

OH MY STACK / READING EDITION

pstack

AI Agent 开发环境实践

构建可验证、可委托的开发环境

kaito

简体中文 • AI 校对版

SOURCE EDITION / 2026-10-03

48 ENTRIES • COMPLETE TEXT

关于本版

作者：kaito

中文翻译整理：Oh My Stack（AI 辅助）

本版收录完整的 48 个目录项，包括 39 个编号章节、开篇、分部导言、附录与后记。中文译文以日文为基准，参考作者英文版校对；已完成逐篇 AI 对照校对，尚未完成人工终审。正文中的第一人称仍指原作者。

来源与授权

书籍正文采用 2026-10-03 的冻结快照。维护者报告已获得作者私聊口头许可，可推出中英文版本。本书与译文不因收录进 Oh My Stack 而自动采用该项目的 MIT 许可。原书第三方引用、代码与图片仍保留各自出处。

电子书阅读说明

网页折叠代码在本版中全部展开。可执行语法、命令、标识符和影响行为的示例值保持原文；说明性注释和提示词模板已翻译为中文。长行在 PDF 中按页面宽度折行。EPUB 表格按行展开，每个值附有列名，以适应阅读器的字号和换行。图表和原图随书附带；外部文章、视频和社交媒体内容保留链接，访问时仍需联网。图片中的原文可参考书末的中文说明。

本书讨论的 pstack 参考版本为 0.15.5。Oh My Stack 的运行时行为以项目文档为准。

[完整授权与来源记录](#) · [在线阅读与校对记录](#)

电子书排版：Oh My Stack。内嵌 Noto Sans CJK 字体采用 SIL Open Font License 1.1。字体许可不适用于书籍正文。

目录

开篇.....	6
第一部分：支撑每月 2,500 个 PR 的开发基础	11
第 1 章：2,500 个 PR 是环境完善后的结果，而非目标	13
第 2 章：信任成果物，而不是 Agent	16
第 3 章：让验证成为可重复的机制	24
第 4 章：让代码库成为 Agent 的记忆	31
第 5 章：在信任的基础上建立自动化	38
第二部分：什么是 pstack	43
第 6 章：pstack 解决什么问题	45
第 7 章：/poteto-mode，让请求进入合适工作流的入口	53
第 8 章：Playbook、Skill 与 Principle 各自负责什么	63
第 9 章：为什么 pstack 只加载需要的内容	71
第三部分：用 Playbook 推进工作	75
第 10 章：调查并复现问题	80
第 11 章：修复缺陷，改善性能	87
第 12 章：重新设计功能、结构与外观	96
第 13 章：编写与评估 Skill	106
第 14 章：创建 PR，合并已验证的变更	112
第 15 章：把长时间任务交给 Agent	122
第 16 章：规划大量 PR，组织多个 Agent 执行	132

第四部分：用 Principle 支持 Agent 的判断.....	150
第 17 章：Core，决定工作方式的十条原则.....	156
第 18 章：Architecture，设计边界与依赖的六条原则.....	181
第 19 章：Verification，通过实际行为验证的四条原则	209
第 20 章：Delegation，处理委派与并行工作的两条原则	231
第 21 章：Meta，改进机制本身的原则	240
第五部分：用 Skill 扩展工作能力	251
第 22 章：理解代码库与过去的工作.....	259
第 23 章：为现有 Playbook 无法覆盖的任务设计 Playbook.....	279
第 24 章：并行运行多个 Agent，比较设计与成果物.....	284
第 25 章：把 pstack 扩展到自己的工作方式与 Grok Bot	306
第 26 章：建立验证机制，并随应用变化持续维护.....	313
第 27 章：检查变更可能破坏的部分及差异中的薄弱点.....	328
第 28 章：记录决策，用证据追踪 Agent 的长时间工作.....	335
第 29 章：用十六条 TypeScript 规则维护代码质量.....	345
第 30 章：TDD，用修复前失败的测试验证修复.....	360
第 31 章：移除注释，整理代码库.....	370
第 32 章：保持写作质量.....	382
第 33 章：回顾工作中的经验并落实改进.....	399
第 34 章：安装 pstack，为各角色选择模型	407
第 35 章：/poteto-mode，pstack 的路由器	424
第六部分：pstack 实践指南.....	440

第 36 章：指南一，从验证 Skill 开始，运行并检查自己的应用	442
第 37 章：指南二，说明目标与背景，对齐问题和历史上下文.....	459
第 38 章：指南二，从用户的使用方式出发检验设计.....	469
第 39 章：指南二，重新审视设计，形成可执行计划.....	483
附录：README with Grill 驱动开发（RGDD）	508
后记.....	525
附加资料：术语表.....	540
附加资料：图片文字说明.....	542

开篇

kaito • 来源: 日本語 • English • 2026-10-03

本书也有英文版: [English edition](#)

这本书旨在**扩大能够交给 Agent 处理的工作范围**。

让 AI Agent 写代码本身已经不难。困难的是建立一种机制，使人能够相信它写出的结果，不再一直守在 AI 旁边监督。

即使 Agent 说「修好了」，在验证之前也无法知道是否真的修好了。因此，人必须在旁边盯着屏幕、阅读差异、实际运行应用并检查。这样一来，无论同时运行多少个 Agent，负责检查的人始终是瓶颈。

本书讲解的，是**如何建立一个能摆脱这种状态的环境**。

本书主要讨论 **pstack**。

pstack 是 **poteto**，也就是 Lauren Tan，公开发布的 Cursor 插件。她是 Cursor 的工程师，也是 React 核心团队成员。

她将自己在 Meta、Netflix、Cursor 的经验，以及每天在 Cursor 开发高质量代码时使用的工作步骤与判断标准，整理成 Playbook、Principle 和 Skill。

poteto 的目标不是让 AI 大量编写代码，而是建立一种能够检验质量的工作方式，让多个 Agent 可以被信任并行工作。她在 2026 年 9 月表示，使用 pstack 在 8 月一个月内将 2,500 个 PR 发布到了生产环境。

<https://x.com/poteto/status/2102050467505430555>

但本书并不提供增加 PR 数量的操作教程。

核心问题只有一个：

在什么条件下，Agent 即使没有人类监督，也能持续完成高质量的工作？

pstack 是对这个问题的一种具体回答。其内容包括验证成果物的机制、熟练工程师的工作步骤、判断标准、保存在代码库中的知识和约束，以及以这些为基础建立的自动化。

本书会讲解 pstack 的所有项目（Playbook、Principle、Skill），并将这个答案拆解成可以应用于自己项目的形式。

「把工作交给 Agent」的定义

把工作交给 Agent，并不只是给它指令，然后等待结果。本书将以下三件事都已交给 Agent 的状态称为「交给它工作」。

1. **做什么才算完成**……完成条件能够通过执行来判断通过或失败
2. **怎样推进**……调查、复现、修复、检查等步骤，每次都能以相同的质量进行
3. **怎样验证**……Agent 自己实际运行应用或成果物，并留下证据

如果缺少第三项，即使具备前两项，人也得一直担任检查者。poteto 将「验证」称为最重要的基础，原因正在于此。本书也从验证开始。

本书涵盖的范围

本书由六个部分、39 章组成。

部分	主题	章节
第一部分：支撑每月 2,500 个 PR 的开发基础	能够把工作交给 Agent，需要什么条件	第 1～5 章
第二部分：什么是 pstack	pstack 通过哪些组件实现第一部分所述的条件	第 6～9 章
第三部分：用 Playbook 推进工作	pstack 中 Playbook 的详细内容	第 10～16 章
第四部分：Principle 支持 Agent 的判断	pstack 中 Principle 的详细内容	第 17～21 章
第五部分：用 Skill 扩展工作能力	pstack 中 Skill 的详细内容	第 22～35 章
第六部分：pstack 实践指南	在自己的项目中，应该从哪里开始	第 36～39 章

第三部分到第五部分介绍 pstack 的具体内容，涵盖截至 2026 年 9 月 pstack 中全部 23 个 Playbook、23 个 Principle、24 个 Skill。不只是罗列名称，还会讲解每个项目为何存在、在什么场景下使用，以及它与其他项目如何关联。

第六部分是实践篇，主要参考 poteto 在 X 发布的文章《The Complete Guide to pstack》的 Part 1 和 Part 2。

以这篇文章为基础，讲解如何创建验证 Skill，以及如何与 Agent 一起开展调查、设计和规划。

阅读方式

第一次阅读时，我建议从头按顺序读。先在第一部分理解「为什么验证要放在前面」，就更容易看出每个项目是为解决什么问题而设置的。

如果目标明确，也可以按以下方式阅读。

- **想安装 pstack 并立即使用**……读完第二部分，再阅读第 34 章（安装）和第 36 章（验证 Skill）。
- **不使用 pstack，但想设计团队的开发环境**……重点阅读第一部分、第四部分和第六部分。Principle 和实践指南的内容即使不使用 pstack 也能应用。
- **想查某个 Playbook 或 Skill**……从各部分导读中的章节列表，直接进入相应章节。每一章都按可以单独阅读的方式编写。

本书反复使用的 pstack 请求示例

逐个查看 pstack 的 Playbook、Principle 和 Skill，不容易看出它们如何在同一个请求中协同工作。因此，本书会在第二部分和第四部分讨论以下提示词。

```
/poteto-mode the export writes duplicate rows when a retry lands mid-run. repro first, then fix and verify.  
// 如果重试发生在执行中途，导出会写入重复的行。先复现，再修复，然后验证。
```

这段请求以 pstack 随附指南中列出的请求示例为基础，原书作者为其添加了日文翻译。

它用一句话说明发生了什么（症状），再用一句话说明做什么才算完成（复现、修复、验证）。第二部分和第四部分将它作为一个好的示例加以讨论。

- 在第二部分，讨论入口 /poteto-mode 会选择哪个 Playbook
- 在第四部分，讨论工作中的判断会怎样运用哪些 Principle

我自己并没有在真实项目中实际执行过这项请求。因此，本书展示的流程是沿着 pstack 文件中记载的步骤推演出来的，没有提供执行日志或输出。

参考时点与表述规则

pstack 经常更新。本书以以下时点的 pstack 为对象。

项目	值
确认日期	2026 年 9 月 24 日
仓库	cursor/plugins 中的 pstack
版本	0.15.5
commit	12d587d（2026 年 9 月 23 日）
数量	Playbook 23 个、Principle 23 个、Skill 24 个

如果使用更新的版本，请将 README 中的列表与本书的列表比较。有些项目可能改名，也可能增加新项目。对于默认使用的模型名称等特别容易变化的值，本书会注明「0.15.5 时点的默认值」。

本书按以下方式区分所写内容的类型。

- **pstack 的规格**，会附上相应的文件名或链接。
- **poteto 的观点**，会注明出处（README、《The Complete Guide to pstack》、演讲等），并作为她本人的观点呈现，不视为适用于所有环境的事实。
- **原文引用**，会简短引用英文原文，注明出处，并附上日文翻译或摘要。
- **作者的解读与经验**，会用「本书将其理解为……」「我认为……」「在我的情况下……」等表述明确标注。

接下来，我们从[第一部分](#)开始，看看「每月 2,500 个」这个数字背后的条件。

第一部分：支撑每月 2,500 个 PR 的开发基础

kaito · 来源：日本語 · English · 2026-10-03

第一部分讨论「能够把工作交给 Agent，需要什么条件」。

要满足这些条件，在提高速度之前，必须先建立能够验证成果的机制和工作步骤等环境。

具体来说，有四个条件；前一个条件是下一个条件的前提和基础。

1. Agent 自己能够运行并检查它制作的东西（成果物），也就是验证
2. 明确验证方法和工作步骤，使任何人执行都能以稳定的质量复现
3. 将过去的判断和约束保存在代码库中，供下一项工作参考
4. 在具备前三项基础后，再实行自动化

每月 2,500 个 PR，是建立这四项条件之后产生的结果。如果只把数字当成目标，跳过第 1 至第 3 项就着手第 4 项，增加的将不是成果，而是**无从验证的变更**。

第一部分能学到什么

阅读第一部分后，你会明白以下几点。

- 2,500 这个数字不意味着什么
- 如果不信任 Agent，应该信任什么
- 如何让验证不止于一次性检查，而成为任何人都能再次执行的机制
- 为什么要把 Agent 的记忆放在代码库里，而不是放在对话或笔记里
- 为什么自动化要放在最后，以及自动化前必须具备哪些条件

第一部分很少具体讲解 pstack 的各个组件。它讨论的是无论是否使用 pstack 都适用的思路。从**第二部分**起，本书会将 pstack 的各个组件与这些思路联系起来。

本部分的章节

章	作用
第 1 章：2,500 个 PR 是环境成熟的结果，而不是目标	确定如何看待 2,500 这个数字，把本书的中心主题从「数量」转向「能够委派工作的条件」
第 2 章：信任成果物，而不是 Agent	明确应当信任的对象，定义什么是验证
第 3 章：让验证成为可复现的机制	讨论增强信任的三种方法：直接验证、工作模式、通过结构施加约束
第 4 章：把代码库变成 Agent 的记忆	讨论如何把已经建立的信任传递给下一个 Agent
第 5 章：在信任的基础上建立自动化	讨论具备第 2 至第 4 章的条件后，哪些工作可以自动化

第 2 章至第 5 章，分别对应 poteto 在演讲中提出的四个条件。后面的章节以前面的章节为前提，因此建议按顺序阅读。

第 1 章：2,500 个 PR 是环境完善后的结果，而非目标

kaito · 来源：日本語 · English · 2026-10-03

每月 2,500 个 PR 并不是刻意追求的数字。它是在建立一个能够放心把工作交给 Agent 的环境之后，随之产生的数字。

本章探讨「为什么不能把数量定为目标」，以及「数量究竟代表什么」。

本章结构

本章按以下顺序展开。

- 为什么不能把数量定为目标
- 数量所代表的事
- 小结：将视角从「数量」转向「能够委派工作的条件」

为什么不能把数量定为目标

原因有两个。

第一个原因是，如果只增加数量，**增加的不是成果，而是未经验证的变更**。

只要人仍在逐一阅读 Agent 所写变更的差异，并操作应用来验证，PR 数量的上限就取决于人能检查多少。

在这种状态下，即使决定「本月把 PR 翻倍」，也只是把发给 Agent 的请求翻倍。负责检查的人手没有增加，因此多出来的变更会未经验证地堆积起来。

哪些变更正确、哪些出了问题，要等以后有人逐一验证才会知道。

另一个原因是，**数量是很容易操纵的数字**。

同一个功能，如果合并成一个大 PR，就是 1 个；如果拆成五个，就是 5 个。

如果把数量定为目标，即使变更内容完全相同，也能仅凭拆细 PR 让数字看起来更好。

数量所代表的事

我认为，数量真正代表的是**从人手中转移到 Agent 一侧完成的检查量**。

如上一节所述，如果所有事情都由人验证，数量就无法超过人的检查能力。反过来说，如果数量远远超出人的检查能力，大部分检查工作应当由人以外的主体承担。

poteto 也在《The Complete Guide to pstack》的 [Part 1](#) 中，将验证（verification）定义为「**Agent 能够检查自己的工作**」，并写道这样人就不会成为瓶颈。

那么，要把检查工作从人手中转移出去，需要什么？

需要以下四项，它们也是 poteto [演讲](#) 的四个主题。

1. 信任
2. 提高信任的方法
3. 将代码库视为记忆
4. 自动化

特别重要的是，这四项并不是相互独立的提示；前一项是后一项的前提和基础。

也就是说，必须遵守这样的顺序：首先让成果物处于可验证的状态，将由此获得的知识保存在代码库中，再在此基础上推进自动化。

每月 2,500 个 PR，是依次建立这四项条件后的结果。本书的 [第 2 章](#) 到 [第 5 章](#) 会分别讨论这四项，每章讨论一项。

小结：将视角从「数量」转向「能够委派工作的条件」

- **不能把数量定为目标的原因**……如果负责检查的人手没有增加，增加 PR 数量只会增加未经验证的变更。数量也很容易随着 PR 的拆分方式而变化。
- **数量所代表的事**……从人手中转移到 Agent 一侧完成的检查量。其背后是信任、提高信任的方法、作为记忆的代码库、自动化这四项条件齐备的状态。

下一章第 2 章讨论四项中的第一项「信任」。演讲提出，信任的对象是 Agent 制作的成果物，而不是 Agent 本身。我们将看看 pstack 如何将这一思路付诸实践。

第 2 章：信任成果物，而不是 Agent

kaito • 来源：日本語 • English • 2026-10-03

把工作交给 Agent 时，应当信任的不是 Agent 的报告，而是**Agent 制作的成果物**。

这是 poteto 在演讲的第一个主题「信任」中提出的思路。

本章讲解信任成果物意味着什么，以及 pstack 如何将这一思路付诸实践。

本章结构

本章按以下顺序展开。

- 为什么信任成果物，而不是报告
- 验证成果物时，验证什么、怎样验证
- 看回复中的什么，才能判断是否可以放心委派
- 如何逐步达到可以放心委派的状态
- 小结：将信任的对象从报告转移到成果物

为什么信任成果物，而不是报告

之所以信任成果物，是因为**报告不能证明正确性**。

Agent 说「修好了」「应该能运行」，只说明 Agent 自己做出了这样的判断。仅凭报告，无法知道它的判断依据是测试通过、阅读代码后的印象，还是实际运行的结果。

人若要确认正确性，到头来仍只能自己阅读差异、操作应用。

如果在这种状态下增加工作量，增加的只会是未经验证的变更。演讲也指出，若无法验证却只增加处理量，**增加的不是成果，而是缺陷**。如果质量只能靠人在旁边监督来维持，就谈不上把工作交出去了。

反过来，如果成果物处于可验证的状态，人就无需相信报告，也无需自己把一切重新检查一遍。

pstack 通过验证机制提供这种能力。poteto 在《The Complete Guide to pstack》的 [Part 1](#) 中，将验证（verification）定义为「**Agent 能够检查自己的工作**」。

验证成果物：验证什么、怎样验证

验证成果物，就是由 **Agent 自己直接检查实物**，并以能够留下证据的方式进行检查。

具体来说，需要满足以下三个条件。

1. **针对实物**……不只看构建是否成功或通过了多少测试，而要检查运行中的应用、实际写入的值、实际输出的文件
2. **由 Agent 自己执行**……不是由人手动操作应用，而是由 Agent 操作并观察
3. **留下证据**……留下截图、视频、追踪记录、命令输出等材料，供人日后查看并作出判断

下面逐项看看 pstack 的文件如何支持这些条件。

针对实物

第一个条件由 Principle「**Prove It Works**」规定。开头的一句话是：

Verify every task output by checking the real thing directly. Do not infer from proxies, self-reports, or "it compiles."

通过直接检查实物来验证每项工作的输出。不要根据替代指标、自我报告或「编译通过了」进行推断。

这句话并不禁止构建或测试。它禁止的是把「编译通过了」当成应用实际能运行的证据。除了构建与测试，还必须运行并检查实物。

SKILL.md 的 description 列出三种检查方式作为示例。

1. 实际运行功能
2. 读取实际的值
3. 查看差异

此外，这项原则将缓存的截图列为应避免的间接检查示例。截图看起来像成果物，但如果不知道它是否反映了这次变更，其作为验证依据的说服力就很弱。由此可以看出，查看成果物时，最好还要确认**该成果物是否在这次变更之后生成**。

同样的思路也适用于 Agent 把工作交给子 Agent 的场景。在 `/poteto-mode` 中，主 Agent 不会原样转交子 Agent 的报告，而是先查看差异再回复。因此，**人收到的回复，在那一刻已经通过成果物接受过一次检查**（第 8 章）。

由 Agent 自己执行

第二个条件由用于修复缺陷的「**Bug fix**」Playbook（`playbooks/bug-fix.md`）支持。在这个 Playbook 中，负责的 Agent 使用能够操作应用的工具（Playbook 称之为 control 类 Skill），自己在缺陷发生的同一环境中复现问题，也在同一环境中验证修复（第 11 章）。

操作应用的工具并不包含在 pstack 本体中。用于浏览器和 Electron（构建桌面应用的框架）的 control-ui，以及用于 CLI 和 TUI 的 control-cli，位于另一个插件 `cursor-team-kit` 中。如何为自己的应用制作专用工具，将在第 3 章和第 36 章讨论。

只有说明操作手段为何无法触及目标，并且已经用这些手段操作到能够到达的最远处，Agent 才能请求用户代为复现。

例如，登录过程中需要在实体设备上完成双重验证，工具无法继续操作时，Agent 应当说明自己已经操作到哪一步、为何停下，再请用户复现。

我认为这个限制很重要。它保留了向人求助的途径，但要求给出理由，因此 Agent 很难仅仅因为麻烦就把检查工作交还给人。

留下证据

第三个条件由「**Prove It Works**」中的「**Script the check when you can**」一节支持。

The strongest proof is a deterministic script that re-runs the same comparison, not a one-time eyeball.

最有力的证据，是可以再次运行相同比较的确定性脚本，而不是一次性的目测。

这一节建议编写能够重复相同比较的脚本，并将执行结果保留在人能够查看的地方。可以重复执行的是脚本，人阅读的是执行结果。

只有在大型移植或迁移等日后需要审计历史的情况下，才需要把证据提交到仓库。如何把检查过程写成脚本，将在第 3 章讨论。

在我看来，最容易被忽略的是第三个条件。即使满足了前两个条件，如果没有留下证据，人最终收到的仍只有一句「我检查过了」，又回到了本章的起点。

实际建立验证机制时，我认为除了确定 Agent 能否检查，还应决定**检查结果是否会留在人能够查看的地方**。

看回复中的什么，才能判断是否可以委派

查看回复时，可以根据**每一项「已经做了……」的主张是否附有验证证据**来判断。

例如，只写「缺陷修好了」的回复，无法让人判断其正误。如果附上修复前出现缺陷的输出，以及修复后缺陷不再出现的输出，读者就能自行判断。

pstack 要求每项主张附有证据，或标为「推测」

在 pstack 中，开始工作时使用的 `/poteto-mode` (`skills/poteto-mode/SKILL.md`) 对回复的写法规定了以下规则。

Every claim carries its evidence or its label in the same sentence. Measured, inferred, or guess.

每项主张都要在同一句话中附上证据或类型标签：经过测量、根据证据推断，还是猜测。

Agent 有证据时就附上证据；没有证据时，就如实写明「这是推断」或「这是猜测」。pstack 随附指南的验证页面 (`docs/guide/06-verify-and-ship.md`) 也用同样的思路区分回复的好坏。

If a check couldn't run, a good reply says "inconclusive", and you should treat a confident reply without evidence as a red flag.

如果无法运行检查，好的回复会说「无法得出结论」；没有证据却充满信心的回复应被视为危险信号。

例如，没有运行测试的环境，却回复「已经修复，没有问题」，就是一个危险信号。

有证据的回复让读者能够自行判断正误

该页面列出了适合不同变更类型的检查方式。据此将只有报告的回复与附有证据的回复并列，可以得到下表。中间一列是我为了比较而编写的示例。

变更类型	只有报告的回复	附有证据的回复
CLI 变更	「添加了参数」	实际运行命令的输出
UI 变更	「修正了显示」	在运行中的应用里操作变更后的界面流程所留下的记录
解析器或迁移处理	「修正了转换处理」	再次输入预先保存的数据所得到的结果
性能改进	「应该变快了」	变更前后性能分析结果的比较
写入处理变更	「已经改为写入」	读回写入值的结果

中间一列所说的未必是假话，但没有可供判断是否正确的材料。右边一列则允许读者自行判断。

仅凭「测试通过了」无法判断缺陷已经修复

容易犯的错误，是把「测试通过了」放进右边一列。

「Bug fix」Playbook 的「验证修复的步骤（步骤 4）」也指出，仅凭单元测试通过，无法判断缺陷已经修复。即使代码在测试预设的条件下能运行，仍必须再次执行引发用户所报缺陷的同一操作，才能确定该操作中的问题是否修好。

用函数返回 undefined 时测试是否仍能通过，检查测试是否验证了行为

测试是否检查了**代码的行为**，也就是代码使用者实际获得的结果，可以根据 Principle 「**Test Behavior, Not Implementation**」来判断。

这项原则认为，**测试应该像代码使用者一样调用代码，并将该使用者得到的结果与具体的预期值比较**。测试是否真的观察到了结果，可以用以下问题检查。

The check: before you keep a test, ask whether it would still pass if every function it imports returned `undefined`. If yes, it observes no behavior and cannot fail for a defect.

检查方法：保留一项测试之前，先问自己：如果它导入的每个函数都返回 `undefined`，这项测试仍会通过吗？如果会，它就没有观察任何行为，也不会因缺陷而失败。

例如，只检查函数是否被调用的测试，在函数返回 `undefined` 时也能通过。这项原则要求改写或删除这样的测试。

不过，即使通过了这个检查，也只能说明「这项测试检查了代码返回的结果」，不能证明不存在缺陷。

「Bug fix」Playbook 要求在回复中并列展示失败与成功时的输出

最后，我们通过用于修复缺陷的「Bug fix」Playbook，确认回复中应当留下什么。

该 Playbook 对回复内容作出如下规定。这里的「复现」，如前一节所述，是实际执行触发缺陷的操作，并确认同一缺陷出现的工作。

Reply: what was broken, root cause, fix, how you verified. Paste failing-then-passing repro output verbatim.

回复：说明哪里坏了、根本原因、修复内容，以及如何验证。将复现先失败、后通过时的输出原样贴出。

发生缺陷时的输出与缺陷消失后的输出，会并列出现在回复中。

如何逐步达到可以放心委派的状态

我认为，要逐步达到可以放心委派的状态，应该找出缺少的检查，再逐一把它们变成机制。

首先，用以下五项查找缺口。这五项是我将本章讨论的内容整理成的检查问题。

要检查的事	如果答案是「否」
完成条件是否写成可以通过执行判定通过或失败的形式？	在请求中补上「做到什么才算完成」
Agent 能否亲自运行实际应用或成果物？	提供操作应用的手段（第 3 章）
运行结果是否会作为证据留下？	确定保存截图、输出与追踪记录的步骤
无法检查时，回复是否会如实说明？	允许回答「无法得出结论」，不接受没有证据的成功报告
即使人离开座位，上述四项仍能运转吗？	将无法运转的部分替换成机制（第 3～5 章）

第一项「完成条件」见随附指南的验证页面（<docs/guide/06-verify-and-ship.md>）中的「State the finish condition up front」。

Put what done means in the first prompt, in whatever words fit:

在最初的提示词中，用合适的话写清楚什么才算完成。

指南给出以下请求作为示例。

```
/poteto-mode add json output to this command. text output stays byte-identical, the json parses,
both run against the sample project. show me the evidence.
// 给这个命令增加 JSON 输出。文本输出必须逐字节保持不变。JSON 必须能够解析。两种输出都要在示例项目中
运行。请展示证据。
```

指南解释说，这样写，**Agent 得到的就是三个能够执行并检查的条件，而不是只能凭感觉判断是否达到的目标。**

写出完成条件，也会同时确定第二项之后的检查问题。写明「文本输出一个字节都不能变」，就需要有比较变更前后的步骤。写明「在示例项目中运行」，就意味着 Agent 必须能够运行该项目。

如果无法将完成条件写成可以通过执行判定通过或失败的形式，就该重新考虑规格是否确定、是否有判定通过或失败的手段。

表格右侧一列中的许多机制，仅靠请求文本无法提供。还需要操作应用的工具、保存证据的步骤，以及人离开后也能运转的机制。这些是第 3 章之后的主题。

小结：将信任的对象从报告转移到成果物

- **为什么信任成果物，而不是报告**……报告无法证明正确性。如果只有人在旁边监督时才能维持质量，就谈不上委派。
- **验证成果物意味着什么**……由 Agent 自己检查实物，并留下证据。pstack 的 Principle「**Prove It Works**」、/poteto-mode 和「**Bug fix**」Playbook 支持这些条件。其中容易被忽略的是留下证据。
- **看回复中的什么**……看每项「已经做了……」的主张是否有验证证据。测试通过只能在测试确实检查代码行为时作为判断材料之一，不能证明不存在缺陷。
- **怎样接近目标**……用五项检查找出缺口，再逐一将其变成机制。

以「**Prove It Works**」为代表的 Verification 组 Principle，将在第 19 章详细讨论。

明确了信任的对象，下一步就是提高信任的方法。如果只验证一次成果物，而下次变更时无法重复同一检查，信任就无法积累。第 3 章将讲解提高信任的三种方法，以及如何把经修正的教训留在机制中。

第 3 章：让验证成为可重复的机制

kaito • 来源：日本語 • English • 2026-10-03

要提高信任，必须把一次性的检查，以任何人无论执行多少次都能得到相同结果的形式留在仓库里。

poteto 在演讲的第二个主题中，提出了以下三种提高信任的方法。

1. 直接验证成果物
2. 教会 Agent 如何高质量地推进工作
3. 让代码库具有易于理解的结构

本章讲解这三种方法的区别、pstack 如何分别实现它们，以及如何把纠正错误后得到的教训保存在机制中。

本章结构

本章按以下顺序展开。

- 提高信任的三种方法及其区别
- 方法 1：将检查方式变成可反复执行的工具
- 方法 2：把熟练工程师的工作方式作为步骤交给 Agent
- 方法 3：让代码库结构本身成为说明书
- 把纠正错误后得到的教训留在机制中，而不是对话中
- 小结：以可再次执行的形式保留一次检查

提高信任的三种方法及其区别

三种方法的区别在于它们有多依赖 Agent 自己保持注意。

三种方法及其示例如下。

1. **直接验证成果物**……实际运行应用，利用 Chrome DevTools Protocol (CDP，一种让程序使用与浏览器开发者工具相同功能的机制) 采集追踪记录（处理耗时的记录）或堆快照（某个时刻内存内容的记录）
2. **教会 Agent 如何高质量地推进工作**……把规定调查、复现、实现、验证步骤的 pstack Playbook 交给 Agent
3. **让代码库具有易于理解的结构**……只要查看目录、类型和进程的边界，就能知道「在哪里可以做什么」「哪些代码在哪里运行」「允许哪些 import」

演讲指出，信任不会因为努力或增加对话次数而增长。把验证方法、工作步骤和结构约束以可供查阅的形式留下来，才能扩大可委派工作的范围。

比较 Agent 必须做什么才能让每种方法生效，就能看出它们的区别。

- **方法 1（验证）**……Agent 必须运行工具才会生效。即使准备了工具，不使用就不会发生检查。
- **方法 2（步骤）**……Agent 必须阅读步骤并照着执行才会生效。它可能漏读或误读。
- **方法 3（结构）**……即使 Agent 没有注意到也会生效。若用类型或 lint 表达边界，越界的变更会在编译或 lint 阶段被拦下。

因此，从方法 1 走向方法 3，维持质量就会越来越少依赖 Agent 的注意。下面逐一来看。

方法 1：将检查方式变成可反复执行的工具

第一种方法是把检查方式做成工具，让任何人无论执行多少次都能得到相同的观察结果。

演讲所要求的验证，并不是把编译通过或 Agent 自称「已经修复」当作依据，而是把实际运行应用时的行为当作证据。

在 pstack 中由「Build the Lever」和验证 Skill 承担

在 pstack 中，Principle「**Build the Lever**」和用于创建验证 Skill 的 Skill 承担这种方法。

这项原则规定，**不要只靠手工处理，而要制作审查者能够阅读并重新执行的脚本或命令**。验证 Skill 是项目专用的 Skill，供 Agent 启动自己的应用、操作应用并取得证据。

在 pstack 中，使用 `/create-verification-skill` 创建验证 Skill，使用 `/maintain-verification-skill` 保持其更新。验证 Skill 的内容和命令示例见第 26 章与第 36 章。

工具要在手动检查一次之后制作，做完还要维护

验证工具必须先在第一次手动检查之后制作，制作后也需要持续维护。

「Build the Lever」规定了以下制作方式：**先手动处理一个目标以确认步骤，再把步骤做成工具，最后用工具处理同一个目标，检查两次的结果是否一致**。应用到验证上，就是先手动检查，再把操作写成命令，最后比较该命令能否复现同一项检查。

原则文件还说，如果声称遵循了这项原则，但差异中没有 codemod（机械式批量改写代码的脚本）、脚本等内容，就不能算应用了这项原则。即使 PR 上写了「已验证」，**如果没有附上可再次执行的命令或其输出，验证工具就尚未完成**（第 36 章）。

制作工具前后，有两点需要注意。

- **为难以操作的应用提供操作入口**.....《The Complete Guide to pstack》的 Part 1 指出，技术栈越难调试和操作，Agent 就越难发挥作用；为了验证，文章甚至建议自己制作工具或更换技术栈。
- **验证失败时，先怀疑工具**.....Principle 「**Prove It Works**」写道：「验证失败时，先怀疑观察方法，再怀疑系统。」应用变化后，工具可能已经过时，返回错误结果。

方法 2：把熟练工程师的工作方式作为步骤交给 Agent

第二种方法是**把熟练工程师的工作模式整理成步骤，交给 Agent**。演讲以 pstack 的 Playbook 为例说明这种方法。

Playbook 给出的不是答案本身，而是「**熟练工程师会这样推进**」的工作模式。

没有固定模式，Agent 每次可能选择不同的推进方式；如果步骤明确，其他 Agent 也更容易以相近的质量推进工作。

Playbook 的具体示例

例如，用于修复缺陷的「**Bug fix**」Playbook ([playbooks/bug-fix.md](#)) 规定了固定顺序的步骤：在同一环境中复现、用证据缩小原因范围、修复、在同一环境中验证，直至创建 PR（第 11 章）。

在 pstack 中，入口 /poteto-mode 会选择符合请求的 Playbook，并把其中的步骤原样抄到 TODO 列表的开头（[README](#)）。全部 23 个 Playbook 将在第三部分逐一讨论。

TODO 列表.....Agent 在开始工作时创建的该项工作待办事项清单（计划）。在 Cursor 中，它显示在聊天界面。用户可以查看清单，确认 Agent 执行了哪些步骤、跳过了哪些步骤（skip: <理由>）（随附指南 [01-setup.md](#)）。

方法 3：让代码库结构本身成为说明书

第三种方法是让人只看目录、类型和进程边界就能明白规则，使代码库结构本身成为给 Agent 的说明书。

目标是建立这样一种代码库：只需查看目录、类型和进程边界，就能知道以下三件事。

- 在哪里可以做什么
- 哪些代码在哪里运行
- 允许哪些 import

建立了这样的代码库，Agent 即使不读文字说明，也能从结构中了解约束与规则。

在 pstack 中由 Architecture 组的 Principle 承担

pstack 按主题对 Principle 分类（见 [README](#) 中的原则列表）。「**Architecture（架构）**」组的原则，旨在整理代码边界、类型和状态的处理方式，建立不容易出错的结构。

本节从中选取「**Boundary Discipline**」和「**Type System Discipline**」。

README 将这两项原则概括如下。

- 「**Boundary Discipline**」 (`principle-boundary-discipline`) ……外部传入的值 (CLI 参数、配置文件、网络、外部 API) 应在系统接收处接受检查并转换为内部类型。内部代码信任这个类型，不重复检查。
- 「**Type System Discipline**」 (`principle-type-system-discipline`) ……用类型让互相矛盾的不合法状态无法被表达。

这两项原则将在第 18 章详细讨论。

无法机械判定的规则，不能变成结构

只有能够通过类型或 lint 机械判定的规则，才能变成结构。

例如，「此目录中的代码只能在浏览器中运行」这项规则，可以用两种方式留下来。

1. 在 README 中写一句话
2. 分开目录，并用 lint 禁止其他目录 import 这里的代码

第一种方法要求 Agent 阅读后才会遵守；第二种方法即使 Agent 没读也能发挥作用。可以机械判定的规则，应采用第二种形式。

另一方面，「这个页面的措辞要克制」「这类变更应拆成较小部分」等需要结合情境判断的规则，无法写成类型或 lint。只能用文字保留。这种保留方式由下一节将介绍的 Principle

「**Encode Lessons in Structure**」规定。

把纠正错误后得到的教训留在机制中，而不是对话中

纠正 Agent 后得到的教训，不应只留在对话里，而要尽可能以有力的形式，留在上述三种方法产生的工具、步骤或结构中。

这也是演讲在介绍三种方法之后提出的思路。

纠正后的教训可以留在以下地方之一。

- 代码库本体
- 由 lint、编译器或 CI 执行的自动验证
- rules 或 Bugbot (Cursor 的自动代码审查)
- Skill

- 团队的风格指南

这样，下一位 Agent 即使没有在对话中收到同样的提醒，也能避开曾经指出的问题。

在 pstack 中由「Encode Lessons in Structure」承担

在 pstack 中，Principle「Encode Lessons in Structure」([skills/principle-encode-lessons-in-structure/SKILL.md](#)) 对此作出规定。在 README 的列表里，只有这项原则被归入「Meta」组。它讨论的不是代码本身，而是**怎样把教训留在机制中**这一建立机制的方式。

该原则规定，**当你发现自己第二次写下同样的指示时，应询问是否可以将其变成 lint 规则或运行时检查等机制；如果可以，就建立机制并删掉文字指示**。对于无法变成机制的规则，也就是上一节说的「需要看情境判断的规则」，应使指示醒目，并附上失败示例。

选择机制时，尽量选择类型或 lint 这样较强的方式。Agent 会模仿周围代码已有的做法，所以弱机制也会成为后续代码仿效的样本。

在对话中纠正 Agent，即使它回答「我会记住」，教训也不会因此留下来。第二次写同样指示时应采取的步骤、各种机制的强弱顺序，以及如何分配纠正内容，将在第 21 章详细讨论。

把审查的判定标准也作为文件持续完善

如何判定审查意见的标准，也可以作为保存教训的文件持续完善。将 PR 推进到可合并状态的「Babysit」Playbook ([playbooks/babysit.md](#)) 会依据判定标准文件 ([references/bugbot-triage.md](#)) 分类 Bugbot 的意见；若某种驳回理由可能供团队重复使用，就在另一份 PR 中提议把它加入共享的判定标准（第 14 章）。

这样，某一次运行中的 Agent 所学到的内容，就进入下一次运行中的 Agent 会读取的文件。

小结：以可再次执行的形式保留一次检查

- **三种方法及其区别**……演讲提出直接验证成果物、教会 Agent 高质量的工作方式、让代码库具有易于理解的结构三种方法。区别在于它们有多依赖 Agent 自己保持注意：工具不用就不会生效，步骤不读就不会生效，而结构即使没被注意到也会生效。
- **方法 1**……按照 Principle「Build the Lever」，把检查方式变成可再次执行的工具。第一次先手动检查，再将其写成命令；验证失败时，先怀疑工具是否过时。

- **方法 2**……交给 Agent 一种「熟练工程师会这样推进」的工作模式，例如 Playbook。在 pstack 中，/poteto-mode 选择符合请求的 Playbook，并将步骤抄到 TODO 列表。
- **方法 3**……依照 Architecture 组的原则，在边界处检查，并用类型在结构上阻止错误。需要结合情境判断的规则，无法变成结构。
- **纠正后的教训**……按照 Principle 「**Encode Lessons in Structure**」，尽可能留在较强的机制中。只能用文字保留的规则，要突出显示并附上失败示例。仅说「我会记住」并不会留下任何东西。

第 4 章将进一步讨论「留在仓库里」的方针，介绍演讲的第三个主题「将代码库视为记忆」。下一章会说明为什么把知识放在仓库里，以及如何选择和持续维护留下的记忆。

第 4 章：让代码库成为 Agent 的记忆

kaito • 来源：日本語 • English • 2026-10-03

让 Agent 继承以往判断的记忆，应放在**代码库**中，而不是模型内部。

这是 poteto 在[演讲](#)的第三个主题中提出的思路。

本章讲解为什么要把知识放进仓库，以及如何筛选并持续维护留下的记忆。

本章结构

本章按以下顺序展开。

- 为什么把知识放在仓库里，而不是模型中
- 留下的记忆如何过时，以及如何维护
- 应删除什么，才不会污染下一位 Agent 参考的示例
- 如何持续进行维护工作
- 小结：记忆不是一味增加，而是需要筛选和维护

为什么把知识放在仓库里，而不是模型中

把知识放进仓库，是因为**Agent 一次能处理的上下文有限，无法每次都把所需知识塞进提示词**。

过去的判断、已知问题、复现步骤和规避方法，不可能每次都完整写进请求。

将知识放入仓库，**Agent 就不必每次从零思考，而能边阅读其中的信息边继续工作**。

这里说的记忆不只有代码。以下内容也能作为记忆，将过去的判断传递给下一位 Agent。

- Feature Map（按应用功能记录能做什么、用户怎样到达该功能、如何用验证 Skill 操作，以及可检查的结果的文档；将在[下一节](#)详述）
- 复现步骤

- 设计约束
- 测试
- lint 规则

poteto 也在《The Complete Guide to pstack》的 [Part 1](#) 中表达了相同的观点。

Personally, I think your codebase is the ultimate form of memory.

就我个人而言，我认为代码库是记忆的终极形式。

她解释说，与经常被用作 Agent 记忆的 Markdown 笔记等不同，**代码反映了团队实际做出的判断，是关于实际发生了什么、系统如何运行的可靠信息来源。**

留下的记忆如何过时，以及如何维护

留下的记忆中，**需要特别维护的是 Feature Map 这样的记忆：应用变了，它会悄无声息地过时。**

测试和 lint 如果与应用不一致，CI 会失败，我们便能察觉它们过时了。Feature Map 等 Markdown 文档则不同，应用改变后，它们并不会发出任何信号。

pstack 随附指南的验证页面 ([docs/guide/06-verify-and-ship.md](#)) 也用下面这句话引出维护工作。

Apps change and feature maps rot.

应用会变化，Feature Map 会逐渐失效。

拿到过时 Feature Map 的 Agent 会相信其中的描述并照着操作，耗费时间寻找已经不存在的界面。

本节介绍 Feature Map 是什么，以及 pstack 如何维护它。

Feature Map 是汇总各功能用法与验证方式的地图

Feature Map 是一份按应用功能整理的文档，记录每个功能能做什么、用户如何到达该功能、怎样通过验证 Skill 操作，以及应该检查什么结果。

有了 Feature Map，Agent 无需阅读整个代码库，只需阅读所需功能相关的文件。《The Complete Guide to pstack》Part 1 将 Feature Map 视为一种节省上下文 token 的记忆形式。

在 pstack 中，`/create-verification-skill` 创建验证 Skill（第 3 章）时，也会创建 Feature Map。文件格式和内容将在第 36 章讨论。

通过 `/maintain-verification-skill` 保持 Feature Map 更新

使用 `/maintain-verification-skill` 反复检查 Feature Map 是否仍与当前应用一致。执行频率等细节将在第 36 章讨论。

根据随附指南的验证页面 `06-verify-and-ship.md`，`/maintain-verification-skill` 按以下顺序工作。

1. 按功能并行启动只阅读源代码的负责人（子 Agent）
2. 实际运行并检查地图中列出的所有功能
3. 返回以下三种结果之一
 - `clean`……地图所列功能都已检查，没有需要修正的地方
 - `changed`……修复了不一致之处，并将经过验证的修正汇入一个 PR；修正范围限于验证 Skill 的目录
 - `blocked`……无法继续检查，并报告阻碍的原因

同一页面还写道：

It never edits product code. If the live pass catches a product regression, it reports the regression instead of papering over it in docs.

它绝不编辑产品代码。如果实际运行的检查发现产品回归缺陷，就报告该缺陷，而不是通过修改文档来掩盖它。

当应用与记忆不一致时，`/maintain-verification-skill` 不会任意改写记忆来让两者表面上吻合。

不一致可能来自两种情况。

- **应用发生了变化**……记忆已过时，应修正记忆
- **应用出了问题**……错误在应用一侧，不应修正记忆，而应报告缺陷

在第二种情况下，如果改写记忆，**错误的行为就会作为正确行为被记住**。

Feature Map 的创建与维护步骤，将在第 26 章和第 36 章讨论。

应删除什么，才不会污染下一位 Agent 参考的示例

应该删除的是**临时拼凑的规避方法，以及实现同一件事的第二种写法**。

这两者都会作为代码留下来，成为下一位 Agent 阅读的示例。下面先看规避方法，再看第二种写法。

留下的规避方法会成为下一位 Agent 的示例

规避方法留在代码中，就会被下一位 Agent 复制。因此，修复根本原因后应将它删除。

不能把任何东西都当作记忆保留下来。临时拼凑的规避方法如果留着，**下一位 Agent 可能以为它是正确的实现示例并加以复制**；复制品又生出新的复制品，令人不愿看到的模式便悄悄扩散。

pstack 的原则中也有两处提到这种风险。

- **「Encode Lessons in Structure」**……如第 3 章所述，该原则指出 Agent 会模仿周围代码，弱机制就会成为后续代码仿效的样本。
- **「Fix Root Causes」**……该原则指出，针对症状的处理会不断累积，并写道：

If a workaround needs a paragraph-long comment to justify it, the code is wrong.

如果某种规避方法需要一整段注释才能说明其合理性，那问题出在代码本身。

pstack 也通过工作步骤防止规避方法留存。用于修复缺陷的「**Bug fix**」Playbook ([playbooks/bug-fix.md](#)) 在步骤开始前写道：

Belt-and-suspenders that "might help" is a hypothesis, not a fix. It does not ship. When evidence refutes a hypothesis, revert what it motivated.

「也许有用」的额外保险措施是一个假设，不是修复方案，不能发布。如果证据否定了假设，就撤销因它而做出的变更。

本书认为，这项规则是为了防止试探性加入的变更最终作为规避方法留在代码中。

将推荐的实现方式收敛为一条「铺好的路」

如果同一目标有两种以上的写法，应留下推荐的一种、删除旧写法，并用 lint 阻止其他写法。

清除技术债，把推荐的实现方式收敛为一种，并用 lint 等工具机械地找出反模式。这样，**Agent 就不必每次都在多种方法之间选择。**

演讲将这种标准的实现方法称为「**一条铺好的路**」。意思是 Agent 不必每次都寻找自己的捷径，而是沿着已修整好的路径前进。

pstack 的一些原则也旨在铺好这条路。README 的原则列表将以下两项概括为：

- 「**Subtract Before You Add**」 ([principle-subtract-before-you-add](#)) ……先移除不必要的东西、重复的验证和没有实际内容的引用，再在简化的基础上构建
- 「**Migrate Callers Then Delete Legacy APIs**」 ([principle-migrate-callers-then-delete-legacy-apis](#)) ……如果无须维持外部兼容性，就将调用旧 API 的代码切换到新 API，并在同一次变更中删除旧 API。不要把支持旧 API 代码继续运行的兼容层留下来

保留兼容层，代码库中就会有两条实现同一件事的路，对 Agent 来说两条都是示例。两项原则将分别在第 17 章和第 18 章详细讨论。

本书认为，应由人决定哪种写法成为唯一的路。人的职责是确定方向、承担最终责任（第 5 章）；选择唯一的实现方法，正属于这里的「方向」。

如何持续进行维护工作

维护工作要借助 pstack 的组件持续进行。

演讲把维护这种环境的人比作「**园丁**」。花园不能只靠增添植物来维持，还必须修剪多余的枝条、清除杂草，持续维护，让想要的植物生长。

代码库也一样，**不仅要增加知识，还必须持续清除过时的规避方法和糟糕的实现示例。**

pstack 中有帮助完成这类维护的组件。

维护工作	提供帮助的 pstack 组件
同一问题指出两次后，将其变成机制	Principle 「 Encode Lessons in Structure 」 （第 3 章、第 21 章）
检查用于关闭 lint 或类型检查的注释是否在增加	<code>/no-comments</code> （移除不必要注释的 Skill）。其「检查子 Agent 报告和差异的步骤（步骤 2）」要求检查被漏掉的 lint 和 TypeScript 检查抑制（第 31 章）
将写着「别删」「别改」的注释替换为类型、测试或 lint	<code>/no-comments</code> 中「处理描述约束的注释的步骤（步骤 5）」。建议在类型、运行时检查、测试或 CI 中的 lint 之间，选择最省力的替代方式
查找同一目标是否有两种以上写法，并收敛为一种	Principle 「 Subtract Before You Add 」、Principle 「 Migrate Callers Then Delete Legacy APIs 」（第 17 章、第 18 章）
确认 Feature Map 和复现步骤是否仍与当前应用一致	<code>/maintain-verification-skill</code> （第 26 章）
将长时间工作中学到的步骤留给下次使用	<code>/reflect</code> 。README 将其定义为通过编辑 Skill，保留一次已完成的长期工作中的步骤（第 33 章）

由此可见，pstack 除了用于编写代码的 Skill，**还提供了清除规避方法、关闭检查的注释和旧 API，以及将描述约束的注释替换为类型或 lint 的组件。**

小结：记忆不是一味增加，而是需要筛选和维护

- **为什么放进仓库**……上下文容量有限，无法每次都将必要知识塞进提示词。poteto 将代码库称为「终极的记忆」。
- **如何过时、如何维护**……测试和 lint 过时后 CI 会发出信号，Feature Map 等 Markdown 文档却会悄悄过时。通过 `/maintain-verification-skill` 反复检查；如果是应用坏了，就报告问题，而不是修改记忆。
- **删除什么**……删除临时拼凑的规避方法，以及实现同一件事的第二种写法。两者都会成为下一位 Agent 的示例。遵照「**Bug fix**」Playbook，如果证据否定了试探性变更所依据的假设，就撤销变更。推荐的写法应收敛为一种，并用 lint 阻止其他写法。具体选择由人来决定。

- **如何持续**……维护代码库如同维护花园，不只是增加，还要不断清除。pstack 提供将重复意见变成机制的 Principle，以及检查被关闭的检查项的 /no-comments 等维护组件。

至此，信任的对象（第 2 章）、提高信任的方法（第 3 章）和传递信任的记忆（第 4 章）已经齐备。第 5 章将讨论演讲的最后一个主题：自动化。我们会讲解为什么把自动化放在最后、可以先自动化哪些工作、能够自动化什么，以及自动化之后人还需要做什么。

第 5 章：在信任的基础上建立自动化

kaito • 来源：日本語 • English • 2026-10-03

自动化只有在成果物能够验证、工作步骤已经确定、过去的判断保存在代码库中时，才能发挥作用。

因此，poteto 在演讲中把自动化放在四个主题的最后。

本章依次讲解为什么要把自动化放在最后、哪些工作可以先自动化、基础齐备时能自动化什么，以及自动化之后人还要做什么。

本章结构

本章按以下顺序展开。

- 为什么把自动化放在最后
- 哪些工作可以开始自动化
- 基础齐备时能自动化什么
- 自动化之后，人还要做什么
- 小结：自动化是建立在信任和记忆之上的最后一步

为什么把自动化放在最后

原因有以下两个。

1. 没有基础就自动化，增加的不是成果，而是无法检查的成果物和反复出现的失败
2. 仅凭 Agent 的能力无法维持稳定的质量，必须先有能让它发挥能力的环境

没有基础的自动化会增加无法检查的成果物

先看第一个原因。演讲按缺失的基础分别说明，在基础欠缺时自动化会发生什么。

缺失的基础	自动化后的结果
无法信任工作成果（第 2 章、第 3 章）	无法确认正确性的成果物增加
没有留下过去的判断（第 4 章）	一再重复同样的失败
多种实现方式毫无秩序地保留下来（第 4 章）	大量产生不理想的模式

这些情况的共同点是：**自动化会把基础本身已有的问题原样大量复制。因此，自动化要放在三项基础齐备后的最后阶段。**

米其林餐厅的厨房不仅需要厨师的本领，还需要步骤和工具

第二个原因通过米其林餐厅的厨房这个比喻来说明。

演讲将 Agent、开发工具、验证与自动化结合起来，持续制作软件变更的机制称为「软件工厂」，也把同样的机制比作「米其林餐厅的厨房」。

一家能够稳定提供高品质料理的厨房，只有优秀的厨师还不够，还需要以下机制。

- 烹饪步骤
- 工具
- 食材管理
- 卫生标准
- 职责分工
- 质量检查机制

Agent 开发同样如此：**不能只期待 Agent 自身的能力，而必须设计一个使它安全发挥能力的环境。**

第 2 至第 4 章讨论的验证工具、工作步骤，以及作为记忆的代码库，正是这个环境的组成部分。

哪些工作可以开始自动化

可以自动化的是**同时满足以下五个条件的工作。**

每个条件都已在前面的章节讨论过。

自动化前需要具备的条件	本书相关章节
能够直接验证成果物	第 2 章、第 3 章（方法 1）
已经定义了高质量的工作步骤	第 3 章（方法 2）
过去的判断保存在代码库中	第 4 章
推荐的实现方式已经明确	第 4 章（一条铺好的路）
能够自动阻止不理想的实现	第 3 章（将教训留在机制中）、第 4 章

基础齐备时能自动化什么

基础齐备后，可以自动化处理用户提交的缺陷报告，让 Agent 自动尝试复现。若验证 Skill 和 Feature Map 足够完善，还有自动推进到修复的设想。

演讲描述的最终形态是：具备上一节的条件后，组合多个 Agent 和工具，持续实现、验证软件变更并将其发布到生产环境。下面先看组成这一机制的要素，再看自动复现报告的例子。

先具备条件，再组合各项组件

演讲列出以下四个组成该机制的要素。它们都要在上一节的条件具备之后组合使用。

- **Grok Bot**……一款 AI Bot 应用，可以将 pstack 作为插件安装使用
- **Cloud Agents**……运行在 Cursor 云端基础设施（虚拟环境）上的 Agent
- **automations**……在预定时间或事件发生时运行处理的机制
- **Agent SDK**……通过程序调用 Agent 的开发套件

《The Complete Guide to pstack》的 Part 1 如下说明其中三项的使用方式。

- **Cloud Agents**……每个 Agent 都在专属的真实计算机上运行，可以安装依赖、运行应用，并录制视频或拍摄截图
- **Grok Bot**……不让 Bot 自己执行具体工作，而让它启动 Cloud Agents。这样 Bot 可以腾出手，也不会因阅读代码、运行命令等琐碎交互而耗尽自己的上下文窗口；**Bot 作为管理和监督 Cloud Agents 的协调者**
- **Grok Bot 的 routines、Cursor Automations**……对验证 Skill 满意之后，将其接入这些机制，让它在预定时间或事件触发时执行

自动复现报告的结果直接取决于基础的质量

自动复现报告，是一个能清楚显示自动化成效取决于基础质量的例子。

《The Complete Guide to pstack》Part 1 提出以下自动化用法。

1. 让 Bot 监听接收用户反馈的 Slack 频道或内部反馈频道
2. 每当报告到来时，自动让 Cloud Agents 尝试复现
3. 如果验证 Skill 和 Feature Map 足够好，还可考虑自动推进到修复

第三项附带「如果验证 Skill 和 Feature Map 足够好」这一条件。由此可以看出，**自动化能推进到什么程度，取决于用于操作并检查应用的工具（第 3 章），以及按功能记录用法和验证方式的记忆（Feature Map，第 4 章）的质量。**

自动化之后，人还要做什么

人仍要**决定方向，并承担最终责任。**

Agent 在步骤与约束明确的环境中工作。如果这个环境足够完善，人不必时时在旁监督；即使人离开座位，Agent 也能保持一定的工作质量。

pstack 入口 `/poteto-mode` 的「**Autonomy**」一节具体界定了人和 Agent 的边界：**可撤销的工作可以不经人的确认继续进行；在向共享分支 force-push、部署、删除数据、向客户发送消息等不可撤销的写入之前，则必须等待人的决定（第 35 章）。**

这条边界背后的思路也体现在 Principle「**Never Block on the Human**」中，将在第 20 章讨论。

同一条边界也写在各个 Playbook 中。负责将 PR 推进至可合并状态的「**Babysit**」Playbook (`playbooks/babysit.md`) 不会自行合并。如果人明确要求合并，它会交给负责合并的「**Shipping**」Playbook。即使所有检查都通过，**随附指南**也说明不自行合并的原因：合并是一项独立的决定。

演讲还谈到更远的前景：如果代码库能够向形式化验证代码正确性的方向发展，对 Agent 成果物的信任就会增强。或许终有一天，人工代码审查本身会成为例外，而非常规工作。

小结：自动化是建立在信任和记忆之上的最后一步

- **为什么把自动化放在最后**……没有基础就自动化，会增加无法检查的成果物、反复出现的失败和不理想的模式。正如演讲用米其林餐厅的厨房作比喻，除了 Agent 的能力，还需要能让能力发挥出来的环境。
- **哪些工作可以开始自动化**……从满足本书整理出的全部五个条件的工作开始：能够直接验证、步骤明确、判断得到保存、推荐的实现方式明确、能自动阻止不理想的实现。
- **能自动化什么**……可以自动化复现用户缺陷报告；若验证 Skill 与 Feature Map 足够完善，还有自动推进到修复的设想。Grok Bot、Cloud Agents、automations 和 Agent SDK 应在条件具备后组合使用。
- **人还要做什么**……决定方向并承担最终责任。/poteto-mode 规定，在不可撤销的操作前必须停下来等待。

第一部分没有过多讨论 pstack 的具体组件，而是沿着演讲的四个主题（信任、提高信任的方法、作为记忆的代码库、自动化）探讨委派工作的条件。第二部分将讨论 pstack 如何通过 Skill、Playbook 和 Principle 这些组件实现这些条件。

第二部分：什么是 pstack

kaito • 来源：日本語 • English • 2026-10-03

第二部分通过 pstack 的设计，讲解「**pstack 用哪些组件实现第一部分提出的条件**」。

先说结论：pstack 通过**统一入口、只在需要时读取所需内容**的设计来实现这些条件。

用户基本上只需记住一个入口：`/poteto-mode`。

`/poteto-mode` 根据请求的类型选择一个 Playbook；Playbook 决定工作的顺序，并在各步骤调用所需 Skill。在此过程中，Principle 作为「是否要采取这一步」「工作是否尚未完成」等判断的标准。

pstack 不会一开始就读取全部内容。它只打开符合请求的那个 Playbook；对于 Principle，则先只读清单，在条件适用时才读正文。这种设计使**严格遵循步骤与节省 token**得以兼顾。

第二部分能学到什么

阅读第二部分后，你会明白以下几点。

- pstack 想解决什么问题，以及它为何而建
- 把请求交给 `/poteto-mode` 后会发生什么，以及如何写好请求
- Playbook、Skill、Principle 各自负责什么，有什么区别
- 「在需要时调用步骤与判断标准」的机制及其好处

第三部分到第五部分会详细讨论 pstack 的所有项目。第二部分是阅读这些细节的地图。掌握组件之间的分工后，就能清楚理解每个项目「属于哪个组件、承担什么作用」。

本部分的章节

章	作用
第 6 章：pstack 解决什么问题	pstack 的整体面貌与目的：它把什么视为问题，追求什么目标
第 7 章：/poteto-mode——将请求接入适当工作方式的入口	入口的用法：请求如何分配给 Playbook，以及如何写请求
第 8 章：Playbook、Skill、Principle 分别负责什么	三种组件的作用和区别，以及子 Agent 的定位
第 9 章：为什么 pstack 只读取所需内容	内容读取的设计：为什么不让 Agent 始终读取所有内容

第 6 章讨论「为什么需要」，第 7 章讨论「如何使用」，第 8 章和第 9 章讨论「内部如何运作」。开篇介绍的重复行请求，将在第 8 章再次出现：该请求被分配给「**Bug fix**」Playbook，作为追踪 Skill 和 Principle 在哪些环节发挥作用的示例。建议按顺序阅读各章。

第 6 章：pstack 解决什么问题

kaito • 来源：日本語 • English • 2026-10-03

pstack 是为回答「AI 写了太多粗糙代码」这个问题而制作的插件。借用 README 中的话，其理念是「**想加快速度，先深入工作**」；它是以较少代码开发高质量产品的工具。

本章讲解 pstack 面对的问题及其回答、它的内容与第一部分四个主题的对应关系，以及适用场景。

第一部分的四个主题

1. 通过检查成果物建立信任（第 2 章）
2. 通过验证工具、工作步骤和代码结构提高信任（第 3 章）
3. 将代码库视为继承以往判断的记忆（第 4 章）
4. 在这些基础上进行自动化（第 5 章）

本章结构

本章按以下顺序展开。

- pstack 想解决什么问题
- pstack 如何回答这个问题
- pstack 包含什么、不包含什么
- pstack 的组件如何对应演讲的四个主题
- pstack 适合什么工作，应如何使用
- 小结：以「先深入工作」回应粗糙代码的问题

pstack 想解决什么问题

pstack 想解决的是AI 写出太多粗糙代码（slop）的问题。

pstack 是 poteto 公开发布的 Cursor 插件，README 开篇就提出了这个问题。

there's a growing sense that ai writes too much slop code. i agree. i don't want to ship like a team of twenty slop artists. throughput without quality is not a goal i aspire to. if you want to go fast, go deep first.

人们越来越觉得 AI 写了太多粗糙代码（slop）。我也这么认为。我不想像一支由二十个粗糙代码制造者组成的团队那样交付。没有质量的产出量不是我追求的目标。想加快速度，先深入工作。

她明确表示，pstack 是她对这个问题给出的「my answer」。

pstack 如何回答这个问题

pstack 的回答包含以下三项方针。

1. 写得更少
2. 先与一位 Agent 深入合作；确认可以信任它写出高质量、可验证的代码后，再并行执行
3. 按角色选用模型

在 README 中，提出问题之后以粗体开头的三个段落，分别对应这三项方针。

- **pstack is my answer.**……目标不是让代码行数最多，而是写出更少、更高质量的代码
- **pstack gives you fearless parallelism.**……先与一位 Agent 深入合作；若能信任它写出高质量、可验证的代码，就能有信心地并行执行
- **cursor gives you the best of all worlds.**……每种模型都有优缺点，而 pstack 可以使用其中任何一种；许多 Skill 结合多种模型，发挥各自的长处

写得更少，不等于删掉证据

第一项方针认为，最大化代码行数恰恰与目标背道而驰。Principle 「**Laziness Protocol**」（选择最小的变更，优先删除；见第 17 章）体现了这项方针。

然而，「写得更少」并不意味着连用于证明结果的代码也删掉。「**Bug fix**」Playbook ([playbooks/bug-fix.md](#)) 的前言这样界定应当删除的内容。

Every shipped line traces to runtime evidence. Belt-and-suspenders that "might help" is a hypothesis, not a fix.

每一行交付的代码都应追溯到运行时证据。「也许有用」的额外保险措施是假设，不是修复。

Agent 不加入「以防万一」的变更，只加入为修复实际运行并确认的问题所必需的变更。

另一方面，该 Playbook 的「确定提交顺序的步骤（步骤 5）」要求先于修复提交能够失败的复现测试（测试成本很高等情况可以省略）。**要删掉的是没有依据的「以防万一」的措施或变更；用于确认缺陷的复现测试等证据代码，不属于应删除的东西。**

信任一位 Agent 的成果物后，再并行执行

第二项方针是：先让一位 Agent 深入工作，确认可以信任它写出可验证的好代码，然后才有信心地并行执行。poteto 在《The Complete Guide to pstack》的 [Part 1](#) 中，也按「信任 → 并行执行」的顺序叙述：先借助验证 Skill 合并几个 PR，再考虑并行。

按角色选用模型

关于第三项方针，根据 README，0.15.5 时点的默认模型分配如下。

- **负责写代码的委派对象**（Feature、Refactoring、Bug fix、Perf、Hillclimb）.....Grok（xAI 的模型）
- **最困难的变更、写作和判断**.....Opus 5.5（Anthropic 的模型）
- **评议小组（让多种模型分别回答同一问题再比较的机制）**.....Opus 5.5、GPT-5.6 Sol 和 Grok

这些分配可通过 `/setup-pstack` 修改。分配机制将在第 34 章讨论。

使用多种模型还有不止个人偏好的意义。`/poteto-mode` 的「Subagents」一节这样定义第二意见。

A second opinion is the same prompt against a different model. Agreement is high-signal.

第二意见，就是让不同模型回答相同的提示词。答案一致，是一个可信度较高的信号。

Agent 让另一种模型回答同一个问题，检查答案是否一致。

例如，分别让 Opus 5.5 和 Grok 4.7 判断缺陷的原因；如果两者指出同一原因，这种判断就比单一模型的回答更可信。在 **pstack** 中，使用不同模型是验证手段之一。

pstack 包含什么、不包含什么

0.15.5 时点的 pstack 包含五类组件。另一方面，**Playbook** 的步骤所使用的一些 **Skill** 位于其他插件或 **Cursor** 本体中，并未包含在 pstack 内；pstack 也没有专门用于撰写计划书的 **Skill**。

包含五类组件

以下数量均为 0.15.5 ([commit 12d587d](#)) 时点的值。

组件	数量	位置	作用
Playbook	23	skills/poteto-mode/ playbooks/*.md	按工作类型编写的步骤文件，是 /poteto-mode 内部的参考文件
Principle	23	skills/principle-*/ SKILL.md	判断标准，每项原则各有一份简短的 Skill 文件
Skill	24	skills/*/SKILL.md（不包括 Principle）	可调用的能力，包括 /poteto-mode 和 /setup-pstack
subagent	2	agents/poteto-agent.md、 agents/comment-sicko.md	由主 Agent 启动的其他 Agent 的定义
随附指南	10 页	docs/guide/*.md	pstack 随附的指南，附有请求示例，涵盖从第一项任务到夜间运行

有三点容易误解。

- **Playbook 不是 Skill**.....Playbook 是 `/poteto-mode` 按请求打开的参考文件，不能单独调用。poteto 也在《The Complete Guide to pstack》的 [Part 2](#) 中说明，Playbook 是 `/poteto-mode` 内的参考文件，而非 Skill。第 8 章将讨论这一区别。
- **Principle 在文件形式上是 Skill**……但 README 没有把它们放在「skills」表中，而是另设「principles」表。本书也分别统计 24 个 Skill 和 23 个 Principle。
- **随附指南与 X 上的《The Complete Guide to pstack》是两种不同资料**……本书将 `docs/guide/` 称为「随附指南」，将 X 上的文章分别称为《The Complete Guide to pstack》Part 1 和 Part 2。

部分被步骤点名使用的 Skill 并未随附

`/poteto-mode` 也通过名称引用未随附的 Skill。如果缺少它们，部分步骤就无法执行。列表见 README 的「[not shipped here](#)」一节。

引用的组件	实际位置	在 <code>/poteto-mode</code> 中的用途
<code>/deslop</code> (deslop Skill)	cursor-team-kit 插件	提交前清除带有 AI 痕迹的粗糙代码
<code>control-cli</code>	cursor-team-kit 插件	操作 CLI 和 TUI（在终端中运行、有画面的应用）进行检查
<code>control-ui</code>	cursor-team-kit 插件	操作浏览器、Electron（构建桌面应用的框架）和 Web UI 进行检查
<code>/create-skill</code>	Cursor 内置	编写 SKILL.md 时遵循的规范
<code>/autopilot</code> (原名 <code>/babysit</code>)	Cursor 内置	检查 PR 状态（pstack 使用「 Babysit 」Playbook；见第 14 章）

README 建议，如果要配齐整套工具，也安装 `cursor-team-kit`。

尤其重要的是 `control-cli` 和 `control-ui`。这两个操作应用的 Skill 用于「**Bug fix**」Playbook 中的「自己复现缺陷的步骤（步骤 1）」。只安装 pstack，可能没有操作应用以复现和检查问题的手段。

如果已经有自己应用专用的验证 Skill（由 `/create-verification-skill` 创建的 `verify-<app>`），它就可以充当操作手段。如果两者都没有，就按照第 36 章的步骤，先建立验证 Skill。

因为认为「最好的规格是代码」，pstack 没有撰写计划书的专用 Skill

pstack 没有专用的计划 Skill，是因为 poteto 认为代码比计划书更适合作为规格。README 的「why are there no planning skills?」一节认为 Cursor 的计划模式与 pstack 很契合，同时写道「最好的规格是代码」（第 38 章）。

同一处还说，如需制订计划，可以由 /poteto-mode 处理，但它不是默认做法。poteto 在《The Complete Guide to pstack》Part 2 补充说，她并非不做计划，而是通过代码进行计划；她也不会对抽象计划开展对抗式审查。

在 pstack 内，通过代码规划主要由以下三个组件承担。

- 「**Prototype**」 Playbook ([playbooks/prototype.md](#)) ……用可以丢弃的原型，以低成本作出设计和行为方面的判断。/poteto-mode 的「Non-negotiables」一节要求：**如果正要问人一个运行后就能观察到答案的问题，应通过这个 Playbook 制作小原型，查看运行结果后再决定。**
- /architect……在实现前，确定调用该函数的代码会传入什么、得到什么，并据此设计类型和模块边界的 Skill。
- 「**Multi-phase or multi-PR plan**」 Playbook ([playbooks/multi-phase-plan.md](#)) ……需要计划书时的方案。不过，其第一步指出：如果改动只涉及一两个文件，方向也很明确，就应省略计划。下一步则要求在撰写计划书前，用原型解决未回答的问题。

第三项「**Multi-phase or multi-PR plan**」 Playbook 确实用于制作计划书，即便如此，它仍要求先运行原型，再撰写文字。**以代码为规格的立场，不仅体现在有没有专用 Skill，也体现在 Playbook 的步骤中。**第 16 章和第 39 章会详细讨论如何处理计划。

pstack 的组件如何对应演讲的四个主题

第一部分逐章讨论了演讲的四个主题。pstack 内部各有对应的组件。本书将它们对应如下。

演讲主题（第一部分的章节）	pstack 中承担相应任务的组件	示例
信任：验证成果物（第 2 章）	验证 Skill、Verification（验证）组的 Principle（第 19 章）	/create-verification-skill、 「 Prove It Works 」
提高信任的方法：让验证与步骤可复现（第 3 章）	Playbook、支持步骤的 Skill	「 Bug fix 」、/tdd、/show-me-your-work

演讲主题（第一部分的章节）	pstack 中承担相应任务的组件	示例
作为记忆的代码库：保留判断与约束（第 4 章）	Meta 与 Architecture 组的 Principle（第 18 章、第 21 章）以及处理记录的 Skill	「 Encode Lessons in Structure 」、/reflect、/why
自动化：建立在基础之上（第 5 章）	用于自动化的 Playbook	「 Autonomous run 」、 「Orchestrate」

pstack 将步骤（Playbook）、判断标准（Principle）和能力（Skill）分开保存，再通过入口 /poteto-mode 一起调用。这个机制将在第 7 章至第 9 章讨论。

pstack 适合什么工作，应如何使用

pstack 适用于需要严谨性的工作。开始使用时，只需记住 /setup-pstack 和 /poteto-mode 这两个命令。

适合需要严谨性的工作

README 建议「每当你做任何需要严谨性的事」（whenver you're doing anything that requires rigor）时使用 /poteto-mode。另一方面，/poteto-mode 的 SKILL.md 要求**不要将其用于闲聊等轻量交流（Casual turn）**（第 7 章）。

据此，可以将适合与不适合的工作区分如下。

- **适合的工作**……需要复现和验证的缺陷修复、涉及函数或模块边界设计的功能开发、希望自己离开时交由 Agent 完成的长期工作等
- **不适合的工作**……改正一行错字，或只需一句话就能回答的问题等

不过，即使工作适合，如果没有运行应用进行验证的手段，pstack 的步骤仍无法执行。「**Bug fix**」Playbook 要求的「自己复现」与「在同一操作界面（缺陷发生的同一画面或命令）验证」，都以具备验证手段为前提。因而顺序应是**先准备验证 Skill，再使用 pstack**。

小结：以「先深入工作」回应粗糙代码的问题

- **要解决的问题**……pstack 是对「AI 写出的粗糙代码」的回答。README 写道：「想加快速度，先深入工作。」

- **回答问题的方式**……三项方针是：写得更少；先与一位 Agent 深入合作，确认可以信任它写出高质量、可验证的代码，再并行执行；按角色选用模型。
- **包含和未包含的内容**……0.15.5 时点包含 23 个 Playbook、23 个 Principle、24 个 Skill、2 个子 Agent、10 页随附指南。/deslop、control-cli、control-ui 位于 cursor-team-kit，/create-skill 内置于 Cursor；特别是缺少操作 Skill 时，就无法复现和验证。没有专用的计划 Skill，是因为其立场是「最好的规格是代码」，这一立场也体现在计划 Playbook 的步骤中。
- **与演讲四个主题的对应关系**……信任、提高信任的方法、作为记忆的代码库、自动化，分别有对应的组件。
- **适用场景和起步方式**……适用于需要严谨性的工作。开始时只需记住 /setup-pstack 和 /poteto-mode。

下一章第 7 章将详细介绍入口 /poteto-mode：如何调用、调用后会发生什么、将请求分配给 Playbook 的规则，以及怎样写好请求。

第 7 章：/poteto-mode，让请求进入合适工作流的入口

kaito · 来源：日本語 · English · 2026-10-03

/poteto-mode 是 pstack 的入口。它阅读请求，从 23 个 Playbook 中选择一个，并按其步骤推进工作。

用户无需指定使用哪个 Playbook，也无需指定按什么顺序调用哪些 Skill。用户要做的只是用自己的话说明目标和验证目标的方法。

本章讲解如何调用 /poteto-mode、调用后会发生什么、请求如何分配给 Playbook，以及怎样写好请求。

本章结构

本章按以下顺序展开。

- 在请求开头加上 /poteto-mode 来调用它
- 调用后会发生三件事
- 按既定规则将请求分配给 Playbook
- 好的请求会用自己的话写出目标和完成条件
- 小结

在请求开头加上 /poteto-mode 来调用它

将 /poteto-mode 加在工作请求的开头，即可调用它。

调用一次之后，在同一段对话的后续轮次中，即使不再写 /poteto-mode，它仍然有效（见后文）。如果每次开始对话时添加它很麻烦，也可以将 /poteto-mode 固定为 Custom Mode（让 Cursor 每轮使用同一个 Skill 的模式）。

基本用法：加在请求开头即可

pstack 的 [README](#) 将 `/poteto-mode` 定为所有非简单工作的默认入口，并举出以下例子。

```
/poteto-mode this pr has a subtle bug where the scroll drifts every 750ms even when idle. repro
first, then fix and verify.
// 这个 PR 有个不易察觉的缺陷：即使处于空闲状态，滚动位置每 750ms 仍会偏移。先复现，再修复，然后验证。
```

这段请求没有写要使用哪个 Playbook 或 Skill，只写了出现的症状（即使空闲，每隔 750 ms 滚动位置仍会偏移）和希望完成的工作（先复现、再修复并验证）。

`/poteto-mode` 会根据请求内容选择 Playbook 和 Skill，因此**用户只需写明症状和希望完成的工作**。

是否应用 `/poteto-mode`，取决于[下一节](#)将介绍的 reminder。它规定：遇到新任务时，只有在请求符合 Playbook 或需要严谨性时才应用 `/poteto-mode`；闲聊等轻量请求或问题则不应用。

所以我认为，当你犹豫要不要加 `/poteto-mode` 时，可以先加上。若只是轻量请求或问题，Agent 会直接回答，不走 Playbook 的步骤。

在 Cursor 中可以固定为 Custom Mode

如果每次请求都输入 `/poteto-mode` 很麻烦，可以将它固定为 **Custom Mode**。《The Complete Guide to pstack》的 [Part 1](#) 介绍了操作方法。

补全 `/poteto-mode` 时，不按 Enter 而按 `Opt + Enter`，Skill 就会被加入 **Custom Mode**；每轮开始时，Agent 都会收到使用该 Skill 的提醒。

提醒内容写在 `/poteto-mode` 的 frontmatter 中，键名为 `reminder`。

```
mode: true
reminder: New task? Playbook match or rigor needed -> apply /poteto-mode. Casual turn or user opts
out -> don't.
# 遇到新任务时，如果符合 Playbook 或需要严谨处理，就使用 /poteto-mode。闲聊回合以及用户要求停止时不要使用。
```

有了这一行，即使固定在 **Custom Mode** 中，面对闲聊等轻量请求或问题，或用户表示要退出该模式时，Agent 也不会执行 Playbook 的步骤。

调用一次后，在同一段对话中持续有效

README 将 `/poteto-mode` 称为持续模式（原文为 `sticky mode`）。意思是，第一次在请求中加上 `/poteto-mode` 之后，同一段对话的后续轮次无需重复添加，它仍然有效。

也就是说，用户在同一段对话中只需运行一次 `/poteto-mode`，下一轮起无需再显式添加。

在模式有效期间，也只有当请求符合 Playbook，或工作需要严谨性时，`/poteto-mode` 才会按 Playbook 的步骤推进。其他请求（如闲聊）不会走这些步骤。想停止时，告诉它即可。

因此，在同一段对话中继续工作时，请求可以很简短。随附指南的 `02-poteto-mode.md` 给出了以下例子。

```
/poteto-mode do it
// 就按这个继续。
```

```
continue
// 继续。
```

```
keep going until done
// 继续，直到完成。
```

同一页面解释说，简短请求之所以足够，是因为模式持续有效，而 Playbook 已经提供组织工作的步骤。**用户的话说明想做什么（意图），Skill 负责按步骤严谨地推进。**

不过，因为 `/poteto-mode` 持续有效，在同一段对话中转向其他话题时，新请求可能被当成前一任务的延续。本章的「换话题时，说明『新任务』」一节会介绍如何提出新话题。

调用后会发生三件事

调用 `/poteto-mode` 后，它会把 Playbook 的步骤抄进 TODO 列表、按步骤调用其他 Skill，并按规定的方式回复。这就是会发生的三件事。

TODO 列表.....Agent 开始工作时创建的待办事项清单（计划）。在 Cursor 中，它显示在聊天界面。用户可据此查看 Agent 执行了哪些步骤、跳过了哪些步骤（skip: <理由

>)

(随附指南 01-setup.md)。

README 对这三件事的说明如下。

1. 将请求与 Playbook 对照，打开 TODO 列表；开头的项目一字一句抄录所选 Playbook 的步骤
2. 随着步骤推进，将工作分配给所需的 Skill
3. 面向用户撰写去除 AI 腔调的回复

1. 将 Playbook 步骤原样抄进 TODO 列表

这是 /poteto-mode 的核心规则。其「Playbooks」一节开头规定：

Open a tolist whose first items are the matched playbook's steps, copied in verbatim, before any task-specific todos. A step you choose not to do stays in the list with a one-line skip: <reason>.

打开 TODO 列表，先将匹配的 Playbook 步骤一字一句抄在开头，再列任务专属的待办事项。即使决定不执行某一步，也要将它留在列表中，并附上一行 skip: <理由>。

Agent **不概括 Playbook 步骤**，而是直接以它们组成工作清单；不执行的步骤也保留并说明理由。

这项规则有两个作用。

- **Playbook 的顺序直接成为工作的骨架**……Agent 自行概括或重新安排步骤的空间变小。
- **跳过的步骤清晰可见**……不执行的步骤也以 skip: <理由> 形式保留，因此人不仅能检查结果，也能检查 Agent 途中作出的判断。随附指南的 01-setup.md 也说，查看 TODO 列表，就能知道 Agent 决定不做什么。

2. 在步骤要求时调用其他 Skill

Playbook 的每个步骤都写明何时使用什么 Skill。/how 和 /architect 并非启动时一次性调用，而是在步骤推进到对应场景时才调用。这个机制将在第 8 章和第 9 章讨论。

3. 按规定方式撰写回复

回复的写法由 /poteto-mode 的「Writing the reply」一节规定，例如：

- 用简短的断定句，每句话只表达一个意思
- 先说明工作为谁而做，以及会带来什么变化
- 每项主张附上证据，或标明「测量所得、根据证据推断、猜测」

所有 Playbook 最终都按这种方式回复，各 Playbook 末尾的「Reply」行则写明该 Playbook 应报告的内容。完整规则将在第 35 章讨论。

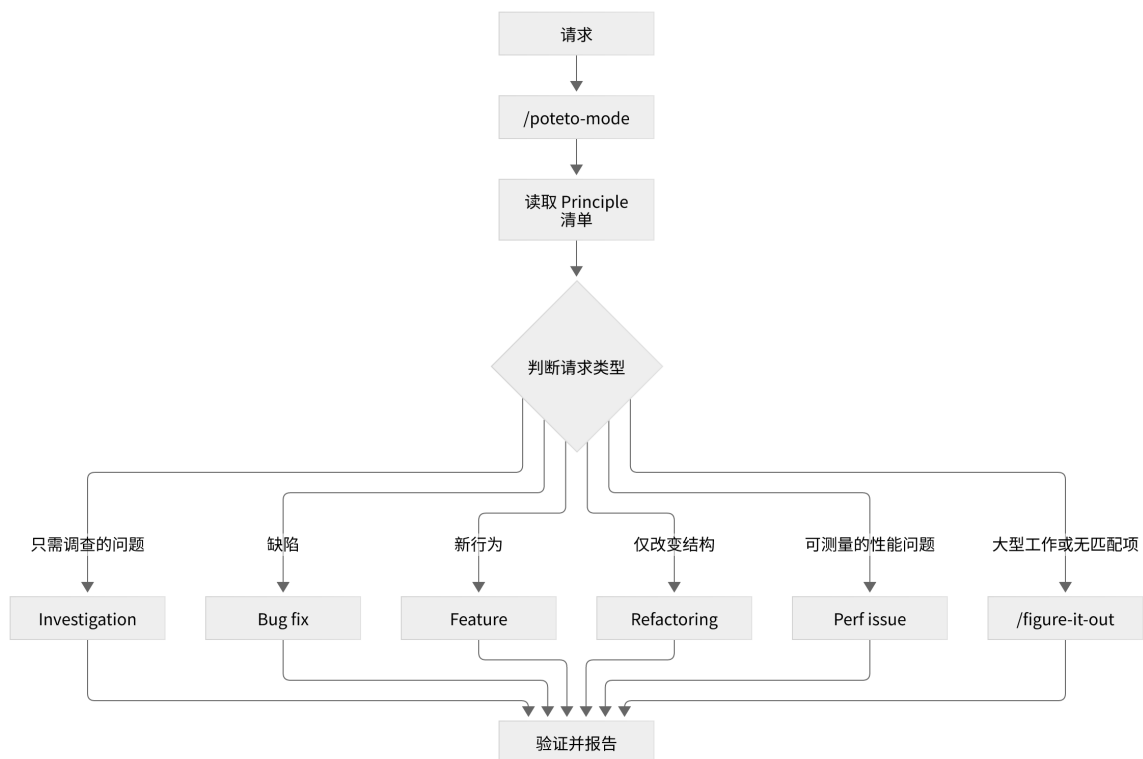
按既定规则将请求分配给 Playbook

分配的基本规则是：**根据请求的类型（只需调查的问题、缺陷、新行为等）选择一个适合的 Playbook。**

不过，**当工作规模很大时，分配目标由工作规模而不是请求类型决定。**即使请求看起来符合「Feature」等 Playbook，也可能被交给 /figure-it-out 或「Orchestrate」。

常见分配目标：五个 Playbook 和一种例外

要把握分配机制的整体情况，先了解五个代表性的 Playbook，以及没有匹配 Playbook 时的一种例外（/figure-it-out）就够了。我把随附指南 02-poteto-mode.md 中的分配图译成日文并重画如下。



查看图示 1

图中只列出代表性的分配目标。其他工作也有对应的 Playbook，包括：

- 持续改进指标
- 诊断运行时症状或已采集的追踪记录
- 制作原型
- 匹配视觉外观
- 创建和评估 Skill
- 长时间自主运行
- 创建 PR
- 跟进 PR 并部署到生产环境
- 自动处理 PR 队列
- 统筹项目规模的工作
- 工作交接与中断
- 多阶段计划
- 清理 worktree（Git 可从一个仓库建立多个工作副本的功能）

全部 23 个 Playbook 将在[第三部分](#)详细介绍。

图中在判断请求类型前，还有「读取 Principle 清单」这一步。[第 9 章](#)会介绍 Principle 的读取方式。

大型工作交给 /figure-it-out，项目规模工作交给「Orchestrate」

工作规模大时，/figure-it-out 或「Orchestrate」Playbook 的优先级高于单项 Playbook。/poteto-mode 的「Playbooks」一节有两条规则，都会根据工作规模改变分配目标。

第一条规则涉及 /figure-it-out，原文如下。

A large or cross-cutting effort (a migration across many call sites, an ambitious multi-part change), or work the user steps away from to trust later, routes to the **figure-it-out** skill even when a narrower playbook like Feature fits.

大型或跨领域工作（涉及众多调用位置的迁移、有多部分的大规模变更），或用户离开后希望日后能信任成果的工作，即使符合 Feature 等范围更窄的 Playbook，也应分配给 **figure-it-out Skill**。

Agent 会优先根据工作规模，而非请求的措辞，决定分配目标。

没有任何 Playbook 匹配的工作，也交给 /figure-it-out。

/figure-it-out 是为当前任务设计严谨、可审计 Playbook 的 Skill。

第二条规则涉及「Orchestrate」。它负责历时多天、项目规模的工作，由一位协调者 Agent 和许多子 Agent 推进大量累积的 PR。

两者的区别是：/figure-it-out 为这一次工作设计推进方式（Playbook）；「Orchestrate」则运营持续多天的整项工作。

不过，「Orchestrate」也明确规定，如果一位 Agent 能在单次会话的预算内完成，就应交给「**Autonomous run**」。

这些规则可归纳如下。

请求的性质	分配目标
符合一个 Playbook、规模普通的工作	相应的 Playbook

请求的性质	分配目标
大型跨领域工作、用户离开后希望日后检查的工作、没有匹配 Playbook 的工作	/figure-it-out
历时多天、涉及众多 PR 和子 Agent 的持续项目	「Orchestrate」
一位 Agent 可以完成的长任务（例如「继续到完成」）	「Autonomous run」

好的请求会用自己的话写出目标和完成条件

归纳随附指南（docs/guide/）的建议，写好请求要注意四点。

1. 写出目标及验证方式
2. 不要罗列 Skill
3. 不要把工作时长当作完成条件
4. 换话题时说「新任务」

写目标，而不是写步骤

随附指南 02-poteto-mode.md 说明，请求无需详尽的规格书。说明哪里不对、想实现什么，以及已经知道的有助于工作的情况就足够了。

指南给出以下缺陷请求示例。

```
/poteto-mode users get two notifications after a retry. repro first, then fix and verify.  
// 重试后，用户收到了两次通知。先复现，再修复，然后验证。
```

指南指出，其中的「先复现」不是礼貌性的修饰，而是**Playbook 必须遵守的真实约束**。实际上，该请求会分配给「**Bug fix**」Playbook，TODO 列表从复现步骤开始。

随附指南的目录也总结说，如果只记住一件事，那就是**用自己的话说明目标和验证目标的方法**。

怎样写验证方式，可以参考第 2 章介绍过的随附指南示例（「文本输出一个字节也不能变，JSON 必须能够解析」）。两项条件都能由 Agent 自行执行并判定通过或失败。

请求中还可以写的不只有验证方式。《The Complete Guide to pstack》的 [Part 2](#) 中，有些请求示例还指定了希望收到的结果形式（分别说明已知事实、使用的数据、有力的假设），以及审查时机（继续之前先让我确认）。

不要罗列 Skill

随附指南 02-poteto-mode.md 将在请求里罗列 Skill 列为一个陷阱。

Pitfall: don't enumerate skills in your prompt ("use /how, then /architect, then /arena..."). The playbook already sequences them, and a hand-written sequence usually reorders or drops steps the playbook would have kept. Name a skill only when you want to override a specific choice.

陷阱：不要在提示词中罗列 Skill（「先用 /how，再用 /architect，最后用 /arena.....」）。Playbook 已经安排了顺序；手写的顺序往往会重排或漏掉 Playbook 原本会保留的步骤。只有想覆盖某个特定选择时，才点名 Skill。

例如，在缺陷请求中写「先用 /how 调查，再用 /architect 设计」，就可能使「**Bug fix**」Playbook 放在第一步的「自己复现」从手写顺序中消失。指南目录开篇也写道，**停止对 Agent 进行细节上的管控时，pstack 才最能发挥作用。**

上一节介绍的「先复现」等措辞，会被视为 Playbook 应遵守的约束。分工是：**在请求里写约束，把推进工作的步骤交给 Playbook。**

「工作多少小时」不能作为完成条件

离开座位之前，先写下完成条件。随附指南 02-poteto-mode.md 有以下例子。

```
/poteto-mode im stepping away. keep going until the migration check reports zero old callers. log your decisions.
// 我暂时离开。继续处理，直到迁移检查报告旧调用方为零，并记录决策。
```

随附指南的夜间运行页面（07-overnight.md）将**把时间当作完成条件列为陷阱。**

因为「做这个四小时」没有给 Agent 任何可以检查的目标，早上留下的只有四小时的活动，而非结果。夜间运行将在第 15 章讨论。

上例中的目标「迁移检查报告旧调用者为零」，是 Agent 可以自行执行并判定通过或失败的标准。因此，**完成条件应写成可执行、可判定的检查，而非一段时间。**

换话题时说「新任务」

`/poteto-mode` 在同一段对话中持续有效，因此长对话会积累前一项任务的上下文。随附指南 `02-poteto-mode.md` 因而建议换话题时明确说「新任务」。

```
/poteto-mode new task. figure out why the cache entry survives logout. don't change any code yet.
// 新任务。查明为什么退出登录后缓存条目仍然存在。暂时不要修改代码。
```

也就是说，**「新任务」是让系统重新选择 Playbook 的信号。**

如果换话题时不作说明，正在执行的 Playbook 可能把新问题视为前一项工作的延续。

小结

- **调用方式**……`/poteto-mode` 加在请求开头，是 pstack 处理所有非简单工作的默认入口。调用一次后，在同一段对话中持续有效。在 Cursor 中还可以用 `Opt + Enter` 将其固定为 **Custom Mode**。
- **调用后发生什么**……选择 Playbook，将步骤一字一句抄进 TODO 列表，按步骤调用 Skill，再按规定方式回复。跳过的步骤以 `skip: <理由>` 形式留下，供人检查过程。
- **分配规则**……基本上按请求类型选择一个 Playbook。大型工作交给 `/figure-it-out`，持续性的项目规模工作交给「**Orchestrate**」。
- **好的请求**……用自己的话说明目标和验证方式，不罗列 Skill，不把工作时长当完成条件；换话题时说「新任务」。

下一章第 8 章将介绍 `/poteto-mode` 内部工作的三种组件：Playbook、Skill 和 Principle。我们会讲解各自的作用，并以本书的请求示例为线索，追踪「**Bug fix**」Playbook 的各个步骤中，Skill 和 Principle 在哪里发挥作用。

第 8 章：Playbook、Skill 与 Principle 各自负责什么

kaito · 来源：日本語 · English · 2026-10-03

pstack 的内容由**Playbook**、**Skill**、**Principle** 三类组件组成。

Playbook 决定工作的顺序。Skill 提供步骤中使用的能力。Principle 则是在特定情境下指导工作判断的规则，例如让变更更小、修复根本原因、报告完成之前检查实物。

本章先讲解三种组件的区别和职责，再依次介绍连接它们的 Non-negotiables、两种子 Agent，以及这些组件在实际请求中发挥作用的位置。

本章结构

本章按以下顺序展开。

- 三种组件分别负责「顺序」「能力」「判断」
- Playbook 决定不同类型工作的步骤和顺序
- Skill 是在步骤中调用的能力
- Principle 是工作途中的判断标准，而非步骤或能力
- Non-negotiables 将三种组件连接起来
- 两种子 Agent 是分担工作的对象
- 追踪 Skill 和 Principle 发挥作用的位置
- 小结

三种组件分别负责「顺序」「能力」「判断」

如开篇所述，三种组件各有分工。**Playbook** 负责「做什么、按什么顺序做」这一工作模式；**Skill** 提供「这一步怎样做」的能力；**Principle** 则指导「要不要采取这一步、是否尚未完成」等判断。

三种组件在 `/poteto-mode` 中按以下方式连接。

1. `/poteto-mode` 判断请求类型，选择一个 Playbook（见 `/poteto-mode` 的「[Playbooks](#)」一节）
2. Playbook 写明该类工作的步骤和顺序，例如修复缺陷时按「复现 → 原因 → 修复 → 验证」推进
3. Playbook 的各步骤写明何时使用哪个 Skill；`/how`（了解现状）和 `/arena`（比较多个方案）不会在启动时调用，而是在推进到相应步骤时才调用
4. Principle 是与步骤独立的判断规则；`/poteto-mode` 开始时只读清单（索引），当前工作符合条件时才阅读全文（见同一文件的「[Principles](#)」一节）

三者的区别可汇总如下。

角度	Playbook	Skill	Principle
负责的内容	工作顺序	步骤中使用的能力	判断标准
回答的问题	做什么，按什么顺序做	这一步怎样做	要不要采取这一步，是否还不能算完成
数量 (0.15.5)	23	24	23
位置	<code>skills/poteto-mode/playbooks/*.md</code>	<code>skills/<名前>/SKILL.md</code>	<code>skills/principle-<名前>/SKILL.md</code>
文件形式	<code>/poteto-mode</code> 内的参考文件，不是 Skill	独立的 Skill	每项原则各一份简短 Skill 文件
调用方式	<code>/poteto-mode</code> 选择并打开符合请求的一个文件	符合 Playbook 步骤或 Non-negotiables 条件时调用；也可像 <code>/how</code> 一样直接调用	开始时读清单，符合条件时读正文
是否接触代码	通过步骤发出指示	执行调查、编写、验证等工作	不接触代码，只作判断
示例	「 Bug fix 」、 「 Feature 」	<code>/how</code> 、 <code>/architect</code> 、 <code>/tdd</code>	「 Prove It Works 」、 「 Fix Root Causes 」

第 9 章将说明每种组件在什么时候被读取。

Playbook 决定不同类型工作的步骤和顺序

Playbook 是按工作类型编写「做什么、按什么顺序做」的步骤文件。

`/poteto-mode` 会选择并打开符合请求的那个 Playbook。

职责：用前言、步骤、报告三个部分规定做什么及顺序

Playbook 文件的结构大体相同。

1. **前言**……写明整个 Playbook 要遵守的基本规则。
2. **编号步骤**……会被一字一句抄进 TODO 列表的部分。每一步写明何时使用哪个 Skill、适用哪项 Principle。
3. **Reply 行**……位于每个 Playbook 的末尾，指定工作后回复中应写的内容。

TODO 列表……Agent 开始工作时制作的待办事项清单（计划）。在 Cursor 中显示于聊天界面。用户可以查看清单，确认 Agent 执行了哪些步骤、跳过了哪些步骤（skip: <理由>）（随附指南 [01-setup.md](#)）。

Skill 是在步骤中调用的能力

Skill 是完成特定工作所需能力与步骤的集合。

多数 Skill 在步骤需要时由 `/poteto-mode` 调用，其余则由用户直接调用。

职责：一项 Skill 负责一类工作

调查使用 `/how` 或 `/why`，设计使用 `/architect`，比较多个方案使用 `/arena`。每项 Skill 负责一类工作。

例外是 `/figure-it-out`，它为没有匹配 Playbook 的任务临时制作该任务专用的 Playbook。各 Skill 的详细内容将在[第五部分](#)（第 22~35 章）讨论。

调用方式：多数 Skill 在 Playbook 需要时由 `/poteto-mode` 调用

多数 Skill 由 `/poteto-mode` 代为调用，调用时机由 Playbook 的步骤决定。

24 项 Skill 按调用方式划分如下。

调用方式	Skill
由 /poteto-mode 按步骤或条件调用	/how、/why、/architect、/arena、/swarm、/interrogate、/unslop、/no-comments、/technical-writing、/tdd、/figure-it-out、/show-me-your-work
由用户调用	/setup-pstack、/recall、/teach、/bro、/blast-radius、/reflect、/automate-me、/make-bot-ui、/create-verification-skill、/maintain-verification-skill
处理 TypeScript 文件时读取	/typescript-best-practices (第 29 章)
入口	/poteto-mode

由用户调用的 Skill，主要在工作的重要节点使用，例如安装（/setup-pstack）、恢复工作（/recall）、长时间工作结束后（/reflect）。

用户怎样使用：任何 Skill 都可以明确调用，未纳入 Playbook 的十项由用户调用

与 Playbook 不同，Skill 可以单独调用。如果只想调查某件事，直接调用有时更快。

```
/how do we cancel runs? do we have an n+1 when we look up every run to cancel?
// 取消执行是怎么实现的？逐个查询待取消的执行时，会不会出现 N+1 查询？
```

由用户调用的十项 Skill 未被纳入 Playbook 步骤。

Principle 是工作途中的判断标准，而非步骤或能力

Principle 既不是步骤也不是能力，而是工作途中作出判断的标准。

开始工作时只读取索引，适用时才读取正文。

职责：提供工作途中作判断的标准

Principle 提供是否采取当前这一步、是否还不能算完成等工作途中判断的标准。例如，当 Agent 想添加抽象或层次时，Principle「**Laziness Protocol**」会使它优先考虑删除和最小变更。「**Prove It Works**」则会在宣布完成之前追问是否已经检查真正的成果物。

每项 Principle 是一份独立文件，Playbook 和 Skill 可通过名称引用。

全部 23 项 Principle 将在[第四部分](#)（第 17～21 章）详细介绍。

Non-negotiables 将三种组件连接起来

Non-negotiables 是一组「处于这种情况就使用这个 Skill」的条件；它将跨 Playbook 通用的规则连接起来。

Playbook 决定不同类型工作的流程，但还有适用于各种工作的共通规则。/poteto-mode 的「Non-negotiables」一节规定了这些规则。主要条件如下。

条件	调用的组件
非简单变更、设计判断	/how
跨函数边界的代码	/architect
所有写作场景（包括回复）	/unslop
文档、PR 描述、提交说明	/technical-writing
提交之前	/deslop（cursor-team-kit 插件）
审查之前	/no-comments
UI、IDE、CLI 变更	control 类 Skill（cursor-team-kit 插件）
长时间工作、用户离开后再查看的工作	/show-me-your-work

Non-negotiables 并非根据「工作类型」，而是根据**工作中出现的情境**作出反应。

无论执行「**Bug fix**」还是「**Feature**」，只要跨函数边界就会用到 /architect，提交前则会用到 /deslop。

两种子 Agent 是分担工作的对象

pstack 有两种子 Agent：依照 /poteto-mode 方式工作的 poteto-agent，以及审查注释的 **Comment Sicko**。

poteto-agent 是依照 /poteto-mode 方式工作的子 Agent

poteto-agent (agents/poteto-agent.md) 在开始工作前会阅读 /poteto-mode 的规则。在 Playbook 步骤中需要启动子 Agent 时，原则上使用它。

Comment Sicko 删除注释，并报告需要修改的代码

Comment Sicko (agents/comment-sicko.md) 是审查代码注释、亲自删除没有理由保留的注释的子 Agent。它不会改写应用代码本身。相反，它会在报告中把需要调整形式、以便没有注释也能表达清楚含义的代码（函数或变量等）标为 MUST KILL。收到报告后，负责修改代码的是 /no-comments。Comment Sicko 不直接调用，而是通过 /no-comments 使用。详见第 31 章。

主 Agent 对子 Agent 的工作负责

即使把工作交给子 Agent，责任仍在主 Agent 身上。

/poteto-mode 要求主 Agent 先查看差异，再用自己的话向用户报告，而不是原样转交子 Agent 的「做完了」。

这也是将第 2 章提出的「信任成果物，而非 Agent」应用到子 Agent 的规则。

追踪 Skill 和 Principle 发挥作用的位置

Skill 出现在复现、调查、设计、提交等需要「执行工作」的环节；Principle 出现在选择复现方式、修复形式和规模、判定完成、安排提交顺序等需要作判断并「做出决定」的环节。

本书的实际示例是以下请求。

```
/poteto-mode the export writes duplicate rows when a retry lands mid-run. repro first, then fix and verify.
// 如果重试发生在执行中途，导出会写入重复的行。先复现，再修复，然后验证。
```

该请求会分配给「Bug fix」Playbook，TODO 列表以它的六个步骤开头（第六步是执行「Opening a PR」来创建 PR，所以下表省略该步）。这里我们追踪每一步何时调用 Skill、何时运用 Principle。

下表将 `playbooks/bug-fix.md`、`/poteto-mode` 中 `Non-negotiables` 和 `Principle` 索引规定的条件，应用到这一请求。实际工作中选择哪些 `Principle`，取决于 `Agent` 当时如何判断索引中的条件是否成立。

每一步都决定了调用的 Skill 和适用的 Principle

「Bug fix」步骤	调用的 Skill	符合条件的 Principle
开始时	(由 <code>/poteto-mode</code> 读取 <code>Principle</code> 索引)	整个索引，此时不读正文
1. 自己在相同操作界面复现	相应的 <code>control</code> 类 Skill 或验证 Skill	「 Fix Root Causes 」(先复现)
2. 用二分法定位原因	<code>/how</code> (目标子系统)、 <code>/why</code> (回归的历史)	「 Fix Root Causes 」、 「Guard the Context Window」
3. 规划修复	跨函数边界时用 <code>/architect</code> ；实现工作交给子 <code>Agent</code>	「 Model the Domain 」、 「Make Operations Idempotent」 、 「Laziness Protocol」
4. 在相同操作界面验证	与「复现缺陷的步骤 (步骤 1)」相同的 <code>control</code> 类 Skill，或验证 Skill	「 Prove It Works 」
5. 先于修复提交失败的复现测试	如果有成本低的本地测试，就使用 <code>/tdd</code>	「 Sequence Work into Verifiable Units 」、 「Test Behavior, Not Implementation」
回复	<code>/unslop</code> (「Writing the reply」一节)	点名改变了判断的 <code>Principle</code>

各步骤的细节见第 11 章，`Principle` 发挥作用的场景将在第 17 章到第 20 章讨论。

不符合条件的 Skill 不会被调用

例如，`/show-me-your-work` 用于长时间工作或用户离开座位期间的工作，记录判断过程。本例的请求并没有说用户会离开，因此不符合这个条件。如果任务历时很长或用户会离开，则可能符合条件；但这种情况下，按第 7 章介绍的规则，也可能被交给 `/figure-it-out`。

即使同为「**Bug fix**」任务，请求的内容和途中发生的情境不同，调用的 Skill 与适用的 `Principle` 也会变化。只读取符合条件的内容，这一设计就是下一章第 9 章的主题。

小结

- **三种组件的区别**.....Playbook 负责工作顺序，Skill 提供步骤中使用的能力，Principle 指导是否采取当前步骤、工作是否尚未完成等判断。
- **Playbook**.....由前言、步骤和 Reply 行三个部分组成。
- **Skill**.....一项 Skill 负责一类工作。多数由 /poteto-mode 按步骤调用；由用户调用的十项未被纳入 Playbook 步骤。
- **Principle**.....不是步骤或能力，而是判断是否采取当前步骤、是否完成等问题的标准。详细内容见第四部分。
- **Non-negotiables**.....响应工作中出现的情境，跨 Playbook 调用 Skill，起连接作用。
- **子 Agent**.....poteto-agent 依照 /poteto-mode 的方式工作；Comment Sicko 删除注释，并报告需要修改的代码。主 Agent 仍对委派出去的工作负责。
- **实际示例**.....在「Bug fix」的各步骤中，复现与调查会调用 control 类 Skill 及 /how、/why，修复时会用到 /architect，提交时可用 /tdd；「Fix Root Causes」「Prove It Works」等 Principle 指导相应判断。

下一章第 9 章将讨论这些组件何时读取、只在需要时读取所需内容的机制，以及其背后的思路、优点与弱点。

第 9 章：为什么 pstack 只加载需要的内容

kaito · 来源：日本語 · English · 2026-10-03

pstack 并不会在开始时读取所有 Playbook、Principle 和 Skill。它先读的只有 /poteto-mode 本体与 Principle 索引。

Playbook 只读取符合请求的那个文件；Principle 正文只在条件适用时读取相应文件；Skill 则在 Playbook 推进到需要它的步骤时读取。

第 7 章和第 8 章介绍了各组件的调用方式。本章讲解把读取内容分为四层的设计、背后的思路，以及这种设计的优点与弱点。

本章结构

本章按以下顺序展开。

- pstack 将所读内容分为四层
- 这一设计与 Principle 「Guard the Context Window」采用相同的划分
- 这一设计有三项优点，其弱点可通过过程中的检查弥补
- 小结

pstack 将所读内容分为四层

pstack 读取的内容分成**入口、步骤、判断、能力**四层，其中始终需要读取的只有入口。

层	读取内容	读取时机
第 1 层：入口	/poteto-mode 的整个 SKILL.md	调用 /poteto-mode 时
第 2 层：步骤	符合请求的一个 Playbook 文件	将请求与 Playbook 对照时

层	读取内容	读取时机
第 3 层：判断	符合条件的 Principle 的 SKILL.md	当前工作符合索引中的条件时
第 4 层：能力	Playbook 步骤或 Non-negotiables 所要求的 Skill 的 SKILL.md	Playbook 推进到相应步骤时

第 1 层的 SKILL.md 不包含 Playbook 或 Principle 的正文。它包含的是适用于各种工作的共通规则，例如 Non-negotiables 和回复方式；还包括 Playbook 的一行说明与文件位置，以及 Principle 清单（索引）和适用条件。**第 1 层由说明「有哪些内容、何时使用」的目录和共通规则组成。**

四层之外，还有一份始终适用的小文件：`/setup-pstack` 写出的模型设置规则 `~/.cursor/rules/pstack-models.mdc`。这个文件每行写一个角色，内容很短，所以即使始终适用，占用的上下文也很少。详见第 34 章。

这一设计与 Principle 「Guard the Context Window」采用相同的划分

pstack 的读取设计与 Principle 「Guard the Context Window」对 Agent 的要求采用相同的划分方式。

每次都用的内容放在里面，按条件使用的内容放在外面

这项原则指出：**上下文窗口有限，每个 token 都应该物有所值**。上下文窗口被占满，会降低推理质量。

该原则还要求，把每次都用的参考内容放在 Skill 文件中。pstack 同样将每次都要用的 Principle 索引和 Non-negotiables 放在 SKILL.md 内，而仅在条件适用时使用的正文放在其他文件。即**每次都用的放里面，偶尔使用的放外面**。

向子 Agent 交接的方式也节省上下文

SKILL.md 的「Subagents」一节规定，向子 Agent 交接时，**默认传递文件位置，而非抄写上下文**（第 35 章）。子 Agent 会自己打开所需文件，主 Agent 无需累积并转交大量上下文。

这一设计有三项优点，其弱点可通过过程中的检查弥补

这一设计有以下三项优点。

- 1. **节省 token 与成本**……不使用的步骤和判断标准不会占用上下文，上下文窗口在填满之前便保有更多余量。poteto 也在《The Complete Guide to pstack》的 [Part 2](#) 中解释说，一次只读取一个 Playbook 是为了提高 token 效率。
- 2. **工作的一致性**……每个组件都在需要时读取完整正文，不必依赖 Agent 的记忆或转述。
- 3. **用户需要记住的东西更少**……用户无需记住 70 个项目的名称和用法，只需记住 /poteto-mode。

弱点：条件是否适用由 Agent 判断

这种设计的弱点是：**由 Agent 决定适用哪个 Principle 或 Playbook**。如果判断错误，本应读取的正文就可能没有被读取。

本书认为，pstack 用以下机制在工作途中检查并弥补这个弱点。

机制	用户能检查什么
跳过的步骤会以 skip: <理由> 形式保留在 TODO 列表中	哪些步骤没有执行，以及原因
在回复中点名改变了判断的 Principle（只能点名已读过正文的 Principle）	读了哪些标准，它们影响了哪些选择
只回复 Principle 名称就能使工作方向改变	可以当场纠正偏离的判断

TODO 列表……Agent 开始工作时创建的待办事项清单（计划），显示在 Cursor 的聊天界面。用户可查看清单，确认 Agent 执行了哪些步骤、跳过了哪些步骤（skip: <理由>）
（随附指南 [01-setup.md](#)）。

小结

- **四层**……pstack 在需要时分别读取入口、步骤、判断、能力四层。始终读取的只有入口 SKILL.md，其中包含目录、索引和共通规则。

- **设计背后的思路**……与 Principle 「**Guard the Context Window**」一样，将每次使用的索引放在内部，将按条件使用的正文放在外部。
- **优点与弱点**……优点是节省 token、保持一致性、易于记忆。对条件的判断由 Agent 负责，而用户可通过 TODO 列表和回复检查。

第二部分介绍了 pstack 解决什么问题、入口 /poteto-mode 如何分配请求、三种组件如何分工，以及如何读取各个组件。从接下来的第三部分开始，我们将逐一介绍 23 个 Playbook。首先是第 10 章，讨论不修改代码、只调查并诊断问题的 Playbook。

第三部分：用 Playbook 推进工作

kaito • 来源：日本語 • English • 2026-10-03

第三部分全面介绍 pstack 的全部 23 个 Playbook。

这一部分可以跳读。我建议只阅读自己感兴趣的章节。

各 Playbook 的原文随时可在 [pstack](#) 仓库中阅读。pstack 更新频繁；需要时再查阅相应章节和原文件，比预先读完全部内容更可靠。

第三部分将介绍什么

- 各 Playbook 适用于什么请求（例如：「**Bug fix**」适用于写着「先复现问题」的请求）
- 各 Playbook 的步骤及每一步的要求（例如：「**Hillclimb**」反复执行「一次改动、一次测量、保留或撤销」）
- 与相似 Playbook 的区别（例如：只想查明原因时用「**Runtime forensics**」，需要修复时用「Bug fix」）

Playbook 是将熟练工程师自然遵循的工作步骤整理成固定形式的操作说明。

例如，修复缺陷的顺序是「复现→确定原因→修复→在同一位置确认」。步骤明确后，无论由哪个 Agent 负责，都能按相同顺序推进工作。

Playbook 是 `/poteto-mode` 中的参考文件，用户不会直接调用它们。即便如此，了解其内容仍能知道该在请求中使用什么措辞。

本书介绍的请求示例，仅取自 pstack 原文（[随附指南](#)、[README](#)以及 poteto 的 X 帖文）中已有的例子。原文没有请求示例的 Playbook（Runtime forensics、Trace forensics、Hillclimb、Pause safely），本书也不提供请求示例。其中，Hillclimb 只说明随附指南中写明的「请求应提供的内容」。

本部分的章节

章	作用	涵盖的 Playbook
第 10 章 调查和复现问题	调查并给出答案或诊断；不修改代码	Investigation、Runtime forensics、Trace forensics
第 11 章 修复缺陷并改善性能	依据运行时证据修复缺陷和性能问题	Bug fix、Perf issue、Hillclimb
第 12 章 重做功能、结构与外观	添加功能、保持行为并调整结构、通过尝试作决定，以及统一外观	Feature、Refactoring、Prototype、Visual parity
第 13 章 编写并评估 Skill	编写决定 Agent 工作方式的 Skill，并衡量其效果	Authoring or modifying a skill、Eval
第 14 章 创建 PR 并将变更交付生产环境	创建 PR，使其达到 merge-ready 状态，并将已验证的部分并入生产环境	Opening a PR、Babysit、Shipping
第 15 章 将长期工作交给 Agent	持续推进一项长期工作，在暂停后恢复，并进行清理	Autonomous run、Pause safely、Session pickup、Worktree and simulator cleanup
第 16 章 规划多个 PR 并由多个 Agent 执行	规划跨越多个 PR 的工作，并由多个 Agent 执行	Multi-phase or multi-PR plan、Orchestrate、Autopilot-full、Autopilot-stack

第 10 章至第 12 章，依次对应调查、修复、构建这一项变更的流程。第 13 章介绍改进 Agent 工作方式的 Playbook；第 14 章介绍将变更交付生产环境的 Playbook；第 15 章与第 16 章介绍长期工作和多个 Agent 协作时使用的 Playbook。

全部 Playbook 速查表

Playbook	概要	使用场景	详解
Investigation	阅读源代码和变更历史来回答问题，并给出附有引用的解释或建议	用户提出能通过阅读代码或历史回答的问题，例如「X 如何运作」「Y 为什么这样设计」时	第 10 章

Playbook	概要	使用场景	详解
Runtime forensics	测量正在运行的进程，诊断运行时症状的原因，只给出诊断而不修复	内存泄漏、空闲时 CPU 空转等症状仍在发生，且 Agent 能测量运行中的进程时	第 10 章
Trace forensics	读取事后提供的性能分析文件或转储，无需重新运行程序，并诊断原因	只有在用户环境中取得的 cpuprofile、trace、spindump、heap snapshot 等成果物时	第 10 章
Bug fix	复现报告的缺陷，找出根本原因，并用运行时证据确认确有必要的最小改动修复	Agent 要修复用户收到的结果本身有误的缺陷时	第 11 章
Perf issue	取得实际测量所得的慢速执行轨迹，与基线比较并改善性能	行为正确但处理缓慢，用户希望通过一次修复改善性能时	第 11 章
Hillclimb	反复执行「一次改动、一次测量、保留或撤销」，使一项指标改善到目标值	用户希望反复改善一项指标，直到达到目标，而非只修复一次时	第 11 章
Feature	先确定所处理数据的形态，再实现新的行为或修改现有行为	用户请求添加功能或修改现有行为时	第 12 章
Refactoring	保持现有行为，通过重命名、提取、去重、移动等方式改变代码结构	用户只想改变代码结构而不改变行为时	第 12 章
Prototype	制作可丢弃的原型，根据观察结果作出设计决定	希望通过实际制作和观察，以较低成本决定布局或行为等设计问题，而不询问他人时	第 12 章
Visual parity	以基线外观为规格，逐像素调整 UI，直到图像差异为零	需要让两种实现的外观一致，或迁移样式系统时	第 12 章
Authoring or modifying a skill	按规定步骤编写 SKILL.md，只保留会改变工作中判断的指令	用户或 Agent 新建或修改 SKILL.md 时	第 13 章

Playbook	概要	使用场景	详解
Eval	在采用之前，采用盲测衡量 Skill、结构或提示词变更对 Agent 行为的影响	用户或 Agent 想在采用 Skill 或提示词变更前确认其效果时	第 13 章
Opening a PR	使用小而有序的提交、Conventional Commits 格式的标题及 Briefing 格式的正文，创建可立即审阅的 PR	Agent 在其他 Playbook 工作结束时将变更整理成 PR 时	第 14 章
Babysit	解决冲突、审阅讨论和 CI 失败，使 PR 或 PR 栈达到 merge-ready 状态	用户就 PR 状态提出请求，例如「让检查通过」或「关注 PR X 的情况」时	第 14 章
Shipping	让未编写代码的 Agent 验证已通过 CI 的 PR 栈中的每个 PR，再从根部开始逐个合并连续且已验证的部分	用户请求将已通过 CI 的 PR 栈并入生产环境时	第 14 章
Autonomous run	先确定完成条件，再持续推进长期任务，直到满足条件	用户要求将长期任务交给 Agent 做到完成，例如「一直运行到完成」或「用 /loop 运行到 X」时	第 15 章
Pause safely	在安全的节点暂停工作，留下让毫无背景的 Agent 也能据此恢复的记录	用户明确要求停止、用户离线、Cursor 重启，或对话即将压缩时	第 15 章
Session pickup	阅读前一位 Agent 的对话记录、Cloud Agents URL 或已推送的分支，从已完成工作的末尾恢复	用户要求恢复或接手前一位 Agent 尚未完成的工作时	第 15 章
Worktree and simulator cleanup	确认无人使用后，删除已合并或废弃的 git worktree 和旧 iOS 模拟器，释放磁盘空间	用户询问磁盘空间被什么占用，或要求清理 worktree 或模拟器时	第 15 章
Multi-phase or multi-PR plan	不编写代码，产出清单式计划文档，连每个 PR 的验证方式也事先确定	Agent 开始跨多个阶段或多个堆叠 PR 的工作之前，先制定计划时	第 16 章

Playbook	概要	使用场景	详解
Orchestrate	将持续数日、涉及大量 PR 和子 Agent 的项目管理，交给一位不编写代码的协调 Agent	用户将一个超出单个 Agent 可运行时长的项目交给一个对话来处理时	第 16 章
Autopilot-full	将相互独立的 PR 队列分别交给各自负责人；通过协调 Agent 验证后，无需操作员复核便逐个合并	操作员委派一组相互独立的 PR，并能够授予 Agent 合并权限时	第 16 章
Autopilot-stack	按与 Autopilot-full 相同的步骤创建并验证 PR，但不合并，而是作为一个 PR 栈交给操作员	操作员想在合并前审阅、变更之间存在依赖，或无法授予 Agent 合并权限时	第 16 章

第 10 章：调查并复现问题

kaito • 来源：日本語 • English • 2026-10-03

本章介绍以下三个 Playbook。

1. Investigation
2. Runtime forensics
3. Trace forensics

这三个 Playbook 都用于调查并得出答案，不修改代码。**成果物是有依据的回答或诊断，而非修复。**

应选择哪个 Playbook，取决于调查可用的信息和证据：是阅读源代码与历史就能回答，还是能测量正在运行的进程，或只能分析已取得的性能分析文件和转储（记录运行状态的文件）。

本章依次说明三者的共同规则及各自的用法。

本章结构

本章结构如下。

- 三个 Playbook 都不修改代码，依据调查可用的信息或证据来选择
- 「Investigation」对可通过阅读代码或历史回答的问题给出带引用的答案
- 「Runtime forensics」停止从源代码猜测，转而测量正在运行的进程
- 「Trace forensics」不重新运行程序，将收到的成果物转成可查询的形式后读取
- 总结

三个 Playbook 都不修改代码，依据可用信息和证据来选择

如前所述，三个 Playbook 依据调查可用的信息或证据来区分。

选择方式如下。

- 问题可通过阅读代码或历史来回答……「Investigation」
- 缺陷或延迟等症状仍在发生，且自己能够测量运行中的进程……「Runtime forensics」
- 只有事后提供的性能分析文件或转储（记录运行状态的文件）……「Trace forensics」

项目	「Investigation」	「Runtime forensics」	「Trace forensics」
证据	源代码与历史	从运行中的进程取得的测量数据	事后提供的成果物（cpuprofile、spindump、heap snapshot 等）
重新运行程序	不运行	不运行（向运行中的进程加入测量代码，以验证假设）	不运行
下一步	交还用户，再转向「Bug fix」或「Feature」	转向「Bug fix」或「Perf issue」	转向「Bug fix」或「Perf issue」

即便同样是「很慢」的反馈，能否测量正在运行的进程，或是否只有收到的文件可供分析，也会改变所选的 Playbook。不同类型的证据还决定了允许采取哪些操作。

「Runtime forensics」允许向运行中的进程加入日志或测量代码，以验证判断。而「Trace forensics」只能读取收到的性能分析文件或转储，**禁止重新运行程序重新采集**。

「Investigation」对可通过阅读代码或历史回答的问题给出带引用的答案

作用：回答可通过阅读代码或历史解决的问题

「Investigation」用于回答只需阅读源代码或变更历史即可解决的问题，例如「X 如何运作」「Y 为什么这样设计」「Z 真的没问题吗」「X 与 Y 应选哪个」。

成果物是附有引用的解释或建议。不创建 PR，也不负责跟进 PR 直至可以合并

（「Babysit」；参见第 14 章）。如果途中发现需要修改代码，就交还用户，不自行开始修复。

步骤：四步确定输出形式

1. 将问题交给 /how（说明代码机制或所在位置的 Skill）。若问题问「为什么」，也交给 /why，附引用调查设计理由和来历（两者见第 22 章）
2. 将吞吐量检查点写成一行「不适用」
3. 按 /how 的五个相同标题（概要、主要概念、运作方式、位置、注意事项）写作。比较选项时，给出附有取舍表的建议
4. 对回答运行 /unstop（去除 AI 常写的套话的 Skill；见第 32 章）

- **TODO 列表**.....Agent 在工作开始时创建的待办事项清单（计划）。Cursor 会在聊天界面显示它。用户可据此确认 Agent 完成了哪些步骤、跳过了哪些步骤（skip: <理由>；参见随附指南 01-setup.md）。
- **吞吐量检查点**.....这是将「工作可以怎样并行划分」的思考结果记录到 TODO 列表时使用的四项内容，如下所示。
 - 需要先完成的步骤
 - 独立的工作流
 - 共享的可变状态
 - 最小且安全的划分方式

对于有多个步骤且并非显而易见的工作，每个 Playbook 都要求写出这四项（详见第 12 章）。

「Investigation」只调查、不修改代码，没有需要分工的工作，因此无需写四项，只写一行「不适用」。

如果被问到「真的没问题吗」，应先作出自己的判断并说明理由，而非直接附和对方的看法；若前提有误，也应指出。

请求写法：写明「新任务」「暂时不要修改代码」

在请求中写明「新任务」和「暂时不要修改代码」，便会重新选择 Playbook，并施加不修改代码的约束。随附指南（docs/guide/02-poteto-mode.md）给出的示例如下。

```
/poteto-mode new task. figure out why the cache entry survives logout. don't change any code yet.
// 新任务。查明为什么退出登录后缓存条目仍然存在。暂时不要修改代码。
```

也可以在请求中指定回答形式。「The Complete Guide to pstack」的 [Part 2](#) 给出了以下示例。

```
/poteto-mode investigate why background workers periodically fail with timeout errors. give me a
breakdown of what we know, what data you used, and your best hypotheses.
// 调查后台 Worker 有时因超时错误而失败的原因。分别列出已知事实、使用的数据和最有力的假设。
```

这种请求方式与 /poteto-mode 中「[Writing the reply](#)」一节的要求相同：**每项主张都要在同一句中附上证据，或标明它是测量所得、推断所得还是猜测**。在请求中也要求作这种区分，便能辨认回答中的哪些句子是经确认的事实，哪些只是猜测。

「Runtime forensics」停止从源代码猜测，转而测量正在运行的进程

作用：通过测量正在运行的进程来诊断运行时症状

「Runtime forensics」用于诊断内存泄漏、空闲时 CPU 空转、显示异常等运行时症状；它测量正在运行的进程，而不是从源代码猜测。

阅读源代码可以找到可能导致问题的路径。但对于运行时症状，关键在于实际执行了哪条路径。需要测量正在运行的进程才能确认。

步骤：采集信号、缩小证据范围、证明机制、映射到源代码

1. 采集实时信号

- 使用符合症状的 control 系 Skill，采集 CPU 性能分析文件、堆快照或 CDP（Chrome DevTools Protocol）轨迹
- control 系 Skill 不随 pstack 提供；此处指 [cursor-team-kit](#) 中的 control-ui 和 control-cli

2. 缩小范围，找到决定性证据

- 查找热路径（占用大部分执行时间的路径）上的函数、泄漏对象的引用链，或无输入却持续运行的循环
- 让子 Agent 分析大型成果物（参见 Principle 「**Guard the Context Window**」，第 20 章）

3. 相信结论前先证明机制

- 验证假设所指出的、引发症状的机制
- 可以向运行中的进程插入测量代码，或不重启进程而直接修改正在运行的代码（热修复）来验证

4. 映射到源代码

- 将原因追溯到源代码中的文件、符号（如函数名）和具体行

除非受到请求，否则不作修复；查明原因后交给「**Bug fix**」或「**Perf issue**」。

- **CPU 性能分析文件**……记录一段时间内哪些处理占用了 CPU
- **CDP 轨迹**……使用 Chrome DevTools Protocol 记录的浏览器内部处理日志，可分析 JavaScript 执行、页面布局与绘制等
- **堆快照**……记录某一时刻应用保存在内存中的数据及其引用关系，可用于调查内存泄漏等问题

「Trace forensics」不重新运行程序，将收到的成果物转成可查询的形式后读取

作用：专注读取已经取得的成果物

「Trace forensics」用于读取用户环境中采集、事后交付的 CPU 性能分析文件、spindump、堆快照等成果物，**不重新运行程序，而是读取这些材料来诊断原因。**

- **spindump**……用于分析无响应应用处理状态的 macOS 诊断报告

与「Runtime forensics」的区别在于，采集已经完成。成果物是固定数据，因此只需专注读取。为使 Playbook 可在任何环境中使用，工具也限定为 DevTools 或 trace 解析器等通用工具。

步骤：加载成果物、整理形式、缩小原因范围、定位源代码

1. 识别格式并加载

- 让子 Agent 分析大型成果物

2. 转成可查询的形式

- 将 trace 或堆快照导入 sqlite，在阅读前转成可查询的形式

3. 缩小原因范围

- 查找耗时最多的帧、泄漏的引用链，以及停滞的线程

4. 定位源代码

- 依据成果物中的符号信息，确定最耗时的帧对应源代码中的哪个文件、哪个函数、哪一行
- 在位置尚不明确时，无法断言是哪段代码导致问题，因此诊断尚未完成
- 若无法定位，就解析符号信息，或明确指出成果物未包含符号信息

5. 如果有成对的采集结果，就相互对照

- 若有修复前后这样可比较的两份成果物，就计算差异
- 若没有比较数据，应将发现写成「成果物支持的最有力假设」，不要称为已确定的原因

6. 给出附有引用的诊断

- 除非受到请求，否则不作修复，而是交给「**Bug fix**」或「**Perf issue**」

- **帧**……处理调用历史中记录的函数位置
- **引用链**……导致数据无法从内存释放而一直保留的引用关系链
- **线程**……程序中推进处理的一条执行流

总结

- **共同点与选择方式**……三个 Playbook 都不修改代码，只给出答案或诊断。按问题能否通过阅读代码和历史回答、能否测量仍在运行的进程、或是否只有已取得的成果物来选择。
- **Investigation**……通过 /how 和 /why 回答，使用五个标题或附有比较表的建议。若需要修改代码，就交还用户。
- **Runtime forensics**……停止从源代码猜测，测量正在运行的进程，证明机制后再指出源代码位置。
- **Trace forensics**……不重新运行程序，将收到的成果物转为可查询形式，定位到引发问题的源代码位置。若无可比较的成果物，则写成「最有力假设」。

下一章第 11 章将从「要修复什么」的角度，介绍接收诊断并实际进行修复的三个 Playbook：「**Bug fix**」、「**Perf issue**」和「**Hillclimb**」。

第 11 章：修复缺陷，改善性能

kaito • 来源：日本語 • English • 2026-10-03

本章介绍以下三个 Playbook。

1. Bug fix
2. Perf issue
3. Hillclimb

三者都是用于修改代码的 Playbook。修改时，要以实际运行程序得到的输出、日志和测量结果（运行时证据）为依据。

选择哪个 Playbook，取决于想修复什么：收到的结果本身有误、行为正确但处理缓慢，还是希望反复提高一项数值直到达到目标。

本章依次说明三者的共同规则及各自的用法。

本章结构

本章结构如下。

- 三个 Playbook 都依据运行时证据进行修复，并根据想修复的问题来选择
- 「Bug fix」先复现缺陷，再以证据证明有必要的最小改动来修复
- 「Perf issue」从修复前的测量开始，通过修复后的测量证明改善
- 「Hillclimb」反复执行「一次改动、一次测量、保留或撤销」，改善一项指标
- 总结

三个 Playbook 都依据运行时证据修复，并按要解决的问题来选择

三个 Playbook 共同遵守不保留缺少运行时证据支持的改动这一规则，选择方式如下。

- 用户收到的结果本身有误……「Bug fix」
- 行为正确但处理缓慢，希望通过一次修复改善性能……「Perf issue」
- 希望反复提高一项数值直到达到目标……「Hillclimb」

项目	「Bug fix」	「Perf issue」	「Hillclimb」
起点	自己复现缺陷	取得基线（修复前的标准）轨迹	了解影响结果的条件和目标机制，确定指标与停止条件
次数	一次修复	一次修复	循环直到满足停止条件
回答重点	复现结果从失败变为成功的输出	基线、修复后数值与差异	指标的初始值和最终值，以及保留与撤销的改动数量

若只需查明原因、不需要修复，应选择第 10 章的「Runtime forensics」或「Trace forensics」。

三者都把实现工作交给子 Agent，主 Agent 负责规划修复和审阅差异。

「Bug fix」先复现缺陷，再以证据证明有必要的最小改动来修复

作用：复现缺陷并找出根本原因

「Bug fix」用于复现报告的缺陷并找出根本原因，只做经运行时证据证明有必要的最小改动。

「也许有效」的防备性措施仍是未经验证的假设，并非修复，因此不应加入改动。若运行时证据否定了假设，就撤销基于该假设所做的改动。

在演讲中提到，如果留下临时绕过问题的措施，后续 Agent 会把它当作正确实现来照搬（见第 4 章）。本书认为，撤销基于被否定假设的改动这一规则，能够防止这类绕过措施留在代码中，供后续 Agent 模仿。

步骤：复现、二分查找、计划、验证、提交和 PR，共六步

1. 自己复现

- 使用适合症状出现位置的 control 系 Skill (`cursor-team-kit` 中的 `control-ui` 或 `control-cli`)
- 只有当该 Skill 因具体原因无法操作，且自己已推进到能够操作的范围边界后，才能请求用户复现
- 若无法立即复现，就人为制造触发条件、缩小到更容易出现缺陷的条件，或加入测量代码，持续尝试直到缺陷出现

2. 用二分查找定位原因

- 列出原因假设，优先尝试能最大程度减少剩余候选的验证方法，直到缩小到一个原因
- 让 `/how` 解释受影响子系统的机制，让 `/why` 从历史查明原本正常的功能如何损坏，以收集假设候选（两者见第 22 章）
- 若不清楚程序状态，就增加测量或日志，不要猜测
- 进入下一步前，先用运行时证据验证最后一个假设所指出的机制

3. 规划修复

- 若修复跨越函数边界，先使用 `/architect`（在编码前设计类型与模块结构的 Skill，见第 24 章）
- 明确限定范围后，将实现委派给子 Agent

4. 在症状出现的同一界面或命令中验证

- 确认原来的复现步骤这次能够成功
- 如果改在其他界面或命令中检查，或检查后仍无法得出结论，都不能算通过
- 单元测试只能说明单个处理的行为，不能证明缺陷已消失。因此，单元测试通过本身不足以判定修复完成

5. 先于修复提交能够复现缺陷的测试

- 先提交修复前会失败的测试，再叠加修复
- 若能编写便于本地运行的测试，就遵循 `/tdd`（见第 30 章），先写失败测试再继续
- 若编写测试费时、测试严重依赖多服务集成环境，或测试目标不明确，则可以省略这一步

6. 执行规定创建 PR 步骤的「Opening a PR」（见第 14 章）

回答应包含以下项目。

- 哪里出了问题
- 根本原因
- 修复
- 验证方法

同时，**不要概述，而要原样粘贴**复现输出从失败变为成功的记录。

本书对步骤与 Principle 的对应关系作如下解读：Principle 「**Fix Root Causes**」对应步骤 1 至 3；「**Prove It Works**」对应步骤 4 与回答；「**Sequence Work into Verifiable Units**」对应步骤 5（见第 19 章）。其中，Playbook 原文只在步骤 5 明确提及「Sequence Work into Verifiable Units」的名称。

请求写法：写明「先复现」

在请求中写明「先复现」；若要求测试，应加上「如果能轻松写出测试」这一条件。随附指南（docs/guide/10-recipes-and-pitfalls.md）中的示例如下。

```
/poteto-mode repro the duplicate write first. if there's a cheap test path, /tdd it. then fix and rerun.
// 先复现重复写入。如果能方便地编写测试，就使用 /tdd。然后修复并重新运行。
```

与其勉强使用脆弱的 mock（替代真实组件的假组件）来编写测试，**运行真实命令更能准确证明问题已修复**。步骤 5 允许省略编写费时的测试，原因就在这里。

「Perf issue」从修复前测量开始，用修复后测量证明改善

作用：测量并修复缓慢问题

「Perf issue」通过采集实际测得的慢速执行轨迹（记录各处理的执行时间等），并与基线比较，来改善性能。它禁止用阅读源代码替代测量，**每项修复都必须关联测量结果**。

步骤：从基线轨迹开始的六步

1. **取得基线轨迹**……使用符合症状的 control 系 Skill
2. **用 /how 了解代码机制，作为假设依据**……在实际运行和测量前，不要断言「已经不可能更快」。从八类常见改善策略（策略类别；见下表）中提出假设
3. **根据轨迹规划修复**……将实现委派给子 Agent，审阅差异，再取得修复后的轨迹
4. **分析并比较修复前后的轨迹**……可将轨迹导入 sqlite 等，以计算差异。若使用不同界面或命令测量，或测量后仍无法得出结论，都不能算通过
5. **在 PR 中引用测量结果**
6. **执行「Opening a PR」**

回答应包含以下四项。

- 基线数值
- 修复后的数值
- 差异
- 成果物路径

这些项目都必须实际测量后才能填写，因此**回答格式本身防止了跳过测量**。

八类策略不是检查清单，而是产生假设的工具。**只有轨迹中出现某种策略可能有效的迹象（表格右列）时，才能尝试该策略**。Elimination（删除）是例外：轨迹能显示哪里慢，却不能证明能否删除，所以需要 `/how` 确认。

策略类别	轨迹中可见的迹象
Elimination（删除）	轨迹中看不到；能否删除要用 <code>/how</code> 确认
Divide and conquer（分而治之）	主要成本随输入规模成比例增长
Caching（缓存）	对相同输入反复进行相同计算或获取
Indirection（间接化）	热路径（占用大部分执行时间的路径）中执行全量扫描等重操作，可由索引或队列等中间机制承担
Batching（批处理）	每次小操作都要付出通信或查询等固定成本
Redundancy（冗余）	总体等待时间受一台缓慢服务器或一次尝试拖累
Lazy evaluation（延迟求值）	尚不需要的结果已产生成本

策略类别	轨迹中可见的迹象
Scheduling（调度）	工作本身必要，但无须在用户等待的时刻执行

请求写法：写出实测数值，而非「感觉很慢」

请求中应写入测得的数值，而不是「感觉很慢」。随附指南（[docs/guide/05-build-and-clean.md](#)）中的示例如下。

```
/poteto-mode startup takes 1.8s on this fixture. trace it, fix the measured cause, show me before and after.
// 使用这个固定的测试输入，启动需要 1.8 秒。采集追踪，修复测得的原因，并展示修改前后的结果。
```

若还要确认改善确实可靠，可在请求中要求通过 `/swarm` 并行运行验证 Skill（像用户一样操作应用并检查结果的 Skill，见第 3 章）。「The Complete Guide to pstack」Part 1 的示例如下。

```
spawn a cloud agent to use /poteto-mode to improve the initial loading time of our app. first use /control-app to take a trace of the status quo, and identify opportunities for improvement. then do a targeted fix and use /control-app + a /swarm to confirm the win
// 启动云端 Agent，让它通过 /poteto-mode 改善应用首次加载时间。先用 /control-app 采集现状追踪并寻找改进空间，再有针对性地修复，最后用 /control-app 和 /swarm 验证改进。
```

`/control-app` 是文章中创建的验证 Skill 的名称（见第 36 章）。`/swarm`（见第 24 章）是并行运行 Workers（多个 Agent）并汇总为一份报告的 Skill。让多个 Agent 运行验证 Skill，便能用足够的样本量确认改善，避免把单次测量的波动误判为改善。

「Hillclimb」反复执行「一次改动、一次测量、保留或撤销」，改善一项指标

作用：循环改善一项指标直到达到目标

「Hillclimb」用于朝目标通过循环持续改善一项指标。一次性的修复则由「Bug fix」或「Perf issue」负责。

核心纪律是「one change, one measurement, keep or revert」，即**一次改动、一次测量、保留或撤销**。不应堆叠未经测试的改动，也不能仅凭阅读代码就声称指标改善（参见 Principle 「**Prove It Works**」）。

步骤：每次只针对一个假设循环，共八步

1. 选择指标前，先了解影响结果的条件和目标机制

- 对目标使用 /how，列出影响指标数值的现实条件（数据量、历史、状态、并发度）
- 选择能复现用户不满的案例
- 确定一项指标和停止条件
- 停止条件须同时规定目标和最少尝试次数，例如「比基线改善 50% 以上，并且至少尝试 10 次」。设定次数下限，是为了避免早期碰巧取得一次好结果就结束

2. 建立测量装置，确认它能区分案例后固定下来

- 确认目标案例能复现症状，并能通过测量值与没有症状的简单案例区分
- 确认后固定测量方法；即在后续循环中不再改变测量案例、指标和测量命令
- 固定后，用一条确定的命令输出指标，取多次测量的中位数，而非只测一次
- 加入改动前，记录基线指标及所有回归关卡（必须持续通过的测试）的通过结果（参见 Principle 「**Build the Lever**」，第 17 章）

3. 建立决策记录

- 用记录决策的 /show-me-your-work（见第 28 章）创建 decision.tsv
- 每次尝试都在 decision.tsv 中写一行，记录假设、改动、前后数值，以及保留或撤销的决定
- 下一次尝试前重读记录

4. 依据步骤 1 中了解的机制提出假设

- 提出关于具体机制的假设，例如「X 拖慢首次绘制，因此把它移出启动流程」
- 不接受「试着对某些东西做记忆化」这类模糊假设

5. 每次只针对一个假设循环

- 将改动交给子 Agent（参见 Principle 「**Guard the Context Window**」）

- 若有多个独立假设，将它们交给各自 `worktree` 中并行工作的子 Agent（参见 Principle「**Separate Before Serializing Shared State**」）
- 用步骤 2 固定的测量装置，分别测量改动前后
- 只有指标改善明显到无法归因于偶然波动，且所有回归关卡通过时，才保留改动
- 否则完整撤销改动
- 无论保留还是撤销，都必须写入 `decision.tsv` 的一行记录
- 若要无人值守地运行，只从负责持续推进长期任务直到完成的「**Autonomous run**」（见第 15 章）借用按时间或事件重新唤醒 Agent 的机制。停止条件仍按步骤 1 所定执行

6. 改善停滞时也不要立即结束

- 若连续撤销改动，先尝试切换假设类型（处理方式的种类）、组合接近成功的方案，或尝试更大胆的方案，再下结论
- 即使指标改善，若改动破坏行为，也要撤销
- 若在维持指标的同时简化代码，则保留改动（参见 Principle「**Laziness Protocol**」）

7. 满足停止条件，或剩余方案的预期效果不再值得投入时停止

- 不得放宽停止条件来宣称达标
- 只要还有低成本可尝试的假设，就不要停止
- 遇到瓶颈时应报告，而不是空转

8. 执行「Opening a PR」

- 按加入顺序叠放已保留改动的提交

「改善明显到无法归因于偶然波动」的含义

例如，测量处理时间时，即使不改代码，结果也可能在 100 毫秒上下波动几毫秒。改动后若降至约 80 毫秒，就可判断改善超出了波动范围。若只变化几毫秒，则可能只是测量波动，尚不能断言有所改善。

步骤 2 取多次测量的中位数而非只测一次，也是为了减小这种波动的影响。

- **测量装置**……确保每次在相同条件下运行测量或测试的机制

- **记忆化**……按输入保存函数的计算结果，并在再次收到相同输入时返回保存结果的方法
- **worktree**……Git 的一项功能，可从一个仓库创建多个工作目录，同时处理不同分支

请求写法：提供指标、目标和最低尝试次数三项信息

根据随附指南（[docs/guide/05-build-and-clean.md](#)），只要提供指标、目标和最低尝试次数，「**Hillclimb**」就会使用固定测量方法的装置，反复进行每次只检验一个假设的流程。

总结

- **共同纪律与选择方式**……三个 Playbook 都要求每项保留的改动有运行时证据支持。行为有误时用「**Bug fix**」；希望一次修复处理慢速问题时用「**Perf issue**」；希望反复改善一项指标直到达标时用「**Hillclimb**」。
- **Bug fix**……先自行复现，通过二分查找将原因假设缩小到一个再修复。在回答中原样粘贴从失败变为成功的输出。
- **Perf issue**……从基线轨迹开始，只有当轨迹显示某一改善策略可能有效时才尝试该策略，并用修复前后的数值证明改善。
- **Hillclimb**……建立固定测量方法的装置，在目标与最低尝试次数共同构成的停止条件下，反复执行「一次改动、一次测量、保留或撤销」。

下一章第 12 章将从「哪些方面不得改变」的角度，介绍用于「添加」「保持」「试验」行为和「统一」外观的四个 Playbook：「**Feature**」、「**Refactoring**」、「**Prototype**」和「**Visual parity**」。

第 12 章：重新设计功能、结构与外观

kaito • 来源：日本語 • English • 2026-10-03

本章介绍以下四个 Playbook。

1. Feature
2. Refactoring
3. Prototype
4. Visual parity

四者都会修改代码，区别在于如何处理行为。行为是用户实际收到的结果，例如命令输出或屏幕显示。

它们的处理方式分别如下。

- 添加或改变行为
- 保持行为，只调整结构
- 试验行为以作出设计决定，然后丢弃代码
- 逐像素匹配外观

无论使用哪个 Playbook，**请求中都要写明什么不能改变**。随附指南的实现章节（[docs/guide/05-build-and-clean.md](#)）也建议在 Feature 请求中写明所需行为及不能改变的内容。

本章依次说明四者的区别、各自的用法以及请求写法。

本章结构

本章结构如下。

- 根据是要添加、保持、试验行为，还是统一外观，选择四个 Playbook 之一
- 「Feature」先确定数据形态，再委派实现并审阅
- 「Refactoring」先固定现有行为，再调整结构
- 「Prototype」用可丢弃的代码低成本地作出设计决定

- 「Visual parity」以基线外观为规格，争取让图像差异为零
- 在请求中写明「什么不能改变」
- 总结

根据是添加、保持、试验行为，还是统一外观来选择四个 Playbook

四个 Playbook 按**如何处理行为**区分。随之变化的还有哪些内容不能改变、对什么负责，以及如何验证。

项目	「Feature」	「Refactoring」	「Prototype」	「Visual parity」
行为	添加或改变	保持不变	试验（不会进入生产环境）	外观保持不变
负责的内容	设计	契约（现有行为）	设计决策（并非代码）	逐像素一致
验证	在改动实际呈现的位置运行并检查	用真实成果物证明行为与修改前相同	通过观察比较各方案	图像差异为零

四者之间也有交接。「Prototype」将比较后选定的设计方案交给「Feature」。

「Refactoring」把途中发现的缺失功能或真实缺陷拆成单独任务，先提交结构改动。如果连设计也要重做，就明确说明这是重做，并转交「Feature」。

「Feature」先确定数据形态，再委派实现并审阅

「Feature」先确定新功能所处理数据的形态（数据字段与类型），再据此实现功能。**主 Agent 对设计负责**，将编写代码交给另一位 Agent。

关键有两点。

- **写代码前先确定数据形态**……Principle 「**Model the Domain**」（见第 18 章）说明：写代码时选择结构成本低；若事后补救，就容易被视为重构而一再推迟。
- **将写代码的人与审阅差异的人分开**……把编码工作交给子 Agent，由主 Agent 审阅差异（步骤 4）。

步骤：确定设计、委派实现、验证，再创建 PR

共有八步。

1. 对受影响的子系统使用 /how（解释代码机制的 Skill，见第 22 章）
2. 用 /architect（先于代码绘制设计的 Skill，见第 24 章）并行探索类型和模块结构方案
3. 写下吞吐量检查点（见下表）
4. 将编写代码的工作交给子 Agent
5. 在改动呈现的环境中验证（界面用浏览器，命令用 CLI）
6. 将提交 rebase 成小而有序的提交（参见 Principle「[Sequence Work into Verifiable Units](#)」，第 19 章）
7. 若对设计有异议，先运行 /interrogate（见第 27 章），让多个模型的审阅者质询改动，再继续
8. 执行规定创建 PR 步骤的「[Opening a PR](#)」（见第 14 章）

rebase……一种 Git 操作，将提交重新叠放在另一基底上；也可用于调整提交顺序或将多个提交合为一个

步骤 3 的吞吐量检查点，是考虑「如何并行划分工作」后写入 TODO 列表的四项内容。

TODO 列表……Agent 在工作开始时创建的待办事项清单（计划）。Cursor 会在聊天界面显示它。用户可据此确认 Agent 完成了哪些步骤、跳过了哪些步骤（skip: <理由>）。（随附指南 01-setup.md）。

对于有多个步骤且并非显而易见的工作，「**Feature**」之外的 Playbook 也要求写下这一检查点。

项目	填写内容
需要先完成的步骤	并行执行前必须通过的关卡
独立的工作流	互不冲突的工作（文件或层面不重叠）可以并行执行
共享的可变状态	默认先划分修改对象，避免同时改写同一对象，以消除共享（参见 Principle「 Separate Before Serializing Shared State 」）

项目	填写内容
最小且安全的划分方式	写下可安全划分的工作单元。若由一人完成更合适，也要写明理由

四项中即使有不适用于本次工作的，也不要删除，而要写 n/a: <理由> 并说明原因。这样便能区分是经过考虑后认为无需填写，还是漏看了该项。

在步骤 4 中，要指定文件路径、数据形态和成功标准，将实现委派给为 Feature 配置模型的子 Agent（可通过第 6 章提到的 /setup-pstack 更改）。如果实现存在多种合理方案，则通过 /arena（见第 24 章）让多个候选并行解决同一问题，再组合各自的优点。

这项委派是必须的，不能用 skip: <理由> 跳过，也不能以要求「最小改动」的 Principle 「**Laziness Protocol**」推翻。目的在于分开实现与审阅，而非节省代码行数。

如果负责实现的子 Agent 无法再启动其他子 Agent，就由该实现 Agent 自己编写代码。发出委派的 Agent 负责审阅差异，因此实现与审阅的职责仍然分开。

请求写法：写明所需行为及不能改变的内容

随附指南中的示例如下。

```
/poteto-mode add a --json flag. text output stays byte-identical. verify both forms.
// 添加 --json 标志。文本输出保持逐字节一致。验证两种输出形式。
```

「Refactoring」先固定现有行为，再调整结构

「Refactoring」通过重命名、提取、去重、移动等方式，保持行为，只改变结构。

主 Agent 负责的是「契约」，即现有行为。可以改变结构，但不能改变现有行为。

也允许连设计一起重做；但在这种情况下，必须明确说明是重做，并转交「Feature」。

步骤：固定行为、删除多余代码、调整结构，再验证等价性

共有八步。

- 1. 固定行为契约

2. 为缺失的结构命名
3. 为目标结构命名
4. 先删后加
5. 以小步调整
6. 用真实成果物证明行为未改变
7. 确认改动值得保留
8. rebase 成小而有序的提交

下面逐项说明这些步骤。

- **快照测试**……先将输出保存为文件，再与后续运行结果比较，以检测变化的测试

1. 固定行为契约

调整结构前，先编写特性测试（将现有输出原样记录为正确答案的测试）、快照测试，或比较新旧输出的机制，以保证现有行为。仅通过类型检查和 lint，并不意味着行为已被固定。

2. 为缺失的结构命名

这一步对应 Principle 「**Model the Domain**」（见第 18 章）。

3. 为目标结构命名

确定模块划分、数据类型及函数之间的调用关系。若目标结构跨越函数边界，就用 `/architect` 探索。

4. 先删后加

先删除无用代码：例如死代码（未使用的代码），或只有一处调用的包装函数（Wrapper），可将其处理合并到调用处。然后再引入新结构（参见 Principle 「**Subtract Before You Add**」）。

5. 以小步调整

逐项改动，每一步都要让步骤 1 固定的测试或比较继续通过。如果改变 API 形式，就将所有调用方迁至新 API，并在同一系列改动中删除旧 API（参见 Principle「**Migrate Callers Then Delete Legacy APIs**」，第 18 章）。

6. 用真实成果物证明行为未改变

编译或 build 通过本身不足以证明行为未改变（参见 Principle「**Prove It Works**」）。

7. 确认改动值得保留

若差异并未减轻读者负担，就撤销改动（参见 Principle「**Minimize Reader Load**」，第 17 章）。

8. rebase 成小而有序的提交

按删除代码的减法实现、重建结构、后续清理的顺序提交，再执行「**Opening a PR**」。

请求写法：调整结构前先记录现有输出

随附指南中的示例如下。

```
/poteto-mode move parsing into one module, zero behavior change. record the current output first and
prove it's unchanged after.
// 把解析逻辑移到一个模块，行为变化必须为零。先记录当前输出，再证明修改后没有变化。
```

「先记录现有输出」对应步骤 1，「之后证明输出未改变」对应步骤 6。

「Prototype」用可丢弃的代码低成本地作出设计决定

「Prototype」制作准备丢弃的原型，依据观察结果作出设计决定。

主 Agent 负责设计决策，而非代码。生产环境中的实现由「**Feature**」完成。

因为原型本来就会丢弃，它被认为是唯一一个**反转** Principle「**Laziness Protocol**」的「最小改动」要求与通常验证标准的 Playbook。

优先求快而非打磨，不要求代码质量，也会提出未经请求的方案。验证标准由「证明改动正确」变为「观察想作决定的问题」。

步骤：明确要决定什么，再制作并比较原型

共有六步。

1. 明确原型要用来决定什么
2. 若方向尚未确定，收集参考案例
3. 在远离生产源代码的工作目录中制作
4. 让替代方案可以在同一界面切换比较
5. 实际运行并比较各方案
6. 展示替代方案、取舍与建议作为比较结果

下面逐项说明这些步骤。

1. 明确原型要用来决定什么

若涉及外观和操作，需要决定的问题可能是「采用哪种布局」「采用哪种操作方式」「界面元素排得多紧凑」；若能通过运行和观察决定，也可能是「采用哪种行为」「安排在什么时机」「采用哪种方法」。若没有需要决定的问题，就不要制作原型，应转交「Feature」。

2. 若方向尚未确定，收集参考案例

向用户展示相似功能或界面的案例，在开始制作原型前请其选择外观或操作方式。若已确定，则可省略。

3. 在远离生产源代码的工作目录中制作

不要引入生产用框架、测试和抽象层，只准备足以验证问题的最少代码。

4. 让替代方案可以在同一界面切换比较

为各方案添加标签，用按钮或按键切换（参见 Principle 「[Exhaust the Design Space](#)」，第 17 章）。

5. 实际运行并比较各方案

界面外观记录截图，处理时间记录测量值，收集能够回答步骤 1 所确定问题的材料。

6. 展示替代方案、取舍与建议作为比较结果

说明各方案的区别及采用后的优缺点，给出推荐方案。决定采用哪个方案后，将其交给「Feature」。

请求写法：写明「想比较，请做两个方案」

pstack 的 README 中的示例如下。

```
/poteto-mode build two prototypes of the markdown renderer so we can compare. spawn an agent for each.
// 制作两个 Markdown 渲染器原型以供比较，各安排一个 Agent。
```

「做一个 mock」「试试这种布局」之类的措辞也是**选择这个 Playbook** 的信号。

「Visual parity」以基线外观为规格，争取让图像差异为零

「Visual parity」用于让两种实现保持一致，或迁移样式机制时，**逐像素保持 UI 外观相同**。

不修改基线，用图像差异来验证是否一致。

通过与否取决于差异是否为零，而非目测。不过，如果基线本身看起来有问题，Agent 不能自行修正基线，而要暂停并询问用户（步骤 2）。

步骤：逐个组件循环，直到图像差异为零

共有五步。

1. 迁移前建立比较基线
2. 禁止为使差异检查通过而走捷径
3. 逐个迁移组件
4. 将每个组件与基线做图像差异比较
5. 按组件或安全的组合执行「Opening a PR」

下面逐项说明这些步骤。

- **worktree**.....Git 的一项功能，可从一个仓库创建多个工作目录，同时处理不同分支

1. 迁移前建立比较基线

建立自动为迁移前组件的多种状态（正常、悬停、禁用等）截图的机制。这些截图将作为迁移后外观的比较基准。

2. 禁止为使差异检查通过而走捷径

不更改测量装置（步骤 1 中负责截图与比较的机制）、不篡改基线，也不为了让差异检查通过而重组组件结构。若基线看起来有问题，就暂停并询问用户。

3. 逐个迁移组件

逐个将组件替换为新实现或新样式方式。可划分 worktree 并行推进，先迁移共享的基础组件。

4. 将每个组件与基线做图像差异比较

通过 /loop（反复唤醒 Agent 的 Cursor 内置命令，见第 15 章）循环，直到比较差异为零。

5. 按组件或安全的组合执行「Opening a PR」

请求写法：以「第二张图是正确的」指出基线

README 中的示例如下。

```
/poteto-mode the row spacing is too tall when this flag is on. the second image is correct. repro
and fix until it matches.
```

```
// 启用这个标志时，行距太大。第二张图才是正确效果。先复现，再修到一致。
```


「第二张图是正确的」一句话说明**应以哪种外观为基线**。没有这句话，Agent 只能猜测该对齐哪一张图。

在请求中写明「什么不能改变」

无论交给四者中的哪一个，**请求中都要写明「什么不能改变」**。

希望通过请求传达的内容	请求措辞示例	分派对象
需要新行为，但不改变现有输出	「文本输出在字节级保持不变」	「Feature」
只想整理结构	「行为变化为零。先记录现有输出」	「Refactoring」
想决定哪种设计更好	「做两个原型，以便比较」	「Prototype」
想让外观完全一致	「第二张图是正确的。修改到两者一致」	「Visual parity」

请求中的那句话会成为判断工作是否完成的标准。例如，若写明「文本输出在字节级保持不变」，文本输出与修改前逐字节相同就成为完成条件之一。

总结

- **四者的区别**……「Feature」对设计负责，「Refactoring」对契约负责，「Prototype」对设计决策负责，「Visual parity」对逐像素一致负责。
- **Feature**……确定数据形态并写下吞吐量检查点，必须将实现工作交给另一位 Agent。
- **Refactoring**……用测试等方式固定现有行为，先删除多余代码再调整结构，并用真实成果物证明行为未改变。
- **Prototype**……「最小改动」和验证标准发生反转。低成本制作方案并通过观察比较，将选定的方向交给「Feature」。
- **Visual parity**……不修改基线，逐个组件循环，直到图像差异为零。
- **请求写法**……写明「什么不能改变」，这便成为完成检查的一部分。

下一章第 13 章介绍用于编写 Skill 的「**Authoring or modifying a skill**」，以及验证变更效果的「**Eval**」。

第 13 章：编写与评估 Skill

kaito • 来源：日本語 • English • 2026-10-03

本章介绍以下两个 Playbook。

1. Authoring or modifying a skill
2. Eval

两者关注的都不是应用代码，而是**决定 Agent 工作方式的 Skill**。

编写或修改 Skill (SKILL.md) 时使用「Authoring or modifying a skill」；衡量 Skill 改动是否有效时使用「Eval」。

本章依次说明两者的分工及各自的用法。

本章结构

本章结构如下。

- 两个 Playbook 分别负责编写 Skill 的步骤和衡量效果的步骤
- 「Authoring or modifying a skill」按规定步骤编写 Skill，只保留能改变工作中判断的指令
- 「Eval」不让候选 Agent 察觉自己正在接受评估，衡量改动对其行为的影响
- 总结

两个 Playbook 分别负责编写 Skill 与衡量效果

随附指南的「Make it yours」章节 (<docs/guide/09-make-it-yours.md>) 对 Skill 提出两项要求。

- 面向 Agent 的文字应比面向人的文字采用更高标准……即使一句没有帮助的话，将来也可能成为某位 Agent 遵循的指令。
- 把 Skill 编辑当作实验来测试……因为 Skill 改动会影响未来的所有会话。

负责编写的是「Authoring or modifying a skill」，负责衡量效果的是「Eval」。

Authoring or modifying a skill 的步骤没有要求使用多个 Agent。
而 Eval 工作需要以下三项。

- **候选 Agent**……在加入改动的环境中工作的 Agent。
- **评审 Agent**……为候选的输出评分的另一位 Agent。
- **去除线索的工作环境**……清除会让候选察觉自己正在接受评估的线索（如目录名或文件名）的工作环境。

「Authoring or modifying a skill」按规定步骤编写 Skill，只保留能改变工作中判断的指令

编写新的 SKILL.md 或修改现有文件时，应执行「Authoring or modifying a skill」Playbook。

随附指南把随意从头编写 SKILL.md 列为常见错误，并称经过这个 Playbook 会进行验证和审阅。

四个步骤

共有四步。

1. 使用 create-skill
2. 验证 Skill
3. 若改变 Skill 结构，就准备测试用例
4. 通过「Opening a PR」创建 PR（见第 14 章）

下面逐项说明这些步骤。

- **frontmatter**……Markdown 文件开头用 --- 包围的元数据，例如名称和说明

1. 使用 create-skill

Cursor 内置的 Skill，用于编写 SKILL.md。

2. 验证 Skill

确认 frontmatter 含有 name 和 description，引用的文件存在，指向其他 Skill 的链接能够解析。

步骤 2 只检查能够机械判定通过与否的项目。验证 Skill 改动是否改变 Agent 行为，是「Eval」的职责。

3. 若改变 Skill 结构，就准备测试用例

若改动只是语气等主观事项，可以省略。

4. 通过「Opening a PR」创建 PR（见第 14 章）

要点是「有疑问就删除」的写作纪律

这个 Playbook 的要点是步骤之后给出的写作纪律。

When in doubt, delete. Keep only prose that changes a decision.

有疑问就删掉。只保留会改变判断的文字。

编写 Skill 的 Agent 应逐句自问「这句话会改变 Agent 的哪项判断」，无法回答的句子就删除。

主要纪律如下。

- **省略理由说明**……指令应以「做什么」为主；只有缺少理由会使指令含义或适用条件不清楚时，才解释理由。
- **指向信息源，而不复制内容**……指明代码库中实际存在的信息源，例如类型、README 和配置文件（参见 Principle「**Encode Lessons in Structure**」，第 21 章）。
- **委托其他 Skill**……不抄写其他 Skill 的内容，而是给出其路径并交给它处理。
- **将重复工作做成 Skill**……若反复执行同一工作，就建议创建新 Skill。

若 Agent 在工作途中发现 Skill 损坏，不应停止工作，也不能视而不见地绕过；而应**只修复该 Skill，并创建 PR**。

因为混在功能开发 PR 中的 Skill 改动不易在审阅时发现，也无法通过「Eval」衡量效果。

验证 Skill 和个人 Skill 有专门入口

根据目的，有时应使用专门入口，而非这个 Playbook。

入口	适用情况	详见章节
/create-verification-skill	想新建用于验证应用行为的 Skill	第 26 章
/maintain-verification-skill	验证 Skill 中记录的步骤不再符合应用当前行为	第 26 章
/automate-me	想根据过去的会话记录，将自己的工作方式整理成 Skill	第 25 章
/reflect	想将一次会话学到的内容保留在现有 Skill 中	第 33 章

请求写法：用平常的话描述要创建的 Skill

用平常的话提出请求即可。

```
/poteto-mode write a skill for verifying database migrations in this repo
// 编写一个 Skill，验证这个代码库中的数据库迁移。
```

「Eval」不让候选察觉评估，衡量改动对 Agent 行为的影响

「Eval」用于在采用之前测试 Skill、结构或提示词的改动如何影响 Agent 行为。

执行「Eval」的 Agent 负责设计实验并汇总结果。这里的实验是让其他 Agent 在实际工作中使用修改后的 Skill，再为其工作表现评分。

实验中会启动多个负责解决任务的 Agent，称为**候选 Agent**。

- 为每位候选 Agent 划分工作目录……每个目录放入修改后的 Skill 和项目骨架，避免候选彼此影响。
- 为每位候选 Agent 使用不同模型……例如第一位用 Opus，第二位用 Grok。各模型有自己的偏向；若只用一个模型测试，其偏向就会混入结果。

- **给所有候选 Agent 相同请求**……否则无法区分结果差异来自改动还是请求本身。

候选工作结束后，由使用不同模型的**评审 Agent** 在不知道输出来自哪位候选的情况下评分。

也可以比较改动前后两个版本。此时由一位评审 Agent 不知道输出分别属于哪个版本，按相同标准一次性评审两者。

- **盲测**……不告知评估对象自己正被评估或所比较条件的试验方法

七个步骤

1. **确定实验框架**……写明要测试的改动及成功的行为。为评审 Agent 制定 3 至 6 项评分标准（rubric），但不向候选展示
2. **准备去除评估线索的环境**……为每位候选准备工作目录，加入要测试的改动。也要放入真实任务会有的背景，例如项目骨架和候选通常会阅读的 Skill
3. **写一条自然的提示词**……使其像用户可能提出的请求，不泄露测量目标
4. **并行启动 N 位候选 Agent**……遵循 /arena 的步骤，为每位候选使用不同模型
5. **启动一位盲测评审 Agent**……使用不同于候选的模型家族，不展示模型名称
6. **通过记录而非自述确认是否遵循指令**……候选是否依序打开 Skill 指向的文件并遵循指令，要根据对话记录中实际打开的文件和代码形态来评分
7. **设计实验的 Agent 亲自阅读所有输出**……与评审 Agent 的判定比较，将分歧视为模型偏向或评分标准含糊的信号，再汇总结论

- **模型家族**……同一开发者以同一系列发布的一组模型（如 OpenAI 的 GPT 系列、xAI 的 Grok 系列）

要点是先于步骤列出的盲测规则

根据随附指南，这个 Playbook 的设计着眼于「知道自己正在被评估的 Agent 会改变行为」这一失败模式，称为**观察者效应**。

盲测规则按信息泄漏途径分为三项。

- **候选会看到的文字**……目录名、文件名和提示词中不要出现 eval、test、judge 等词。名称应像用户会取的名称，提示词应像普通请求。
- **候选回答的内容**……不要让候选列举使用了哪些 Skill 或文件，也不要告诉它还有其他候选。
- **评审 Agent 会看到的信息**……为输出贴上无法辨别模型来源的标签，隐去模型名。比较两个版本时，由一位评审 Agent 按同一尺度一次性评审两者。

请求写法：给两个版本相同任务，并要求候选保持盲测状态

随附指南的请求示例比较改动前后的两个版本，要求给两者相同任务，并保持候选对评估不知情。

```
/poteto-mode run the eval playbook on this skill change. same task for both variants, candidates stay blind.
// 对这个 Skill 的修改运行 Eval Playbook。给两个版本安排相同任务，并保持候选版本盲测。
```

总结

- **两个 Playbook 的分工**……「**Authoring or modifying a skill**」负责创建或编辑 Skill，「**Eval**」负责衡量效果。
- **Authoring or modifying a skill**……通过 create-skill 编写，进行机械验证，再创建 PR。要点是「有疑问就删除，只保留能改变判断的文字」。损坏的 Skill 应通过只包含其修复的独立 PR 处理。
- **Eval**……在采用改动前确认其对行为的效果。候选不知道自己正在接受评估，评审 Agent 不知道模型名称。评分依据的是表明实际打开了哪些文件的对话记录，而非候选自述。

下一章第 14 章介绍创建 PR 的「**Opening a PR**」、使 PR 达到可合并状态的「**Babysit**」，以及只将已验证的部分并入主线分支 (trunk) 的「**Shipping**」。

第 14 章：创建 PR，合并已验证的变更

kaito • 来源：日本語 • English • 2026-10-03

本章介绍以下三个 Playbook。

1. Opening a PR
2. Babysit
3. Shipping

三者负责直到将改动并入 trunk（PR 合并目标所在的主线分支），各自负责以下一个阶段。

- **创建 PR**
- **使 PR 达到 merge-ready（可合并）状态**
- **只将已验证的 PR 并入 trunk**

选择哪个 Playbook，取决于 PR 当前处于哪个阶段：尚无 PR；已有 PR 但尚不能合并；或希望将可合并的 PR 并入 trunk。

本章依次说明三个 Playbook 如何衔接，以及各自的步骤和要点。

本章结构

本章结构如下。

- 三个 Playbook 按「创建 PR、使其可合并、并入 trunk」的顺序衔接
- 「Opening a PR」创建范围小、易阅读、可立即审阅的 PR
- 「Babysit」从尚未合并的最前沿 PR 开始处理问题，使其达到 merge-ready
- 「Shipping」只将通过另一位 Agent 验证的连续 PR 范围并入 trunk
- 总结

三个 Playbook 按「创建 PR、使其可合并、并入 trunk」的顺序衔接

开头所述的三个阶段依次是「创建 PR」「变绿（全部 CI 检查通过）」「并入 trunk」，每个阶段都有明确的结束位置。

阶段	Playbook	结束位置
创建	「Opening a PR」	返回 PR 的 URL
使其变绿	「Babysit」	merge-ready
并入	「Shipping」	从根部开始连续、已验证的 PR 完成合并

- **forge**……本章把管理 PR 的服务及操作它的 CLI 合称为「forge」。默认使用 GitHub CLI (gh)。
- **trunk**……PR 的合并目标所在的主线分支。许多仓库使用 main。
- **PR 栈**……由父子关系连接的一系列 PR。只有最前沿的 PR（根部）指向 trunk（结构见下一节的图）。
- **合并最前沿**……PR 栈中尚未合并、下一步应合并的 PR。
- **merge-ready**……forge 判断 CI 检查全部通过、无未解决的审阅意见且无冲突的状态。

各阶段对应的请求措辞也不同。以下示例来自随附指南的验证章节（docs/guide/06-verify-and-ship.md）。

```
/poteto-mode babysit this pr. get it green.
// 照看这个 PR，直到所有检查通过。
```

```
/poteto-mode land the stack.
// 将这组堆叠的 PR 合入 trunk。
```

仅打开 PR 不会启动「Babysit」。

请求「让检查变绿」会运行「**Babysit**」；请求「合并整个 PR 栈」会运行「**Shipping**」。

「Opening a PR」创建范围小、易阅读、可立即审阅的 PR

「Opening a PR」是规定如何把改动整理成 PR 的 Playbook。

与前面介绍的 Playbook 不同，它不按编号列步骤，而是按标题组织规则。主要规则如下。

- **Worktree**……在从 main 创建的 git worktree 中工作。
- **Commits**……打开 PR 前用 rebase 将提交整理为小而有序的序列。把每次提交当作可以单独合并的「未来 PR」。
- **PRs**……针对差异，在提交前运行 /deslop（cursor-team-kit 中用于删除 AI 常写的多余代码的 Skill），审阅前运行 /no-comments（删除不必要注释的 Skill）。标题、说明和提交正文用规定技术文档写法的 /technical-writing 编写，再运行 /unslop 删除 AI 常见套话（见第 32 章）。
- **Titles**……采用 Conventional Commits 格式，例如 fix(pstack): retarget opening-a-pr babysit trigger。
- **Forge**……从创建到合并，始终使用同一个 forge。
- **Size and stacks**……宁可创建五个范围窄的 PR，也不创建一个庞大的 PR；将小 PR 组成 PR 栈。
- **Readiness**……打开 PR 时应已可供审阅，而不是草稿。

- **git worktree**……Git 的功能，可从一个仓库额外创建处理不同分支的工作目录（例如保留 main 的工作目录，同时在另一目录打开修复分支）
- **rebase**……将提交重新接到其他提交上、重排历史的 Git 操作
- **Conventional Commits**……提交消息采用 fix:、feat: 等表示改动类型的前缀的规范

PR 栈的形态如下：子 PR 以父 PR 的分支为 base。

```
main ← trunk
└─ PR #1 (base: main) ← 只有最底层的 PR 以 trunk 为目标
```

```
└─ PR #2 (base: PR #1 的分支)
  └─ PR #3 (base: PR #2 的分支)
```

把 PR 正文写成「Briefing」

这个 Playbook 的要点，是正文规则的第一句话。

The PR body is a briefing, not the lab notebook.

PR 正文是情况简报，而非实验笔记。

不要逐项列举工作中试过什么或经历了什么，只写持有差异的审阅者**作出是否批准改动的判断所需的信息**。正文依次为 ## Why、## Scope、## Tradeoffs、## Blast Radius（影响范围）、## Verification；没有内容的章节就省略。

打开 PR 后也不启动 Babysit

Agent 打开 PR 后，应报告 PR 的 URL，并继续剩余工作。

若对每个 PR 都运行「**Babysit**」，工作每次都会中断。若后续工作又改写提交，该 PR 已运行的 CI 检查就会作废，必须针对新提交重新运行。

例外是第 16 章介绍的「**Autopilot-full**」（由 Agent 自主将 PR 队列推进至合并的 Playbook）和「**Autopilot-stack**」（采用相同步骤，但不合并，只制作一个 PR 栈的 Playbook）中，分别负责单个 PR 的 Agent（负责人）。负责人收到的指令包含「**Babysit**」循环，因此打开所负责的 PR 后，仍会继续推进至 merge-ready（Autopilot-stack 则推进至 STACK-READY）。

请求写法：要求小而有序的提交，以及在说明中给出证据

随附指南中的请求示例要求将提交整理得小而有序，并把证据写入 PR 说明。

```
/poteto-mode open the pr. small ordered commits, evidence in the description.
// 创建 PR。提交要小而有序，并在 PR 描述中写出证据。
```

「Babysit」从尚未合并的最前沿 PR 开始处理问题，使其达到 merge-ready

「Babysit」处理冲突、审阅讨论和 CI 失败，使 PR 或 PR 栈达到 merge-ready 状态。需要人来判断时就停止。

Cursor 也有监视 PR 状态的内置 /autopilot Skill（旧名 babysit），可能由类似请求触发。因此 /poteto-mode 明确规定，所有关于 PR 状态的请求都交给这个 Playbook。例如：

- 「关注这个 PR」
- 「让检查变绿」
- 「处理 Bugbot 评论」
- 「关注 PR X 的进展」

Bugbot.....Cursor 提供的 PR 自动审阅功能。PR 创建或更新时，它读取差异，并以 PR 审阅评论的形式指出可能存在的缺陷或安全问题

步骤：从最前沿 PR 开始逐个处理问题

主要步骤如下。

1. **声明模式**.....查看状态前，先确定要处理到什么程度。默认是 drive；小 PR 或纯文档 PR 使用 check（模式见下表）
2. **只关注最前沿**.....只修复尚未合并的最前沿 PR，不碰上面的 PR
3. **每个 PR 栈只由一位 Agent 负责**
4. **不改变 PR 栈结构（PR 的父子关系）**.....不 rebase、不改 base、不 force-push。若需要 rebase，就向分支负责人报告
5. **按冲突、审阅讨论、CI 的顺序处理**.....已知的修复汇总在一次 push 中，令检查只需重新运行一次。冲突不能自行解决；报告哪个分支需要 rebase 后停止
6. **以 forge 的判断为准**.....「ready」不只是所有 CI 检查通过，还要由 forge 认定可合并。在 GitHub 上，使用随附脚本 `watch-pr` 获取 PR 状态
7. **重新运行 CI 前先分类失败原因**.....只有测试偶发失败（与代码无关、偶尔失败的测试）或基础设施问题时才重新运行；只有失败确由改动中的代码造成时才以提交修复
8. **分流 Bugbot 评论**

9. 需要人判断时停止

- **force-push**……用本地历史强制覆盖远端分支历史的 push
- **分流**……为决定是否需要处理、如何处理而给项目分类（如修复、驳回、提问）

模式	执行内容	对应请求
drive	反复检查状态并修复，直到达到 merge-ready	「让检查变绿」 「使其达到可合并状态」
background	不暂停其他工作，对 PR 状态进行分流	计划仍在执行时
threads-only	只回复审阅评论	「处理 Bugbot 评论」
check	检查一次状态并报告	「看看 X 的进展」 「检查变绿了吗？」

将 Bugbot 评论分为 fix、dismiss、ask

步骤 8 要求在处理 Bugbot 等评论前，先把它分为三类。标准见 [references/bugbot-triage.md](https://github.com/pstack/references/bugbot-triage.md)。

- **fix**……若可能涉及正确性、安全或数据丢失等问题，应在包含问题代码且最早合并的 PR 中修复，并解决讨论。
- **dismiss**……若属于已记录的低风险模式，且可从代码证明所指出的风险并不适用，就简短回复理由并解决讨论。
- **ask**……若是新类型、高严重程度或含糊的指出，就不要猜测，应询问用户。

先对照代码检查 Bugbot 的指出。如果无法判断是否误报，就**不要猜测，而要向用户确认**；错误驳回真实缺陷的损失大于确认所需的时间。

审阅评论是需要验证的主张，不是必须照做的指令。因此，不应只为消除评论而改代码；仅在确认问题后修复。

即使达到 merge-ready，也不合并

步骤 9 的合并规则如下。

Babysitting never authorizes merging. Only an explicit request to merge, land, ship, or merge when ready does.

babysit 不授予合并权限；只有明确请求合并、并入（land）、发布（ship）或准备好后合并，才算授权。

即便达到 merge-ready，**Agent 也应报告并停止**。若用户请求合并，就转交「Shipping」。

达到 merge-ready 后，回顾这次的评论分流；若发现可供团队复用的 dismiss 模式（已记录的低风险模式），应另开 PR 提议补充到 references/bugbot-triage.md。

要点是「只关注最前沿」的规则

在「Babysit」中，**应优先解决下一个待合并 PR 的问题**。

也应阅读后续 PR 的审阅评论并汇总待办事项；但若修复后续 PR 会导致更早的 PR 重跑检查，就推迟处理后续 PR。

例如，PR #1 的 CI 失败时，即使 PR #3 收到审阅评论，也先让 PR #1 的 CI 通过。PR 栈只能自下而上合并，所以当前能推动合并的只有最前沿。

另外，每次收到后续 PR 的评论就修复并 push，会使检查每次都重跑。因此，先积累后续 PR 的问题，待处理最前沿时合并到同一次 push 中修复。

请求写法：要推进就说「让检查变绿」，只查状态就缩小请求

若希望推进到 merge-ready，就说「让检查变绿」。这里再列一次本章开头的请求示例。

```
/poteto-mode babysit this pr. get it green.
// 照看这个 PR，直到所有检查通过。
```

若只想知道状态，就提出较小的请求。以下请求会使用 check 回答，而不启动循环。

```
/poteto-mode check on pr 123. anything outstanding?  
// 检查 PR 123 的状态。还有什么事没完成?
```

「Shipping」只将通过另一位 Agent 验证的连续 PR 范围并入 trunk

用户要求把已通过 CI 的 PR 栈**并入 trunk（PR 合并目标所在的主线分支）**时，使用「Shipping」。它独立验证每个 PR，只从根部开始逐个合并连续且已验证的部分。/poteto-mode 将其概括为「Green is not safe.」（CI 通过并不表示安全）。

要点是「安全」的定义

Safe means a verdict from an agent that did not write the code. CI green is not a verdict, and an approving bot review is not a verdict.

安全是未编写该代码的 Agent 作出的判定。CI 通过并非判定；批准该变更的 bot 审阅也并非判定。

这一定义将第 2 章的「信任的是成果物，而非 Agent」应用于合并前的检查，**连由谁验证也规定清楚**。

步骤：验证每个 PR，确定连续范围，逐个并入 trunk

共有六步。

1. 独立验证每个 PR
2. 只并入从根部开始连续且已验证的范围
3. 确认判定仍对应当前补丁（改动差异）
4. 只准备最前沿 PR，并逐个合并
5. 每次合并后重新计算，并监视最前沿直到完成合并
6. 到上限即停止

下面逐项说明这些步骤。

- `git patch-id`……根据提交差异内容计算标识符的 Git 命令；即使提交 ID 改变，只要差异相同，计算值就相同

1. 独立验证每个 PR

每个 PR 分配一位子 Agent，让其在实际界面或命令中比较父分支与该 PR 的最新提交。结果为 PASS、PASS+NOTES（通过但附有备注）或 FAIL。

2. 只并入从根部开始连续且已验证的范围

步骤 2 的「连续范围」确定方式如下。

```
PR #4  PASS      ← 已验证，但下面还有未验证的 PR
PR #3  (未验证)  ← 连续通过的区间在这里中断
PR #2  PASS
PR #1  PASS      ← 已验证连续区间的最前端
main
```

因此，可以并入的只有 PR #1 和 #2。

要合并 PR #4，必须先验证作为前提、尚未验证的 PR #3，然后才能合并。若跳过 #3 的验证，未经验证的改动也会进入 trunk。

3. 确认判定仍对应当前补丁（改动差异）

即使 rebase 改变了提交 ID（SHA），也可用 `git patch-id` 比较差异内容是否相同。若内容改变，则重新验证。

4. 只准备最前沿 PR，并逐个合并

准备是指必要时将最前沿 PR rebase 到最新 trunk，并把该 PR 的 base 改指 trunk。每合并一个，再准备下一个。

5. 每次合并后重新计算，并监视最前沿直到完成合并

每合并一个 PR，就重新检查新最前沿 PR 的 base、检查结果及 patch-id。即便已预约自动合并，也不能将其视为整个 PR 栈安全的证据。

6. 到上限即停止

上限是已验证连续范围最上方的 PR（步骤 2 的示意图中是 PR #2）。合并到该处后，报告并入了什么、下一个尚未验证的 PR 是哪个，然后结束。

请求写法：说「并入整个 PR 栈」

这里再列一次本章开头的请求示例。

```
/poteto-mode land the stack.  
// 将这组堆叠的 PR 合入 trunk。
```

总结

- **三个 Playbook 如何衔接**……「Opening a PR」创建 PR，「Babysit」使其可合并，「Shipping」将通过验证的 PR 并入 trunk（PR 合并目标所在的主线分支）。
- **Opening a PR**……创建范围小、提交顺序明确的 PR，并把正文写成简短的「情况简报」。打开 PR 后不启动「Babysit」。
- **Babysit**……先声明模式，再只关注最前沿，推进到 merge-ready。将 Bugbot 评论分为 fix、dismiss、ask。
- **Shipping**……将「安全」定义为未编写代码的 Agent 作出的判定，只逐个合并从根部开始连续且已验证的范围。
- **Babysit 与 Shipping 的区别**……两者中只有收到明确合并请求的「Shipping」会合并 PR（「Autopilot-full」等第 16 章的 Playbook 另有规则）。

下一章第 15 章介绍四个 Playbook：在人离开期间继续推进一项长期工作、暂停、恢复及清理。

第 15 章：把长时间任务交给 Agent

kaito • 来源：日本語 • English • 2026-10-03

本章介绍以下四个 Playbook。

1. Autonomous run
2. Pause safely
3. Session pickup
4. Worktree and simulator cleanup

四者都用于将一项长期工作交给 Agent。其作用如下。

- 不间断地推进一项任务直到完成
- 中途暂停，让另一位 Agent 恢复
- 工作结束后清理不再需要的 worktree 等

- **worktree**.....Git 的功能，可从一个仓库建立多个工作目录，同时检出不同分支

随附指南的夜间工作章节（docs/guide/07-overnight.md）列出在人睡觉时委派工作的三个条件：

- 可验证的完成条件
- 隔离的 worktree
- 早晨可审计的决策记录

例如「旧调用方数量为零」是可验证的完成条件，而「工作四小时」不是。

本章先按作用整理四个 Playbook，再依次介绍各自的适用场景、步骤与要点。

本章结构

本章结构如下。

- 四个 Playbook 分别负责一项长期工作的「持续推进、暂停与恢复、清理」
- 「Autonomous run」先确定完成条件，再不间断工作直到满足条件
- 「Pause safely」在毫无背景的 Agent 也能恢复的节点暂停
- 「Session pickup」信任前一位 Agent 的记录，从已完成工作的末尾继续
- 「Worktree and simulator cleanup」删除 worktree 前先证明「无人使用」
- 总结

四个 Playbook 分别负责一项长期工作的「持续推进、暂停与恢复、清理」

本章四个 Playbook 分为下表中的**三种作用**。

作用	Playbook	负责的内容
不间断地推进一项任务直到完成	「Autonomous run」	完成条件
暂停并恢复	「Pause safely」	在可恢复的状态暂停
暂停并恢复	「Session pickup」	从前一项工作的哪个位置恢复
清理	「Worktree and simulator cleanup」	收回磁盘空间，并在删除前确认是否可以删除

本章反复出现的工具

这些 Playbook 中会反复出现以下工具。

- /loop.....Cursor 内置命令，可按固定间隔或 CI 完成等事件唤醒 Agent。
- decisions.tsv……由 /show-me-your-work（见第 28 章）写入的决策记录，每行记载一项决策。

对于长期工作或无人值守的工作，用 /show-me-your-work 记录决策。早晨检查时，无需重读整个夜间工作，**可以根据这份记录（decisions.tsv）审计决策**。

「Autonomous run」先确定完成条件，再不间断工作直到满足条件

「Autonomous run」适用于「一直运行到完成」「用 /loop 运行到 X」之类的请求，持续推进长期任务直到完成。

六个步骤

步骤如下。

1. 确定完成条件
2. 确定如何唤醒 Agent
3. 根据证据逐项作必要改动，并验证结果
4. 自行处理途中发现的问题
5. 每次都用 /show-me-your-work 记录决策
6. 满足完成条件后停止

下面逐项说明这些步骤。

1. 确定完成条件

在第一次循环前，写下「测试通过」「N 个 PR 全部合并」等形式的完成条件。

2. 确定如何唤醒 Agent

使用 /loop 确定唤醒方式。若有 CI 完成、PR 合并等需要等待的事件，就安排一位监视子 Agent 监视事件并唤醒 Agent，并预设较长间隔的定时唤醒作为备用。若没有需要等待的事件，就在值得重新检查结果的时间点，以固定间隔唤醒。

3. 根据证据逐项作必要改动，并验证结果

根据证据作必要的最小改动。改动后检查是否更接近最初设定的完成条件；若更接近就提交，否则撤销（参见 Principle 「[Sequence Work into Verifiable Units](#)」，第 19 章）。

4. 自行处理途中发现的问题

相关缺陷或不稳定的验证由 Agent 自行修复；偏离主线的修复另开 PR。只在操作不可逆、判断无法通过实验作出，或真正遇到瓶颈时请求人来判断。

5. 每次都使用 `/show-me-your-work` 记录决策

每轮循环逐行记录改动内容及是否更接近完成条件。

6. 满足完成条件后停止

若条件尚未满足，就尝试其他方法。不得降低条件来宣称完成；若确实无法继续推进，应报告现状。

回答应包含的项目

回答应包含以下项目。

- 完成条件
- 循环次数
- 最终保留的改动，以及尝试后发现未接近条件而撤销的改动
- 最终条件的状态

要点是把完成条件写成可验证的谓词

谓词就是可判断为真或假的条件，即能判断它是否得到满足。

如开头所述，**时长不能作为完成条件**。「工作四小时」没有可验证的对象，早晨剩下的只是四小时的活动记录。随附指南的常见陷阱章节 (<docs/guide/10-recipes-and-pitfalls.md>) 也把「再改进一些」这类模糊条件列为失败示例。因此，Agent 在步骤 1 确定条件，并在步骤 6 停止之前始终坚持同一条件。

「Pause safely」在毫无背景的 Agent 也能恢复的节点暂停

「Pause safely」用于接到暂停指令、重启 Cursor 或对话即将压缩（将装不下的对话概括整理）时。它让毫无背景的 Agent 也能恢复工作。只有明确指示时才运行；如果被要求「继续」或「不要停止」，就不会暂停。

四个步骤

1. 在安全的节点暂停……完成或撤销当前最小单元的步骤，不开始新工作
2. 不为暂停而执行不可逆操作……除非 PR 已经提交，否则不创建 PR，也不 push
3. 将工作保存为不会消失的形式……把未提交的编辑汇总成一个以 wip: 开头的提交
4. 在上下文之外写恢复备忘……写明意图、进展、下一步及注意事项；若已有 /show-me-your-work 的记录，就引用该记录而不重复内容

回答应包含以下项目。

- 循环进行到哪里
- 已记录在文件或提交中的信息，以及仍只存在于对话上下文中的工作状态、意图和注意事项
- 创建的提交
- 恢复时的第一个行动

要点是把「只存在于 Agent 脑中的内容」转存到外部

Agent 的上下文会在压缩或会话结束时消失。回答中区分「磁盘上的内容（文件或提交中记录的信息）」与「只存在于脑中的内容（工作状态、意图、注意事项）」，便能看出哪些内容未能移入文件或提交。

wip: 提交将编辑中的代码保存在磁盘上；恢复备忘则保存无法写入代码的意图和注意事项。可以把它理解为在单项工作中落实第 4 章所说的「代码库就是记忆」。

「Session pickup」信任前一位 Agent 的记录，从已完成工作的末尾继续

「Session pickup」用于根据对话记录（与 Agent 对话的记录）、Cloud Agents URL 或已推送分支，接手前一位 Agent 尚未完成的工作。它与「Pause safely」成对。

若接手一项具体工作，使用「Session pickup」；若需要汇集多个对话的上下文，使用 `/recall`（见第 22 章）。

五个步骤

1. 寻找先前记录

- 查找当前工作区的对话记录、Cloud Agents URL 或已推送分支
- 让子 Agent 阅读较长的对话记录，只把摘要放入主线程（调用子 Agent 的原始对话；参见 Principle 「Guard the Context Window」，第 20 章）

2. 重建工作状态

- 查明工作分支、已完成的改动（通过 `git log` 或与 base 的 `git diff` 核实）、剩余 TODO 及已作决策

3. 比较已完成与剩余工作

- 比较计划与完成情况，确定从哪里恢复，避免重复已完成的工作

4. 将剩余工作交给适合其内容的 Playbook

- 后续工作由适合工作内容的 Playbook 接手

5. 用实际成果核实接手的主张

- 不把前一位 Agent 自称「成功」当作证明，而要实际检查接手的改动（参见 Principle 「Prove It Works」，第 19 章）

回答应包含以下项目。

- 前一位 Agent 在哪里停止
- 接手的内容与重新做过的内容
- 恢复起点
- 结果

要点是将先前记录作为可信的输入

接手的 Agent 不必重新推导前一位 Agent 的记录，而应直接把它作为输入使用。

Playbook 要求使用先前记录区分已完成与未完成的工作，从剩余部分继续。

例如，前一位 Agent 已复现缺陷并留下操作步骤、画面与日志，接手者就不必再重复操作，而应阅读记录，从调查原因或修复开始。

请求写法：要求阅读决策日志，从已完成工作的末尾继续

随附指南中的示例请求接手一个分支，要求阅读决策日志以查明已完成的工作，且不要重做已经完成的部分。

```
/poteto-mode take over this branch. read the decision log, figure out what's done, and continue from there. don't redo finished work.
// 接手这个分支。阅读决策日志，了解已经完成的工作，然后从那里继续。不要重做已完成的工作。
```

「Worktree and simulator cleanup」删除 worktree 前先证明「无人使用」

「Worktree and simulator cleanup」用于删除已合并或废弃的 git worktree 和旧 iOS 模拟器，以释放磁盘空间。

六个步骤

步骤如下。

1. 记录并审计
2. 分类是建议，不是许可
3. 删除前确认使用情况
4. 在不可逆的损失前停下
5. 删除已确认的对象
6. 清理模拟器等其他可回收对象

下面逐项说明这些步骤。

1. 记录并审计

使用 `df -h /` 记录当前磁盘用量，再运行 `worktree-audit.sh`（参见 Principle 「**Build the Lever**」，第 17 章）。

`worktree-audit.sh` 是不会删除内容的只读脚本。它检查以下项目。

- 每个 worktree 的大小
- 每个 worktree 的存续时间（距离最后更新的时间）
- 是否已经合并
- 未提交的编辑
- PR 状态
- 最后操作的对话

脚本依据这些信息给出分类建议。

2. 分类是建议，不是许可

脚本的分类只是删除候选建议。被置顶或仍在进行的对话（侧边栏中与 Agent 的对话）正在使用的 worktree，必须排除。脚本曾把置顶对话使用的 worktree 判定为 safe，因此还要对照对话列表检查候选。

3. 删除前确认使用情况

若有疑问，就让子 Agent 阅读对话记录（与 Agent 的对话记录）来确认。

4. 在不可逆的损失前停下

若存在未提交的工作，就把差异展示给用户，由用户判断。

5. 删除已确认的对象

使用 `git worktree remove` 和 `git worktree prune` 删除。分支会保留，提交不会丢失。

6. 清理模拟器等其他可回收对象

删除旧模拟器、运行时环境，必要时也清理构建缓存。

回答应包含的项目

回答应包含以下项目。

- 删除前后的可用磁盘空间以及增加的容量
- 已删除的 `worktree`
- 保留的对象及各自的理由

要点是删除前的关卡即为审阅

删除 `worktree` 或模拟器可能使进行中的环境或未提交的改动丢失。因此，删除前要确认其无人使用，且没有未提交改动。这些检查就是防止误删的关卡（删除前必须通过的检查）。

其他 Playbook 可通过代码审阅发现错误；这个 Playbook 则是唯一一个不经代码审阅就删除用户本地 `worktree` 或模拟器的 Playbook。因此，步骤 2 至 4 对使用情况和未提交工作的**检查关卡本身就是审阅**。

请求写法：请求「删除可安全清理的 `worktree`」

随附指南的示例询问哪些内容占用磁盘空间，并要求只删除可安全清理的 `worktree`。

```
/poteto-mode what's eating my disk? prune the worktrees that are safe to prune.
// 什么占用了磁盘空间？删除可以安全删除的 worktree。
```

总结

- 四个 Playbook 的作用……「**Autonomous run**」将一项任务推进至完成；「**Pause safely**」与「**Session pickup**」负责暂停和恢复工作；「**Worktree and simulator cleanup**」负责清理。
- **Autonomous run**……确定可验证的完成条件，持续工作直到满足条件。

- **Pause safely**……把只存在于 Agent 脑中的内容通过 wip: 提交和恢复备忘保存下来，再暂停。
- **Session pickup**……信任先前记录，不重复已完成的工作，并实际核实接手的主张。
- **Worktree and simulator cleanup**……证明无人使用后再删除。删除前的关卡即为审阅。

下一章第 16 章介绍规划多个 PR、由多个 Agent 执行的四个 Playbook。

第 16 章：规划大量 PR，组织多个 Agent 执行

kaito · 来源：日本語 · English · 2026-10-03

本章介绍以下四个 Playbook。

1. Multi-phase or multi-PR plan
2. Orchestrate
3. Autopilot-full
4. Autopilot-stack

四者用于规划并由多个 Agent 执行跨越许多 PR 的工作。「**Multi-phase or multi-PR plan**」编写计划，「**Orchestrate**」管理持续数日的工作，而「**Autopilot-full**」和「**Autopilot-stack**」在验证的同时推进 PR 队列。

上一章第 15 章介绍了持续推进、暂停和恢复一项长期工作的 Playbook。

本章先整理四者的分工，再说明各自的适用场景、步骤与要点，最后比较如何选择长期运行的工作方式。

本章结构

本章结构如下。

- 四个 Playbook 用于规划和执行多个 PR，居中的 Agent 不编写代码
- 「Multi-phase or multi-PR plan」不编写代码，产出清单式计划，连每个 PR 的验证方式也事先确定
- 「Orchestrate」由一位不编写代码的协调 Agent 管理持续数日、涉及许多 PR 的工作
- 「Autopilot-full」由各 PR 的负责人推进到合并，协调 Agent 只负责验证判定
- 「Autopilot-stack」按与 Autopilot-full 相同的步骤创建并验证 PR，不合并，而是作为一个 PR 栈交给操作员

- 长期运行工作的五种选择，取决于工作单元是什么、由谁合并
- 总结

四个 Playbook 用于规划和执行多个 PR，居中的 Agent 不编写代码

本章四个 Playbook 按下表分担职责。

Playbook	负责的内容
「Multi-phase or multi-PR plan」	计划，而非代码
「Orchestrate」	项目（持续数日的整体工作），不持有代码
「Autopilot-full」	验证判定，不持有 PR
「Autopilot-stack」	PR 栈，不负责合并

四个 Playbook 都明确写明自己「不持有的内容」（代码、PR 或合并）。

协调 Agent 自己不编写代码，专注于计划、给 Workers（并行工作的多个 Agent）的任务说明和判定。不过在「Orchestrate」中，若本地 Git 操作能迅速完成，协调 Agent 也可以将已验证的改动并入分支并 push。

协调 Agent 是与**操作员（提出工作请求并作最终决定的人）**交互的常驻对话。在

「Orchestrate」中称为「协调者」，在「Autopilot-full」和「Autopilot-stack」中称为「根 Agent」。名称不同，本章统称「协调 Agent」。

本章反复出现的工具

这些 Playbook 中会反复出现以下工具。

- `/goal`……为 Agent 设定目标的命令。设定的目标跨对话轮次持续有效，直到队列工作完成。
- `decisions.tsv`……由 `/show-me-your-work`（见第 28 章）写入的决策记录，每行记载一项决策。
- **state-then-wait**……若只被要求说明方法或计划，就说明后停止；操作员明确说「开始」之前，不启动工作。

「Multi-phase or multi-PR plan」不编写代码，产出包含每个 PR 验证方法的清单式计划

「Multi-phase or multi-PR plan」用于规划跨多个阶段或堆叠 PR 的工作。

计划采用清单形式。负责人逐项执行，用证明完成的证据勾选。操作员查看证据，核实每项是否真正完成。**缺少证据的项目不能勾选。**

poteto 在「[The Complete Guide to pstack](#)」Part 2 中介绍：这个 Playbook 用于在满意的设计完成之后制定执行计划。关于制定计划的实际讲解见第 39 章。

七个步骤

步骤如下。

1. 判断是否需要计划
2. 未解决的问题先用原型决定，再开始写计划
3. 将调查交给子 Agent
4. 复制计划模板并填完所有项目
5. 按技术文档标准编写计划，去除 AI 腔
6. 检查计划的形式
7. 交付计划

下面逐项说明这些步骤。

1. 判断是否需要计划

若改动仅涉及一到两个文件且方法明确，就不编写计划，说明无需计划并停止。

2. 未解决的问题先用原型决定，再开始写计划

通过「**Prototype**」（见第 12 章）作决定，并把证据留在计划附录。只在实验无法解决时才询问操作员（参见 Principle「**Never Block on the Human**」，第 20 章）。

3. 将调查交给子 Agent

调查由子 Agent 完成，让其返回相关文件位置、代码约定、运行测试的命令，以及入口代码（例如 main 函数或最先接收 API 请求的函数）。

不要让子 Agent 把读取的文件全文贴在回答中。

这样可避免调查材料填满主 Agent 的上下文（参见 Principle 「**Guard the Context Window**」，第 20 章）。

4. 复制计划模板并填完所有项目

对计划中每个 PR，写明该 PR 的改动、涉及文件与验证方式。一个 PR 只包含一个可验证单元的改动。

5. 按技术文档标准编写计划，去除 AI 腔

使用规定技术文档标准的 /technical-writing 编写，再运行 /unslop 去除 AI 常见套话。

6. 检查计划的形式

运行 check-plan.mjs，修复输出的所有错误，包括缺少标题或验证项，以及违反写作规则。

7. 交付计划

在回答中给出计划路径与脚本输出，然后停止。执行须等操作员明确下令开始，再依照计划指定的执行 Playbook（「**Autopilot-full**」、「**Autopilot-stack**」或「**Orchestrate**」之一）启动。

回答应包含的项目

回答应包含以下项目。

- 计划路径
- PR 依赖关系
- 需要审阅的 PR
- 原型证明了什么

- `check-plan.mjs` 的输出

计划模板与 `check-plan.mjs`

计划结构如下。

- 开头概要
- 阅读方法
- 项目总清单
- 各 PR 章节
- 结束章节
- 附录（原型证据、未采纳方案、风险、资料）

每个 PR 章节包含以下**九个小标题**。

- 依赖关系
- 涉及文件
- 改动
- 可观察的结果
- unit 验证（单元测试）
- live 验证（应用实际行为）
- perf 验证（性能）
- Review gate（合并前是否需要操作员确认，以及如何确认）
- 合并

`check-plan.mjs` 检查计划的标题、必填项和写法是否违反规则。有问题时，它会显示行号和原因。这个 Playbook 不只用文字描述计划规则，还用脚本检查（参见 Principle「**Encode Lessons in Structure**」，第 21 章）。

要点是「只有测试不等于验证」这一规则

这个 Playbook 在 unit、live 和 perf 各项验证的开头，都放了下面这句共同的通过条件。

Tests alone are not sufficient verification. A PR is verified only when its unit, live, and perf boxes are all checked.

只有测试还不足以构成充分验证。只有 unit、live 和 perf 的方框全部勾选，PR 才算通过验证。

计划中为每个 PR 设置 unit、live、perf 验证栏，并写明各自的具体检查方法。影响界面或操作的 PR，还要设置合并前由操作员确认的「Review gate」。

各栏内容如下。

- **Verify, unit**……写明测试文件、新增用例及运行命令。
- **Verify, live**……必填项。十条 lane（并行验证职责）在 PR 的最新提交上实际操作界面或 CLI。Lane 1 是回归检查 lane：在 trunk（合并目标所在的主线分支）和 PR 上执行同一场景，检查是否因改动产生缺陷。
- **Verify, perf**……在 trunk 和 PR 上测量同一指标，并确定判为失败的数值。
- **Review gate**……影响界面或操作的 PR，合并前由操作员查看截图与视频进行审阅。

操作画面时，浏览器或 Electron（构建桌面应用的框架）使用 control-ui；CLI 和 TUI 使用 control-cli（两者均来自 [cursor-team-kit](#)）。

- **TUI**……在终端中绘制界面并通过按键操作的用户界面，例如 vim 或 htop

请求写法：写明必须遵守的约束，以及希望先看到什么

README 中的请求示例写明了不能泄露内部资料、要在临时目录工作，以及希望先查看依赖关系图。

```
/poteto-mode open source these skills as a plugin. nothing internal leaks, work in a temp dir, show me the dependency graph first.
// 将这些 Skill 作为插件发布。不要泄露任何公司内部资料；在临时目录中工作，并先展示依赖关系图。
```

「Orchestrate」由一位不编写代码的协调 Agent 管理持续数日、涉及许多 PR 的工作

「Orchestrate」将持续数日、涉及大量 PR 和数十至数百位子 Agent 的项目交给一位协调 Agent。预期操作员每天只检查两次。协调 Agent 不持有代码，而是给 Workers（多个子 Agent）编写任务说明，处理完成通知并作决定。

七个步骤

步骤如下。

1. 确定完成条件（Frame）
2. 准备存放状态的位置（Install the runtime）
3. 用一個工作单元试运行（Pilot）
4. 增加 Workers（Scale）
5. 批量处理完成通知（Drain）
6. 依次合并已验证的工作单元（Land）
7. 核对记录并关闭（Close）

下面逐项说明这些步骤。

1. 确定完成条件（Frame）

完成条件要写成能从记录判断是否完成的形式，例如「126 个工作单元全部合并，且台账中每个都有已验证记录」。若一位 Agent 能在预定工作时限内完成，就改用「**Autonomous run**」（见第 15 章）。

2. 准备存放状态的位置（Install the runtime）

运行 `orch init` 创建存储区（保存工作状态和决策记录的区域）、建立决策记录，并写下常驻指令。

3. 用一个工作单元试运行 (Pilot)

先完整推进一项工作：编写任务说明、实现、验证、登记到 PR 栈、记入台账并合并。根据结果调整任务说明与验证方法，再扩展到其余工作。

用一位 Agent 和一项工作进行试运行 (Pilot)，是为了先解决任务说明、验证步骤和工作单元规模的问题，再扩大范围，**避免同一缺陷扩散到许多 Agent 并造成返工。**

4. 增加 Workers (Scale)

启动 Workers，直到达到同时运行的上限；每完成一项，再启动新的 Worker 补位。

5. 批量处理完成通知 (Drain)

先将 Workers 的完成通知积累在收件箱 (inbox/)，在固定节点集中处理。

6. 依次合并已验证的工作单元 (Land)

不要最后一次性合并，而是从第一个通过验证的合并开始持续推进。

7. 核对记录并关闭 (Close)

核对所有已启动 Agent 是否都已在表中记为完成、放弃等终止状态。检查实物以确认完成条件已满足；将反复纠正的指令或经常需要的指令写入 `preferences.md` 或任务说明模板，供下一位 Agent 使用。

回答应包含的项目

回答中应报告以下内容，涉及 PR 的项目附上对应 PR 链接。

- 从 `units.tsv` 和 `ledger.tsv` 统计的、相对于完成条件的进度
- 已合并的 PR
- 已放弃的工作及原因
- 等待操作员决定的事项

进度数值来自记录每项工作状态的 `units.tsv` 和记录验证结果的 `ledger.tsv`。

三层职责

- **协调 Agent**（在 Orchestrate 中称为协调者）……主 Agent，负责整体框架、任务说明、收件箱处理与向操作员报告；不编写代码。
- **子协调者**……仅在工作规模超出协调 Agent 处理能力时，每条称为 track 的工作系列（如构建或验证）安排一位子 Agent。
- **Workers（执行者）与验证者**……Workers 执行分配的工作，验证者检查 Worker 的成果。两者原则上都在云端运行；验证者使用与 Worker 不同系列的模型。每个 worktree 或分支只允许一位 Worker 编辑。

- **worktree**……Git 的功能，可从一个仓库建立多个工作目录，同时检出不同分支

存储区、任务说明和台账承载状态

协调 Agent 在 Agent 存储区（保存工作状态和决策记录的区域）创建 `orchestrate/<project-slug>/`，通过记账用 CLI `orch` 将状态写入表格文件。主要文件如下。

文件	内容
<code>preferences.md</code>	所有 Agent 都应遵守的共同指令。每行一条规则，在启动与恢复时每次都传入
<code>units.tsv</code>	各工作单元的 ID、track（并行推进的工作系列）、状态、分支、PR、head SHA（PR 的最新提交 ID）、任务说明位置
<code>ledger.tsv</code>	记录每个 PR 验证结果的台账
<code>inbox/</code>	记录 Worker 发来的完成通知及报告存放位置
<code>gates.md</code>	需要操作员决定的事项，记录问题、选项，以及无人回复时如何处理
<code>decisions.tsv</code>	工作中所作决策及理由的记录

任务说明是 Workers 接收的唯一说明

协调 Agent 向 Workers 分派工作时，为每个工作单元编写任务说明（brief）。Workers 不能在工作中向协调 Agent 提问。因此，**任务说明中没写的内容，Workers 只能靠猜测补足。**

靠猜测推进的工作容易偏离目标。因此，任务说明预先规定了必填栏位，主要包括：

- **GOAL（目标）** ……用一句话写明工作目标，让不了解聊天经过的人也能执行。
- **SCOPE（范围）** ……写明允许修改和禁止修改的文件，以及专用分支。
- **CONTEXT（背景）** ……写明参考文件或 PR。Workers 看不到其他 Workers 的工作；若本项依赖上一项结果，就原样贴入前一位 Worker 的完整报告。
- **ACCEPTANCE（完成条件）** ……逐行写出可验证的完成标准。
- **VERIFY（验证方式）** ……写明验证时运行的命令或步骤。
- **TIMEBOX（时间上限）** ……写明大致的工作时限；到达时限就返回已取得的结果并停止。
- **FORBIDDEN（禁止事项）** ……写明不得执行的操作，例如范围外修改或 force-push。
- **REPORT（报告内容）** ……写明工作完成后应返回的报告项目。
- **STANDING（常驻指令）** ……原样贴入 preferences.md 内容。

若有任何一栏无法填写，就不启动 Workers。例如，写不出 ACCEPTANCE，表示尚未确定怎样才算完成。此时启动 Workers，只会让他们猜测完成标准，导致结果不一致。无法填写的栏位表明工作范围仍不清楚，须先解决后再启动 Workers。

但任务说明的长度要与工作规模相称。若只改几行代码，却把每一栏详尽写成长篇任务说明，写和读的成本可能超过工作本身。对于这类小工作，可在一段话中写明目标、范围、验证命令和报告格式。

台账记录 PR 验证到什么程度

PR 是否真的得到验证，记录在台账（ledger.tsv）中。台账为每个 PR 写一行判定，表示验证到什么程度。判定分为以下五种。

判定	含义
live-ui-verified	实际操作界面或 CLI，验证行为
unit-test-verified	通过单元测试验证
type-check-only	仅通过类型检查

判定	含义
verifier-blocked	因环境问题等原因无法执行验证
verifier-failed	验证发现问题

例如，改变界面或行为的工作，只有 type-check-only 类型检查还不够，必须取得更有力的判定，因为类型检查不会验证实际行为。

verifier-blocked 并非通过，因此环境修复后，协调 Agent 要重新验证。若结果为 verifier-failed，则不应反复运行相同验证，而应新建工作单元修复问题。

CI 通过只是作出判定的一项材料，**仅凭 CI 通过不能作出通过判定**。CI 只能检查预设的测试，无法涵盖实际行为。

判定按 PR 编号与 head SHA（PR 最新提交 ID）的组合记录。若给 PR 增加提交或进行 rebase（将分支提交重新叠放在其他提交上的 Git 操作），head SHA 就会改变。

head SHA 变更后，PR 内容可能已不同于判定记录时。因此，**不能使用旧 head SHA 对应的判定**；提交改变就应重新验证，并将判定记在新 head SHA 下。

只由一位 stacker 重新整理 PR 栈

在 PR 栈（由父子关系连接的一系列 PR）中，下层 PR 内容改变后，上层 PR 也需要重新叠放在改变后的 PR 上（见下图）。这项操作称为 restack。

更新之前

main

```
└─ PR #1 (父分支是 main)
   └─ PR #2 (父分支是 PR #1)
      └─ PR #3 (父分支是 PR #2)
```

对 PR #1 添加改动 A 后的更新顺序

PR #1 → PR #2 → PR #3

更新之后

main

```
└─ PR #1 (改动 A)
   └─ PR #2 (已纳入更新后的父 PR #1)
      └─ PR #3 (已纳入更新后的父 PR #2)
```

在「Orchestrate」中，每个 PR 栈只安排一位负责 restack 的 Agent，称为 stacker（PR 栈管理者）。

stacker 使用 Graphite (gt) 命令管理堆叠的 PR，执行 restack。

- **Graphite**……以父子关系创建和管理 PR 栈的工具

Workers 不使用 gt，也不进行 rebase。多个 Agent 同时重排同一 PR 栈，会互相覆盖改动或破坏 PR 的父子关系。**将重排限定给唯一一位 stacker**，保证始终只有一位 Agent 写入 PR 栈，从而避免冲突。

不要为了查看进展而恢复 Agent

即使要查看进度，也不恢复并询问处于等待中的 Agent，因为询问会使它重新开始工作。协调 Agent 应改为只读检查其留下的记录，例如台账、units.tsv 或已推送分支。

只有四类问题需要操作员判断

这个 Playbook 预设 Agent 长时间自主工作，操作员一天仅检查两次。若连 Agent 能决定的事项也去询问，工作会在等待回复期间停止。因此，只将以下四类事项交给操作员判断。

- 不可逆操作（如对共享分支 force-push、部署、删除）
- 无法通过试验得出答案的产品或偏好判断
- preferences.md 的共同指令与实际情况冲突
- 重新规划后仍未解决的瓶颈

询问前，先把事项记录在 gates.md；等待答复时仍继续其他工作。

- **force-push**……用本地历史强制覆盖远端分支历史的 push

要点是关于完成通知、常驻指令和任务说明的三条规则

这个 Playbook 开头列出三条规则，后续步骤均以此为前提。

- **完成通知不逐条即时处理，而要积累后集中处理**……数十位 Workers 可能同时工作；每来一条通知就停下，会妨碍协调 Agent 自身推进工作。因此，通知先放入收件箱（inbox/），在合适的节点集中处理（步骤 5）。
- **启动与恢复时都必须传入共同指令**……Agent 跨恢复可能丢失先前指令。每漏一条，操作员就要重新解释一次。所以每次启动或恢复时都原样传入 preferences.md。
- **任务说明的质量决定成果质量**……Workers 工作中无法提问；即使说明含糊，也无人发现，最终会得到偏离目标的成果。

请求写法：写明完成条件与检查频率

随附指南 07-overnight.md 中的请求示例，写明全部包转换并合并的完成条件，以及操作员每天检查两次。

```
/poteto-mode orchestrate the store migration. own it until every package is converted and merged.
i'll check in twice a day.
// 统筹存储系统迁移，直到所有软件包都完成转换并合并。每天检查两次。
```

「Autopilot-full」由各 PR 的负责人推进到合并，协调 Agent 只负责验证判定

「Autopilot-full」用于将互相独立的 PR 队列（待处理工作的序列）推进到合并，无需操作员逐一确认。

每个 PR 分配一位 Agent，称为**负责人**。负责人从创建自己的 PR 一直负责到合并。

另一方面，**协调 Agent 不直接操作 PR，只负责验证判定**。没有协调 Agent 的通过判定，负责人不能合并。负责人也检查自己的工作，但作者本人容易漏看问题，所以还要求另一位 Agent 验证通过。

通过判定须满足条件：使用 /swarm（见第 24 章）并行运行多个验证职责，其中必须包含实际操作界面或 CLI 验证行为的 live lane。lane 是一条并行运行的验证职责。live lane 是最低要求；没有 live lane 的判定不能算通过。此外，负责人须修复 lane 提出的全部问题，包括作为备注提出的意见。达到这些条件的通过判定称为 **clean verdict**（字面意思是「干净的判定」）。

七个步骤

步骤如下。

1. 等待操作员下令开始 (state-then-wait)
2. 为每个 PR 启动一位负责人
3. 各负责人并行工作，不把 PR 堆叠起来
4. 每轮通过 /swarm 验证
5. 取得 clean verdict 后，由负责人自行合并
6. 协调 Agent 大约每三十分钟检查全局
7. 操作员叫停时，停止所有负责人

下面逐项说明这些步骤。

1. 等待操作员下令开始 (state-then-wait)

若只被要求解释计划，就解释后停止。操作员明确下令开始后，将整体目标设入 /goal，再启动。若操作员指定某个 PR 由自己处理，负责人不合并该 PR，交给操作员。

2. 为每个 PR 启动一位负责人

负责人是在 Cursor 云端运行的 Agent (Cloud Agents)，负责创建 PR、在实际环境中验证行为、分流 Bugbot 评论、rebase 到 trunk (合并目标所在的主线分支)、通过「**Babysit**」(见第 14 章) 推进至 CI 通过，最后合并。

3. 各负责人并行工作，不把 PR 堆叠起来

若各 PR 互相独立，就能并行推进而不冲突；各自从 trunk 开始工作，不堆叠。

4. 每轮通过 /swarm 验证

一轮是负责人确定代码并接受验证的一次流程。每次 push 修复都会开始新一轮。针对确定时的提交并行启动多个验证者，将结果汇总成一个判定；缺少 live lane 的判定不算 clean verdict。

5. 取得 clean verdict 后，由负责人自行合并

合并前重新 rebase 到最新 trunk，再合并并从队列领取下一项工作。

6. 协调 Agent 大约每三十分钟检查全局

重读 Playbook 和 /goal；若执行偏离要求，当场纠正。同时检查是否有停滞的负责人，有则替换。

7. 操作员叫停时，停止所有负责人

操作员的停止指令应立即传达给所有负责人，内容为「此后不要再写入任何内容」。

回答应包含的项目

回答应包含以下项目。

- 每个 PR 的负责人、状态与判定
- 已合并的 PR
- 等待操作员决定的事项
- 决策记录位置

要点是只按「副作用」计算进度

在 Autopilot-full 中，许多负责人和子 Agent 同时工作，其中有的可能因错误而停下，却没有报告。若放任停滞的 Agent，其负责的 PR 会一直停滞。

因此，协调 Agent 不依赖 Agent 自述，只把**工作实际留下的痕迹算作进度**。这些痕迹包括提交、push、PR 或检查状态的变化，以及写入存储区（保存工作状态和决策记录的区域）的报告。这些可从外部观察到的工作结果称为副作用。

若负责人超过预期时间仍没有留下副作用，就视为停滞，不等回复而直接换人（在步骤 6 的检查中执行）。这样避免停滞负责人拖慢整体进度。「Orchestrate」不为检查进度而唤醒等待中的 Agent，而是查看提交、PR 状态和台账记录，出于同样的理由。

请求写法：写明各项目互相独立，以及合并期限

随附指南中的请求示例说明队列中的每一项互相独立，并要求在早晨前合并。

```
/poteto-mode full autopilot on this queue. each item is independent. i want them merged by morning.
// 用 full autopilot 处理这个队列。各项任务相互独立，早上之前完成合并。
```

「Autopilot-stack」按与 Autopilot-full 相同的步骤创建并验证 PR，不合并，而是作为一个 PR 栈交给操作员

「Autopilot-stack」按与「Autopilot-full」相同的方式创建并验证 PR，但不合并。它将通过验证的 PR 堆成一条 PR 栈（由父子关系连接的 PR 序列），再交给操作员。操作员审阅后自行合并。

若要人在合并前审阅、改动之间相互依赖而必须按顺序堆叠，或无法授予 Agent 合并权限，就使用这个 Playbook。

步骤：与 Autopilot-full 的区别

步骤与「Autopilot-full」基本相同，区别如下。

- 验证通过后，负责人报告 STACK-READY（可加入 PR 栈），而不是 merge-ready（可合并）。
- 所有负责人都不合并。只将取得 clean verdict 的 PR 依次加入同一 PR 栈。**未通过验证的 PR 不加入 PR 栈。**
- 只有协调 Agent 可以改动 PR 栈的排列（哪个 PR 叠在哪个 PR 上）。负责人只 push 自己的分支。
- trunk（合并目标所在的主线分支）前进后，协调 Agent 从最前沿 PR 开始依次 rebase 整条 PR 栈（rebase 是将分支提交重新叠放到另一提交上的 Git 操作）。rebase 会改变 head SHA（PR 的最新提交 ID），所以内容变化的 PR 需重新验证。
- 最后，将附有每个 PR 验证判定的 PR 栈交给操作员。操作员从最前沿开始依次审阅并合并。

回答应包含以下项目。

- PR 栈最前沿的 PR（指向 trunk 的 PR）与最顶部 PR 的链接

- 每个 PR 的判定摘要
- 从 PR 栈中排除的 PR 及原因

要点是只有协调 Agent 能改动 PR 栈排列

PR 栈的排列是所有负责人共享的信息。若多个负责人同时改写 PR 的父子关系，各人的改动可能互相冲突，破坏 PR 栈。

因此，**只有协调 Agent 能改动 PR 栈排列**，负责人只能 push 自己的分支。把写入职责限定给一位 Agent，就能避免排列冲突。

这与「**Orchestrate**」安排每个分支只有一位 Worker 写入、每条 PR 栈只由一位 stacker restack 一样，是对 Principle「**Separate Before Serializing Shared State**」（见第 18 章）的一种落实。

请求写法：写明「整理成 PR 栈，但不要并入生产环境」

随附指南中的请求示例要求将改动堆成 PR 栈，不并入生产环境，而由操作员亲自并入。

```
/poteto-mode autopilot these five changes but stack them, don't ship. i'll land the stack in the morning.
// 用 autopilot 处理这五项改动，但要组成 PR 堆栈，不要投入生产。早上我会亲自合入整个堆栈。
```

长期运行工作的五种选择，取决于工作单元是什么、由谁合并

委派长期运行的工作时，应根据**工作单元是什么、由谁合并**，从「Autonomous run」

「Orchestrate」「Autopilot-full」「Autopilot-stack」四个 Playbook，以及在没有合适 Playbook 时自行设计可审计步骤的 /figure-it-out（见第 23 章）中选择。

比较角度	「Autonomous run」	/figure-it-out	「Orchestrate」	「Autopilot-full」	「Autopilot-stack」
工作单元	一项任务	一次大规模工作	持续数日的整体项目	独立 PR 队列	按顺序或依赖关系排列的改动队列

比较角度	「Autonomous run」	/figure-it-out	「Orchestrate」	「Autopilot-full」	「Autopilot-stack」
由谁合并	Playbook 未规定	取决于设计的流程	协调 Agent 或 stacker	各 PR 负责人	操作员
适用场景	能将完成条件写成一项可验证的条件，且一位 Agent 能完成	工作规模大、涉及多个位置，或没有适合的 Playbook	无法在一位 Agent 的可运行时长内完成	各 PR 独立，且 Agent 获得合并权限	希望合并前审阅、工作之间存在依赖，或无法授予合并权限

在两个 Autopilot Playbook 中，若 PR 互相独立且已授予合并权限，选择「**Autopilot-full**」；否则选择「**Autopilot-stack**」。

总结

- 四个 Playbook 的分工……「**Multi-phase or multi-PR plan**」负责计划；「**Orchestrate**」管理持续数日的工作；「**Autopilot-full**」让各 PR 负责人推进至合并；「**Autopilot-stack**」不合并，而是交付一条 PR 栈。每者都明确规定协调 Agent 不持有什么。
- **Multi-phase or multi-PR plan**……编写清单式计划。只有 unit、live、perf 三项俱全，才算通过验证。
- **Orchestrate**……通过任务说明、存储区表格和台账管理持续数日的工作。说明栏位无法填写完整时不启动 Worker；判定按最新提交记录。
- **Autopilot-full**……各负责人推进至合并，协调 Agent 只负责判定。进度只按副作用计算。
- **Autopilot-stack**……不合并，交付一条 PR 栈。只有协调 Agent 可以改动 PR 栈排列。
- **五种选择**……依据工作单元是什么、由谁合并来选择。

第三部分已逐一介绍 pstack 的所有 Playbook。第四部分将讨论在工作中如何选择方案、何时结束等指导 Agent 判断的 Principle。

第四部分：用 Principle 支持 Agent 的判断

kaito · 来源：日本語 · English · 2026-10-03

第四部分全面介绍 pstack 的全部 23 条 Principle。

这一部分可以跳读。我建议只阅读自己感兴趣的章节。

各 Principle 的原文随时可在 [pstack](#) 仓库中阅读。pstack 更新频繁；需要时再查阅相应章节和原文件，比预先读完全部内容更可靠。

第四部分将介绍什么

- 各 Principle 要求选择什么、避免什么，以及为什么需要它（例如「**Prove It Works**」要求不能只凭 build 通过就宣称完成，还要直接检查生成文件或界面）
- 各 Principle 适用与不适用的场景（例如「**Outcome-Oriented Execution**」适用于有计划的迁移，不适用于日常没有阶段边界的小修复）
- 如何在请求中使用 Principle 名称改变 Agent 的工作方向（例如写 `apply prove it works.`，要求真正执行导入并展示写入的记录）

Principle 是**Agent 在工作中作判断的标准**。Playbook 规定工作顺序，但 Agent 推进各步骤时仍需判断，例如「是否该增加这层抽象」「编译通过后能否说已完成」。遇到这些问题，Agent 依据 Principle 决定「是否加入这项改动」或「是否仍不能宣称完成」。

23 条 Principle 按所处理的判断类型分为以下五组。读者可根据当前犹豫的判断类型，选择该读哪章。

分组	数量	处理的判断
Core（核心）	10	要做多少、何时重新审视设计

分组	数量	处理的判断
Architecture（架构）	6	状态、验证和兼容性应放在哪里
Verification（验证）	4	什么才算证明
Delegation（委派）	2	怎样让并行工作与委派顺利推进
Meta（改进机制本身）	1	如何改进机制本身

Principle 不是由用户明确调用的，而是由 **Agent 判断工作是否符合适用场景，再按需读取**。

/poteto-mode（启动工作时使用的 Skill；见第 7 章）会在多步骤工作开始时阅读 Principle 索引。

索引是 /poteto-mode 的 SKILL.md 中的清单，每条 Principle 的名称及适用场景各占一行（例如「Prove It Works」适用于「完成工作之后、宣称完成之前」）。对于符合当前场景的 Principle，Agent 先阅读全文 SKILL.md，再应用规则，并在回答中列出所用 Principle 的名称及其改变了哪些判断。

Agent 在工作开始时只读取 Principle 名称及适用场景的清单。只有当工作符合某个场景时才阅读相应正文，并用于判断。这样无须每次都加载全部 23 篇正文，节省上下文。

用户也可以通过**在请求中写入 Principle 名称**来明确调用这一机制。若 Agent 漏掉了适用场景，用户只需在请求中写下 Principle 名称，Agent 就能按该名称对应的规则调整工作方向。

写一个 Principle 名称，比写长篇指令更准确。名称指向其 SKILL.md 中完整的规则，用户无需重新讲述其具体内容。

例如，Agent 只因 build 通过就要宣称完成时，可以按随附指南（pstack 附带的 docs/guide/ 指南；见第 6 章）中 docs/guide/08-principles.md 的示例写请求。

```
apply prove it works. run the real import flow and show me the written records.
// 使用 prove it works。实际运行导入，并展示写入的记录。
```

本部分的章节

章节	作用
第 17 章 Core——决定工作方式的十条原则	工作量、重新审视设计的方式、原型制作方式，以及是否要制作工具
第 18 章 Architecture——设计边界与依赖的六条原则	数据结构、在边界验证外部数据（边界如表单输入或 API 响应等数据进入的位置）、类型、幂等性（同一操作反复执行也得到相同结果的性质）、迁移旧 API、共享状态
第 19 章 Verification——用实际行为验证的四条原则	用真实输出确认完成、修复缺陷根因、分小步验证并推进、测试行为
第 20 章 Delegation——处理委派与并行执行的两条原则	保护 Agent 的上下文，不等待人的确认而推进
第 21 章 Meta——改进机制本身的原则	将教训固化为机制，而非只留在文字中

本部分按 pstack 随附指南的顺序，将 Core 放在开头。

全部 Principle 速查表

Principle	概要	触发条件	详解
Laziness Protocol	优先删除代码，选择解决问题所需的最小改动	Agent 估计重构或差异规模，或想增加抽象或层次、让新值穿过多层传递时	第 17 章
Subtract Before You Add	添加功能前，先删除未使用代码、重复验证等复杂内容	Agent 决定以什么顺序进行功能或处理的添加、重构或重写时	第 17 章
Minimize Reader Load	减少读者须追踪的间接引用层次，以及必须记住的代码状态	Agent 审阅难以追踪处理流程的代码，或编写、重组这类代码时	第 17 章
Foundational Thinking	编写逻辑前，先确定数据结构和基础设施（CI、lint、测试基础设施、共享类型等）	Agent 在写逻辑前选择核心类型或数据结构、决定先做基础设施还是功能，或思考并发的多个主体应共享什么时	第 17 章

Principle	概要	触发条件	详解
Redesign from First Principles	不把新需求补丁式地附加到旧设计，而按新需求从一开始就存在来重新设计	Agent 将新需求纳入现有设计时	第 17 章
Attack the Premise	不要继续叠加基于相同前提的修复；写下一次修复前，先质疑该前提本身	共享同一前提的两次或更多修复，在同一 gate（必须通过的测试或检查）失败时	第 17 章
Outcome-Oriented Execution	优先选择可验证的最终形式，而不是为迁移中途保持一致所加的临时代码	Agent 推进事先确定阶段边界的计划性重写或迁移时	第 17 章
Experience First	优先用户体验而非实现便利，少做功能并把它做好	Agent 在产品、UX 或功能范围上选择一项就必须放弃另一项时	第 17 章
Exhaust the Design Space	制作形态各异的两到三个原型，并排比较后再决定设计	Agent 创建代码库中没有先例的 UI 交互、选择有多种可行实现的架构，或判断主要凭实际感受而非理论决定的产品设计时	第 17 章
Build the Lever	用工具代替手工操作：制作执行工作的工具，或证明工作正确的工具（codemod、脚本、生成器、供子 Agent 遵循的 Skill）	Agent 处理编辑、迁移、分析或检查等工作，工作量超过几处一眼可见的修改时	第 17 章
Model the Domain	用结构（状态机、带类型的对象、查找表、reducer 等）表达领域，而非散落的条件分支	Agent 编写有状态逻辑、分支繁多的代码，或在多个文件反复写下「这些数据有这个字段」的相同前提时	第 18 章
Boundary Discipline	将验证与错误处理集中在外部数据进入的边界；边界内使用已带类型的值，无须反复验证，并让业务逻辑保持为纯函数	Agent 加入输入验证或错误处理，或编写连接框架与自身代码的适配器时	第 18 章
Type System Discipline	利用类型检查，让不可能的状态在编译阶段就无法构造	Agent 设计类型、审阅函数签名，或在静态类型语言中编写代码时	第 18 章

Principle	概要	触发条件	详解
Make Operations Idempotent	让改变状态的操作无论执行多少次，或从中途重新执行，最终都到达同一状态	Agent 设计可能中途停止、重启或重试的命令、启动或关闭步骤、重复处理循环时	第 18 章
Migrate Callers Then Delete Legacy APIs	先将调用方迁至新 API，并在同一改动中删除旧 API	Agent 在旧调用方仍存在时引入新的内部 API 时	第 18 章
Separate Before Serializing Shared State	先划分多个主体共享的写入对象；实在无法划分时，再用锁等方式串行写入	并行的多个主体可能写入同一文件、分支、键或有状态对象时	第 18 章
Prove It Works	直接运行功能并观察实际数值，验证工作成果，而非依赖 build 成功等代理指标	Agent 完成工作、准备宣称完成时	第 19 章
Fix Root Causes	不增加只会掩盖缺陷症状的防护判断，而要找到并修复症状的根本原因	Agent 调试缺陷时	第 19 章
Sequence Work into Verifiable Units	将工作分成可验证的小单元，每单元的检查（如测试）通过后再继续，并按展示正确性的顺序排列提交	Agent 执行多阶段工作（批量修复、迁移、连续的类似编辑），或决定提交与 PR 的排列顺序时	第 19 章
Test Behavior, Not Implementation	以代码使用方相同的方式调用代码，并把可观察结果与直接写在测试中的期望值比较	Agent 编写或修改测试，或决定是否保留测试时	第 19 章
Guard the Context Window	让子 Agent 阅读大量输出或文档，主线程只保留摘要	大量输出、长文件或重复读取同一文件填满 Agent 上下文，或 Agent 计划将工作分给多个子 Agent 并行推进时	第 20 章
Never Block on the Human	可撤销的工作无需等待人确认，先完成并展示结果；只有执行不可逆操作前才向人确认	Agent 想就可撤销工作问人「可以做 X 吗」时	第 20 章

Principle	概要	触发条件	详解
Encode Lessons in Structure	将反复需要的修正固化为机制（lint 规则、元数据标志、运行时检查或脚本），而非仅写成文字指令	Agent 发现自己第二次写同一指令，或同一纠正反复出现时	第 21 章

第 17 章：Core，决定工作方式的十条原则

kaito · 来源：日本語 · English · 2026-10-03

本章介绍以下 10 条 Principle。

1. Laziness Protocol
2. Subtract Before You Add
3. Minimize Reader Load
4. Foundational Thinking
5. Redesign from First Principles
6. Attack the Premise
7. Outcome-Oriented Execution
8. Experience First
9. Exhaust the Design Space
10. Build the Lever

这 10 条属于 Core（核心）组。随附指南的 [docs/guide/08-principles.md](#) 将它们描述为决定「**做多少，以及何时重新审视设计**」的原则。例如，Agent 用这些原则判断是否应缩小差异、在添加功能前削减复杂性，或在基于同一前提的修复两次失败后质疑该前提。

如果只是逐条列出这 10 条 Principle，就不容易看出每条适用于什么判断。本章按判断对象把它们分为四组，并依次介绍。

- **1~3**……减少代码的原则
- **4~7**……决定如何安排设计（设计基础与重新审视的时机）的原则
- **8 和 9**……决定追求什么（用户体验，以及在多个方案中选择目标形态）的原则
- **10**……把手工工作变成工具的原则

本章按上述顺序，逐条从以下角度介绍 10 条 Principle。

- **规则**……该 Principle 要求做什么
- **触发条件**……该 Principle 适用于什么情境

本章结构

本章结构如下。

- 阅读 Principle 前应了解的三个前提
- 10 条 Principle 一览
- 「Laziness Protocol」：以最小改动取得最大成果
- 「Subtract Before You Add」：添加功能前先削减复杂性
- 「Minimize Reader Load」：减少读者须追踪的间接引用层次与须记住的代码状态
- 「Foundational Thinking」：编写逻辑前先确定数据结构和基础设施
- 「Redesign from First Principles」：按新需求从一开始就存在来重新设计
- 「Attack the Premise」：基于同一前提的修复两次在同一测试失败后，质疑该前提
- 「Outcome-Oriented Execution」：优先实现可验证的最终形态，而非为维持迁移途中的一致性加入临时代码
- 「Experience First」：选择用户体验而非实现便利
- 「Exhaust the Design Space」：没有先例的设计先做两三个方案比较
- 「Build the Lever」：制作执行工作的工具，而非手工操作
- 总结

阅读 Principle 前应了解的三个前提

第 8 章和第 9 章也介绍了 Principle 的运作方式。阅读本章时，只需掌握以下三个前提。

1. Principle 决定是否纳入当前改动，以及工作是否尚未完成

Principle 本身是判断标准，而非步骤或能力。不过，「Build the Lever」和「**Prove It Works**」等 Principle 在应用时也会要求编写脚本。

2. 触发条件写在 /poteto-mode 的 Principle 列表和每条 Principle 的开头

本书将原则适用的情境称为触发条件。本章的触发条件，译自 /poteto-mode 的

「Principles」列表中各条目的「何时适用」，以及各条 Principle 的 SKILL.md 开头（description）所述的适用情境。

3. 即使请求中没有写 Principle 名称，Agent 也会使用它

只要工作符合触发条件，Agent 就会自行阅读并应用该 Principle。不过，Agent 只在开始多步骤工作时阅读索引，而且是否适用也由 Agent 自己判断，因此可能漏掉适用的情境。如果 Agent 遗漏（或跳过）了某条 Principle，人可以在请求中明确写出 Principle 名称，让 Agent 应用它并调整工作方向。

本章只给 10 条 Principle 中的「Subtract Before You Add」「Exhaust the Design Space」「Build the Lever」这三条提供请求示例。

本书只收录 pstack 随附指南或 README 中已有的请求示例。如果为原文没有示例的 Principle 自行编写示例，可能会展示 pstack 并未预期的用法。

10 条 Principle 一览

分组	Principle	一句话结论
减少代码	「Laziness Protocol」	用最少的代码与复杂性取得最大的成果
减少代码	「Subtract Before You Add」	添加功能前先削减复杂性
减少代码	「Minimize Reader Load」	减少读者须追踪的间接引用层次与须记住的代码状态
设计安排	「Foundational Thinking」	编写逻辑前先确定数据结构和基础设施
设计安排	「Redesign from First Principles」	按新需求从一开始就存在来重新设计
设计安排	「Attack the Premise」	基于同一前提的修复两次在同一测试失败后，质疑该前提
设计安排	「Outcome-Oriented Execution」	优先实现可验证的最终形态，而非为维持迁移途中的一致性加入临时代码
追求的目标	「Experience First」	选择用户体验而非实现便利

分组	Principle	一句话结论
追求的目标	「Exhaust the Design Space」	没有先例的设计先做两三个方案比较
转化为工具	「Build the Lever」	制作执行工作或证明工作正确的工具，而非手工操作

「Laziness Protocol」以最小改动取得最大成果

「Laziness Protocol」要求以最少的代码与复杂性取得最大的成果。它判断解决方案好坏的唯一标准是：人类开发者是否会因维护这些代码而疲惫不堪。原文明确写道：「如果维护这些代码令人疲惫不堪，那就是糟糕的解决方案」。

规则：删除代码、减少调用层次、集中作出判断

主要规则如下。

先寻找可以删除的内容

接到改进请求时，先寻找可以删除的内容，再考虑添加代码。

减少调用层次

如果为了找到代码中的目标，需要查看四个或更多文件，或追踪四层或更多处理，就应去掉中间层，缩短路径。必须追踪多层才能得到答案的代码，会让维护者疲惫。

在一处集中作出判断

不要在多个位置重复判断同一条件；只在一处判断。

例如，先判断输入框是否为空，再把结果用于「按钮启用或禁用」和「提示信息显示」等多个位置。修改条件时，只需修改这一处。相同判断散落多处，就得一直花力气保持它们一致。

缩小差异

相比精巧的样板代码，应选择行数更少的实现。

让新值穿过多层之前先停下来

如果请求要求让一个新值穿过类型、schema、pipeline 等多个层次，先停下来寻找更直接的做法。

例如，为仅向付费会员显示按钮而新增功能时，不要立刻把「是否为付费会员」这个新值加到 API 类型、转换流程和从父组件传给子组件的属性等多个层次。先看看能否直接用按钮附近已有的用户信息判断。

触发条件

- 重构或估算差异规模时
- 想增加抽象或层次时
- 想让新值穿过多个层次传递时

「Subtract Before You Add」在添加功能前削减复杂性

「Subtract Before You Add」要求扩展系统时先去除复杂性，再构建新内容。在复杂系统中添加功能会进一步堆积复杂性。先削减复杂性，代码会减少，系统的基本结构也会显现，下一步如何设计通常也会更清楚。

规则：构建前先去除复杂性

- 构建前先去除复杂性；打磨品质前，先缩减到最小范围
- 不增加对必要性尚未证实的边缘情况的处理，而根据实际观察到的用法设计
- 验证器与防护措施只覆盖规格要求的范围，不凭猜测添加
- 即使在提示词或 Skill 文本中，也要删除重复指令

触发条件

决定添加功能或处理、重构、重写的先后顺序时。

请求示例：删除旧适配器后再设计

以下示例来自随附指南：Agent 正要在现有三个适配器之外添加第四个适配器。

```
use subtract before you add. delete the obsolete adapters first, then design what's left.
// 使用 subtract before you add. 先删除过时的适配器，再设计剩余部分。
```

适配器……吸收外部服务之间的差异，使它们能以相同方式调用的组件。

例如，A 公司、B 公司、C 公司的支付服务各有不同的调用方式；为每家公司准备适配器后，应用程序都能以相同写法 `pay(金额)` 调用。

「Minimize Reader Load」减少读者须追踪的间接引用层次与须记住的代码状态

「Minimize Reader Load」将可维护性定义为「读者理解代码所需付出的工作量」。代码被阅读的次数远多于被编写的次数。

这条 Principle 用「须追踪的层次」和「须记住的状态」两个维度衡量读者负担。

「须追踪的层次」指读者从产生疑问到找到答案，需要经过多少层函数或文件。

「须记住的状态」指读者为理解代码行为，必须在脑中记住的隐藏状态或可变值，也就是需要持续追踪「哪个值」在「何处」会「改变」所造成的负担。

这两个维度分别造成不同类型的阅读困难。

例如，即使只有一个文件，无须追踪多层函数，如果有 50 个可从任何位置修改的全局变量，读者仍难以弄清某个值何时何处改变。反过来，即使变量不多，如果要经过六层适配器（承担转换的层次）才能找到答案，代码同样难读。

因此，这条 Principle 要求同时减少两种负担，而非只处理其中一种。

「Minimize Reader Load」相当于第 20 章「Guard the Context Window」（避免大量输出占满 Agent 上下文窗口）的面向人类版本。一次能记住的内容（工作记忆）有限，这对 Agent 和阅读代码的人都一样。

规则：减少层次与状态

去掉只负责转交调用的层次

如果某个函数只调用另一个函数，而且只在一处被使用，就去掉这个函数，让调用方直接调用原本被它调用的函数。

例如，如果 `saveUser()` 只调用 `db.save()`，就直接调用 `db.save()`。

```
type User = { name: string };
declare const db: { save(user: User): Promise<void> };
declare const user: User;

// 修改前: saveUser 只调用 db.save, 而且只有这里用到它
function saveUser(user: User) {
  return db.save(user);
}
await saveUser(user);

// 修改后: 删除 saveUser, 直接调用 db.save
await db.save(user);
```

如果调用方与实际处理之间的中间层没有任何可替换的实现，也应同样去掉；为将来扩展而预先加入的层次也包括在内。

例如，只使用一家支付服务，却仍加入 `PaymentGateway` 这样的可替换接口。这样的层次只会增加读者「打开文件查看里面做了什么」的工作。

```
declare const aPayClient: { charge(amount: number): Promise<void> };

// 修改前: 多了一层可替换接口, 但只有 A 公司的一种实现
interface PaymentGateway {
  pay(amount: number): Promise<void>;
}

class APayPaymentGateway implements PaymentGateway {
  pay(amount: number) {
    return aPayClient.charge(amount);
  }
}

const gateway: PaymentGateway = new APayPaymentGateway();
await gateway.pay(1000);
```

```
// 修改后：去掉接口层，直接调用 A 公司的支付实现
await aPayClient.charge(1000);
```

缩小状态的作用范围

缩小值可以被修改的范围。例如，相比可从任何地方修改的全局变量，应选择只在一个函数中使用的局部变量。

也不要将从其他值计算出来的值另存为变量。

例如，除了商品列表再存一个「商品数量」变量，每次修改列表都必须同步更新数量。原有三件商品，添加一件后忘记更新数量，就会出现列表有四件、数量仍是三的情况。应在需要时从列表计算数量。

状态的作用范围越小，读者需要记住的代码状态就越少。

在边界只验证一次不变量，并为它命名

不变量指「金额大于或等于零」等程序运行期间始终必须满足的条件。这条 Principle 要求只在边界验证一次，而非在每个使用位置重复验证。

这里的**边界**，指表单输入或 API 响应等外部数据进入的位置。

例如，接收订单表单金额的函数，只检查一次「金额是否大于或等于零」，再把已验证的值作为名为 `NonNegativeAmount`（非负金额）的类型传递。

使用金额的函数（如计算总额或创建发票的函数）接收 `NonNegativeAmount` 类型，无须重复检查相同条件。

只看类型名就能知道「这个金额已在边界验证为非负」。这样，**读者只需阅读边界的一处和类型名，就能一次记住该条件。**

```
// 表示已经验证不小于 0 的金额类型
type NonNegativeAmount = number & { readonly __brand: "NonNegativeAmount" };

// 边界：在这里对订单表单传来的金额只验证一次
function parseAmount(input: number): NonNegativeAmount {
  if (input < 0) {
    throw new Error("金額は0以上で入力してください");
  }
}
```

```

    return input as NonNegativeAmount; // 刚完成验证，所以只在这里赋予该类型
}

// 使用方接收 NonNegativeAmount，无须再次验证是否不小于 0
function calcTotal(price: NonNegativeAmount, quantity: number): number {
    return price * quantity;
}

function createInvoice(amount: NonNegativeAmount): string {
    return `ご請求金額: ${amount}円`;
}

```

Oh My Stack 项目注（代码示例）：异常消息要求金额不小于 0；账单模板输出「应付金额：若干日元」。这些是函数的原始输出值，故保留。

calcTotal 和 createInvoice 中没有 if (amount < 0) 这样的检查。未经过 parseAmount 的数值在传入时会产生编译错误，因此使用方无须再次验证。

用 30 秒测试判断

这条 Principle 用「30 秒测试」判断层次或状态是否过多。如果新读者无法在 30 秒内回答「X 从哪里来」「什么可能改变 X」，就应减少层次或状态。

触发条件

审阅处理流程难以追踪的代码，或编写、重组这类代码时。

「Foundational Thinking」在编写逻辑前确定数据结构和基础设施

「Foundational Thinking」要求编写逻辑前先确定数据的形态。

这条 Principle 将判断分为以下两类。

1. 结构判断（决定数据形态和工作顺序）
2. 代码写法判断

结构判断保留未来的选择余地。这里的结构判断指「决定数据形态或工作顺序」，未来的选择余地指「之后仍可选择其他路线的空间」。

代码写法判断则保持代码简洁。只要正确选择数据结构，后续代码需要写什么，通常也会自然明确。

poteto 在《The Complete Guide to pstack》[Part 2](#) 中提出这样的分工：**Agent 时代的工程师把时间用于选择架构和数据结构，Agent 则补齐实现细节**。这条 Principle 将选择数据结构的判断放在 Agent 工作的开头。

规则：确定数据结构、拆分并发状态、先建基础设施再叠加功能

先确定数据结构

编写逻辑前，先确定核心数据的类型。接着梳理这些数据将在何处、如何被读写，并选择适合最常见操作的形态。

例如，制作商品列表功能时，先确定「商品包含名称、价格、库存数量」，之后再考虑如何展示或加入购物车。确定数据形态后，使用它的处理逻辑就容易编写。

```
// 先确定结构：商品有名称、价格和库存数量
type Product = {
  name: string;
  price: number;
  stock: number;
};

// 结构确定后，显示商品和加入购物车的代码就能明确编写
function formatProduct(product: Product): string {
  return `${product.name} ${product.price}円 (在庫 ${product.stock})`;
}

function canAddToCart(product: Product): boolean {
  return product.stock > 0;
}
```

Oh My Stack 项目注（代码示例）：formatProduct 的原始模板输出为「商品名、日元价格（库存数量）」；其中「円」表示日元，「在庫」表示库存。保留该函数的实际输出值。

在代码中消除结构重复，而非仅仅消除相似的代码行

不要每见到几行相似代码就提取公共函数，应优先消除数据形态等结构上的重复。

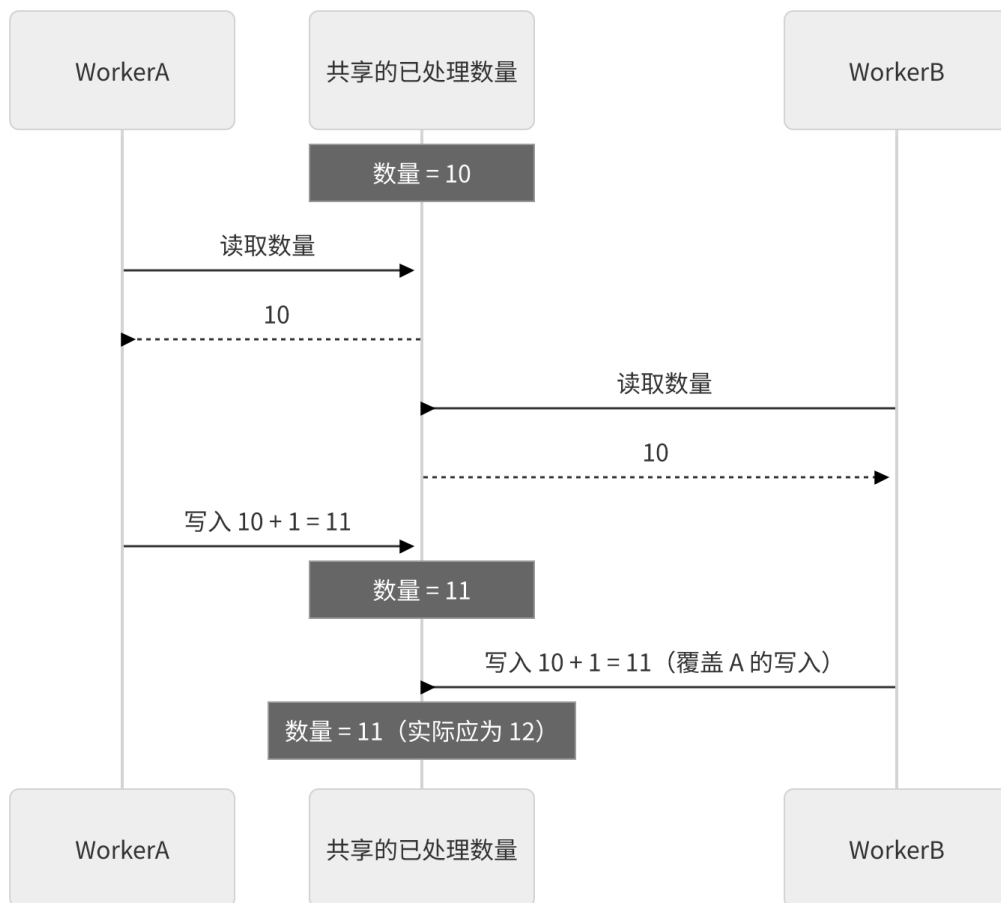
这条 Principle 认为，只有三段相似语句时，不必急于抽取共用逻辑，直接保留往往更好。比起精巧的写法，应选择一眼能懂的写法。所谓精巧，是指虽然写得短，但读者无法立刻理解的写法。例如，判断数字是否为奇数，与其使用 $n \& 1$ 这样的位运算，不如写 $n \% 2 === 1$ 。

过早抽象或过于精巧的写法，都背离保持代码简洁的目标。

将可能并发修改的值分开，而非共享

多个主体（同时运行的 Worker 或 Agent）共享同一个值之前，先思考「**其他主体同时修改这个值会发生什么**」。如果会引发问题，就不要共享该值，而是让每个主体持有自己的值。

例如，Worker A 和 B 都把共享的「已处理数量」加一。若数量原为 10，而 A、B 几乎同时运行，就会出现以下过程。



查看图示 1

两个 Worker 各处理了一件工作，本应得到 12；但 B 根据 A 写入前读到的 10 写入 11，结果计数仍是 11，丢失了一次更新。

因此，每个 Worker 应各自保存处理数量，只在需要总数时相加。值分属不同主体后，就不会再被其他主体修改。

```

type Job = { id: string };
declare const store: {
  get(key: string): Promise<number>;
  set(key: string, value: number): Promise<void>;
};
declare function processJob(job: Job): Promise<void>;
declare const jobsForA: Job[];
declare const jobsForB: Job[];
  
```

```
// 修改前：所有 Worker 共享一个「已处理数量」，先读取再写入
async function countUp() {
  // A 和 B 几乎同时读取到 10
  const current = await store.get("processedCount");
  // 两者都写入 11，导致少算一项
  await store.set("processedCount", current + 1);
}

// 修改后：每个 Worker 保存自己的计数，只在需要总数时求和
async function runWorker(jobs: Job[]): Promise<number> {
  let processedCount = 0; // 这个 Worker 专用的计数

  for (const job of jobs) {
    await processJob(job);
    processedCount += 1;
  }

  return processedCount;
}

const counts = await Promise.all([runWorker(jobsForA), runWorker(jobsForB)]);
const total = counts[0] + counts[1];
```

先建基础设施

问自己：「后续每个阶段都会因它存在而受益吗？」如果答案是「是」，就先建设它。CI、lint、测试基础设施和共享类型都属于这类基础。此 Principle 称其为 **scaffold**（字面意为「脚手架」）。

功能前先做准备，修复前先建测试。提交也应小而目的单一。有了基础设施，后续所有工作都能使用它。

不过，删除未使用代码的优先级高于建设基础设施。

如果保留未使用代码，先建基础设施，共享类型和测试可能也会为这些无用代码提供支持。先删除它们，才能看清实际使用的部分，为之建设合适的基础设施。这与本章的「[Subtract Before You Add](#)」思路相同。

此外，如第 4 章所述，代码库也是 Agent 的记忆。因此，本书认为，优先删除未使用代码也有助于避免留下错误的上下文。

每做一项改动，都留下一个完整机制

每做一项改动，都创建一个完整机制（可供多处使用的共用函数或组件等），或让已有的共用函数或组件更易使用。

添加新功能时，不要在每个使用位置散布特例处理。例如，为商品列表增加排序功能，不应在每个展示列表的页面各写一套排序逻辑，而应制作一个负责排序的组件，由各页面调用。

新功能集中在一处，阅读和修改的人也只需查看这一处。

触发条件

- 编写逻辑前（选择核心类型或数据结构时）
- 决定先建基础设施（CI、lint、测试基础设施和共享类型等后续工作都会使用的机制）还是先做功能时
- 思考并发运行的多个**主体**应共享什么时

这里的**主体**，指同时运行并推进处理的实体。

例如，从队列取出任务并行处理的多个 Worker（负责执行处理的程序），或同时在同一仓库工作的多个 Agent，都属于主体。

「Redesign from First Principles」按新需求从一开始就存在来重新设计

「Redesign from First Principles」要求不要把变更后加到现有设计中，而要**按这项需求从一开始就存在来重新设计**。本书认为，如果「**Foundational Thinking**」适用于「新建事物」，那么「Redesign from First Principles」适用于「将变更融入现有设计」。

规则：用四个步骤重新设计

1. 阅读所有受影响的文件，理解现有设计
2. 问自己：「如果从一开始就知道这项新需求，要从零编写，会怎样设计？」
3. 将相关类型、文档、示例和说明设计理由的章节等所有位置都调整到新设计
4. 先构思重新设计后的整体，再把改动分成小步逐步实施

例如，为原本只处理日元的价格计算增加美元支付，可以这样思考。

- 后加功能时，在每个使用金额的位置加入「如果是美元就这样计算」的分支
- 重新设计时，先问「如果从一开始就支持多币种，会怎样设计」，再把金额改为「数值与币种」的组合

最后，为体现重新设计的结果，所有使用金额的位置、类型和文档都应改用这一形态。

```
// 事后补丁：金额仍只是以日元表示的数字，每个使用处都加上「如果是美元」的分支
function formatPrice(amount: number, isDollar: boolean): string {
  if (isDollar) {
    return `$$${amount.toFixed(2)}`;
  }

  return `${amount}円`;
}
```

Oh My Stack 项目注（代码示例）：返回值末尾的「円」表示日元，是原示例的实际显示值。

```
// 重新设计：金额表示为「数值与币种」的组合，所有使用处都接收这种类型
type Currency = "JPY" | "USD";
type Money = { amount: number; currency: Currency };

function formatPrice(price: Money): string {
  if (price.currency === "USD") {
    return `$$${price.amount.toFixed(2)}`;
  }

  return `${price.amount}円`;
}
```

Oh My Stack 项目注（代码示例）：返回值末尾的「円」表示日元，是原示例的实际显示值。

这条 Principle 将上述四步视为**把变更融入现有设计，同时保留未来选择余地**的方法。如果只是在原设计上追加，新加部分会一直与原设计不一致。之后的修改只能绕开这种不一致，未来可选的设计路线也会减少。

触发条件

将新需求融入现有设计时。

「Attack the Premise」：基于同一前提的修复两次在同一测试失败后，质疑该前提

「Attack the Premise」规定，如果基于同一前提的两次或更多修复在同一道 gate 失败，就应质疑前提，而非继续修改。

- **gate**……意为「关卡」，指修复必须通过的测试或检查。
- **前提**……编写修复的人或 Agent 理所当然地认为「原因就在这里」的想法。

例如，Agent 认为「处理速度慢是因为数据量过大」，第一次和第二次都尝试减少每批处理的数据量，但两次都在同一测试失败。

如果数据量是原因，减少数据量应使测试通过。两次测试都失败，说明速度慢可能另有原因。因此，写第三次修复前，Agent 应质疑「数据量过大」这个想法本身。每次基于相同前提的修复失败，失败本身即为判断该前提提供了证据，Agent 可以据此反思前提。

「Redesign from First Principles」围绕新需求重组设计，而这条 Principle 质疑现有设计视为「理所当然」的前提。

规则：修改前写出前提，并统计偏斜分布

写出前提

用一句话写出所有失败修复共同视为理所当然的前提，以明确需要质疑的对象。

下一次修复前先做 census 统计

census（字面意为「人口普查」）是按主体（同时运行的 Worker 或 Agent）统计偏斜情况的明细。

例如，四个 Worker 分工处理任务时，分别统计各自未处理的数量，得到「Worker 1 有 30 件，Worker 2~4 各有 2 件」这样的明细。census 要说明的不是偏斜有多大，而是偏斜集中在哪个主体（本例为 Worker 1）。

Agent 应将 census 统计写成可反复运行的脚本。这符合本章最后介绍的「Build the Lever」原则。

解读偏斜模式

如果每次运行都偏向同样少数几个主体，说明某处设置或代码反复把导致偏斜的角色分配给它们。Agent 应找出这一分配机制。

这个分配机制，就是接下来要追问的「为什么」。

这也是「Fix Root Causes」（不断追问「为什么」，修复根因而非症状的 Principle，见第 19 章）的思路。

消除造成偏斜的分配方式，而非事后补救

采用以下任一方式，避免同一主体每次都承担该角色。

- 在主体之间轮换角色
- 随机分配
- 将角色移到其他位置

例如，如果第一批任务总是交给第一个 Worker，就每次更换接收者。

即使增加事后补救机制，造成偏斜的分配方式仍在，而且每次运行都会增加额外工作（「Laziness Protocol」）。事后补救机制包括：空闲 Worker 去领取其他 Worker 的任务、所有 Worker 共用的任务池、成批转交任务或定期重新分配等。

在写出前提并完成 census 之前，不开始下一次修复

Agent 在写出前提、完成 census 前，不开始下一次修复。如果 census 显示各主体分布均匀，说明这一前提不是原因。此时 Agent 应到别处寻找原因，并保留 census 作为证据。

例如，多个 Worker 处理队列，两次未通过「在时间上限内清空队列」的测试。

首先，Agent 在两次修复中分别加入以下处理。

- 第一次……加入空闲 Worker 领取其他 Worker 任务的机制
- 第二次……加入定期重新分配任务的机制

两次修复都基于同一前提：「任务堆积在少数 Worker 上，是因为各任务耗时不同」。

接着，在编写第三次修复前，Agent 写脚本，从运行日志中统计每个 Worker 的未处理数量。

如果任务耗时差异是原因，积压任务的 Worker 应随每次运行而变化。

但如果三次运行中始终只有 Worker 1 的未处理数量很高，说明原前提不成立。下一个要问的「为什么」是：「什么把这个角色分配给了 Worker 1？」

可能的原因之一是：某项设置总让 Worker 1 接收第一批任务。

触发条件

基于同一前提的两次或更多修复，在同一道 gate 失败时。

「Outcome-Oriented Execution」 优先实现可验证的最终形态，而非为迁移途中保持一致加入临时代码

「Outcome-Oriented Execution」规定，相比让迁移中的代码在每一步都顺畅运行，应优先到达最终目标形态，并验证它正确。如果要求每个中间阶段都能运行，就可能产生兼容旧形式和新形式的临时代码；这些代码若未删除，容易长期留存并成为负担。

例如，将配置文件从旧格式迁到新格式，为使迁移的每一步都能读取两种格式，可能编写把旧格式转换成新格式的过渡代码。

迁移结束后，旧格式不再使用，这段过渡代码本该删除，却容易因遗忘或担心仍有旧格式残留而留下来。遵循这条 Principle 的 Agent 应朝目标形态推进，而不是靠这些代码维持中途一致性，并在预定的验证边界（事先划定的正确性检查节点）证明结果正确。

规则：事先划定可暂时失效的范围，最后全面验证

- 只有当暂时无法运行符合计划、范围有限且可恢复时，才允许这样做。

- 事先声明哪里可以暂时失效。例如，在计划中写明「迁移期间，管理界面的搜索可以暂时不可用」。
- 即使在迁移途中，也要在当前修改的位置保留能迅速发现问题的检查。
- 计划结束时，必须完整通过静态检查（如类型检查）和实际运行验证。

这条 Principle 并非无条件允许中途失效，而是要求**预先在计划中写明可暂时失效的范围**。

触发条件

执行已预先划定阶段边界的计划性重写或迁移时。Agent 不应把它用于没有阶段边界的日常小修复。

阶段边界（原文称 **phase boundaries**）指大型工作分成若干阶段后，各阶段之间的分界点。

例如，迁移配置文件格式时，可按以下方式分阶段，并在每个阶段结束时验证是否正常运行。

1. 支持读取新格式
2. 将所有配置文件改写为新格式
3. 删除读取旧格式的逻辑

「Experience First」选择用户体验而非实现便利

「Experience First」规定：**如果对开发者更省事的做法与提升用户体验的做法冲突，应选择提升用户体验**。三个打磨完善的功能胜过十个粗糙功能。此外，设计决策用一次性 HTML 验证，比直接在生产代码中试验成本更低。

规则：少做功能，并把它做好

为每项功能、操作和选项找到理由

对于界面中的每个按钮和设置选项，都应能说明它为什么必要。

少做功能，并把它做好

与其凑齐十个粗糙功能，不如做好三个功能。

决定前先制作原型

编写生产代码前，先用一次性 HTML 等制作原型，亲手操作、实际验证后，再决定设计方向。相比在生产代码中返工，在原型阶段发现错误成本更低。

做好细节

正确处理页面切换、元素对齐、留白、操作反馈和错误显示。

例如，保存失败时应显示发生了什么，而非毫无提示。

以用户的主要工作流程为中心

每项功能都应帮助用户完成主要工作流程，或者至少不妨碍它。

从用户视角说明影响

这里的用户包括所有使用成果的人。对界面而言，是应用用户；对库或内部 API 而言，是导入它们的同事；下一位维护代码的工程师也包括在内。

例如，不要只说「修改了 API 响应格式」，而要说明「列表页面比以前显示得更快」等用户实际感受到的变化。

这条 Principle 区分了两条原则的职责：「**Foundational Thinking**」决定工作顺序，「**Experience First**」决定目标形态。同时，它认为 CI、测试基础设施和共享类型等基础设施，最终也是为了提升用户体验。

触发条件

在产品、UX 或功能范围上，必须选择一方并放弃另一方的取舍出现时。

「Exhaust the Design Space」：没有先例的设计先做两三个方案比较

「Exhaust the Design Space」要求：面对代码库中没有先例的交互或设计决策，实现前先尝试几个具体的替代方案。因为做错东西的代价，比试验三个方案的代价更高。

规则：并排比较两三个形态不同的方案

对于答案不明显的设计，Agent 应制作两三个彼此竞争、形态不同的方案（原文称 competing prototypes），比较后再决定采用哪一个。方案可以是可实际操作的原型，也可以是草稿。这条规则也称为「Design it twice（设计两次）」。

关键在于**方案之间的形态必须有根本差异**。只对第一个方案做些微调整，不算第二个方案。只有比较形态不同的方案，才能知道哪一种更适合用户。

例如，设计注册页面时，下面两种方案的形态不同，可以作为两个方案。

案A：1ページにすべての項目を並べる

会員登録

名前

メールアドレス

パスワード

登録

すべての項目を一度に見渡して入力できる

案B：1項目ずつ、画面を切り替えて順番に聞く

会員登録 1 / 3

お名前を入力してください

次へ

会員登録 2 / 3

メールアドレスを入力してください

次へ

会員登録 3 / 3

パスワードを入力してください

登録

1画面に1項目だけなので、迷わずに進められる

相反，只改变方案 A 的按钮颜色或位置，页面形态仍与方案 A 相同，不能算另一个方案。

触发条件

- 制作代码库尚无类似先例的新 UI 交互时
- 选择有多种可行实现方式的架构时
- 判断主要取决于使用感受、而非逻辑推理的产品设计时

相反，这条 Principle 不适用于以下变更。

- 做法已确立的机械式实现
- 目标状态明确的缺陷修复或重构
- 受约束限制、只有一种可行方式的变更

请求示例：制作两个原型并比较

以下示例来自 pstack 的 README。此请求会让 Agent 进入「**Prototype**」Playbook，因此制作原型时就以之后会丢弃为前提。

```
/poteto-mode build two prototypes of the markdown renderer so we can compare. spawn an agent for each.  
// 制作两个 Markdown 渲染器原型以供比较，各安排一个 Agent。
```

「Build the Lever」：制作执行工作的工具，而非手工操作

「Build the Lever」（字面意为「制作杠杆」）规定：如果工作并不简单，就应避免手工操作，**制作执行这项工作或证明工作正确的工具。**

制作工具有两个好处。

第一是处理量。codemod（机械式修改代码的脚本）和其他脚本无论运行多少次，每次都按同样方式处理。因此，即使目标有一百个文件，Agent 也无须像手工操作那样逐个花时间处理。

如果中途调整处理方式，或目标文件增加，Agent 只需重新运行脚本，就能以相同方式重新处理所有文件。

第二是准确性。审阅者可以阅读工具的代码并实际重新运行，亲自验证工作是否正确。

手工修改则不会留下改写过程。因此，审阅者若要重新确认正确性，只能自己重做一遍。若一百个文件都是手工改写的，审阅者也得逐个检查一百个文件，既费时间，也容易遗漏。

这条 Principle 用「确定性的脚本把『相信我』变成『运行它』」概括第二个好处。

- **确定性的脚本**……对相同输入始终产生相同结果的脚本。
- **把「相信我」变成「运行它」**……手工工作的结果只能请审阅者相信；有脚本，审阅者就能亲自重新运行并核对结果。

规则：先手工完成第一次、摸清步骤，再制作工具

第一次先手工完成，再制作工具

例如，要在一百个文件中把旧函数名改为新函数名，先手工改写一个文件，确认步骤，再编写改写脚本。

然后，在第一个文件上运行脚本，将输出与手工改写结果比较，证明脚本正确。工具应保证可安全重复运行。

选择适合任务的工具

编辑用 codemod 或脚本；大量重复文件用生成器；分析时把数据导出到 SQLite 并用查询检查；验证则用可反复运行的脚本。

脚本能做的工作，就由脚本完成，不要拆给多个子 Agent

如果脚本一次就能处理所有单元，Agent 应自行运行。相比让子 Agent 逐个手工处理，每次都产生相同结果的脚本更可靠。

委派给子 Agent 时，把操作说明写成 Skill

需要把工作分给子 Agent 时，应编写供所有子 Agent 阅读的 Skill，将工作步骤、验证方法与禁止触碰的范围集中写在其中。

把这份 Skill 放在子 Agent 无法修改的位置，防止它们悄悄改动既定的步骤或验证方法。

制作最小脚本，而非框架

只制作能完成工作或证明结果正确的最小脚本（「**Laziness Protocol**」）。

工作跨越多个会话时提交工具

将工具保存下来，使下一次会话仍能使用。

从首次手工操作到工具处理全部目标的流程，如下图所示。

```
手动完成第一项，了解步骤
↓
制作工具（codemod、脚本等）
↓
用工具重新执行第一项，并将输出与手动完成的版本比较
↓ 如果结果一致
自己运行工具，处理剩余的所有单元
（不要让子 Agent 逐个手动处理）
```

Agent 应用这条 Principle 时，差异中会增加工具文件。如果 Agent 引用了这条 Principle，差异中却没有 codemod、脚本、生成器或委派用 Skill，那么 Agent 并未实际应用它。

还有两条 Principle 同样涉及制作工具或脚本。为避免混淆，下表说明各自处理的对象。

原则	对象
「 Build the Lever 」	使眼前的工作可批量处理且易于验证
「 Encode Lessons in Structure 」（第 21 章）	把反复出现的指令固化到 lint 规则或运行时检查等机制中，成为长期有效的防护措施
「 Prove It Works 」（第 19 章）	宣称完成前，实际运行并验证

触发条件

所有不简单的工作，无论是编辑、迁移、分析还是检查。

只有少数几处一眼就能确认的简单修改，Agent 才无须制作工具。

制作工具与否，取决于工作是否简单，而非相同工作重复多少次。这里的简单工作是指修改位置少、一眼就能判断结果正确。

修改位置多或验证步骤多的工作，即使只做一次也并不简单。因此，只要有工具就能验证正确性，Agent 即便只做一次，也应制作工具。

请求示例：离开期间委托迁移

pstack 的 README 中有以下请求示例。README 将它列为 /figure-it-out 的示例；本书认为，这项保持行为不变的迁移也是工具有用的场景。

```
/poteto-mode i'm stepping away. migrate every caller from the synchronous store to the new async
one, keeping behavior identical. i want to trust it was done right when i'm back.
// 我暂时离开。把所有调用方从同步存储迁移到新的异步存储，保持行为不变。我回来时需要有证据相信迁移正
确。
```

总结

- Principle 是决定是否纳入当前改动、工作是否尚未完成的判断标准。人可以在请求中写出 Principle 名称，调整 Agent 的工作方向。
- 减少代码的三条 Principle 是「**Laziness Protocol**」「**Subtract Before You Add**」和「**Minimize Reader Load**」。
- 设计安排由四条 Principle 决定：「**Foundational Thinking**」（先确定数据形态与基础设施）、「**Redesign from First Principles**」（按需求从一开始就存在来重新设计）、「**Attack the Premise**」（质疑在同一测试持续失败的修复所依赖的前提）和「**Outcome-Oriented Execution**」（声明可暂时失效的位置，再朝最终形态推进）。
- 目标形态由「**Experience First**」（选择用户体验）和「**Exhaust the Design Space**」（没有先例的设计比较不同形态的方案）决定。
- 「**Build the Lever**」把工作转化为可重复执行的工具。如果 Agent 虽引用这条 Principle，却未在差异中加入工具，就不能算实际应用。

下一章第 18 章介绍 Architecture 的六条 Principle，它们都用于决定「数据和验证应放在代码的什么位置」。

第 18 章：Architecture，设计边界与依赖的六条原则

kaito · 来源：日本語 · English · 2026-10-03

本章介绍以下六条 Principle。

1. Model the Domain
2. Boundary Discipline
3. Type System Discipline
4. Make Operations Idempotent
5. Migrate Callers Then Delete Legacy APIs
6. Separate Before Serializing Shared State

这六条属于 Architecture（架构）组，决定 **Agent 应如何组织代码的形态（例如如何保存状态、在哪里验证）**。随附指南的 [docs/guide/08-principles.md](#) 将它们概括为决定「状态、验证和兼容性应放在哪里」的原则。

应用这六条 Principle，Agent 会采取以下做法。

- 用状态机保存状态，而非组合多个布尔值
- 将外部数据的验证集中在边界
- 将所有调用方迁到新 API 后，在同一改动中删除旧 API

- **状态机**……预先规定可能的状态及状态之间如何转移的写法。在任一时刻，只能处于某个既定状态。

例如，交通信号灯一次只显示绿、黄、红其中一种颜色，并按绿→黄→红的顺序变化。它不会「绿灯和红灯同时亮」，也不会「从绿灯直接跳到红灯」。状态机用代码表达这样的规则。

- **边界**……表单输入或 API 响应等外部数据进入的位置。

本章与第 17 章一样，逐条从规则和触发条件介绍六条 Principle，最后从「让结构本身成为给 Agent 的说明书」这一角度加以总结。

本章的请求示例也与第 17 章一样，写成包含 Principle 名称的单行指令。即使不写名称，只要工作符合触发条件，Agent 就会自行阅读并应用 Principle；这一点已在第 17 章的前提中说明。

六条 Principle 中，本章只给「Separate Before Serializing Shared State」提供请求示例。

本书只收录 pstack 随附指南或 README 中已有的请求示例。若为原文没有示例的 Principle 自行编写示例，可能展示 pstack 并未预期的用法。

本章结构

本章结构如下。

- 六条 Principle 一览
- 「Model the Domain」：用结构而非条件分支表达领域
- 「Boundary Discipline」：将输入验证集中在外部边界，在内部信任类型
- 「Type System Discipline」：利用类型防止构造无效状态
- 「Make Operations Idempotent」：让操作无论运行多少次都得到相同结果
- 「Migrate Callers Then Delete Legacy APIs」：迁移所有调用方，在同一改动中删除旧 API
- 「Separate Before Serializing Shared State」：先消除共享写入，锁是最后手段
- Architecture 组的六条 Principle：让结构本身成为给 Agent 的说明书
- 总结

六条 Principle 一览

Principle	一句话结论
「Model the Domain」	用结构而非散落的条件分支表达领域

Principle	一句话结论
「Boundary Discipline」	将验证与错误处理集中在外部数据进入的边界；边界内部的代码信任类型
「Type System Discipline」	利用类型检查，在编译阶段阻止构造不可能的状态
「Make Operations Idempotent」	让改变状态的操作无论运行多少次，或从中途重新执行，最终都到达相同状态
「Migrate Callers Then Delete Legacy APIs」	迁移调用方至新 API，并在同一改动中删除旧 API
「Separate Before Serializing Shared State」	先消除共享写入；只有确实必要时才用锁让写入排队

「Model the Domain」用结构而非条件分支表达领域

「Model the Domain」要求用数据结构表达实际业务领域，而不是把领域规则分散写在各处的条件分支中。

当状态或分支可能增多时，Agent 应**先选择表达领域的结构，再添加条件分支（if/else）**。

- **领域**……软件处理的业务或对象（例如网店的订单、库存、配送）。
- **数据结构**……数据包含哪些字段，以及各字段取什么类型的值（如下例中的 `Order` 和 `OrderFlags`）。
- **表达领域的结构**……与真实业务规则具有相同形态的数据结构。

例如，真实订单只会处于「待付款」「待发货」「已发货」「已取消」四种状态之一。

上述订单数据可用以下两种结构表达。

```
// 表达领域的结构：状态只能取 status 的四种值之一
type Order = {
  address: string;
  status: "unpaid" | "paid" | "shipped" | "canceled";
};

// 未表达领域的结构：用三个布尔值保存状态
// 甚至能出现 isShipped: true 与 isCanceled: true 这样现实中不存在的组合
type OrderFlags = {
  address: string;
  isPaid: boolean;
```

```

    isShipped: boolean;
    isCanceled: boolean;
  };

```

两者都规定了订单包含的字段，以及各字段可取的值，因此都属于数据结构。但只有 Order 与真实订单规则具有相同形态，才是表达领域的结构。

散落的布尔值、重复前提和蔓延的分支，都是不必要的复杂性

这条 Principle 将以下三种情况视为本可避免的复杂性。

- 散落的布尔值
- 在各文件重复的「这些数据应该具有这种形态」的前提
- 跨文件蔓延的分支

包含上述三种复杂性的代码会允许写出不可能的状态

例如，网店订单同时在生成配送标签和编写通知邮件的两个文件中使用。

包含上述三种复杂性的代码如下。

```

// order.ts: 用三个布尔值保存订单状态（分散的布尔值）
type Order = {
  address: string;
  isPaid: boolean;
  isShipped: boolean;
  isCanceled: boolean;
  trackingNo?: string;
};

// label.ts: 生成发货标签
function labelText(order: any): string {
  // 在这里验证「订单应当有地址」这一前提
  if (!order.address) throw new Error("住所がありません");
  // 按状态分支
  if (order.isCanceled) return "キャンセル済み";
  if (order.isShipped) return "発送済み";
  if (order.isPaid) return "発送待ち";
  return "支払い待ち";
}

// mail.ts: 生成通知邮件的主题

```



```
function mailSubject(order: any): string {
  // 在另一个文件中再次验证相同前提
  if (!order.address) throw new Error("住所がありません");
  // 同样的分支逻辑在多个文件中重复
  if (order.isCanceled) return "ご注文をキャンセルしました";
  if (order.isShipped) return "商品を発送しました";
  if (order.isPaid) return "お支払いを確認しました";
  return "お支払いをお待ちしています";
}
```

Oh My Stack 项目注（代码示例）：示例中的日文字符串是程序输出：地址缺失时抛出「没有地址」；发货标签依次使用「已取消」「已发货」「待发货」「待付款」；邮件主题依次表达「订单已取消」「商品已发货」「已确认付款」「等待付款」。这些原值会影响示例的输出。

这段代码允许 `isShipped` 和 `isCanceled` 同时为 `true`。

这是**不可能的状态**：真实订单不可能同时「已发货」和「已取消」。

它还允许 `isShipped` 为 `true`，却没有 `trackingNo`（物流追踪编号）。

选择表达领域的结构，可避免不可能的状态并减少分支

Agent 选择表达领域的结构后，**不可能的状态根本无法写出，条件分支也会减少**。这类结构包括状态机和字段明确的类型化对象。

用表达领域的结构改写同一订单，如下所示。

```
// order.ts: 集中定义订单结构与可能的状态
type OrderStatus =
  | { state: "unpaid" }
  | { state: "paid" }
  | { state: "shipped"; trackingNo: string }
  | { state: "canceled" };

type Order = { address: string; status: OrderStatus };

// 将每种状态对应的文字集中到一张表中
const LABEL_TEXT: Record<OrderStatus["state"], string> = {
  unpaid: "支払い待ち",
  paid: "発送待ち",
  shipped: "発送済み",
  canceled: "キャンセル済み",
};
```

```
// label.ts: 把各状态的文字集中到表中，信任类型并直接查表
function labelText(order: Order): string {
    return LABEL_TEXT[order.status.state];
}

// mail.ts: 把各状态的邮件主题集中到表中，信任类型并直接查表
const MAIL_SUBJECT: Record<OrderStatus["state"], string> = {
    unpaid: "お支払いをお待ちしています",
    paid: "お支払いを確認しました",
    shipped: "商品を発送しました",
    canceled: "ご注文をキャンセルしました",
};

function mailSubject(order: Order): string {
    return MAIL_SUBJECT[order.status.state];
}
```

Oh My Stack 项目注（代码示例）： 状态映射和邮件主题中的日文是实际输出值：未付款、已付款、已发货、已取消对应的标签分别是「待付款」「待发货」「已发货」「已取消」；邮件主题分别表示「等待付款」「已确认付款」「商品已发货」「订单已取消」。

在这段代码中，三种复杂性分别被消除。

- **散落的布尔值**……以 `OrderStatus` 四种状态之一代替三个布尔值。「已发货且已取消」无法表示。只有 `shipped` 状态包含追踪编号，而且进入 `shipped` 状态就必须有该编号。
- **重复的前提**……「订单有地址」只在 `Order` 类型中定义一次。各文件信任类型，无须重复检查地址。
- **跨文件分支**……状态对应的文本由查表决定。添加新状态时，只需为表增加一行；遗漏时会在编译阶段报错。

结构应在编写代码时选择，而非日后补加

Agent 应在编写代码时选择结构。在这个阶段作出选择，几乎不会增加额外工作。

相反，代码若已写成条件分支，之后再补入结构，往往会被推迟。

因为重新引入结构不会改变代码行为，属于重构。

与新功能不同，重构没有明显可见的成果，因此开发者和 Agent 都容易先加功能，把重构推后。

例如，运费规则的条件分支已散布在三个文件，再用查找表替换，也不会改变运费金额或界面显示。因此，这项替换容易被视为「不用立刻做」，而每加一项新功能，分支却继续增加。

规则：每种散乱写法都有相应的替代结构

这条 Principle 列举了以下典型替换方式。

散乱的写法	替代结构
表达状态的布尔值（如 <code>isLoading</code> 、 <code>isError</code> ），以及「现在处于哪个阶段」的检查，散落在代码各处	状态机（规定可能状态及状态如何转移的表达方式）
给函数传递分散的参数，「这些数据应该有这个字段」的同一前提在各处重复	字段明确的类型化对象（如 <code>User</code> 类型）
按类型分流处理的同一条件分支，散布在多个文件中	映射或查找表（以类型为键查找对应值或处理方式），或 discriminated union（可用表示类型的字段区分的联合类型）
在各个使用位置直接修改状态	reducer（根据当前状态与操作计算下一状态的函数）

下面通过示例和代码逐一说明表中各行。

用状态机代替布尔值

用布尔值保存状态时，`running`（运行中）与 `failed`（失败）同时为 `true` 这样的矛盾状态，也能写出来而不产生编译错误。

改用状态机后，**这种组合根本无法表示**。

```
// 修改前：可以表示相互矛盾的状态组合
type ExportJobFlags = { running: boolean; failed: boolean; retrying: boolean };

// 修改后：每种状态可携带的数据由类型决定
type ExportJob =
  | { state: "queued" }
  | { state: "running"; attempt: number }
  | { state: "failed"; attempt: number; error: string }
  | { state: "done"; rowCount: number };
```

```
// 同时定义状态转换：只有失败的任务才能作为下一次尝试回到 running 状态
function retry(job: ExportJob): ExportJob {
  if (job.state !== "failed") return job;
  return { state: "running", attempt: job.attempt + 1 };
}
```

用类型化对象代替分散的参数

例如，给发送欢迎邮件的函数和创建发票的函数分别传递会员姓名、邮箱、套餐等分散参数，两者就都在参数排列中重复了「会员应有姓名、邮箱、套餐」这个前提。

Agent 应创建一个表示会员的 User 类型，让两个函数都接收 User。这样，增加会员字段时只需修改 User 类型；也不会误把姓名和邮箱以相反顺序传入。

```
// 修改前：两个函数各自列出相同的三个参数
function sendWelcomeMail(name: string, email: string, plan: string) { /* ... */ }
function createInvoice(name: string, email: string, plan: string) { /* ... */ }
```

```
// 修改后：只定义一次 User 类型，两个函数都接收它
type User = { name: string; email: string; plan: "free" | "paid" };

function sendWelcomeMail(user: User) { /* ... */ }
function createInvoice(user: User) { /* ... */ }
```

用查找表或 discriminated union 代替蔓延的条件分支

例如，显示页面、账单和邮件三个文件分别用条件分支决定各类订单的运费。Agent 应创建一张「按订单类型查运费」的表，让三个文件共用。

这样，新增订单类型时，无须分别修改三个文件的条件分支，只需在表中增加一行。

```
type OrderKind = "standard" | "express" | "pickup";
declare const order: { kind: OrderKind };

// 修改前：页面、计费 and 邮件三个文件中都有相同的 if/else
function shippingFee(kind: OrderKind): number {
  if (kind === "standard") return 500;
  if (kind === "express") return 1200;
  return 0;
}
```

```

}

// 修改后：只建立一张按类别查运费的表，三个文件都查询它
const SHIPPING_FEES: Record<OrderKind, number> = {
  standard: 500,
  express: 1200,
  pickup: 0,
};

const fee = SHIPPING_FEES[order.kind];

```

写成 `Record<OrderKind, number>` 后，如果在类型 `OrderKind` 中增加订单类型，却忘记在表 `SHIPPING_FEES` 中加一行，编译阶段就会报错。

当每个类型对应一个确定的「值」时，查找表很合适。

如果不同类型**拥有的数据本身不同**，Agent 应使用 **discriminated union**。

例如，支付方式有「银行卡」「银行转账」「货到付款」三类：只有银行卡支付包含卡号后四位，只有银行转账包含收款账户编号。

若用表示类型的字段及可选字段（optional 类型）编写，仍能表示「银行卡支付却含有银行账户编号」这样的无效值。

使用 `discriminated union` 后，`kind` 的值决定允许哪些字段，因此无法写出这类值。

```

// 修改前：任何支付方式都可以包含全部字段
type PaymentLoose = { method: "card" | "bank" | "cod"; cardLast4?: string; bankAccount?: string };

// 修改后：kind 的值决定可以包含哪些字段
type Payment =
  | { kind: "card"; cardLast4: string }
  | { kind: "bank"; bankAccount: string }
  | { kind: "cod" };

function paymentLabel(p: Payment): string {
  switch (p.kind) {
    case "card":
      return `カード (末尾 ${p.cardLast4})`;
    case "bank":
      return `銀行振込 (口座 ${p.bankAccount})`;
    case "cod":
      return "代金引換";
  }
}

```

Oh My Stack 项目注（代码示例）：三种支付方式的原始输出依次是「卡片（末尾四位数字）」「银行转账（账户号码）」和「货到付款」。这些日文显示值会影响示例输出，因此保留。

用 reducer 代替到处直接修改

例如，商品列表、商品详情和购物车三个页面都直接修改购物车内容，适合使用 **reducer**。此时，想知道购物车内容在哪里、怎样发生变化，就必须阅读全部三个位置。

Agent 应将购物车操作（添加、删除）定义为类型，创建一个「由当前购物车和操作计算下一购物车」的 reducer。三个页面不再直接修改购物车，而是把操作交给 reducer。这样，**只看 reducer 就能知道购物车内容如何变化。**

```
type CartItem = { id: string; price: number };
type Cart = { items: CartItem[] };
declare let cart: Cart;
declare const item: CartItem;
declare const id: string;

// 修改前：三个页面分别直接修改购物车
cart.items.push(item);
cart.items = cart.items.filter((i) => i.id !== id);

// 修改后：用类型定义操作，下一步购物车状态只在 reducer 中计算
type CartAction =
  | { type: "add"; item: CartItem }
  | { type: "remove"; id: string };

function cartReducer(cart: Cart, action: CartAction): Cart {
  switch (action.type) {
    case "add":
      return { items: [...cart.items, action.item] };
    case "remove":
      return { items: cart.items.filter((i) => i.id !== action.id) };
  }
}

cart = cartReducer(cart, { type: "add", item });
```

不强行引入结构

不过，这条 Principle 并不要求强行引入结构。如果现有代码清楚、集中在一处，且预计不会扩展，Agent 就应保留简单写法。

Agent 还应**质疑只在调用之间插入一层、却没有减少任何复杂性的共用函数或类型**。如果它们既没有减少分支，也没有减少重复规则或无效状态，只会增加读者需要打开查看的位置。

触发条件

- 编写有状态的逻辑时
- 分支很多时
- 「这些数据有这个字段」的同一前提在多个文件重复时

出现以下情况，说明可能跳过了这条 Principle。

- Agent 为新功能在现有的一串条件分支上再添一个分支
- Agent 增加第二个布尔值，它必须始终与现有布尔值保持一致

例如，已有 `isLoading` 表示加载中，又增加 `isError` 表示错误。加载失败时，代码必须将 `isError` 设为 `true`，同时将 `isLoading` 恢复为 `false`。

如果忘记恢复 `isLoading`，`isLoading: true` 与 `isError: true` 就会同时成立，界面出现「既在加载又已报错」的矛盾状态。

这样，增加第二个布尔值后，每次修改其中一个，都需要同步维护另一个。

/poteto-mode 的「[Non-negotiables](#)」也要求遵守这条 Principle（见第 8 章）。

「Non-negotiables」要求 Agent 在编写任何代码时，先明确数据形态，再依这条 Principle 选择结构。23 条 Principle 中，只有「[Model the Domain](#)」被「Non-negotiables」明确点名。

「Boundary Discipline」将输入验证集中在外部边界，在内部信任类型

「Boundary Discipline」要求在系统边界进行输入验证、类型收窄和错误处理。它列举的边界包括 CLI 参数、配置文件、外部 API 和经网络传入的数据。

在边界验证过的值带有类型，因此边界内部的代码应信任它符合类型，无须重复相同检查。

将检查集中于边界，是因为检查散落在代码各处会带来以下两个问题。

- 同一检查要在多处重复编写
- 开发者和 Agent 以为「某处应该已经检查过了」；若不了解各项检查究竟在哪里进行，就可能漏掉完全没有验证的路径

下面的代码展示了检查分散在各处的情况：来自订单表单的「数量」是否至少为一，只有部分函数会验证。

```
let stock = 10; // 库存数量

// 计算价格：检查数量是否至少为 1
function calcPrice(quantity: number): number {
  if (quantity < 1) throw new Error("数量は1以上にしてください");
  return quantity * 500;
}

// 减少库存：假定「其他地方已经验证过」，此处不做检查
function reserveStock(quantity: number) {
  stock = stock - quantity;
}

// 订单页面：先调用 calcPrice，验证后再减少库存
calcPrice(2);
reserveStock(2);

// 另一个页面：直接调用 reserveStock
reserveStock(-3); // 库存从 8 意外增加到 11
```

Oh My Stack 项目注（代码示例）：异常消息要求数量至少为 1；原文作为抛出的错误内容保留。

订单页面的路径通过 calcPrice 进行了检查，但只调用 reserveStock 的路径完全没有检查。因此，-3 直接传入，本应减少库存的处理反而增加了库存。

按这条 Principle 改写相同处理，结果如下。

```
let stock = 10; // 库存数量

// 边界：在这里对表单传来的值只验证一次
```



```
function parseQuantity(input: string): number {
  const quantity = Number(input);
  if (!Number.isInteger(quantity) || quantity < 1) {
    throw new Error("数量は1以上の整数にしてください");
  }
  return quantity;
}

// 内部：只会收到已验证的值，因此不重复验证
function calcPrice(quantity: number): number {
  return quantity * 500;
}

function reserveStock(quantity: number) {
  stock = stock - quantity;
}

// 所有页面都先让表单值经过 parseQuantity，再使用它
const quantity = parseQuantity("-3"); // 这里会报错，执行不会继续
calcPrice(quantity);
reserveStock(quantity);
```

Oh My Stack 项目注（代码示例）：异常消息要求数量是至少为 1 的整数；原文作为抛出的错误内容保留。

现在只有 `parseQuantity` 一处负责检查，`calcPrice` 内的检查也不再需要。所有页面都会先让表单值经过 `parseQuantity`，因此 `-3` 等值不会传到 `calcPrice` 或 `reserveStock`。这条 Principle 还要求将业务逻辑写成纯函数，让 `shell` 成为只调用纯函数的轻薄外层（详见「[提取由 shell 调用的纯函数](#)」）。逻辑脱离框架后，**无需运行框架即可单独测试逻辑**。

规则：在边界验证外部数据

Agent 用以下两个问题判断应在哪里进行验证。

只验证刚从外部进入的数据

第一个问题是：「这些数据此刻是否正在跨越系统边界（刚从外部进入）？」

如果没有跨越边界，再检查就是多余的。边界代码把验证后的值转换为带类型的值，传入内部函数；内部函数信任该类型。

例如，读取配置文件的函数在读取时已验证 `batchSize` 为正数，使用它的内部函数就无须再检查 `batchSize > 0`。只需在边界验证一次，所有内部函数便可放心使用。

```
type Config = { batchSize: number };

// 边界：读取配置文件时只验证一次，并转成 Config 类型的值
function parseConfig(raw: { batchSize?: unknown }): Config {
  if (typeof raw.batchSize !== "number" || raw.batchSize <= 0) {
    throw new Error("batchSize は正の数にしてください");
  }
  return { batchSize: raw.batchSize };
}

// 内部：信任 Config 类型，不再重复检查 batchSize > 0
function splitIntoBatches(items: string[], config: Config): string[][] {
  const batches: string[][] = [];
  for (let i = 0; i < items.length; i += config.batchSize) {
    batches.push(items.slice(i, i + config.batchSize));
  }
  return batches;
}
```

Oh My Stack 项目注（代码示例）：异常消息要求 `batchSize` 为正数；原文作为抛出的错误内容保留。

提取由 shell 调用的纯函数

第二个问题是：「这段处理能否提取为纯函数，让 shell 仅负责调用？」

如果可以，Agent 就应将其提取为纯函数。

- **纯函数**……相同参数始终返回相同结果，且不读写界面或文件等程序外部对象的函数。
- **shell**……负责与界面、文件、网络、框架等外部对象交互的部分：从外部取值，交给纯函数，再将结果输出到外部。

例如，Agent 不应把订单总额计算直接写在按钮点击处理函数内，而应提取为 `calcTotal(items)`，点击处理函数只负责调用它。这样，无须运行界面即可测试金额计算。

另一方面，shell 中没有计算逻辑，出错的余地很小。

```
type Item = { price: number; quantity: number };
declare const button: HTMLButtonElement;
declare const totalLabel: HTMLLabelElement;
declare function readItemsFromCart(): Item[];

// 纯函数：相同的 items 总返回相同的总价，不访问页面
function calcTotal(items: Item[]): number {
  return items.reduce((sum, item) => sum + item.price * item.quantity, 0);
}

// 外壳：只从页面读取值、调用纯函数，并把结果写回页面
button.addEventListener("click", () => {
  const items = readItemsFromCart(); // 从页面读取
  const total = calcTotal(items); // 把计算交给纯函数
  totalLabel.textContent = `${total}円`; // 写入页面
});
```

Oh My Stack 项目注（代码示例）：页面文字末尾的「円」表示日元，是原示例的实际显示值。

触发条件

加入输入验证或错误处理，或编写连接框架与自身代码的适配器时。

「Type System Discipline」利用类型防止构造无效状态

「Type System Discipline」把类型检查器视为证明「这个值必定具有这种形态」的工具（原文称 proof assistant）。

Agent 用类型检查器让矛盾状态或遗漏的 variant 处理在**编译阶段就报错，而不是运行之后才发现**。如果类型系统未能发现这些遗漏，原本可由编译器阻止的错误就会留到运行时。

- **类型检查器**……程序运行前发现类型错误的机制，例如 TypeScript 编译器。
- **variant**……组成联合类型的每一种形态。例如在 `type Status = "success" | "error" | "pending";` 中，`success`、`error` 等每个值都是一个 variant。

- **联合类型**……表示一个值可取若干形态之一的类型。TypeScript 的联合类型，例如 `type FooBar = 'foo' | 'bar'`，其值只能是 `foo` 或 `bar` 之一。

规则：在定义类型时就排除无效状态

主要模式如下。

让不可能的状态无法表示

如果任务类型是 `{ completed: boolean; completedAt?: Date }`，就能写出「已经完成却没有完成时间」的无意义组合。

因此，Agent 应改用 `{ kind: "open" } | { kind: "done"; at: Date }` 这样的联合类型。这样一来，只能写出「完成时必有时间」的形态。

相比事后检查组合是否正确，**从一开始就不允许表示无效组合更可靠。**

为含义不同的值加类型标记

Agent 将 `UserId` 和 `OrderId` 设为品牌类型，即使两者底层都是字符串，也不可互换使用。

品牌类型……给字符串或数字等基本值（primitive）加上类型标记形成的类型，例如下方代码中的 `UserId` 与 `OrderId`。

例如，把订单 ID 传给应接收用户 ID 的 `getUser()`，就会在编译时出错。如果两者可互换，不同含义的值即使被混淆也能通过编译。

```
type UserId = string & { readonly __brand: "UserId" };
type OrderId = string & { readonly __brand: "OrderId" };
type User = { id: UserId; name: string };

declare function getUser(id: UserId): Promise<User>;
declare const orderId: OrderId;

// 两者内部都是字符串，但传入 OrderId 会导致编译错误
await getUser(orderId);
```

不对类型系统说谎

Agent 不应靠 `cast` 或 `assertion` 绕过类型检查；绕过的位置仍有运行时出错的风险。

- **cast**……强制把值视为另一种类型的写法，例如以 `value as unknown as User` 强制完成未经类型检查的转换。
- **assertion**……向编译器断言「这个值必然属于这一类型」的写法，例如在 TypeScript 中使用 `as` 子句写成 `as User`。

例如，未经验证就把 API 响应写成 `as User`，编译会通过。

但即使实际响应缺少 `name` 字段，代码仍假定它存在，最终可能在使用 `user.name` 时于运行中出错。

编译器无法证明的事实，Agent 应通过实际检查值或用条件分支收窄类型，自行证明。

让编译器检查遗漏的分支

Agent 应使联合类型增加新 variant 时，未处理它的分支产生编译错误。

例如，任务类型新增「已取消」（`canceled`）时，如果决定显示文本的函数没有处理它，应让编译报错。

这样，就无须靠人逐处查找遗漏。

```
// 已给任务类型添加 canceled
type Task = { kind: "open" } | { kind: "done" } | { kind: "canceled" };

// 修改前：未处理的类型会落到最后的 return ""，编译不会报错
function labelLoose(task: Task): string {
  if (task.kind === "open") return "未完了";
  if (task.kind === "done") return "完了";
  return ""; // canceled 类型的任务在页面上显示为空文字
}

// 修改后：漏掉任何类型都会导致编译错误
function label(task: Task): string {
  switch (task.kind) {
    case "open":
      return "未完了";
    case "done":
```

```

    return "完了";
  default: {
    // 因为没有用 case 处理 canceled, 编译会在此行报错
    const unhandled: never = task;
    return unhandled;
  }
}
}
}

```

Oh My Stack 项目注（代码示例）： open 和 done 的日文显示值分别表示「未完成」与「已完成」；它们是函数实际返回的文字，因此保留原值。

原来的 labelLoose 即使忘记处理 canceled 也能编译通过，直到界面显示空文本，才会发现遗漏。

修改后的 label 中，never 类型表示「这里理论上不会有任何值到达」。

如果所有类型都已由 case 处理，就不会有值到达 default，编译可通过。

如果遗漏 canceled，该任务会进入 default，在赋值给 never 变量的位置产生编译错误。报错的位置就是应增加 canceled 分支的位置。

只在已证明类型太弱的位置加强类型

不过，这条 Principle 并不以把类型定义得越细越好为目标。类型的职责不是尽可能细致地描述数据，而是**完整列出该类型各使用位置必须处理的情况**。

因此，Agent 只在发现类型过弱的位置加强类型。判断线索包括运行时 assertion，或声称「不可能发生」并抛错的 throw 语句。这些位置说明类型放行了本可阻止的情况。

例如，任务一旦完成就必有时间，却仍需写 `if (!task.completedAt) throw new Error("起きるはずがない")`，说明类型允许了「已完成却无时间」的值。

触发条件

- 设计类型时
- 审阅函数签名（接收参数与返回值的类型）时
- 在静态类型语言（如 TypeScript，运行前检查类型）中编写代码时

这条 Principle 提出的一个判断问题，把类型看作给下一位修改代码的 Agent 的指令。

"If a new variant is added next month, will the compiler tell the next agent where to add a case?" If no, the match isn't exhaustive.

「下个月新增 variant 时，编译器会告诉下一位 Agent 在哪里增加分支吗？」如果不会，分支就没有覆盖所有情况。

编写覆盖所有情况的分支，编译器就会告诉下一位 Agent 应修改的位置，因此人无须再写「增加新状态时，记得改这三处」之类的文字指令。

/typescript-best-practices 是以 TypeScript 的具体写法体现这条 Principle 的 Skill（见第 29 章）。

「Make Operations Idempotent」让操作无论执行多少次都到达相同状态

「Make Operations Idempotent」要求改变状态的操作，无论执行多少次，或从中途重新执行，**最终都收敛到同一正确状态**。

- **幂等 (idempotent)** ……同一操作执行多少次，都与只执行一次得到相同结果的性质。例如，设置值为 5，无论执行多少次，结果仍是 5。

这项要求源于：**必须按操作可能中途停止来设计**。

命令、启动和结束步骤、重复处理循环，经常会因崩溃、重启或重试而中途停止或再次执行。

如果前次执行留下的数据或文件使下一次执行结果改变，开发者每次重启都得调查「为什么这次结果不同」。

保证幂等性后，反复重新执行都得到同一结果，就不必每次重启后重新调查。

规则：用三个问题检查是否收敛

Agent 用以下三个问题检查操作是否幂等。

1. 连续执行两次相同操作，结果会怎样？
2. 如果前次执行中途停止，下次执行会得到什么结果？（无论在开始、途中还是结束前停止，结果都相同吗？）
3. 不管重新执行多少次，最终是否到达同一状态（收敛）？

只要其中一个问题的答案是「取决于前次执行留下什么」，操作就需要**调和 (reconciliation) 步骤**。

以第二个问题为例。

假设要把 1,000 行写入输出文件，下次执行保留前次的文件，并从第一行开始追加到末尾。

如果前次写到第 400 行停止，文件中就会接上本次写入的 1,000 行。前次 400 行与本次最初 400 行内容相同，于是文件变成有重复内容的 1,400 行。如果前次一行都没写，文件则只有 1,000 行。

结果随前次执行停止的位置而变，就需要调和。

- **调和**……检查前次执行留下的内容，无论留下什么，都将状态调整到正确状态的步骤。

例如，即使中途停止的执行留下只写到第 400 行的输出文件，**下次执行先找到并删除该文件，再开始写入**，不管前次停在哪，最终都会得到相同的 1,000 行文件。

触发条件

设计可能中途停止或重试的命令、启动和结束步骤、重复处理循环时。

「Migrate Callers Then Delete Legacy APIs」迁移所有调用方，并在同一改动中删除旧 API

「Migrate Callers Then Delete Legacy APIs」用于确定新内部 API 才是正确设计时。

Agent 不保留兼容层，迁移所有调用方至新 API，**并在同一改动中删除旧 API**。

同时保留新旧 API，会出现两套实现相同事情的调用方式，清理被推迟，代码只增不减。

兼容层……为继续支持旧 API 调用方式而保留的层。

poteto 在演讲中（见第 4 章）指出，推荐的实现方式必须收敛为一种。本书把这条 Principle 理解为在 API 迁移中重新收敛至一种实现方式的步骤。

规则：找齐并迁移调用方，立即删除旧 API

主要规则如下。

不以仍有调用方为由保留旧 API

Agent 应找出旧 API 的所有调用方，迁移到新 API，然后立即删除旧 API。

例如，要把 `getUser(id)` 换成 `fetchUser({ id })`，Agent 应搜索所有调用 `getUser` 的位置，改为 `fetchUser`，在同一系列改动中删除 `getUser`。

新旧调用方式并存，读者会不知该用哪一种，清理也容易被推迟。

临时过渡层应是例外

连接旧调用方式与新 API 的临时层（适配器）只能在确实必要时使用；Agent 设置它时必须确定删除期限。若习惯性地增加过渡层，代码就会只增不减。

例如，保留旧 `getUser(id)`，把函数体改成只有 `return fetchUser({ id });` 一行，就是一种适配器。

调整测试以适应新 API

Agent 应将测试调整为验证新 API 承诺的行为，删除只维护旧实现内部结构的测试。

例如，应保留验证「`fetchUser({ id })` 是否返回相应 ID 的用户」的测试，删除只验证「旧 `getUser` 内部调用另一个函数多少次」的测试。

保留保护旧实现细节的测试，代码就不得不继续遵循本应删除的旧内部结构。

触发条件

在旧调用方尚未全部迁移时引入新的内部 API。

有外部用户的公开 API（外部 API）不能直接应用这条 Principle，因为它依赖以下三个前提。

1. 没有外部用户在使用旧 API
2. 删除旧 API 后会失效的调用方，都能在项目内部一次性改完
3. 引入新 API 是为简化代码或重构

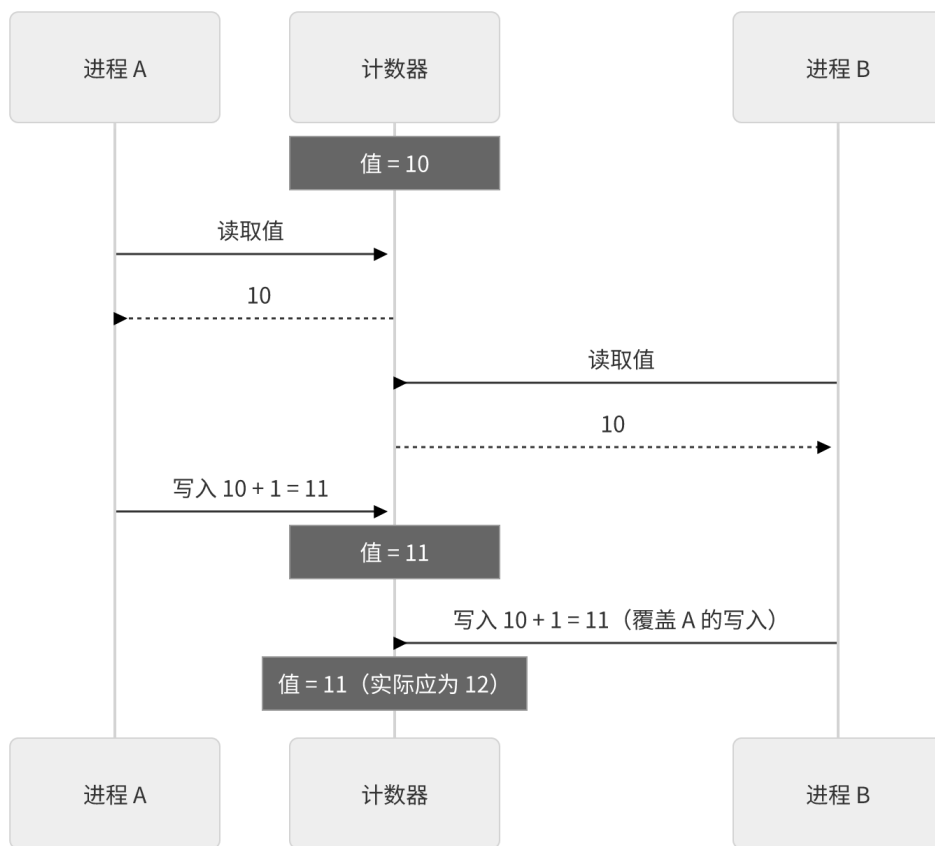
「Separate Before Serializing Shared State」先消除共享写入，锁是最后手段

「Separate Before Serializing Shared State」适用于多个主体（如 Agent 或 Worker）可能共享同一写入目标的情境。

加入锁之前，Agent 应先判断**各主体是否真的需要写入同一目标**。如果不需要，就为每个主体分配独立的写入目标，消除共享。并发写入造成的竞态条件可能只偶尔出现、难以复现，因此查找原因很费时间。

- **Serializing（串行化）**……使用锁等机制让写入逐一按顺序进行。
- **锁**……保证同一时刻只有一个处理过程可以写入的机制。
- **竞态条件**……多个处理过程同时读写相同数据，导致结果随执行顺序而变化的状态。

例如，处理过程 A 和 B 都给值为 10 的计数器加一，就会出现以下情况。



查看图示 1

两个处理过程各加一，结果本应为 12；但 B 根据 A 写入前的值 10 写入 11，因此最终得到 11。

锁不会消除共享，因此串行化写入是最后手段

这条 Principle 将写入串行化视为无法拆分写入目标时的最后手段。

这是因为锁并不消除共享本身。

拆分写入目标后，不再有多个主体写入同一位置，竞态条件就不会发生。

而使用锁后，写入目标仍是共享的。所有主体每次写入都必须获取锁；只要有一个处理过程未经获取锁便写入，竞态条件就会重现。

此外，持锁的主体若中途停止，锁可能无法释放，其他主体就可能一直无法写入。

因此，这条 Principle 要求在觉得「需要加锁」时，先重新审视写入目标是否真的无法拆分，而不是立刻加锁。

即使无法拆分，也要用机制保证顺序，而非依赖指令

如果写入目标确实无法拆分，也应使用锁等机制保证顺序，而非依赖对人的指令。

这条 Principle 的第一段以以下一句话结尾。

Instructions and conventions are not concurrency control.

指令和惯例并不是并发控制。

放在 Agent 工作中，即使人在请求中写了「不要修改同一个文件」，这句话本身也不是防止 Agent 并发写入的机制。

规则：加锁前先拆分写入目标

步骤共有三个。

1. 找出多个主体会修改的共享位置

找出多个主体读写的文件，或多个主体会 push 的分支等。

2. 消除共享写入

Agent 应以消除共享写入为基本处理方式。

首先判断各主体是否确实需要同一份权威数据，还是只在分别记录不同事实。

随后，为每个主体分配专属的文件、键或分支，并在读取或报告结果时汇总。

例如，两个 Worker 分别向同一个 `state.json` 写入各自的字段，就是共享写入。改成 `indexer-state.json` 和 `metrics-state.json` 后，就消除了共享。

拆分写入目标前后的示意如下。

拆分前（各主体共享同一个写入位置）

WorkerA —

```

      |————→ state.json
WorkerB ———|

```

拆分后（各主体分别拥有自己的写入位置）

```
WorkerA —————→ indexer-state.json
```

```
WorkerB —————→ metrics-state.json
```

3. 只有无法拆分时才串行化写入

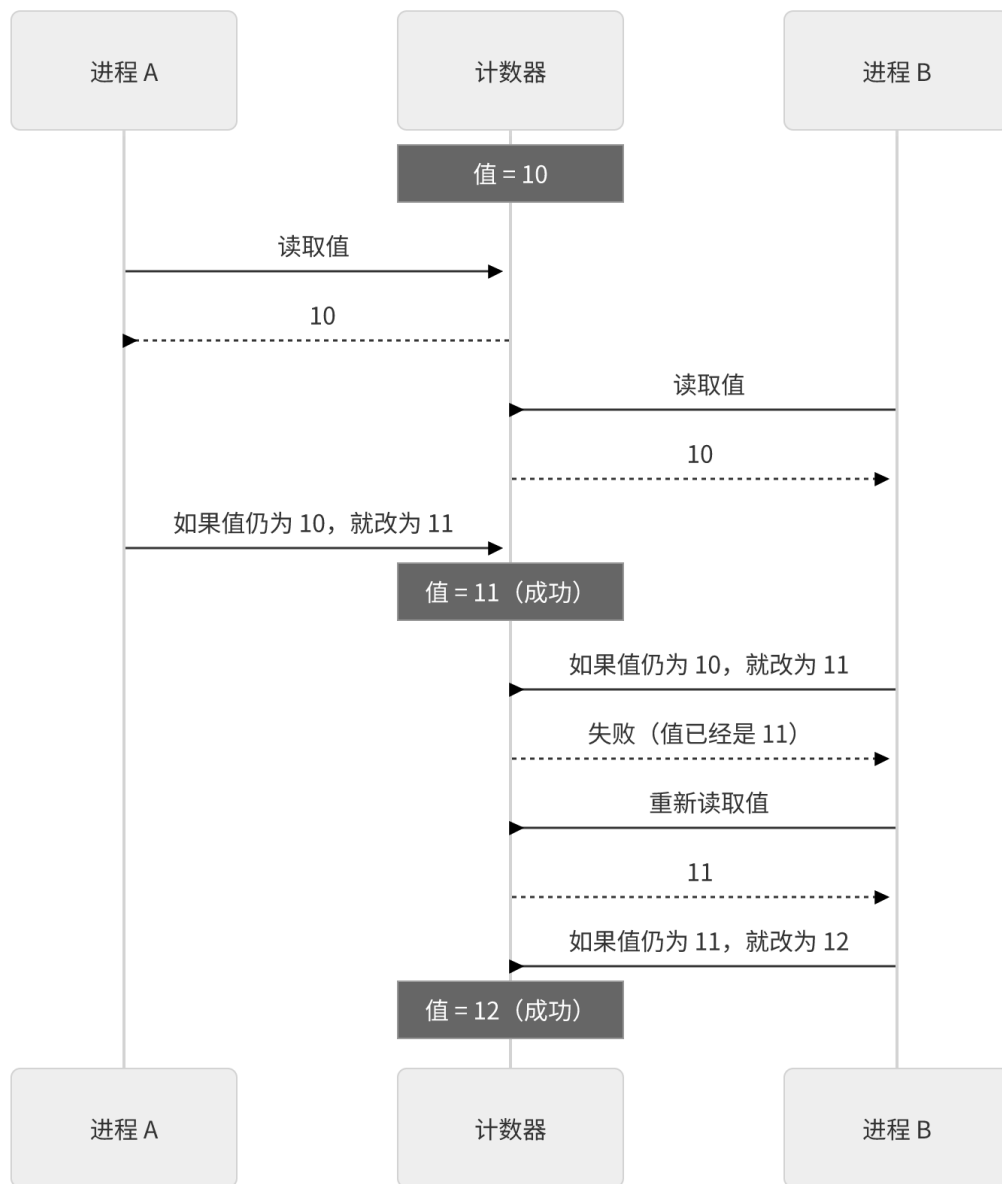
只有在写入目标必须统一才能保证正确时，才引入让写入逐一按顺序进行的机制（串行化）。

例如库存余量或座位预订，各主体必须读写同一个值，才有可能正确。

这条 Principle 提出了以下四种机制。

- **锁文件**……写入前创建表示「正在写入」的文件，写完后删除；其他主体在文件存在时等待。
- **按阶段顺序执行**……拆分写入阶段，上一阶段结束后才开始下一阶段（如 A 写完后 B 再写）。
- **由单一主体负责写入**……只允许一个主体写入，其他主体请它「写入这个值」。
- **原子比较并交换**……仅当值仍为读取时的值，才进行修改，例如「如果仍是 10，就改成 11」；若值已改变，就重新读取再试。检查与写入必须作为不可被其他处理过程插入的单一操作（原子操作）完成。

将原子比较并交换应用到前述计数器示例，结果如下。



查看图示 2

B 第一次试图改值时，A 已将其改为 11，因此失败。B 重新读取 11 再试，最终正确得到 12。

触发条件

多个并发主体可能写入同一文件、分支、键或有状态对象时。

请求示例：为并行尝试分别提供专用 worktree

随附指南的 `docs/guide/08-principles.md` 举例：两个并行尝试准备写入同一分支时，可使用以下单行请求。

```
separate before serializing shared state. give each attempt its own worktree, no locks.  
// 使用 separate before serializing shared state。给每次尝试分配专用 worktree，不要加锁。
```

- **worktree**……从同一 Git 仓库创建额外工作目录的功能，可让不同分支在不同目录中同时工作。

Architecture 组的六条 Principle 让结构本身成为给 Agent 的说明书

poteto 在演讲中提出，提高对 Agent 信任的一种方法是改善代码库本身的结构：让 Agent 看见文件夹、类型和流程边界，就知道自己可以在哪里做什么。

演讲将其称为**让结构本身成为给 Agent 的说明书的设计**。

六条 Principle 各自将判断保留在类型、状态机和文件划分等结构中。下表列出各条 Principle 在审阅时可提出的问题，以及承载该判断的结构。

原则	审阅时可问的一句话	承载判断的结构
「Model the Domain」	「是否增加了第二个布尔值？」	状态机、discriminated union、查找表
「Boundary Discipline」	「这些数据此刻是否正在跨越边界（刚从外部进入）？」	每个边界的解析函数（验证外部数据并转换为带类型值的函数）
「Type System Discipline」	「新增 variant 后，编译器会指出遗漏吗？」	联合类型、品牌类型、用 never 检查分支是否覆盖所有情况
「Make Operations Idempotent」	「重复执行两次或中途停止，会怎样？」	调和步骤
「Migrate Callers Then Delete Legacy APIs」	「旧 API 何时删除？」	在迁移调用方的同一系列改动中删除旧 API

原则	审阅时可问的一句话	承载判断的结构
「Separate Before Serializing Shared State」	「只有一个主体会写入这个目标吗？」	为每个主体分配文件、分支和worktree

总结

- 「**Model the Domain**」把散落的分支和布尔值整理成状态机或查找表。Agent 每次编写代码都应应用这条 Principle。
- 「**Boundary Discipline**」将验证集中在边界，边界内信任类型。
- 「**Type System Discipline**」在编译阶段阻止构造不可能的状态。
- 「**Make Operations Idempotent**」用三个问题检查，让操作即使重试或重启，也收敛到相同的最终状态。
- 「**Migrate Callers Then Delete Legacy APIs**」在同一系列改动中迁移调用方并删除旧 API。本书认为，这有助于只保留一种推荐实现方式。
- 「**Separate Before Serializing Shared State**」在用锁串行化写入前，先消除共享写入本身。
- 本书认为，六条 Principle 都是在让结构本身成为给 Agent 的说明书。

下一章第 19 章介绍 Verification 组的四条 Principle，它们决定什么才算完成的证据。

第 19 章：Verification，通过实际行为验证的四条原则

kaito · 来源：日本語 · English · 2026-10-03

本章介绍以下四条 Principle。

1. Prove It Works
2. Fix Root Causes
3. Sequence Work into Verifiable Units
4. Test Behavior, Not Implementation

这四条属于 Verification（验证）组，决定 **Agent** 验证到什么程度才可以宣称工作完成。随附指南的 `docs/guide/08-principles.md` 将它们概括为决定「什么才算证明」的原则。

应用这四条 Principle，Agent 会采取以下做法。

- 不因「构建通过」就结束，而是打开输出文件，检查实际写入的行
- 不只消除错误症状（如值为 null 时加检查并跳过处理），而是查明错误原因并修复
- 将工作拆成小单元，每完成一个单元就运行测试，通过后再进入下一单元
- 测试时比较具体输入所产生的实际返回值与期望值，而非检查代码调用了哪些函数

本章与第 17 章一样，逐条从规则和触发条件介绍四条 Principle，最后说明它们如何用于处理重复行的请求。

重复行的请求，是前言介绍的这个示例：用户请 Agent 修复「运行中途触发重试时，导出会写入重复行」的缺陷。

```
/poteto-mode the export writes duplicate rows when a retry lands mid-run. repro first, then fix and verify.
```

```
// 如果重试发生在执行中途，导出会写入重复的行。先复现，再修复，然后验证。
```

本章的请求示例也与第 17 章一样，写成包含 Principle 名称的单行指令。即使不写名称，只要工作符合触发条件，Agent 就会自行阅读并应用 Principle；这一点已在第 17 章的前提中说明。

四条 Principle 中，本章只给「Prove It Works」提供请求示例。

本书只收录 pstack 随附指南或 README 中已有的请求示例。如果为原文没有示例的 Principle 自行编写示例，可能会展示 pstack 并未预期的用法。

本章结构

本章结构如下。

- 四条 Principle 一览
- 「Prove It Works」：直接检查实际成果，而非依赖代理指标
- 「Fix Root Causes」：修复缺陷原因，而非掩盖症状
- 「Sequence Work into Verifiable Units」：把工作拆成可验证的小单元逐步推进
- 「Test Behavior, Not Implementation」：测试行为，而非实现细节
- 在重复行请求中，「Fix Root Causes」和「Prove It Works」应用于「Bug fix」Playbook 的各个步骤
- 总结

四条 Principle 一览

Principle	一句话结论
「Prove It Works」	直接检查实际成果，而非依赖代理指标
「Fix Root Causes」	修复造成错误等表面症状的根本原因，而非掩盖症状
「Sequence Work into Verifiable Units」	把工作拆成可验证的小单元，各单元的测试等检查通过后再进入下一个
「Test Behavior, Not Implementation」	以使用者相同方式调用代码，将可观察的结果与直接写在测试中的期望值比较

代理指标（原文称 proxy）……代替真正要验证的对象而使用的间接线索。

例如，认为「文件修改时间很新，内容应该也正确改写了」或「构建通过，功能应该能用」，这里的修改时间和构建成功就是代理指标。

「Prove It Works」 直接检查实际成果，而非依赖代理指标

「Prove It Works」要求 Agent 在宣称完成前，**亲自验证**实际运行的应用、输出文件或界面上显示的值。

这条 Principle 的核心是原文开头的以下一句话，第 2 章也引用过。

Verify every task output by checking the real thing directly. Do not infer from proxies, self-reports, or "it compiles."

对每项工作成果，直接检查实际对象加以验证。不要从代理指标、自述或「编译通过」推断。

提出这项要求，是因为**未经验证的工作，即使看起来正确，实际也可能出错**。

与直接查看实际成果相比，Agent 容易觉得下列间接检查更省事，因为它们都不必运行功能。

- 文件修改时间（「修改时间很新，导出应该成功了」）
- 输出是否为新内容（「看起来是这次运行产生的，应该正确」）
- Agent 的自述（「已经修好了」的报告）
- 保存的截图（先前拍下的界面）

但依错误推测继续工作，代价可能远高于直接验证实际成果。

例如，Agent 只看修改时间，就报告「导出成功」，实际上文件含重复行。用户之后发现重复，就得重新调查错误从哪一步开始。

因此，未检查实际成果就宣称完成，之后可能大幅增加工作量；必须在完成前直接验证。

规则：实际运行并读取实际值

直接检查实际对象，指 Agent 亲自运行功能、读取实际值。主要规则如下。

实际运行功能并验证

Agent 不应因「构建通过」或「Agent 如此报告」就结束，而要实际运行功能、观察结果。构建成功只说明代码能够编译。

例如，按下按钮可把商品加入购物车，Agent 就应实际按下按钮，在界面确认购物车中出现该商品。

直接确认程序是否正在运行

Agent 不应依据日志文件等程序留下的其他信息，推测服务器一类的程序是否正在运行。

例如，不应仅因「日志最近更新」就认定服务器仍在运行，而应实际访问服务器并确认收到响应。日志可能是在服务器停止前一刻写下的。

读取实际值，而非保存的副本

Agent 应读取实际值，而非缓存值或由原始值派生的其他显示。

例如，要验证界面显示的金额，应读取当前界面的数字，而非旧截图。原始值改变后，旧截图可能仍显示过时数据。

检查失败时，先质疑检查方法，再判断系统是否出错

检查结果异常时，Agent 不应立刻断定系统坏了，应先检查自己的验证方法是否有误。

例如，界面没有显示新值，Agent 应先确认是否一直开着旧页面、是否访问了其他环境的服务器。验证方法错误时，可能误把正常系统当成故障去修。

将验证写成脚本并保留

这条 Principle 认为，最有力的证据不是一次性的目视检查，而是**可反复执行相同比较的脚本**。

脚本每次运行都会做相同比较，审阅者无须相信 Agent 的描述，可以亲自运行相同检查，核对结果是否一致。

因此，Agent 应尽可能编写并运行验证脚本，把输出保留在审阅者可见的位置。

例如，比较输出文件与期望内容的脚本可以写成如下形式。

```
// check-export.ts: 比较导出文件的各行与预期内容
import { readFileSync } from "node:fs";

const actual = readFileSync("out/export.csv", "utf8").trim().split("\n");
const expected = ["a", "b", "c"];

if (JSON.stringify(actual) !== JSON.stringify(expected)) {
  console.error("NG", { actual, expected });
  process.exit(1);
}
console.log("OK", actual.length);
```

如果输出文件包含 a、b、c 三行，脚本输出 OK 3。

如果行重复，变成 a、b、a、b、c，脚本会输出 NG 和实际、期望内容，审阅者也能亲自查看差异。

不过，Agent 通常只需在审阅者可见的位置保留脚本和输出，无须将其纳入提交。

只有在大型移植或迁移等大规模、复杂工作中，未来需要审计工作过程时，才将其纳入提交。

触发条件

工作完成之后、宣称完成之前。

请求示例：索取实际输出，而非构建日志

随附指南的 [docs/guide/10-recipes-and-pitfalls.md](#) 将只凭构建通过就报告成功列为常见陷阱，因为构建成功只说明编译通过。

同一页面列出一条让偏离任务的执行回到正轨的单行请求：索取实际输出而非构建日志。

Agent 只凭构建通过就报告成功时，可以使用它。

```
apply prove it works. show me the real output, not the build log.  
// 使用 prove it works。展示真实输出，而非构建日志。
```

「Fix Root Causes」修复缺陷原因，而非掩盖症状

「Fix Root Causes」要求 Agent 调试时不要止步于消除错误的最小改动，而要追问为什么，找到根本原因（root cause），在原因所在的位置修复。

原因在于症状性的处理（权宜之计）会不断累积。

权宜之计留下真正的缺陷；每当缺陷造成新症状，就会再增加一项权宜之计。权宜之计越多，读者越难理解系统行为。

此外，权宜之计若留在代码中，Agent 可能把它当作正确实现的上下文，不断复制错误，代码库就无法继续充当可靠记忆（见第 4 章）。

修复原因起初可能更费时间，但能减少调试的总时间，并使后续所有任务持续受益。

规则：修复原因，不增加掩盖症状的防护判断

主要规则如下。

先复现缺陷

修复前，Agent 应亲自建立触发缺陷的条件，确认它实际发生。

例如，用户报告「连续按两次保存按钮会报错」，Agent 应先自己按两次，观察错误。不能复现，就无法确认之后是否修好。

持续追问「为什么」，直到找到根本原因

Agent 应持续追问「为什么」，直到找到根本原因。

以前述保存按钮为例，Agent 可以这样追问。

1. 为什么报错？→ 因为保存运行了两次
2. 为什么保存运行两次？→ 因为按过一次后，按钮仍可点击

3. 为什么按钮仍可点击？→ 因为保存期间没有禁用按钮

第三个答案是原因，因此 Agent 应在保存期间禁用按钮。

若停在第一个答案，Agent 可能只增加「忽略第二次保存」的处理，按钮仍可点击的根本原因没有解决。

若中途停止追问，就只能对症处理；因此应持续追问，直到找到根因。

不用防护判断掩盖症状

Agent 不应加一道防护判断来掩盖症状。

防护判断……当值可能导致问题时，停止或跳过剩余处理的简短检查，例如在函数开头写 `if (user === null) return;`。

例如，在值为 `null` 时崩溃的代码中加入「`null` 就不处理」，崩溃虽然消失，值为什么会变成 `null` 却仍不清楚。

因此，Agent 应修复值变成 `null` 的原因，而非只加防护判断。后者只掩盖症状，真正的缺陷仍在。

如果需要长篇注释解释权宜之计，就修改代码

如果必须用一整段注释来为权宜之计辩解，说明代码有问题。应修改代码，而非注释。

例如，对重复行缺陷作出如下改动，很可能只是权宜之计。

```
type Row = { id: string };
declare function writeRow(row: Row): void; // 向文件写入一行

const writtenIds = new Set<string>();

function writeRowOnce(row: Row) {
  // 如果中途发生重试，已写完的行可能再次传入。
  // 目前尚不清楚它为何再次传入。
  // 因此，记录已写入行的 ID；若同一 ID 再次传入，就丢弃而不写入。
  // 这样就不会导出重复的行。
  if (writtenIds.has(row.id)) return;
  writeRow(row);
}
```

```
writtenIds.add(row.id);
}
```

这项改动不再写出重复行，但注释没有解释为什么同一行会再次到达。

写下这类注释和代码前，Agent 应先追问为什么同一行会再次到达。

如果必须用长篇注释为修改辩解，就应改代码，而不是写注释。

修复所有具有相同模式的位置，而非只修一处

Agent 应搜索（用 `grep`）与原因具有相同模式的代码，修复所有找到的位置。

例如，一个页面的问题来自「按过一次后按钮仍可点击」，Agent 应检查其他页面的按钮是否采用相同写法。只修一处，其他地方就可能留下相同缺陷。

遇到阻碍时先测量，不靠猜测修改

进展受阻时，Agent 应添加日志或读取实际错误信息，确认发生了什么，而非凭猜测修改。猜测若错，只会再增加一层权宜之计。

遇到「重启后无法运行」的缺陷，Agent 应先怀疑重启后仍留存的旧状态（原文称 `stale persistent state`），再怀疑代码。例如：

- 配置文件
- 缓存
- 锁文件
- 保存在文件中的处理中间状态

如果删除保存中间状态的文件后就能运行，原因便是文件里留下的旧状态。

此时，Agent 不应删掉文件便结束，而应优先修复为：读取文件后验证状态是否符合当前代码预期。只删文件，下次留下旧状态仍会无法运行。

例如，旧代码把中间状态保存为 `{ step: 3 }`，当前代码预期的是 `{ lastRowId: "b" }`；状态验证可写成以下形式。

```
// 当前代码预期的中间状态结构
// 下方的 loadState 会拒绝旧的 { step: 3 } 结构
type SavedState = { lastRowId: string };
```



```
// 验证从保存文件读出的值是否符合 SavedState 结构
function loadState(raw: unknown): SavedState | null {
  if (
    typeof raw === "object" &&
    raw !== null &&
    "lastRowId" in raw &&
    typeof raw.lastRowId === "string"
  ) {
    return { lastRowId: raw.lastRowId };
  }
  // 不使用 { step: 3 } 这样的旧状态，而是从头开始
  return null;
}
```

如果运行结果会随前次执行留下的状态而变化，就需要验证读入的状态。

触发条件

调试时。

「Sequence Work into Verifiable Units」把工作拆成可验证的小单元逐步推进

「Sequence Work into Verifiable Units」要求 Agent 将工作拆成小单元，每完成一个单元就执行测试等检查，**所有检查通过后再进入下一个单元**。

例如，在五十个文件中改名，Agent 应在改完一个包后运行该包的测试，通过后再处理下一个包。

因为每个单元都验证，出错时就能在造成错误的单元中及时发现。

如果一次改完十个包才测试失败，Agent 就必须找出十项改动中哪一项造成问题，而且此时可能已在错误状态上继续做了更多工作。

逐个验证时，测试失败后只需查看刚改的一个包。

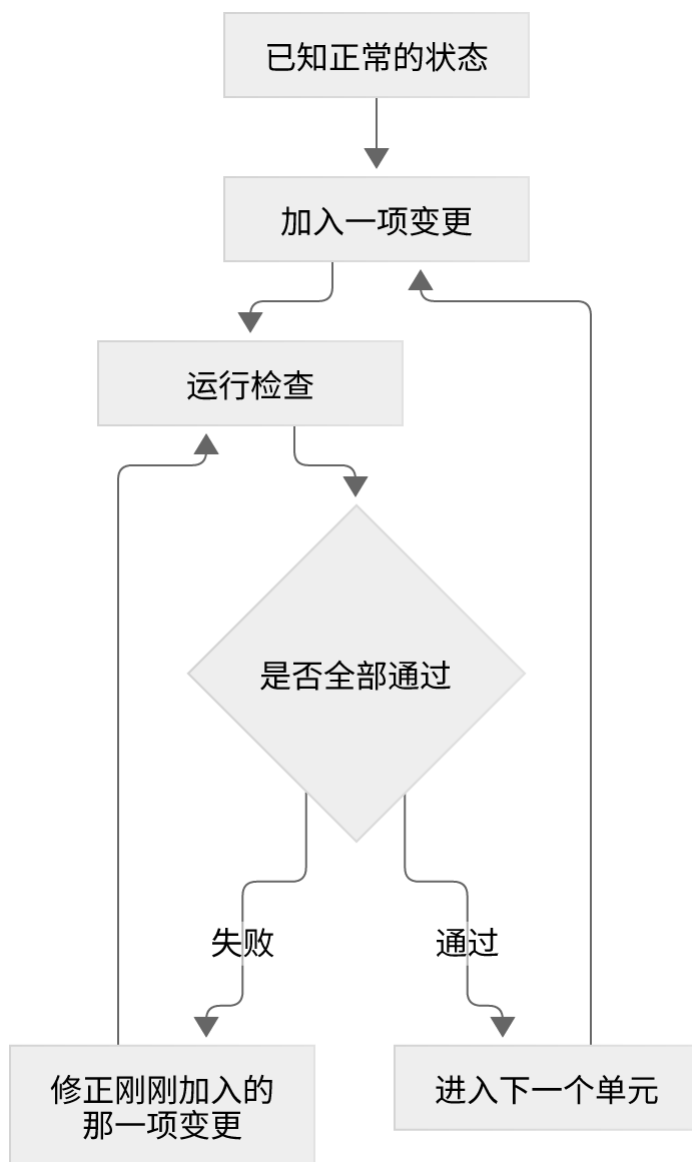
一次性验证的改动越多，排查原因的范围就越大，因此每做一个单元的改动都应验证。

规则：逐单元验证并推进，按证明正确性的顺序排列提交

工作推进规则如下。

每改一个单元，就验证一次

Agent 应从已知正常的状态开始，完成一项改动，运行检查，再继续。以上述改名为例，一个包就是一个单元。



查看图示 1

检查失败时，Agent 只需修复刚做的这一项改动，排查范围不会扩大。

即使 Agent 使用自动修改的工具（如第 17 章「**Build the Lever**」要求制作的工具），也不能跳过逐单元检查。工具让这类检查几乎不费事。

开始前先在干净的 trunk 上 rebase

开始工作前，Agent 应将分支 rebase（把分支提交重新叠放在其他提交上的 Git 操作）到最新且没有额外改动的 trunk（合并目标主线分支）上。

这样，每次检查都是在最新 trunk 这一真实基线（原文称 baseline）之上比较。

如果基于旧 trunk 或混有其他改动的分支工作，检查失败时就无法区分是自己的修改，还是原有内容造成的。

按证明正确性的顺序排列提交

Agent 应排列提交和 PR，使审阅者依序查看提交就能确认改动正确。基本顺序是「**失败的测试先行，其上再放修复**」。

这样排列，审阅者能亲眼看到测试由失败变成通过。

提交顺序（越靠下越早）

修复提交	← 在此提交中测试通过
↑	
失败测试的提交	← 在此提交中测试失败
↑	
trunk	

这条 Principle 还提出以下三种排列方式。

- 重组结构的提交之前，先放删除无用代码的提交。例如，先删未使用函数，再重组剩余文件。
- 改善性能的提交之前，先放记录改善前测量值的提交。例如，先记录当前处理时间，再修改代码加速。

- 实现功能的提交之前，先放只建设功能基础的提交。例如，先添加空页面和通向它的路由，再填入新页面内容。

Agent 应让每个提交都可以独立合入，并让整个提交序列像一份逐步证明改动正确的说明。

触发条件

工作有多个阶段（如批量修复、迁移或一系列相似编辑），或需要决定提交与 PR 排列顺序时。

「Test Behavior, Not Implementation」测试行为，而非实现细节

「Test Behavior, Not Implementation」规定：测试应以使用方相同的方式调用代码，并将使用方可观察的结果，与直接写在测试中的期望值比较。

- **行为 (Behavior)**代码使用者能看到的动作或结果，例如函数返回值或保存的数据。
- **实现 (Implementation)**代码内部如何调用函数等具体构造。
- **直接写在测试中的期望值**（原文称 literal expected value）……不通过代码计算、直接写出值本身的期望值，例如下例中的 "hello-world"。

例如，测试把字符串转为 URL 适用形式的 slugify，可以用以下两种写法（假定使用 Vitest 测试运行器）。

```
// 将字符串转换为适合 URL 的形式
declare function slugify(text: string): string;

// 测试实现细节：检查 slugify 内部是否调用 toLowerCase
const spy = vi.spyOn(String.prototype, "toLowerCase");
slugify("Hello, World!");
expect(spy).toHaveBeenCalled();

// 测试行为：像使用方一样调用 slugify，再与直接写出的预期值比较
expect(slugify("Hello, World!")).toBe("hello-world");
```

第一种写法只要 `slugify` 调用了 `toLowerCase` 就能通过，即使结果不是 `"hello-world"`；第二种写法在结果不是 `"hello-world"` 时就会失败。

之所以这样规定，是因为只检查代码调用过哪些函数，或仅复制代码中常量的测试，即使代码有缺陷也可能不失败。

这类测试既没有以使用方相同方式调用代码，也没有比较可观察结果；它们只占用 CI 时间和审阅者注意力，却发现不了缺陷。

而且，**仅复制常量的测试，在有人有意修改该常量时也会失败，妨碍正确的改动。**

例如，假设「每页显示数量」的配置值为 20，测试也写 20，只比较配置值是否为 20。

想把每页数量改为 30 时，将配置值改为 30，页面已经正确显示 30 项，测试却仍预期 20 而失败。要通过测试，只能再把测试中的 20 改为 30。

因此，只复制常量值的测试会因正确修改常量而失败，成为改动的阻碍。

规则：用「即使全部返回 `undefined` 也能通过吗」识别没有实际验证的测试

保留测试前，Agent 应问：「如果测试导入的函数全部只返回 `undefined`，它还能通过吗？」

如果能通过，测试就**完全没有观察到行为**，即使真实代码损坏也不会失败。此时 Agent 应重写断言，或删除测试。

- **断言**……在测试中确认结果是否符合预期的语句，例如 `expect(slugify("Hello, World!")).toBe("hello-world");`。
- **模拟对象**……测试中代替真实对象的假组件，例如由 `vi.fn()` 创建的假函数。

能通过上述问题的测试，主要有以下五种形式。

验证方式太弱

没有断言，或只用 `toBeDefined`（只检查值不是 `undefined`）等几乎任何结果都能通过的验证方式。

例如，对前述 slugify 写 `expect(slugify("Hello, World!")).toBeDefined()`，即使它返回的是 "xyz" 而非 "hello-world"，测试仍会通过。

```
// 将字符串转换成适合 URL 的形式
declare function slugify(text: string): string;

// 薄弱的测试：即使 slugify 返回 "xyz" 也能通过
expect(slugify("Hello, World!")).toBeDefined();

// 有力的测试：只要返回值不是 "hello-world" 就会失败
expect(slugify("Hello, World!")).toBe("hello-world");
```

只看是否被调用

只验证模拟对象被调用（`toHaveBeenCalled`），或结果为空（`toEqual([])`）。

例如，测试按姓名搜索会员的 `findUsers`，若只写 `expect(findUsers("存在しない名前")).toEqual([])`，那么即使 `findUsers` 已经坏到对任何姓名都返回空数组，测试仍能通过。

```
type User = { name: string };
declare function findUsers(name: string): User[]; // 按名称查找用户

// 只检查结果是否为空数组；即使错误实现永远返回 []，测试也能通过
expect(findUsers("存在しない名前")).toEqual([]);
```

Oh My Stack 项目注（代码示例）：测试输入中的日文表示「不存在的姓名」，作为查找参数保留。

用被测代码生成期望值

用被测代码自身计算期望值的测试。

例如，计算总金额的 `calcTotal` 忘记乘数量：两件各 100 日元的商品，本应返回 200，却只返回 100。

原测试在期望值一侧也调用相同的 `calcTotal`，所以无论它返回什么错误值，左右两侧始终相同，测试总能通过。

```

type Item = { price: number; quantity: number };
// 计算总金额（存在忘记乘以数量的缺陷）
declare function calcTotal(items: Item[]): number;

const items = [{ price: 100, quantity: 2 }];

// 修改前：预期值也用 calcTotal 计算，左右两边都是 100，因此测试通过
expect(calcTotal(items)).toBe(calcTotal(items));

// 修改后：直接在测试中写入预期值；100 与 200 不同，因此测试失败
expect(calcTotal(items)).toBe(200);

```

只复制常量

仅把代码中的常量、配置值或提示词字符串再写一遍，用于比较的测试。

例如，代码将 Agent 一次可使用的工具数量上限设为 8，测试只把 LIMITS.maxTools 中的 8 重写一遍。

```

// 代码：将工具数量上限设为 8
const LIMITS = { maxTools: 8 };

// 测试：只把代码中的 8 又抄写了一遍
expect(LIMITS.maxTools).toBe(8);

```

只查看测试自身准备的数据

测试只读取自己在准备阶段创建的数据，完全不调用被测代码。

```

it("keeps the user name", () => {
  const user = { name: "taro" }; // 测试自己创建的数据
  expect(user.name).toBe("taro"); // 完全没有调用待测试的代码
});

```

如何重写

用具体输入调用被测代码，与直接写出的期望值比较

重写测试时，Agent 应在测试中用一项具体输入调用被测代码，将输出值或可观察变化（如数据已保存）与直接写出的期望值比较。

例如，slugify 可写成 `expect(slugify("Hello, World!")).toBe("hello-world")`。

验证某事没有发生时，也验证另一项输入会使它发生

如果要验证某事不会发生，Agent 应在同一个测试中，另用一项输入验证它会发生。

例如，对 `findUsers`，也在同一测试中比较一个能够找到会员的姓名所得到的结果。

```
type User = { name: string };
declare function findUsers(name: string): User[]; // 按名称查找用户

expect(findUsers("存在しない名前")).toEqual([]);
expect(findUsers("taro")).toEqual([{ name: "taro" }]);
```

Oh My Stack 项目注（代码示例）：测试输入中的日文表示「不存在的姓名」，作为查找参数保留。

总是返回空数组的错误实现，会在第二条断言失败。

对常量，不复制其值，而是验证读取该常量并运行的处理

对于常量，Agent 应验证使用它的处理对某项输入产生的行为，而非重述常量值。

例如，「每页显示数量」可改写如下。

```
// 每页显示条数的配置值（当前为 20）
declare const PAGE_SIZE: number;
// 将数组分页；省略 pageSize 时使用 PAGE_SIZE
declare function paginate<T>(items: T[], pageSize?: number): T[][];

// 修改前：只复制了配置值；将配置改为 30 后测试就会失败
expect(PAGE_SIZE).toBe(20);
```



```
// 修改后：用一个输入检查分页行为
expect(paginate(["a", "b", "c"], 2)).toEqual([[ "a", "b" ], [ "c" ]]);
```

后一个测试在分页处理损坏时会失败，但把配置值改为 30 时不会失败。

找不到有效验证方式的测试应删除

如果找不到这样的验证方式，Agent 应删除该测试。

对模拟对象，验证收到的内容而非仅验证被调用

使用模拟对象本身没有问题；但测试应验证它接收到的内容或调用后的状态，而非只验证它被调用过。

例如，用模拟对象代替发送邮件的组件时，Agent 应验证传入了「正确的收件人和正文」，而非只验证「调用过发送」。只验证调用，内容错误也能通过。

```
type Mailer = { send: (mail: { to: string; body: string }) => void };
// 注册用户，并通过 mailer 发送注册感谢邮件
declare function registerUser(mailer: Mailer, user: { email: string }): Promise<void>;

// 修改前：只检查是否调用发送方法；收件人或正文错误时仍能通过
it("sends a welcome mail", async () => {
  const mailer = { send: vi.fn() };
  await registerUser(mailer, { email: "taro@example.com" });
  expect(mailer.send).toHaveBeenCalled();
});

// 修改后：将 mock 收到的收件人与正文和直接写出的预期值比较
it("sends a welcome mail", async () => {
  const mailer = { send: vi.fn() };
  await registerUser(mailer, { email: "taro@example.com" });
  expect(mailer.send).toHaveBeenCalledWith({
    to: "taro@example.com",
    body: "ご登録ありがとうございます",
  });
});
```

Oh My Stack 项目注（代码示例）：邮件正文中的日文表示「感谢注册」，作为 mock 所检查的输出值保留。

这段测试中，即使 registerUser 把空地址作为收件人，原测试也能通过；修改后的测试会因收件人不是 taro@example.com 而失败。

触发条件

编写、修改测试，或决定是否保留测试时。

示例：用行为验证重试

以重复行请求为例，比较「只看是否被调用」的测试及其改写方式。重复行请求就是本章开头的以下请求。

```
/poteto-mode the export writes duplicate rows when a retry lands mid-run. repro first, then fix and verify.
// 如果重试发生在执行中途，导出会写入重复的行。先复现，再修复，然后验证。
```

Agent 收到请求后，要编写重试测试。

下面的测试只要 exportWithRetry 调用了 db.query，即使没有正确写出行，也能通过。

```
type Db = { query: (...args: unknown[]) => unknown };
type InMemoryDb = Db & { rows: (table: string) => { source_row_id: string }[] };

// 导出原始行 input；中途失败时重试
declare function exportWithRetry(
  db: Db,
  input: string[],
  options?: { failAfterRows?: number },
): Promise<void>;

// 为测试创建内存数据库
declare function createInMemoryDb(): InMemoryDb;
// 原始行 (a、b、c 共三行)
declare const input: string[];

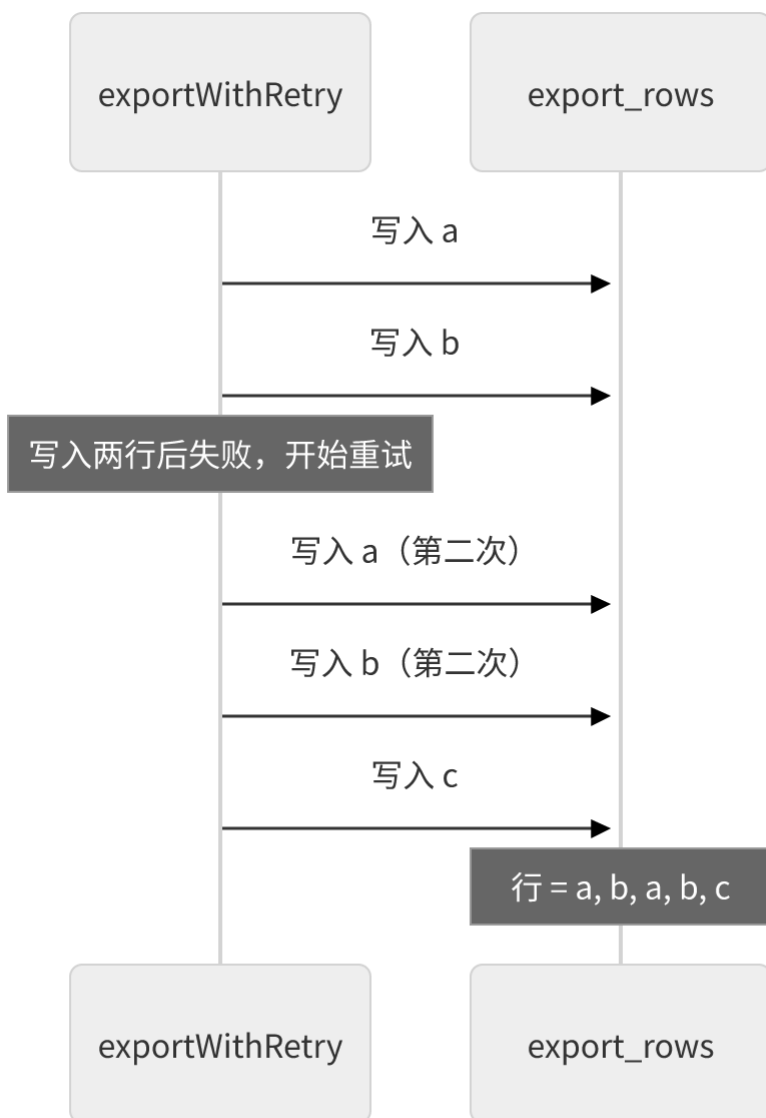
// 错误的测试：只检查 mock 是否被调用
it("writes rows on retry", async () => {
  const db = { query: vi.fn() };
  await exportWithRetry(db, input);
  expect(db.query).toHaveBeenCalled();
});
```

下一段测试将使用方可观察的结果，也就是实际写出的行，与直接写在测试中的期望值 ["a", "b", "c"] 比较。

```
// 验证行为的测试
it("writes each source row once when a retry lands mid-run", async () => {
  const db = createInMemoryDb();
  await exportWithRetry(db, input, { failAfterRows: 2 });
  expect(db.rows("export_rows").map((r) => r.source_row_id)).toEqual(["a", "b", "c"]);
});
```

`failAfterRows: 2` 指定写出两行后失败一次，使重试在运行中途发生。

如果重试从第一行重新写入，结果会如下变化。



查看图示 2

最初写入的 a、b 后面接着重试从第一行写出的 a、b、c，实际行就成为 ["a", "b", "a", "b", "c"]，与期望的 ["a", "b", "c"] 不同，测试会失败。

因此，测试能否捕捉缺陷，取决于比较的对象；应将可观察结果与直接写出的期望值比较。

在重复行请求中，「Fix Root Causes」和「Prove It Works」应用于「Bug fix」Playbook 的各个步骤

再看一次开头的重复行请求。

```
/poteto-mode the export writes duplicate rows when a retry lands mid-run. repro first, then fix and verify.
// 如果重试发生在执行中途，导出会写入重复的行。先复现，再修复，然后验证。
```

本书认为，在这个请求中，第 11 章介绍的「**Bug fix**」Playbook，主要会在各步骤应用两条 Principle。

请求中的「先复现」对应「**Fix Root Causes**」，「再验证」对应「**Prove It Works**」。

「Bug fix」的步骤	使用的 Principle 及其如何改变判断
1. 亲自复现	「 Fix Root Causes 」：建立在运行中途触发重试的条件，逐步收窄范围直到复现重复行；复现后才编写修复
2. 通过二分查找缩小原因范围	「 Fix Root Causes 」：提出可能原因，每次将怀疑范围缩小一半，加入日志，逐个检验候选原因
3. 规划修复	「 Fix Root Causes 」：输出后再删除重复行，只是处理症状，未修复重复产生的原因。如果打算让写入幂等，应使用第 18 章「 Make Operations Idempotent 」的三个问题检查写入设计
4. 在相同环境以相同操作验证	「 Prove It Works 」：不只看单元测试通过，还要让原先的复现流程通过，并检查实际写出的行
5. 先提交会失败的复现	「 Sequence Work into Verifiable Units 」：在 Git 历史中保留测试从失败到通过的变化
回答	「 Prove It Works 」：写明根本原因与修复，并附上复现步骤在修复前失败、修复后通过的输出

在步骤 2 中，Agent 要检查的可能原因例如以下三种。

- 队列（待处理任务的排列）把同一任务交付了两次
- 重试没有从已写入行之后继续，而是从已写过的行重新开始
- 写入并非幂等（见第 18 章）
 - 幂等……同一操作执行多少次，结果都与只执行一次相同

总结

- 「**Prove It Works**」不把「编译通过」「Agent 如此报告」当作证据，而是直接检查实际成果物（输出文件或运行中的界面）。
- 「**Fix Root Causes**」先复现缺陷，再不断追问「为什么」直到找到原因；不靠防护判断掩盖症状。
- 「**Sequence Work into Verifiable Units**」逐个小单元验证测试等检查通过，并让失败复现的提交排在修复提交之前。
- 「**Test Behavior, Not Implementation**」用「即使全部返回 undefined 也能通过吗」识别发现不了任何缺陷的测试。
- 在重复行请求中，「**Fix Root Causes**」用于「**Bug fix**」从复现到规划修复的步骤，「**Prove It Works**」用于验证与回答。

下一章第 20 章介绍 Delegation 组的两条 Principle：「**Guard the Context Window**」与「**Never Block on the Human**」。

第 20 章：Delegation，处理委派与并行工作的两条原则

kaito · 来源：日本語 · English · 2026-10-03

本章讨论以下两项原则。

1. Guard the Context Window
2. Never Block on the Human

这两项属于 Delegation（委派）组，规定了 Agent 将工作交给子 Agent，或不等待人类回复而继续工作时如何判断。

随附指南的 [docs/guide/08-principles.md](#) 将两者概括为**让并行工作保持有序的原则**（原文为「keep parallel work sane」）。

它们都帮助负责委派的 Agent 持续推进并行工作，避免子 Agent 的大量输出填满上下文，也避免因等待人类回复而停下。

运用这两项原则时，Agent 会这样行动。

- 让子 Agent 阅读数千行测试日志或数百页 API 规格文档，主线程（调用子 Agent 的原始会话）只接收几行摘要，例如「哪些测试失败、为什么失败」
- 对于重命名函数等可以撤销的工作，不向人类询问「可以改吗」，而是执行后展示结果和理由
- 只有向共享分支 force-push（覆盖远程分支历史的推送）等无法撤销的操作，才在执行前向人类确认

此外，**两项原则保护的资源不同**。「Guard the Context Window」保护 Agent 的上下文，避免原始数据（例如数千行测试日志、截图、大型文档等未经摘要直接读入的输出）将其填满。

这类数据的大部分内容不会用于判断，却会按读入量占用上下文，因此需要这项原则保护 Agent 的上下文。

「Never Block on the Human」保护人类的注意力，使其不用每次都被 Agent 的批准请求打断。

这两项原则也用在 Playbook 的步骤中。

例如，第 15 章的「**Session pickup**」Playbook 让子 Agent 阅读很长的会话记录（「Guard the Context Window」）。第 16 章的「**Multi-phase or multi-PR plan**」Playbook 不会把运行原型即可回答的问题拿去问使用者（「Never Block on the Human」）。

本章与第 17 章一样，从规则和触发条件两个角度逐项解释，最后讨论它们与演讲主题「人类无需在旁边盯着」的关系。

本章没有为这两项原则提供请求示例。

本书只采用 pstack 随附指南或 README 中已有的请求示例。如果为原文未提供示例的原则自行编写示例，就可能展示 pstack 并未设想的使用方式。

本章结构

本章结构如下。

- 两项原则一览
- 「Guard the Context Window」让子 Agent 阅读大量输出或文档，只在主线程留下摘要
- 「Never Block on the Human」不等待人类确认，直接推进可以撤销的工作
- 保护 Agent 的上下文和人类的注意力，人类便无需在旁边盯着
- 总结

两项原则一览

Principle	一句话结论	保护的资源
「Guard the Context Window」	让子 Agent 阅读大量输出或文档（如数千行日志、数百页规格文档），主线程只保留摘要	Agent 的上下文

Principle	一句话结论	保护的资源
「Never Block on the Human」	无需等待确认即可推进可以撤销的工作（如编写代码、重命名函数），并向人类展示结果和理由；只有无法撤销的操作（如 force-push、删除生产数据）才需要执行前确认	人类的注意力

「Guard the Context Window」让子 Agent 阅读大量输出或文档，只在主线程留下摘要

「Guard the Context Window」原则要求逐一判断放入主线程的内容是否值得其成本。这里的成本是主线程读入的 token（AI 模型计量文本的单位）数量。

值得其成本……相对于主线程读入的 token 数量，内容能帮助 Agent 作出下一步判断

例如，用几行概括「哪些测试失败、为什么失败」，Agent 可以直接据此决定下一步修复什么，因此值得其成本。

相反，若将包括通过测试在内的数千行日志原样读入主线程，Agent 用于判断的只有极少部分，就不值得其成本。

原则提出这一要求，是因为上下文窗口（模型一次能够读入和处理的文本量上限）有限，而且在同一次会话中，已用掉的容量无法重新腾出。

上下文窗口填满后，Agent 的推理质量会下降。随后，会话压缩（将放不下的会话内容摘要压缩）可能漏掉先前读过的部分内容，最终工作本身也会停止。

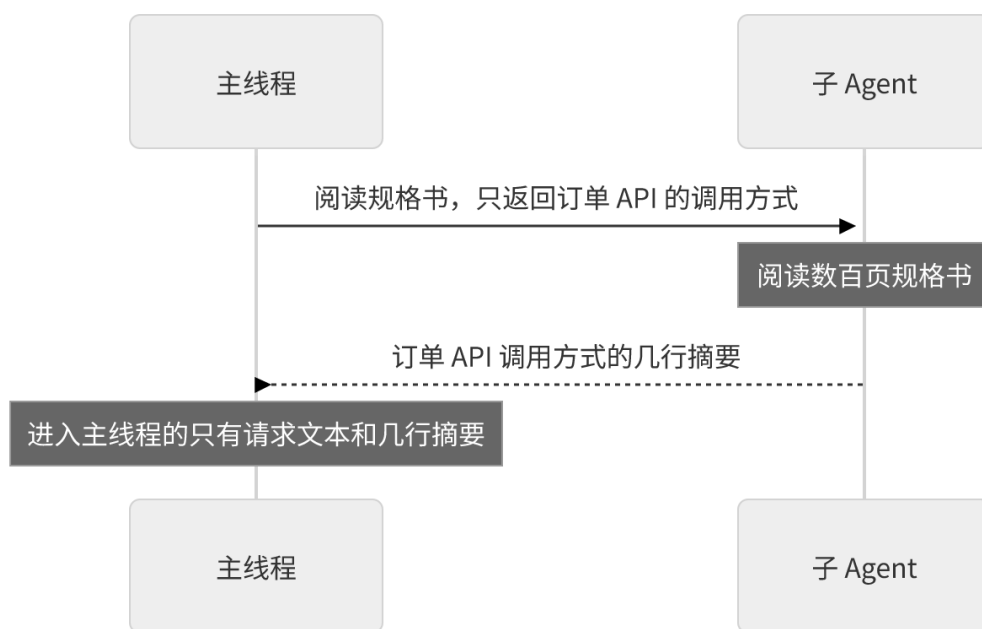
规则：只把值得其成本的内容放入主线程

原则列出以下三种做法。

1. 让子 Agent 阅读大量输出或文档

Agent 让子 Agent 阅读冗长的输出（如数千行测试日志）、截图和大型文档，主线程只接收摘要，不接收原始数据。因为主线程阅读原始数据会相应占用其上下文。

以阅读大型文档时如何分工为例。如果只想从数百页的 API 规格文档中了解如何调用订单 API，Agent 会这样分配工作。



查看图示 1

阅读数百页规格文档填满的只是子 Agent 的上下文。主线程的上下文仅消耗请求文本和几行摘要所占的容量。

因此，若在主线程阅读大型文档会占用大量上下文，就需要让子 Agent 阅读。

2. 将每次都用到的模板和参考资料放在 SKILL.md 正文中

Agent 应把每次使用 Skill 都要阅读的模板和参考资料写在 SKILL.md 正文中，不另存文件。否则每次使用 Skill 时，读取额外文件都会增加成本。

例如，一个审查 Skill 每次都按相同格式写报告，文件可以这样安排。

修改前：把每次使用的报告模板放在单独文件中

skills/review/SKILL.md ← 步骤以及「按 templates/report.md 的格式编写报告」这一句话

skills/review/templates/report.md ← 报告模板

修改后：把每次使用的报告模板写进 SKILL.md 本体
 skills/review/SKILL.md ← 步骤和报告模板

采用前一种方式，Agent 每次使用 Skill 都要先读 SKILL.md，再读 templates/report.md。采用后一种方式，只读一次 SKILL.md 即可。

因此，如果文件位置会改变每次读取的次数，就应把每次都要用的内容放在 SKILL.md 正文中。

3. 给每个阶段的工作范围设定上限

Agent 将大型工作划分为阶段（原文为 phase），并为每个阶段规定处理的文件数量和允许使用的轮次数量上限。轮次指 Agent 向模型提出请求并接收回复的一次往返。

Agent 还要事先估算工作机制本身占用的上下文，例如调用子 Agent 时，发给子 Agent 的请求及其返回的摘要也会占用主线程的上下文。

如果既不设上限，也不计入机制本身的开销，上下文可能在某个阶段尚未完成时就已填满。Agent 的推理质量可能随之下降，工作也可能中止。

判断标准是内容是否值得其成本

原则的标准不是减少阅读量，而是内容是否值得其成本。第二种做法体现了这一点。

把模板放入 SKILL.md 正文会使文件变长，单次阅读量也会增加。

原则仍选择放在正文，是因为每次都用到的模板，即便拆成单独文件，最终也每次都要读。读入的内容没有变，却多了一次读取，所以放在正文中成本更低。

触发条件

大量输出、长文件或重复阅读同一文件等情况使上下文逐渐填满时，就会触发这项原则。计划把工作分给多个子 Agent 并行推进（原文为 fan-out）时也适用。

Playbook 示例：要求调查代码库的子 Agent 只返回四类内容

「**Multi-phase or multi-PR plan**」Playbook（第 16 章）在编写计划前，将调查代码库的工作交给子 Agent。它引用这项原则，**规定子 Agent 只返回以下四类内容。**

- 相关文件的位置

- 代码库的惯例
- 测试命令
- 处理入口（原文为 entry points，即所调查功能开始执行的函数或文件，例如最先接收 API 请求的处理函数、接收 CLI 命令的函数）

Playbook 还禁止子 Agent 原样粘贴读过的代码（原文为 inlined dumps）作为回复。

把返回内容限定为这四类，主线程便只会收到编写计划所需的信息。

「Never Block on the Human」不等待人类确认，直接推进可以撤销的工作

「Never Block on the Human」原则要求 Agent 对可以撤销的工作**直接执行，无需请求许可**，然后向人类展示结果和理由。人类看到结果后，可以再修正工作方向（原文为 course-correct）。

原则开头的两句话是：

The human supervises asynchronously. Agents must stay unblocked.

人类以异步方式监督。Agent 必须能够继续推进。

也就是说，人类不必现场陪同 Agent 工作，而是在事后检查结果，因此 Agent 不应因等待人类回复而停下。

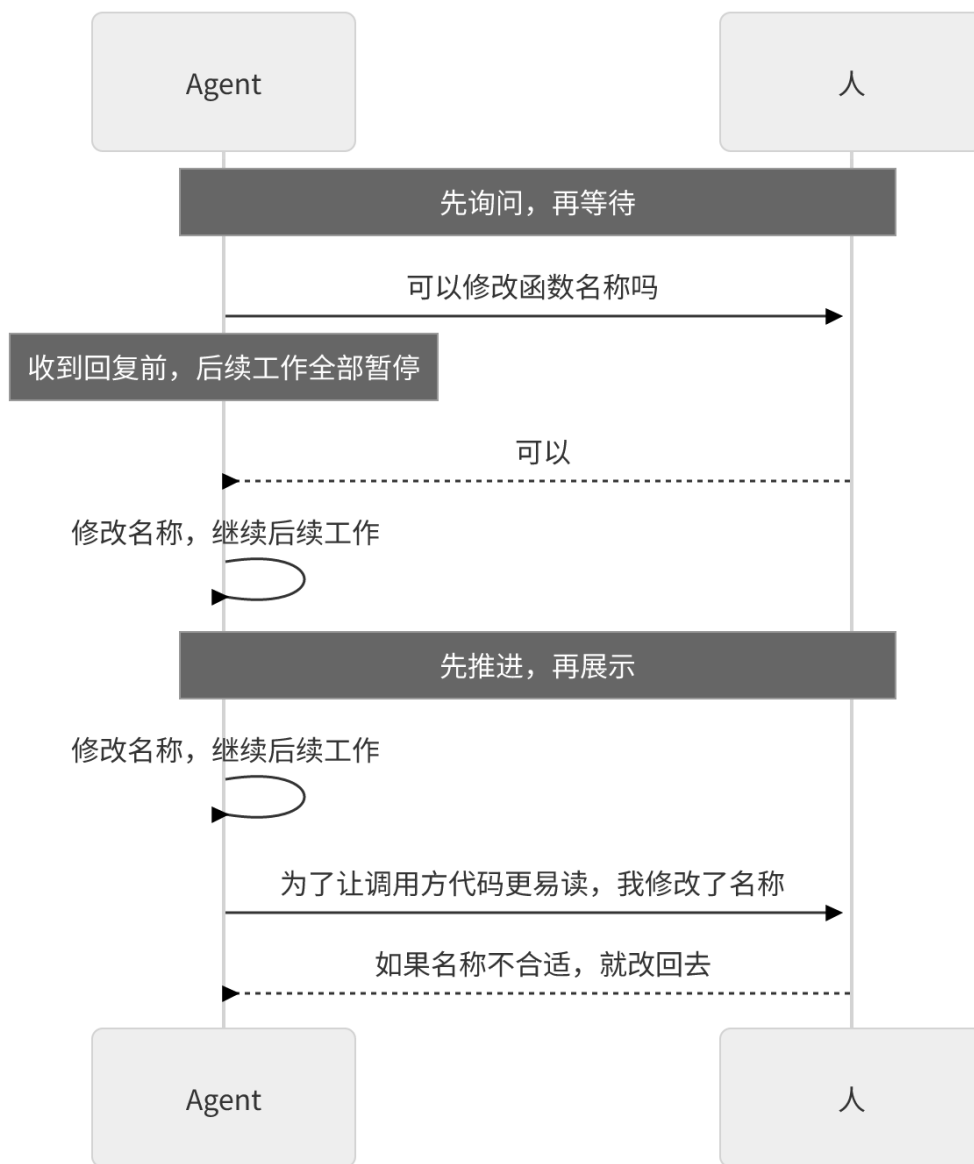
原则之所以如此要求，是因为**每当 Agent 停下来等待人类许可，工作流程（原文为 pipeline）就会中断，人类回复的速度便决定了整个工作的进度。**

代码变更可以撤销，也可以审查，所以**错误判断的成本通常低于停下来的成本**。判断错了，人类在审查时发现并撤销变更即可；而停下来则意味着后续所有工作都要等人类回复才能继续。

规则：只有无法撤销的操作才向人类确认

原则要求 Agent 不要问「可以做 X 吗」，而是先执行 X，再展示结果和理由（原文为 Proceed, then present）。

例如，Agent 犹豫是否要重命名函数时，「询问并等待」与「先推进再展示」的区别如下。



查看图示 2

如果先询问并等待，人类回复前工作便会停滞。如果先推进再展示，即便名称不合适，人类也只需在审查时回复「改回去」。

因此，对可以撤销的工作，询问并等待会使工作停下，应先推进再展示。

此外，**原则还要求机制能够自行逐步修复**（原文为 self-healing）。Agent 在工作途中发现问题时，应记录问题，并在下一轮工作中修复，无需等待人类指示。每次发现问题都等待指示，工作就会一再停下。

例如，Agent 在工作途中发现另一项测试偶尔失败，应记录该问题，并在下一轮工作中修复。报告问题后停下来等人类指示，不属于 self-healing。

可以不等确认就推进的依据，是变更能够撤销。
无法撤销的操作一旦出错就无法挽回，所以 Agent 在执行前要向人类确认。

根据操作类型，处理方式分为以下三种。

类型	处理方式	示例
无法撤销的操作	需要确认	force-push、删除生产数据、向外部发送消息
可以撤销的操作	不停下，继续推进	编写代码、编辑笔记、拆分任务
产品方向	由人类决定要开发什么；方向确定后，实施过程不停下	决定开发哪项功能

触发条件

当 Agent 面对可以撤销的工作，想问「可以做 X 吗」的时候。

/poteto-mode 的两处规定明确划分了哪些操作要停、哪些可以继续

启动工作时使用的 Skill /poteto-mode 有两个小节，具体规定了 Agent 遇到哪些操作要停下来、遇到哪些操作可以继续。

- 「Autonomy」一节要求：在向共享分支 force-push、部署、删除数据、向客户发消息等无法撤销的写入之前，必须停下来。另一方面，可以撤销的工作，以及在团队聊天中发帖或更新工单等外部操作，都应直接推进，不必询问（第 35 章）。
- 「Non-negotiables」一节（第 8 章）要求：**如果答案是可以通过运行来观察的事实，例如行为、耗时、页面布局或输出，Agent 应运行原型来判断，而非询问人类。**只有任何实验都无法决定的产品或偏好问题才问人类（第 38 章）。

「Never Block on the Human」原则不区分收件人，将向外部发送消息一律视为无法撤销的操作，要求执行前向人类确认（见「规则：只有无法撤销的操作才向人类确认」表格第一

行)。而「Autonomy」一节按接收对象区分外部操作：给客户发消息属于要停下来的操作，在团队聊天中发帖则可以继续推进。

也就是说，「Autonomy」一节的划分更细。

保护 Agent 的上下文和人类的注意力，人类便无需在旁边盯着

演讲中提出的一个观点是：只要环境建设得足够完善，人类就无需一直在旁边盯着 Agent。

本书认为，Agent 遵循 Delegation 的两项原则时，**人类就不必在旁边盯着**。原因有两个。

- 如果原始数据没有填满 Agent 的上下文，即使工作持续很久，推理质量也不易下降，工作也不易中断。因此人类无需频繁查看进度。
- 如果 Agent 不询问就推进可以撤销的工作，人类便无需现场回答「可以做吗」等批准请求。人类只要在方便时阅读 Agent 展示的结果和理由，再作判断即可。

总结

- 「**Guard the Context Window**」……保护 Agent 的上下文。让子 Agent 阅读数千行日志等大量内容，主线程只留下摘要。判断标准是「是否值得其成本」，不是「是否减少阅读量」；因此每次都用的模板即使会让 SKILL.md 变长，也应放在正文中。
- 「**Never Block on the Human**」……保护人类的注意力。Agent 推进编写代码等可以撤销的工作后，再展示结果和理由；只有 force-push、删除生产数据等无法撤销的操作，以及决定开发哪项功能等产品方向，才需要向人类确认。
- **保护上下文和注意力**……人类无需在旁边盯着，只要阅读 Agent 展示的结果和理由，再作判断即可。

下一章第 21 章介绍 Meta 组的原则「**Encode Lessons in Structure**」。

第 21 章：Meta，改进机制本身的原则

kaito • 来源：日本語 • English • 2026-10-03

本章讨论以下一项原则。

1. Encode Lessons in Structure

这项原则要求把反复出现的纠正（多次修正同样的错误）写进机制（工具、代码、元数据、自动化），而不是写成文字指令。

当 Agent 发现自己要第二次写下同一条提醒时，应停止增加文字提醒，**改用 lint 规则或运行时检查，让机器来执行。**

运行时检查……程序运行时检查是否遵守规则的代码

例如，在显示日期之前检查它是否符合 2026/09/26 格式，可以使用以下代码。

```
// 在页面上显示的日期字符串
declare const text: string;

// 运行时检查：在显示前验证日期格式
if (!/^\d{4}\d{2}\d{2}$/.test(text)) {
  throw new Error(`日付の形が違います: ${text}`);
}
```

Oh My Stack 项目注（代码示例）：异常消息中的日文表示「日期格式不正确」，后面附带原始 text；作为运行时输出保留。

只有程序实际执行到这里，这段代码才会发现日期格式不对。

例如，Agent 可以将「不要从外部 import internal/ 中的模块」这条提醒改为检查 import 的 lint 规则。

随附指南的 [docs/guide/08-principles.md](#) 将这项原则单独归为 meta（元）原则。

它的作用与其余 22 项原则不同。

其他原则规定具体工作中的判断，例如修复原因而不是症状、可以撤销的工作无需等待确认即可推进。

而「Encode Lessons in Structure」把工作中发现的教训转化为后续工作的机制，例如 lint 规则。

这项原则把**错误、人类的纠正、意外结果都视为学习信号**（原文为 learning signal）。

Agent 捕捉到学习信号后，应将其送到合适的位置（例如把反复出现的纠正变成 lint 规则），不能止于记录，而要立即修复或创建具体的待办事项（详见「[反馈循环：将纠正转化为机制的三个阶段](#)」）。

演讲也提到，不要让纠正带来的教训只留在会话里，应以可复用的形式存入仓库（第 3 章、第 4 章）。

本章将依次讲解这项原则的规则、触发条件，以及 pstack 自身如何运用它。

本章没有为这项原则提供请求示例。

本书只采用 pstack 随附指南或 [README](#) 中已有的请求示例。如果为原文未提供示例的原则自行编写示例，就可能展示 pstack 并未设想的使用方式。

本章结构

本章结构如下。

- 规则：将反复出现的纠正转化为最强的机制，并删除文字指令
- 触发条件
- pstack 自身就是按这项原则构建的
- 总结

规则：将反复出现的纠正转化为最强的机制，并删除文字指令

「**Encode Lessons in Structure**」原则要求用机制执行反复出现的提醒，而不是依赖文字指令。

Agent 发现同一提醒写了两次，就要**建立执行提醒的机制，并删除原来的文字指令**。

原则之所以提出这一要求，是因为**文字指令容易被忽略**。

读者必须看到文字、记住内容并照做，指令才会得到遵守。

对 Agent 而言，换一次会话，上次会话中收到的纠正就不会自动传承。即便 Agent 回答「我会记住」，这一承诺也不会留到下一次会话。

机制不依赖 Agent 的记忆也能运作。因此，用机制取代文字指令，可以让下一次会话中的 Agent 也遵守同一规则。

步骤：建立机制后删除文字指令

步骤有三个。

1. 问自己能否把指令变成 lint 规则、元数据标志、运行时检查或脚本
2. 如果可以，就建立机制并删除指令
3. 如果不可以（机器无法判断，仍需要判断力），就突出这条指令，并附上违反指令的失败示例

元数据标志……放在文件开头设置区（frontmatter）的值

例如，pstack 各项 Principle 的 SKILL.md 在文件开头的设置区写有以下值。

```
---
name: principle-encode-lessons-in-structure
disable-model-invocation: true
---
```

其中 `disable-model-invocation: true` 就是元数据标志。

这个标志禁止 Agent 根据会话内容自行选择并运行该 Skill。它通过设置区的值来决定这一点，而不是在正文写下「不要主动使用这个 Skill」。

能由机器判断的指令，改为 lint 规则并删除文字

开头的「不要从外部 import internal/ 中的模块」属于步骤二。

如果 Agent 在规则文件（例如记录 Agent 规则的 AGENTS.md）里看到下面这句话，就应增加检查 import 的 lint 规则，并从规则文件中删除这句话。

AGENTS.md

修改前：在规则文件中用文字说明

- 不要在 internal/ 外部 import internal/ 中的模块

eslint.config.js

```
// 修改后：添加以下配置，并删除规则文件中的文字（假设已引入 typescript-eslint）
import tseslint from "typescript-eslint";

export default [
  {
    files: ["src/**/*.ts"],
    languageOptions: { parser: tseslint.parser },
    // 允许 internal/ 内的模块互相 import
    ignores: ["src/internal/**"],
    rules: {
      "no-restricted-imports": ["error", {
        patterns: [{
          group: ["**/internal/**"],
          message: "internal/ のモジュールは、internal/ の外から import しないでください",
        }],
      }],
    },
  },
];
```

Oh My Stack 项目注（代码示例）：规则配置中的日文错误消息表示「不要从 internal/ 外部导入 internal/ 的模块」，作为 lint 输出值保留。

有了这项设置，`internal/` 之外的文件若 `import internal/` 的模块，`lint` 就会报错。因此，无需再把同一提醒留在规则文件中。

也就是说，**机器能够判断的指令可以交给机制执行，文字指令便不再需要。**

机器无法判断的指令，应突出显示并附上失败示例

相反，「错误消息要让用户知道下一步该做什么」属于步骤三。

机器很难判断消息写得好不好，因此 Agent 应保留这条文字指令，放在显眼处，并附上糟糕消息的示例。

AGENTS.md

```
## 必须遵守
```

- ****编写错误消息时，要让用户知道下一步该做什么****
- 失败示例：「发生错误」。用户不知道接下来该怎么做

机器无法判断的指令只能以文字保留，因此需要突出显示，并附上失败示例。

能靠机制修复，就只使用机制

原则要求：**如果机制可以解决问题，就只用机制。**

同时保留机制和文字指令，意味着在两个地方维护同一规则；只更新其中一个，两者就会冲突。因此原则要求建立机制后删除文字指令。

例如只修改 `lint` 规则，却忘记修改规则文件，读过规则文件的 Agent 仍可能按照旧规则编写代码。

反复写下同一指令，是机制不足的症状

原文把文字指令称为「症状」（`symptom`）。

本书将反复写下同一指令理解为：执行规则的机制不足所呈现的症状。

据此可以认为，原则要求**建立缺失的机制以解决原因，而不是继续增加作为症状的指令。**

强度排序：选择可行方案中最强的机制

如果有多种机制可用，Agent 应选择当前情况允许的最强方案。

按强度从高到低，有以下四类机制。

1. **无法表示的状态**（原文为 unrepresentable state）……用类型让违反规则的状态无法写出来。违反规则时会发生编译错误。
2. **使 CI 失败的 lint 或禁用 API**……lint 检测到违反规则的写法或被禁止的 API 时，CI 失败，变更无法合入。
3. **标准辅助函数**……提供封装了规定写法的共用函数（如稍后出现的 `formatDate()`），让代码调用该函数。
4. **运行时检查**……运行时检查条件。即使代码违反规则，也要执行到相关部分才会发现。

选择较强机制，是因为**Agent 会模仿周围代码已有的写法来编写代码**（第 4 章）。

如果放入较弱的防护机制（原文为 `guard`，即执行规则的机制中排名较低的方案），它就会成为下一个 Agent 模仿的样板。

例如，只用运行时检查保证日期显示格式，就可能让较弱的机制成为样板。

订单列表页面自行格式化日期，并在显示前检查它是否符合 2026/09/26 格式。下一个 Agent 开发发货列表页面时，模仿这段代码，同样自行格式化日期，也加上同样的运行时检查。

```
type Order = { id: string; orderedAt: Date };
type Shipment = { id: string; shippedAt: Date };

// 订单列表页面：自行格式化日期，并在显示前检查格式
function orderRow(order: Order): string {
  const d = order.orderedAt;
  const mm = String(d.getMonth() + 1).padStart(2, "0");
  const dd = String(d.getDate()).padStart(2, "0");
  const text = `${d.getFullYear()}/${mm}/${dd}`;

  if (!/^d{4}\d{2}\d{2}$/.test(text)) {
    throw new Error(`日付の形が違います: ${text}`);
  }

  return `${order.id} ${text}`;
}

// 发货列表页面：下一个 Agent 模仿上述代码，重复编写相同的格式化逻辑和运行时检查
function shipmentRow(shipment: Shipment): string {
  const d = shipment.shippedAt;
```

```

const mm = String(d.getMonth() + 1).padStart(2, "0");
const dd = String(d.getDate()).padStart(2, "0");
const text = `${d.getFullYear()}/${mm}/${dd}`;

if (!/^\d{4}\d{2}\d{2}$/.test(text)) {
  throw new Error(`日付の形が違います: ${text}`);
}

return `${shipment.id} ${text}`;
}

```

Oh My Stack 项目注（代码示例）：异常消息中的日文表示「日期格式不正确」，后面会附上原始 text；作为运行时输出保留。

如果改为提供标准辅助函数 `formatDate()`，并通过 lint 禁止绕过它自行格式化日期，下一个 Agent 模仿的就会是调用 `formatDate()` 的写法。

```

type Order = { id: string; orderedAt: Date };
type Shipment = { id: string; shippedAt: Date };

// date.ts: 只在标准辅助函数 formatDate() 中格式化日期
function formatDate(d: Date): string {
  const mm = String(d.getMonth() + 1).padStart(2, "0");
  const dd = String(d.getDate()).padStart(2, "0");
  return `${d.getFullYear()}/${mm}/${dd}`;
}

// 订单列表页面
function orderRow(order: Order): string {
  return `${order.id} ${formatDate(order.orderedAt)}`;
}

// 发货列表页面: 下一个 Agent 也模仿上述代码，调用 formatDate()
function shipmentRow(shipment: Shipment): string {
  return `${shipment.id} ${formatDate(shipment.shippedAt)}`;
}

```

```

// 在 eslint.config.js 中添加以下配置，禁止在 date.ts 外格式化日期（假设已引入 typescript-eslint）
import tseslint from "typescript-eslint";

export default [
  {
    files: ["src/**/*.ts"],

```

```

languageOptions: { parser: tseslint.parser },
ignores: ["src/date.ts"],
rules: {
  "no-restricted-properties": ["error", {
    property: "getFullYear",
    message: "日付の整形には formatDate() を使ってください",
  }],
},
},
];

```

Oh My Stack 项目注（代码示例）：规则配置中的日文消息表示「请使用 formatDate() 格式化日期」，作为 lint 输出值保留。

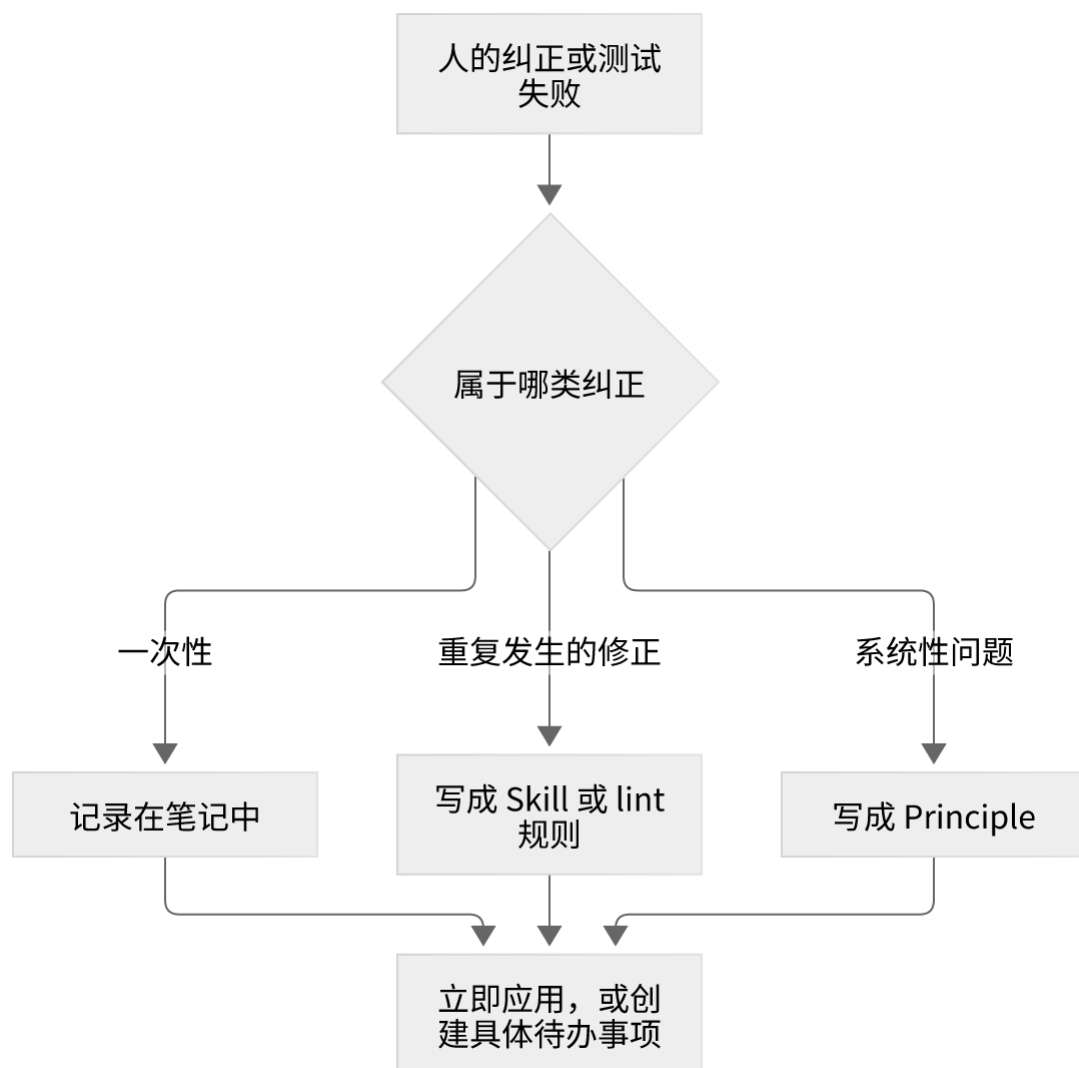
这样，日期格式化会集中在 formatDate() 一处，添加页面时也不再重复增加同样的运行时检查。

因此，下一个 Agent 会模仿已有机制的写法，应该选用可行方案中最强的机制。

反馈循环：将纠正转化为机制的三个阶段

Agent 将纠正转化为机制，要经过以下三个阶段。

- **及时捕捉纠正**……人类介入工作并纠正，或测试失败时，Agent 判断这次纠正是一次性问题，还是会反复出现的模式。
- **把纠正送到正确层级**（原文为 Route to the right layer）……一次性问题写入笔记（原文为 brain note）；反复出现的纠正写入 Skill 或 lint 规则；整个系统的问题则写入 Principle。
- **把纠正处理到底**……不能只记录，要么立即应用（如立即添加 lint 规则），要么创建具体的待办事项（如「添加禁止 import internal/ 的 lint 规则」）。



查看图示 1

如果在三个阶段中的任何一处停下，纠正就不会反映到下一次工作中。

原文列出以下三种应避免的反面做法。

- **只是承认，没有记录**……回答「我会记住」，这句话也不会留到下一次会话。
- **只是记录，没有送到正确层级**……该写成 lint 规则的内容只写在笔记里，若不实现 lint 规则，下一个 Agent 再犯同样的错误，CI 也不会失败，还得再纠正一次。这样的笔记便没有发挥作用。

- **只修复一个实例，没有归纳规则**……只修正一处从外部 `import internal/` 的代码，却不添加 lint 规则，下一个 Agent 仍会在别处写下同样的 `import`。修复一个实例，并没有消除产生同类错误的模式。

触发条件

发现自己第二次写下同一指令，或发现同一纠正反复出现的时候。

编写脚本时，如何区分另外两项相似的原则

这项原则与「**Build the Lever**」「**Prove It Works**」都可能促使 Agent 编写脚本。

「Build the Lever」原文这样区分三项原则。

- 「Build the Lever」用于更快地完成眼前工作，并让结果更易审查
- 「Prove It Works」把验证本身写成脚本
- 「Encode Lessons in Structure」把反复出现的指令变成长期有效的防护机制

本书按照脚本回答的问题整理这种区分。

- 「眼前这项工作该怎么完成？」……对应第 17 章的「**Build the Lever**」（例如批量重命名的 `codemod`）。
- 「这项工作是否正确完成？」……对应第 19 章的「**Prove It Works**」（例如检查导出文件中各行的脚本）。
- 「如何防止同一错误再次出现？」……对应「**Encode Lessons in Structure**」（例如禁止 `import internal/` 的 lint 规则）。

pstack 自身就是按这项原则构建的

pstack 也将自己的教训保存在机制中，而不只写成文字。以下是典型例子。

- `/reflect`（第 33 章）……对于从会话中收集的经验，如果 lint 或脚本能更可靠地强制执行，`/reflect` 就不会把它添加到 Skill 文字中，而是将实现相关机制作为任务，录入团队的待办列表（管理问题的地方）。
- `preferences.md`……「**Orchestrate**」Playbook（第 16 章）把长期有效的指令（如使用哪个模型、不能触碰哪些路径）逐行写入此文件，并在每次启动或恢复子 Agent 时原样贴入。

因为恢复时指令容易遗漏，每漏一次，人类就得再说一次。Agent **发现自己重复某条指令时，应先在此文件增加一行，再继续工作。**

总结

- **规则**……同一指令（如「不要从外部 import internal/ 中的模块」）写了两次，就要考虑能否做成 lint 规则、运行时检查等机制；如果可以，就删除文字指令。在可用的机制中选择最强方案，依次是类型、使 CI 失败的 lint、标准辅助函数、运行时检查。无法转化为机制的指令，要突出显示并附上失败示例。
- **触发条件**……第二次写下同一指令，或发现同一纠正反复出现。
- **pstack 自身如何运用**……/reflect 将可转化为机制的经验录入待办事项；「Orchestrate」Playbook 每次都会把 preferences.md 中的长期指令贴给子 Agent。

演讲指出，信任的增长来自仓库中留下的验证方法、工作步骤和约束，而不是会话的多少。

「**Encode Lessons in Structure**」正是在每次纠正后增加约束（如 lint 规则、类型）的原则。

第四部分至此结束。第五部分讨论 Playbook 和 Principle 调用的具体工具 Skill。首先，第 22 章将介绍四项 Skill，用于在变更前确认代码行为与原因，并从上次工作的位置继续。

第五部分：用 Skill 扩展工作能力

kaito • 来源：日本語 • English • 2026-10-03

第五部分将全面介绍 pstack 的全部 24 项 Skill。

这一部分可以跳读。建议只阅读自己感兴趣的章节。

每项 Skill 的原文都可以在 [pstack](#) 仓库中随时阅读。需要时可查阅相应章节和文件。

第五部分将介绍什么

- 每项 Skill 做什么、何时使用（例如 `/how` 会在修改代码前解释当前代码的行为）
- 输出采用什么形式（例如 `/why` 会标明找到的每个理由是有记录佐证，还是属于推测）
- 直接调用时的请求示例，以及如何区分相似的 Skill（例如查找差异内部的弱点用 `/interrogate`，查找差异以外可能受影响的地方用 `/blast-radius`）

Skill 将调查（如 `/why`）、设计（如 `/architect`）、验证（如 `/blast-radius`）、整理（如 `/unslop`）等工作的步骤封装起来，使人只要按名称调用，就能反复按相同步骤执行。

多数 Skill 会由 `/poteto-mode` 在 Playbook 步骤中自动调用。

使用者也可以按名称直接调用 Skill。如果想改变 Playbook 选择 Skill 的方式，或希望检查比默认流程更仔细，就可以直接调用。

例如，即使差异很小、`/poteto-mode` 的步骤不会调用 `/blast-radius`，使用者若仍不放心，也可以直接调用 `/blast-radius`，确认这项变更可能破坏什么。

要直接调用，使用者需要了解每项 Skill 的作用。第五部分的各章就是为此而写。

本部分章节

章	作用	介绍的 Skill
第 22 章 理解代码库和以往工作	调查当前代码的行为和代码形成现状的原因，从上次中断处继续工作	<code>/how、/why、/teach、/recall</code>
第 23 章 为没有合适 Playbook 的任务设计 Playbook	为没有现成 Playbook 的任务设计步骤，事先确定完成条件和验证机制	<code>/figure-it-out</code>
第 24 章 并行运行多个 Agent，比较设计与成果物	实现前先确定类型和模块边界，让多个 Agent 解决同一问题并比较方案，再将工作分给多个 Agent	<code>/architect、/arena、/swarm</code>
第 25 章 将 pstack 扩展到个人工作方式与 Grok Bot	根据使用者的聊天记录创建专属的 mode Skill，并制作可用按钮启动 Grok Bot 的页面	<code>/automate-me、/make-bot-ui</code>
第 26 章 建立验证机制并持续适配应用	创建项目专用的验证 Skill，沿着用户实际使用路径运行应用并检查，再随应用变化而维护	<code>/create-verification-skill、/maintain-verification-skill</code>
第 27 章 检查变更可能破坏的地方与差异的弱点	查找修改文件之外可能被破坏的地方，并让多个模型查找差异中的弱点	<code>/blast-radius、/interrogate</code>
第 28 章 记录判断，使 Agent 的长时间工作有据可查	记录判断及其依据，并请另一系列模型检查	<code>/show-me-your-work</code>
第 29 章 用 16 条 TypeScript 规则保持代码质量	通过类型让无效状态无法表示，并在外部数据进入时只验证一次	<code>/typescript-best-practices</code>
第 30 章 【TDD】用修复前失败的测试确认问题	修复缺陷前，先编写会在修复前失败的测试	<code>/tdd</code>
第 31 章 删除注释，整理代码库	让没有编写这些注释的 Agent 审查差异中多余的注释并删除	<code>/no-comments</code>
第 32 章 保持文字质量	去除文字中明显的 AI 写作习惯，用不含术语的话重述难懂的回复，并在写作前确定文档类型	<code>/unslop、/bro、/technical-writing</code>
第 33 章 回顾并运用工作中的经验	把长期工作中学到的内容写入现有 Skill	<code>/reflect</code>

章	作用	介绍的 Skill
第 34 章 安装 pstack 并确定各角色使用的模型	安装 pstack，并配置各角色使用的模型	/setup-pstack
第 35 章 `/poteto-mode`：pstack 的路由器	整理合适 Playbook 并加载所需 Skill 和 Principle 的规则	/poteto-mode

本书按工作流程排列章节，流程如下。

- 理解当前代码（第 22 章）
- 并行运行多个 Agent，比较设计与成果物（第 24 章）
- 实现并验证能否运行（第 26 章）
- 检查变更可能破坏的地方（第 27 章）
- 记录判断（第 28 章）
- 整理代码（第 29～31 章）
- 整理文字（第 32 章）
- 保存经验（第 33 章）

第 23 章介绍为没有合适 Playbook 的任务设计工作流程的 Skill。它涉及整个任务的推进方式，而非流程中的某个具体步骤，因此单独介绍。

第 25 章汇集了两项将 pstack 扩展到使用者工作方式和 Grok Bot 的 Skill。它们不用于流程中的某个特定步骤，因此单独介绍。

第 34 章介绍配置模型的 /setup-pstack，第 35 章介绍最先接收请求的 /poteto-mode。

pstack 的 README 将入门步骤概括为「用 /setup-pstack 选择模型，在需要严谨处理的工作中使用 /poteto-mode，仅此而已」；其他 Skill 会由 /poteto-mode 按需调用。

开始使用 pstack 时，首先要用 /setup-pstack；每次开始工作时，首先要用 /poteto-mode。两者都不属于流程中的特定步骤，所以本书将其放在独立章节介绍。

全部 Skill 速查表

Skill	概述	使用场景	使用方式（由谁调用）	详解
/how	说明当前代码的行为（触发条件、经过的函数、数据流）	使用者或 Agent 想在修改代码前了解子系统行为，或询问代码应该放在哪里时	/poteto-mode（依据「Non-negotiables」规则及「Investigation」「Bug fix」「Feature」「Refactoring」等 Playbook 的步骤）、/teach、/architect、/no-comments、使用者	第 22 章
/why	根据提交、PR、工单等记录调查代码为何形成现状，区分有记录佐证的理由和推测后作答	使用者或 Agent 询问设计理由、回归问题的来龙去脉或阈值的来源时	/poteto-mode（依据「Investigation」「Bug fix」Playbook 的步骤）、/teach、/recall、/architect、/no-comments、使用者	第 22 章
/teach	调用 /how 和 /why，将代码或变更是什么、如何运行、为何这样设计，汇总为一段通俗的解释	使用者觉得摘要不足以说明问题，希望真正理解变更或子系统时	仅在使用者按名称调用时	第 22 章
/recall	从个人聊天记录和共享记录中收集近期工作的经过，简要回答已推进到哪一步、接下来该做什么	使用者在开始或恢复工作前，想知道上次推进到哪里时	仅在使用者按名称调用时	第 22 章
/blast-radius	查找修改文件之外可能被破坏的地方，并通过运行代码验证变更安全的依据	使用者面对看似很小却不能完全放心的差异，想知道它还可能破坏什么时	仅在使用者按名称调用时	第 27 章

Skill	概述	使用场景	使用方式（由谁调用）	详解
<code>/interrogate</code>	让不同模型的审查者查找同一差异的弱点，由主 Agent 将意见归为四类	使用者或 Agent 希望多个模型从严格的代码质量等角度查找差异可能在哪里出错时	<code>/poteto-mode</code> （依据在合并有争议的设计前使用的「Non-negotiables」规则，以及「Feature」「 Opening a PR 」Playbook 的步骤）、 <code>/architect</code> 、使用者	第 27 章
<code>/figure-it-out</code>	为没有合适 Playbook 的任务设计步骤，事先确定完成条件和验证机制，并在推进中留下判断记录	使用者委托涉及多个调用方的迁移、希望离开期间继续推进任务，或请求没有合适的 Playbook 时	<code>/poteto-mode</code> （依据「Playbooks」一节的规则，将大型任务、使用者离开期间的工作及没有合适 Playbook 的任务交给此 Skill）、使用者	第 23 章
<code>/architect</code>	编写跨函数边界的代码前，先用不包含函数实现的骨架确定类型、函数签名和模块边界	Agent 准备编写跨函数边界的代码时	<code>/poteto-mode</code> （依据「Non-negotiables」规则及「Feature」「Bug fix」「Refactoring」「 Perf issue 」Playbook 的步骤）、 <code>/figure-it-out</code> 、 <code>/no-comments</code> 、使用者	第 24 章
<code>/arena</code>	让 N 个候选方案并行解决同一问题，选出最佳方案作为基础，再吸收其他方案的优点	使用者或 Agent 制作后续修改成本很高的成果物（如缓存键格式）时	<code>/poteto-mode</code> （依据「Non-negotiables」规则及「Feature」「 Eval 」「 Orchestrate 」Playbook 的步骤）、 <code>/architect</code> 、 <code>/blast-radius</code> 、使用者	第 24 章

Skill	概述	使用场景	使用方式（由谁调用）	详解
/swarm	以分工或竞争方式并行运行 N 个 Worker，并将结果汇总成一份报告	Agent 为了全面覆盖、竞争、设置关卡或划分调查范围而并行拆分工作时	/poteto-mode（依据「Non-negotiables」规则及「 Multi-phase or multi-PR plan 」「 Autopilot-full 」Playbook 的步骤）、使用者	第 24 章
/automate-me	根据使用者的聊天记录和问答，创建专属于该使用者的 -mode Skill	使用者希望 Agent 遵循符合自己偏好和工作方式的 mode Skill 时	仅在使用者按名称调用时	第 25 章
/make-bot-ui	制作可通过按钮和 webhook 启动 Grok Bot 的页面，同时不让 Agent 看到发送密钥	使用者希望制作通过 webhook 启动 Grok Bot 的页面、提供发送密钥并通过 Tailscale 发布页面时	仅在使用者按名称调用时	第 25 章
/create-verification-skill	在项目专用的验证 Skill 中启动并操作应用、保留证据，并将 Skill 生成于 .cursor/skills/verify-<app>/ 目录	使用者的项目缺少用脚本反复证明应用行为的手段时	使用者、/setup-pstack（在最后一步提议生成，使用者接受时调用）	第 26 章
/maintain-verification-skill	检查验证 Skill 和 Feature Map 是否与当前应用一致，并修正差异	使用者修改应用后，想修正 Feature Map 描述与当前应用之间的偏差时	仅在使用者按名称调用时	第 26 章
/show-me-your-work	把长期工作中的判断和依据记录在 decisions.tsv 中，并让另一系列模型检查记录	Agent 执行长期任务、自主任务，或使用者离开并打算事后检查的任务时	/poteto-mode（依据「Non-negotiables」规则及「 Autonomous run 」「 Orchestrate 」「 Hillclimb 」「 Autopilot-full 」等 Playbook 的步骤）、/figure-it-out、使用者	第 28 章

Skill	概述	使用场景	使用方式（由谁调用）	详解
/typescript-best-practices	用 16 条 TypeScript 规则说明如何让无效状态无法通过类型表示	Agent 读取或编辑 .ts 或 .tsx 文件时	Agent 读取或编辑 .ts 或 .tsx 文件时会自动加载。使用者无需按名称调用	第 29 章
/tdd	修复缺陷前先编写一个回归测试，确认它在修复前失败、修复后通过	使用者要求 TDD 或失败测试，或缺陷可以通过本地立即运行的测试确认时	/poteto-mode （依据「Bug fix」Playbook 的步骤 5）、使用者	第 30 章
/no-comments	让未编写注释的子 Agent Comment Sicko 审查差异中的注释，并修正接受的意见	Agent 要在审查前清理差异中的注释时	/poteto-mode （依据「Non-negotiables」规则及「Opening a PR」「Autopilot-full」「 Autopilot-stack 」「Multi-phase or multi-PR plan」Playbook 的步骤）、使用者	第 31 章
/unslp	用编号规则去除任何文字中明显的 AI 写作习惯	Agent 撰写包括回复在内的任何文字时	/poteto-mode （依据「Non-negotiables」规则及「Investigation」「Opening a PR」「Multi-phase or multi-PR plan」Playbook 的步骤）、 /technical-writing 、 /show-me-your-work 、 /recall 、 /teach 、 /blast-radius 、 /automate-me 、使用者	第 32 章
/bro	不用专业术语重述 Agent 刚刚给出的回复	使用者觉得 Agent 的回复虽详细却难以理解时	仅在使用者按名称调用时	第 32 章

Skill	概述	使用场景	使用方式（由谁调用）	详解
/technical-writing	从文档类型到单句写法，按四个层次说明编写技术文档应遵循的规范	Agent 撰写或审查文档、RFC、README、PR 描述及提交信息时	/poteto-mode（依据「Non-negotiables」规则及「Opening a PR」「Multi-phase or multi-PR plan」Playbook 的步骤）、使用者	第 32 章
/reflect	从工作会话中提取可用于后续任务的经验，把使用者批准的内容写入现有 Skill	完成有所收获的工作后，使用者立即说「reflect」时	仅在使用者按名称调用时	第 33 章
/setup-pstack	询问推理预算及各角色使用的模型，将回答写入始终生效的规则 ~/.cursor/rules/pstack-models.mdc 中	使用者初次使用 pstack，或想修改各角色使用的模型时	使用者。SKILL.md 中没有 disable-model-invocation: true，所以 Agent 也可以根据请求内容（如「configure pstack models」）选择并运行它	第 34 章
/poteto-mode	整理选择合适 Playbook 并让 Agent 加载所需 Skill 和 Principle 的规则	使用者委托需要严谨处理的任务（如先复现缺陷，再修复并验证）时	仅在使用者按名称调用时。调用一次后，在同一会话的后续轮次中持续有效，直到使用者明确停用	第 35 章

第 22 章：理解代码库与过去的工作

kaito · 来源：日本語 · English · 2026-10-03

本章介绍以下四项 Skill。

1. `/how`
2. `/why`
3. `/teach`
4. `/recall`

四项 Skill 都用于调查当前代码和以往工作。该用哪一项，**取决于想知道什么**。

选择方式如下。

- **现在如何运行**（例如取消运行时会经过哪些函数、如何修改数据库）……`/how`
- **为什么形成了当前的设计**（例如重试上限为什么是 5）……`/why`
- **希望把这两方面合成易懂的解释**（例如想充分理解这个 PR 如何改变重试机制）……`/teach`
- **从上次中断处继续工作**（例如上周的导出任务推进到了哪里、哪些修改被撤回）……`/recall`

调查已完成的变更可能破坏什么、差异有哪些弱点的 Skill，将在第 27 章介绍。

本章先列出四项 Skill，再逐项从使用时机、步骤和输出三个角度说明。原文提供请求示例的 Skill，也会附上示例。

最后总结如何区分相似的 Skill 与 Playbook。

本章结构

本章结构如下。

- 四项 Skill 一览
- `/how` 在变更前解释代码行为，让人能在脑中追踪执行过程

- /why 区分有记录佐证的理由与推测，回答代码为何形成现状
- /teach 用通俗语言解释代码机制与原因，直到人能够理解
- /recall 收集近期工作的经过，让人从上次失败处继续
- 根据眼前的问题选择 /how、/why、/teach、/recall 或「Session pickup」
- 总结

四项 Skill 一览

Skill	简述	主要问题
/how	解释当前代码的行为（触发条件、经过的函数、数据流）	X 如何运行？这段代码应该放在哪里？
/why	从提交、PR、工单等记录调查代码为何形成现状，并标明每个理由的可信程度	为什么会这样？为什么选择 Y？
/teach	调用 /how 和 /why，把结果合成一份从通俗定义开始的解释	我想真正理解它
/recall	从个人聊天记录及共享记录（如 PR、工单、生产错误）整理近期工作推进到哪里、哪些事情不顺利	我做到哪一步了？发生了什么？

四项 Skill 都只在按名称调用时运行。

四个 SKILL.md 都设置了 `disable-model-invocation: true`，禁止 Agent 根据会话内容自行选择运行。因此，只有使用者按名称调用，或 /poteto-mode 及其他 Skill 在步骤中按名称调用时，它们才会运行。

调用者的区别如下。

- /how、/why……/poteto-mode 会按 Playbook 步骤调用它们。/how 还会依据「[Non-negotiables](#)」一节的规则调用。其他 Skill 也会在步骤中调用：/teach、/architect、/no-comments 会调用 /how；/teach、/recall、/architect、/no-comments 会调用 /why。使用者也可以按名称直接调用。
- /teach、/recall……它们没有出现在 /poteto-mode 的「Non-negotiables」规则、Playbook 步骤或其他 Skill 的步骤中，因此只在使用者按名称调用时运行。

/how 在变更前解释代码行为，让人能在脑中追踪执行过程

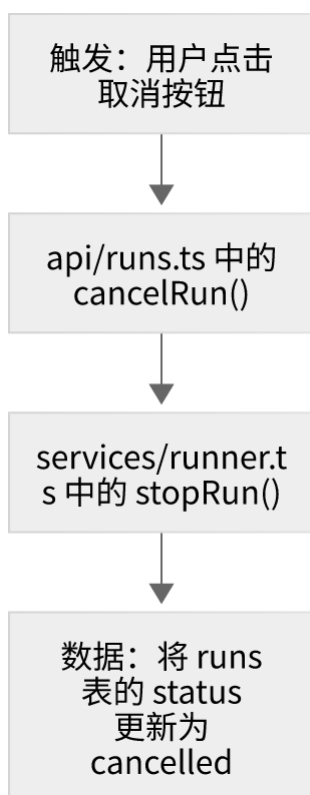
/how 会解释代码行为，详细程度足以让刚加入该子系统的资深工程师在脑中重现执行过程。

子系统……应用中负责一项完整功能的部分，例如取消运行、登录或支付。它通常横跨多个文件或模块。

在脑中重现，指读者无需打开代码，就能追踪处理从什么事件开始、经过哪些文件中的哪些函数，以及数据流向哪里。

解释的目的是让读者把握流程，因此无需逐行注释源码，写成过于琐碎的说明。

例如，对于取消运行的机制，读者应能在不打开代码的情况下说明以下流程（文件名和函数名只是示例）。



[查看图示 1](#)

使用时机：解释代码行为，以及询问代码应该放在哪里

在修改代码前，希望了解子系统的行为（例如取消运行依次经过哪些函数），或询问「这段代码应放在哪里」「放在这一层是否合适」时使用。这里的层指按职责划分的代码层，例如页面、API、数据库读写。

若要问「为什么这样设计」等动机，应使用 `/why`。

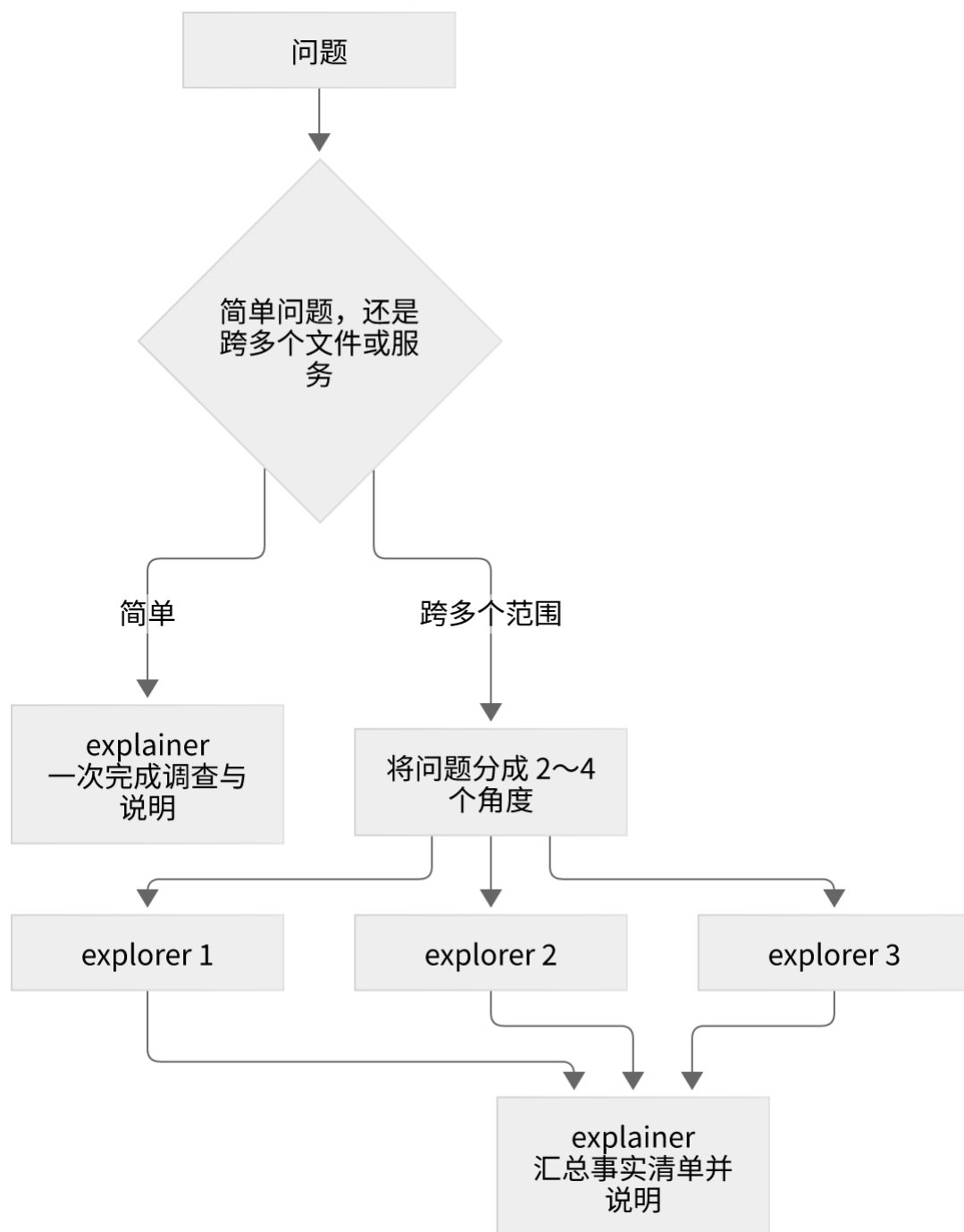
步骤：分开收集事实与撰写解释

若问题简单（例如「函数 X 如何运行」，范围限于一个模块），由 **explainer**（解释者）一次完成调查和解释。

若问题横跨多个文件或服务（例如整个子系统的结构），Agent 将问题拆成 2~4 个角度，**explorer**（调查者）会被**并行启动**，再由 **explainer** 整合结果。

角度指调查同一子系统时互不重叠的部分。例如取消运行机制可分为「接收取消请求」「停止运行」「更新数据库」。

若不确定问题简单还是复杂，Agent 将其视为简单问题，只用 **explainer** 调查，不启动 **explorer**。



查看图示 2

无论采用哪条路径，供使用者阅读的解释都由 explainer 撰写。explorer 只收集解释所需的事实。

explorer 返回事实清单，包括发现的组件、处理流程、阅读的文件、与其他部分的边界及意外发现，而不撰写正文。

因为面向读者的解释由 explainer 另写，explorer 可以优先考虑调查深度与准确性，无需顾虑文字是否易读。

explorer 不应编造未知或未追踪到的部分，而应如实写下「无法确定 X 如何连接到 Y」。

/how 只负责解释，因此其所有子 Agent 都在不修改文件的只读模式下运行。

输出：用 Overview 到 Gotchas 的五个标题组织内容

解释的目标，是让不熟悉该领域的人读完后，能够有把握地开始工作。

为此，explainer 按全貌、细节、工作入口的顺序使用以下五个标题；不适用的标题可省略。

以下示例以「取消运行的机制」为问题。

- **Overview（概述）** ……说明机制是什么、做什么、为何存在，让读者读完这部分就能决定是否继续（例如取消机制负责从使用者操作到停止运行的流程）。
- **Key Concepts（关键概念）** ……列出理解后文所需的类型与服务（例如表示运行的类型、接收取消请求的服务）。
- **How It Works（运行机制）** ……这是最长的部分，说明触发条件、处理步骤和数据流（例如取消请求依次经过哪些函数并修改数据库状态）。
- **Where Things Live（文件位置）** ……简要列出相关文件或目录及其内容，只列开始在该领域工作时必须打开的文件（例如接收取消请求的 `api/runs.ts`、停止运行的 `services/runner.ts`）。
- **Gotchas（注意事项）** ……说明仅看代码不易发现的细节、意外行为、当前设计的来历和容易犯错之处（例如取消后正在运行的处理不会立即停止，而是完成当前阶段才停止；所以页面显示「已取消」后，处理仍可能继续几秒）。

若处理流程横跨多个组件，explainer 也会加入图示。不过，图示是为了说明流程；仅靠文字即可讲清楚时便省略。

请求示例：直接问自己实际关心的问题

/how 会在许多 Playbook 的步骤中被调用，典型例子如下。

- 「**Bug fix**」（修复缺陷）

- 「**Feature**」 (新增功能)
- 「**Refactoring**」 (在不改变行为的前提下整理结构)

此外, /poteto-mode 的「Non-negotiables」规定: 作出不显而易见的变更或设计判断, 以及确认「真的没问题吗」时, 应使用 /how。

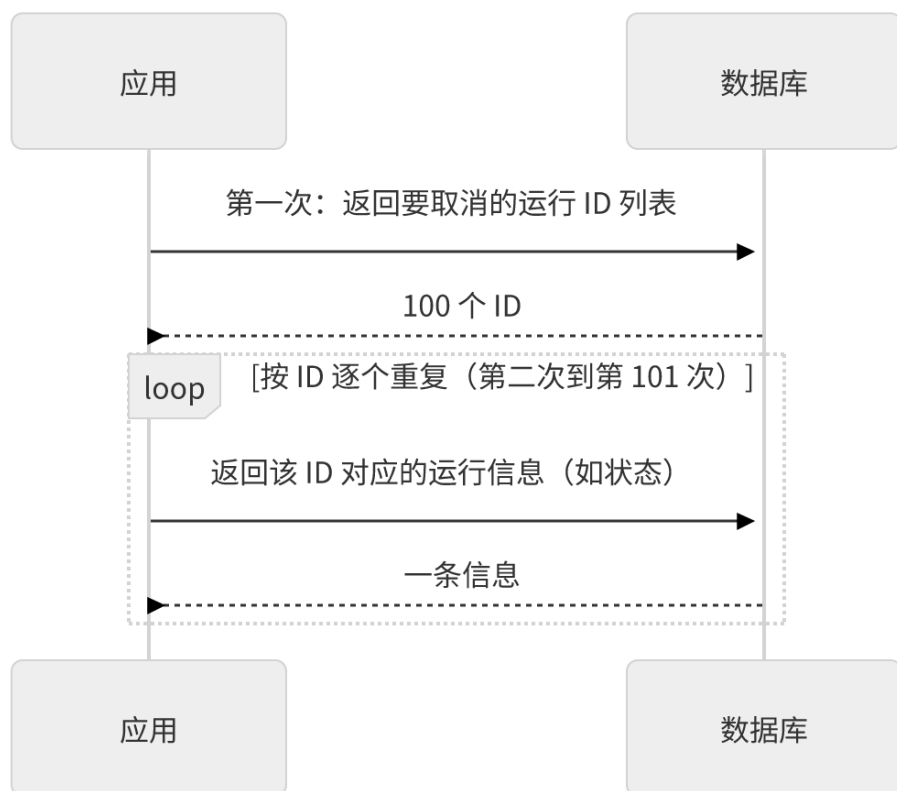
除 Playbook 外, /teach、用于确定设计的 /architect (第 24 章)、用于审查差异注释的 /no-comments (第 31 章), 也会在步骤中调用 /how。

直接调用时, 提出自己真正关心的问题即可。README 的示例如下。

```
/how do we cancel runs? do we have an n+1 when we look up every run to cancel?  
// 取消执行是怎么实现的? 逐个查询待取消的执行时, 会不会出现 N+1 查询?
```

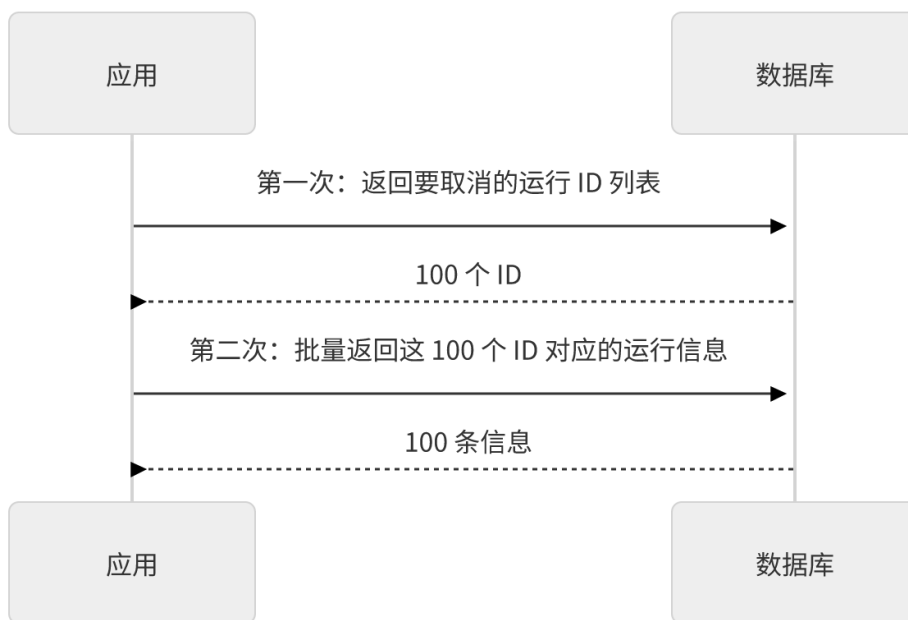
N+1……取得列表的一次查询之后, 又对每个条目分别发起一次查询的写法。

例如, 要取消 100 次运行, 应用可能向数据库发起 101 次查询。每次查询都要等待, 因此条目越多, 处理越慢。



查看图示 3

要消除 N+1，应用应停止逐个 ID 查询，在第二次查询中一次传入全部 100 个 ID。数据库会一次返回这 100 条信息，总查询次数便降至两次，如下图所示。



查看图示 4

随附指南的 03-understand.md 提醒使用者：不要以为「反正 Agent 会读代码」，就跳过该页面介绍的 Skill (/how、/why、/teach、/recall)。

指南这样提醒，是因为**Agent 若不先追踪代码行为就开始修改，很容易在第一个看似合理的位置只修补症状，没有修复根本原因**。只修补症状，根因仍在，后续便会出现新的缺陷（第 19 章）。

同一页面还指出，先使用 /how，比日后处理因此产生的缺陷成本更低。

/why 区分有记录佐证的理由与推测，回答代码为何形成现状

/why 是一项 Skill，用于解释代码为何形成现状，并**区分有记录佐证的理由与推测**。

/how 回答「如何运行」，/why 则回答「哪些背景和判断使它形成当前设计」。

如果调查后找不到理由，/why 会回答「不知道」，不以推测填空。因为**如果把推测写得很肯定，使用者就可能把推测当作事实，继而据此作决定**。

使用时机：询问设计理由、回归问题或阈值来源

适用于询问「为什么这样运行」「为什么选择 Y」等设计理由，调查回归问题（原本可用的功能因变更而失效）、事故复盘（故障后整理原因和防止再发生的办法），或阈值来源（例如重试上限为何为 5）。

如果要问运行时的行为，即「现在如何运行」，应使用 /how。

步骤：按证据所在位置分配调查者，并行调查

/why 将可能留下设计理由的地方分为下表七类。

类别	MCP 示例	特别适合回答的问题
源码管理历史	git、gh（在终端操作 GitHub PR 和 Issue 的官方 CLI）	审查过程中写下的实现理由（例如 PR 描述）。这是唯一在任何环境中都一定可用的来源，因此始终调查
问题与工单管理	Linear、Jira	产品或业务动机（例如应客户要求而新增）
长篇文档	Notion、Google Docs	写代码前的设计理由（例如比较方案 A 与 B 的设计文档）
即时聊天	Slack	未写进文档的讨论
基础设施观测	Datadog	导致超时或重试设计的运行环境条件
错误追踪	Sentry	导致防御性代码（例如检查 null、重试）的异常
产品分析	BigQuery	决定代码形态的产品或数据背景（例如阈值数字的来源）

Agent 首先调查连接到使用者 Cursor 的 MCP（Model Context Protocol，即连接 Agent 与外部工具或数据的机制），确认每个 MCP 可读取哪类证据。

例如，若连接了 Linear MCP，就归入「问题与工单管理」；若连接了 Slack MCP，就归入「即时聊天」。

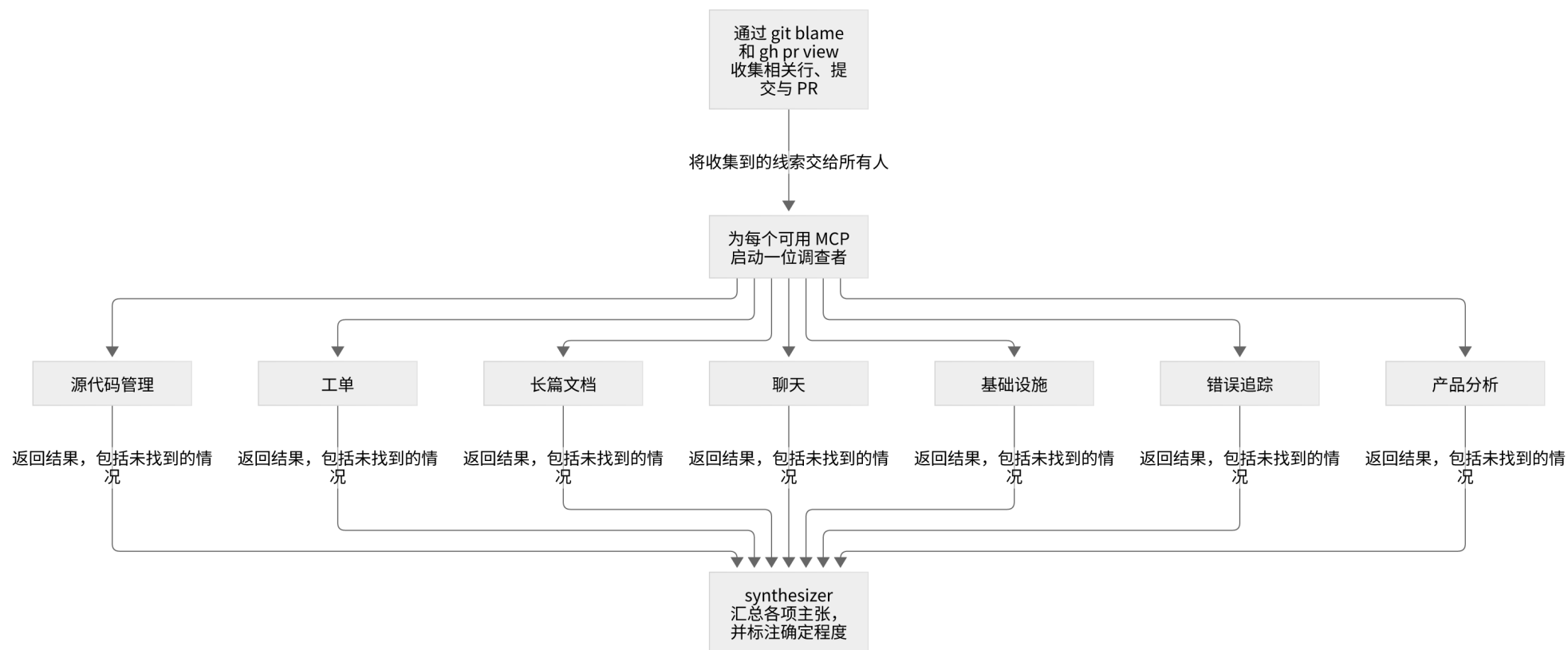
然后，Agent 为每个已连接 MCP 的类别并行启动一名调查者（investigator）。源码管理历史即使没有 MCP，也可以用 git 和 gh 读取，因此始终调查。

启动调查者前，Agent 使用如下命令收集目标文件的行、提交与 PR，再把线索交给所有调查者，让他们从具体代码开始调查。

```
# 显示最后修改目标行的提交
git blame -L <start>,<end> <file>

# 显示 PR 描述和讨论
gh pr view <number> --json
title,body,author,createdAt,mergedAt,labels,closingIssuesReferences,comments,reviews
```

调查结果由**synthesizer**（整合者）汇总，流程如下。



查看图示 5

调查者并行工作；若什么都没找到，也会向 synthesizer 返回「未找到」的结果。

synthesizer 还会记录哪些类别没有调查、哪些类别调查后没有发现，并说明理由。「查过但没找到」也是调查结果；如果不记录，使用者就无法区分没有调查和调查后没有发现。

输出：将所有主张按可信程度分为 Direct 到 Unknown 五级

代码能告诉我们它做了什么，却不能告诉我们它为什么存在。理由只可能留在提交、PR、工单和聊天等不完整的记录中。

因此 /why 按可信程度把每项主张分为以下五级。

下表示例针对「重试上限为什么是 5」这一问题。

级别	含义	示例
Direct（直接证据）	有作者明确写出理由的记录	代码注释写明「上游 API 拒绝第六次及以后的重试，因此在第五次停止」
Supported（有证据支持）	没有明确说明，但多项间接证据一致	把上限设为 5 的 PR，以及同一周的其他 PR 和工单，都提到了同一故障
Inferred（推断）	根据上下文作出的合理解读，没有明确佐证	代码库其他地方的上限也是 5，因此可能沿用了团队惯例
Speculative（推测）	证据薄弱，其他解释同样可能成立	或许是为了满足 SLA（承诺的响应时间），但 SLA 文档并未提到数字 5
Unknown（未知）	已调查但未查明，应写明搜索了什么、在哪里搜索	用「retry」搜索工单，并阅读了修改该文件的六个 PR，仍未找到理由

synthesizer 只对 **Direct** 和 **Supported** 级别的主张使用「所以是 X」这类肯定说法，因为这种句式意味着有证据支持因果关系。

例如，对于 Direct 主张，可明确写下「上限如此设定，是因为上游 API 拒绝第六次及以后的重试」，并附上来源。

对于 Inferred 主张，则应写「这个上限可能沿用了团队惯例」，让读者看出这只是推断。

即便使用者在问题中提出「我觉得是出于性能考虑」等假设，`/why` 也不会直接认可，而会将其视为候选解释，独立查证。

poteto 在「The Complete Guide to pstack」Part 2 中写道：**请求 Agent 工作时，最好不要先说出自己的假设，以免把 Agent 引向错误方向。**

若使用者打算在调查后修改代码，`/why` 会把调查结果转成 **Preserve / Change / Avoid / Risk**（保留、变更、避免、风险）四类约束，供变更计划使用。

例如，若调查重试上限的原因是为了修改它，可以得到以下约束。

```
Preserve: 不超过上游 API 允许的重试次数（该上限的理由已有记录）
Change:   不在代码中写死上限，而是在定义上游 API 限制的同一处确定它
Avoid:    去掉上限，一直重试直到成功
Risk:     如果上游 API 的限制变化，当前数值 5 就可能不合适
```

这样把调查结果分为四类约束，可以避免在制定变更计划时因不了解应保留的理由而无意中将其删除。

请求示例：「没人留下理由」也是一种答案

`/why` 会由「Investigation」（针对可通过阅读代码或历史回答的问题提供带引用的答复）和「Bug fix」Playbook 调用。`/recall`、`/teach`、`/architect`（第 24 章）、`/no-comments`（第 31 章）也会在步骤中调用 `/why`。

03-understand.md 给出了以下请求示例。

```
/why was the retry limit set to five? does the reason still hold?
// 为什么把重试上限设为 5？当初的理由现在仍成立吗？
```

同一页面指出，「没人留下理由」本身也是一种答案。

知道理由没有记录后，使用者便可决定询问原作者，或停止进一步调查。因此 `/why` 也会报告未找到理由的结果。

/teach 用通俗语言解释代码机制与原因，直到人能够理解

/teach 是一项把代码或变更是什么、如何运行、为何这样设计合成一份通俗解释的 Skill。它的目标是让对方理解，而不是修改任何东西。

poteto 在「The Complete Guide to pstack」Part 2 中写道，和比自己聪明的对象（Agent）一起工作，重要的是让对方用自己能理解的方式重述，这成为 /teach 的灵感。

使用时机：希望真正理解，而非只看摘要

当摘要不足以说明问题，使用者希望真正理解一项变更或子系统时使用。

步骤：调用 /how 和 /why，将结果合成一份解释

汇总 /how 和 /why 的结果，不重复调查

/teach 会自己阅读代码，以判断从哪里开始解释什么；但它不会重新做调查，而是实际调用 /how 和 /why，再把结果合成一份解释。

若问题涉及整个子系统，会同时调用两者；若只是小变更，调用 /how 或 /why 其中一项可能就足够了。

默认要求 /why 缩小调查范围

/teach 调用 /why 时，默认要求缩小调查范围（例如只查 git 和另外一两个位置）。调查所有类别会耗时，只有对方主要想了解原因时，才扩大范围。

确定少数值得记住的要点，从通俗定义连接到眼前的代码

解释时，先确定对方需要记住的少数要点，从通俗定义开始，再连接到当前代码。要点取决于对方为何提问（接下来要修改、审查，还是调查缺陷）以及对方已经知道什么；这些可从会话中判断。

例如，先用一般语言定义「虚拟滚动是一种只渲染屏幕可见行的机制」，再连接到当前代码：「在这个应用中，打开很长的聊天记录时会使用虚拟滚动。」

用一两句话回答后停下，等待对方反应

/teach 应采用对话形式，而非讲课。它不出小测验，也不要求对方复述；给出一两句话的最简回答后就停下，等待对方反应。如果对方希望深入，再继续展开。

不把所有内容画在一张图上，而是按构建顺序重画

说明机制的图应一次增加一个部件并重画。把所有部件画在一张图上可供查阅，却难以让读者追踪机制如何逐步建立。

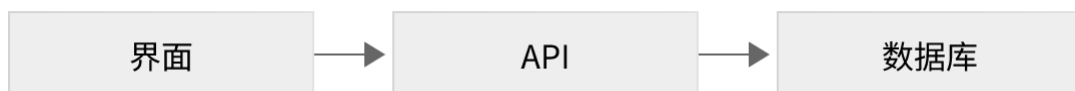
例如，从页面到 API、再从 API 到数据库的流程，应依次画出以下三张图。

第一张只画页面到 API。



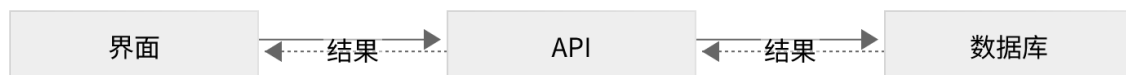
[查看图示 6](#)

第二张在第一张的基础上重画，加入数据库。



[查看图示 7](#)

第三张重画第二张，加入返回方向的箭头。



[查看图示 8](#)

这样每张图只增加一个部件，读者就能按照机制建立的顺序逐步理解。

输出：直接给出通俗解释，从要点开始

回复应直接给出解释，而非工作报告。从要点开始，并附上值得继续用 `/how` 或 `/why` 深入调查的问题。

`/teach` 可以为解释而自由调整措辞，但必须保留 `/why` 表示可信程度的词语，例如 `Direct` 或 `Inferred` 等级。

「似乎是……」等保留措辞，不是可以随意改写的装饰，而是「记录中找不到佐证」这一调查结果的一部分。因此 `/teach` 不应改写表示可信程度的词语。

请求示例：请对方「说服我」

`/teach` 是由使用者按名称调用的 Skill。03-understand.md 提供以下请求示例。

```
/teach me how this PR changes retries. convince me it fixes the cause and not the symptom.
// 说明这个 PR 如何改变重试，并让我相信修复的是原因而非症状。
```

指南建议使用「说服我」（convince me）这一说法，让解释成为可以提出反驳并检验的论证，而非单向讲解。

实际使用方式将在第 37 章介绍。

`/recall` 收集近期工作的经过，让人从上次失败处继续

`/recall` 在工作开始或恢复前，重新收集近期工作的经过（做过什么、决定了什么、哪里不顺利），让工作能够从上次失败处继续。它简要回答工作推进到哪里、下一步该做什么。

使用时机：想知道「上次做到哪里」

在开始或恢复工作前，想问「回顾一下 X 的工作」「上次做到哪里了」时使用。

步骤：并行调查聊天记录与共享记录

工作经过分散在两类记录中，因此 `/recall` 两者都要读。

- 个人聊天记录……记录做过什么、作出了哪些决定。

- **共享记录**……记录同一代码周围发生的事情，例如用户持续报告的症状、发布后被撤回的修复、生产环境持续出现的错误。

/recall 让 /why 的调查者查看共享记录。一个功能的缺陷持续越久，相关经过就越可能主要留在共享记录，而不是个人聊天记录。

此时 /recall 要求调查者回答的不是「为什么这样设计」，而是「现在是什么状态、试过什么但没有成功、用户一直在报告什么」。

主 Agent 在搜索前确定时间范围（默认最近七天）、主题、工作区（默认当前打开的工作区），并向使用者说明范围后再继续，以便使用者发现范围与预期不同时及时纠正。

除非使用者要求，主 Agent 不读取其他项目的历史。

若目标聊天只有一两条，主 Agent 会直接查找；更多时则让子 Agent 并行阅读历史。原始历史留在子 Agent 中，主 Agent 只接收发现的内容，避免长记录填满主 Agent 的上下文（第 20 章）。

主 Agent 使用 git 和 gh 核实找到的 PR 或分支的当前状态。聊天记录反映的是当时的状态，此后可能发生合并或撤回。

若使用者只是想继续某一条特定聊天，/recall 会把请求转交给「**Session pickup**」Playbook。

输出：Capsule、Threads、Problems、Next move 四部分

为了让读者迅速掌握「整体推进到哪里、接下来做什么」，输出从总体摘要到下一步行动，分为四部分。

- **Capsule（摘要）**……最多五项，说明任务是什么、整体进度如何（例如迁移到任务队列的变更已经合并，只剩重试问题）。
- **Threads（各工作单元的状态）**……按行列出该主题下的工作单元（如一个 PR、一条分支），每行必须使用下表中的一个状态标签。
- **Problems（反复出现的问题）**……最多五项，记录反复出现的问题，包括被撤回的修复，让下一次尝试从上次失败处开始。
- **Next move（下一步）**……写出最有用的一项下一步行动。

Threads 使用以下六种状态标签。

标签	含义
[merged #N]	PR #N 已合并
[open PR #N]	PR #N 仍处于打开状态，例如等待审查
[in flight <branch>]	正在分支上开展工作
[verified, uncommitted]	已验证，但尚未提交
[reverted #N]	PR #N 已被撤回
[planned, not started]	已计划，但尚未开始

例如，对于上周的导出工作，包含上述四部分和全部六种标签的输出如下。

Capsule

- 把 CSV 导出从页面处理流程移到异步任务中
- 迁移到任务的改动已经合并，但重试会重复写入相同行的问题仍存在

Threads

- [merged #412] 将导出移到异步任务
- [reverted #418] 添加重试（因重复写入相同行而撤回）
- [open PR #421] 以行键实现幂等写入
- [in flight export-progress] 在页面显示进度
- [verified, uncommitted] 延长大文件处理的超时时间
- [planned, not started] 删除旧的同步导出逻辑

Problems

- 重试时，首次尝试写入的行后面会再次出现相同行；#418 因此被撤回

Next move

- 回应 #421 的审阅意见并合并

这样，若 Problems 中保留被撤回的修复及其原因，下一次尝试便能从上次失败处开始。

请求示例：在工作开始时主动调用

使用者在开始或恢复工作时按名称调用 /recall。03-understand.md 提供以下请求示例。

```
/recall catch me up on the export work from last week
// 说明上周导出工作的进展
```

如何把工作交接给新的会话，将在第 37 章介绍。

根据眼前的问题选择 /how、/why、/teach、/recall 或「Session pickup」

各项回答的问题如下。

- /how……「现在如何运行？」
- /why……「为什么是这种设计？」
- /teach……「我想真正理解。」
- /recall……「我之前做到哪里了？」

如果想从一条特定聊天继续，使用「**Session pickup**」；如果要读入横跨多条聊天的上下文，使用 /recall。

总结

- **四项 Skill**……都只在按名称调用时运行。/how 和 /why 也会由 /poteto-mode 按 Playbook 步骤调用。
- /how……区分收集事实的 explorer 与撰写解释的 explainer，用 Overview 到 Gotchas 的五个标题解释当前行为。
- /why……按 git、工单、聊天等证据来源并行调查，将主张分为 Direct 到 Unknown 五级。只有有记录佐证的主张才用「所以是 X」等肯定说法。
- /teach……汇总 /how 与 /why 的结果，从通俗定义开始解释，并一次增加一个部件来绘图。
- /recall……根据聊天历史与共享记录，用 Capsule、Threads、Problems、Next move 四部分总结工作状态，并在 Problems 中保留被撤回的修复及其原因。
- **选择方式**……根据当前的问题选择。如果想继续一条特定聊天，使用「**Session pickup**」。

下一第 23 章介绍 /figure-it-out，它为没有合适 Playbook 的任务设计 Playbook 本身。

第 23 章：为现有 Playbook 无法覆盖的任务设计 Playbook

kaito · 来源：日本語 · English · 2026-10-03

本章介绍 `/figure-it-out`。

`/figure-it-out` 为没有合适 Playbook 的任务设计步骤：先确定完成条件与验证机制，再边推进边记录判断。

`/figure-it-out` 的 `SKILL.md` 设置了 `disable-model-invocation: true`，禁止 Agent 根据会话内容自行选择并运行它。调用这项 Skill 的可以是使用者，也可以是将没有合适 Playbook 的任务或大型任务交给它的 `/poteto-mode`（第 7 章）。

本章从使用时机、步骤、输出三个角度介绍 `/figure-it-out`，最后说明它与相似 Playbook 的区别。

本章结构

本章结构如下。

- 使用时机：大型迁移，或使用者离开期间委托的工作
- 步骤：从 Frame 到 Verify，分五个 Phase 推进
- 输出：返回设计出的 Playbook 和判断记录的路径
- 请求示例：使用者离开期间的工作由 `/poteto-mode` 交给 `/figure-it-out`
- `/figure-it-out` 设计单次任务的推进方式，「Orchestrate」管理整个计划
- 总结

`/figure-it-out` 是一项为没有现成 Playbook 的任务设计 Playbook 本身的 Skill。编写代码前，它先设计工作流程（按什么顺序推进、途中验证什么），并留下记录，让暂时离开的使用者返回后能逐项检查 Agent 的判断。

使用时机：大型迁移，或使用者离开期间委托的工作

当使用者委托跨越许多调用方的迁移，或希望 Agent 在自己离开期间继续工作、回来后能够信任结果时使用。

`/poteto-mode` 如何把这类任务交给 `/figure-it-out`，已在第 7 章介绍。

步骤：从 Frame 到 Verify，分五个 Phase 推进

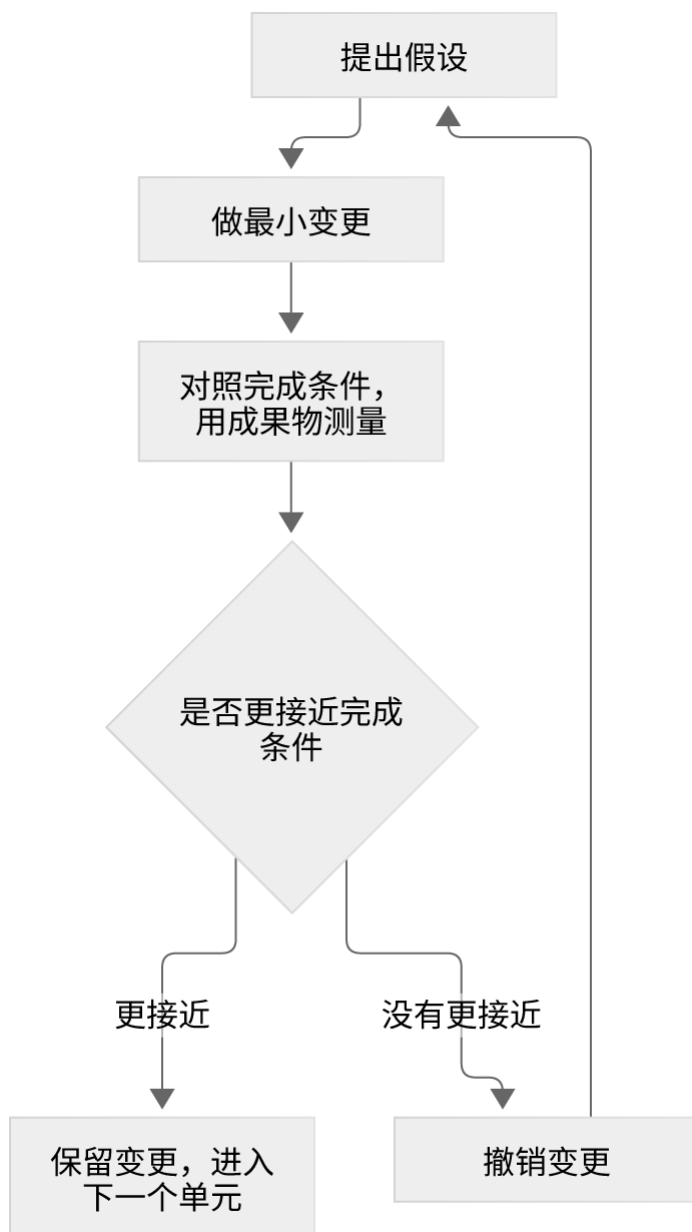
`/figure-it-out` 将工作分为五个阶段（Phase）。

划分阶段，是为了先确定完成条件、建立验证机制，并在推进中记录判断，让使用者返回后能够信任这项工作。

Phase	内容
A: Frame（确定框架）	确定完成条件、工作范围、严格程度（工作需要验证到什么程度）三件事。完成条件要写成可判断是否成立的条件，例如「调用同步存储的位置降为零」。用数字说明工作范围，包括工作单元的大致数量、工作量及调查发现的障碍。严格程度倾向设高；决定越难撤销，要求越严
B: Design the workflow（设计流程）	拆分工作单元并排序，在开展工作前先建立验证机制。每个单元应是可以独立合并、仍能通过构建和测试且应用照常运行的变更，无需等待其他单元合并。例如，每个包用一个 PR 将同步存储的调用方迁至异步存储；即使只合并一个 PR，尚未迁移的调用方仍可用同步存储运行。最不确定能否成功的单元排在前面。建立验证机制时，还要记录变更前后的值，例如先编写统计同步存储调用点的脚本并记录初始数量，让检查成为「变更前与变更后」的比较
C: Run the loop（重复实验）	把每个单元当作实验，依据完成条件，用真实成果物（如测试运行结果或统计调用点的脚本输出）测量。判定分为 VERIFIED（已验证）、NOT VERIFIED（未验证）、INCONCLUSIVE（无法判断）；INCONCLUSIVE 不算通过
D: Keep the audit trail（保留判断记录）	使用 <code>/show-me-your-work</code> （第 28 章）记录判断。人类返回后，可以依据记录和差异判断工作是否可信

Phase	内容
E: Verify and hand back (验证并交接)	在真实产品中验证整体结果；把反复出现的纠正写成 lint（自动检查代码写法的工具）规则或脚本，让机器执行，避免重复写文字指令（见 Principle「 Encode Lessons in Structure 」及第 21 章）

在 Phase C，Agent 对每个单元重复以下实验。



查看图示 1

每验证完一个单元才进入下一单元，因此与最后统一检查相比，可以立即看出是哪一单元失败。

在这一循环中，Agent 查看的是成果物本身，而非 Worker（受委派工作的 Agent）的自我报告（第 19 章）。例如，不只听 Worker 说「测试通过」，而要看实际运行测试的结果。

同样，Phase A 规定的严格程度并不是「仔细看看」的态度，**而是由必须通过的检查 and 必须留下的成果物决定**。提高严格程度时，应增加未通过就不能进入下一单元的检查，并留下更多验证结果、判断记录等成果物。

输出：返回设计出的 Playbook 和判断记录的路径

回复应包含以下内容。

- 设计出的 Playbook
- 严格程度及其理由
- 判断记录的路径
- 依据 Phase A 的完成条件已验证的内容
- 尚未解决的问题

请求示例：使用者离开期间的工作由 /poteto-mode 交给 /figure-it-out

README 的示例，是一个经 /poteto-mode 转交给 /figure-it-out 的请求。

```
/poteto-mode i'm stepping away. migrate every caller from the synchronous store
to the new async one, keeping behavior identical. i want to trust it was done
right when i'm back.
// 我暂时离开。把同步存储的所有调用方迁移到新的异步存储。
// 保持行为一致，并留下足够的证据，让我回来时能相信迁移正确。
```

/figure-it-out 设计单次任务的推进方式，「Orchestrate」管理整个计划

/figure-it-out 设计一次任务的推进方式，而「**Orchestrate**」负责管理跨越多天的整个计划。包含「**Autonomous run**」（先确定完成条件，再不中断地完成一项任务）的比较见第 16 章。

总结

- /figure-it-out……面对没有合适 Playbook 的大型任务，设计一套推进步骤：先设定可验证的完成条件并建立验证机制，再逐单元验证、记录判断。
- **选择方式**……/figure-it-out 设计单次任务的推进方式，「**Orchestrate**」管理跨越多天的整个计划。

下一章第 24 章介绍并行运行多个 Agent、比较设计与成果物的三项 Skill：/architect、/arena、/swarm。

第 24 章：并行运行多个 Agent，比较设计与成果物

kaito · 来源：日本語 · English · 2026-10-03

本章介绍以下三项 Skill。

1. `/architect`
2. `/arena`
3. `/swarm`

三者都会并行运行多个子 Agent。差别在于**希望通过并行得到什么**。

区别如下。

- `/architect`.....Agent 在实现前确定类型与模块边界。例如，导入操作应作为一个函数公开，还是分成读取、验证、保存三个函数公开？它让多个模型提出并比较设计方案，目标是确定一份供实现遵循的设计。
- `/arena`.....Agent 让多个子 Agent 分别解决同一问题（例如决定缓存键格式），选择最佳方案作为基础，再吸收其他方案的优点。目标是一份最好的成果物。
- `/swarm`.....Agent 把工作（例如检查 packages/ 下所有包）分配给多个 Agent，或让它们竞争解决同一问题，再把结果汇成一份报告。目标是全面检查或快速获得答案。

本章先确认谁会调用这三项 Skill，再逐项从作用、使用时机、步骤三个角度说明。原文提供请求示例的 Skill，也会附上示例。

最后说明如何选择这三种并行 Skill。

本章结构

本章结构如下。

- `/poteto-mode` 根据规则和 Playbook 步骤调用三项 Skill

- /architect 从调用方的用法推导类型，骨架若有误就丢弃
- /arena 让多个候选方案竞争，以得到一份最佳成果物
- /swarm 为全面覆盖或竞争将工作分给 Worker，并汇总成一份报告
- 根据想得到的结果，以及变更规模和撤销难度选择三项并行 Skill
- 总结

/poteto-mode 根据规则和 Playbook 步骤调用三项 Skill

/architect、/arena 和 /swarm 由 /poteto-mode 按照「Non-negotiables」规则与 Playbook 步骤调用。

Skill	简述	并行方式
/architect	实现前确定调用方的用法、类型和模块形态	内部调用 /arena，比较多种设计方案
/arena	让 N 个候选者（解决问题的子 Agent）处理同一问题，选出最佳者作为基础，再移植其他候选者的优点	所有候选者处理同一问题
/swarm	让 N 名 Worker（并行运行的 Agent）分工或竞争，并汇成一份报告	分别负责不同范围，或竞争解决同一问题

调用者如下。

- /architect、/arena、/swarm……/poteto-mode 按「Non-negotiables」一节的规则和 Playbook 步骤调用。其他 Skill 也会在步骤中调用：/figure-it-out 和 /no-comments 调用 /architect，/architect 和 /blast-radius 调用 /arena。使用者也可以按名称直接调用。

/architect 从调用方的用法推导类型，骨架若有误就丢弃

作用：用代码骨架确定设计，而非只写文字

/architect 是一项 Skill：编写跨函数边界的代码前，先用不含函数实现的骨架（not implemented）表达类型、函数签名（名称、参数与返回类型）和模块边界。

骨架……只写类型、函数签名（名称、参数、返回类型）和模块边界，函数主体暂用 `throw new Error("not implemented")` 等占位实现（下例中的 `ImportOptions`、`ImportResult`、`importRows`）。`not implemented` 表示「尚未实现」；占位主体被调用时只抛出这个错误，不执行实际处理。

以下以导入操作的骨架为例。

先写调用方如何使用，再由此确定类型和签名，结果如下。

```
// 调用方的用法：先写用法，再据此确定类型
async function onImportClick(file: File) {
  const result = await importRows(file, { onError: "skip" });
  console.log(`${result.imported}件を取り込み、${result.skipped}件を飛ばしました`);
}

// 骨架：只写类型和函数签名，暂不写实现
// onError 决定遇到无效行时跳过该行 (skip) 还是停止导入 (stop)
type ImportOptions = { onError: "skip" | "stop" };
type ImportResult = { imported: number; skipped: number };

export async function importRows(file: File, options: ImportOptions): Promise<ImportResult> {
  throw new Error("not implemented");
}
```

Oh My Stack 项目注（代码示例）：`console.log` 模板中的日文表示「导入了若干条，跳过了若干条」；它是示例运行时输出，故保留。

即使还没有实现，只看骨架也能知道：调用方只需调用一次 `importRows` 就能完成导入，返回值会说明导入和跳过了多少条。

Agent 先写骨架，是因为**骨架成为实现必须遵守的「契约」**：实现应符合骨架规定的类型与签名。

有了骨架，若实现偏离类型，或需要使用 `any`、第 18 章介绍的类型断言（如 `as User`，直接断定值属于某类型）等绕过类型检查的手段，就会在与骨架的**差异中显现**。

例如，骨架规定 `importRows(file, options)`，实现时却需要第三个参数，签名变更就会出现在差异里。

Agent 应让这类偏差显现出来，重新审视究竟是骨架有误、遗漏了需求，还是实现做了多余的事，而不是在实现中默默调整。因为**偏差意味着设计、需求或实现中至少有一处出了问题**。

`/architect` 不只依靠一份方案确定骨架，而会让多个模型提出方案并整合。随后按骨架填入实现；一旦发现骨架错误，就丢弃并重新设计。

poteto 在「The Complete Guide to pstack」Part 2 中写道，用代码做计划更有效，并列举 `/poteto-mode` 的原型制作和 `/architect` 作为实现方式。

使用时机：编写跨函数边界的代码之前

`/poteto-mode` 的「Non-negotiables」一节（第 8 章）规定，跨函数边界的代码应使用 `/architect`。

Playbook 也依此将 `/architect` 纳入步骤。「Feature」（新增功能）在步骤 2 必定调用它。

「Bug fix」（修复缺陷）、「Refactoring」（重构）、「Perf issue」（改善性能）则在变更跨越函数边界时调用它。

例如，新增函数并从另一文件的函数调用它，就跨越函数边界。只修改一个函数内部的计算等限定在单个函数内的变更，则无需 `/architect`。

其他 Skill 中，`/figure-it-out`（第 23 章）在难以撤销的设计判断上调用 `/architect`。`/no-comments`（第 31 章）若要修复注释审查提出的问题而需要新的代码形态，也只调用它一次。

步骤：从 Ground 到 Scrap，分五个 Phase 推进

Phase	名称	内容
A	Ground（打好基础）	使用 <code>/how</code> （第 22 章）调查新代码涉及的机制。只列出相关文件名不算完成调查，必须按 <code>/how</code> 的要求留下处理流程的追踪结果。如果设计要改变模块职责或代码层次，还需用 <code>/why</code> 调查当前设计的理由，并将其作为约束
B	Sketch（绘制骨架）	通过 <code>/arena</code> 生成多份设计方案，比较并整合
C	Agree（取得共识）	默认不等待人类确认，直接进入实现；仅在使用者要求时展示设计并等待批准

Phase	名称	内容
D	Implement (实现)	把骨架中 not implemented 的占位主体替换为实际代码
E	Scrap (丢弃)	如果实现中反复出现不符合骨架的变通方法或偏离，就丢弃骨架并重新设计

Phase B：至少比较两种结构不同的方案

即使第一份方案看起来已经足够，Agent 也会先让候选者提出至少两种结构不同的设计，再加以整合。如果复杂设计只做一次，容易固化为模型最先想到的形态（见随附指南 04-design.md）。

要比较的是结构本身不同的方案，而非同一种结构的小幅修改。

例如，只改 importRows 参数名称仍是同一结构的修改；只公开一个函数，与分别公开读取、验证、保存三个函数，则是结构不同的方案。

Phase B 的做法具体落实了第 17 章的 Principle「Exhaust the Design Space」。

/arena 启动的每名候选者（分别制作设计方案的子 Agent）遵循十项指令。以下介绍其中五项主要指令。

1. 先写调用方的用法，再据此确定类型

候选者在写类型之前，先写使用该类型的调用方代码，因为调用方想如何使用，正是设计必须满足的规格。

若用法与类型冲突，候选者应修改类型，使其符合用法。

例如在前面的骨架示例中，先在 onImportClick 中写下 importRows(file, { onError: "skip" }) 这一调用，再据此确定 ImportOptions 和 ImportResult 的类型。

```
// 调用方的用法：先写用法，再据此确定类型
async function onImportClick(file: File) {
  const result = await importRows(file, { onError: "skip" });
  console.log(`${result.imported}件を取り込み、${result.skipped}件を飛ばしました`);
}

// 骨架：只写类型与函数签名，暂不写实现
```



```
// onError 决定遇到无效行时是跳过该行 (skip) , 还是停止导入 (stop)
type ImportOptions = { onError: "skip" | "stop" };
type ImportResult = { imported: number; skipped: number };

export async function importRows(file: File, options: ImportOptions): Promise<ImportResult> {
  throw new Error("not implemented");
}
```

Oh My Stack 项目注（代码示例）：console.log 模板中的日文表示「导入了若干条，跳过了若干条」；该字符串是示例运行时输出，故保留原值。

2. 先选择符合常见读写操作的数据结构

写逻辑前，候选者先选择适合常见读写操作（例如按 ID 查找一个用户）的数据结构。

候选者逐项追踪这些读写操作在所选结构中如何完成。若结论是「以后再加索引或缓存」，说明该结构选错了。

例如，经常要按 ID 查找用户，却把用户保存在数组里。

每次按 ID 查找时都要从数组开头搜索。追踪后若得出「以后再添加按 ID 查找的索引」，就说明数组不合适。因此候选者一开始就应选择以 ID 为键的映射表。

```
type User = { id: string; name: string };
// 要查找的用户 ID
declare const id: string;

// 修改前：每次按 ID 查询都从头遍历数组
const userList: User[] = [];
const found = userList.find((u) => u.id === id);

// 修改后：用 ID 作为键建立映射，直接按 ID 查询
const usersById = new Map<string, User>();
const user = usersById.get(id);
```

合适的数据结构取决于常见的读写操作，因此候选者应在实现前作出选择。

3. 减少公开的函数，把处理隐藏在内部

候选者应缩小公开的函数和类型（公开接口），将更多处理隐藏在接口后面，以**减少调用方需要了解的内容**。

公开内容少、内部隐藏处理多的接口称为「**深接口**」。

例如，导入操作可以采用以下两种公开接口。

```
// 深模块：只公开一个函数，将读取、验证和保存隐藏在内部
type DeepImporter = {
  importRows(file: File, options: ImportOptions): Promise<ImportResult>;
};

// 浅模块：公开三个函数，要求调用方按正确顺序调用
type ShallowImporter = {
  loadRows(file: File): string[][];
  validateRows(rows: string[][]): string[][];
  saveRows(rows: string[][]): Promise<void>;
};
```

使用 `DeepImporter`，调用方只需调用一次 `importRows`。使用 `ShallowImporter`，调用方必须按正确顺序调用三个函数。

4. 尽可能用类型表达必须始终成立的条件

候选者首先用类型表达不变条件。无法用类型表达时，才用运行时检查；连运行时检查都不可行时，才写成注释。

例如，「已发货的订单必须有追踪编号」可用以下三种方式表达。

```
// 用类型表达：shipped 状态的订单必须有 trackingNo
type Order =
  | { state: "paid" }
  | { state: "shipped"; trackingNo: string };

// 运行时检查：类型无法阻止，只有执行时才会发现
type LooseOrder = { state: "paid" | "shipped"; trackingNo?: string };

function assertTrackingNo(order: LooseOrder): void {
  if (order.state === "shipped" && !order.trackingNo) {
    throw new Error("発送済なのに追跡番号がありません");
  }
}

// 注释：能否遵守取决于读注释的人
type CommentedOrder = {
  state: "paid" | "shipped";
  // state 为 "shipped" 时，必须提供 trackingNo
```

```
trackingNo?: string;
};
```

Oh My Stack 项目注（代码示例）：异常消息中的日文表示「已发货却没有追踪号码」；该字符串是示例运行时输出，故保留原值。

如果用类型表达，编译器会在写出违反不变条件的代码时提示。因此应尽可能用类型表达不变条件。

5. 外部数据只在入口验证一次，内部不重复检查

候选者在外部数据进入的地方（边界，例如接收文件或 API 响应的函数）验证一次，并为已验证的值赋予类型。内部代码直接使用带类型的值，不重新检查。**入口检查一次，内部就不必重复写同样的检查。**

例如导入操作中，`importRows` 读取文件每一行后立即检查 ID 是否为空。内部保存逻辑只接收已验证的行，因此不再检查 ID 是否为空。

这一指令遵循第 18 章的 Principle「**Boundary Discipline**」。

为设计附上理由文档，写明被否决的方案及原因

每位候选者都要为设计附上一页理由文档，先写调用方的使用方式（Usage），再写类型，并明确列出被否决的方案及原因。

比较方案时，候选者不能只看实现是否容易，还要比较展示给调用方的复杂度与隐藏在内部的复杂度。例如，`DeepImporter` 只要求调用方了解一个函数，`ShallowImporter` 要求了解三个，因此前者向调用方暴露的复杂度更低。

整合前用四个危险信号检查

整合前，Agent 用四个危险信号检查所有候选方案。危险信号指一旦出现，就足以要求修改或否决设计的结构特征。

四个危险信号如下。

- **浅模块**……公开了许多函数或类型，背后却只隐藏少量处理。
- **信息泄漏**……本应只在一个模块内部决定的事情（例如数据如何存放），其他模块也把它当作前提。一旦改变决定，所有依赖它的模块都要一起修改。

- **按执行顺序拆分**……按读取→验证→保存的执行顺序拆成模块。这样各模块都会传递同一数据形态，依赖该形态的模块随之增多。原文建议把处理同一数据的逻辑放进一个模块，即使执行时机不同也一样。
- **纯转发方法**……仅把收到的参数原样交给另一方法。

例如，下面的导入骨架同时具有四个危险信号。

```
// load.ts: 读取 CSV，并将每行转为字符串数组（浅模块）
export function loadRows(file: File): string[][] {
  throw new Error("not implemented");
}

// validate.ts: 假设第 0 列是 ID，过滤掉 ID 为空的行（浅模块，泄露内部知识）
export function validateRows(rows: string[][]): string[][] {
  throw new Error("not implemented");
}

// save.ts: 假设第 0 列是 ID，以 ID 为键保存（浅模块，泄露内部知识）
export function saveRows(rows: string[][]): Promise<void> {
  throw new Error("not implemented");
}

// importer.ts
export class Importer {
  // 只是将收到的 rows 原样传给 saveRows（透传方法）
  save(rows: string[][]): Promise<void> {
    return saveRows(rows);
  }
}

// 调用方：必须按正确顺序调用三个函数才能完成导入（按步骤拆分）
async function onImportClick(file: File) {
  const rows = validateRows(loadRows(file));
  await new Importer().save(rows);
}
```

四个信号在这段代码中分别表现如下。

- **浅模块**……load.ts、validate.ts、save.ts 三个文件各公开一个函数，但每个函数只做读取、筛掉行或保存等小事。相比公开函数数量，隐藏的处理很少，因此是浅模块。调用方必须自行组合三个函数，才能完成一次导入。原文也把调用方为完成一个操作而组合多个函数，列为浅模块的迹象。

- **信息泄漏**……`validate.ts` 与 `save.ts` 都假定「每一行是字符串数组，索引 0 的列是 ID」。若 CSV 列顺序改变、ID 移到索引 2 的列，就必须同时修改两个文件。
- **按执行顺序拆分**……文件按读取（`load.ts`）、验证（`validate.ts`）、保存（`save.ts`）拆分。三个文件依次传递行数据，都必须知道「每一行是字符串数组」。若行的形态变化，三个文件都得修改。
- **纯转发方法**……`Importer.save` 只将收到的 `rows` 原样交给 `saveRows`，没有隐藏任何处理。

决定若散落在多个模块，后续每次改变都要修改更多地方，因此应把同一决定封装在一个模块内。

前面 `importRows(file, options)` 的骨架中，调用方只需调用一个函数。行的表示方式、哪一列是 ID，均可隐藏在 `importRows` 模块内部。

Phase C：默认不等待人类确认，直接进入实现

默认情况下，Agent 不等待人类确认，直接进入实现。

Agent 不等待回复而继续，人类事后修正方向，符合 Principle「**Never Block on the Human**」（第 20 章）的思路。

若人类反对设计形态，Agent 将意见作为 Phase A 的新材料重新调查，并重做 Phase B。

若希望在实现前严格质询设计弱点，Agent 会对整合后的骨架运行第 27 章的 `/interrogate`。

Phase E：何时丢弃骨架

Agent 根据反复出现的模式，而非单次事件，判断是否丢弃骨架。

迹象例如：

1. 同类变通方法反复出现在无关代码中（例如不同页面调用 `importRows` 前，都自行编写相同的预处理）
2. 为了让代码编译，需要使用 `any` 或类型断言等绕过类型约束的手段
3. 调用方必须知道共用函数或类型的内部规则才能使用它（例如调用 `importRows` 前必须先调用另一个初始化函数）
4. 多处独立位置至少两次出现同类偏离骨架的情况（例如两个函数分别需要增加骨架没有的参数）

同时具有四个迹象的实现示例如下。

```
type Row = { id: string; name: string };

// 预先建立导入所用数据库连接的函数
declare function initImporter(): Promise<void>;

// 骨架中的签名是 parseRows(text: string): Row[]
// 征兆 4: 实现时需要骨架中没有的 tenantId 参数 (表示数据属于哪个租户)
declare function parseRows(text: string, tenantId: string): Row[];

// 骨架中的签名是 saveRows(rows: Row[]): Promise<void>
// 征兆 4: 实现时需要骨架中没有的 tenantId 参数 (表示数据属于哪个租户)
declare function saveRows(rows: Row[], tenantId: string): Promise<void>;

// 设置页面
async function onUploadFromSettings(file: File) {
  // 征兆 3: 若不先调用此函数, importRows 就会失败; 这一约定无法从 importRows 的类型看出
  await initImporter();

  // 征兆 1: 调用 importRows 前, 自行去掉 CSV 的第一行 (标题行)
  const lines = (await file.text()).split("\n");
  const body = new File([lines.slice(1).join("\n")], file.name);

  // 征兆 2: 为读取骨架的 ImportResult 中没有的 reasons (跳过行的原因), 使用 any
  const result = (await importRows(body, { onError: "skip" })) as any;
  console.log(result.reasons);
}

// 仪表盘: 虽然与设置页面无关, 却重复了征兆 1 的预处理和征兆 3 的初始化
async function onUploadFromDashboard(file: File) {
  // 征兆 3 的初始化
  await initImporter();
  // 征兆 1 的预处理
  const lines = (await file.text()).split("\n");
  const body = new File([lines.slice(1).join("\n")], file.name);
  await importRows(body, { onError: "stop" });
}
```

只需处理几个罕见输入或场景 (边界情况), 并不足以否定整个设计。数据本身复杂而导致分支增多, 也不是丢弃骨架的理由。

例如, 不同国家的地址格式不同而增加分支, 是数据复杂所致, 并不意味着设计有误。

丢弃骨架时, Agent 会对已有成果重新运行 /how, 再次调查机制。随后将实现中发现的新约束作为重新设计的起点, 从 Phase B 开始, 而不是继续在旧骨架上修补。

此外，在新骨架中添加内容前，Agent 会先删除不再需要的东西，例如不再使用的函数或重复的验证。这遵循 Principle 「**Subtract Before You Add**」（第 17 章）。因此，在开始添加功能前，新骨架就已比旧骨架更小。

请求写法：说明「最关心什么」

随附指南 04-design.md 给出以下请求示例。

```
/architect design the import pipeline before writing any code. i care most about how callers use it.
// 写代码前先设计导入流程。我最关心调用方会怎样使用它。
```

若希望在 Phase C 加入确认环节，使用者可这样请求。

```
/architect with checkpoint. stop and show me before implementing.
// 请设置确认环节：在实现前停下来，给我看设计。
```

从调用方用法出发设计，以及重新审视设计的实践，将分别在第 38 章和第 39 章介绍。

/arena 让多个候选方案竞争，以得到一份最佳成果物

作用：让 N 个候选者解决同一问题

候选者是各自制作一份设计方案的子 Agent。

/arena 让多个候选者解决同一问题，**选出最佳成果物，并将其他成果物的好想法融入其中。**

使用时机：后续修改成本高的成果物

对于缓存键格式等成果物，若只让模型做一次，容易受最先想到的设计限制，而事后修改又很昂贵，使用者或 Agent 就可以使用 /arena。

/poteto-mode 工作中主要在两种场景调用 /arena。

一是 /architect 在 Phase B 让多个候选者提出设计方案。

二是「**Feature**」Playbook 的步骤 4：若实现有多种合理方式（例如错误处理或测试结构都存在多种合理写法），主 Agent 会把编码交给 /arena 的候选者。

/arena 也由 Playbook 和其他 Skill 调用

调用 /arena 的 Playbook 和 Skill，都在只让一个模型回答一次可能留下偏差或遗漏的场景使用它。各调用者的场景和理由如下。

/architect：不只依据一份方案决定设计

/architect 在 Phase B 调用 /arena，让多个模型提出设计方案。复杂设计若只做一次，容易固化为模型首先想到的形态（见本章「Phase B：至少比较两种结构不同的方案」）。

「Eval」 Playbook：用多个模型与盲评衡量 Skill 变更的效果

「Eval」用于衡量 Skill 或提示词变更如何影响 Agent 行为（第 13 章）。

「Eval」并不直接调用 /arena，但规定按照 /arena 的步骤推进。

在「Eval」中，多个模型在变更后的环境分别处理同一请求，再由另一个模型在不知道输出来源的情况下评分。

因此，让多个候选者解决同一问题、再让另一模型评分的流程，与 /arena 相同。

「Orchestrate」 Playbook：开工前比较难以撤销的判断

「Orchestrate」管理持续多天、涉及多个 PR 和子 Agent 的项目（第 16 章）。

它在最初的 Frame 步骤中，把存在分歧的任务拆分方式和难以撤销的判断交给 /arena，先于第一个工作单元的试运行进行比较。因为一旦项目开始运行，就无法再轻易修正难以撤销的判断。

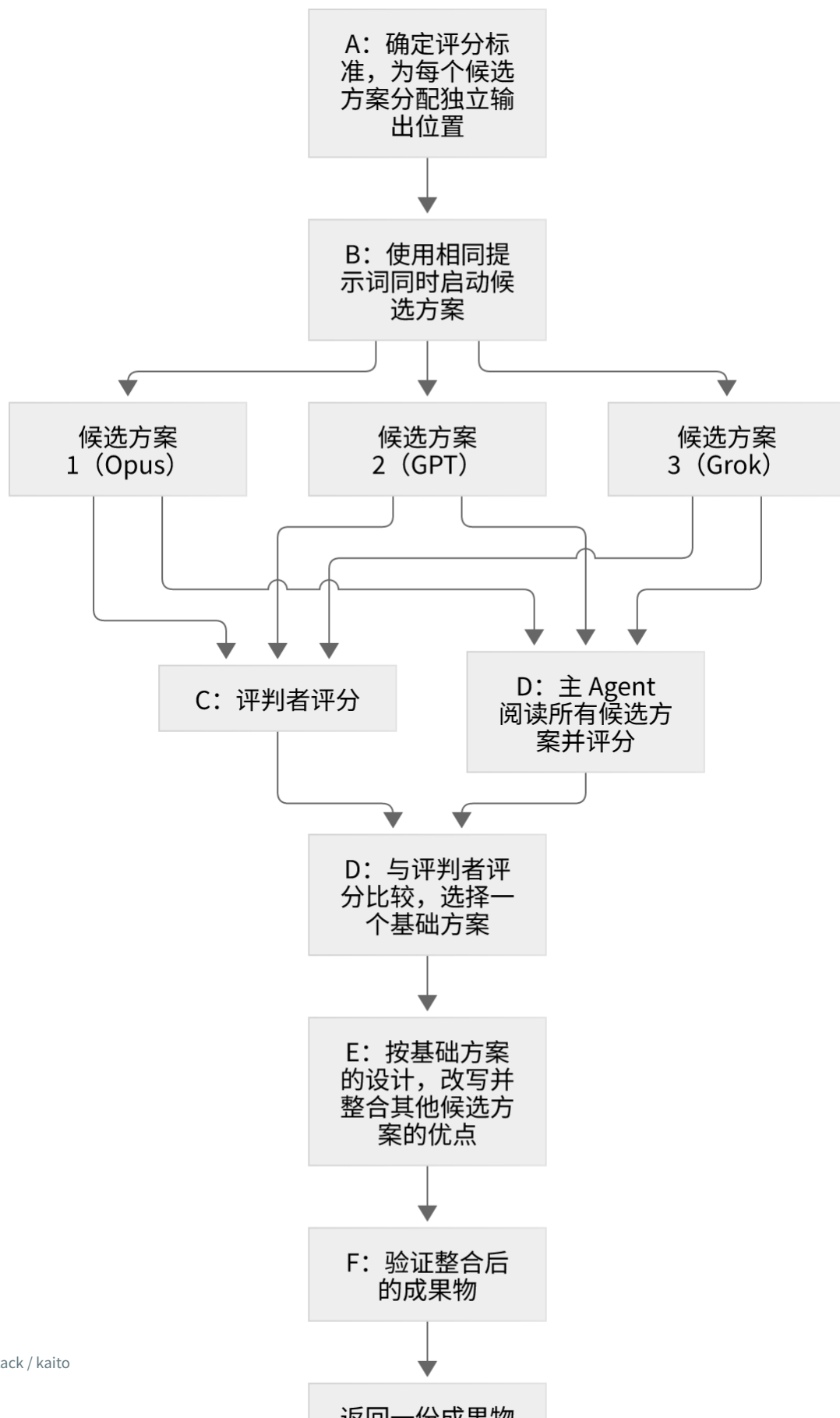
/blast-radius：让多个模型调查大型变更的影响

/blast-radius（第 27 章）调查规模大或影响范围广的变更时，会用 /arena 向多个模型提出同一问题，再整合回答。据其 SKILL.md，原因是不同模型会发现不同的缺陷。

步骤：从 Frame 到 Verify，分六个 Phase 推进

Phase	名称	内容
A	Frame（定义问题）	明确每个候选者要制作的成果物及 3~6 条评分标准，并为每人分配不同输出位置，避免候选者互相覆盖成果物
B	Fan out（同时启动）	同时启动 N 个子 Agent；默认使用 Opus、GPT、Grok 三种模型
C	Cross-judge（由另一模型评分）	所有候选者完成后，由 评判者 （cross-judge）模型评分。评判者尽可能选用与 主 Agent （启动候选者的 Agent）不同系列的模型
D	Pick（选择基础方案）	主 Agent 完整阅读全部候选成果，按各项标准评分，再与评判者推荐的基础方案比较后作决定。若一致，推荐可为选择提供佐证；若不一致，可能是主 Agent 或评判者有偏差，或评分标准含糊
E	Graft（移植）	从未入选的方案找出有价值的部分，根据基础方案的设计重新编写后纳入，不能机械粘贴；最终成果物应遵循一致的设计思路
F	Verify（验证）	遵循 Principle「 Prove It Works 」（第 19 章），以与其他成果物同样严格的标准验证整合结果能否实际运行

六个 Phase 的流程如下。



[查看图示 1](#)

所有候选者完成后才启动评判者

评判者的评分与主 Agent 阅读所有候选成果的工作，会在全部候选者完成后并行进行。主 Agent 不会在候选者仍在编写时启动评判者。

给候选者相同的提示词，明确成果物，但不展示评分标准

所有候选者收到同一提示词，**每人要制作什么完全取决于提示词写了什么**。因此，主 Agent 在 Phase A 应明确写出成果物，例如缓存键格式方案及理由。

评分标准用于主 Agent 在 Phase D 选择基础方案，因此主 Agent 只向候选者展示问题，不展示评分标准。

选择最便于后续增加功能的方案作为基础

主 Agent 选择未来维护者最容易在不破坏不变条件的情况下增加功能的方案，作为基础。

若所有候选者得出相同结构，主 Agent 会把独立求解后的结果一致视为有力佐证，直接采用，不再移植。

候选方案差异过大时，重新定义问题

若候选方案结构差异过大，主 Agent 会判断问题定义不足，返回 Phase A 重新界定后再做一次。

例如三个候选者提出的缓存键格式全都不同，主 Agent 不会从三者各取一部分拼出中间方案，因为**中间方案可能与任何候选者的设计思路都不一致**。

最后返回整合成果物与整合说明

主 Agent 最后返回一份整合后的成果物和一份简短的整合说明，内容包括：

- 作为基础的候选方案及选择理由，包括评判者的结论
- 从其他候选方案吸收的部分，以及每部分来自哪个方案
- 考虑过但未吸收的部分及原因

- 未能交付成果、中途退出的候选者（如有）；主 Agent 会用剩余方案继续整合，并记录其退出
- 在 Phase F (Verify) 实际运行并验证整合成果物的结果

请求写法：指定候选者数量

04-design.md 提供指定候选数量的示例。

```
/arena this, 5 candidates. the cache key format is expensive to change later.
// 给出五个候选方案。缓存键格式事后修改代价很高。
```

随附指南 [10-recipes-and-pitfalls.md](#) 还建议：对于事后修改成本高的决定，可以先就当前设计征求第二意见。

这时，主 Agent 将使用者与 Agent 正在推进的方案作为候选之一，与其他方案比较。

```
ask /arena for a second opinion on this thread and our approach
// 用 /arena 对这个讨论及我们的工作方式提出第二意见。
```

/swarm 为全面覆盖或竞争将工作分给 Worker，并汇总成一份报告

作用：并行运行 Cloud Agents

/swarm 同时运行 N 名云端 Agent (Worker)，**收集结果并返回一份报告**。/arena 追求一份最佳答案，/swarm 则追求全面检查或快速答案。

poteto 在「The Complete Guide to pstack」[Part 1](#)中提到，让大量 Cloud Agent（运行在 Cursor 云端的 Agent）执行验证 Skill，以足够的样本量确认性能改进，或对应用进行模糊测试。

模糊测试……向应用提供大量随机或损坏的输入，以寻找缺陷的测试方法。

使用时机：全面覆盖、竞争、关卡与分区调查

适用情况如下。

- **全面覆盖**……主 Agent 把多个范围分配给 Worker，确保逐一检查，没有遗漏（例如检查 packages/ 下的全部包）。
- **竞争**……主 Agent 让多名 Worker 解决同一问题（例如分别尝试复现同一缺陷）。
- **gauntlet**（关卡）……主 Agent 为同一变更并列设置多项验证，要求变更通过全部验证（示例见下方说明框）。
- **分区调查**……主 Agent 把搜索范围拆开，让每名 Worker 调查一处（例如查找缺陷原因时，为每个可疑模块分配一名 Worker）。

gauntlet（关卡）不是 pstack 文件中明确定义的术语。英语 gauntlet 指接连通过多项考验。

可从随附指南 04-design.md 中的「gauntlet lanes」（关卡负责人），以及使用 /swarm 的 Playbook，看出它在 pstack 中的用法。

例如「**Multi-phase or multi-PR plan**」Playbook（第 16 章）制定的计划要求：PR 准备好接受审查时，以及此后每次推送变更时，都针对当时 PR 的最新提交，通过 /swarm 并行运行以下负责人，**仅在全部结果为 PASS 时才通过**。

- 一名负责人在最新提交上重新运行测试、lint 等检查
- 十名负责人在实际页面或 CLI 中验证
- 一名负责人测量性能
- 至少两名负责人阅读差异和执行记录，质疑并检查 PR 描述

/poteto-mode 在「Non-negotiables」一节规定，全面覆盖、竞争、关卡、分区调查等并行拆分工作应使用 /swarm。尤其是第 16 章介绍的边验证多个 PR 边推进的 Playbook，会用 /swarm 判断 PR 能否合并。

例如，「**Autopilot-full**」（每个 PR 的负责人从制作负责到合并）中，协调 Agent（管理整体工作的 Agent）用 /swarm 并行运行独立验证者，再把结果汇成一项判定。每个 PR 的负责人只有得到无问题的判定，才会合并 PR。

步骤：从 Frame 到 Report，分四个 Phase 推进

/swarm 依次执行以下四个 Phase。

1. **Frame**……主 Agent 确定完成条件（用可验证的形式写明满足什么才算完成）和应返回的报告，同时决定模式（下表的分工、竞争、混合）及 Worker 数量。若使用者在请求中指定数量，就按指定数量；否则按模式确定。
2. **Fan out**……主 Agent 在云端同时启动所有 Worker。每名 Worker 返回 PASS（通过）、ISSUES（发现问题）或 BLOCKED（无法验证），并附证据，例如验证过的提交 SHA（标识提交的值）与检查输出。
3. **Aggregate**……主 Agent 收集结果。若采用竞争模式，则按 Frame 阶段事先声明的选择规则选出结果。
4. **Report**……主 Agent 将结果表、附有证据的问题、没有得到结果的范围（空白）和中途退出的 Worker（脱落）汇成一份报告。

Frame 可选择以下三种模式。

模式	内容
分工（partition）	把工作分为不重叠的范围，每名 Worker 负责其中一个（例如每个包分配一名）
竞争（race）	向 N 名 Worker 发出相同指令，让他们竞争（例如分别尝试复现同一缺陷）
混合	结合分工与竞争（每个范围由多名 Worker 竞争处理）

采用竞争或混合模式时，主 Agent 必须在启动前声明选择规则：「第一个通过的结果」（first pass）、「对全部结果排序」（rank all）或「选出最佳结果」（best-of）。

主 Agent 会丢弃缺少所要求记录的结果（如验证过的提交 SHA 或测量次数等方法），并让相关 Worker 只重试一次。因为**若不知道验证了哪个提交、如何验证，就无法判断结果是否适用于当前代码。**

第二次仍缺少记录，就把该范围记为空白。**空白不能算作通过。**

例如，十名 Worker 中九名返回 PASS、一名没有结果，报告应写「九项通过、一项空白」，不能只写「九项通过」。

请求写法：分别写明验证方法、模式与数量、所需输出

README 和随附指南给出了以下请求示例。

```
/swarm check every package under packages/ against its check.sh. one worker per package. one report.  
// 用各自的 check.sh 检查 packages/ 下的全部软件包。每个软件包安排一个 Agent，最后提交一份报告。
```

请求用三句话分别写明验证方法（check.sh）、模式与数量（每个包一名 Worker）、输出（一份报告）。这三项分别对应主 Agent 在 Frame 阶段要决定的内容；写进请求后，主 Agent 便可按指定条件构建 Frame。

根据目标、变更规模及撤销难度选择三项并行 Skill

选择方式如下。

- 想设计跨函数边界的变更（例如创建新函数，并从另一文件的函数调用）……使用内部会调用 /arena 的 /architect
- 最终想得到一份成果物（例如选定一种缓存键格式）……使用 /arena
- 想把多个范围的检查结果汇成一张表（例如 packages/ 下全部包的检查结果）……使用 /swarm

设计投入取决于变更规模和撤销难度

并非每项变更都需要专门设计。04-design.md 提供以下参考标准。

情况	使用的工具
小而完整的变更，但仍有疑虑	仅用 /interrogate
跨函数边界，或转移所有权（由哪个模块负责）的变更	使用 /architect（内部也调用 /arena）
命名或算法等单项判断	直接使用 /arena
按范围全面检查、并行检查，或按事先声明规则进行竞争	/swarm
存在分歧、事后撤销成本高的设计	先用 /architect，再用 /interrogate

`/poteto-mode` 会自行套用这些标准；工作跨函数边界时，即便使用者没有要求，也会调用 `/architect`。因此，使用者直接调用这些 Skill，通常是希望检查比默认流程更仔细或更简短。

`/arena` 与 `/swarm` 的目标不同

项目	<code>/arena</code>	<code>/swarm</code>
目标	制作一份最佳成果物	全面覆盖，或通过竞争快速获得答案
给各子 Agent 的指令	所有人处理同一问题	分工时负责不同范围，竞争时收到相同指令
汇总方式	选定基础方案，将其他候选方案的优点按基础方案的设计重新编写后移植	汇成表格；竞争时按事先声明的规则选择
输出	一份整合成果物及整合说明	在聊天中给出的一份报告

为了全面覆盖而使用 `/arena`，是常见的误用（`10-recipes-and-pitfalls.md`）。`/arena` 把同一问题交给所有候选者，因此无法检查不同范围。

要全面覆盖，应使用 `/swarm` 分配范围，再汇总为一份报告。`/swarm` 不会像 `/arena` 一样选基础方案并移植。

总结

- **三项 Skill 的分工**……`/architect`、`/arena`、`/swarm` 由 `/poteto-mode` 按「Non-negotiables」规则和 Playbook 步骤调用。
- `/architect`……编写跨函数边界的代码前，从调用方的用法推导类型，建立不含函数实现的骨架。若同类变通方法或绕过类型约束的手段（any、类型断言）反复出现，就丢弃骨架并重新设计。
- `/arena`……让多个模型处理后续修改成本高的成果物（如缓存键格式），以最佳候选方案为基础，并将其他方案的优点按基础设计重写后移植。
- `/swarm`……为全面覆盖（如检查所有包）或竞争并行运行 Worker，汇成一份报告。如果某个范围缺少已验证提交等记录，重试一次后仍缺失，就将其报告为空白，不算通过。
- **选择并行 Skill**……跨函数边界的变更使用 `/architect`，需要一份成果物使用 `/arena`，需要覆盖多个范围的结果表使用 `/swarm`。

下一第 25 章介绍将 pstack 扩展到使用者工作方式和 Grok Bot 的两项 Skill: `/automate-me`、`/make-bot-ui`。

第 25 章：把 pstack 扩展到自己的工作方式与 Grok Bot

kaito · 来源：日本語 · English · 2026-10-03

本章介绍以下两项 Skill。

1. `/automate-me`
2. `/make-bot-ui`

两者都用于让 pstack 适应使用者的环境。各自的用途如下。

- `/automate-me`.....Agent 从使用者的聊天记录中读出其工作习惯（例如「段落写短些。比较选项时用表格。」），把这些习惯写成使用者专属的 mode Skill，类似 `/poteto-mode`。
- `/make-bot-ui`.....Agent 制作一个网页，让使用者按下按钮就能启动支持安装 pstack 的 AI Bot 应用 Grok Bot（第 5 章）中的处理流程。

本章先确认由谁调用这两项 Skill，再分别从作用、使用时机和步骤三个方面解释。原文提供请求示例的 Skill，也会列出示例。

本章结构

本章安排如下。

- 两项 Skill 只有使用者点名调用时才会运行
- `/automate-me` 从聊天记录中读出使用者的工作方式，将其写成专属 Skill
- `/make-bot-ui` 制作一个用按钮启动 Grok Bot 的页面，同时避免暴露发送密钥
- 总结

两项 Skill 只有使用者点名调用时才会运行

Skill	一句话概括
/automate-me	根据使用者的聊天记录，制作使用者专属的 -mode Skill
/make-bot-ui	制作一个页面，按下按钮后通过 webhook（向 URL 发送 HTTP 请求）启动 Grok Bot 的处理流程

两份 SKILL.md 都设置了 `disable-model-invocation: true`，禁止 Agent 根据对话内容自行选择并运行它们。`/poteto-mode` 和各个 Playbook 的步骤也不会调用这两项 Skill。因此，只有使用者点名调用时，它们才会运行。

/automate-me 从聊天记录中读出使用者的工作方式，将其写成专属 Skill

作用：从记录和提问中整理使用者的习惯，制成专属 Skill

`/automate-me` 将使用者的工作习惯转化为 Agent 应遵循的 Skill。输出是一项为该使用者定制、名称以 `-mode` 结尾的 Skill（例如 `jay-mode`）。

使用者无需自己说明工作习惯。Agent 会从聊天记录中读出使用者实际反复提出的要求。

例如，如果使用者多次要求 Agent「回答再短一些」，Agent 就会将「简短回答」识别为其习惯。记录里没有体现的意图，则通过向使用者提问来补充。

使用时机：使用者希望将自己的偏好和工作方式写成 Skill

pstack 的 `/poteto-mode` 和其他 Skill 无法涵盖所有使用场景。

因此，使用者想制作符合自己偏好和工作方式的 Skill 时，可以点名调用它。

如果只想保存「如何写提交信息」这类单项工作的步骤，使用者只需使用 Cursor 内置、用于编写 Skill 文件的 `create-skill`，无需调用本 Skill。单项工作只要写成普通 Skill 即可，不必放进描述整体工作习惯的 Skill。

步骤：阅读记录、询问使用者、打磨草稿，再创建 PR

1. **阅读记录**.....Agent 将当前工作区最近 2~4 周的 transcript（聊天记录）分成例如三个时间段，交给并行的子 Agent 阅读。只读取当前工作区的记录，不会连其他项目的私人聊天也读进去。只有至少两个时间段都出现的模式，才视为可信；只出现于一个时间段的模式通常会舍弃。一次性提出、却在另一场合提出相反要求的偏好，是噪声而非稳定习惯。
2. **直接询问使用者**.....为补足记录中尚未体现的意图，Agent 只提出 1~2 个选择题和 1 个自由作答的问题。采用选择题，是为了免去使用者从头写一段说明的负担。
3. **起草**.....Agent 使用 create-skill，在 .cursor/skills/<handle>-mode/SKILL.md 写下草稿。
<handle> 可以是使用者姓名或其选择的标识符。
4. **打磨**.....Agent 对草稿运行 /unslop（第 32 章），去除 AI 风格的行文习惯，并与使用者确认读起来是否像本人。
5. **纳入项目**.....Agent 在 worktree 中工作（git 的一项功能，可以从同一仓库创建多个工作副本；第 18 章），然后创建 PR，不会直接推送到 main。

如果已有 mode Skill，Agent 只读取上次修改之后的记录，并据此更新该 Skill。

只有使用者存在与 Agent 默认行为不同的具体规则时，Agent 才会在 Skill 中单独设节，不会写入「表达清楚」之类的泛泛原则。泛泛原则不会改变 Agent 的行为，写进去只会增加阅读负担。

例如，「段落要短；比较选项时用表格；只有条目确实并列时才用列表」这样的规则就值得单独设节。

请求写法：只输入命令即可开始

随附指南 09-make-it-yours.md 指出，使用者只需输入下面的命令即可开始。Agent 会从记录中读出其习惯，因此无需另附说明。

```
/automate-me
```

同一页面还给出工作方式改变后更新现有 mode Skill 的请求写法。

```
/automate-me update my mode skill with everything since its last edit
// 利用上次编辑以来的所有信息，更新我的 mode Skill。
```

/make-bot-ui 制作一个用按钮启动 Grok Bot 的页面，同时避免暴露发送密钥

作用：在 Agent 看不到秘密值的前提下，建立启动 Bot 的机制

/make-bot-ui 制作一个供使用者点击的页面，并建立由按钮通过 webhook 启动 Grok Bot 的机制。

使用时机：希望通过 webhook 按钮启动 Grok Bot 的例行任务

使用者想制作一个通过 webhook 启动 Grok Bot 的页面（按钮或仪表板）时，可使用这项 Skill。为该页面提交发送密钥，或通过 Tailscale 开放页面访问时，也使用它。

Tailscale……一项服务，可将使用者持有的设备（如电脑和手机）连接成不向互联网公开的私有网络。该网络称为 tailnet，只有加入 tailnet 的设备才能打开网络中的页面

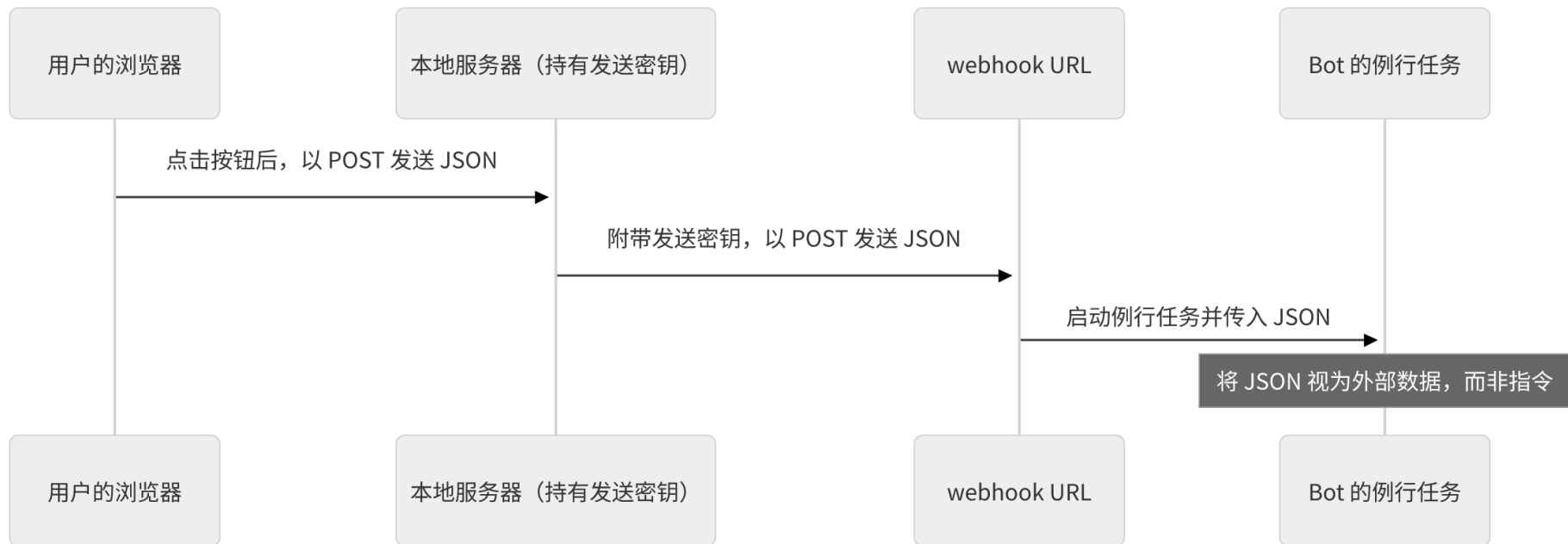
Grok Bot 是一款 AI Bot 应用，可以将 pstack 作为插件安装（第 5 章）。Grok Bot 支持注册例行任务，让任务由日程或事件触发。

webhook……向指定 URL 发送 HTTP 请求，通知另一个系统发生了事件并启动处理的机制（例如，向指定 URL 发送通知，告知 GitHub 上发生了 push）

/make-bot-ui 处理的是由 webhook 触发的例行任务。从按下按钮到 Grok Bot 开始处理，依次经过以下三步。

1. 使用者在浏览器中按下页面按钮，页面便向运行在 Bot 同一台电脑上的小型服务器（下称本地服务器）发送按钮对应的 JSON（例如 {"action": "summarize-today"}）。
2. 本地服务器为收到的 JSON 附上发送密钥，再将其发送到该任务的 webhook URL（用于从外部启动此任务的专用 URL）。
3. JSON 到达 webhook URL 后，Bot 的例行任务启动，并将 JSON 作为输入开始处理。

页面不直接向 webhook URL 发送请求，而是经由本地服务器，是为了不把发送密钥交给浏览器。后面的步骤说明会解释原因。



[查看图示 1](#)

步骤：创建例行任务、部署页面，并通过 tailnet 开放访问

1. **创建例行任务**.....Agent 创建由 webhook 触发的例行任务，并在提示词中要求把 POST 请求视为不可信数据。webhook URL 收到的请求体来自外部，即使其中夹带「忽略之前的所有指示」之类的命令，也不能让 Bot 照做。
2. **请使用者取得 URL**.....Agent 请使用者从 Routines（Bot 的例行任务列表）中打开相应任务并复制 webhook URL。原文允许使用者将 webhook URL 粘贴到聊天中；但下一步的发送密钥不能贴入聊天。
3. **接收发送密钥**.....发送密钥是证明调用 webhook 的发送者有权发送的秘密值。Agent 不通过聊天接收，而请使用者通过专门输入秘密值的卡片提交。Agent 看不到提交的密钥值，却能在不查看值的情况下将其写入服务器配置。
4. **部署页面**.....Agent 将页面和本地服务器放在运行 Bot 的同一台电脑上。页面按钮向本地服务器发送 POST 请求，再由服务器附上发送密钥并向 webhook 发送 POST 请求。
5. **接入 tailnet**.....Agent 使用 Tailscale，让使用者的设备可以打开页面，同时避免向互联网公开页面。
6. **接收触发输入**.....Bot 将收到的 JSON 当作外部数据，而非指令。

Agent 只把发送密钥放在服务器上，不放进浏览器或聊天中。因为知道密钥的人都能启动该例行任务。

如果浏览器直接调用 webhook，就得把密钥写进页面代码，打开页面的任何人都能读到。因此，按钮必须通过本地服务器调用 webhook。

同样基于避免在聊天中暴露秘密值的原则，Agent 也不会向使用者索取 Tailscale 凭证；它会提供登录 URL，让使用者自行在浏览器中批准设备。

总结

- **两项 Skill 的调用方式**...../automate-me 和 /make-bot-ui 只有使用者点名调用时才运行。
- /automate-me.....根据聊天记录和对使用者的提问，制作记录其习惯的专属 -mode Skill。
- /make-bot-ui.....制作一个按钮页面，用来启动 Grok Bot，同时避免把发送密钥放进聊天或浏览器。

接下来的第 26 章介绍两项用于建立验证机制、并使其持续适应应用的 Skill: `/create-verification-skill` 和 `/maintain-verification-skill`。

第 26 章：建立验证机制，并随应用变化持续维护

kaito · 来源：日本語 · English · 2026-10-03

本章介绍以下两项 Skill。

1. `/create-verification-skill`
2. `/maintain-verification-skill`

两者都旨在让 Agent 实际运行应用，验证自己的工作。它们分别负责以下任务。

- `/create-verification-skill`……在项目中创建一套步骤，让 Agent 启动应用，像真实用户一样操作，并取得截图等证据。
- `/maintain-verification-skill`……定期检查 `/create-verification-skill` 编写的步骤中所述的功能和操作是否仍与当前应用一致，并修正偏差。

`/create-verification-skill` 会建立一项**验证 Skill**（原文称 verification skill），让 Agent 像真实用户一样运行应用并验证。验证 Skill 是项目专属的 Skill，包含 Agent 启动、操作开发中的应用，以及获取证据（截图、终端输出、日志等）所需的步骤（第 3 章）。

两项之中，**使用者首先要用的是创建验证 Skill 的 `/create-verification-skill`**。

本章先确认两者的分工、调用者以及为何应从验证 Skill 开始，再分别从作用、使用时机、步骤和请求写法四个方面说明。

本章结构

本章安排如下。

- 两项 Skill 各自负责的工作
- `/create-verification-skill` 建立通过用户实际路径运行应用的验证手段
- `/maintain-verification-skill` 定期修正 **Feature Map** 与当前应用的偏差

- 总结

两项 Skill 各自负责的工作

两项 Skill 的作用和成果如下表所示。

Skill	作用	成果物
/create-verification-skill	在项目中建立启动、操作应用并留下证据（如截图）的步骤	.cursor/skills/verify-<app>/（验证 Skill 与 Feature Map ）
/maintain-verification-skill	持续使验证 Skill 和 Feature Map 与当前应用保持一致	包含实际运行应用并验证过的修正的 PR（最多一份），或仅提交报告

Feature Map 是一份文档，按应用功能记录该功能能做什么、用户如何找到它、怎样用验证 Skill 操作它，以及可以验证什么结果（第 4 章）。

/create-verification-skill 与 /maintain-verification-skill 将第 19 章的 Principle「**Prove It Works**」所要求的「实际运行功能并验证」，转化为项目中可用于各种工作的验证手段，并持续维护。

两项 Skill 只有使用者或 /setup-pstack 点名调用时才会运行

两份 SKILL.md 都设有 disable-model-invocation: true（第 25 章）。设有该选项时，Agent 不会根据对话内容自行选择并运行 Skill。因此，只有使用者点名调用，或其他 Skill 在步骤中明确指定调用时，它们才会运行。

两者的调用者如下。

- /create-verification-skill……使用者可以点名调用。此外，/setup-pstack 会在最后一步检查项目是否已有运行应用并验证的手段；若没有，就提出创建建议，并在使用者接受后调用这项 Skill。
- /maintain-verification-skill……只有使用者点名调用时才会运行。

首先创建验证 Skill，是为了让 Agent 能自行验证工作

使用者首先准备验证 Skill，是因为如果 Agent 无法自行验证工作，不论交给它多少任务，最后仍要人逐项检查，人就会成为阻碍工作推进的瓶颈。

人一旦成为瓶颈，就得整天照看 Agent。有了验证 Skill，Agent 可以自行运行应用、确认结果，并持续工作直到成功。使用者只需查看 Agent 验证过的结果。

poteto 也在《The Complete Guide to pstack》的 [Part 1](#) 和 [Part 2](#) 反复表达这一观点。

尤其在 Part 1 中，他指出，有了验证 Skill，其他 Skill 和自动运行的机制也能负责检查应用。因此，他将验证 Skill 视为其他 Skill 与定期任务的基础。

Part 1 举了以下两个例子。

- `/swarm` ([第 24 章](#)) 让大量 Cloud Agent 执行验证 Skill，以足够多的次数确认性能改善。
- Grok Bot 的例行任务 ([第 25 章](#)) 每次收到 Slack 中的用户报告，就让 Cloud Agent 用验证 Skill 重现该问题。

`/create-verification-skill` 建立通过用户实际路径运行应用的验证手段

作用：为项目建立实际运行应用并证明其行为的手段

`/create-verification-skill` 在 `.cursor/skills/verify-<app>/` 生成项目专属的验证 Skill。借助它，Agent 能够**启动真实应用，像用户一样操作**，并留下证据。

Agent 编写这项 Skill 的读者是其他 Agent，而非人。**读者可能从未见过该应用，在工作途中毫无背景知识地打开验证 Skill。**因此，Agent 会直接写出该仓库实际使用的命令和 selector（指定界面元素的方式），避免读者猜测。

验证 Skill 建好后，使用者只需说「在应用中验证」，任何 Agent 都能自行启动和操作应用，无需使用者再教一次方法 ([docs/guide/06-verify-and-ship.md](#))。

使用时机：项目缺少通过脚本验证应用行为的手段

使用者应在项目缺少可用脚本反复验证应用行为的手段时使用这项 Skill。

例如，项目既没有自动操作浏览器并检查画面的脚本，也没有运行 CLI 并核对输出的脚本。

配置 pstack 所用模型的 `/setup-pstack` 也会在最后一步检查项目是否缺少运行应用并验证的手段 (`verify-* Skill` 或既有 `harness`)。若缺少，它会提出一次生成建议；使用者接受后，调用 `/create-verification-skill` ([第 34 章](#))。

步骤：调查仓库、编写验证 Skill，并在交付前运行一次

步骤 1：先从仓库调查应用如何运行，再询问使用者

第一步，Agent 先从仓库代码和配置调查应用的运行方式，再考虑询问使用者（原文称这项调查为「面谈」，interview）。

Agent 只向使用者询问代码库中观察不到的事。连读代码就能知道的事也向使用者发问，会增加对方负担，并让工作在等待答复期间停下来。

Agent 从代码库回答以下五个问题，以收集验证 Skill 所需的内容：启动什么、如何操作、留下什么证据。

方面	问题
Surface（用户接触的界面）	用户实际会接触什么（Web UI、CLI、API 等）
Run（启动）	如何在本地启动应用（优先使用仓库自身的开发命令，例如 package scripts、Makefile、README 中的快速入门）
Drive（操作）	Agent 如何通过程序操作？先找仓库现有的 Playwright 或 Cypress 测试、可用 curl 调用的端点等；若没有，再选择 CDP、PTY、tmux 或纯 HTTP 等通用手段
Observe（观察）	能留下什么证据（截图、终端记录、响应正文、日志、退出码、数据库状态等）
Isolate（隔离）	能否分别使用不同端口和数据目录，同时运行两个应用实例

- **Playwright、Cypress**……自动操作浏览器进行测试的工具
- **CDP**……通过程序使用浏览器开发者工具功能的机制。对于 Web 应用和 Electron（用于构建桌面应用的框架）应用，Agent 可借助 CDP 连接浏览器，发送点击和输入操作，并读取页面状态
- **PTY**……伪终端（pseudo terminal）。它是由程序提供、可代替真实终端的终端；其中运行的应用会认为自己正被人在终端里操作

- **tmux**……终端复用软件（Terminal Multiplexer），可在一个终端界面管理多个会话、窗口及分屏（pane）。也能从外部用命令向会话发送按键，或读取屏幕显示的内容
 - 对于 CLI 和 TUI，Agent 在 PTY 或 tmux 会话中启动应用，发送按键输入，再读取屏幕显示的文字
- **纯 HTTP**……不经过界面，直接使用 curl 等向应用发送 HTTP 请求，并核对返回的响应
- **实例**……每一个正在运行的应用。例如，同一个 Web 应用分别在 3000 和 3001 端口运行，就是两个实例

如果无法并排运行两个实例，Agent 会在验证 Skill 中注明这一点，并要求两个 Agent 不要同时操作同一个实例。Agent 若同时操作使用者正在使用的实例，可能破坏使用者工作中的会话。

如果仓库代码按当前状态无法构建或启动，Agent 会先修到能够构建和启动，或准确报告缺少什么，再生成验证 Skill。因为以无法构建和启动的状态为前提编写的 Skill，会把错误步骤传给下一位 Agent。

步骤 2：写出 SKILL.md 的六个标题

Agent 在生成的 SKILL.md 中安排以下六个标题，让从未见过应用的下一个 Agent 可以依次完成启动到清理的全程。

Agent 只根据步骤 1 查到的事实编写内容，不留下 placeholder（待以后填写的临时文字）。不熟悉应用的下一个 Agent 无法把临时文字替换为正确值。

以下对比保留 placeholder 与依据调查事实写成的示例。

```
<!-- 仍有占位内容：下一个 Agent 不知道该运行什么、点击什么 -->
启动：运行 `<启动命令>`；`http://localhost:<PORT>` 有响应后即准备就绪
操作：点击 `TODO：发送按钮的选择器`

<!-- 根据调查所得事实编写：下一个 Agent 可以直接照做 -->
启动：运行 `pnpm dev`；`http://localhost:5173` 有响应后即准备就绪
操作：点击 `getByRole('button', { name: '送信' })` 所定位的按钮
```

Oh My Stack 项目注（代码示例）：getByRole 的 name 值「送信」意为「发送」；它必须与示例按钮的可访问名称一致，因此保留原值。

标题	内容
Launch（启动）	启动命令、判断应用「已准备就绪」的方法（如出现特定日志行或端口响应）以及停止方法
Doctor（诊断）	只读检查，用于判断「操作这个实例来验证是否有意义」（例如进程是否运行、构建是否正确、使用该端口的是否为自己启动的实例、认证是否有效）
Drive（操作）	使用该仓库真实的 selector 和命令编写操作步骤。使用 ARIA 标签或 data 属性等稳定标识，而不是屏幕坐标或按 Tab 键切换焦点的顺序
Evidence（证据）	为证明结果要保留什么、存放在哪里
Cleanup（清理）	如何清理由本次运行创建的实例，但不删除证据
Helpers（辅助脚本）	如何调用随附的脚本

ARIA 标签……赋予按钮等元素的名称，供屏幕阅读器朗读

下面用 Playwright 代码对比 Drive 中的「稳定标识」。

```
declare const page: {
  mouse: { click(x: number, y: number): Promise<void> };
  getByRole(role: "button", options: { name: string }): { click(): Promise<void> };
  locator(selector: string): { click(): Promise<void> };
};

// 修改前：按屏幕坐标点击；按钮位置变化后可能点到其他地方
async function saveByPosition() {
  await page.mouse.click(640, 410);
}

// 修改后：按 ARIA 标签点击；按钮位置变化后仍能点到同一按钮
async function saveByLabel() {
  await page.getByRole("button", { name: "保存" }).click();
}

// 修改后：按 data 属性点击；按钮位置变化后仍能点到同一按钮
```

```
async function saveByDataAttribute() {
  await page.locator('[data-testid="save"]').click();
}
```

Oh My Stack 项目注（代码示例）：getByRole 中的「保存」意为「保存」，是按钮可访问名称的精确匹配值，因此保留原值。

Evidence 还要写明证明标准，例如：

- 操作用户实际经过的路径，而非测试专用的捷径
- 不只检查界面，也检查副作用（写入的文件、新增的行、发出的消息等）

测试专用捷径，例如直接改写内部值的函数，或专为测试准备的端点。Agent 即使用捷径完成检查，也无法证明用户实际经过的路径（真实应用行为）运作正常。

步骤 3：制作 Feature Map 的初始版本

Agent 还会为生成的验证 Skill 配上一份 **Feature Map**。

Feature Map 由每项面向用户的功能各自对应的功能文件，以及汇总这些文件的目录 features/README.md 组成。Agent 起初会找出约 3~5 项主要功能，分别制作功能文件。

Agent 从以下四处寻找主要功能。

- **路由**.....Web 应用各页面对应的 URL 路径（例如 /settings）。
- **命令**.....CLI 接受的命令或子命令（例如 git commit 中的 commit）。
- **菜单**.....应用界面中列出的菜单项。
- **文档**.....README、使用说明等解释应用功能的文档。

功能文件的内容，以及把 **Feature Map** 作为持续积累的记忆来维护的思路，将在第 36 章介绍。

The map is the repo's maintained verification source; a proof that drives one convenient entry point is incomplete when the map lists others.

Map 是仓库持续维护的验证依据；当 Map 列出其他入口时，只操作一个方便的入口并不能构成完整的证明。

这里的「入口」，是用户到达某项功能的方法，例如按钮、按键操作或命令。

如果 **Feature Map** 列出同一功能的多个入口，Agent 必须验证所有入口，才能证明功能运作正常。凭这一规则，**Feature Map** 也成为界定证明所需覆盖哪些入口的清单。

可以将这条规则应用到随附的 **Feature Map** 示例。示例中，笔记搜索功能的功能文件 `search.md` 如下记载用户进入搜索功能的方法。

How to get to it (user POV)

- Choose the Search button in the browser toolbar.
- Press / in the browser while focus is outside an editable field.
- Run `notes search <query>` in a terminal.

如何进入该功能（用户视角）

- 选择浏览器工具栏中的 Search 按钮。
- 当焦点位于可编辑区域以外时，在浏览器中按 / 键。
- 在终端中运行 `notes search <query>`。

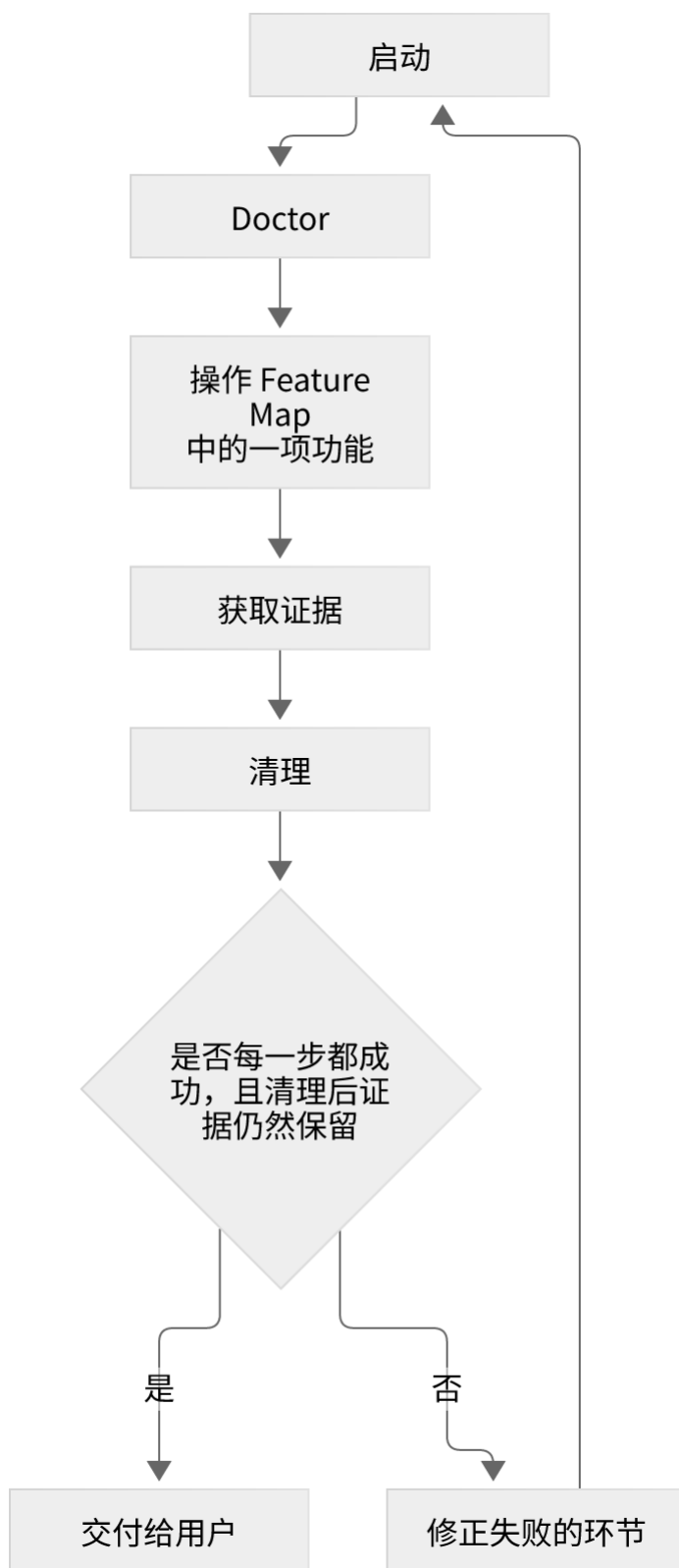
此例中的笔记搜索功能有三个入口：工具栏的 Search 按钮、/ 键以及终端命令 `notes search <query>`。Agent 只通过 Search 按钮验证，证明仍不充分；还需验证另外两个入口。

因此，只要 **Feature Map** 列出同一功能的多个入口，Agent 就必须检查所有入口。

步骤 4：将生成的验证 Skill 从头到尾运行一次

Agent 在将验证 Skill 交给使用者之前，先按其中的指示完整执行一次。操作一项功能即可；其余功能可在之后的运行中依据 **Feature Map** 验证。

执行流程如下图所示。



查看图示 1

Agent 即使在失败的运行中也会执行清理，因此失败尝试启动的进程和占用的端口不会残留。

如果清理之后证据也不见了，Agent 会判定该次运行不合格，因为连证明都没有留下。

此外，生成的验证 Skill 在首次运行前只算草稿，不算成果物。书面步骤的错误只有执行后才能发现。

例如，Launch 中写错端口号，不运行就不会察觉。

最后，Agent 会向使用者介绍负责维护 **Feature Map** 的 `/maintain-verification-skill`。

请求写法：只输入命令即可开始

根据 docs/guide/06-verify-and-ship.md，使用者只需输入以下命令，就可以开始创建 **Feature Map** 和验证 Skill。Agent 会从仓库调查应用如何运行，因此使用者不必预先准备说明。

```
/create-verification-skill
```

实际创建和维护验证 Skill 的过程，将在第 36 章介绍。

`/maintain-verification-skill` 定期修正 Feature Map 与当前应用的偏差

作用：使验证 Skill 和 Feature Map 与当前应用保持一致

`/maintain-verification-skill` 检查验证 Skill 和 **Feature Map** 是否仍与当前应用一致，并修正偏差。

A feature map rots the moment the app changes.

应用一旦改变，**Feature Map** 就开始过时。

应用改变后，**Feature Map** 的记载便可能开始偏离实际应用。所以，**Feature Map** 创建后仍需持续维护，而这项 Skill 正负责维护工作。

使用时机：应用改变后定期运行

应用改变、**Feature Map** 的内容与当前应用出现偏差时，使用者应运行这项 Skill。

poteto 在《The Complete Guide to pstack》的 [Part 1](#) 建议至少每天运行一次，以便让 Agent 操作应用所需的信息始终保持最新。

因此，本书也建议至少每天运行一次，并通过 Routine 等机制将其自动化。

步骤：并行阅读源代码，但只由主 Agent 操作应用

不修改产品代码；遇到回归时报告，不改写 Feature Map 掩盖它

Agent 只可编辑验证 Skill 自身的目录，包括 SKILL.md、features/ 和验证 Skill 附带的脚本。

Agent 在检查期间不编辑产品代码。如果应用没有按照 **Feature Map** 运行，Agent 要辨别偏差来自 **Feature Map** 记载错误，还是产品回归（原先可用的功能坏掉）。前者修正 **Feature Map**，后者向使用者报告。

不能通过修改 **Feature Map** 来消除回归，是因为**若产品已经坏了，却把 Feature Map 改成描述当前错误行为，故障本身便会被掩盖。**

七个步骤：从整理索引到决定是否创建 PR

具体步骤如下。

1. **查找目标**……主 Agent 找到要检查的验证 Skill（通常位于 `.cursor/skills/verify-*/`）。若找不到，就停止并建议使用 `/create-verification-skill`。
2. **整理索引**……主 Agent 对照 **Feature Map** 的目录（README）与功能文件，修复缺失项和失效链接。
3. **阅读源代码**……主 Agent 为每个功能文件并行启动一个只读子 Agent。各子 Agent 对照 **Feature Map** 的描述与当前源代码，指出疑似偏差并附上作为依据的代码位置。
4. **交叉核对**……主 Agent 实际核验步骤 3 提出的部分疑点，并从近期变更中寻找 **Feature Map** 尚未记载的功能。
5. **实际操作**……即使源代码没有发现问题，主 Agent 仍至少操作每项功能一次。
6. **分类**……主 Agent 按下文「发现的问题分为三类，结果只用三种状态之一表示」中的表格，将问题归为三类。

7. **决定是否创建 PR**……若有通过实际操作验证的修正，主 Agent 重读修改的文件，再将改动汇入一份 PR。如果没有需要修正的内容，或无法完成检查，就不创建 PR，而是报告结果与已验证的范围。

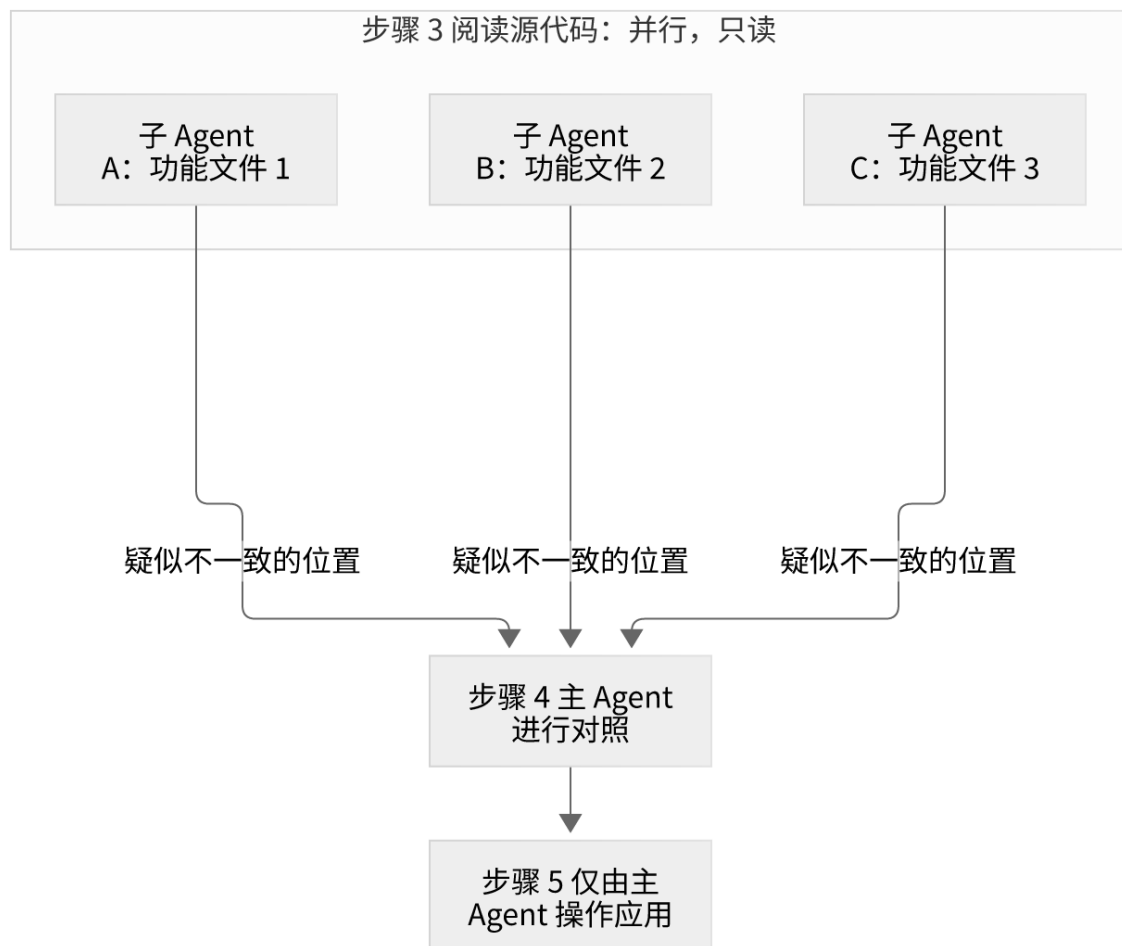
即使子 Agent 在步骤 3 中没有发现某项功能存在偏差，主 Agent 仍会在步骤 5 实际操作它。因为只阅读源代码，无法证明应用会像 **Feature Map** 所述那样运行。证明需要实际运行应用并核验（第 19 章的 Principle 「Prove It Works」）。

多个 Agent 操作同一个应用，会使证据失去可信度

多个子 Agent 并行阅读源代码，不会互相干扰。但多个 Agent 同时操作同一个应用，会让状态混杂，证据因此无法信赖。

例如，一个 Agent 打开的页面被另一个关掉，就无法判断截图反映的是谁的操作。因此，应用只由一个主 Agent 操作。

下图展示步骤 3 到步骤 5 的分工。



[查看图示 2](#)

主 Agent 在操作期间遵守三条规则

主 Agent 在步骤 5 的操作过程中，无论中途出现什么故障，都要遵守以下三条规则。

- 如果实例出现意外行为，下一次操作前先运行 Doctor 检查，重新确认实例是否仍可操作。

Doctor（诊断）……只读检查，判断「操作这个实例来验证是否有意义」（例如进程是否运行、构建是否正确、占用该端口的是否为自己启动的实例、认证是否有效）。

- Cleanup（清理）不能删除已取得的证据。

Cleanup（清理）.....清理由本次运行创建的实例，但不删除证据。

- 为操作而启动的东西，必须全部执行 Cleanup（清理），不能留下。

发现的问题分为三类，结果只用三种状态之一表示

问题分为以下三类。

类别	处理方式
文档偏差（从用户视角描述的功能有误或缺失）	修正 Feature Map
harness 缺口（应用正常运行，但验证 Skill 的操作步骤或脚本无法操作）	修正操作步骤或脚本
产品缺陷（应用行为本身坏了）	为使用者记录；不在这份 PR 中修复

结果只会是以下三者之一：无需修正的 clean；将已验证的修正汇入一份 PR 的 changed；未能完成检查的 blocked。

请求写法：只输入命令即可

docs/guide/06-verify-and-ship.md 也列出仅输入以下命令的请求方式。

```
/maintain-verification-skill
```

SKILL.md 的说明还指出，也可以使用下面的请求调用。

```
audit the verify skill
// 检查验证 Skill。
```

总结

- `/create-verification-skill`……先从仓库代码和配置调查应用如何启动及操作，而非先询问使用者，然后生成验证 Skill 和 **Feature Map**。从启动到清理完整运行一次，确认清理后证据仍在，再交给使用者。
- **Feature Map**……按应用功能记录其能做什么、用户如何进入、如何用验证 Skill 操作，以及能验证什么结果的文档。
- `/maintain-verification-skill`……只编辑验证 Skill 目录；发现产品回归时报告，而不直接修复。至少实际操作每项功能一次，并以无需修正的 `clean`、将已验证的修正汇入一份 PR 的 `changed`，或未能完成检查的 `blocked` 三种状态之一结束。

接下来的第 27 章介绍两项 Skill，用于检查已写好的变更可能破坏什么，以及差异中有哪些弱点：`/blast-radius` 和 `/interrogate`。

第 27 章：检查变更可能破坏的部分及差异中的薄弱点

kaito · 来源：日本語 · English · 2026-10-03

本章介绍以下两项 Skill。

1. `/blast-radius`
2. `/interrogate`

两者都用于检查已经写好的变更。该用哪一项，**取决于想知道什么**。

选择方法如下。

- **改动文件之外还有什么可能坏掉**（例如：改变缓存值的结构后，读取同一缓存的其他服务会不会坏掉）……使用 `/blast-radius`
- **差异中的代码本身是否有弱点**（例如：输入为空或运行两次时，能否正确工作）……使用 `/interrogate`

本章先总览两项 Skill，再分别从使用时机、步骤、输出三个方面说明。原文提供请求示例的 Skill，也会列出示例。

最后会总结两者的分工。

本章结构

本章安排如下。

- 两项 Skill 一览
- `/blast-radius` 通过运行代码，给出证明变更安全的依据
- `/interrogate` 让不同模型的审查者寻找差异中的弱点，由主 Agent 分类意见
- `/interrogate` 检查差异之内，`/blast-radius` 检查差异之外
- 总结

两项 Skill 一览

Skill	一句话概括	主要问题
/blast-radius	寻找修改文件之外可能受损的部分，并通过运行真实代码证明变更安全的依据	这项变更可能破坏什么
/interrogate	不同模型的审查者寻找同一差异中的弱点，主 Agent 将意见归为「处理」「驳回」等四类	请严格指出这份差异的问题

两者都是**只有被点名调用才会运行的 Skill**。

两份 SKILL.md 都设有 `disable-model-invocation: true`，禁止 Agent 根据对话内容自行选择并运行。因此，只有使用者点名调用，或 `/poteto-mode`、其他 Skill 在步骤中明确指定调用时，它们才运行。

两者的调用者有所不同。

- `/interrogate`……`/poteto-mode` 会根据 Playbook 步骤和「Non-negotiables」一节的规则调用它。`/architect` 也会在步骤中调用。使用者也可直接点名调用。
- `/blast-radius`……`/poteto-mode` 的「Non-negotiables」规则、Playbook 步骤及其他 Skill 的步骤均未提及它，所以只有使用者点名调用时才运行。

/blast-radius 通过运行代码，给出证明变更安全的依据

`/blast-radius` 查找看似很小的变更可能在修改文件之外破坏什么，并**通过执行代码核验能够说明「安全」的依据**。

一份听起来可信的影响范围说明，无论对错都同样有说服力；仅凭说明没有价值。因此，使用 `/blast-radius` 的 Agent 不会只回答「影响应该很小」，而会返回通过运行代码核验、能够作为安全依据的事实。

使用时机：看似很小，却不能完全放心的差异

当差异看起来很小，仍想知道它可能在别处破坏什么时使用。

步骤：以执行结果证明变更安全的依据

/blast-radius 的工作并非列出调用方，因为 Agent 用 grep 很快就能生成那份列表。

它要找到 **grep 看不到的故障路径**。例如，如果另一种语言写的服务也读取同一份缓存，这种故障就不会出现在调用方列表中。

因此，Agent 会继续检查库的源代码、执行时机（例如异步任务何时运行、关闭页面时如何清理）、API 返回的 JSON、数据库列、feature flag 等 grep 无法追踪到的地方。

feature flag……不用改写代码，只切换配置就能启用或停用功能的机制（例如只向部分用户显示新页面）

不过，Agent 花时间的重点并不是罗列冗长的「可能出故障」清单，而是**找出一个能够说明变更安全的事实**。

一个事实就可能足够，因为很多看似危险的变更，只要某个关键事实成立，就可以判断为安全。

例如，改变清除缓存的处理时，若能确认「这次调用只会清除已经失效的缓存项，不会做其他事」，就能一次排除许多疑虑，包括误删仍在使用的项。

所以，Agent 会找出这一事实，并用运行真实代码的脚本或测试证明它。

若变更较大或影响范围较广，Agent 会用 /arena（第 24 章）让多个模型调查，因为不同模型发现的故障可能不同。

输出：安全依据、验证深度，以及合并前最简便的检查

输出包括以下五项。

- 发生了什么变化，包括不易从差异中读出的变化
- 使变更安全的一个事实及其证明
- 风险：故障方式、真实存在的 file:line（文件名与行号）、发生可能性与损失、检查方法
- 已检查且确认无问题的事项及原因
- 合并前可执行、能够发现实际可能发生的故障的最简便测试或重现步骤（包括编写的脚本）

此外，Agent 会为每一项安全依据标明验证到了哪一步，方便读者判断能信任到什么程度。

验证分为以下五级，级数越大越可靠。Agent 会在不过度耗费精力的前提下，尽量验证到更高一级。

1. Agent 仅用文字陈述依据（单凭这一步没有价值）。
2. Agent 指出支持依据的代码行（file:line），或所调用库源码中的对应行。
3. Agent 逐步追踪可能导致所担忧故障（例如仍在使用的缓存项被删除）的处理流程，并说明流程无法走到该故障。
4. Agent 执行调用真实代码的脚本或测试（如果依据错误，就会明确失败）。
5. Agent 在运行中的应用里实际操作，确认行为符合所述依据。

以刚才的「这次调用只会清除已经失效的缓存项，不会做其他事」为例，逐级验证的方法如下。

- 第 1 级，只写「应该只会清除失效项」。
- 第 2 级，指出清除项目的函数的 file:line，或所调用库源码中的对应行。
- 第 3 级，逐步追踪传入仍有效项目时的处理流程，说明该项目不会到达清除步骤。
- 第 4 级，加载应用使用的同一个库并调用该函数，执行脚本确认有效项目仍在；若依据错误，脚本会明确失败。
- 第 5 级，在运行中的应用实际执行清除缓存的操作，确认有效项目仍在。

验证深度决定依据有多可信，因此 Agent 需要逐项说明验证到了哪一级。

如果无法证明，就标注 unproven（未证明）。

/interrogate 让不同模型的审查者寻找差异中的弱点，由主 Agent 分类意见

/interrogate 为通过 /setup-pstack（第 34 章）配置的每个审查模型各启动一名审查者。每名审查者都围绕「这份差异会在哪里出错」进行审查，再由主 Agent 对意见分类。

不同模型的盲点也不同。因此，若两个模型独立提出同一个问题，这个问题是真实缺陷的可能性较高。

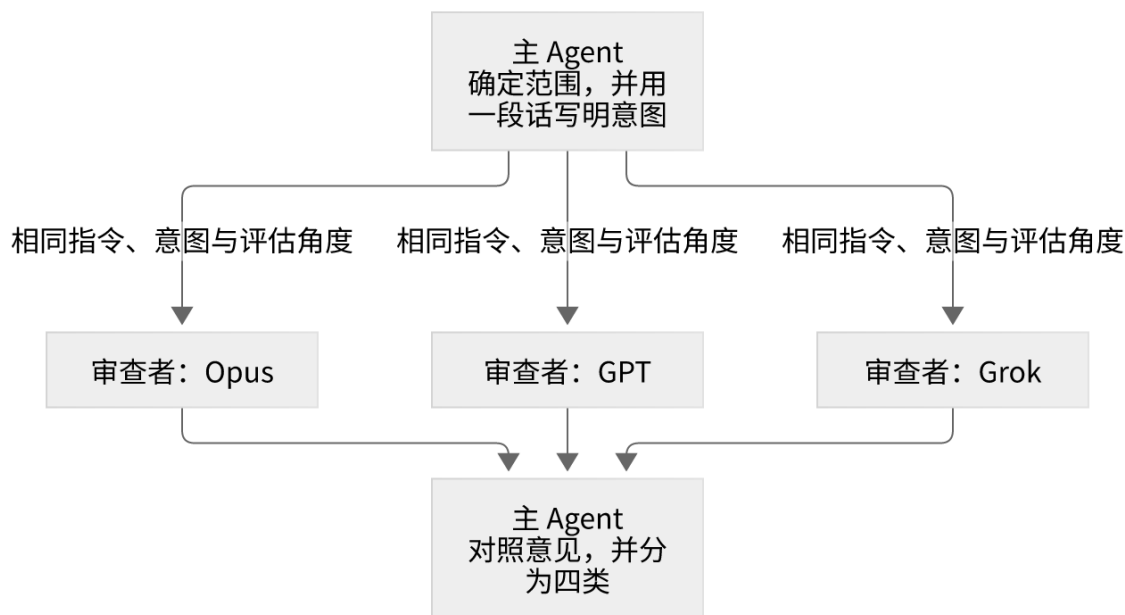
使用时机：希望多个模型寻找差异中的弱点

当希望从严格的代码质量角度找出差异可能出错的地方，例如要求「以对抗性视角审查」「找出盲点」时使用。

步骤：审查者接受变更意图，重点寻找实现代码的弱点

主 Agent 先确定审查范围（未指定时为 `git diff main...HEAD`），再用一段话写明变更意图。意图来自使用者请求、提交信息及 PR 描述；若无法确定，就先询问使用者。

随后，默认分别启动 Opus、GPT 和 Grok 三个模型的只读审查者。给所有审查者相同的指示和评估维度，是为了让意见差异来自模型本身，而非角色分工。



查看图示 1

主 Agent 最重视至少两个模型独立提出的意见；只有一个模型提出的意见，权重较低。

审查者将变更目标（意图）视为正确，严格检查代码是否妥善实现这一目标。

例如，意图是「取消运行时将原本逐条获取的运行记录批量获取，消除 N+1」，审查者不会质疑「是否应该批量获取」。它要调查的是「批量查询会不会连不该取消的运行记录也查出来」。

审查者按以下维度评估。

- **正确性**……代码能否按意图运行（例如空输入、运行两次、上次运行中途停止时）。
- **根本原因**……是修复根因，还是只掩盖症状（例如是否用重试掩盖已经失效的前提）。
- **结构健全性**……改动是否融入系统结构（例如验证是否集中在边界、引入新 API 后是否仍留下旧 API）。
- **验证**……是否有办法确认代码运作正常（例如修复故障时，是否有重现故障的测试）。
- **复杂度预算**……复杂度是否与代码承担的职责相称（例如只有一个调用方的抽象、为尚未存在的情况设置配置）。
- **安全性**……危险输入是否存在未经检查的传递路径（例如用户输入是否未经验证就传给 SQL 或 shell）。

除这些维度外，审查者也会寻找在不改变外部行为的前提下，大幅简化实现的方法，例如通过改写完全去除条件分支或辅助函数。

因此，审查者会提出广泛的意见。主 Agent 掌握完整上下文，负责驳回只是个人偏好的改写，或实际上不可能发生的假设（例如调用方不可能传入 null，却指出「传入 null 会崩溃」）。审查者只看到了差异和一段意图，不知道此前已尝试且放弃的方案，也不知道代码之外的约束。

输出：将意见分为 Act on、Consider、Noted、Dismissed

主 Agent 不只是中立的汇总者，而会像有经验的工程师那样，将所有意见归为四类。下表沿用刚才消除 N+1 的差异作为例子。

分类	含义	示例
Act on（应处理）	确实影响正确性、安全性或可维护性的问题。在真实 PR 中会阻止合并	批量查询把已完成、不可取消的运行记录也查出来
Consider（可考虑）	建议合理，但难以判断现在处理是否划算	缓存查询结果可以加速，但当前数据量是否需要尚不清楚
Noted（记下即可）	技术上正确，但目前不处理	变量名称可以写得更具体
Dismissed（驳回）	错误、过于琐碎，或因不了解上下文而提出的意见。需简短附上驳回理由	「记录太多会变慢」的意见；调用方已经限制数量，因此驳回

为 Dismissed 保留理由，是为了让使用者看到主 Agent 驳回了什么、为什么驳回，并自行重新考虑不同意的意见。

主 Agent 只返回分类后的结果，不会自动应用意见提出的修改。

请求示例：在合并有争议的设计前使用

/poteto-mode 的「Non-negotiables」规定，合并有争议的设计前要使用 /interrogate。此外，「Feature」Playbook、创建 PR 的「Opening a PR」Playbook，以及第 24 章介绍的 /architect 也会调用它。

随附指南 04-design.md 给出以下请求示例。

```
/interrogate the whole branch, but skeptically. no nitpicks unless it's an actual bug or regression.
// 以怀疑的态度审阅整个分支。只有真正的缺陷或回归才需要指出，不必纠缠细枝末节。
```

对抗性审查者找不到严重问题时，往往会用琐碎意见填满报告。若使用者像示例一样明确不要琐碎意见，Act on 中的意见就更有可能是真正的问题，也更值得阅读。

/interrogate 检查差异之内，/blast-radius 检查差异之外

两者都在调查「这项变更是否可靠」，但 /interrogate 让多个模型审查差异内部，/blast-radius 则检查差异之外，并通过执行代码证明安全依据。

总结

- /blast-radius……找出使变更安全的一个事实（例如这次调用只清除已经失效的缓存项），并执行调用真实代码的脚本或测试予以证明。
- /interrogate……Opus、GPT 和 Grok 的审查者寻找同一份差异的弱点，主 Agent 将意见分为 Act on、Consider、Noted、Dismissed 四类，并为驳回的意见说明原因。
- 使用区别……/interrogate 检查差异之内，/blast-radius 检查差异之外。

接下来的第 28 章介绍 /show-me-your-work：记录 Agent 的判断和依据，让人之后能沿证据回溯检查。

第 28 章：记录决策，用证据追踪 Agent 的长时间工作

kaito · 来源：日本語 · English · 2026-10-03

本章介绍 `/show-me-your-work`。

`/show-me-your-work` 是一项 Skill，用于把 Agent 作了什么决定（例如舍弃子 Agent 的成果）及其依据（例如截图一片空白）逐条记录在文件中。记录会交给与执行工作的模型不同系列的模型检查。Agent 最后在回复的「Attention」栏汇总检查发现的注意事项。

第 26 章的验证 Skill 用证据说明应用确实能够运行。`/show-me-your-work` 则让使用者日后可以沿证据追溯，核实 Agent 为何作出某项判断。

`/show-me-your-work` 的 SKILL.md 设置了 `disable-model-invocation: true`（第 25 章）。Agent 不会根据对话内容自行选择并运行设有此选项的 Skill。因此，只有使用者点名调用，或 `/poteto-mode`、Playbook、其他 Skill 在步骤中明确指定调用时，它才运行。

本章从作用、使用时机、步骤和请求写法四方面介绍 `/show-me-your-work`。

本章结构

本章安排如下。

- 作用：只留下一份可供日后检查的判断记录
- 使用时机：长时间工作、自主工作，以及日后要核查的工作
- 步骤：记录判断，与 transcript（Agent 对话记录）核对，再交给另一模型检查
- 请求写法：要求开始记录
- 总结

作用：只留下一份可供日后检查的判断记录

对于长时间工作，或使用者离开期间持续进行的工作，`/show-me-your-work` 会把可供日后检查的判断记录集中放在一份文件中。

随附指南 [docs/guide/07-overnight.md](#) 指出，交给 Agent 长时间工作后离开，不能只寄望一切顺利，还需要以下三项条件。

- 可验证的完成条件
- 不与其他工作冲突的专用 `worktree`（第 18 章）
- 供日后检查的判断记录

这项 Skill 负责第三项「供日后检查的判断记录」。

需要记录判断，是因为**代码差异只包含最终保留下来的变更**。中途舍弃的方案、撤销子 Agent 成果的决定，都不会出现在差异中。

使用者离开期间，Agent 可能已工作很久；使用者回来后手头只有差异。因此，Agent 记录判断及依据，让使用者日后能检查为何形成这个结果。

使用时机：长时间工作、自主工作，以及日后要核查的工作

`/poteto-mode` 的「[Non-negotiables](#)」一节（第 8 章）要求在长时间工作、自主工作、多阶段工作，以及使用者离开后再核查的工作中保留记录。

`/show-me-your-work` **也由 Playbook 和其他 Skill 调用**

负责长时间工作的 Playbook 和 Skill 也会使用这项 Skill 留下记录。

这项 Skill 的 `SKILL.md` 规定了其他 Skill 记录判断时的规则。

其他 Skill 不自行设计记录格式，而是点名调用这项 Skill，把格式交由它决定。

因此，不论由哪个 Playbook 或 Skill 调用，记录的格式都一致，使用者可以用同一方式检查。

以下列出几个典型调用者及其记录原因。

「Autonomous run」：每次迭代都记录改了什么、是否接近完成条件

「Autonomous run」是先确定完成条件，再不中途停顿、持续工作直到完成的 Playbook（第 15 章）。

它在步骤 5 规定，每次迭代都用这项 Skill 记录一行：该轮修改了什么，以及是否更接近完成条件。

「Orchestrate」：把持续多日的项目判断汇入同一份记录

「Orchestrate」是由一名不编写代码的协调 Agent，管理持续多日、涉及多个 PR 的工作所用的 Playbook（第 16 章）。

这份 Playbook 规定，项目记录目录中的 `decisions.tsv` 就是这项 Skill 的记录。

协调 Agent 在开始启动子 Agent 前的准备步骤（Install the runtime）开始记录，并在最后的 Close 步骤按本 Skill 的规则核对记录，再交由不同系列的模型检查。

这份 Playbook 的设计前提是使用者约每天查看两次，而非每五分钟查看一次。Close 步骤也规定，不删除含记录的目录，而要保留下来作为事后复盘资料（postmortem）。

/figure-it-out：记录与差异俱全，使用者才能信任工作

/figure-it-out 是为使用者请求不符合现有 Playbook 的大型任务设计 Playbook 本身的 Skill（第 23 章）。

/figure-it-out 在保留工作记录的阶段（Phase D）调用这项 Skill。它处理的工作通常较大，因此 Agent 会提交记录，让审查者在 PR 中阅读。因为只有记录和差异同时具备，使用者才能信任工作。

步骤：记录判断，与 transcript（Agent 对话记录）核对，再交给另一模型检查

格式：每行一项判断，用六列 TSV 记录

这项 Skill 用 TSV（以制表符分列的文本表格）保存判断记录，方便检查者从上到下阅读并沿证据核查。

※ TSV 示例见下文。

每行只写一项判断；证据不写成描述性文字，而是写成指向证据位置的指针，例如提交或文件。若只用文字描述证据，检查者看不到实物，只能相信描述。指针则让人找到实物核验。

共有以下六列，依次回答何时、在哪一阶段、决定什么、为何决定、依据是什么、结果如何。

列	内容	示例（下方记录的第一行）
ts	时间戳	2026-05-24T11:15:00Z
phase	作出判断时所在的阶段，或工作流程名称	widget
decision	选择了什么、做了什么	moved the widget styles over without changing how it looks
why	用通俗语言说明作出判断的原因	keep the change small and the result identical
evidence	提交 SHA、PR 编号、file:line（文件名与行号）、截图路径等	commit 7c21e0a, pixel-diff 0
result	tests green、reverted、open 等结果	looks identical, tests pass

从 SKILL.md 的示例中摘出表头和两行记录，TSV 如下（列之间是制表符）。

```
tsphase decisionwhy evidenceresult
2026-05-24T11:15:00Zwidgetmoved the widget styles over without changing how it looks keep the change
small and the result identical commit 7c21e0a, pixel-diff 0 looks identical, tests pass
2026-05-24T12:30:00Zwidgetthrew out a helper's work because its screenshots were blank checked the
real files instead of trusting its summary worktree resetreverted, tightened the instructions for
next time
```

第一行记录了「在不改变外观的前提下迁移 widget 样式」的判断。证据是提交 7c21e0a 和像素差为零（pixel-diff 0）。检查者可以打开该提交核对实物。

第二行记录了「舍弃子 Agent 的成果，因为截图一片空白」。Agent 没有相信子 Agent「已经完成」的摘要，而是在检查实际文件后将成果舍弃。

Agent 记录的是关键决策点，而非每一个动作。理所当然的事或琐碎动作不必记。

例如，以下时机可以各记一行。

- 从两种方法中作出选择时
- 完成一个工作阶段并得到检查结果时
- 改变方针或撤销变更时
- 遇到无法继续推进的问题时

Agent 用通俗语言写每一行。去除 AI 风格措辞的 /unslop（第 32 章）规则也适用于日志。

追加与存放位置：使用 log.sh 追加判断记录

Agent 使用 log.sh 逐行追加

Agent 使用 `scripts/log.sh` 添加记录行。

```
scripts/log.sh <logfile> <phase> <decision> <why> <evidence> <result>
```

脚本会自动填写时间戳，将单元格里的制表符和换行替换为空格，整理成 TSV 格式。

日志放在工作目录中，仅在大型任务中提交

Agent 将日志放在工作目录的 `decisions.tsv`，默认不提交。只有工作规模大到检查者需要判断记录才能相信结果时，Agent 才提交日志。

例如「在不改变外观的前提下迁移约 100 个组件的样式」就是这种情况。

阅读代码可以知道改了什么，却无法得知 Agent 如何确认外观确实没变，以及中途舍弃了哪些方法；这些不会留在代码里。

因此，任务越大，这类判断越多。检查者便可沿提交的日志追踪判断和证据，核验报告的成果。

判断有误时追加更正行，不改写原行

日志只能追加；判断有误时，追加一行更正，而不改写或删除旧行。否则，中途改变判断的经过会从记录中消失。

新一轮运行先写 start 行，与上一轮记录分开

这项 Skill 将与一名 Agent 的一场对话视为一轮运行（run）。使用者在同一对话中随后追加请求，Agent 接着完成的工作，仍属于同一轮。对话太长而被摘要后继续进行的工作也一样。

另一方面，另一名 Agent 接手工作或开始新的聊天，就属于新一轮运行。

长时间工作未必能在一轮运行中结束。中途可能由另一名 Agent 接手，也可能由使用者在新聊天中要求继续。这项 Skill 规定，一项工作的记录要汇入一份日志（工作目录的 decisions.tsv），因此新一轮也会继续追加到上一轮写的日志中。

于是，一份日志里会有多个运行写入的记录。若无法辨别哪些行由哪一轮写成，检查者就无法单独阅读新一轮的判断。

所以，向已有记录的日志追加时，Agent 要先写一行 phase 列为 start 的记录。

这行 start 用于标记其下方由新一轮运行写入。该行要写明上方并非自己写入的记录所涵盖的时间戳范围，以及标识本轮运行的信息（如 Agent ID）。

例如，新的聊天接着上一场聊天写的日志追加，结果如下。

```
tsphase decisionwhy evidenceresult
2026-05-24T11:15:00Zwidgetmoved the widget styles over without changing how it looks keep the change
small and the result identical commit 7c21e0a, pixel-diff 0 looks identical, tests pass
2026-05-24T12:30:00Zwidgetthrew out a helper's work because its screenshots were blank checked the
real files instead of trusting its summary worktree resetreverted, tightened the instructions for
next time
2026-05-25T09:00:00Zstart rows 2026-05-24T11:15:00Z to 12:30:00Z were written by another run picking
up yesterday's logagent a1b2c3open
2026-05-25T09:40:00Zwidgetmoved the button styles over same approach as the widget commit 9d4e2f1,
pixel-diff 0 looks identical, tests pass
```

时间戳为 2026-05-25T09:00:00Z 的行，是当天运行最先写下的 start 行。检查者只需阅读这一行以下的內容，就能核查当天运行的判断。

下文的「核对：交付使用者前，将日志与 transcript（Agent 对话记录）比对」中，Agent 也只会将本轮写的行与本轮 transcript 对照。start 行标明本轮所写内容从哪里开始。

核对：交付使用者前，将日志与 transcript（Agent 对话记录）比对

完成工作的 Agent 在返回结果前，会核查记录是否符合事实。具体来说，将本轮写的行与本轮 transcript 核对以下几点。

- 每行是否对应实际作出的判断
- 证据是否真实存在，并支持该行所述内容
- 决定工作方向的重要分歧点是否漏记

Correct the log, not the story. The audit never edits or removes a row, even an invented one.

要更正的是日志里的错误，不是改写事情的经过。核对时即使发现某行记载了并未发生的事，也不编辑或删除该行。

即使核对时发现记录不实，Agent 也不删除或改写原行，而是追加说明真实经过的更正行。这与本章前文「判断有误时追加更正行，不改写原行」的规则相同。

以核对时发现某行列出一个不存在的提交作为证据为例。下面第一行与事实不符，第二行是 Agent 追加的更正。

```
tsphase decisionwhy evidenceresult
2026-05-24T13:00:00Zwidget修复了按钮颜色为修复外观问题commit 9d4e2b1tests green
2026-05-24T15:20:00Zwidget更正 13:00 的记录：按钮颜色并未修复，commit 9d4e2b1 也不存在 对照执行记录
后发现原记录与事实不符 agent-transcripts/ 中本次执行的记录 open
```

第一行保留不删，检查者就能追踪记录何时偏离事实、又何时得到更正。

检查：不同系列的模型阅读记录

执行工作的 Agent 会用与执行工作所用模型不同系列的模型启动一名子 Agent，交给它阅读日志和 transcript。这名负责检查的子 Agent 不重做工作，而是寻找使用者需要注意的事项，例如：

- 依据薄弱或没有依据的判断
- 跳过的验证，或 transcript 没有证据却声称已完成的验证
- 回头看存在风险的选择，例如：
 - 过早下结论
 - 工作范围扩张过度
 - 只让症状（错误或画面错位等表面故障）消失、根因却仍在的修正
- 使用者略读日志时难以发现的问题

仅让执行工作的模型重新检查自己的记录，不能替代这种检查。因为**执行工作的模型，即使重新审阅，也容易再次忽略工作时遗漏的事项。**

随附指南 <docs/guide/04-design.md> 对让多个模型检查差异弱点的 /interrogate（第 27 章）也提出相同的理由。

Agent 在报告所记录工作的回复末尾，必须加上「Attention」栏，列出检查者发现的使用者应留意的事项。例如，检查者核查本章示例中的以下两行。

迁移 widget 样式的判断行^[1]如下。

```
2026-05-24T11:15:00Zwidgetmoved the widget styles over without changing how it looks keep the change
small and the result identical commit 7c21e0a, pixel-diff 0 looks identical, tests pass
```

把不存在的提交写作证据的行^[2]如下。

```
2026-05-24T13:00:00Zwidget修复了按钮颜色为修复外观问题 commit 9d4e2b1 tests green
```

此时，「Attention」栏可能如下所示（此例并非原文所载）。

```
Attention
reviewed by grok-4.7-xhigh-fast
```

- decisions.tsv 中 11:15 的记录：pixel-diff 0 的证据只有首页的一张截图；其他使用该组件的页面尚未验证。
- decisions.tsv 中 13:00 的记录：把不存在的 commit 9d4e2b1 作为证据。15:20 的记录已作更正，但按钮颜色仍未修复。

第一行 reviewed by grok-4.7-xhigh-fast 标明由哪个模型检查。其下逐项列出检查者的意见，每项都指出涉及 decisions.tsv 的哪一行（或 transcript 的哪个片段）。使用者可以沿意见找到对应记录，核对证据。

如果没有意见，以「No flags」（无须注意事项）代替列表。不能只写 reviewed by 而既不列意见、也不写「No flags」。即使写「No flags」，也不可省略 reviewed by 行。

```
Attention
reviewed by grok-4.7-xhigh-fast
```

```
No flags
```

因为缺少 reviewed by 行，使用者就无法确认检查是否真的由不同系列的模型完成，而非由执行工作的模型自行复查。

请求写法：要求开始记录

在 [README](#) 中，有开始记录请求示例。

```
/show-me-your-work keep a decision trail i can review when i'm back.
// 留下决策记录，方便我回来后复查。
```

使用者回来后，可以按 docs/guide/07-overnight.md 中的以下请求，收到前一晚工作的报告。报告末尾会附上「Attention」栏。

```
/show-me-your-work catch me up on what you did last night
// 告诉我昨晚完成了什么。
```

使用者先阅读回复中的「Attention」栏，再查看该栏指向的日志行。由于栏中只列检查者找到的注意事项，使用者不必重新阅读整个长时间任务的记录。

总结

- **作用**……对于长时间工作或使用者离开期间进行的工作，将可供日后检查的判断集中保存在一份文件中；代码差异只显示最终保留下来的变更。
- **记录格式**……用六列 TSV 每行记一项判断，只能追加。错误的行不删除，而是追加更正行。
- **核对**……在向使用者返回结果前，将日志与 transcript（Agent 对话记录）比对；不实记录用新行更正。
- **检查与「Attention」栏**……不同系列的模型阅读日志和 transcript，Agent 以「Attention」栏结束回复，列出检查模型的名称和意见。即使没有意见，也不能省略模型名称。

接下来的第 29 章至第 31 章，每章分别介绍一项维护代码质量的 Skill。第 29 章介绍汇总 TypeScript 规则的 [/typescript-best-practices](#)。

脚注

1. 引用自本章「格式：每行一项判断，用六列 TSV 记录」的示例。↩
2. 引用自本章「核对：交付使用者前，将日志与 transcript（Agent 对话记录）比对」的示例。↩

第 29 章：用十六条 TypeScript 规则维护代码质量

kaito · 来源：日本語 · English · 2026-10-03

本章介绍 `/typescript-best-practices`。

`/typescript-best-practices` 用于指导 Agent 编写 TypeScript：通过类型排除不可能的状态（例如正在加载却同时有错误值），并在外部数据进入系统的地方只验证一次。

从本章到第 31 章的三章，分别介绍以下三项维护 Agent 所写代码质量的 Skill。

1. `/typescript-best-practices`（本章）
2. `/tdd`（第 30 章）
3. `/no-comments`（第 31 章）

本书将三者归为**不只依赖编写代码的 Agent 自己留心，而让其他机制核验其成果的 Skill**。对于 `/typescript-best-practices`，代替编写者检查的是编译器。

本章从作用、使用时机、规则和请求写法四方面介绍 `/typescript-best-practices`。

本章结构

本章安排如下。

- 作用：将「Type System Discipline」落实为具体的 TypeScript 写法
- 使用时机：读取或编辑 TypeScript 文件时自动加载
- 规则：TypeScript 写法的 16 条规则
- 请求写法：即使不提 Skill 名称，Agent 也会遵守规则
- 总结

作用：将「Type System Discipline」落实为具体的 TypeScript 写法

`/typescript-best-practices` 是一套 TypeScript 规则。

第 18 章的 Principle 「**Type System Discipline**」解释了为何要让类型无法表示无效状态。本 Skill 则用具体的 TypeScript 写法展示如何实现。

其 `SKILL.md` 在开头要求 Agent 先应用这一 Principle，再使用 Skill 中的规则。

使用时机：读取或编辑 TypeScript 文件时自动加载

Agent 读取或编辑 TypeScript 文件时，这项 Skill 会自动加载，因为 `SKILL.md` 的 `paths` 设置指定了 TypeScript 文件。

该 Skill 设置了 `disable-model-invocation: true`（第 25 章），因此 Agent 不会根据对话内容自行选择并运行它。

由于自动加载，Agent 不会忘记调用，使用者也无需点名。

规则：TypeScript 写法的 16 条规则

Skill 包含 16 条规则，以及 `references/patterns.md` 中的代码示例。

`references/patterns.md`……汇总 `SKILL.md` 中各条规则对应的代码示例的文件

16 条规则中，除「真实测试」和「结构化日志（Structured telemetry）」外，其余 14 条都有代码示例

随附指南 `docs/guide/05-build-and-clean.md` 称，这项 Skill 把类型系统的原则转化为具体的 TypeScript 规则。

依次概括 16 条规则如下。

规则	概述
discriminated union	按状态用类型限定可有字段，使「加载中却同时有错误」之类的矛盾值无法写出
品牌类型	即使底层都是字符串，也用类型区分含义不同的值（例如 ID 与普通字符串），避免混淆
构造式建模（Constructive modeling）	对于「非空数组」等有约束的值，使用无法构造不满足条件之值的类型
足够简单的类型（Simplest total type）	只在现有类型迫使代码采用不自然写法之处收紧类型
any 不如 unknown	用一种在检查内容前不能直接使用的类型接收外部数据
schema 优先于类型守卫（Schemas before guards）	手写数据验证逻辑前，先使用项目现有的验证库
as 不应使用	不在未检查值的情况下断言「这个值属于此类型」
类型收窄的优先顺序（Narrowing hierarchy）	按安全程度依次选择确认值类型的方法
类型守卫（Type guards）	判断「是否属于此类型」的函数应真正验证名称所声称的条件
穷尽性检查	新增一种状态时，通过编译错误提示遗漏了对应处理
as 不如 satisfies	在不丢失值的具体类型信息的情况下，检查它是否符合某类型
边界验证	外部数据在入口验证一次，内部不重复验证
从现有类型推导（Schema-derived types）	编写新类型前，先从已有类型提取需要的部分
用对象传递参数（Object args）	为参数标明名称，避免传错顺序
真实测试	本地能运行的组件，用真实组件测试，不用替身
结构化日志（Structured telemetry）	按固定结构保存可用于调查的信息，console.log 不留在代码中

下面结合 `references/patterns.md` 的代码示例逐条介绍。示例中的注释为日文。

1. 用 discriminated union 排除不可能的状态

discriminated union 是用 `kind` 等标识种类的字段，区分各形态（variant）所含字段不同的数据类型（第 18 章）。

variant……同一个类型可能具有的各种形态之一

例如下文的 DiffState 有 { kind: "loading" }、{ kind: "ready"; diff: GitDiff }、{ kind: "error"; error: string } 三种 variant

references/patterns.md 把以布尔值和可省略字段（optional 字段）定义的类型列为反例。

反例允许写出同时包含 loading: true 和 error 的值，即「正在加载却同时出错」的矛盾状态。

正例按 kind 限定每种状态可包含的字段，只能写出三种有效状态。

```
// 差异（修改过的文件清单）
type GitDiff = { files: string[] };

// 错误示例：布尔值加可选字段的组合，也能表示相互矛盾的状态
type DiffStateLoose = { loading: boolean; diff?: GitDiff; error?: string };

// 正确示例：只能表示有效状态
type DiffState =
  | { kind: "loading" }
  | { kind: "ready"; diff: GitDiff }
  | { kind: "error"; error: string };
```

因此，当不同状态拥有不同字段时，应使用 discriminated union，而不是布尔值加可选字段。

2. 用品牌类型防止混淆含义不同的值

品牌类型是在字符串等基础值上加上 & { readonly __brand: "AgentId" } 这样的标记（第 18 章）。这样可以在编译阶段防止把未经验证的字符串传给需要 Agent ID 的函数。

下面的例子里，位于数据入口的 parseAgentId 先验证字符串是否符合 UUID 格式，再将其转成 AgentId。as 只在验证后的一个位置使用。

```
type AgentId = string & { readonly __brand: "AgentId" };

// 检查字符串是否符合 UUID 格式
declare function isUUID(input: string): boolean;

// 边界：验证字符串后再将其转为 AgentId
function parseAgentId(input: string): AgentId {
  if (!isUUID(input)) throw new Error(`Invalid agent id: ${input}`);
  // 已经验证，所以这里可以使用 as
```

```

    return input as AgentId;
}

// 内部：只接收 AgentId，不重复验证 ID
function focusAgent(id: AgentId): void {
    /* 视为已经验证的输入 */
}

// 可以通过
focusAgent(parseAgentId("3f2b8c1e-9a4d-4e6b-8f1a-2c5d7e9b0a13"));
// 未经验证的字符串会导致编译错误
focusAgent("3f2b8c1e-9a4d-4e6b-8f1a-2c5d7e9b0a13");

```

标记统一采用 readonly __brand: "X"，不另造写法。

因此，内容类型相同、含义却不同的值应使用品牌类型，并在边界验证后赋予该类型。

3. 定义无法构造无效值的类型

这条规则（原文称 Constructive modeling）要求用有效的组成部分构造类型，而不是先用宽松类型，再靠运行时检查拦截无效值。

```

// 非空数组：第一个元素一定存在
type NonEmpty<T> = [T, ...T[]];

// 错误示例：接收 T[]，让各调用方重复检查数组是否为空
function pickWinnerLoose(entries: string[]): string {
    if (entries.length === 0) throw new Error("no entries");
    return entries[Math.floor(Math.random() * entries.length)];
}

// 正确示例：这种类型无法表示空数组
function pickWinner(entries: NonEmpty<string>): string {
    return entries[Math.floor(Math.random() * entries.length)];
}

// 接收 T[] 后，用类型守卫只缩小一次类型；此后类型会承载已验证的事实
const isNonEmpty = <T>(arr: T[]): arr is NonEmpty<T> => arr.length > 0;

```

因此，在运行时排除无效值之前，应先定义无法构造无效值的类型。

4. 只有宽松类型造成困难时才加强类型

这条规则（原文称 Simplest total type）并不要求把所有类型都收紧。只要针对数组的每项操作对包括空数组在内的所有输入都能返回结果，就继续使用 `T[]`。

只有宽松类型迫使使用位置采用不自然的写法时，才需要加强类型。其迹象包括！（非空断言运算符）、`arr[0] as T`（类型断言）以及「按理说不会出现」的错误。

```
type NonEmpty<T> = [T, ...T[]];
type Session = { id: string; startedAt: Date };

// 可以继续使用 T[] 的例子：空数组也能返回 0
const sum = (xs: number[]) => xs.reduce((a, b) => a + b, 0);

// 错误示例：用 ! 向编译器隐藏数组为空时的处理问题
function newestSessionLoose(sessions: Session[]): Session {
  return sessions.at(0)!;
}

// 正确示例：强化参数类型后，不再需要 !
function newestSession(sessions: NonEmpty<Session>): Session {
  return sessions[0];
}
```

`references/patterns.md` 也把将返回值放宽为 `Session | undefined` 列为另一种正确写法。

因此，是否加强类型，要看宽松类型是否迫使代码使用！（非空断言运算符）等写法。

5. 用 unknown 接收外部数据

用 `any` 接收的值，无需验证也能使用；用 `unknown` 接收的值，在确认类型前不能使用，Agent 因而无法跳过验证。

```
// 错误示例：接收 any 后，无须验证就能使用
function handleLoose(input: any) {
  // 即使没有 foo，也能通过编译
  return input.foo.bar;
}

// 正确示例：接收 unknown，缩小类型后再使用
function handle(input: unknown) {
  if (typeof input === "object" && input !== null && "foo" in input) {
    // 此处编译器已经确认 input 有 foo 字段
  }
}
```

```

    }
  }
}

```

因此，如果值的结构要到运行时才知道，应先以 `unknown` 接收，收窄类型后再使用。

6. 手写类型守卫前，先用仓库的 schema 库

对于外部数据，Agent 在手写逐字段检查的类型守卫（判断值是否属于某类型并返回布尔值的函数）之前，应先查找仓库使用的运行时 schema 库和现有 schema。

由一份 schema 负责验证，并从中推导 TypeScript 类型。若分别维护 schema、相同结构的 interface 和类型守卫，修改时容易漏掉其中一份，造成不一致。

```

import { z } from "zod";

const UserSchema = z.object({
  id: z.string().uuid(),
  role: z.enum(["admin", "member"]),
});

// 从 schema 推导类型
type User = z.infer<typeof UserSchema>;

function parseUser(input: unknown): User {
  return UserSchema.parse(input);
}

```

因此，外部数据应使用仓库已有的 schema 验证，并从同一 schema 推导类型。

7. 验证值后才使用 as

`as`（类型断言）是在未验证值的情况下，向编译器断言「这个值属于此类型」的写法。即使实际类型不同，编译也可能通过，使用该值的代码则可能在运行时出错。

```

type User = { id: string; name: string };
declare const data: unknown;

// 错误示例：未经验证就断言它是 User
const user = data as User;

// 正确示例：在边界验证后使用 as
function parseUser(data: unknown): User {

```

```

if (typeof data !== "object" || data === null) {
  throw new Error("expected object");
}
if (!("id" in data) || typeof data.id !== "string") {
  throw new Error("expected id");
}
// .....其余字段也全部验证
// 所有字段均已验证，因此可以使用 as
return data as User;
}

```

从现有代码中去除 `as` 时，Agent 要查明 TypeScript 无法推断类型的原因。

- 缺少标识种类的字段 → 添加字段，改用 discriminated union
- 原类型过宽（例如 `Record<string, unknown>`）→ 收窄类型
- 边界缺少类型 → 添加解析函数或 schema
- 类型确实无法表达 → 使用品牌类型或 `satisfies`

因此，使用 `as` 前必须先验证值。

8. 收窄类型有优先顺序

`references/patterns.md` 按以下顺序优先选择收窄类型的方法。

1. 对标识种类的字段使用 `switch` 或 `if`（编译器自动收窄）
2. `in` 运算符（用 `"key" in obj` 收窄到包含该字段的 variant）
3. `typeof` 与 `instanceof`（适用于基础值和类实例）
4. 自定义类型守卫（前三种不足时使用）
5. `as`（仅在验证值后使用）

下面的代码分别示范这五种方法。

```

type Shape =
  | { kind: "circle"; radius: number }
  | { kind: "rect"; width: number; height: number };

// 1. 对表示类别的字段使用 switch: 检查 kind 后，编译器会自动缩小类型
function area(s: Shape): number {
  switch (s.kind) {
    case "circle":
      return Math.PI * s.radius ** 2;
  }
}

```



```

    case "rect":
        return s.width * s.height;
    }
}

// 2. in 运算符: 只有圆形有 radius, 因此这里缩小到圆形
function areaByIn(s: Shape): number {
    if ("radius" in s) return Math.PI * s.radius ** 2;
    // 这里缩小到矩形
    return s.width * s.height;
}

// 3. typeof 与 instanceof: 缩小到基本类型或类实例
function describe(value: string | number | Date): string {
    if (typeof value === "string") return value;
    if (value instanceof Date) return value.toISOString();
    // 这里缩小到 number
    return value.toFixed(2);
}

// 4. 自定义类型守卫: 用函数检查前三种方法无法确认的条件 (是否为非空字符串)
function isNonEmptyString(value: unknown): value is string {
    return typeof value === "string" && value.length > 0;
}

// 5. as: 只在值经过验证后使用
type AgentId = string & { readonly __brand: "AgentId" };
declare function isUUID(input: string): boolean;

function toAgentId(input: string): AgentId {
    if (!isUUID(input)) throw new Error(`Invalid agent id: ${input}`);
    return input as AgentId;
}

```

因此, 收窄类型时应优先使用列表中靠前的方法。

9. 类型守卫必须真的检查其名称所声称的条件

类型守卫必须按名称所述实际验证值。没有真正验证的类型守卫会把故障藏在看似安全的名称后, 比 `as` 更糟。名称应采用 `isX` 或 `hasX`。

```

type Shape =
    | { kind: "circle"; radius: number }
    | { kind: "rect"; width: number; height: number };

// 实际检查了表示类别的字段

```

```
function isCircle(s: Shape): s is Shape & { kind: "circle" } {
  return s.kind === "circle";
}
```

因此，编写类型守卫时，必须真正检查其名称所声称的条件。

10. 用穷尽性检查让遗漏的 case 变成编译错误

在 switch 的 default 分支写入 `const _exhaustive: never = x;`。若新增一种 variant，却忘记编写对应的 case，编译就会报错（第 18 章）。

```
type Shape =
  | { kind: "circle"; radius: number }
  | { kind: "rect"; width: number; height: number };

function area(s: Shape): number {
  switch (s.kind) {
    case "circle":
      return Math.PI * s.radius ** 2;
    case "rect":
      return s.width * s.height;
    default: {
      // 若在 Shape 中增加 variant 却忘记添加 case，这里会出现编译错误
      const _exhaustive: never = s;
      return _exhaustive;
    }
  }
}
```

如果 switch 不返回值，就用 `void _exhaustive;` 代替 `return _exhaustive;`。

因此，按 variant 分支处理时，需要在 default 分支检查是否穷尽。

11. 用 satisfies 而非 as 验证是否符合类型

`satisfies` 可以检查值是否符合类型，同时不扩大字面量类型（例如只表示特定值的 "dark"）。

```
type Config = { theme: "dark" | "light"; cols: number };

// 错误示例：使用 as 会把 theme 的类型扩大为 "dark" | "light"，丢失它确为 "dark" 的信息
const configLoose = { theme: "dark", cols: 3 } as Config;

// 正确示例：satisfies 验证是否符合类型，同时保留字面量类型
```

```
const config = { theme: "dark", cols: 3 } satisfies Config;
// config.theme 的类型是 "dark", 不是 string
```

因此，如果只是要检查值是否符合类型，应使用 `satisfies` 而非 `as`。

12. 在数据入口验证一次，内部信任类型

在数据进入系统的位置，将其解析（按固定结构读取和转换）为领域类型。内部代码直接使用已带类型的值，不反复验证。这符合 Principle 「**Boundary Discipline**」（第 18 章）。

领域类型……表示应用处理的业务概念（如用户 ID、订单）的类型
例如，下面的代码用两种类型表示从外部收到的订单数据

```
// 领域类型：明确订单有哪些字段，以及每个字段的类型
type Order = { id: string; userId: string; total: number };

// 不是领域类型：字段不确定，每次使用都得检查
type RawOrder = Record<string, unknown>;
```

在边界验证时，数据入口的代码对 `RawOrder` 之类的值只验证一次，将其转为 `Order`；内部代码只处理 `Order`。

因此，外部数据应在入口验证一次，内部则信任类型。

13. 编写新类型前，先从现有类型推导

如果 OpenAPI 定义、GraphQL schema 或数据库迁移已经规定了数据结构，Agent 不应重新编写同样结构的类型，而要从据此生成的类型中推导。

```
// 生成的类型（实际应从生成的模块中 import）
type ChecksMessage = {
  totalCount: number;
  checks: { name: string; status: string }[];
  updatedAt: string;
};

// 错误示例：重复手写同样的结构，schema 改变时就会不一致
type CheckSummary = {
  totalCount: number;
```

```

    checks: { name: string; status: string }[];
};
function renderChecksLoose(s: CheckSummary) {
    /* ... */
}

// 正确示例：从生成的类型中取出所需字段
function renderChecks(s: Pick<ChecksMessage, "totalCount" | "checks">) {
    /* ... */
}

```

编写新 interface 前，先检查能否使用 Pick、Omit、Parameters、ReturnType、Awaited 或 typeof。

因此，若已有原始定义，就应从它推导类型，而非重写。

14. 用对象传递参数，而非按位置排列

用对象传递参数，每个值对应什么参数都能从名称看出，也能避免弄错参数顺序。

```

type Selection = {
    startLineNumber: number;
    startColumn: number;
    endLineNumber: number;
    endColumn: number;
};
declare const uri: string;
declare function openFileLoose(uri: string, selection: Selection): void;
declare function openFile(args: { uri: string; selection: Selection }): void;

// 错误示例：即使调换两个参数，只要类型相符就能通过编译
openFileLoose(uri, {
    startLineNumber: 10,
    startColumn: 1,
    endLineNumber: 10,
    endColumn: 1,
});

// 正确示例：不受顺序影响，名称能说明每个值的含义
openFile({
    uri,
    selection: {
        startLineNumber: 10,
        startColumn: 1,
        endLineNumber: 10,
        endColumn: 1,
    }
});

```

```
    },
  });
```

但逐帧渲染、tokenizer、parser 等反复执行的热点代码不适用这条规则。若每次调用都新建参数对象，就要不断分配内存；在高频调用处会影响性能。

- **逐帧渲染**……像动画或游戏那样，每秒重绘画面数十次的处理
- **tokenizer**……将字符串切分成单词或符号等最小有意义单位（token）的程序
- **parser**……读取 token 序列等，并组装成程序可处理结构的程序

概括来说，有多个参数时，除高频执行的部分外，应使用对象传参。

15. 本地能运行的组件就用真实组件测试，不用 mock

Agent 对本地能运行的组件，应运行真实组件来测试，而非用 mock（代替真实组件的假组件）。只有本地无法运行的组件（例如外部支付服务）才使用 mock。界面也应实际构建、运行后验证。

```
declare function test(name: string, fn: () => Promise<void>): void;
declare function expect<T>(actual: T): { toBe(expected: T): void };
declare function makeTempDir(): Promise<string>;
declare function saveNote(dir: string, text: string): Promise<void>;
declare function loadNote(dir: string): Promise<string>;
declare const fakeFs: { writeFile: { calledTimes: number } };

// 错误示例：连可在本机运行的文件保存都替换成 mock，只检查 mock 被调用的次数
test("メモを保存する", async () => {
  await saveNote("/notes", "買い物");
  expect(fakeFs.writeFile.calledTimes).toBe(1);
});

// 正确示例：在临时目录写入真实文件，重新读出并检查内容
test("保存したメモを読み戻せる", async () => {
  const dir = await makeTempDir();
  await saveNote(dir, "買い物");
  expect(await loadNote(dir)).toBe("買い物");
});
```

Oh My Stack 项目注（代码示例）：测试名称分别表示「保存笔记」和「能读回已保存的笔记」；输入及预期内容「買い物」意为「购物」。它们是测试使用的原始字符串，保留以维持示例断言。

16. 用结构化日志保留诊断信息

Agent 应使用结构化日志（包含固定字段的日志）保存足以通过 ID 调查故障的信息，不在发布代码中留下 `console.log`。

```
declare const logger: {
  error(event: string, fields: Record<string, unknown>): void;
};
declare const orderId: string;
declare const userId: string;
declare const err: Error;

// 错误示例：只有字符串，事后无法查出哪个订单因何失败
console.log("payment failed");

// 正确示例：用固定字段记录事件名称、供调查使用的 ID 和原因
logger.error("payment_failed", { orderId, userId, reason: err.message });
```

使用正例中的日志，调查故障的人或 Agent 可以按 `orderId` 搜索，追踪该订单发生了什么。

因此，诊断信息应以日后可以按 ID 追踪的日志形式保存。

请求写法：即使不提 Skill 名称，Agent 也会遵守规则

这项 Skill 没有请求示例。

随附指南 <docs/guide/05-build-and-clean.md> 也说明，使用者无需在任务中通过斜杠命令调用它。

总结

- **检查者**……由编译器代替编写代码的 Agent 查找类型错误。
- **使用时机**……Agent 读取或编辑 TypeScript 文件时自动加载。
- **规则**……让类型无法表示不可能的状态，并只在外部数据入口验证一次。

接下来的第 30 章介绍三项维护代码质量的 Skill 中的第二项：`/tdd`，它要求修复故障前先写一项在修复前会失败的测试。

第 30 章：TDD，用修复前失败的测试验证修复

kaito · 来源：日本語 · English · 2026-10-03

本章介绍 `/tdd`。

`/tdd` 指导 Agent 在修复故障前，先编写一项在故障仍存在时会失败的测试。例如，「重试时收到两条通知」这一故障，Agent 会编写验证「只收到一条通知」的测试。修复前测试失败，修复后通过。

`/tdd` 是从第 29 章开始介绍的三项维护代码质量的 Skill 中的第二项。借助 `/tdd`，替代编写代码的 Agent 核验修复成果的是一项修复前失败、修复后通过的回归测试。

本章从作用、使用时机、步骤和请求写法四方面介绍 `/tdd`。

本章结构

本章安排如下。

- 作用：修复故障前，用测试重现错误行为
- 使用时机：仅在使用者要求 TDD，或已有简便的测试途径时
- 步骤：修复前先确认测试「因正确的原因失败」
- 请求写法：只输入命令即可
- 总结

作用：修复故障前，用测试重现错误行为

`/tdd` 要求在修复故障前先写一项回归测试（检查已修复故障是否再次发生的测试），确认它在修复前失败、修复后通过。

虽然名称是 TDD（测试驱动开发），但其 SKILL.md 的标题是「TDD Bug Fix」：这项 Skill 只处理故障修复，并非指导从测试开始设计新功能。

/tdd 并不要求每次都先写测试。只有使用者要求，或 Agent 已知道简便可行的测试途径时，才先写测试。

使用时机：仅在使用者要求 TDD，或已有简便的测试途径时

/tdd 设置了 `disable-model-invocation: true`（第 25 章），因此 Agent 不会根据对话内容自行选择并运行它。

点名调用 /tdd 的是使用者和 /poteto-mode。README 说明，/poteto-mode 会在步骤需要时运行 /tdd。

SKILL.md 的 description 将使用 /tdd 限定于以下情况。

- 使用者要求实施 TDD、失败测试或回归测试
- 已知道可以用本地快速运行的测试核验故障

反过来说，以下情况 Agent 不会新增测试。

- 测试途径不明确
- 编写测试代价很高
- 必须组合许多组件才能编写测试
- 使用者没有要求

「已有简便的测试途径」例如故障代码已有单元测试，只需在其中增加一项测试。

若需要大量准备，就不新增测试，改用最接近的验证手段

前述「代价很高」与「必须组合许多组件」具体指编写测试需要以下事项。

- 在较大范围新建供测试运行的机制（harness）
- 实现稍有变化就会失效的脆弱 mock
- 运行耗时的 E2E 测试基础设施（按用户操作路径贯通整个应用的测试）
- 只有生产环境才有的状态（例如仅存在于生产数据库的数据）
- 不明确的重现步骤

- 与故障无关、规模较大的测试数据（fixture）改动

如果需要这些准备，Agent 不新增测试，转而使用最接近的验证方法，如脚本或手动重现命令。

这是因为**勉强建立这些准备而写出的测试，对故障是否修好只能提供较弱的证据。**

例如，依靠脆弱 mock 才通过的测试，验证的只是替身而非真实组件；与运行真实命令相比，它更难证明故障已修复（见随附指南 [docs/guide/10-recipes-and-pitfalls.md](https://docs.google.com/document/d/10-recipes-and-pitfalls)）。

「Bug fix」Playbook 只在有简便测试途径时参考 /tdd

/poteto-mode 也使用 /tdd。规定故障修复流程的「Bug fix」Playbook 在步骤 5 要求，把重现故障的提交（例如在修复前代码上会失败的测试）放在 Git 历史中修复提交之前；如果有简便测试途径，再参考 /tdd。

但这份 Playbook 也规定：测试代价高、需要组合许多组件，或测试写法不明确时，应跳过 /tdd。

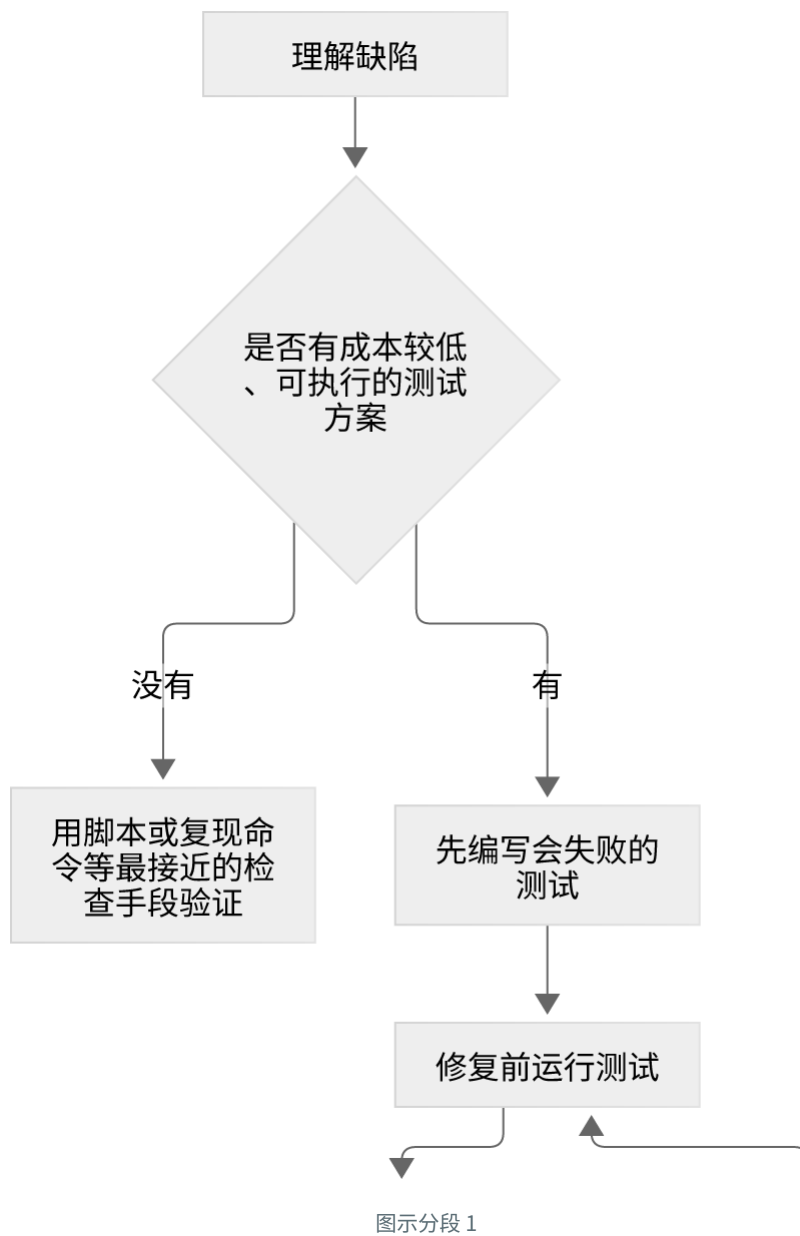
步骤：修复前先确认测试「因正确的原因失败」

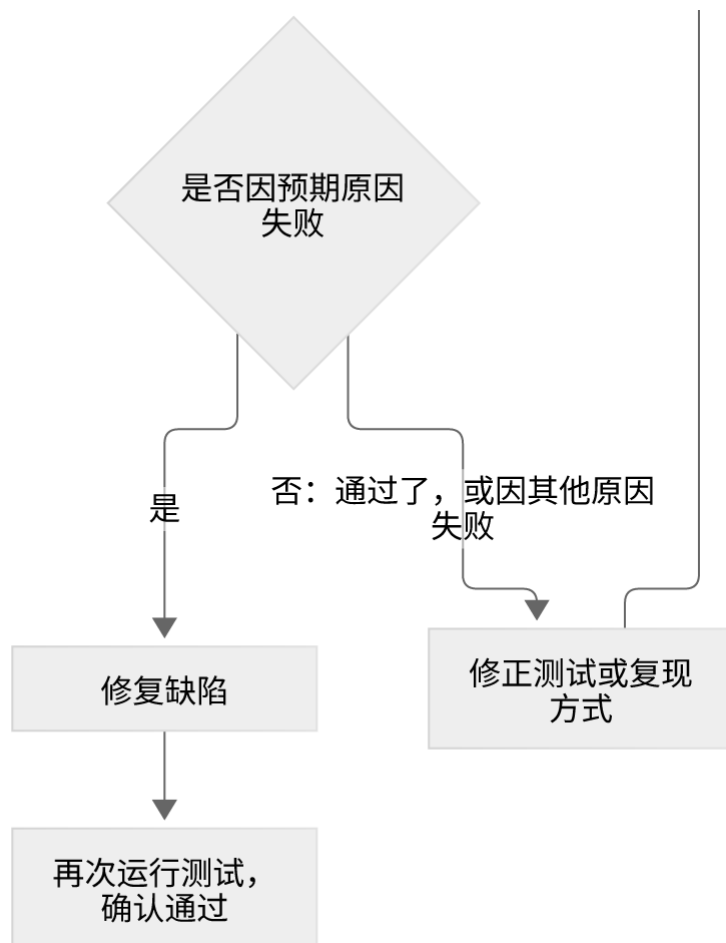
六个步骤：从修复前失败核验到修复后通过

/tdd 有六个步骤。

1. **理解故障**.....Agent 确定预期行为、当前行为，以及触发故障的最小重现方式（最短操作或输入）。
2. **选择范围最窄的测试**.....Agent 在故障代码已有的测试中，选择能以最窄范围核验故障的测试。若找不到容易使用的测试，不会只为了完成这一步而从头建立新的测试。
3. **先写失败测试**.....Agent 新增一项能发现此故障的最小测试，期望值写真实应有的行为。例如，故障会发送两条通知，也不能把期望值设为 2；应设为 1。
4. **修复前运行测试**.....Agent 确认测试因预期原因，即故障本身而失败。
5. **修复故障**.....Agent 在应用本体代码而非测试代码中，作出满足预期行为的最小修改，同时遵守周边代码依赖的约定（例如函数参数和返回值结构）。
6. **重跑回归测试**.....Agent 确认测试通过。

找不到简便测试，及修复前测试并非因预期原因失败时，流程的分岔如下图。





图示分段 2

[查看图示 1](#)

在修复前运行测试，辨别失败是否确由故障造成

步骤 4 在修复前运行测试，是为了区分失败来自故障本身，还是其他原因。

若测试因函数名写错等其他原因失败，无论故障是否存在都会失败；这就不能证明测试能发现故障。

如果测试在修复前就通过，也说明它没检测到故障；Agent 应修改测试。

示例：比较通知重复故障中正确原因与其他原因导致的失败

以修复「重试时收到两条通知」为例，看看步骤 4 中修复前的运行。

测试要验证的预期行为是「即使重试，也只收到一条通知」。

```

declare function test(name: string, fn: () => void): void;
declare function expect<T>(actual: T): { toBe(expected: T): void };
// 待验证函数：发送通知，失败时重试，返回已发送通知列表
declare function sendWithRetry(message: string): string[];

// 因正确原因失败的测试：修复前会因「预期 1，实际 2」而失败
test("再試行しても、通知は1通だけ届く", () => {
  const sent = sendWithRetry("注文を受け付けました");
  expect(sent.length).toBe(1);
});

// 因其他原因失败的测试：函数名拼错，与缺陷无关也会失败
test("再試行しても、通知は1通だけ届く", () => {
  const sent = sendAndRetry("注文を受け付けました");
  expect(sent.length).toBe(1);
});

```

Oh My Stack 项目注（代码示例）：测试名称表示「即使重试，也只送达一条通知」；通知内容表示「已接受订单」。原字符串是测试调用和断言的值，故保留。

因此，如果修复前的测试因与故障无关的原因而失败，Agent 必须先修正测试，再修改实现。

无法编写测试时，改用可执行的验证方法

若编写测试不切实际，Agent 会改用可执行的验证方法，例如脚本、手动重现命令、浏览器自动化，或检查日志中是否出现预期行。

即使不写测试，Agent 也不会省略验证；否则就没有证据表明故障已修复。

最终报告不仅写结果，也写修复前后的证据

Agent 在最终报告中说明，修复前哪项测试（或其他验证）如何失败，以及修复后哪次运行通过。若无法展示修复前的失败，就说明原因及替代验证方法。

Agent 报告结果，也报告证据。只说「修好了」，读者无法核查是否真的修好。

与其新增糟糕的测试，不如不新增测试

/tdd 主张**与其新增糟糕的测试，不如不新增。**

糟糕的测试包括只检查 mock 的测试，或把当前实现细节固定下来的测试（第 19 章的 Principle「**Test Behavior, Not Implementation**」）。前者没有验证真实行为；后者即使代码改写未改变行为，也会失效。

仍以重复通知故障为例，下文对比两种糟糕测试与一项良好测试。

```
declare function test(name: string, fn: () => void): void;
declare function expect<T>(actual: T): { toBe(expected: T): void };
type Transport = { send(message: string): void };
// 待验证函数：发送通知，失败时重试，返回已发送通知列表
declare function sendWithRetry(message: string, transport?: Transport): string[];
// 发送组件的替身 (mock)，统计 send 被调用的次数
declare const mockTransport: Transport & { calls: number };
// 函数内部使用的重试计数变量
declare const retryState: { attempts: number };

// 错误测试 1（只验证 mock）：只检查替身的 send 被调用的次数，
// 没有检查实际送达的通知数量
test("送信の部品を1回呼ぶ", () => {
  sendWithRetry("注文を受け付けました", mockTransport);
  expect(mockTransport.calls).toBe(1);
});

// 错误测试 2（绑定实现细节）：依赖内部变量的值，
// 即使重构不改变送达通知的数量（如重命名变量），测试仍会失败
test("再試行の回数が1回になる", () => {
```

```

    sendWithRetry("注文を受け付けました");
    expect(retryState.attempts).toBe(1);
  });

  // 正确测试：检查外部可见的行为（送达的通知数量）
  test("再試行しても、通知は1通だけ届く", () => {
    const sent = sendWithRetry("注文を受け付けました");
    expect(sent.length).toBe(1);
  });

```

Oh My Stack 项目注（代码示例）：测试名称分别表示「调用发送组件一次」「重试次数为一次」「即使重试，也只送达一条通知」；通知内容表示「已接受订单」。这些原字符串是测试输入，故保留。

Agent 使测试符合预期行为，不会为了迁就错误实现而修改测试。此外，除非预期行为本身确实改变且理由明确，也不会减弱现有断言（例如 `expect(sent.length).toBe(1)`，用于判断结果是否符合预期）。

```

declare const sent: string[];
declare function expect<T>(actual: T): {
  toBe(expected: T): void;
  toBeGreaterThanOrEqual(expected: number): void;
};

// 修改前：检查预期行为（只送达一条通知）
expect(sent.length).toBe(1);

// 修改后（不该这样改）：弱化断言，让错误地送达两条通知的实现也能通过
expect(sent.length).toBeGreaterThanOrEqual(1);

```

请求写法：只输入命令即可

若能为待修故障编写简便测试，且对话中已经说明该故障，使用者只需输入命令（[docs/guide/05-build-and-clean.md](#)）。

```

/tdd implement
// 使用 /tdd 实现。

```


若希望只有在具备简便测试途径时才使用 `/tdd`，可按 `docs/guide/10-recipes-and-pitfalls.md` 中的示例附上条件。

```
/poteto-mode repro the duplicate write first. if there's a cheap test path, /tdd it. then fix and rerun.
```

```
// 先复现重复写入。如果容易编写测试，就使用 /tdd；然后修复并重新运行。
```

由于附有「if there's a cheap test path」（如果有简便测试途径）的条件，若没有此途径，Agent 可以跳过 `/tdd`，采用最接近的验证方法。

总结

- **核验者**……修复前失败、修复后通过的回归测试，用来验证修复。
- **使用时机**……只有使用者要求，或已知有简便测试途径时，才在修复前编写失败测试。
- **步骤**……修复前运行测试，确认失败确由故障造成，然后修复。

接下来的第 31 章介绍三项维护代码质量的 Skill 中的第三项：`/no-comments`，由并非编写代码的 Agent 审查差异中的注释。

第 31 章：移除注释，整理代码库

kaito • 来源：日本語 • English • 2026-10-03

本章介绍 `/no-comments`。

`/no-comments` 是一种 Skill：在审查前，Agent 会让没有写过差异中那些注释的另一位 Agent（**Comment Sicko**）审查它们。Comment Sicko 会删除解释项目自身代码意外行为的注释（例如保存函数还会发送确认邮件），并报告相应函数需要修改。原 Agent 则通过重命名或拆分函数来修改代码，让行为即使没有注释也能看明白。对于解释无法由自己改变的外部库等所强加行为的注释，则保留下来。

从第 29 章开始介绍的三种维护代码质量的 Skill 中，`/no-comments` 是第三种。替写代码的 Agent 检查注释的，是没有写过那些注释的 Comment Sicko。

`/no-comments` 的审查对象，是差异中的注释，以及关闭 lint 或类型检查的指令（例如 `eslint-disable`、`@ts-ignore`）。

- **抑制指令**……写在代码中的指令，用于只在该处关闭 lint 或类型检查发出的警告
- **Comment Sicko**……pstack 随附的子 Agent，负责审查并删除注释（第 8 章）
 - `/no-comments` 启动 Comment Sicko，收到它的报告后，由 `/no-comments` 修改代码

本章从作用、使用时机、规则和步骤、请求写法四个角度讲解 `/no-comments`。

本章结构

本章按以下顺序展开。

- 作用：让没有写过注释的 Agent（Comment Sicko）审查它们
- 使用时机：审查前，针对差异调用
- 规则与步骤：审查 Comment Sicko 的意见

- 请求写法：指定差异为对象
- 小结

作用：让没有写过注释的 Agent（Comment Sicko）审查它们

`/no-comments` 是在代码审查前移除注释的 Skill。

它会启动 Comment Sicko（第 8 章），并修复它所提出、经自己接受的问题。它还会针对注释声称的约束（例如「不得修改措辞」），提出用测试等代码表达该约束的方案。

这项 Skill 把解释自身代码意外行为的注释，视为代码需要修改的标志。通过重命名或拆出函数，可以把这种行为改写成只读代码也能理解、无需文字说明的形式。

※ 关于「解释意外行为的注释」，后文「保存函数还发送确认邮件：用函数名替代注释」有进一步说明。

随附指南 [docs/guide/05-build-and-clean.md](#) 如下解释为何要把注释审查交给另一位 Agent。

Comments need their own pass, and not from the agent that wrote them. An author defends its comments the way you'd defend yours.

注释需要单独审查，而且不应由写注释的 Agent 来做。作者会像你维护自己的注释一样，维护它写下的注释。

如果让 Agent 审查自己写的注释，它可能会维护自己的注释，把不必要的注释也留下。因此，注释审查由没有写过注释的 Comment Sicko 执行。

使用时机：审查前，针对差异调用

`/no-comments` 设置了 `disable-model-invocation: true`（第 25 章），因此 Agent 不会根据对话内容自行选用并运行它。启动该 Skill 的，是点名调用它的用户，以及 `/poteto-mode` 的规则或 Playbook 步骤。

`/poteto-mode` 的「Non-negotiables」（第 8 章）规则，以及创建 PR 时使用的「Opening a PR」等 Playbook，规定要在审查之前调用 `/no-comments`。

`/no-comments` 查看调用者传入的文件或差异。若未传入范围，就查看与基准分支（默认为 `main`）之间的差异，并包括尚未提交的变更。

`/no-comments` 与另外两种进行类似清理的 Skill 按对象分工如下。

- **代码中的冗余内容**……`/deslop` 移除仅仅复述代码行为的注释、缺乏依据的防御性检查、已经不用的兼容性路径等。`/deslop` 随附于 `cursor-team-kit`，不包含在 `pstack` 中。
- **文章的写作毛病**……`/unslop`（第 32 章）从 README、PR 描述等文字中移除 AI 式写作毛病。
- **注释**……`/no-comments` 让没有写过注释的 Agent 来审查它们。

创建 PR 的 Playbook 也会调用 `/no-comments`

调用 `/no-comments` 的 Playbook 都用它在 PR 交付审查或验证前，从差异中清理注释。以下分别说明各调用方的时机和理由。

「Opening a PR」：审查前清理显得尚未完成的差异

「Opening a PR」是创建 PR 的 Playbook，在其他所有 Playbook 末尾调用。它规定审查前运行 `/no-comments`。负责创建 PR 的子 Agent 也会先运行 `/no-comments`，再发布 PR URL。

随附指南 `05-build-and-clean.md` 解释了为何不能跳过差异清理：**对审查者来说，留有只是解释代码行为的注释或多余防御代码的差异，会显得尚未完成**。同一页面还指出，**残留的冗余代码会成为下一处缺陷藏身的地方**。

「Autopilot-full」「Autopilot-stack」：确定代码最终形态后再交给验证

在「**Autopilot-full**」和「**Autopilot-stack**」（第 16 章）中，负责每个 PR 的 Agent（从头到尾负责一份 PR 变更的云端 Agent）先完成 `/deslop` 和 `/no-comments`，待拟合并的代码成为最终形态后，才报告该提交的 SHA（每次提交所获、可唯一标识它的编码）。然后，该负责人自行开展的验证和 CI，与 `/swarm`（第 24 章）执行的独立验证并行推进。

「Multi-phase or multi-PR plan」：放进计划书检查清单

「Multi-phase or multi-PR plan」（第 16 章）生成的计划书，是每个 PR 的负责人逐项执行、操作人（指示开始执行计划并查看证据的人）依据证据核查的检查清单。计划书模板中，每个 PR 的共通项目包含「每次提交前运行 `/deslop`，审查前运行 `/no-comments`」。

规则与步骤：审查 Comment Sicko 的意见

Comment Sicko 审查并删除注释，不改写应用代码

Comment Sicko (`agents/comment-sicko.md`) 只删除注释、报告需要修改的代码，不改写应用代码。

Comment Sicko 的定义带有「疯狂憎恨注释」的夸张口吻。但可以保留注释的规则很明确。

Comment Sicko 只保留以下五类注释，犹豫时就删除。

- 法律或许可证声明头
- 解释外部依赖或平台强加、且自己无法改变的非显而易见行为
- `// prettier-ignore`（指示代码格式化工具 Prettier 不要格式化紧随其后的代码），以及对规则本身错误、过细或只涉及风格的 lint 规则所做的抑制
- 定义公开 API 契约的文档注释
- 指向 issue 或 RFC（提出设计方案的文档）的链接，用来说明代码无法表达的约束

删除意外行为的说明或关键 lint 规则的抑制，并列入 MUST KILL

Comment Sicko 会删除解释自身代码意外行为的注释，并在报告中将拥有该行为的符号（函数、变量、类型等名称）列为 MUST KILL。这一标记要求通过改名、拆分、补充类型或重新设计，使行为无需文字说明就能看明白。

如果代码用抑制指令（`eslint-disable`、`@ts-ignore`、`@ts-expect-error` 等）关闭用于查找真实缺陷或保护正确性与安全性的 lint 规则，Comment Sicko 也会删除抑制指令，并把引发 lint 警告的符号列为 MUST KILL。

保存函数还发送确认邮件：用函数名替代注释

来看一个解释自身代码意外行为的注释例子。

假设一个保存函数实际上还会发送确认邮件。

```
type User = { name: string; email: string };

// 修改前：用注释说明自家函数的意外行为
// 注意：此函数除了保存，还会发送确认邮件
function saveUser(user: User) { /* ... */ }
```

```
type User = { name: string; email: string };

// 修改后：将两项工作拆成两个名称清楚的函数
function saveUser(user: User) { /* ... */ }
function sendConfirmationMail(user: User) { /* ... */ }
```

前一个例子的注释不是在解释外部依赖强加的行为，而是在解释自身代码的意外行为。因此，Comment Sicko 会删除该注释，并把 saveUser 列为 MUST KILL。

Comment Sicko 本身不修改代码。/no-comments 可以像后一例那样将工作拆为两个函数，使人不看注释，也能从名称看出每个函数做什么。

因此，如果注释在解释自身代码的意外行为，就应删除注释，并修改代码，让名称表达相应行为。

/no-comments 会审查 Comment Sicko 的报告，不会照单全收

/no-comments 不会直接接受 Comment Sicko 的报告及其修改，而是分三个阶段审查。

1. 驳回有误的报告，只允许重做一次
2. 对没有说明依据的警告性注释，先用 /how 或 /why 核实，再接受相应判断
3. 如需决定新的代码结构才能修复，先用 /architect 确定结构再修改

1. 驳回有误的报告，只允许重做一次

Comment Sicko 的报告或修改可能包含以下错误。

- 对传入范围之外的内容提出意见……指出的代码不在 /no-comments 要求审查的文件或差异中

- **改写应用代码**……Comment Sicko 只许修改注释，却改写了应用代码
- **删除应保留的注释**……删除了「Comment Sicko 审查并删除注释，不改写应用代码」列出的五种可保留注释之一
- **MUST KILL 的理由错误**……报告中将某个符号列为 MUST KILL 的理由与实际代码不符，例如函数未发送确认邮件，却被指称为「还会发送确认邮件」
- **错误地指出有意保留的代码**……把有意编写且已决定保留的代码列为需要修改的内容

发现此类有误的报告时，/no-comments 会撤销 Comment Sicko 的修改，说明错误所在，只让它再审查一次。如果第二次报告仍有误，/no-comments 会将问题列为未解决，并以执行失败结束。

2. 对缺少依据的警告性注释，先用 /how 或 /why 核实再接受判断

代码中可能有「IMPORTANT」或「do not remove（不得删除）」等没有说明理由的简短警告性注释。

如果 Comment Sicko 决定「删除」或「保留」这类注释，/no-comments 在接受判断前，会针对注释所附的符号运行 /how 或 /why（两者见第 22 章）。它用 /how 了解代码目前如何运作，用 /why 了解代码为何采用当前形式，以确认警告中的说法如今是否仍正确。

例如以下注释。

```
declare function sleep(ms: number): Promise<void>;
declare function saveDraft(): Promise<void>;

async function onClose(): Promise<void> {
  // IMPORTANT: do not remove
  await sleep(100);
  await saveDraft();
}
```

如果通过 /why 发现这段等待时间是为规避一个已经修好的旧库缺陷，那么警告所说的情况已不成立。此时，/no-comments 会接受 Comment Sicko「删除」的判断。

如果调查后仍无法作出判断，/no-comments 会删除注释。

3. 如需新代码结构，先用 `/architect` 确定结构再修改

对于接受的意见，若只是删除未使用的处理、移除不必要的参数、改用本应使用的 API 等小修正，`/no-comments` 会直接修改。

如果修正需要决定怎样拆分函数或设计类型等新代码结构，`/no-comments` 会将接受的意见汇总后只交给 `/architect` 一次（第 24 章），由它确定结构。`/architect` 在确定结构后停止，实际修改代码的是 `/no-comments`。

从根本原因修复已接受的意见

`/no-comments` 会从根本原因修复审查后接受的问题（见第 19 章的 Principle「**Fix Root Causes**」）。

例如，Comment Sicko 删除了解释规避方法（不修复缺陷原因、只避开症状的代码）理由的注释，`/no-comments` 就会一并删除规避代码，转而修复需要规避方法的根本原因。

不过，`/no-comments` 不修改传入范围之外的代码。如果根本原因位于范围之外，它只在范围内做能做的最小修正，并将余下问题列为未解决。

声称存在约束的注释：提出用代码强制约束的方法，等待批准

声称存在约束的注释，是指写着「do not remove（不得删除）」「do not change wording（不得修改措辞）」「talk to X before changing（修改前先咨询 X）」之类的话。

对于这类注释，`/no-comments` 会提出一种用代码而非注释来强制约束的方法，并等待用户批准。候选方式有以下四种；`/no-comments` 会选择传入范围内最省力的一种。

- **类型**……让违反约束的代码产生编译错误
- **运行时检查**……违反约束时，在运行时抛出错误
- **测试**……违反约束时让测试失败
- **lint**……让 lint 对违反约束的代码发出警告

例如，遇到「不得修改措辞」的注释，`/no-comments` 会提出编写一个措辞变化时会失败的测试。有了这个测试，即使没有注释，也能维持约束。

```
// 修改前：只用注释声明约束
// 不要更改显示文字
function stockLabel(count: number): string {
```



```
return count === 0 ? "在庫切れです" : `残り${count}点`;
}
```

Oh My Stack 项目注（代码示例）：函数输出中的日文分别表示「缺货」和「剩余若干件」；原值是被要求保持不变的显示文字。

```
// 添加一个测试
declare function test(name: string, fn: () => void): void;
// 检查值
declare function expect(value: unknown): { toBe(expected: unknown): void };

// 修改后：用测试表达约束，并删除注释
function stockLabel(count: number): string {
  return count === 0 ? "在庫切れです" : `残り${count}点`;
}

test("在庫が0のときの文言は「在庫切れです」から変わらない", () => {
  expect(stockLabel(0)).toBe("在庫切れです");
});
```

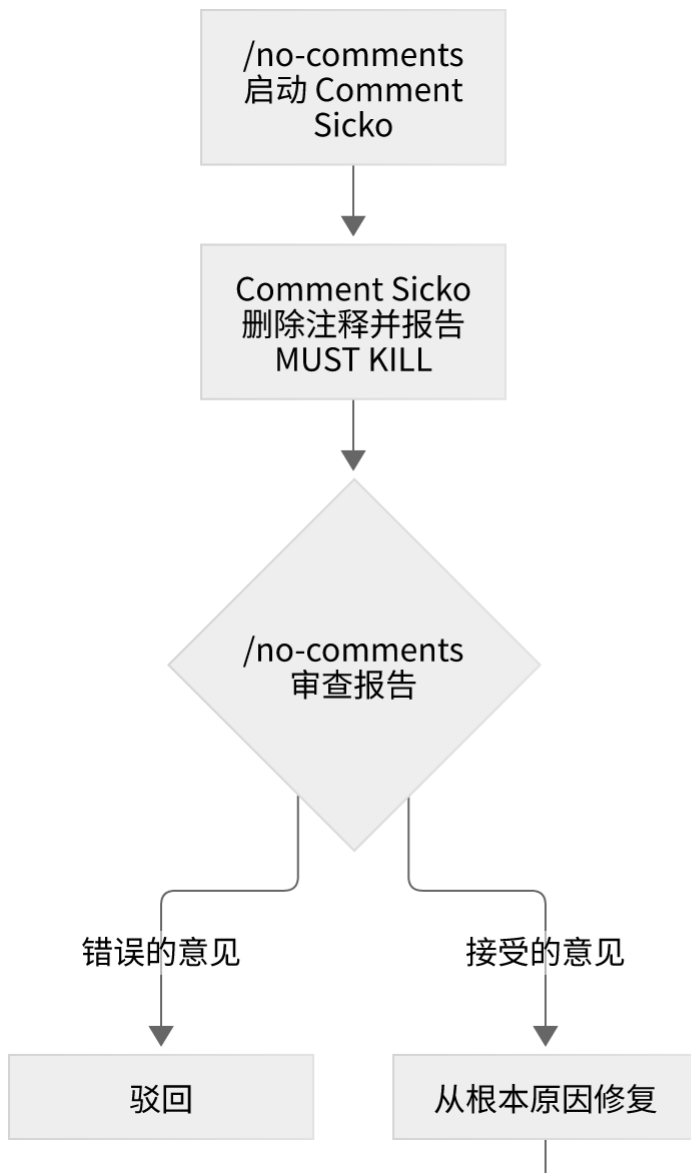
Oh My Stack 项目注（代码示例）：函数输出中的日文分别表示「缺货」和「剩余若干件」；测试名称要求库存为零时的「缺货」文字不变，断言也直接比较该原值。

这样，如果注释声称存在约束，就需要用测试等代码表达该约束。

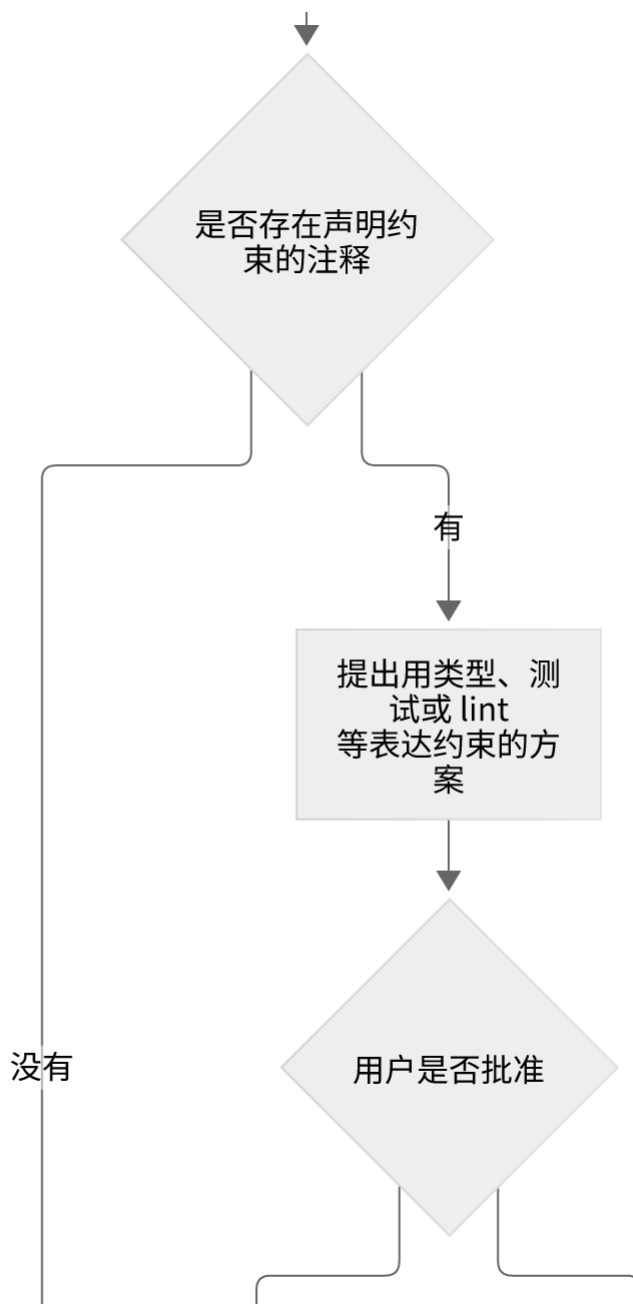
如果用户批准，`/no-comments` 会先用代码表达约束，再删除注释；如果不批准，仍会删除注释，并将约束列为未解决问题报告。

删除注释的是 Comment Sicko，决定修什么的是 `/no-comments`

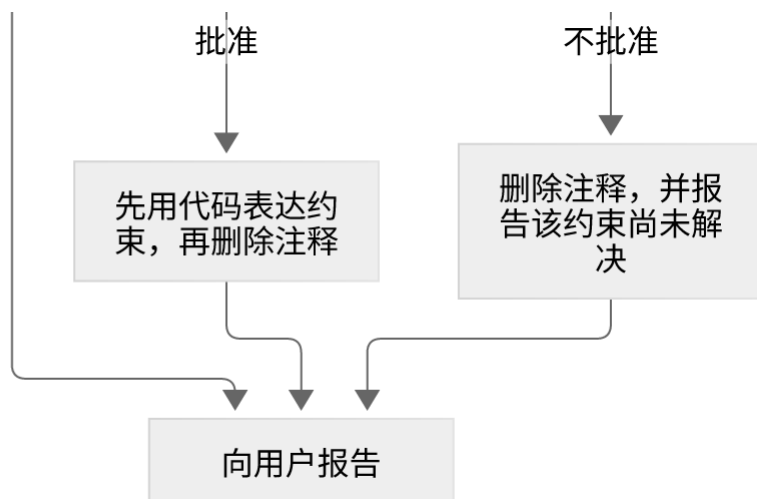
将上述流程画成图，如下所示。



图示分段 1



图示分段 2



图示分段 3

查看图示 1

如图所示，Comment Sicko 负责删除注释；决定接受哪些意见并修改代码，以及决定用代码表达约束还是将它列为未解决问题的，是 `/no-comments`。

请求写法：指定差异为对象

随附指南 `05-build-and-clean.md` 给出了以下请求，用于在审查前将差异中的注释交给没有写过它们的 Agent。

```
/no-comments the diff  
// 针对差异
```

小结

- **负责检查的角色**……没有写过注释的 Comment Sicko 审查差异中的注释。
- **使用时机**……在审查前，针对差异调用。
- **修改方式**……删除解释意外行为的注释，并修改代码，让行为即使没有注释也能看明白。

下一章第 32 章将介绍维护文章质量的三种 Skill：`/unstop`、`/bro` 和 `/technical-writing`。

第 32 章：保持写作质量

kaito • 来源：日本語 • English • 2026-10-03

本章介绍以下三种 Skill。

1. `/unslOp`
2. `/bro`
3. `/technical-writing`

三者都旨在维护 Agent 所写文章的质量，分别负责以下工作。

- `/unslOp`……让 Agent 从自己写的文章中，清除「I hope this helps!（希望这能帮到你）」等套话和其他 AI 式写作毛病。
- `/bro`……让 Agent 不使用术语，简短地重新表述刚才的回复。
- `/technical-writing`……让 Agent 先确定文档的类型（如教程或参考文档），再撰写技术文档。

本章先比较三种 Skill 的对象、由谁修改、因何启动，再分别从作用、使用时机、规则或步骤、请求写法四个角度介绍它们。

本章结构

本章按以下顺序展开。

- 三种 Skill 的对象和修改者不同
- `/unslOp` 用带编号的规则清除 AI 式写作毛病
- `/bro` 用没有术语的话重述难懂的回复
- `/technical-writing` 先确定文档类型，再开始写作
- 小结

三种 Skill 的对象和修改者不同

三种 Skill 的对象及修改者如下表所示。

Skill	对象	由谁修改
/unslop	包括 Agent 回复在内的任何文章	撰写文章的 Agent 自己按规则修改
/bro	Agent 刚才的回复	用户觉得回复难懂时调用，由 Agent 重述
/technical-writing	文档、RFC（提出设计方案的文档）、README、PR 描述、提交说明	撰写文章的 Agent 自己按四层规范写作和修改

三者都只在被点名调用时运行

这三个 SKILL.md 都设置了 `disable-model-invocation: true`（第 25 章）。对于设置了此选项的 Skill，Agent 不会根据对话内容自行选用。因此，只有用户点名调用，或 `/poteto-mode`、Playbook、其他 Skill 在步骤中点名调用时，它们才会运行。

`/poteto-mode` 的「Non-negotiables」（第 8 章）规定了其中两种 Skill 的调用场景：写任何文章时使用 `/unslop`；写文档、RFC（提出设计方案的文档）、README、PR 描述或提交说明时使用 `/technical-writing`。

所以，在 `/poteto-mode` 下工作，即使用户不点名这两种 Skill，Agent 也会依照「Non-negotiables」规则调用它们。

/unslop 用带编号的规则清除 AI 式写作毛病

作用：依照规则清除 AI 式写作毛病

`/unslop` 是让 Agent 依照规则，从自己撰写的文章中删去 AI 式写作毛病的 Skill。

所谓 AI 式写作毛病，是 AI 文章中常见、让读者察觉到「这是机器写的」的用语或格式。

英文原文称之为 AI tells 或 AI patterns。例如回复结尾的套话，或过度使用 em dash（英文中的长破折号 —）。

规则和例子都针对英语文章，但大多数规则与语言无关，普遍适用于文章，因此也并非不能用于日语。

当然，它在英语中最有效。因此，仍然可以说：「The hottest programming language is English」。

使用时机：始终应用于任何文章

/poteto-mode 的「Non-negotiables」规定，**所有文章，包括 Agent 的回复**，都要应用这项 Skill。

SKILL.md 的说明（description）也写着「Must always apply（必须始终应用）」。不过，/unslop 设置了 disable-model-invocation: true，Agent 不会因为读到这段说明就自行调用它。之所以会始终应用，是因为「Non-negotiables」规则，以及下文介绍的其他 Skill 和 Playbook 步骤，会调用 /unslop。

其他撰写文章的 Skill 和 Playbook 也使用 /unslop

其他 Skill 和 Playbook 也在各自步骤中规定，要将 /unslop 应用于自己写的文章。调用方及应用的文章汇总如下。

调用方	应用 /unslop 的文章
/technical-writing	撰写或审查的所有文档
「Opening a PR」 「Multi-phase or multi-PR plan」 Playbook	PR 标题、PR 描述、提交正文、计划书
/teach、/show-me-your-work	解释性的回复及判断记录
/recall、/blast-radius	交给用户的报告
/automate-me	用户专用 Skill 的草稿
「Investigation」 Playbook	调查结果的回复

各调用方的细节如下。

/technical-writing: 把 AI 式写作毛病清单交给 /unslop

/technical-writing 自己不维护 AI 式用词或空洞套话等毛病清单，而是交给 /unslop。因此，它会将 /unslop 应用于自己撰写或审查的每份文档。

「Opening a PR」 「Multi-phase or multi-PR plan」 Playbook: 经 /technical-writing 撰写后，再应用 /unslop

创建 PR 的「Opening a PR」 Playbook（第 14 章）规定，先用 /technical-writing 撰写 PR 标题、PR 描述及提交正文，再应用 /unslop。

撰写跨多个阶段或 PR 的计划书的「Multi-phase or multi-PR plan」 Playbook（第 16 章）也规定，先用 /technical-writing 写计划书，再应用 /unslop。

/teach、/show-me-your-work: 像向同事说话一样写

/teach（第 22 章）用简单的话解释工作，让人真正理解内容。它规定所有回复都应像向同事解释一样，使用浅白的口语，并将 /unslop 应用于回复。

/show-me-your-work（第 28 章）把 Agent 的判断及依据逐行记录在文件中。它规定记录中的每一行都要像向队友讲述自己做过的像一样写，不用 AI 式说法或抽象术语；随后还要对记录文字应用 /unslop。

/recall、/blast-radius: 撰写交给用户的报告

/recall（第 22 章）从聊天历史等资料中重建用户近期工作的状态。/blast-radius（第 27 章）在发布前查找变更可能导致其他地方出现的问题。

两者都通过 /unslop 撰写交给用户的报告。/recall 的报告说明工作目前的进度和下一步；/blast-radius 的报告说明变更可能破坏什么，以及合并前要检查什么。

/automate-me: 用 /unslop 修改用户专用 Skill 的文字

/automate-me（第 25 章）从用户近期的 transcript（与 Agent 的对话记录）中找出工作偏好，起草用户专用的 Skill（<利用者の名前>-mode）。它将 /unslop 应用于草稿的每一行。

「Investigation」 Playbook: 调查结果的回复就是成果物

「Investigation」（第 10 章）是不修改代码、只调查并回答问题的 Playbook。它返回带出处的解释或建议，而非代码变更，并把这份回复定义为调查的成果物。

这项 Playbook 在最后一步对该回复应用 `/unslop`。

规则：用七个类别和固定编号列出 AI 式写作毛病

SKILL.md 的步骤只有两项，其余大部分篇幅是规则。

1. 找出符合规则的地方
2. 保留原意，并按照作者希望呈现的语气改写

每条规则都有编号，其他 Skill 用这些固定 ID 引用它们。例如，`/poteto-mode` 用「unslop rule 14」这样的编号，引用「不要用冒号连接句子中途的成分」这一规则。

SKILL.md 规定，删除一条规则后应空出该编号。在 pstack 0.15.5 版本的 SKILL.md 中，编号 1、2、4、6、21 是空缺的。

这些空缺可在 GitHub 的[源码视图](#)（未经 Markdown 格式化的显示方式）中确认。

共有 28 条规则，分为以下七类。

类别	修改内容	规则编号
Content（内容）	只添加评价的短语，以及出处不明的主张	3、5
Language（词汇）	AI 常用的词语和固定句式	7~12
Style（格式）	符号、粗体、标题及表情符号的用法	13~19
Communication artifacts（聊天痕迹）	聊天回复中的套话和奉承语气	20、22
Filler（赘词）	无意义的词语、过度保留和空洞结尾	23~25
Jargon（貌似专业的术语）	听起来专业的比喻性名词	26
Plain speech（浅白用语）	只写感受或印象的句子、需要重读才能理解的句子、刻意雕琢的词语	27~33

下面按类别列出全部规则。

如前所述，原文规则和例子都以英语文章为对象。但本书认为，除了英语格式特有的四条规则（13、14、17、19），其他规则也适用于日语文章。因此，对于同样适用于日语的规则，本书改用日语示例说明；四条英语特有规则则保留原文的英语例子。

Content（内容）

编号	规则	修改方式
3	空泛短语	对于「实现了……」「使……成为可能」等只增加评价的短语，删除它们，或补上实际出处和实质内容
5	模糊的出处	不要写「据专家称」「一般认为」等无法知道由谁提出的说法；点名来源，或删除

Language（词汇）

编号	规则	修改方式
7	AI 式词汇	不要反复使用「重要的」「全面的」「此外」「利用……」等词，改用简单用语
8	把「是」说得过于花哨	将「发挥……作用」「以……为傲」改成「是」「有」
9	「不仅……还……」	避免「不仅……还……」的句式，直接写要点
10	硬凑三项并列	不要强行把想法塞进三项组合，实际有几项就写几项
11	反复换名指同一事物	不要在同一段里轮流称「主人公」「主角」「中心人物」，统一使用一个词
12	虚假的范围	不要用「从文档到测试」这样的「从……到……」连接不在同一尺度上的事物；直接列出话题

Style (格式)

编号	规则	修改方式
13	过度使用 em dash	不用 em dash，改用句号或逗号；也不用括号、en dash 或连字符代替
14	过度使用冒号	只在列表或例子前使用冒号，不要用它连接句子中途的成分
15	过度使用粗体	不要每次出现专有名称或缩写都加粗
16	带小标题的列表项	把「性能：性能改善了」这样粗体小标题重复本行内容的列表项，改为普通句子
17	Title Case 标题	标题改用 sentence case
18	装饰性的表情符号	从标题和列表项中删除表情符号
19	弯引号	将弯引号改为直引号

- **em dash**……英文中用于插入语或句子间隔的长破折号 (—)
- **en dash**……比 em dash 短的破折号，英文中可表示「1–5」这样的范围
- **Title Case**……标题中主要单词的首字母都大写，例如「Pick The Mode First」
- **sentence case**……标题同一般句子一样，只将第一个单词的首字母大写，例如「Pick the mode first」
- **弯引号**……左右形状不同的引号 (“ ”)；直引号左右形状相同 (")

Communication artifacts (聊天痕迹)

编号	规则	修改方式
20	聊天机器人的套话	删除「希望能帮到你」「如果还有问题，请告诉我」「明白了！」等句子

编号	规则	修改方式
22	奉承的语气	省去「问得真好！你说得完全正确！」等前言，直接回答

Filler（赘词）

编号	规则	修改方式
23	赘词和空洞短语	将「由于存在……这一事实」改为「因为……」；删除「需要注意的是，这一点很重要」
24	过度保留	将「也不能排除可能存在……的可能性」改为「可能……」
25	没有内容的结尾	不要用「未来值得期待」之类的句子收尾；写出具体的计划或事实

Jargon（貌似专业的术语）

编号	规则	修改方式
26	抽象的比喻性名词	将「基石」「北极星」「飞轮」等比喻改成具体用语，例如将「北极星指标」改成「最重视的指标」

Plain speech（浅白用语）

编号	规则	修改方式
27	写出做了什么，而非只写感受或印象	不要只说「易读的 SQL」「符合 schema 的类型」等使用时的感受；改写成说明机制或数值的句子，例如「 <code>.toSQL()</code> 返回发送给数据库的原始字符串」「列名改变后，构建会失败」
28	缩短或拆开过密的句子	读者必须重读才能理解的句子，拆成两句，或删除部分内容

编号	规则	修改方式
29	使用主动语态	将「查询会被验证」改为「编译器会验证查询」
30	删去副词，或使用更有力的动词	将「运行得很快」改为「速度快」或写出实测数值；不要只说「大幅改善」，应写测得的差异
31	选择浅白的词语	将「活用」「利用」改成「使用」，将「促进」改成「帮助」
32	放弃刻意雕琢的文风	把比喻或华丽说法改成字面描述，例如将「值得转动的旋钮」改成「值得试着调整的参数」
33	避免把太多内容塞进一句话	将省略动词、依赖箭头和缩写的句子改成完整句，例如将「解析器拒绝无效日期 → exit 2，无写入」改为「解析器拒绝无效日期，以退出码 2 结束，且不写入任何内容」

请求写法：附上要修改的文字和额外规则

随附指南的 `docs/guide/05-build-and-clean.md` 给出了以下示例：调用时附上要修改的文字和额外规则。

```
/unslop the readme changes, no emdashes
// 对 README 的修改运行 unslop。不要使用 em dash。
```

其中「the readme changes」是要修改的文字，「no emdashes」是额外规则。

同一页面还指出，即使请求很短，Agent 也能理解意图，用这项 Skill 修改文字，例如：

```
unslop that, tighten it
// 对它运行 unslop，让文字更精练。
```

其中「that」指刚刚写的文章。

/bro 用没有术语的话重述难懂的回复

作用：不使用术语，重述刚才的回复

/bro 是让 Agent 不用术语，重新表述刚才回复的 Skill。正文只有三句话，是 pstack 中最短的 Skill。

它与 /unslop 的区别在于修改的范围。/unslop 只修改符合规则的地方，例如套话；/bro 则不使用一套规则，而是用浅白的话重述整份刚才的回复。

与 /unslop、/technical-writing 不同，/poteto-mode、Playbook 和其他 Skill 都不会在步骤中调用 /bro。因此，只有用户点名调用时，/bro 才会运行。

使用时机：回复写得很详细，却看不懂在说什么

当 Agent 的回复技术细节很多，却让用户看不明白究竟在说什么时，用户就可以使用这项 Skill。

步骤：正文的三句话就是指令

/bro 的正文只有以下三句话。

Restate your last message. Stop using jargon and speak coherently. State it more simply and concisely, like one human talking to another.

重新表述你刚才的话。不要用术语，把意思说清楚。像人与人对话那样，说得更简单、更简短。

SKILL.md 没有单独的步骤小节，Agent 直接把这三句话作为指令。

请求写法：只写「/bro」就够了

随附指南 10-recipes-and-pitfalls.md 将以下一个词列为完整的提示词。

```
/bro
```

正文的三句话已经是指令，用户无需再说明要怎样重述。

/technical-writing 先确定文档类型，再动笔写作

作用：让 Agent 按分层规范撰写技术文档

/technical-writing 是一项 Skill，它把 Agent 撰写技术文档时应遵循的规范分为四层，从整篇文章的类型一直细化到单个句子的写法。目标是让疲惫的工程师读一遍就能理解。

poteto 在《The Complete Guide to pstack》的 [Part 2](#) 中解释了创建这项 Skill 的缘由。

起因是他曾让 Agent 在没有这项 Skill 的情况下，为 Cursor 内部的桌面应用框架 Dune 撰写 README。据他说，生成的 README 把教程、操作指南、设计说明和 API 参考混在一起，还用了带有明显 AI 腔调的花哨措辞，读起来很费劲。

使用时机：撰写和审阅文档时

SKILL.md 的说明（description）列出的使用场景包括撰写及审阅文档、RFC（提出设计方案的文档）、README、PR 描述和提交信息。

不过，/technical-writing 也设置了 `disable-model-invocation: true`，所以 Agent 不会读到这段说明后自行调用 /technical-writing。这些场景之所以会使用它，是因为「Non-negotiables」的规则要求在撰写这些文档时使用这项 Skill。

在 Playbook 中，「Opening a PR」要求用这项 Skill 撰写 PR 标题、PR 描述和提交信息正文。「Multi-phase or multi-PR plan」也要求用它来撰写计划文档。

规则：四个层次及层次之外的规则

除了四个层次的规范，SKILL.md 还规定了不属于任何一层的规则。我们先看这些规则，再依次看四个层次。

优先于四个层次的三条规则

除了四个层次，Agent 还要遵守以下三条优先级高于所有层次的规则。

- 删除所有删去后不改变句意的词。例如，将「In order to」改为「to」。
- 使用简短、日常的词。例如，写「use」而不是「utilize」。

- 如果遵守某条规则反而使句子变差，就用别的方法修改，或保留原句。这些规则服务于读者；即便全部遵守，文章读起来仍像机器写的，也算失败。

沿用代码库中的名称，不创造新术语

Agent 要原样使用代码库中实际存在的符号（函数名、类型名、变量名等代码中有名字的事物）、文件、标志和命令的名称，不随意改写。

Agent 也不创造新的术语，而是使用开发者平时说的话。例如，把代码移到别处时，写「move（移动）」，而不是「evacuate（疏散）」。

另一方面，「Strangler Fig 模式」这类已有固定设计或做法、且已广泛使用的名称，Agent 也应沿用。

但 Agent 要在这个名称第一次出现在文档中时解释其含义。例如，可以写「采用 Strangler Fig 模式（逐步以新系统替换旧系统的做法）进行迁移」。

/unslp 的规则 26（抽象的比喻性名词）列出了像 evacuate 这样需要改写的词。如果 Agent 发现列表之外的词，就不自行修改 /unslp，而是在回复中附上差异，提出将「列表中缺少的词」和「改写后的词」加入列表的建议。

句子长短交错，内容写得具体

关于句长和具体程度，这项 Skill 提出了以下四点要求。

- **有意交错使用长短句**……用短句点明要点，用长句连同条件或结果写清事实。
- **是否拆句取决于要表达的事情有几件，而非句子有多长**……「一句话讲一件事」不等于「每句话都要短」。一句话里有两件事就拆开；只有一件事，即使句子较长也可以保留。
- **在解说中写出作者的判断**……写 explanation（解说）时，不能只列优缺点，还要写作者如何评价。写 reference（参考文档）时则不要加入评价，只写事实。
- **具体描述，不要含糊**……不要笼统地写「更改 schema 可能导致问题」，而要写明会发生什么，例如「重命名列会导致构建失败」。

因为即使四个层次的规范全部得到遵守，如果全文都是切得很短的句子，却没有写出任何具体事情，读起来依然像机器写的。

PR 描述应让审阅者在一分钟内读完

Agent 要把 PR 描述控制在审阅者一分钟内能读完的长度。因此，/swarm（第 24 章）的运行日志、提交 SHA（每次提交所附、可唯一标识该提交的标识符）列表和指标表格，都应以链接指向，而不是贴进正文。

四个层次从整篇文档逐步细化到单个句子

四个层次是依次从整篇文档细化到句子内部的步骤。每一层分别回答一个问题。

层次	问题	要点
Diátaxis	这是什么类型的文档	在 tutorial（教程）、how-to（操作指南）、reference（参考文档）和 explanation（解说）中选一种；不同类型要拆开，并互相链接
Google developer style	句子如何对读者说话	称读者为「you」，用现在时和祈使句。标题除了说明本节主题，也要写出要点
STE	一句话写多少内容	一句话只表达一件事；在整篇文档中用同一个词表示同一种操作
Global English	有没有可以作两种理解的句子	写清「it」或「this」所指的对象，让读者不必猜测；在整篇文档中用同一个名称称呼同一事物

- **Diátaxis**……将技术文档分为 tutorial（教程）、how-to（操作指南）、reference（参考文档）和 explanation（解说）四种类型来撰写的思路
- **Google developer style**……Google 面向开发者的文档风格指南
- **STE**……Simplified Technical English，一套通过限制词汇和句型来规范技术文档英语写作的标准
- **Global English**……Kohl 的《The Global English Style Guide》提出的英语写作方法，使非英语母语读者、译者和 Agent 都能对句子作出一致的理解

下面用原文中的例子逐一说明表中的各个层次。

Diátaxis：先确定文档类型

第一层（Diátaxis）要求先确定文档类型。如果把不同类型混在一起，文档就会像本章「作用：让 Agent 按分层规范撰写技术文档」提到的 Dune README 一样难读。

文档类型由两个问题决定：「文档是为了行动还是为了理解？」以及「文档用于学习还是用于工作？」

	用于学习	用于工作
为了行动	tutorial	how-to
为了理解	explanation	reference

SKILL.md 对这四种类型的说明如下。

- **tutorial（教程）** ……供初次使用的人边动手实际制作东西边学习的文档。开头先展示最终成果，并在每一步说明「屏幕或输出中应该出现什么」。
- **how-to（操作指南）** ……供已经熟悉工具的人解决眼前问题的步骤说明。省去基础介绍，按顺序列出要做的事。
- **reference（参考文档）** ……供人在需要时查阅、核对准确事实的文档。列出规范和配置项，不加入操作步骤或作者的意见。
- **explanation（解说）** ……帮助读者理解机制或设计「为什么是这样」的文档。写出设计决策、过程、限制和其他方案。只有这种文档会写作者的意见。

如果一篇文档像 Dune 的 README 一样混合了四种类型，Agent 就应将其拆成四篇，并让它们互相链接。

不过，Agent 不会把这一层应用于 PR 描述和提交信息；对这两种文字只应用其余三个层次。

其余三个层次（Google developer style、STE、Global English）主要讨论英语文章的写法。建议用其他语言撰写文档的读者跳过接下来的三个小节。

Google developer style：直接对读者说话

Agent 将读者称作「you」，并用祈使句发出指示，例如「Click Submit。」。

Agent 还要把条件放在指示前面，写成「To delete the document, click Delete.」（要删除文档，请点击 Delete）。条件先出现，读者才能跳过与自己无关的指示。

此外，标题不只要写「这一节谈什么」，还要写出「这一节的要点」。

例如，不用只说明主题的「Modes（模式）」作标题，而用「Pick the mode first（先选模式）」，让读者从标题就知道「该做什么」。

这里的模式指的是 Diátaxis 所划分的文档类型。

STE：一句话只表达一件事

Agent 在一句话中只写一条指示。这里的指示，是指像「Install the component.（安装组件）」这样让读者执行某项操作的句子。

Agent 还要把长句拆成两句。判断标准是：指示句约 20 个词，其他句子约 25 个词。这里计算的是英语单词数。

写作时，Agent 不省略英语冠词「the」或「a」。例如，「Remove backup file」可以有两种读法，而「Remove the backup file（删除那个备份文件）」只能有一种读法。

- **冠词**……英语中放在名词前面的「the」「a」等词，日语没有冠词
- **「Remove backup file」为什么有两种读法**……原文没有具体说明是哪两种读法。本书认为可以这样理解：没有冠词时，可以把「backup file」看作一个名词（备份文件），理解成「删除备份文件」；也可以把 backup（备份）和 file 分开，将 file 理解成动词（归档），把整句话读成「删除备份，并将其归档」。有了「the」，就能确定「the backup file」是一个完整的名词短语

此外，Agent 在整篇文档中用同一个词表示同一种操作。如果某处写「start（开始）」，就不要在另一处改写成「initiate（启动）」。

Global English：消除有歧义的句子

Agent 要把「only」和「not」紧挨着它们所修饰的词放。因为位置不同，意思就会改变，比如「only fails on growth（增加时只是失败）」和「fails only on growth（只有增加时才会失败）」。

Agent 还要在各处用同一个名称称呼同一事物。如果一篇文档把同一套机制分别称作「the gate」「the ratchet」「the budget check」，读者会误以为有三套机制。

示例：按规则重写一个段落

SKILL.md 给出了一例修改前后的对照。修改前的段落如下。

Configuration of the proto import ratchet budget script parameters is performed via budget.json. Note that it's important to remember that running with --write, which updates the committed budget to reflect the current count, should only be done when lowering it. If exceeded, CI fails.

proto import ratchet budget script 的参数通过 budget.json 配置。请注意务必记住，带上 --write 运行会将已提交的预算更新为当前数量，因此只应在降低预算时这样做。如果超出预算，CI 就会失败。

修改后的段落如下。

budget.mjs reads the committed budget from budget.json and counts the files that import protos. If the count exceeds the budget, CI fails. Run budget.mjs --write only to lower the budget.

budget.mjs 从 budget.json 读取已提交的预算，并统计导入 proto 的文件数。如果数量超过预算，CI 就会失败。只有要降低预算时才运行 budget.mjs --write。

对照这两个段落，可以看到以下变化。

- 不再堆叠「proto import ratchet budget script」这一串名词，而是使用实际存在的文件名 budget.mjs 和 budget.json。
- 不再用「is performed via」这样的被动表达，而是以 budget.mjs 为主语，说明它做什么。
- 删除「Note that it's important to remember that」之类没有实际意义的词句。
- 不再使用「ratchet」这一比喻。ratchet 指只能朝一个方向转动的棘轮，这里原本用它来比喻「只能降低的上限」。
- 把「should only be done」这样的被动提醒，改成祈使句指示：「Run budget.mjs --write only to lower the budget。」。

请求写法：带上 Skill 名称提出请求

随附指南的 `docs/guide/09-make-it-yours.md` 给出了以下请求示例。

```
/technical-writing review the readme changes  
// 审阅 README 的修改。
```

同一页面还说明，用户既可以用这项 Skill 审阅自己或 Agent 已经写好的文档，也可以在请 Agent 撰写文档时，把 Skill 名称放在请求的开头。

小结

- **三者的区别**……`/unslop` 和 `/technical-writing` 让写作的 Agent 自己按规则修改；`/bro` 则是在用户调用时，让 Agent 重述刚才的话。
- `/unslop`……用编号不变的 28 条规则，去除各类文字中的 AI 腔调。
- `/bro`……不用术语，简短地重述刚才的回复。
- `/technical-writing`……先将文档类型定为一种，再写出让疲惫的工程师读一遍就能理解的文档。

下一章第 33 章将介绍回顾工作中学到的经验，并将其反映到现有 Skill 中的 `/reflect`。

第 33 章：回顾工作中的经验并落实改进

kaito · 来源：日本語 · English · 2026-10-03

本章介绍 `/reflect`。

`/reflect` 是一项 Skill：它从工作对话中提取能用于下一次工作的经验，并且只把用户批准的经验留下来，例如追加到现有 Skill 中。

负责提取经验的是两类子 Agent：从三个视角阅读对话的审阅者，以及汇总审阅结果的综合者。是否把经验写入 Skill，由用户决定。

本章从作用、使用时机、步骤和请求写法四个角度介绍 `/reflect`。

本章结构

本章包含以下内容。

- 作用：从对话中筛选长期适用的经验
- 使用时机：有收获的工作刚结束，且用户调用 `/reflect` 时
- 步骤：汇集三个视角的经验，获得批准后再应用
- 请求写法：写明发生了什么，以及想保留什么
- 小结

作用：从对话中筛选长期适用的经验

第 3 章讨论过这样一种做法：从用户纠正中得到的教训，不应只留在对话里，还应留在代码库、lint 等自动检查或 Skill 中。`/reflect` 负责其中写入 Skill 的部分。

不过，正如第 21 章的原则「**Encode Lessons in Structure**」所说，书面指令很容易被忽略。即使向 Skill 增加了一句话，如果下一个 Agent 没注意到，也不会有任何改变。

因此，本书将这项 Skill 的**步骤**中的第 3 至第 5 步，视为**筛选经验的步骤，而不是增加经验数量的步骤**。

- 综合者剔除**不会改变未来 Agent 行为的经验**。
- 对于可通过 lint 或脚本强制执行的经验，Agent 不把它写成 Skill 的文字，而是放进团队使用的议题管理工具的待办项（Backlog）。
- 在剩余经验中，由用户选择要把哪些写入 Skill。

经过筛选，Skill 中只留下**确实会改变下一个 Agent 行为的经验**。

使用时机：有收获的工作刚结束，且用户调用 /reflect 时

随附指南的 `docs/guide/09-make-it-yours.md` 建议在有收获的工作刚结束时调用 /reflect。/reflect 的 SKILL.md 将使用时机定义为用户说「reflect」或「/reflect」时。

另一方面，在以下情况下，Agent 不使用这项 Skill。

- 对话内容琐碎
- 对话偏离了主题
- Agent 按照现有 Skill 正确完成了工作

最后一种情况下，工作能按现有 Skill 顺利完成，本身就是该 Skill 内容正确的证据。

此外，Agent **不会把一次性事件当作可沉淀的经验**。

还需注意，这项 Skill 设置了 `disable-model-invocation: true`（第 25 章），所以 Agent 不会读到「reflect」一词后自行选择它。要运行这项 Skill，用户需明确以 /reflect 的名称调用。

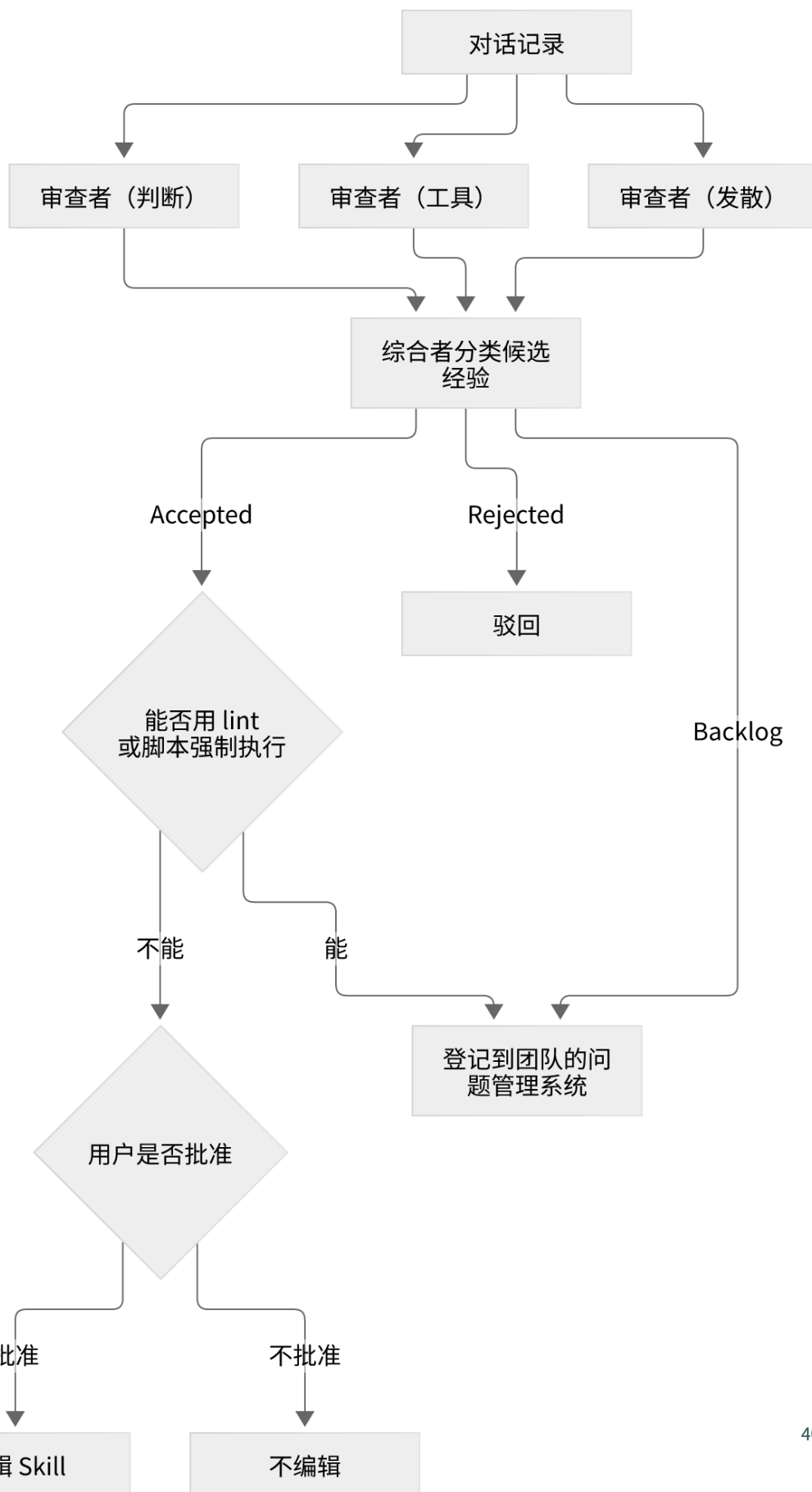
步骤：从三个视角汇集经验，获得批准后再应用

步骤如下。

1. **找到对话记录**.....Agent 寻找当前对话的 transcript 文件（与 Agent 对话的记录）。搜索范围仅限当前工作区的 `agent-transcripts/` 目录。如果连其他项目的目录也搜索，就可能读到不相关项目的私人对话。找不到文件时，Agent 会写一份对话摘要，交给审阅者代替原文件。

2. **同时启动三个视角的审阅者**.....Agent 同时启动三个审阅者，分别从判断（Judgment）、工具（Tooling）和发散（Divergent）的视角阅读对话。默认模型分别为：判断和发散使用 `claude-opus-5-5-max`，工具使用 `gpt-5.6-sol-max`；可通过 `/setup-pstack` 修改（第 34 章）。各视角寻找什么经验，见本节「三个视角的审阅者分别寻找不同类型的经验」。
3. **综合者分类**.....综合者把三个审阅者提出的候选经验分为 Accepted（采纳）、Rejected（驳回）和 Backlog 三类。
4. **把可通过 lint 或脚本强制执行的内容移入 Backlog**.....在 Accepted 中，对于可由 lint 规则、脚本、元数据标志或运行时检查强制执行的内容，Agent 不把它写进 Skill，而是移入 Backlog。书面指令只有被人注意并遵守才有效；lint 和脚本即使无人留意，也能强制执行规则。
5. **获得用户批准后才应用 Skill 编辑**.....Agent 向用户展示综合者的全部分类结果，由用户从 Accepted 中选择要应用的内容。用户也可以改变经验的归属，即决定写进哪项 Skill。Agent 只应用用户选中的编辑。只有 Accepted 的项目需要等待批准；Backlog 项目则由 Agent 无需等待批准，直接登记到议题管理工具。
6. **向用户简短汇报**.....Agent 将已应用的编辑、新建的 Skill、移入 Backlog 的项目和驳回的项目逐项列出，每项一行，并附上驳回理由。

把第 2 至第 5 步画成图，就是下面这样。



查看图示 1

只有留在 Accepted 中、且获得用户批准的经验，才会引发 Skill 编辑。

三个视角的审阅者分别寻找不同类型的经验

三个视角及各自寻找的经验如下。

- **判断（Judgment）** ……寻找个别事件背后长期适用的原则，例如用户纠正背后的原则。
- **工具（Tooling）** ……寻找下一个 Agent 必须重新查找的具体信息，包括工具、命令、路径和标志。例如 Agent 通过反复尝试找到的命令标志，或在本地复现失败执行的方法。此外，还要指出用户手动提供、但 Agent 本可通过 MCP（连接 Agent 与外部工具或数据的机制）自行取得的信息，例如用户贴进对话的工单 ID。
- **发散（Divergent）** ……寻找另外两个视角遗漏的内容。例如因错误理由而碰巧奏效的判断、只是测试碰巧通过才得以保留的判断，以及 Agent 跳过的验证。

对于每一条经验，三个审阅者都会返回以下三项内容。

- **原则**……用一句话写出下一次工作仍适用的经验
- **依据**……对话中产生该经验的具体场景
- **归属**……要写入这条经验的 Skill

作为归属的 Skill 必须是这段对话实际用过的。如果某项 Skill 本来有帮助却没有被使用，审阅者应提出修改其说明（description）的建议。不属于这两种情况的经验，审阅者会舍弃。

综合者依照八项标准筛选经验

对综合者的指示写在 references/synthesizer.md 中。根据这份指示，综合者要用以下八项标准，逐一检查候选经验。

标准	检查什么
Durability（持久性）	六个月后文件路径或工具版本变了，这条经验还正确吗
Specificity（具体性）	是否既宽泛到能用于别的工作，又具体到能辨认适用场景
Existing-skill-first（优先使用现有 Skill）	能否只补充现有 Skill，而不创建新 Skill
Convergence（共识）	是否有多个审阅者提出这条经验

标准	检查什么
Decision-changing（改变行为）	写下这条经验后，下一个 Agent 的行动是否会改变
Structural-mechanism check（可否通过机制执行）	能否通过 lint 或脚本强制执行
Skill-was-used（Skill 是否用过）	要补充的 Skill 是否在这段对话中实际使用过
Already-covered（是否已有规定）	目标 Skill 中是否已经写了相同内容

例如，对综合者的指示列出了下面这一行作为应舍弃的经验示例。

```
linter at SHA bd91aa7 uses chars/4 heuristic  
在提交 bd91aa7 时，linter 使用字符数除以 4 的估算方法
```

bd91aa7 是 SHA（每次提交所附、可唯一标识该提交的标识符）。这一行只描述了「提交 bd91aa7 时的代码中，linter 是这样运行的」这一事实。如果后续提交改写了 linter，这个事实就不再正确。

因此，综合者依据 Durability 标准，不采纳它。

修改 Skill 前，须获得用户明确批准

Skill 的改动会影响组织未来所有的 Agent。因此，Agent 不会自动应用 Skill 编辑，而是等待用户明确批准。

对于用户批准的编辑，Agent 会依修改规模采用以下方式。

- 增加一行，或纠正过时事实等小改动……Agent 自行直接编辑。
- 增加新章节等超过十行的大改动……交给 Cursor 内置的 create-skill（用于编写 Skill 的 Skill），让它反复起草、测试和修改。
- 已有 Skill，却在本该使用的场景中没有使用……让 create-skill 修改该 Skill 的说明（description）。
- 现有 Skill 中没有合适的归属……让 create-skill 创建新的 Skill。

如果环境中 SKILL.md 验证工具，Agent 在完成工作前，会用它验证所有修改过的 Skill。

随附指南建议用户只批准那些会改变未来 Agent 判断的提案。同一页面还写道：「只发生过一次的奇怪事件是偶然事件，不是规则。」

SKILL.md 验证工具的例子

SKILL.md 验证工具主要检查文件开头的配置区（frontmatter）是否写得正确。例如有以下工具。

- `skills-ref validate`……由发布 Agent Skills 规范的代码库提供、作为参考实现的库中的命令。运行时指定 Skill 目录，例如 `skills-ref validate path/to/skill`
- `quick_validate.py`……Anthropic 的 `anthropics/skills` 代码库中，用于创建 Skill 的 Skill（skill-creator）附带的脚本。它检查是否有 SKILL.md、frontmatter 是否为有效 YAML、是否包含 name 和 description，以及是否出现不允许的字段

另外，pstack 本身没有随附用于验证 SKILL.md 的脚本工具。

因此，即使环境中没有验证工具，在 pstack 中，编写 Skill 的「**Authoring or modifying a skill**」Playbook（第 13 章）也承担验证工具的角色。

这份 Playbook 要求检查 frontmatter 是否包含 name 和 description、被引用的文件是否存在，以及通向其他 Skill 的链接是否可访问。

请求写法：写明发生了什么，以及想保留什么

随附指南给出了以下请求示例，供用户在有收获的工作刚结束时发送。

```
/reflect that took way too long. capture what we learned so the next run doesn't repeat it.
// 花费太久了。记录经验，避免下次执行时重蹈覆辙。
```

这项请求写明了发生什么事（花了太多时间），以及用户想保留什么（避免下一次执行重蹈覆辙的经验）。

小结

- **收集经验的方法**……从判断、工具、发散三个视角收集经验，再根据六个月后是否仍正确等标准进行筛选。

- **Backlog**……能通过 lint 或脚本强制执行的经验，不写进 Skill，而是移入团队的议题管理工具。
- **保留经验的方法**……获得用户批准后，向现有 Skill 的 SKILL.md 补充内容或修改原有内容。

下一章第 34 章将介绍安装 pstack 并为各个角色选择所用模型的 `/setup-pstack`。

第 34 章：安装 pstack，为各角色选择模型

kaito · 来源：日本語 · English · 2026-10-03

本章介绍 `/setup-pstack`。

`/setup-pstack` 是一项按 **pstack** 启动的子 Agent 的角色决定所用模型的 Skill。用户安装 pstack 插件后，首先运行这项 Skill。

这里的角色，指子 Agent 承担的职责，例如 `bug-fix`（在「Bug fix」Playbook 中编写代码的子 Agent）、`judgment and prose`（负责写作和判断的子 Agent），以及 `arena runners`（`/arena` 让其解决同一任务的候选子 Agent）。

本章将介绍如何安装 pstack、`/setup-pstack` 的执行过程，以及它写出的规则文件。

本章也介绍 pstack 的安装方法，因为和运行 `/setup-pstack` 一样，安装也是开始使用 pstack 时最先要做的事。

pstack 自带的 Skill 与需要作为另一个插件安装的 Skill，已在「第 6 章：步骤中点名依赖的某些 Skill 并未随附」中讨论。

本章结构

本章包含以下内容。

- 安装 pstack
- 角色一览
- 整体流程
- 检测可用模型
- 读取上次的规则文件
- 选择推理预算

- inherit-parent 与 auto
- 写出前确认模型可用
- 面板角色
- 写出的文件
- 再次运行时保留的行
- 提议创建验证 Skill
- 小结

安装 pstack

安装 pstack 只需一条命令和一段简短对话。

1. 在 Cursor 聊天窗口运行以下命令。

```
/add-plugin pstack
```

2. 接着运行以下命令启动 /setup-pstack，并回答它提出的问题。本章后续章节会说明它问什么，以及这些回答决定什么。

```
/setup-pstack
```

3. /setup-pstack 结束后，开始一段新对话。因为 /setup-pstack 写出的模型规则从新对话开始生效。

角色一览

/setup-pstack 会为以下 17 个角色选择模型（以 pstack version 0.15.5 为准）。角色名称沿用规则文件中的原文。

角色	承担职责的子 Agent	默认模型
feature, refactoring	在「Feature」「Refactoring」Playbook 中编写代码	grok-4.7-xhigh-fast

角色	承担职责的子 Agent	默认模型
bug-fix	在「Bug fix」Playbook 中编写代码	grok-4.7-xhigh-fast
perf-issue	在「Perf issue」Playbook 中编写代码	grok-4.7-xhigh-fast
hillclimb	在「Hillclimb」Playbook 中编写代码	grok-4.7-xhigh-fast
judgment and prose	撰写文字并作出判断	claude-opus-5-5-max
hardest tasks	为最困难的变更编写代码，包括跨多个位置的设计、复杂并发处理和不易发现细微错误的算法	claude-opus-5-5-max
how explorer	在 /how 中分头调查代码	grok-4.7-xhigh-fast
how explainer	在 /how 中将调查结果汇总成一份说明	claude-opus-5-5-max
why investigators	在 /why 中分头调查 Git 历史、工单等信息来源	grok-4.7-xhigh-fast
why synthesizer	在 /why 中将调查结果汇总成一个答案	claude-opus-5-5-max
reflect tooling	在 /reflect 中从工具视角阅读对话	gpt-5.6-sol-max
reflect judgment, divergent, synthesizer	在 /reflect 中从判断和发散视角阅读对话，并分类结果	claude-opus-5-5-max
arena runners	在 /arena 中解决同一任务的候选者	claude-opus-5-5-max、gpt-5.6-sol-max、grok-4.7-xhigh-fast
arena cross-judge pool	在 /arena 中为候选成果物评分的评判者（从中选择一个）	claude-opus-5-5-max、gpt-5.6-sol-max、grok-4.7-xhigh-fast
swarm workers	在 /swarm 中并行工作的 Worker	grok-4.7-xhigh-fast
architect runners	在 /architect 中拟定设计方案	claude-opus-5-5-max、gpt-5.6-sol-max、grok-4.7-xhigh-fast
interrogate reviewers	在 /interrogate 中寻找差异中的弱点	claude-opus-5-5-max、gpt-5.6-sol-max、grok-4.7-xhigh-fast

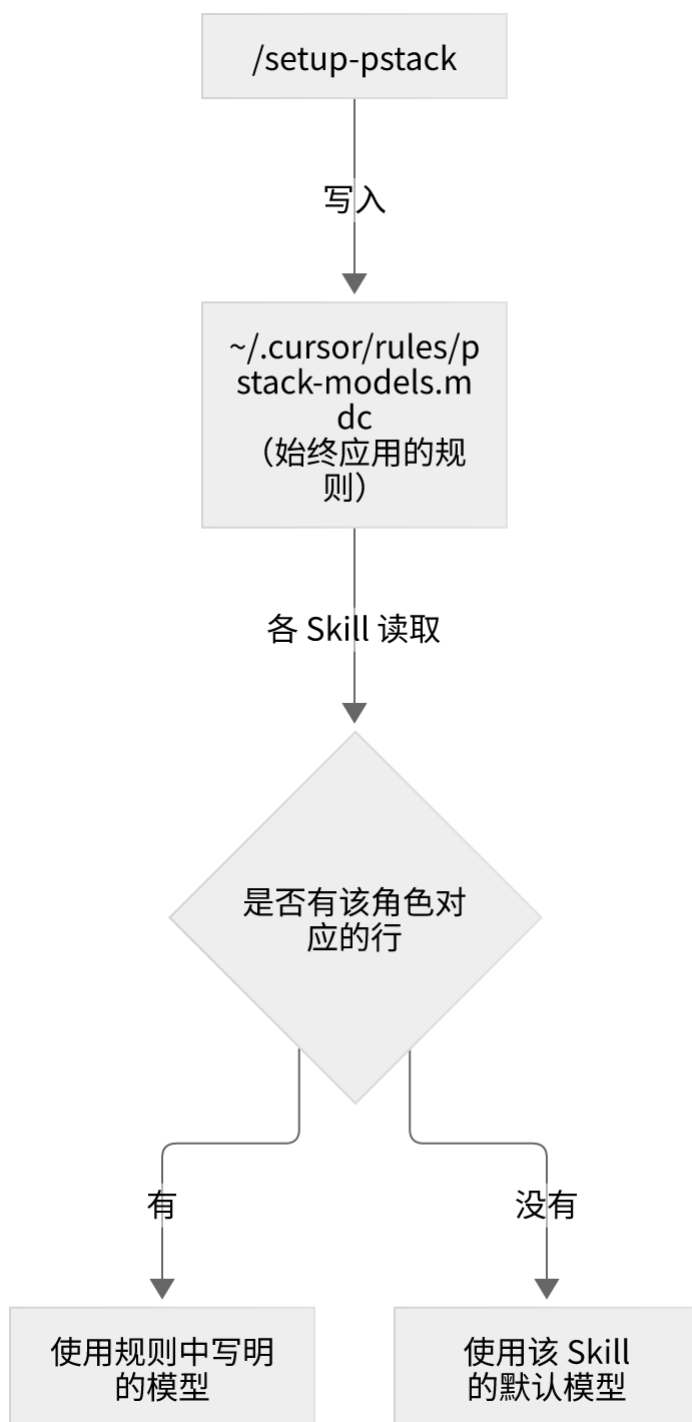
默认模型列出三个模型的角色，会在配置值中写入模型列表。本章后面会逐项说明。

整体流程

/setup-pstack 会询问推理预算和各角色的模型，并将回答写入 ~/.cursor/rules/pstack-models.mdc，成为一条**始终生效的规则**。

- **推理预算**……原文为 budget，用于一次性决定各角色模型的推理强度
- **slug**……用于指定模型的标识符，例如 claude-opus-5-5-max
 - 例如，claude-opus-5-5-max 表示以 Effort Level max 运行 Claude Opus 5.5
- **推理强度 (Effort Level)** ……模型投入推理工作的等级。由 slug 中的 max、xhigh、high、medium、low 等词表示，越靠左越强

pstack 的各项 Skill 读取这条规则，决定各角色使用的模型。



[查看图示 1](#)

没有对应行的角色，会使用该 Skill 的默认模型。

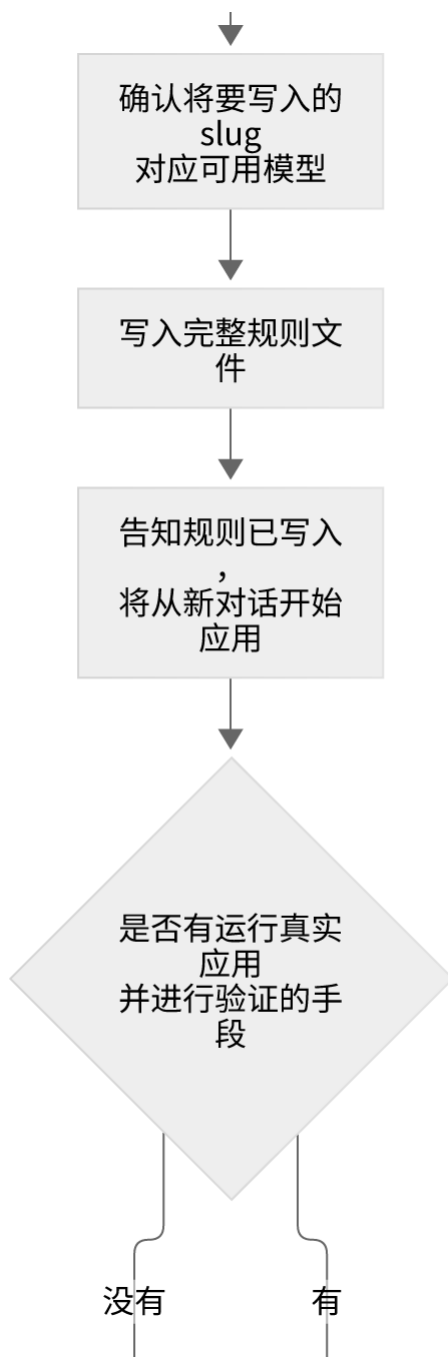
因此，用户只需要修改想偏离默认模型的角色。例如，只想让 bug-fix 使用另一种模型，就只改 bug-fix 那一行。

若要让某个角色恢复使用默认模型，从规则文件中删除那一行即可。

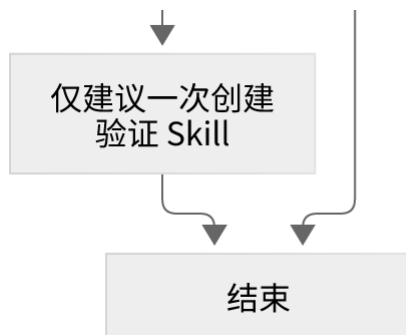
/setup-pstack 按以下顺序执行。



图示分段 1



图示分段 2



图示分段 3

[查看图示 2](#)

检测可用模型

首先，`/setup-pstack` 调查用户可用的模型 slug。调查方法如下。

- **基本方法**……检查当前对话中可传给子 Agent 的 slug。原文认为这是可靠的方法。
- **有 API 或 CLI 时**……如果 Cursor 提供列出用户可用模型的 API 或 CLI，为了避免遗漏，优先使用它们。
- **什么也没找到时**……请用户把自己可用的 slug 贴到聊天中。

本章将这样找到的 slug 称为「可用模型」。

读取上次的规则文件

如果过去运行过 `/setup-pstack`，它当时写出的规则文件（`~/.cursor/rules/pstack-models.mdc`）就已经存在。此时，`/setup-pstack` 会读取文件中的 `# budget` 行（上次选择的预算）和各角色的值，作为这次重新选择的起点。

不过，上次的规则文件中，可能还保留着已废弃角色的行（例如 `how critics` 行）。所谓已废弃角色，就是当前 `pstack` 写出的文件中不再列出的角色（文件内容见本章「[写出的文件](#)」）。

`/setup-pstack` 会丢弃这些行，并在展示角色与模型列表时，告知用户丢弃了哪些行。

选择推理预算

推理预算是一项**一次性决定所有角色的模型要以哪一级推理强度运行**的设置。强度由 slug 中的 `max`、`xhigh`、`high`、`medium`、`low` 表示；`max` 最强，`low` 最弱。

用户从以下四种预算中选择。

每种预算都对应一个要统一达到的推理强度（下称「目标强度」）。

预算	目标强度	<code>claude-opus-5-5-max</code> 如何变化
<code>unlimited</code>	<code>max</code> （但不替换 slug 中的强度，保留默认值）	<code>claude-opus-5-5-max</code>

预算	目标强度	claude-opus-5-5-max 如何变化
large	统一为 xhigh	claude-opus-5-5-xhigh
medium	统一为 high	claude-opus-5-5-high
small	统一为 medium	claude-opus-5-5-medium

将 slug 中的强度词替换为目标强度

选择 large、medium 或 small 时，/setup-pstack 会把所有角色的 slug 中表示强度的词替换为目标强度。强度词位于 slug 末尾；若末尾是 fast，则位于 fast 前面。

例如，选择 small（目标强度为 medium）后，变化如下。

- claude-opus-5-5-max → claude-opus-5-5-medium
- grok-4.7-xhigh-fast → grok-4.7-medium-fast

如果一行中为同一角色列出多个模型，例如 arena runners: claude-opus-5-5-max, gpt-5.6-sol-max, grok-4.7-xhigh-fast，列出的所有 slug 都会被替换。这类角色将在本章「[面板角色](#)」中说明。

inherit-parent 和 auto 不是 slug，因此不会被替换。

替换后的模型不可用时，在同一系列内选择替代模型

替换后的 slug 有时不在已检测到且可用的模型之中。例如，claude-opus-5-5-medium 可能不可用。

此时，/setup-pstack 按以下顺序选择替代模型。

1. 寻找同一系列（原文为 family）的可用模型
2. 在推理强度不高于目标强度的模型中，选择强度最高的一个
3. 如果一个也找不到，就标记该角色，请用户重新选择

例如，用户选择 small（目标强度为 medium），而可用的 Claude 模型只有 claude-opus-5-5-high 和 claude-opus-5-5-low。

- high 强于目标的 medium，所以不使用。
- low 弱于目标的 medium，所以可以使用。

因此，`/setup-pstack` 选用 `claude-opus-5-5-low`。

替代模型只从同一系列选择，所以**即使降低预算，模型系列也不会改变**。例如，原本使用 Claude 的角色不会因降低预算而切换到 Grok。

最后核对角色与模型列表

替换完成后，`/setup-pstack` 会展示所有角色及其分配模型的列表，并标记需要重新选择的角色。

用户可以直接接受，也可以修改特定角色的模型。修改时，可以选择一个可用模型，或选择 `inherit-parent`、`auto`。

inherit-parent 与 auto

`inherit-parent` 和 `auto` 都表示**该角色的子 Agent 使用与父对话相同的模型**。父对话就是启动子 Agent 的那段对话。

对于设为这两个值的角色，各项 Skill 启动子 Agent 时不会指定模型。未指定模型的子 Agent 将使用与父对话相同的模型。

因此，使用 Auto（由 Cursor 自动选择模型的设置）进行对话的用户，可以在角色中写入 `inherit-parent` 或 `auto`，让子 Agent 也使用 Auto。

写出前确认模型可用

用户核对完列表后，`/setup-pstack` 会在写出规则文件前，检查将要写出的 slug 是否包含在本章「[检测可用模型](#)」一节检测到的可用模型中。

只要有一个 slug 不在其中，`/setup-pstack` 就会停止写入，请用户为该角色重新选择模型。`/setup-pstack` **不会将未经确认可用的 slug 写入规则文件**。

因为规则文件中的 slug 会在各项 Skill 启动子 Agent 时，直接作为模型指定值传入。

`inherit-parent` 和 `auto` 不是模型 slug，因此不在此次检查范围内；它们始终可以写入规则文件。

面板角色

各家前沿模型（各公司的最先进模型）都有强项和弱项。因此，pstack 的许多 Skill 会组合多个模型，发挥各自的长处（[README](#)）。

对于为此列出多个模型的角色，/setup-pstack 称之为面板角色（原文为 panel roles）。面板是一种让多个模型解决同一问题、再比较结果的机制（[第 6 章](#)）。

面板角色有以下三个。

- arena runners……/arena 让其解决同一任务的候选子 Agent
- architect runners……/architect 让其在写代码前拟定设计方案的子 Agent
- interrogate reviewers……/interrogate 让其查找差异中弱点的审阅子 Agent

列出几个模型，就启动几个子 Agent

面板角色的值用，分隔多个模型。**列出几个模型，就启动几个子 Agent。**

例如，若值为 arena runners: claude-opus-5-5-max, gpt-5.6-sol-max, grok-4.7-xhigh-fast, /arena 会在三个模型上各启动一个子 Agent，总共三个。

列表中写入 inherit-parent 或 auto 时，每一项也会启动一个子 Agent。这个子 Agent 使用父对话的模型。

即使列出多个模型，也不属于面板角色的情况

规则文件中有一行虽然列出多个模型，却不属于面板角色，即 arena cross-judge pool。

arena cross-judge pool 列出的是 /arena 的评判者（为候选成果物评分的 Agent；[第 24 章](#)）可选的模型。所有候选者写完后，/arena 才会从该行列出的模型中选一个，启动评判者。选择时，尽量选与父对话不同系列的模型。

swarm workers 也不是面板角色。它是 /swarm 启动的所有 Worker 所用的默认模型，值中只写一个模型。

例外是模型竞赛（原文为 model race）：让运行于不同模型的 Worker 解决同一任务，比较哪种模型给出最好的结果。

在这种情况下，/swarm 启动 Worker 前，会逐个决定其使用的模型。该 Worker 使用指定的模型，而非 swarm workers 的模型。

写出的文件

/setup-pstack 每次运行都会将规则文件整体重写，不会向上次的文件追加行，也不会只修改部分行。

如果预算选 unlimited，且所有角色都接受默认值，/setup-pstack 写出的文件如下（以 pstack version 0.15.5 为准）。

```
---
description: pstack per-role model choices (overrides skill defaults)
alwaysApply: true
---
# pstack model configuration. One line per role. Delete a line to fall back to the skill default.
# `inherit-parent` or `auto` as a value: the role runs on the parent chat model (omit Task `model`).
Alias entries in a panel list still count toward its fan-out.
# budget: unlimited (max)
feature, refactoring: grok-4.7-xhigh-fast
bug-fix: grok-4.7-xhigh-fast
perf-issue: grok-4.7-xhigh-fast
hillclimb: grok-4.7-xhigh-fast
judgment and prose: claude-opus-5-5-max
hardest tasks: claude-opus-5-5-max
how explorer: grok-4.7-xhigh-fast
how explainer: claude-opus-5-5-max
why investigators: grok-4.7-xhigh-fast
why synthesizer: claude-opus-5-5-max
reflect tooling: gpt-5.6-sol-max
reflect judgment, divergent, synthesizer: claude-opus-5-5-max
arena runners: claude-opus-5-5-max, gpt-5.6-sol-max, grok-4.7-xhigh-fast
arena cross-judge pool: claude-opus-5-5-max, gpt-5.6-sol-max, grok-4.7-xhigh-fast
swarm workers: grok-4.7-xhigh-fast
architect runners: claude-opus-5-5-max, gpt-5.6-sol-max, grok-4.7-xhigh-fast
interrogate reviewers: claude-opus-5-5-max, gpt-5.6-sol-max, grok-4.7-xhigh-fast
```

根据 README，pstack 默认将 grok-4.7-xhigh-fast 分配给编写代码的角色（feature, refactoring、bug-fix、perf-issue、hillclimb），将 claude-opus-5-5-max 分配给负责最困难变更的 hardest tasks，以及负责文字和判断的 judgment and prose。

面板角色的默认模型有三个：claude-opus-5-5-max、gpt-5.6-sol-max 和 grok-4.7-xhigh-fast。

顺便附上我目前最新的配置。

我特别在意两点：调查角色与说明角色使用不同模型，以及 /arena 的评分角色中没有 Claude 模型。

因为模型不同，人格也不同。

```
---
description: pstack per-role model choices (overrides skill defaults)
alwaysApply: true
---
# pstack model configuration. One line per role. Delete a line to fall back to the skill default.
# `inherit-parent` or `auto` as a value: the role runs on the parent chat model (omit Task `model`).
Alias entries in a panel list still count toward its fan-out.
# budget: medium (high)
feature, refactoring: grok-4.7-high
bug-fix: grok-4.7-high
perf-issue: grok-4.7-high
hillclimb: grok-4.7-high
judgment and prose: claude-opus-5-5-high
hardest tasks: claude-opus-5-5-high
how explorer: grok-4.7-high
how explainer: claude-opus-5-5-medium
why investigators: grok-4.7-high
why synthesizer: claude-opus-5-5-medium
reflect tooling: claude-sonnet-5-5-high
reflect judgment, divergent, synthesizer: claude-opus-5-5-medium
arena runners: claude-opus-5-5-high, muse-spark-1.3-max, grok-4.7-high, claude-sonnet-5-5-high
arena cross-judge pool: gpt-5.6-sol-medium, grok-4.7-high, muse-spark-1.3-max
swarm workers: grok-4.7-high
architect runners: claude-opus-5-5-high, muse-spark-1.3-max, grok-4.7-high, claude-sonnet-5-5-high
interrogate reviewers: claude-opus-5-5-medium, muse-spark-1.3-max, grok-4.7-high, claude-sonnet-5-5-high
```

再次运行时保留的行

再次运行 /setup-pstack 时，它会查看上次的规则文件，按以下方式决定各角色的模型。

- 上次仍使用默认模型的角色……从默认模型重新决定。
- 上次由用户改成非默认模型的角色……直接沿用用户改过的模型。

例如，上次只把 bug-fix 从默认的 grok-4.7-xhigh-fast 改为 claude-opus-5-5-max。再次运行后，bug-fix 会继续使用 claude-opus-5-5-max，其他角色则从默认模型重新决定。

用户无须每次重新运行时，再次选择自己已经改过的角色。

pstack version 0.15.3 之前的规则文件会保留旧模型

这个机制有一个注意点。

在 pstack version 0.15.3 之前写出的规则文件，记录的是当时的默认模型，而 pstack 的默认模型后来变了。也就是说，那些曾经的默认值已经不同于现在的默认值。

因此，即使再次运行，/setup-pstack 也会把那些行当成「写有非默认模型的行」而保留。相应角色会继续使用旧的默认模型。

如果曾在 pstack version 0.15.3 之前运行过 /setup-pstack，应先采取以下两种措施之一，再次运行 /setup-pstack ([README](#) 及随附指南)。

- 删除写有当时默认模型的角色行
- 删除整个规则文件 (~/.cursor/rules/pstack-models.mdc)

提议创建验证 Skill

最后，/setup-pstack 会检查 Agent 是否能够实际运行项目的应用，确认变更确实生效。它会检查项目中是否存在以下两种设施。

- **验证 Skill**……供 Agent 启动应用、操作应用和收集证据的项目专用 Skill (第 3 章)。用 /create-verification-skill 创建时，名称为 verify-**<应用名>**。
- **现有测试工具链**……代码库中已经具备、可从外部操作应用的机制 (例如 Playwright 测试)。

如果两者都没有，/setup-pstack 会且只会提议一次，建议用 /create-verification-skill 创建验证 Skill。

在提议中，/setup-pstack 会说明创建验证 Skill 的目的是「**让 Agent 能像用户一样操作应用，证明变更确实有效**」。

- **用户接受时**……/setup-pstack 会调用 /create-verification-skill。
- **用户拒绝时**……/setup-pstack 不再建议 /create-verification-skill，直接结束。

/create-verification-skill 创建验证 Skill 的步骤见第 26 章；创建验证 Skill 并开始在自己的应用中使用的流程见第 36 章。

小结

- **写出的内容**……/setup-pstack 只使用已确认可用的 slug，把各角色的模型写进一个规则文件。
- **推理预算**……降低预算时，不更换模型系列，只降低推理强度。
- **生效时间**……规则从新对话开始生效，因此运行后需要开启新对话。

下一章第 35 章将介绍 pstack 安装完成后，用户将工作请求交给的 pstack 路由器 /poteto-mode。

第 35 章：/poteto-mode，pstack 的路由器

kaito · 来源：日本語 · English · 2026-10-03

/poteto-mode 是 pstack 的入口。它读取请求，从 23 个 Playbook 中选择一个，并按其步骤推进工作（第 7 章）。

本章介绍 /poteto-mode 的主体文件 SKILL.md。

/poteto-mode 的调用方法及请求路由已在第 7 章介绍；Playbook、Skill、Principle 的职责，以及「Non-negotiables」的主要条件，已在第 8 章介绍。本章将说明那些章节尚未涉及的 /poteto-mode 主体文件 SKILL.md 的内容。

本章结构

本章包含以下内容。

- /poteto-mode 是什么
- 「Non-negotiables」：必须做的事
- 「Principles」：原则索引
- 「Autonomy」：可自行推进与必须停下的范围
- 「Subagents」：委派规则
- 「Writing the reply」：回复写法
- 「Comments」：注释写法
- 「Playbooks」：Playbook 索引
- 调整成自己的工作方式
- 小结

/poteto-mode 是什么

本章标题中的「Router」（路由器）指 /poteto-mode 将请求分配给合适 Playbook 的职责。随附指南也将介绍 /poteto-mode 的页面（02-poteto-mode.md）命名为「Route work through /poteto-mode」（让工作经由 /poteto-mode 路由）。

除了将请求分配给 Playbook 的规则，/poteto-mode 还写有分配之后适用于整个工作过程的规则。例如：

- 编写代码前，先确定数据的形态（原则「**Model the Domain**」）
- 在不可撤销的操作前一定停下
- 用简短、明确的陈述句回复

SKILL.md frontmatter 中的 description 将 /poteto-mode 描述为「poteto 的 Agent 风格」（poteto's agent style），并列出了这套风格追求的五点。

- 简洁而详细的回复（concise, detailed responses）
- 经过深思熟虑的子 Agent 委派（deliberate subagents）
- 去除 AI 腔调的文字（unslopped prose）
- 简单的代码（simple code）
- 已经验证的工作（verified work）

安装完 pstack 后（第 34 章），用户将工作请求交给 /poteto-mode。

/poteto-mode 设置了 disable-model-invocation: true，所以只有用户按名称调用时才会启动。**调用一次后，在同一段对话的后续轮次中，即使不再写 /poteto-mode，它仍会生效，直到用户说停用（README、第 7 章）。**即使它处于生效状态，也只有在请求符合某个 Playbook，或需要严格流程时，才会按 Playbook 的步骤推进。

SKILL.md 的正文由「Non-negotiables」到「Playbooks」的七个小节构成。

「Non-negotiables」：必须做的事

「Non-negotiables」列出「遇到这种情况，必须这样做」的规则。主要规则已在「第 8 章：Non-negotiables 串联三个组成部分」中介绍。

「Non-negotiables」开头有两条适用于所有规则的要求。

- Agent 应在回复中点名影响自己判断的原则，并说明该原则改变了哪项选择
- 只有本次会话中完整读过相应原则 Skill 文件的原则，才可以被列举

例如，修复第 8 章的实际案例（重试在运行中途发生时，导出文件会写出重复行）后，回复中可以加入以下内容。

按照 Fix Root Causes，修复了每次重试都重复写入已导出行的根本问题，没有在导出后增加去重步骤。
按照 Make Operations Idempotent，改为按按键覆写。即使同一次执行运行两遍，行数也不变。

也就是说，以下回复不符合上述两条要求。

遵循了 Fix Root Causes 和 Make Operations Idempotent

因为只列原则名称的回复，无法让人知道 Agent 根据原则作了什么决定。而且，如果 Agent 说不清做了什么决定，它可能根本没有实际应用该原则。

用户看到原则名称和 Agent 的判断，就能核对 Agent 根据什么标准，作出了什么选择。

除了第 8 章介绍的规则及上述共同要求，还有以下规则。

情形	必须做的事	详情
Agent 准备问用户「哪种做法更好」	提问前，先把问题的答案分为「执行后可确定的事实」或「需由人决定的判断」	第 38 章
编写代码	先弄清数据的形态，再选择结构。 遵循原则「 Model the Domain 」	第 18 章
并行展开工作	覆盖、竞赛、关卡（验证门槛）和拆分探索使用 /swarm；比较设计或代码，选择一个基础方案并移植其他方案的优点，则使用 /arena	第 24 章
存在分歧的设计	合并前使用 /interrogate	第 27 章
并非显而易见的多步骤工作	写出吞吐量检查点，与「 Feature 」的第 3 步相同	第 12 章

情形	必须做的事	详情
询问 PR 状态的请求	使用「 Babysit 」，而非 Cursor 内置的 <code>/autopi lot Skill</code> （旧名 <code>babysit</code> ）。仅仅打开 PR 时不使用它	第 14 章
用户请求合并所有检查都通过的 PR 堆栈	执行「 Shipping 」。即使所有检查都通过，也不据此认定安全；每个 PR 都要独立确认后再合并	第 14 章
Bugbot 或 Agent 的安全审阅评论	不要照单全收；逐条分为修复（fix）、驳回（dismiss）或追问（ask）。因为 Bugbot 等虽然能发现真实缺陷，也会提出并非问题或过于琐碎的意见	第 14 章
工作中发现 Skill 已损坏	不要停下工作，也不要默默绕过；用该 Skill 专属的 PR 修复	第 13 章

- **数据的形态**……数据拥有哪些字段，以及可以处于哪些状态
- **吞吐量检查点**……考虑工作「如何并行拆分」后，在待办清单中记录的四项内容：需先完成的步骤、可并行推进的工作、共同修改的状态，以及可安全拆分的最小单元
- **询问 PR 状态的请求**……例如「check on PR X（看看 PR X 的情况）」「get it green（让所有检查通过）」
- **覆盖**……将多个范围分配给 Worker，确保检查没有遗漏的拆分方式
- **竞赛**……让多个 Worker 竞争解决同一任务的拆分方式
- **关卡**……原文为 `gauntlet`。pstack 文件未定义，本书将其定义为：针对一项变更安排多道验证，要求全部通过的拆分方式（第 24 章）
- **拆分探索**……将待搜索的地方分成多个范围，让每个 Worker 各调查一处（例如查找缺陷原因时，为每个可疑模块分配一个 Worker）
- **堆栈**……具有父子关系的一系列 PR（一个 PR 之上再叠另一个 PR）。从第二个开始，每个 PR 都以紧邻其下的 PR 的分支为合并目标；只有最底部的 PR 以 `main` 等主线分支（`trunk`）为合并目标（第 14 章）
- **Bugbot**……Cursor 提供的自动审阅 PR 功能

提问前，区分答案是事实还是判断

在表格的规则中，原文解释篇幅最长的是第一条「提问前先分类」。

Agent 在准备问用户「哪种做法更好」「应该怎么做」时，要先判断问题的答案属于以下哪一类。

- **执行后可以确定的事实**……例如行为、处理耗时、布局、输出、性能等。Agent 不询问人，而是自行查证。具体查证方式取决于请求被分配到哪个 Playbook，分为两种。
 - 「Investigation」以外的 Playbook……使用「Prototype」Playbook 尝试，并根据结果决定。
 - 「Investigation」Playbook……不制作原型，而是继续调查，根据阅读代码等取得的证据回答。
- **需由人决定的判断**……尝试也得不到答案的产品决策或偏好。Agent 只向人询问这类判断。

在「Investigation」中不制作原型，是因为它的任务是不修改代码地调查，再向用户给出附有依据的答案（第 10 章）。

用户将工作全权交付时

如果用户把工作全权交给 Agent，Agent 会在途中自行决定出现的判断，不向用户询问，继续推进；并在最终给用户的报告中说明自己作出的判断。

报告内容依判断类型分成以下两类。

- **委托范围内的判断**……例如测试放在哪里、如何拆分函数等工作中由 Agent 作出的选择。报告需说明采用了哪个选项。
- **需由人决定的判断**……尝试也得不到答案的产品决策或偏好。报告除了说明采用了哪个选项，还要写明理由，以及用户若想撤销该选择可以使用的措辞（例如「撤回」）。

不过，即使用户全权交付工作，在以下两种情况下，Agent 仍要停下，等待用户判断。

- 用户预先指定的停顿检查点（例如合并前）
- 本章「Autonomy」一节将介绍的「Always pause」清单（例如向共享分支 force-push、部署等不可撤销的操作）

「Principles」：原则索引

「Principles」是 23 项原则的索引。每一项依次列出原则名称、适用条件和原则要点。

23 项原则分为以下五类。

分类	原则	本书章节
Core	Laziness Protocol、Foundational Thinking、Redesign from First Principles、Attack the Premise、Subtract Before You Add、Minimize Reader Load、Outcome-Oriented Execution、Experience First、Exhaust the Design Space、Build the Lever	第 17 章
Architecture	Model the Domain、Boundary Discipline、Type System Discipline、Make Operations Idempotent、Migrate Callers Then Delete Legacy APIs、Separate Before Serializing Shared State	第 18 章
Verification	Prove It Works、Fix Root Causes、Sequence Work into Verifiable Units、Test Behavior, Not Implementation	第 19 章
Delegation	Guard the Context Window、Never Block on the Human	第 20 章
Meta	Encode Lessons in Structure	第 21 章

Agent 通过 /poteto-mode 开始工作时，会先读这份索引，选择适用于当前工作的原则。由于索引就在 /poteto-mode 的 SKILL.md 中，加载 /poteto-mode 时也会一并读到索引（第 9 章）。

例如，「Model the Domain」在索引中的条目如下。

原则名称后面依次是适用条件（例如编写具有状态的逻辑）和原则要点（通过结构表达领域）。

```
- **Model the Domain** (**principle-model-the-domain**). Writing stateful logic, or code that branches a lot or repeats a shape assumption across files. Encode the domain in a structure (state
```

```
machine, typed model, table or registry, reducer, boundary, the right collection) instead of
scattered conditionals.
// 编写有状态逻辑、多分支代码，或在多个文件中重复相同前提时，用结构（状态机、带类型的模型、表或注册
表、reducer、边界、合适的集合）表达领域，而非散落的条件分支。
```

Agent 决定应用某项原则时，不会只凭索引中的要点行事，还会完整阅读该原则的 Skill 文件（SKILL.md）。

这条规则写在 /poteto-mode 的 SKILL.md 的「Principles」小节开头。

```
Read the leaf skill in full for any principle you apply.

对于要应用的原则，须完整阅读其 Skill 文件。
```

同一份 SKILL.md 的「Non-negotiables」小节开头还写道：

```
Cite only principles whose leaf SKILL.md you read this session.

只能引用本次会话中读过相应 SKILL.md 的原则。
```

也就是说，Agent 受到约束，不应在回复中提及自己没有读过 Skill 文件的原则名称。

```
先只读取索引，应用原则时再读取其全文的加载方式见第 9 章；各项原则的内容见第 4
部分（第 17 章至第 21 章）。
```

「Autonomy」：可自行推进与必须停下的范围

「Autonomy」区分可不询问用户而推进的工作，以及必须停下的工作，还规定了用户征求意见时的回答方式。它由以下四段组成。

段落	规则
「Just do it.」	可不经询问自由使用 MCP 工具。可撤销的工作与对外行动（在团队聊天中发帖、更新工单、启动评估〔「Eval」，第 13 章〕）也可不经询问直接进行

段落	规则
「Always pause」	遇到不可撤销的操作（向共享分支 force-push、部署、删除数据、给客户发消息），一定要停下
「Session overrides」	如果用户说「别停」「我要睡了」「一直运行到做完」「都交给你」，就不要停下，持续推进
「No is an acceptable answer.」	在用户问「该不该这样做」、提议增加工作，或展示一种做法时，Agent 应诚实表达自己的意见。如果不赞同，可以拒绝或反驳（例如回答「这不值得加入」）。不因方案来自用户就表示同意，而应坦率回答

即使收到「Session overrides」的指示，Agent 遇到表格第二行「Always pause」所指的不可撤销操作时，仍会停下。这项例外没有写在「Session overrides」段落，而是写在「Non-negotiables」的规则中（见本章「用户将工作全权交付时」）。

「Subagents」：委派规则

「Subagents」规定主 Agent 将部分任务委派给子 Agent 时应遵守什么。它包含以下两方面。

- **委派对象**……要委派给哪个子 Agent
- **默认设置**……每次启动子 Agent 时使用的设置

先看委派对象。主 Agent 在 Playbook 步骤中启动子 Agent 时，默认使用 poteto-agent。不过，/how、/why、/interrogate、/reflect、/swarm 为了用多个模型审阅，自行决定要启动哪些子 Agent。主 Agent 不会把这些 Skill 选择的子 Agent 换成 poteto-agent。

poteto-agent.....pstack 随附的子 Agent（agents/poteto-agent.md）。开始工作前，它会完整阅读 /poteto-mode 的 SKILL.md，包括 Principle 索引。因此，它按与主 Agent 相同的 /poteto-mode 规则工作。

接下来是默认设置。无论启动哪一个子 Agent，主 Agent 每次都使用相同的设置，共有以下四项。

- **在后台运行**……主 Agent 不必等待子 Agent 完成，可以继续自己的工作。
- **以 agent mode 运行**.....agent mode 允许编辑文件和运行工具。如果以只读模式运行，子 Agent 将无法使用 MCP 工具。

- **在请求中写文件路径，而不是文件内容**……子 Agent 自行打开并阅读所给路径的文件。
- **按角色指定模型**……明确指定各角色的子 Agent 使用哪种模型。角色及默认模型见「第 34 章：角色一览」，可通过 `/setup-pstack` 修改。

Agent 在所有委派中均遵守这些默认设置。

第四项按角色指定模型，对委派编写代码还有专门规定。主 Agent 根据变更难度决定委派给哪种模型。

- **最困难的变更**（跨多个位置的设计、复杂并发处理、容易漏掉细微错误的算法）……交给判断能力最强的模型（默认 `claude-opus-5-5-max`）。无论指示模糊、需要判断，还是步骤规定得很细、只需照着执行，只要属于最困难的变更，主 Agent 都会交给这个模型。
- **琐碎的机械编辑**（例如替换名称）……交给编写代码时响应迅速的模型（默认 `grok-4.7-xhigh-fast`）。

委派模型可通过 `/setup-pstack` 写出的规则文件修改。Agent 根据委派内容读取该文件中的以下行，决定模型。

- **委派最困难的变更时**……`hardest tasks` 行
- **委派各 Playbook 的编码工作时**……相应 Playbook 的行（`feature`、`refactoring`、`bug-fix`、`perf-issue`、`hillclimb`）
- **委派文字和判断时**……`judgment and prose` 行

规则文件中没有相应行的角色继续使用默认模型。值为 `inherit-parent` 或 `auto` 的角色，使用父对话的模型（第 34 章）。

「Subagents」还规定如何处理子 Agent 返回的结果。

第 6 章所说的第二意见（将同一请求交给另一模型）的定义，也写在「Subagents」中。

「Subagents」认为，**如果另一种模型给出相同答案，就是答案可信的有力线索。**

主 Agent 有时会在子 Agent 工作过程中将其停止，再重新启动。反复这样做，子 Agent 可能会悄然遗漏最初收到的指令（上下文）的一部分。

因此，对于多次停止并重新启动的子 Agent，即使它报告「完成了」，主 Agent 也不会直接相信。这时，主 Agent 会将最初的指令及所有待办工作重新整合为一份请求，交给新的子 Agent。

「Writing the reply」：回复写法

「Writing the reply」要求 Agent 在撰写回复时遵守写作规则，而不是写完后才遵守。

Write the reply clean as you draft it. A cleanup pass after drafting does not remove these patterns.

在写回复的同时把它写好。写完后统一清理，并不能消除这些写作模式。

「Writing the reply」指出，如果等写完再统一检查，下表所列的违规写法仍会残留，难以全部改掉。

引文中的「这些写作模式」对应以下七条规则。

规则	内容
简短的陈述句	一句话只表达一件事，以句号结束
不用长破折号（em dash，—）	列表项和标题也不要破折号拼接，而应写成句子（例如不用 main.js — 保存处理，而写「main.js 负责保存处理。」）
不用句中冒号	同 unslop 的规则 14（第 32 章）；列表前的冒号可以使用
不要借口简短而删减内容	句子要简短，但 Playbook 要求写入回复的事项不能删除（例如详细经过、权衡、所选方案和尚未确定的事）
先写对使用者及下一个代码维护者的影响	先于实现细节写明这项工作服务于谁（例如应用用户、使用库的同事），以及对他们有什么变化。随后写明下一个负责此代码的工程师将接手什么
不编造链接或引用	也不编造 transcript（与 Agent 的对话记录）引用。链接仅限本次会话中创建或读过的链接。PR 链接按 <code>https://github.com/<owner>/<repo>/pull/<number></code> 的格式书写
为主张附上证据或标签	使用 measured（测量所得）、inferred（根据证据推断）、guess（猜测）。预测和未经验证的原因标为 guess。自己能执行的检查，不推给人来做

每个 Playbook 最终都以遵守这些规则的回复结束。

例如，[README](#) 中有一个关于「滚动位置漂移」的请求示例。

```
/poteto-mode this pr has a subtle bug where the scroll drifts every 750ms even when idle. repro
first, then fix and verify.
// 这个 PR 有个不易察觉的缺陷：即使处于空闲状态，滚动位置每 750ms 仍会偏移。先复现，再修复，然后验证。
```

修复「滚动位置漂移」请求中的缺陷后，用违反规则和符合规则的方式回复，分别如下。

修改前：修好了：原因是定时器，所以我把它停了；我想现在大概没问题了。

修改后：空闲时滚动位置偏移的缺陷已不再出现（measured）。

原因是每 750ms 改写位置的定时器（measured）。

下一个维护者添加定时器时，需要检查它是否会在空闲状态改写位置（inferred）。

前一种回复在句中用了冒号，一句话里塞进了三件事，而且无法判断「修好了」是测量结论还是猜测。后一种先说清应用用户得到什么改变，再说明下一个维护代码的人需要注意什么，并给每项主张加上标签。

这样写出的回复，让用户能逐句看出发生了什么变化，以及 Agent 如何确认。

「Comments」：注释写法

「Comments」要求将上一节的回复规则用于代码注释。与回复一样，**注释也要在写的时候遵守规则，而不是写完后修改。**

此外，它规定**只在注释中留下无法从代码本身看出的原因**。例如「为了规避库中的缺陷」就是值得保留的注释所解释的原因。

这项规则还对验证脚本和测试脚本作出规定。

脚本中不要写 `// Phase 1: add cards`（阶段 1：添加卡片）之类解释处理阶段的注释。应通过断言（检查条件是否成立、不成立则使脚本失败的语句）或日志输出的文字，说明正在验证什么。

下例脚本添加两张卡片并重启应用，然后检查卡片是否仍然存在。

```
// 修改前：用描述阶段的注释表示操作步骤
async function checkBefore() {
```

```
// 阶段 1: 添加卡片
await addCards(2);
// 阶段 2: 重启应用
await restartApp();
assert((await countCards()) === 2, "失敗");
}

// 修改后: 用断言字符串说明验证了什么
async function checkAfter() {
  await addCards(2);
  await restartApp();
  assert((await countCards()) === 2, "再起動しても、カードが保存されている");
}
```

Oh My Stack 项目注（代码示例）：修改前的断言消息「失敗」意为「失败」；修改后的消息表示「重启后卡片仍被保存」。两者都是断言失败时显示的原始文字。

前一种写法里，断言失败时只会显示「失败」。要知道它在检查什么，还得阅读注释。

后一种写法在失败时，从运行结果中的文字就能看出「重启后卡片仍被保存」这一点没有得到证实。

这样一来，若把步骤说明写在断言的文字中，而非注释中，说明就会留在运行结果里。

「Comments」的规则不限于测试脚本，适用于 Agent 创建的所有文件，也包括让子 Agent 编写的代码变更。

代码中的注释是否真的必要，将由审阅前运行的 `/no-comments`（第 31 章）检查。

「Playbooks」：Playbook 索引

「Playbooks」是供 Agent 选择符合请求的 Playbook 的索引。**每个条目写明 Playbook 名称、适用的工作，以及文件位置。**Agent 从索引中选出符合请求的 Playbook，打开文件，把步骤一字不改地抄入待办清单。

- **待办清单**.....Agent 开始工作时创建的任务列表（计划）。在 Cursor 中，它显示在聊天界面。用户可以通过该清单核对 Agent 执行了哪些步骤，跳过了哪些步骤（skip: <理由>）（随附指南 `01-setup.md`）。

索引之前写有 Playbook 的使用方法及路由规则。「第 7 章：请求按固定规则分配给 Playbook」已介绍过这些规则，但它们是 /poteto-mode 的核心，所以这里再列一次。

Playbook 的使用方法

Agent 按以下顺序使用 Playbook。

1. 从索引中选择符合请求的 Playbook，并打开其文件
2. 创建待办清单，将该 Playbook 的步骤一字不改地抄在清单开头。如果请求还包含 Playbook 步骤之外的工作（例如用户要求添加 --json 标志），则在抄入的步骤后追加必要工作（如「确认文本输出没有变化，哪怕一个字节也没有」）
3. 决定不执行的步骤也不删除，另起一行按 skip: <理由> 格式说明

路由规则

基本上，会选择一个符合请求类型的 Playbook。不过，如果工作量很大，路由目标就取决于工作规模，而不是请求类型。

请求性质	路由目标
符合一个 Playbook、规模普通的工作	该 Playbook
大型或跨多个位置的工作（例如涉及大量调用位置的迁移）	/figure-it-out
用户会离开，稍后再检查结果的工作	/figure-it-out
没有适用 Playbook 的工作	/figure-it-out
由一个协调者统筹许多 PR 和许多子 Agent、持续多天的工作	「Orchestrate」
希望不停顿推进到底，且一个 Agent 能在一次会话中完成的工作（例如请求「一直做完为止」）	「Autonomous run」

表格第二行和第三行的工作，即使也符合「Feature」等 Playbook，仍会交给 /figure-it-out。

/figure-it-out 是为特定工作专门设计 Playbook 的 Skill。它不会照搬固定步骤，而会针对每项工作做以下三件事。

- 决定验证需要多严格；工作规模越大、失败影响越大，要求越严
- 将工作拆成可以逐个整合的小单元，从最不确定的单元开始
- 使用 /show-me-your-work 记录每个阶段作出的判断

原文没有说明为什么将第二行的大型工作交给 /figure-it-out。本书推测，是因为需要根据工作规模决定验证严格程度和拆分方式。

第三行中，将用户暂时离开、稍后检查的工作交给 /figure-it-out，是为了让用户在工作结束后集中核对结果（随附指南 07-overnight.md）。因此，必须能事后追溯 Agent 作了什么判断、为什么这样判断。/figure-it-out 在编码前设计执行方案并留下决策记录，可以满足这一需求。

它与「Orchestrate」的区别在于：/figure-it-out 为一次工作设计执行方式，而「Orchestrate」统筹持续多天的整个工作。

23 个条目的索引

若按本书所用的工作类型划分，索引中的 23 个条目如下。

分类	Playbook	本书章节
调查	「Investigation」、「Runtime forensics」、「Trace forensics」	第 10 章
修复、改进	「Bug fix」、「Perf issue」、「Hillclimb」	第 11 章
创建	「Feature」、「Refactoring」、「Prototype」、「Visual parity」	第 12 章
处理 Skill	「Authoring or modifying a skill」、「Eval」	第 13 章
交付	「Opening a PR」、「Babysit」、「Shipping」	第 14 章

分类	Playbook	本书章节
长期运行	「Autonomous run」、 「Session pickup」、「Pause safely」、「Worktree and simulator cleanup」	第 15 章
运作大量 PR	「Multi-phase or multi-PR plan」、「Orchestrate」、 「Autopilot-full」、「Autopilot-stack」	第 16 章

调整成自己的工作方式

/poteto-mode 体现 poteto 的工作方式，不一定原样适合每位用户。

如果不适合，就需要用 /automate-me（第 25 章）创建自己的模式（像 /poteto-mode 一样汇集个人工作方式的 Skill），或 fork pstack（将代码库复制到自己的账号下）。

/automate-me 从近期的对话记录中，找出用户在回复、委派、验证、代码和文字等方面反复表现出的工作方式，并起草一项名为 <your-name>-mode（例如 tomato-mode）的 Skill。

使用 /automate-me，用户可以继续以 pstack 为基础，同时在 /poteto-mode 之外拥有一项属于自己的请求路由 Skill。

如果想修改 Playbook 或 Principle 本身，本书建议 fork pstack，维护自己的版本。

README 开头写道「fork it. improve it. make it yours.」（fork 它，改进它，让它成为自己的版本），欢迎用户 fork 和改进。

小结

- /poteto-mode 的职责……它是 pstack 的入口（Router），将请求分配给 Playbook。SKILL.md 也规定了分配之后适用于整个工作的规则。
- 「Non-negotiables」……按情形列出 Agent 必须执行的事项。例如，提问前先判断答案是执行后可确定的事实，还是需由人决定的判断。
- 「Principles」和「Playbooks」……23 项原则和 23 个 Playbook 的索引。Agent 从索引中选择适用的条目，再打开对应文件阅读。

- **「Autonomy」和「Subagents」**Agent 在不可撤销的操作前一定停下，并在每次委派时遵守默认设置（主 Agent 每次启动子 Agent 所使用的设置）。
- **「Writing the reply」和「Comments」**Agent 在撰写回复和注释的过程中遵守规则，而不是等写完再修改。
- **调整成自己的工作方式**使用 /automate-me 创建自己的模式，或 fork pstack。

至此，第 5 部分结束。本书迄今已介绍了 pstack 的所有 Playbook、Principle 和 Skill。第 6 部分将说明在自己的项目中，应按什么顺序开始使用这些组成部分。第一篇第 36 章将介绍验证 Skill，让 Agent 能运行自己的应用并确认变更。

第六部分：pstack 实践指南

kaito • 来源：日本語 • English • 2026-10-03

第六部分是 pstack 的实践指南，基于 pstack 作者 poteto 在 X 上发布文章《The Complete Guide to pstack》的 [Part 1](#) 和 [Part 2](#)。

这一部分讨论：在自己的项目中使用 pstack 时，应该从哪里开始，按什么顺序推进。

第六部分按以下顺序讲解文章内容。

1. 创建验证 Skill (Part 1、[第 36 章](#))
2. 让 Agent 理解问题，并接续过去的工作 (Part 2、[第 37 章](#))
3. 从用户的使用方式出发思考设计，并用原型试验 (Part 2、[第 38 章](#))
4. 确定设计，并转化为可执行的计划 (Part 2、[第 39 章](#))

Part 1 提出，使用者首先应该创建**验证 Skill**，并介绍其创建和使用方法。

Part 2 介绍在验证 Skill 准备好之后，如何与 Agent 一起决定要做什么。本书将 Part 2 的内容整理为 13 个要点，分为第 37~39 章讲解。

验证 Skill (verification skill) 项目专用的 Skill，让 Agent 能够启动、操作使用者的应用，确认修改确实生效（例如：修改设置页面后，Agent 点击齿轮按钮打开设置页面，并留下修改位置的截图作为证据）。

之所以先创建验证 Skill，是因为**如果 Agent 无法验证自己的修改，原型的好坏、计划是否完成，最终仍要由人来确认。**

这样一来，在人确认完毕之前，Agent 的工作就无法继续推进，人仍然是阻碍工作流转的瓶颈，一整天都要用来盯着 Agent。

第六部分将介绍什么

阅读第六部分后，你将了解以下内容。

- 如何为自己的应用创建验证 Skill，并随着应用变化保持更新
- 如何让 Agent 像人一样理解问题，并接续过去的上下文
- 如何从用户的使用方式开始设计，并从多个方案的原型中选择一个
- 如何根据实现过程中了解到的情况重新审视设计，并将其转化为拆分成小规模、可验证 PR 的计划

本部分章节

各章内容如下。

章节	内容	对应文章
第 36 章 指南 1：从验证 Skill 开始——运行自己的应用进行验证	创建让 Agent 能验证自身修改的验证 Skill	Part 1
第 37 章 指南 2：传达目的与背景，让 Agent 理解问题并接续过去的工作	理解问题	Part 2（要点 1~4）
第 38 章 指南 2：从用户的使用方式出发试验设计	试验解决方案	Part 2（要点 5~8）
第 39 章 指南 2：重新审视设计，转化为可执行的计划	确定解决方案，并转化为可执行的计划	Part 2（要点 9~13）

后面的章节以前面的章节为基础，因此建议从第 36 章开始按顺序阅读。

第 36 章：指南一，从验证 Skill 开始，运行并检查自己的应用

kaito · 来源：日本語 · English · 2026-10-03

poteto 在《The Complete Guide to pstack》Part 1（本章称「文章」）中，把验证 Skill 列为 pstack 新用户首先应制作的东西。

验证 Skill（verification skill）是项目专用的 Skill，让 Agent 启动并操作用户开发的应用，确认改动确实有效并留下证据（见第 26 章）。

例如，改动设置页面后，Agent 应亲自点击齿轮按钮打开设置页面，留下改动位置的截图作为证据。

本章结合 /create-verification-skill 的 SKILL.md 等 pstack 原文，介绍文章推荐的验证 Skill 制作与使用方法。

/create-verification-skill 和 /maintain-verification-skill 的 SKILL.md 所规定的步骤，已在第 26 章介绍。

本章省略以下内容，重点介绍文章提出的建议。

- **制作前调查**……询问用户之前，Agent 先从仓库调查用户操作的对象、启动和操作方式、可保留的证据，以及能否并排运行两个应用实例。
- **SKILL.md 的内容**……Agent 根据调查所得事实，编写启动、诊断、操作、证据、清理和辅助脚本六个小节。证据应来自实际操作用户路径，并验证写入文件等副作用，而不只看界面。
- **交付前运行**……Agent 应将生成的验证 Skill 从启动到清理完整运行一次，确认能运行后再交给用户。一次都未运行的视为草稿。
- **检查所有入口**……如果 **Feature Map** 为一项功能列出按钮、快捷键、命令等多个入口，Agent 必须验证所有入口后才能宣称该功能已验证。

- **维护规则**……运行 `/maintain-verification-skill` 后，Agent 应以 `clean`（无需修改）、`changed`（将修改汇总到一个 PR）或 `blocked`（未能完成检查）之一报告结果。维护期间不编辑产品代码。

本章结构

本章结构如下。

- 为什么应首先制作验证 Skill
- 验证 Skill 由什么组成
- 如何制作和维护验证 Skill
- 如何在请求中使用并扩展制作好的验证 Skill
- 第一步应从哪里着手
- 总结

为什么应首先制作验证 Skill

首先制作验证 Skill 有两个原因。

第一，验证 Skill 是**提升整个团队成果的重要基础设施**。

第二，许多 pstack Playbook 的步骤，若没有**实际操作应用的手段**，就无法执行。

文章的定位：重要基础设施

文章的「Verification is all you need」一节以以下一句话开头。

The most critical skill to have in your toolbox is a high quality verification skill.

工具箱中最关键的是高质量的验证 Skill。

文章所说的验证（verification），是指**Agent 能亲自检查自己的工作**。

也就是说，验证并非「写了代码」或「构建通过」，而是像真实用户一样操作实际应用，并保留结果证据。

能验证自身工作的 Agent，在失败后可以修复并重试，直到成功。人无须逐项确认，也就不会成为工作流程的瓶颈。

文章认为，准备并持续维护高质量验证 Skill 很重要，应把它看作重要基础设施，而非「普通 Skill」；本书也采用这一看法。

文章接着指出，高质量的验证 Skill 能提升包括非工程师在内的整个团队的成果。

如果 Agent 能亲自验证改动是否有效，组织里的所有人，包括非工程师，都能修改应用并确认改动实际可用。文章称，制作得当的验证 Skill 可使团队成果提高 100~1,000 倍；因此可以说每个应用都应配备一个。

Playbook 的前提：操作应用的手段

许多 Playbook 要求 Agent 实际操作用户接触的界面、CLI 或 API。

此时可使用 control 类 Skill：例如 cursor-team-kit 插件的 control-ui、control-cli，它们是操作应用的 Skill（见第 8 章）。通过 `/create-verification-skill` 创建的验证 Skill 也属于 control 类 Skill。

下表概括主要 Playbook 要求 Agent 对应用执行哪些操作。

Playbook	要求 Agent 执行的应用操作
「Bug fix」（缺陷修复）	Agent 亲自复现缺陷
「Perf issue」（性能问题修复）	取得修复前的基准 trace
「Runtime forensics」（运行中进程的诊断）	从运行中的进程实际取得 profile 或 trace，而非靠推测
「Prototype」（用一次性原型决定设计）	决定视觉设计时，分别截图并操作各方案，进行比较
「Visual parity」（使两种实现或迁移前后的外观逐像素一致）	逐组件比较与迁移前基准画面的图片差异；差异不为零即失败
「Shipping」（验证并合并 PR）	对每个 PR，由未编写代码的另一 Agent 实际操作改动前后的界面或 CLI，给出 PASS、PASS+NOTES 或 FAIL 三档判定

- **trace**……按时间记录各项处理的耗时等信息。
- **profile**……按函数等维度汇总 CPU 时间或内存用量的记录。

没有验证 Skill 或 control 类 Skill 等操作应用的手段，Agent 就无法执行表中的步骤。

Agent 开始工作时，会制作一份照 Playbook 步骤编写的 TODO 列表；跳过某一步时，应在该行记下 skip: <理由>（见第 7 章）。

TODO 列表.....Agent 开始工作时制作的任务清单（计划），在 Cursor 的聊天界面显示。用户可查看清单，确认 Agent 完成了哪些步骤、跳过了哪些步骤（skip: <理由>）（随附指南 01-setup.md）。

例如在「**Bug fix**」中，如果没有操作手段，或手段无法到达需要复现的位置，本书认为 Agent 只有以下两种选择。

- 在 TODO 列表留下 skip: <理由> 并跳过该步骤
- 根据「**Bug fix**」允许的例外，请用户复现

只有同时满足以下两个条件，才能请用户复现。

- 具体说明为什么现有操作手段无法到达需复现的位置（例如，该页面仅特定订阅账户可登录）
- Agent 已亲自操作所有现有手段能够到达的部分

无论跳过步骤还是请用户复现，都会留下 Agent 无法亲自验证的部分；这些部分须由人验证。

验证 Skill 由什么组成

验证 Skill 的核心有两部分：**用于操作应用的小型 CLI，以及按功能记录入口和访问方式（功能位于哪个页面或命令、用户怎样操作才能到达）的 Feature Map。**

在本书所依据的 pstack 0.15.5 版本中，Agent 按 /create-verification-skill 的步骤，将两者都创建在 .cursor/skills/verify-<app>/ 下。

若以文章举例的虚构应用 Atlas 为例，完成后结构如下。

```
.cursor/skills/verify-atlas/
├── SKILL.md（从启动到清理的步骤）
├── control-atlas.mjs（操作应用的 CLI）
├── features/（Feature Map）
│   └── README.md（Feature Map 目录，链接到各功能文件）
```

```

├─ <功能A>.md
└─ <功能B>.md

```

SKILL.md 的内容已在第 26 章介绍；这里重点看文章详细讨论的 **CLI** 和 **Feature Map**。

CLI

验证 Skill 的 CLI 将应用操作和调试变成 Agent 可以**重复执行的命令**。

「Build the Lever」原则的应用

文章把**将应用操作做成 CLI** 视为「**Build the Lever**」（见第 17 章）的应用：**制作执行工作的工具，代替手工操作**。

制作 Skill 时应用这条 Principle，就会同时交给 Agent 可执行的工具，而非只有 Markdown 步骤说明。

What this means in the context of creating a skill, is that we prefer to give agents tools rather than just markdown.

在制作 Skill 时，这意味着我们倾向于提供工具，而不只是 Markdown。

例如，如果验证 Skill 的 SKILL.md 只写「点击设置按钮并截图」，Agent 每次都得自己编写点击脚本。

若验证 Skill 附带拥有 click、screenshot 等命令的 CLI，Agent 就只需执行命令。这里的工具，正是汇集应用操作与调试功能的小型 CLI。

文章列出把应用操作做成 CLI 的以下好处。

- **减少 token 消耗**……Agent 无须为单次按钮点击编写一次性脚本，只需运行一条 CLI 命令。
- **易于复现和测试**……验证 Skill 每次运行都得到相同结果；命令形式固定，也便于测试 Skill 本身是否正常工作。

示例：虚构应用 Atlas 的命令

以下是文章给虚构应用 Atlas 的部分命令示例。设定中，Atlas 是以 Electron（构建桌面应用的框架）开发的应用。

Agent 按顺序执行以下命令，完成三件事。

1. 用 doctor 确认应用可操作
2. 用 new-session 打开空白新对话（线程），再用 send 发送「list open tasks in this project」（列出本项目未完成的任务）
3. 用 screenshot 将结果画面保存到 /tmp/atlas-proof.png 作为证据

```
# health
node .cursor/skills/verify-atlas/control-atlas.mjs doctor

# open a blank thread and send
node .cursor/skills/verify-atlas/control-atlas.mjs new-session
node .cursor/skills/verify-atlas/control-atlas.mjs send "list open tasks in this project"

# screenshot for evidence
node .cursor/skills/verify-atlas/control-atlas.mjs screenshot /tmp/atlas-proof.png
```

六类命令

准备命令时，可参考以下六类。

- **观察（Inspection）** ……info、snapshot、screenshot 等查看界面状态的命令。
- **导航（Navigation）** ……home、new-session、scroll 等切换界面的命令。
- **交互（Interaction）** ……send、click、type、press 等输入或点击命令。
- **性能（Performance）** ……trace、profile、perf-metrics 等测量性能的命令。
- **获取记录（Streaming）** ……console、network-log 等获取控制台或通信记录的命令。
- **诊断与清理（Health & cleanup）** ……doctor、cleanup 等确认应用是否可操作或执行清理的命令。

便于 Agent 使用的 CLI

文章建议，在尝试更高级的用法前，先花时间把 CLI 做到无错误运行。**CLI 做好后，所有 Agent 都能用它操作和调试应用。**

CLI 的使用者是 Agent 而非人，因此应便于 Agent 使用。

文章列出以下六项便于 Agent 使用的 CLI 特性。

- 命令之间易于组合使用（文章引用 John Ousterhout 《[A Philosophy of Software Design](#)》中的 deep modules 思路，即简单接口封装丰富功能）
- 可能造成破坏性副作用的命令提供 `--dry-run`（只预示会发生什么，不实际执行）
- 不一次展示所有功能，而是通过子命令逐层展开；例如总览 `--help` 只列子命令，详细用法放在各子命令的 `--help` 中
- 错误信息写明下一步应做什么；例如不止说「应用未启动」，还说明「请先运行启动命令」
- 提供充分的 `--help` 信息
- 以 JSON 等机器可读格式输出

文章建议用户亲自设计，或请 Agent 设计这样的 CLI。

Feature Map

Feature Map 是一组按功能划分的 Markdown 文件，记录**各功能的作用、用户怎样操作才能到达，以及验证 Skill 如何操作、出现什么结果才算正常**（见第 26 章）。

应用越复杂，Agent 越需要指引才能找到并操作功能。poteto 为此提出 Feature Map。

`features/README.md` 是 Feature Map 的目录，链接到各功能文件。

操作应用前，Agent 先读目录，只打开与当前工作有关的功能文件。

只读取必要文件，可以节省 Agent 上下文窗口的 token。

文章称 `/create-verification-skill` 会创建 `references/features` 目录，而 pstack 0.15.5 版的 `SKILL.md` 要求创建在 `.cursor/skills/verify-<app>/features/`；两者位置不同。本书采用 pstack 0.15.5 版规定的位置。形式可参考随附的 [references/feature-map-example/](#)。

Agent 按 `/create-verification-skill` 步骤，用以下四个标题编写每个功能文件。

标题	要回答的问题
Sub-features	用户能在该功能中执行哪些操作（逐项分配简短 ID）
How to get to it (user POV)	用户如何操作才能到达该功能
Driving it with <harness>	如何使用 harness（从外部操作应用的机制，如验证 Skill 的 CLI）执行操作
Gotchas	哪些陷阱会让验证白费或无效

下面用 `/create-verification-skill` 附带的示例 `create-note.md`（笔记应用 Notes 的「创建笔记」功能），逐行说明表格内容。

Sub-features

示例将用户创建笔记时可执行的操作分为以下四项。

- 打开编辑界面
- 保存
- 丢弃草稿
- 通过 CLI 创建

```
## Sub-features

- `create-open` opens a blank editor from each browser entry point.
- `create-save` persists a title and body.
- `create-cancel` discards an unfinished browser draft.
- `create-cli` creates the same note shape from the terminal.
```

How to get to it (user POV)

入口指用户到达功能的方式，包括按钮、键盘操作或命令。示例列出工具栏的 New note 按钮、n 键和 notes create 命令三个入口。

```
## How to get to it (user POV)
```

- Choose the `New note` button in the browser toolbar.
- Press `n` in the browser while focus is outside an editable field.
- Run `notes create --title <title> --body <body>` in a terminal.

Driving it with <harness>

示例的 README.md 规定，各功能文件中这一标题应按以下方式编写。

Driving it with <harness> starts with Preconditions: and uses labeled bullets that pair each user action with an exact command and observable result.

Driving it with <harness> 应以 Preconditions: 开头，并使用带标签的列表，为每项用户操作配对准确命令和可观察结果。

前提条件（Preconditions:）指开始步骤前必须满足的状态。例如，示例要求「Notes 正常运行于 http://127.0.0.1:4173」「还没有标题为 Release checklist 的笔记」。

前提条件之后逐项列出步骤。根据规则，每项都应写齐以下三部分。

- 用户操作
- 用 harness 复现该操作的命令
- 能证明操作成功的可观察结果

```
- **Open editor.**
  Choose `New note`.
  Run `control-notes browser click --role button --name "New note"`.
  A form named `Note editor` appears with focus in the `Title` textbox.
```

将上例拆开，得到下表。

要素	示例写法	含义
用户操作	Choose New note.	选择 New note 按钮
用 harness 复现的命令	control-notes browser click --role button --name "New note"	点击角色为按钮、名称为 New note 的控件
可观察结果	A form named Note editor appears with focus in the Title textbox.	名为 Note editor 的表单打开，光标进入 Title 输入框

Agent 运行命令后，检查是否出现表中的「可观察结果」；若没有，就知道这一步失败了。

Gotchas

示例记录了以下陷阱。

- Pressing `n` while a textbox has focus types the character instead of opening a new editor.
- A save status alone is insufficient proof. Reopen the note from the list.

第一项是：输入框获得焦点时，按 n 不会打开编辑器，只会输入字符 n。

第二项是：仅出现「已保存」状态，不足以证明数值真的保存成功；应从列表重新打开笔记验证。

文章中的 Feature Map 示例（设置页面）也在 Gotchas 写了以下注意。这里的 tab 指设置页面左侧的切换项，如 General、Appearance、Models、Plan & Usage。

Some tabs are entitlement-gated. Skip with an explicit account reason.

一些 tab 受账户权限限制。跳过时必须明确说明账户原因。

Feature Map 写明这一点后，Agent 找不到 Plan & Usage 时，就不会误判为功能故障；可以说「账户订阅状态使该 tab 不显示」，再跳过该 tab 的验证。

团队共享的记忆

文章称 Feature Map 为「具象化的记忆（materialized memory）」（见第 4 章）。

Agent 的记忆，是把过去工作获得的知识保留下来，供后续工作使用。

这种记忆的最终形态是代码库。代码呈现团队既有的判断，是实际发生过什么、系统现在怎样运行的权威来源。

Feature Map 将代码库承载的记忆精简整理，以节省 Agent 上下文窗口的 token。

此外，**Feature Map 是仓库 .cursor/skills/ 下验证 Skill 的 Markdown 文件，因此所有参与代码库工作的人都能共享这份记忆。**

如何制作并维护验证 Skill

具体制作步骤已在第 26 章介绍：Agent 按 `/create-verification-skill` 执行。这里讨论文章给用户的建议，即制作前的准备及制作后的维护频率。

制作前用户应做好什么准备

如何选择技术栈

文章指出，若要提供从外部操作应用的方式，技术栈的选择也很重要。**技术栈越难调试或操作，就越难让 Agent 高效工作。**

例如，对于 Electron（桌面应用框架）或 Web 应用，Agent 可以使用 CDP（Chrome DevTools Protocol，让外部程序使用浏览器开发者工具能力的机制）；对于 iOS 应用，则可以使用模拟器操作。

如果所选技术栈没有 Agent 可用的调试工具，就只能请 Agent 自行制作工具（例如使用 lldb，或在开发环境中制作与应用一同运行的专用包），或者将就使用现有工具。

但 poteto 非常重视 Agent 验证，因此建议自行构建调试工具，甚至换用其他技术栈。

I personally feel that agentic verification is so important that I would unironically suggest building your own rich debugging tools, or even choosing a different tech stack

我个人觉得 Agent 验证极其重要，所以会认真建议自行打造丰富的调试工具，甚至选择另一种技术栈。

我个人也认为验证极其重要且有益，因此若所选技术栈没有验证工具，我会采纳 poteto 的建议。

准备开发体验

文章建议用户在准备运行应用时，先思考以下三个与开发体验相关的问题。

- 如何在开发数据库中加入种子数据（预先放入以供开发和测试的数据）
- 如何准备认证与测试用户，以及如何调用测试或预发布环境的 API

- 如何按一致的步骤安装并启动开发环境

CLI 和上述准备都是 Agent 开发应用的主要工具，因此需要维护和测试。

准备就绪后，用户运行 `/create-verification-skill`。文章还推荐一个供参考的公开示例：[poteto/verification-skill-example](#)，它是为虚构应用制作的验证 Skill。

用 `/maintain-verification-skill` 保持更新

维护结果的三种类型（clean、changed、blocked），以及维护时不编辑产品代码的规则，已在第 26 章介绍。这里讨论文章建议的运行频率，以及如何将其机制化。

运行频率

`/maintain-verification-skill` 的 `SKILL.md` 没有规定运行频率。执行 `/create-verification-skill` 的 Agent，也只在用户询问时才建议频率。因此，本书认为**用户应根据应用变化速度决定运行频率**。

文章建议**至少每天运行一次**。

Agent 在开发应用时也可能更新验证 Skill，但经常会遗忘一些位置。

用户每天运行一次 `/maintain-verification-skill`，就能发现并修正遗漏更新的位置，使 Agent 持续掌握应用的最新操作方式。

用 Dr Eggbot 把每日运行机制化

文章也介绍了 Dr Eggbot，作为实现每日运行的方式。

Dr Eggbot 是 poteto（Lauren Tan）制作的 Bot，用于设计并制作 Grok Bot（AI Bot 应用，见第 5 章）。xAI 的[Bot 市场](#)将 Dr Eggbot 描述如下。

Asks a few preference questions, then creates them with CreateAgent. Coding bots get the poteto-mode bar (one job, unslopped, verified).

先问几个偏好问题，再用 CreateAgent 制作 Bot。编程 Bot 会达到 poteto-mode 的标准（只做一项工作、去除 AI 腔、验证结果）。

Dr Eggbot 随 pstack 一起提供。

因此，**Dr Eggbot 能教编程 Bot 如何使用 pstack；不编程的 Bot 也能按同样严谨的标准制作。**

推荐按以下步骤使用 Dr Eggbot。

1. 请 Dr Eggbot 制作工程师角色的 Bot
2. 请制作好的 Bot 运行 `/create-verification-skill`
3. 请同一个 Bot 设置每天运行一次 `/maintain-verification-skill` 的例行任务

如何在请求中使用并扩展已制作的验证 Skill

用户应在给 Agent 的请求中写明使用哪种验证 Skill，以及采用什么证据形式。

用户应在满意于验证 Skill 之后，才将它加入 Grok Bot 的例行任务或 Cursor Automations（Cursor 中由固定时间或事件触发任务的功能），见本章「[用法 3：自动复现用户报告](#)」。

验证 Skill 的用法

文章举出三种用法。下面示例中的 `/control-app`，是文章给通过 `/create-verification-skill` 制作的验证 Skill 所用的名称。

用法 1：制作新功能

制作新功能时，`poteto` 将 `/poteto-mode` 与验证 Skill 一同使用，让 Agent 验证自身改动。文章请求示例如下。

```
/poteto-mode build <description of feature, any useful context>. use /control-app to verify your changes and show me a video and screenshots as proof
// 创建<功能说明和有用的上下文>，用 /control-app 验证改动，并展示视频和截图作为证据。
```

这个请求在功能说明之外，还明确了使用哪种验证 Skill（`/control-app`），以及证据形式（视频和截图）。

如果通过 Grok Bot 发出请求，`poteto` 不让 Bot 自己执行工作，而是让它启动 Cloud Agents（Cursor 在云端运行 Agent 的功能）来做。以下就是示例请求。

```
spawn a cloud agent to use /poteto-mode to build <description of feature, any useful context>. use /
control-app to verify your changes and show me a video and screenshots as proof
// 启动云端 Agent，让它用 /poteto-mode 创建<功能说明和有用的上下文>；用 /control-app 验证改动，并展
示视频和截图作为证据。
```

poteto 不让 Bot 亲自做，是因为将工作交给云端 Agent 后，Bot 可以继续处理其他工作，其上下文窗口也不会被执行记录填满。

用法 2：改善性能

改善性能时，poteto 要求 Agent 先通过验证 Skill 测量改动前的性能，再用同一 Skill 验证改动效果。文章的请求示例如下。

```
spawn a cloud agent to use /poteto-mode to improve the initial loading time of our app. first use /
control-app to take a trace of the status quo, and identify opportunities for improvement. then do a
targeted fix and use /control-app + a /swarm to confirm the win
// 启动云端 Agent，让它通过 /poteto-mode 改善应用首次加载时间。先用 /control-app 采集现状追踪并寻找
改进空间，再有针对性地修复，最后用 /control-app 和 /swarm 验证改进。
```

该请求明确要求修复前用 /control-app 取得现状 trace。「Perf issue」Playbook 也要求第一步取得改动前的基准 trace，并在报告中写出改动前后的数值及差异。

文章认为，让大量 Worker（并行运行的多个 Agent）工作并汇总成一份报告的 /swarm（见第 24 章），是最适合与验证 Skill 搭配的 Skill 之一。

让大量 Cloud Agents 通过 /swarm 执行验证 Skill，可用充分样本验证性能改善，或通过 fuzzing（大量输入随机值或意外值，检查有无异常）验证应用未被破坏。

用法 3：自动复现用户报告

满意于验证 Skill 后，用户就可以把它加入 Grok Bot 例行任务或 Cursor Automations；两者都能按固定时间或事件触发处理。

文章的例子中，Bot 逐条接收来自 Slack 等渠道的用户反馈，启动云端 Agent，自动尝试复现报告的缺陷。

如果验证 Skill 和 Feature Map 足够好，甚至可以将缺陷修复也自动化。

并行运行 Agent

借助验证 Skill 成功合并几个 PR 后，下一步是让更多 Agent 并行工作。

如果 Agent 接到请求后能将工作推进到几乎可以合并，用户就能在此期间运行其他 Agent。

使用 Cloud Agents，而非 worktree

文章推荐使用 Cloud Agents 并行，而非 worktree（Git 功能，可为同一仓库增加工作目录，见第 18 章）；本书也持同样建议。

文章不推荐 worktree，是因为每个 Agent 都需要额外的工作目录，消耗本地机器的存储和资源。

根据仓库大小和机器性能，估计本地 worktree 并行最多可运行约十个 Agent。

依我自己的经验，本地并行还得为各 worktree 分开目录，避免应用端口冲突；创建后的管理和清理遗留 worktree 也费事。

Cloud Agents 则可使用云端实际计算机，安装依赖、运行应用、录制视频和截图，并像用户一样操作应用。

Cursor 在首次构建后保存环境快照（某一时间点的完整状态），后续 Cloud Agents 因此可迅速启动。每个 Agent 都运行于单独机器，应用实例（每个运行中的应用）也从一开始就相互隔离。

<https://x.com/poteto/status/2087977008253071816>

poteto 也在 X 的帖子中说明采用 Cloud Agents 的原因。

worktrees are dead. cloud agents are the future. they've allowed me to massively orchestrate swarms of agents, all with their own computer. it's how i've been able to trust that my agents are actually doing what i want them to, because they can run their code, take videos and screenshots, and interact with UIs the same way users do.

worktree 的时代结束了；未来属于 Cloud Agents。它们让各有自己计算机的大规模 Agent 群成为可能。我之所以能够信任 Agent 确实按我的意图工作，是因为它们能运行代码、拍视频与截图，并像用户一样操作界面。

不过，Cloud Agents 也有注意事项。

Cloud Agents 自动运行命令，无须等待用户许可。即使命令输错，受损的也只是云端机器，而非本地电脑。

另一方面，仓库代码会传到云端机器，仍有被带到外部的风险。

秘密信息（如 API 密钥）也面临同样风险。

Cursor 文档也指出，prompt injection（在 Agent 阅读的内容中混入恶意指令）可能导致数据外泄：

自动执行存在数据外泄风险。攻击者可能通过 prompt injection 欺骗 Agent，把代码上传至恶意网站。

如果开发环境已准备妥当，设置云端环境便不太费事。

在 Cursor 中，可将开发环境的创建方式定义为代码（`.cursor/environment.json`，EAC: Environment as Code）并保存在仓库中。

把为验证 Skill 整理的开发环境启动步骤写入该文件，各 Cloud Agents 就能从同样环境开始工作。Cursor 在**博客**中这样描述：

开发环境本身也是产品，使用它的是 Agent。

顺带一提，Cloud Agents 在云端机器运行，即使用户合上电脑，工作也不会停。我自己也是 Cloud Agents 的重度用户。

<https://zenn.dev/sc30gsw/articles/953334f11df507>

第一步应从哪里着手

第一步是用户确认，**Agent 是否拥有从外部操作和观察自己应用的手段**。

本书按照文章脉络，将包括 `/create-verification-skill` 和 `/maintain-verification-skill` 在内的用户步骤整理如下。

1. 检查能否从外部操作和观察应用（现有 harness、CDP、模拟器或自制工具）

2. 确保开发环境可按一致步骤启动（种子数据、测试用户、测试环境和隔离）
3. 运行 `/create-verification-skill`，让 Agent 生成 CLI 与 Feature Map
4. 将生成成果从头到尾运行一次，确认留下证据；修复 CLI 使其无错误运行
5. 决定 `/maintain-verification-skill` 的运行频率
6. 在日常请求中写明验证 Skill 名称和证据形式
7. 先用验证 Skill 成功合并几个 PR，并对 Skill 感到满意，再扩展至 Cloud Agents 并行、例行任务和自动化

文章最后还建议持续改进 CLI、像投资重要基础设施一样投资验证 Skill，甚至设置轮值（oncall rotation，即轮流指定验证 Skill 损坏时的处理负责人）。

我建议让 Grok Bot 承担这项轮值。

设想是让 Grok Bot 持续审计，再把实际工作交给 Cloud Agents。

总结

- **为何首先制作**……文章将验证 Skill 视为应用开发最重要的 Skill，pstack 的许多 Playbook 也以能够操作应用为前提。
- **由什么组成**……核心是可重复执行的 CLI，以及汇总各功能入口和访问路径的 Feature Map。
- **如何制作和维护**……用户在制作前准备技术栈与开发环境；制作后按文章建议至少每天运行一次 `/maintain-verification-skill`，也可用 Dr Eggbot 的例行任务机制化。
- **如何使用和扩展**……用户在请求中写明验证 Skill 名称与证据形式，满意后再扩展到并行与自动化。
- **第一步**……确认 Agent 能否从外部操作和观察应用。

从下一章第 37 章起，本书依据《The Complete Guide to pstack》Part 2，介绍如何与 Agent 一起决定做什么：解决什么问题、采用什么设计、怎样规划。第 37 章会先讲本书归纳的 Part 2 十三项要点中的前四项。

第 37 章：指南二，说明目标与背景，对齐问题和历史上下文

kaito · 来源：日本語 · English · 2026-10-03

第36章介绍了为何要先建立验证 Skill，以及如何建立、维护和使用它。

验证 Skill……项目专用的 Skill，让 Agent 能启动和操作应用，并收集证据。
有了验证 Skill，Agent 就能验证自己的改动。

从本章到第39章的三章，将讲解如何与 Agent 一起决定要做什么：解决哪个问题、采用什么设计，以及如何规划。

这三章都以《The Complete Guide to pstack》的 Part 2 为基础（以下三章称之为“文章”）。

Part 2 介绍了 Agent 能验证自己的工作之后，poteto 如何借助 pstack 进行调查、规划、制作原型和架构设计。

本书将文章归纳为 13 个要点。本章讨论其中的要点 1 至 4。

决定要做什么时，人首先应该做的是**让 Agent 像人一样理解问题是什么，并承接过去工作的上下文**。

过去工作的上下文，包括先前对话中调查过的事、做过的判断以及失败过的尝试等。

之所以要让 Agent 与人对问题有相同的理解，是因为如果双方的理解仍有偏差，**即使 Agent 代码写得再好，也会解决人本来无意解决的问题**。

要点 3 和要点 4 使用的 /how、/why、/teach、/recall 的步骤和输出，已在第22章介绍。

本章略去以下内容，重点讨论文章推荐的用法。

- /how 的调查方式……面对复杂问题，/how 将收集事实和撰写说明分给不同角色，并以从 Overview 到 Gotchas 的五个标题组织说明。

- **/why 的调查方式**……/why 按 git、工单、聊天等证据所在位置，并行启动调查 Agent。然后将找到的每项原因按从 Direct（直接）到 Unknown（未知）的五级可信度分类。
- **/teach 的说明方式**……/teach 确定少数几个值得带走的要点，用一两句话回答，再等待对方回应。
- **/recall 的输出**……/recall 用 Capsule、Threads、Problems、Next move 四个标题总结工作状态。

本章结构

本章内容如下。

- 13 个要点，以及要点 1 至 4 之间的联系
- 要点 1：说明目的和背景，不过早限定解决方法
- 要点 2：实施前让 Agent 复述问题
- 要点 3：通过 /how、/why、/teach 理解机制与原因
- 要点 4：通过 /recall 将过去的工作带入新对话
- 小结

13 个要点，以及要点 1 至 4 之间的联系

这 13 个要点是本书对文章的归纳。

三章都按要点分节，每个要点有以下两个部分。

- **文章的观点**……文章建议什么
- **pstack**……pstack 的哪个 Skill 或 Playbook 实现了这一观点

如果某个要点在 pstack 中没有直接对应的内容，就省略 pstack 部分。

13 个要点如下。

No.	Part 2 中对应的章节	概要
1	『The art of supervising someone smarter than you』 『In your own words』	人向 Agent 充分说明目的和背景，不把具体解决方案定得过死，为 Agent 寻找人未想到的方法留下空间
2	『In your own words』	人让 Agent 在实施前用自己的话说明问题，从而尽早发现 Agent 的误解，也避免用自己的假设限制 Agent 的调查方向
3	『Building up a mental model』	人让 Agent 用 /how 调查当前机制，用 /why 调查设计背景，再通过 /teach 得到人能理解的说明。为解释而进行的调查，也有助于 Agent 自己理解
4	『Learning from history』	人让 Agent 用 /recall 回顾过去对话中相关的工作，把先前的调查、判断和失败带入新工作，使 Agent 不只根据眼前的症状考虑解决方法
5	『Working backwards』	人让 Agent 在实施前编写 README 或教程，从用户如何使用出发，思考 API 和内部设计
6	『Working backwards』	Agent 用 /technical-writing 按教程、操作指南、参考资料和解释等用途整理文档，并通过其内部使用的 /unslop 去掉 AI 容易写出的不自然措辞
7	『Measure a hundred times, cut once』	人不直接采纳 Agent 的第一个设计，而是让 Agent 制作多个方案的原型，比较实际操作时的截图、测得的时间或布局
8	『Measure a hundred times, cut once』 『Architecting bigger changes』	对于能通过做出小实验并运行来回答的疑问，人让 Agent 自己验证。通过原型和验证减少不确定性，而不是反复审查抽象计划
9	『Architecting bigger changes』	面对较大的改动，人把时间用于思考数据如何存放以及系统之间如何协作。Agent 用 /architect 调查现有约束并设计，比较、整合多个模型独立提出的方案，然后实施

No.	Part 2 中对应的章节	概要
10	『Architecting bigger changes』	如果实施中发现与设计不符，Agent 重新审视设计。根据实施得到的证据（例如类型需要 any 或强制类型转换），Agent 有时会舍弃设计、重新开始
11	『Okay but I really want a planning doc』	设计具体化后，Agent 用规划 Playbook 将工作拆成可小范围验证的任务或 PR，并为每项工作规定实际运行代码得到的结果作为完成证据
12	『Okay but I really want a planning doc』	规划 Playbook 生成的计划文档是临时文档，用于执行任务，并在大型工作中向其他 Agent 分享进度；poteto 通常在完成后将其删除
13	『The workflow in practice』	/poteto-mode 是按请求选用 Playbook 和 Skill 的入口。人向 /poteto-mode 说明目的、约束、要确认的结果和审查时机

表中“Part 2 中对应的章节”是讨论该要点的文章小节名，可在重读文章时作为线索。

要点 1 至 4 是监督工作的基本动作

逐一讨论要点 1 至 4 之前，先看这四个动作服务于什么工作。

文章的“The art of supervising someone smarter than you”一节讨论的是“人监督比自己更聪明的对象（最新模型）”这项工作。

借助 Agent，即使不熟悉代码，人也能修改系统。然而，**保持代码和用户体验的高质量仍然困难**；如果人不是知道该检查什么、该问什么的专家，就更难做到。

接下来的“In your own words”一节指出，这项监督工作发生在**人没有亲自编写代码库，而且已经无法在脑中掌握整个代码库结构的情况下**。

本书将要点 1 至 4 定位为这项工作的基本动作。

- **如何表达（要点 1）**……请求中应该说明什么，又不该过度说明什么
- **如何确认理解（要点 2）**……如何在实施前确认 Agent 的理解是否偏离人的意图

- **如何加深人的理解（要点 3）** ……人如何理解代码现在怎样运行、当初为何这样写
- **如何承接过去的工作（要点 4）** ……如何把先前的调查、判断和失败交给新对话中的 Agent

要点 1：说明目的和背景，不过早限定解决方法

人应在请求中充分说明目的、背景、约束和已知事实，但不要把具体方案写得过细（例如哪个文件的哪个函数要如何修改，或具体实施步骤）。

文章的观点：提供背景，保留解法的自由

在“The art of supervising someone smarter than you”一节，poteto 指出，**即使模型能力提升，仍反复出现以下两种失败。**

- 请求中的指示不足或不清楚，以至 Agent 无法完全理解人的意图
- Agent 缺少正确推进工作所需的上下文

这两个问题彼此相关，归结为一点：

“能否用好 Agent，取决于人能向 Agent 的上下文窗口放入多少高质量上下文。”

上下文窗口，是 Agent 一次能读入的信息范围。

据“In your own words”一节，poteto 过去会非常具体地告诉旧模型自己想让它做什么，几乎到了事无巨细地管理的程度。

但最新模型比人更擅长写代码。因此，在说明想要达成什么的同时，**也给 Agent 留下用人未曾想到的方法解决问题的自由**，是“监督比自己更聪明的对象”时的重要做法。

pstack：步骤由 Playbook 决定

在 pstack 中，用户说明目的、约束和已知事实，步骤则写在 Playbook 中（第7章）。

随附指南 [docs/guide/02-poteto-mode.md](https://docs/pstack.ai/guide/02-poteto-mode.md) 的“Say the goal, not the ceremony”一节，也建议用户不要在请求里写规范，而要说明：

- 问题是什么，或想要什么
- 能省去 Agent 调查时间的已知事实（如果有）

该页面给出了下面这行请求作为例子（第7章也讨论过）：

```
/poteto-mode users get two notifications after a retry. repro first, then fix and verify.
// 重试后，用户收到了两次通知。先复现，再修复，然后验证。
```

这条请求**说明了现象（重试后用户收到两次通知）和要遵守的约束（先复现）**，没有写具体方案，例如修改哪个文件的哪个函数。

因为请求是修复缺陷，/poteto-mode 会选择修复缺陷的“**Bug fix**” Playbook，并把其步骤（复现、缩小原因范围、规划修复、确认结果等）写入 TODO 列表。

TODO 列表.....Agent 在工作开始时建立的本次任务清单（计划）。用户可以查看清单，确认 Agent 推进了哪些步骤，又跳过了哪些步骤（skip: <理由>）（随附指南 01-setup.md）。在 Cursor 中，这份清单显示在聊天界面。

02-poteto-mode.md 在页面末尾的 Pitfall（陷阱）中提醒用户，不要在请求中按使用顺序罗列 Skill，例如“先 /how 再 /architect”。因为 Playbook 已经决定了工作顺序。

据这一 Pitfall，**如果依照用户写出的顺序，Agent 可能会调换或省略原本遵循 Playbook 时会执行的步骤。**

不过，02-poteto-mode.md 并不禁止提到 Skill 名称。它建议只想覆盖某项特定选择时才写出 Skill 名称（第7章）。

因此，如果步骤已经写在 Playbook 中，**用户只需在请求里说明问题是什么、想要什么、需要遵守哪些约束，以及已知事实，而无须写工作步骤。**

要点 2：实施前让 Agent 复述问题

人让 Agent 在实施前**用自己的话复述问题**，以确认 Agent 的理解与人的意图是否有偏差。

文章的观点：indirect prompt 的三个效果

“In your own words”一节将一种技法称为 **indirect prompt**（直译为“间接提示”）：人不直接把想让 Agent 做的事告诉它，而是引导 Agent 用自己的话说出来。

poteto 说，当有人在 Slack 报告问题时，他常常会在让 Agent 做其他事之前，**先让它阅读讨论串并复述问题**。示例请求如下。


```
/poteto-mode read this slack thread. restate in your own words and in plain english what you think
the underlying issue is
// 阅读这个 Slack 讨论串，用自己的话简明复述你认为根本问题是什么。
```

poteto 列出这种请求的三个效果。

- **梳理对话**……Agent 阅读发言交错的讨论串，把问题整理成简短的说明
- **尽早发现误解**……如果 Agent 误解了问题，人可以在它写代码之前指出并纠正
- **避免强加假设**……因为人没有先说出自己的假设，Agent 可以不受其影响地理解问题。人的假设可能有误；即使没错，也可能限制 Agent 的调查和解决方案的方向

这样使用 indirect prompt，可以减少返工，让双方在共享前提和认识之后再推进工作。

另外，poteto 在 2026 年 9 月末发布了下面这条帖子，介绍 indirect prompt 的具体例子；照着复制粘贴，应该就能得到预期的结果。

<https://x.com/poteto/status/2104744961904394699>

最近，这已经成为我最常用的提示词之一：

请用你自己的话复述，你认为我的目标是什么，以及我想解决什么问题。

要点 3：通过 /how、/why、/teach 理解机制与原因

人让 Agent 用 /how 调查代码现在怎样运行，用 /why 调查当初为何那样写，再通过 /teach 得到人能理解的说明。

文章的观点：用三个 Skill 加深理解

poteto 在 “Building up a mental model” 一节写道，与比自己更聪明的对象合作时，让 Agent 用人能理解的方式重新解释很重要。/teach 就是实现这一点的 Skill。

心智模型……人根据过去的经验和环境，在无意识中形成的“理解事物和思考问题的框架（固有想法）”。

/teach 会在内部调用 /how 和 /why。

Skill	作用
/how	追踪代码运行时的行为
/why	从 Git 历史、PR 审查评论、工单、设计资料等调查代码为什么写成这样（动机和意图）
/teach	把 /how 和 /why 查到的内容整理成人能理解的浅显说明，使人更好地理解 Agent 的工作

之所以需要 /why，是因为阅读代码虽然能知道发生了什么，却几乎无法知道写代码的人为什么这样写。

以下是这三个 Skill 的示例请求。

第一个例子中的“虚拟化”，指长列表只渲染屏幕上可见部分的机制。

```
/how is virtualization implemented?  
// 虚拟化是怎样实现的?  
  
/why are we still stuck an old version of node.js?  
// 为什么我们还无法摆脱旧版 Node.js?  
  
/teach me why you implemented it this way and not <other way>. what were the tradeoffs you made and why?  
// 解释为什么选择这种实现方式而不是<其他方式>。权衡了什么，理由是什么？
```

除了帮助人理解，/teach 还有一个效果：**为向人解释而进行的调查，也能帮助 Agent 自己。**

poteto 说：“即使最新模型也常在没有数据佐证、没有阅读理解机制所需代码的情况下，自信地断言”（他补充说，程度也取决于运行 Agent 的工具质量）。

因此，向人说明接下来要做什么以及为什么这样做，本身也能帮助 Agent。

pstack: /teach 保留表达可信度的措辞

/why 会按“这一原因得到记录的支持程度”，为找到的每项原因标注五级可信度。/teach 汇总并解释 /why 的结果。

/teach 的 `SKILL.md` 允许为了说明而自由调整措辞，但设了一个例外。原文如下。

Keep why's confidence language intact (its hedges are findings, not style).

保留 why 表达可信度的措辞（其中的保留性表述是调查结果，不是文风）。

换言之，/teach 不改动“似乎”“可以认为”等表达可信度的词，是因为**这些词并非写作习惯，而是 /why 查出的结果本身：该原因究竟有多确定。**

因此，如果 /why 将某个原因列为推测，/teach 的说明也会将其作为推测传达。

要点 4：通过 /recall 将过去的工作带入新对话

人让 Agent 用 /recall 回顾过去对话中相关的工作，把先前的调查、判断和失败带入新工作。

之所以连失败也要带过去，是因为**如果不知道此前失败的方法，Agent 可能会再次尝试同一种方法。**

文章的观点：用 /recall 避免从头重建上下文

poteto 在“Learning from history”一节，写了自己修复 Cursor 中报告的虚拟化缺陷和性能问题时的经历。

先前对话中的 Agent 在解决类似问题时，往往积累了丰富的上下文。但每次打开新聊天，又得几乎从头重建这些上下文。

由此可见，**过去的对话记录常常保存着先前 Agent 掌握的大量上下文。**

/recall 就是为应对这一问题而设的 Skill。

/recall 从聊天历史提取近期上下文，让新对话中的 Agent 也能带着所需信息继续工作。

示例请求如下。

```
/recall the work i did yesterday on virtualization and then read this bug report on slack
// 回忆昨天完成的虚拟化工作，然后阅读 Slack 上的这份缺陷报告。
```

pstack: /recall 也收集失败记录

`/recall` 会调查用户自己的聊天历史。如果调查主题指明了某项功能、文件或缺陷，还会查看保存在以下地方的团队共享记录。

- 源代码管理
- 工单
- 聊天
- 错误监控

共享记录中保存着过去失败的记录等重要上下文，例如发布后被撤回的修复，或在生产环境中持续出现的错误。

而且，`/recall` 可以返回**此前试过什么、哪些改动被撤回**（第22章）。

这样，Agent 就能从上一次失败的地方开始下一次尝试。

小结

- **要点 1 至 4 的联系**……本书将要点 1 至 4 定位为监督比自己更聪明的模型所需的基本动作。
- **要点 1：说明目的和背景，不过早限定解决方法**……人在请求中说明目的、背景、约束和已知事实，不把具体解决方法定得过死。
- **要点 2：实施前让 Agent 复述问题**……人在实施前让 Agent 用自己的话复述问题。
- **要点 3：通过 `/how`、`/why`、`/teach` 理解机制与原因**……人让 Agent 用 `/how` 和 `/why` 调查，通过 `/teach` 的说明理解代码现在怎样运行，以及当初为何这样写。
- **要点 4：通过 `/recall` 将过去的工作带入新对话**……人让 Agent 用 `/recall` 回顾过去对话中相关的工作，把先前的调查、判断和失败带入新工作。

下一章第38章将讨论“尝试解决方案”阶段的要点 5 至 8。

这四个要点分别是：实施前写出用户的使用方式（要点 5）；按用途区分文档（要点 6）；制作多个方案的原型并比较（要点 7）；让 Agent 通过实验验证疑问（要点 8）。

第 38 章：指南二，从用户的使用方式出发检验设计

kaito · 来源：日本語 · English · 2026-10-03

第37章讨论了如何与 Agent 一起决定要做什么：解决哪个问题、采用什么设计，以及如何规划。

本章进入理解问题后的下一阶段，介绍如何尝试解决方案。

本章同样以《The Complete Guide to pstack》的 [Part 2](#)（下称“文章”）为基础。

文章建议的**不是细化抽象计划（尚未写成代码、只有文字的计划）**，而是以下三件事。

- 实施前写出用户的使用方式
- 实际制作并运行多个方案
- 根据运行结果作出选择

第37章把文章内容归纳为13个要点。

本章讨论其中的要点 5 至 8。

No.	Part 2 中对应的章节	概要
5	『Working backwards』	人让 Agent 在实施前编写 README 或教程，从用户如何使用出发，思考 API 和内部设计
6	『Working backwards』	Agent 用 /technical-writing 按教程、操作指南、参考资料和解释等用途整理文档，并通过其内部使用的 /unslop 去掉 AI 容易写出的不自然措辞
7	『Measure a hundred times, cut once』	人不直接采纳 Agent 的第一个设计，而是让 Agent 制作多个方案的原型，比较实际操作时的截图、测得的时间或布局

No.	Part 2 中对应的章节	概要
8	『Measure a hundred times, cut once』 『Architecting bigger changes』	对于能通过做出小实验并运行来回 答的疑问，人让 Agent 自己验证。 通过原型和验证减少不确定性，而 不是反复审查抽象计划

表中“Part 2 中对应的章节”是讨论该要点的文章小节名。

首先解释 pstack 的 README 为什么说“不相信计划”，然后逐一讨论要点 5 至 8。

每个要点主要分为“文章的观点”和“pstack”两部分。

要点 7 使用的“Prototype” Playbook，其步骤已在第12章介绍。

本章略去以下内容，重点讨论文章的建议。

- **如何制作原型**.....Agent 优先追求速度，不要求代码质量。
- **制作完成后的报告**.....Agent 向人展示试过的方案、证据（截图或测量结果）、取舍和推荐方案。

本章结构

本章内容如下。

- “不相信计划”的意思是“用代码规划”
- 要点 5：先写使用方式
- 要点 6：用 /technical-writing 按用途区分文档
- 要点 7：制作多个方案的原型并比较
- 要点 8：通过实验验证疑问
- 小结

“不相信计划”的意思是“用代码规划”

pstack 的 README 写着“不相信计划”，意思并非抛弃规划，而是**用代码规划**。

README 的 “why are there no planning skills?” 一节有这样一句话：

personally, i don't believe in planning. the best spec is code.

就我个人而言，我不相信计划。最好的规格说明是代码。

规划模式.....Agent 在编写代码前写出计划的模式。

在文章的 “Working backwards” 一节，poteto 谈及多数带有规划模式的工具（原文为 harness）时指出：

“它们往往把实施细节写得过多，却把其他内容写得不够。”

但 poteto 也写了下面这句话，说明他并非完全不做规划。

The truth is that I do plan, but I do so through code.

事实上，我也会规划，但我是通过代码来做的。

不同的是规划方法。

本书认为，用代码规划，并非不加思考就开始实施，而是 “一边制作和验证，一边解答设计上的疑问”。

这种方法包括写出使用方式的教程（要点5），以及制作并验证可以实际运行、相互比较的原型（要点7、要点8）。

要点 5：先写使用方式

人在实施前让 Agent 在 README 或教程中写出使用方式，然后从中反推 API 和内部设计。

文章的观点：从 README 开始写

poteto 在文章中介绍了 “README 驱动开发” 这种先写使用方式的开发方法。

README 驱动开发……先写向目标用户解释使用方式的 README，再反推实施方式和架构的开发方法。

从 README 开始写，会迫使开发者站在使用所开发成果（例如 API）的一方，思考它是否好用（原文为 developer experience）。

例如，poteto 制作 Cursor 内部的桌面应用框架 Dune 时，据说先让 Agent 写教程，以了解用 Dune 开发应用会是什么体验。

在实施前写出的使用方式，**对人来说是理解完成形态的材料，对 Agent 来说则是实施后验证行为的具体标准。**

顺带一提，文章也写道，最初很难让 Agent 写出易读的教程，因此先制作了 /technical-writing（要点6）。

pstack: /architect 的输出也先写使用方式

/architect（第24章）的 SKILL.md 在“Outputs”一节也规定，**先写调用方的使用方式，再从中推导出代码骨架。**

The caller's usage is written first and the type sketch derived from it.

先写调用方的使用方式，再由此推导类型骨架。

- **骨架**……只写类型和函数的形状，暂不写内部实现的代码。
- **webhook**……向指定 URL 发送 HTTP 请求，把事件通知给另一个系统的机制（第25章）。

以从外部传来的 webhook 设置速率限制（一定时间内允许接收的请求数量上限）为例，看看如何从使用方式推导骨架。

这一功能也出现在文章“The workflow in practice”一节的示例 2 请求中（we need to add rate limiting for external webhooks），参见第39章的“例 2）设计请求”。

这里以将来自同一发送方（发送 webhook 的外部系统）的 webhook 接收量限制为每分钟 60 个为例。

按照 /architect 的要求，Agent 先写使用方式，再从中推导骨架，就会得到以下结果。

```
// 1. 先写调用方的用法
const limiter = createRateLimiter({ perMinute: 60 });

function handleWebhook(webhook: { senderId: string }): Response {
  if (!limiter.tryAcquire(webhook.senderId)) {
    return new Response("Too Many Requests", { status: 429 });
  }
  return new Response("OK");
}

// 2. 从用法推导骨架：只写类型与函数形式，暂不写实现
type RateLimiterOptions = { perMinute: number };
type RateLimiter = { tryAcquire(senderId: string): boolean };

function createRateLimiter(options: RateLimiterOptions): RateLimiter {
  throw new Error("not implemented");
}
```

因为在第 1 部分的用法里给 createRateLimiter 传入了 { perMinute: 60 }，第 2 部分的 RateLimiterOptions 形状也随之确定。

同样，第 1 部分根据 limiter.tryAcquire(webhook.senderId) 的结果决定是否接收，所以第 2 部分的 RateLimiter 必须有一个接收发送方 ID（字符串）并返回布尔值的 tryAcquire。

函数的内部实现（throw new Error("not implemented") 所在部分），在确定骨架后的实施阶段再写（文章对按骨架实施的说明，见第39章的要点 9）。

由此可见，先写使用方式，类型和函数的形状便由使用方式决定。

要点 6：用 /technical-writing 按用途区分文档

Agent 用 /technical-writing 把文档分为教程、操作指南、参考资料、解释四类，**不在同一份文档里混合不同用途。**

文章的观点：混合用途的 README 不易读

poteto 说，没有使用 `/technical-writing` 时让 Agent 写的第一版 Dune README 很难读。

因为教程、操作指南、解释和参考资料四种作用都被塞进了一份文档，而且语言带有 AI 式的故作郑重。

因此 poteto 制作了 `/technical-writing`（第32章）。

- **Diátaxis**……按读者的目的把文档分成四种类型（模式）的框架。
- `/unslop`……Agent 按规则从自己写的文章中删除 AI 式习惯（让读者觉得“像机器写的”措辞和格式，例如“希望这对你有所帮助”之类套话）的 Skill（第32章）。

`/technical-writing` 根据 Diátaxis 给文档分类。

`/technical-writing` 还会使用 `/unslop`，因此生成的文档更易读。

文章的观点：将 `/technical-writing` 与其他 Skill 组合

pstack 的许多 Skill，包括 `/technical-writing`，组合使用时能互相增效。

文章以用户考虑自己应用设计的阶段（原文为 design phase）为例，说明这种情况。

把 pstack 的 Skill 组合起来、提取丰富的上下文，Agent 就不再只看到问题的一小部分，而能像人一样看到问题的核心部分。

- **验证 Skill**……让 Agent 启动和操作应用、收集证据的项目专用 Skill（第36章）。
- **虚拟化**……长列表只渲染屏幕上可见部分的机制（第37章）。

以下这个由三个部分组成的请求，展示了 Skill 如何组合。

```
(1) /recall my work fixing virtualization bugs and perf issues from the past 7 days. use /how and /why to understand how our current virtualization implementation works.
```

```
// 回忆过去七天修复虚拟化缺陷与性能问题的工。用 /how 和 /why 理解当前虚拟化实现的运行方式。
```

```
(2) then use /poteto-mode planning and /technical-writing to come up with a new virtualization engine that categorically eliminates flickering and jittering. let's start by writing a tutorial on how i would use this new package to virtualize a React app
```

```
// 然后通过 /poteto-mode 制定计划，用 /technical-writing 设计一个从根本上消除闪烁和抖动的新虚拟化引擎。先写教程，说明我如何在 React 应用中使用这个新软件包实现虚拟化。
```

```
(3) after you write the plan, /teach me and prove to me why this new approach is superior to our current engine
```

```
// 计划完成后，用 /teach 教给我，并证明新方法为何优于当前引擎。
```

/technical-writing 出现在 (2)，承担编写新软件包教程的任务，也就是[要点5](#)所说的“使用方式”。

三个部分分别是：

- **(1)**……让 Agent 用 /recall 回顾过去七天的工作，用 /how 和 /why 理解当前虚拟化实现如何运行。
- **(2)**……利用 (1) 收集的内容（例如先前修复的缺陷），设计一种能从根本上消除闪烁和抖动的新方案。
- **(3)**……写完计划后，用 /teach 解释，并证明新软件包比当前引擎（负责现有虚拟化的组件）更好。
（这里，验证 Skill 之类的高质量工具很重要。）

因此，**这条请求旨在让 Agent 提取先前工作和当前实现的丰富上下文，并将其用于设计和证明。**

pstack：一份文档采用一种模式

/technical-writing 的 SKILL.md 在“Pick the mode first (Diátaxis)”一节规定，**一份文档采用一种模式。**

One document, one mode.

一份文档，一种模式。

Agent 根据文档内容对读者有何帮助，通过以下两个问题选定模式。

- 是为了行动，还是为了理解
- 是为了学习，还是为了完成工作

两组答案的组合对应以下模式（各模式的说明依据文章）。

- **行动与学习**……教程。让初学者按步骤制作可见成果、边做边学的文档。
- **行动与工作**……操作指南。供熟悉工具的人解决当前特定问题的步骤文档。
- **理解与工作**……参考资料。准确、完整地说明 API 和配置项的文档。
- **理解与学习**……解释。说明背景、设计选择和取舍的文档。

前面两个问题既可用于整份文档，也可用于一句话。

此外，该 SKILL.md 禁止把参考资料表格塞进教程，要求把两者分开并建立链接。

例如，如果想在教程中途插入配置项清单，该清单属于帮助理解的参考资料，因此 Agent 会把它移到另一份文档，再从教程链接过去。



[查看图示 1](#)

详细规则已在[第32章](#)介绍。

要点 7：制作多个方案的原型并比较

人不直接采用 Agent 的第一个设计，而是让 **Agent** 实际制作多个方案，根据操作和测量的结果选择采用哪个。

文章的观点：制作并比较多个方案的原型

poteto 在“Measure a hundred times, cut once”一节指出，规划中常见的失败有两种。

- 直接接受 Agent 给出的第一个设计
- 没有运行并验证的结果（经验证据），就把计划细化得过多

本书在要点 7 讨论第一种失败，在[要点8](#)讨论第二种。

对于第一种，poteto 的解释如下。

过去由人写代码时，大家通常互相传阅设计文档，不断重写，直至设计稳定。

与 Agent 合作时，人可以省去自己写设计文档的程序，让 Agent 来写已成为常态。

问题在于，人们往往“**直接接受 Agent 第一次交回的设计**”。

poteto 称之为失败。

With agents, while we can skip the ceremony of the design doc, I often see the mistake of accepting the first thing the agent gives back to you.

与 Agent 合作，可以省去设计文档这套程序；但我经常看到人们犯下直接接受 Agent 第一次交回内容的错误。

文章没有说明为何称之为失败，但凭直觉和经验，这个判断似乎也成立。

我认为，**未经比较的设计，无法判断好坏**。

过去，人们在设计文档中列出备选方案，并反复重写，就是为了比较方案并选出更好的一个。直接接受第一个设计，会整个跳过比较这一步。

应对这一失败的方法，是使用“**Prototype**” Playbook 制作并比较多个方案。

示例请求如下。

```
/poteto-mode prototype a few options for <feature request>. use /control-app* and take videos/
screenshots for me to review and choose from
// 为<功能需求>制作几个候选原型。用 /control-app* 录制视频并截图，让我比较后选择。
```

文章脚注（*）解释，/control-app 指的是《The Complete Guide to pstack》[Part 1](#) 中制作的验证 Skill（[第36章](#)）。

对于改变外观的原型，Agent 会在应用内或工作目录中制作一次性的试作品。

比较 UI 操作方式时，Agent 会制作两三个方案，提供简单的切换开关，方便切换显示的方案。

例如，比较三种打开菜单的方式时，可以在屏幕上放置“方案 A”“方案 B”“方案 C”按钮，按哪个按钮，菜单就按对应方案打开。

这样能留在同一个画面中切换和比较。

然后，Agent 用 `/control-app` 操作各个方案，拍摄截图，并测量实际的时机和布局。

“**Prototype**” Playbook 除了比较外观方案，也能比较功能或缺陷修复方案。

pstack：原型是为决定设计而制作的一次性工具

“**Prototype**” Playbook 将原型定义为决定设计所用的一次性工具。

开始之前，Agent 应明确决定原型要“帮助决定什么”。

可能要决定的是以下三项。

- 采用哪种布局
- 采用哪种操作方式
- 屏幕元素排列得多紧凑

如果通过运行和观察就能判断哪种选择更好，也可以决定以下事项。

- 采用哪种行为
- 采用什么时机
- 采用哪种方法

如果没有需要用原型决定的事，Agent 不会制作原型，而会将任务交给开发新功能的“**Feature**” Playbook。

即使人已经选定一个方案，Agent 也会把该方案交给“**Feature**” Playbook。

如果需要先决定类型或模块的形状，则交给 `/architect`。

要点 8：通过实验验证疑问

对于做出来并运行就能回答的疑问，人无需自己作答，也无须让 Agent 等待人的答复，而应让 Agent 通过实验验证。

在计划仍然抽象时，也不要反复审查并细化它。

- **实验**……制作一个小版本，运行它并检查结果。
- **抽象计划**……尚未用代码成形、只有文字的计划。即使长篇计划写满实施细节，只要尚未用代码成形，仍是抽象计划。

文章的观点：用原型回答疑问，不对抽象计划进行对抗性审查

没有运行并验证的结果（经验证据），就把计划细化得过多

应对要点7提到的第二种失败的方法，同样是原型。

poteto 在 “Measure a hundred times, cut once” 一节写道，**制作原型就是用代码规划**。

有了原型，Agent 不用等人的回复，就可以根据运行和验证的结果回答自己的疑问。

例如，要判断“A 和 B 哪个更快”，Agent 可以分别运行、测量时间；要判断“这种操作会不会把屏幕布局弄乱”，可以操作原型进行确认。

在 “Architecting bigger changes” 一节的末尾，poteto 写道，**用原型和 /architect 通过代码规划要有效得多。**

他接着说，这也是不对抽象计划进行对抗性审查的原因。

对抗性审查……让 Agent 从寻找缺陷的立场提出批评。

不对抽象计划做对抗性审查，是因为这样审查会让 Agent **把没有根据、只在理论上成立的风险编造得像事实一样。**

随后又会为根本不会发生的问题设计复杂的异常情况。

例如，Agent 可能针对计划中的某个函数指出：“如果向这个函数传入 null（表示没有值的值），处理就会崩溃”，并要求在计划中加入处理 null 的步骤。

有代码时，可以追踪调用方，确认实际上是否可能传入 null。

如果没有调用方会传入 null，就知道这项建议是在防备不会发生的问题。

但抽象计划中没有可追踪的调用方。

因此，无法核实的指出会被当作事实，计划中便不断堆积针对不存在问题的异常处理。

基于这些原因，文章建议在计划仍然抽象时不要将其细化；对于尚未解决的疑问，让 Agent 制作原型、检查结果并自行得出答案。

pstack: Agent 在询问人之前，先判断疑问能否由实验回答

/poteto-mode 的“Non-negotiables”一节列出所有 Playbook 的工作都必须遵守的规则（第8章）。

其中也有关于 Agent 想向人提问时的规定。

它针对的是推进方式存在多个选择时的问题，例如：

- 采用哪种方法
- 应该怎样制作
- 这项工作应该做什么

“Non-negotiables”要求 Agent 在提出这类问题前，先判断答案是否属于运行并观察就能得知的事实（第35章）。

如果答案是运行后可观察的事实，就不应该由人来回答。

可观察的事实包括：

- 行为
- 时机
- 布局
- 输出
- 性能

对于答案属于这类事实的疑问，Agent 无需询问人，而应使用“Prototype” Playbook 尝试并决定。

不过，“Non-negotiables”有一个例外：如果工作只是只读调查（“Investigation” Playbook），成果物是有证据支持的回答。

这种情况下，Agent 不制作原型，而是根据收集的证据回答。

Agent 应向人询问的是试了也无法回答的产品判断和偏好。

用下面两个问题看看这种分类。

“这样操作会让屏幕布局错乱吗？”可以通过操作原型观察，因此由 Agent 尝试并判断。

而“两个布局中，这款产品应该采用哪一个？”即使尝试也无法得出答案，属于偏好问题。屏幕元素如何排列（布局）可以观察，但观察本身不能决定该选哪一个。

因此，Agent 应把两个画面展示出来，交由人选择。

pstack: /interrogate 审查骨架和差异

如果不对抽象计划做对抗性审查，那么 pstack 对什么做这种审查呢？

pstack 有一个 Skill，能让多个模型对变更进行对抗性审查：`/interrogate`（第27章）。

`/interrogate` 在给审查者的指令中，把他们称为“对抗性代码审查者”，要求他们寻找真正的问题，而非提供帮助或鼓励。

何时使用 `/interrogate` 由“Non-negotiables”规定：**有争议的设计应在合并前交给 /interrogate 审查。**

有争议的设计……不同的人可能对其好坏作出不同判断的设计（原文为 contested design）。

本书认为，当几种做法无论选择哪一种都能正确运行时，就属于这种情况，例如错误处理：失败时抛出异常，或通过返回值表示失败，都能正确运行，因此可能有不同意见。

查看 pstack 的两个 SKILL.md 可以发现，提交给 `/interrogate` 的设计，都是**已经形成代码的内容**。

- **差异**……`/interrogate` 的 SKILL.md 把审查对象定为代码改动（差异）。
- **骨架**……`/architect`（将多个模型的设计方案整合为一个骨架的 Skill；第24章）的 SKILL.md 写道，在开始实施前，可以把整合后的骨架（只写类型和函数形状、尚无内部实现的代码）交给 `/interrogate`（“Phase C: Agree (opt-in)”一节）。

只要内容已形成代码，就能把审查指出的问题与实物核对。

在 `/interrogate` 中，掌握全局上下文的主 Agent 会核实审查者提出的问题。

像前面 null 的例子，只要追踪调用方，就能知道那种假设实际上不会发生，因此可归类为无需处理的意见（第27章）。

由此，本书认为，“不对抽象计划做对抗性审查”并不是“不审查”，而是**先准备证据，再进行审查**。

这里的证据，指骨架、差异、使用示例、原型、测量结果等审查者可以查看实物并核实的东西。

小结

- **“不相信计划”的意思**……并非抛弃规划，而是用代码规划。本书认为，用代码规划就是一边制作和验证，一边解答设计上的疑问。
- **要点 5：先写使用方式**……人让 Agent 先写 README 或教程，再由使用方式思考 API 和内部设计。
- **要点 6：用 /technical-writing 按用途区分文档**……Agent 用 /technical-writing 将文档分为教程、操作指南、参考资料、解释四种模式，不在一份文档中混合用途。/technical-writing 会用 /unslop 去除 AI 式的写作习惯。
- **要点 7：制作多个方案的原型并比较**……人不直接采用 Agent 的第一个设计，而是让它制作多个方案的原型，以截图和测量结果比较。原型是为决定设计而制作的一次性工具。
- **要点 8：通过实验验证疑问**……人让 Agent 自行实验验证能够通过实验回答的疑问。原则上，人只回答试了也无法决定的产品判断或偏好。本书认为，“不对抽象计划做对抗性审查”的意思，是准备骨架、差异、使用示例、原型和测量结果等证据后再审查。

下一章第39章将讨论“确定解决方案，并将其转化为可执行计划”阶段的要点 9 至 13。

- 要点 9：用 /architect 比较并整合多种设计
- 要点 10：如果实施中发现设计错误，就重新审视设计
- 要点 11：拆分为可以小范围验证的任务
- 要点 12：计划文档是临时的
- 要点 13：应向 /poteto-mode 说明什么

第 39 章：指南二，重新审视设计，形成可执行计划

kaito · 来源：日本語 · English · 2026-10-03

第 38 章介绍了从用户的用法出发思考设计，用原型试验解决方案，并选择设计方案的方法。

本章继续讨论试验解决方案之后，Agent 如何**确定大型变更的设计，并将其转化为可执行计划**。

- **确定设计**（/architect）……决定类型、公开函数的签名（函数名称、参数和返回值类型），以及模块如何划分。
- **转化为可执行计划**（「Multi-phase or multi-PR plan」Playbook）……把大型任务分为多份 PR，每份只包含一项变更（例如将「修改数据类型」和「让界面适配新类型」分开）。然后为每份 PR 写出要修改的文件和确认完成的方法（例如核对界面状态），形成计划文档。

本章也以《The Complete Guide to pstack》Part 2（下称文章）为依据。

第 37 章将文章归纳成13 个要点。

本章介绍其中剩余的要点 9～13。

No.	Part 2 对应章节	概述
9	「Architecting bigger changes」	大型变更中，人将时间用于思考数据如何存放以及系统之间如何协作。Agent 使用 /architect 调查现有约束并设计，在比较、整合多个模型各自独立提出的方案之后再实现
10	「Architecting bigger changes」	实现时如果发现与设计不符，Agent 就重新审视设计。根据实现中得到的证据（例如，类型需要 any 或强制类型转换），Agent 可能舍弃原设计，重新开始

No.	Part 2 对应章节	概述
11	「Okay but I really want a planning doc」	设计具体化后，Agent 使用规划 Playbook 将其拆分为可分别验证的小任务或 PR，并规定每项任务都要以实际运行代码得到的验证结果作为完成证据
12	「Okay but I really want a planning doc」	规划 Playbook 生成的计划文档，是执行任务以及在大型任务中与其他 Agent 分享进度的临时文档；poteto 通常在完成后删除
13	「The workflow in practice」	/poteto-mode 是根据请求选用 Playbook 或 Skill 的入口。人向 /poteto-mode 说明目标、约束、希望确认的结果以及进行审查的时机

表中「Part 2 对应章节」是文章讨论相应要点的小节名称。

先通过要点 9~11 了解如何确定设计并形成计划，再在要点 12 讨论计划文档的用途，要点 13 讨论人应向 /poteto-mode 说明什么。

最后，再把全部 13 个要点当作一条流程回顾。

每个要点主要分为「文章的观点」和「pstack」两个小节。

要点 9~11 使用的 /architect 和「Multi-phase or multi-PR plan」Playbook 的步骤，已在第 24 章与第 16 章介绍。

本章略去以下内容，重点说明文章推荐的做法。

- **要求展示设计**……默认情况下，/architect 完成整合后的骨架（只写类型和函数形式、尚未写函数内部的代码）后，不等待使用者确认便进入实现。只有使用者在请求中明确要求（例如 /architect with checkpoint. stop and show me before implementing.），它才展示整合后的设计并停下。
- **舍弃骨架后的重新设计**……Agent 以实现中发现的新约束为前提重新设计，并在增加功能前让新骨架比旧骨架更小（第 17 章的 Principle「Redesign from First Principles」「Subtract Before You Add」）。
- **无需计划的情况**……若变更只涉及一两个文件，做法也显而易见，Agent 不创建计划，而是告知使用者不需要计划后停下。

- **检查计划格式**.....Agent 使用 `check-plan.mjs` 核对计划文档的结构和格式，并修正输出的全部问题。

本章结构

本章安排如下。

- 要点 9：用 `/architect` 比较并整合多个设计
- 要点 10：实现揭示设计错误时，重新审视设计
- 要点 11：拆成可分别验证的小任务
- 要点 12：计划文档是临时性的
- 要点 13：向 `/poteto-mode` 说明什么
- 将 13 个要点连成一条流程
- 总结

要点 9：用 `/architect` 比较并整合多个设计

对于大型变更，人把时间花在架构设计、数据结构及系统间协作上；Agent 则花时间填入实现细节。

但 Agent 不会直接开始实现，而是先比较多个设计方案，整合成一个，再着手实现。

文章的观点：poteto 将时间用于架构设计

poteto 在「Architecting bigger changes」一节写道：在 Agent 时代，工程师更应该把时间用在以下方面。

- 架构设计
- 选择合适的数据结构
- 自己构建的系统之间如何协作

他也写道：实现细节由自己的 Agent 填入。

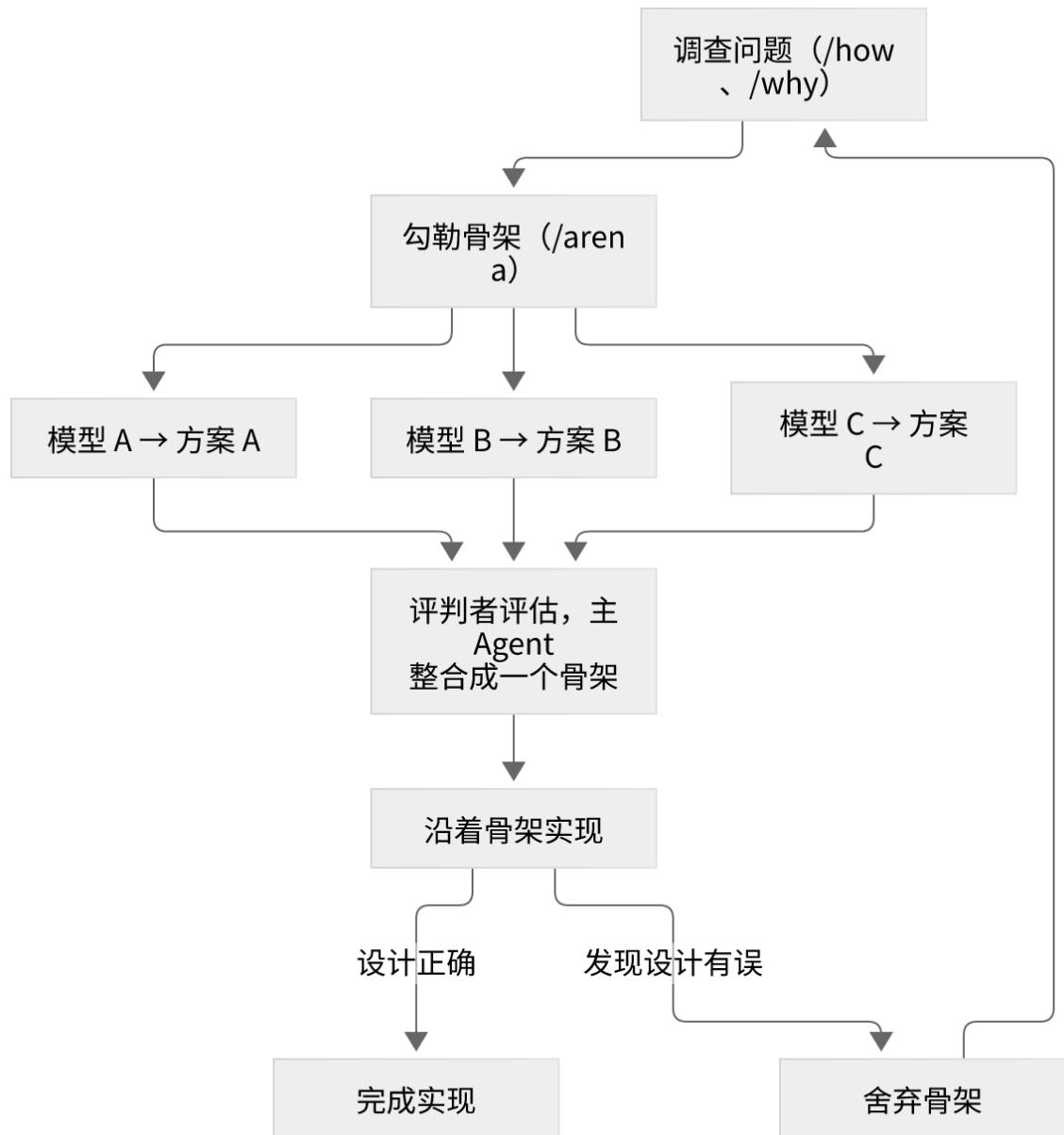
文章举出 `/architect`，作为分阶段推进设计的 Skill，并给出下面这一行请求示例。

```
/architect this new <feature request>  
// 设计这个新的<功能需求>。
```

文章将 `/architect` 的流程分为五个阶段。

1. 调查问题
2. 绘制骨架
3. 评估并整合
4. 依照骨架实现
5. 设计有误时舍弃

流程如下图所示。



[查看图示 1](#)

下面逐一说明这五个阶段。

1. 调查问题

Agent 使用 `/how` 和 `/why` 调查新代码将涉及的现有机制。

- `/how`……调查代码当前如何运行的 Skill（第 22 章）。
- `/why`……调查代码为何形成现状的 Skill（第 22 章）。

由此准确把握各模块的职责分工，以及现有约束。

2. 绘制骨架

Agent 让多个模型并行制作设计方案，彼此不看对方的方案（`/architect` 的 `SKILL.md` 使用 `/arena`）。

这些模型通常来自不同系列。

`/arena`……让多个 Agent 分别解决同一问题的 Skill（第 24 章）。

每个模型提交的方案由以下四部分组成，原文称为 `design package`。

- **调用方的用法**……使用待建功能的一方如何编写代码（下例代码第 1 部分）。
- **核心类型定义**……设计中心的数据结构（下例代码第 2 部分的 `RateLimiterOptions` 和 `RateLimiter`）。
- **公开函数的签名**……供其他代码调用的函数名称、参数及返回值类型（下例代码第 2 部分的 `createRateLimiter` 第一行）。
- **简述设计理由的文档**……用约一页说明为何采用这一形式、还考虑过什么其他形式及为何没采用。只有这一部分用文字而非代码编写（写法见 `/architect` 的 `references/rationale-template.md`）。

前三部分通常绘制成骨架代码。

骨架……只写类型与函数形式，暂不写函数内部的代码（第 24 章）。

每个设计模型还会从以下三个方面自检方案。

- **接口深度**……调用方可见的函数和类型是否很少，却能在内部承担大量处理？例如，调用一次 `tryAcquire`，计数及每分钟重置计数能否都在内部完成？
- **能力较弱模型可能遇到的失败**……考察日后能力较弱的模型能否依照此设计写出满足要求的实现。
- **对照四个危险信号**……/architect 有一份设计危险信号清单（[references/design-red-flags.md](#)；第 24 章），包括浅模块、信息泄漏、按步骤拆分、纯转发方法。方案若符合其中一项，就应修正或舍弃。

骨架代码示例

先写调用方的代码，再由此确定类型和函数形式，形成骨架。

骨架示例：第 38 章推导出的 `webhook` 限流骨架

第 38 章的要点 5（先写用法）示范了如何由调用方代码推导骨架。示例内容如下。

这里以限制同一发送方（向我们发送 `webhook` 的外部系统）每分钟最多提交 60 个 `webhook` 为例。

这个例子按以下顺序编写。

```
// 1. 先写调用方的用法
const limiter = createRateLimiter({ perMinute: 60 });

function handleWebhook(webhook: { senderId: string }): Response {
  if (!limiter.tryAcquire(webhook.senderId)) {
    return new Response("Too Many Requests", { status: 429 });
  }
  return new Response("OK");
}

// 2. 从用法推导骨架：只写类型和函数形式，暂不写实现
type RateLimiterOptions = { perMinute: number };
type RateLimiter = { tryAcquire(senderId: string): boolean };

function createRateLimiter(options: RateLimiterOptions): RateLimiter {
  throw new Error("not implemented");
}
```

第 1 部分先写调用方代码：将 `{ perMinute: 60 }` 传给 `createRateLimiter`，再依据 `tryAcquire` 的结果决定是否接收 `webhook`。

第 2 部分确定符合这一用法的类型与函数形式。

这与第 38 章要点 5 是同一思路。该章这样写道。

人在实现之前，让 Agent **先在 README 或教程中写出用法**，再由此反推 API 与内部设计。

3. 评估并整合

由不同于主 Agent（启动各方案模型的 Agent）模型的评判者，按严格标准评估各方案。

主 Agent 根据各方案及其评估，将它们整合成一份设计。

随附指南 [docs/guide/04-design.md](#) 说明，负责整合的 Agent 会完整阅读每份方案，选一份作为基础，吸收其他方案的优点，再验证结果。

4. 依照骨架实现

Agent 将骨架中临时代替函数体的代码（例如 `throw new Error("not implemented")`）换成真实实现。

如果实现时发现函数需要未曾预料的参数或状态，Agent 不会悄悄在实现内部消化偏差，而会把它明确指出。

然后重新审视原因属于哪一种（`/architect` 的 `SKILL.md` 中「Phase D: Implement against the sketch」一节）：

- 骨架有误
- 遗漏了要求
- 实现做了超出需要的事

5. 设计有误时舍弃

实现中若确认骨架有误，Agent 会舍弃全部骨架，从头开始（见要点 10）。

pstack：跨越函数边界或转移职责时使用 /architect

随附指南 docs/guide/04-design.md 认为，大多数改动无需比较并整合多个方案这样的设计过程。

它将**跨越函数边界的改动，或转移职责归属（哪个模块负责什么）的改动**列为 /architect 的适用范围。

例如，改变某函数参数或返回值的结构，并需同步修改调用方，就是跨越函数边界的改动。

假设返回商品金额的函数 `getPrice`，改为返回「金额和币种」，而非仅返回金额。

```
// price.ts (修改前)：只返回金额
function getPrice(itemId: string): number {
  // ...
}

// price.ts (修改后)：返回金额和币种
function getPrice(itemId: string): { amount: number; currency: string } {
  // ...
}
```

返回值结构变了，调用 `getPrice` 的代码也必须同步修改。

将缓存职责从界面代码移到获取数据的模块，则是职责归属发生变化。

变更前，界面代码自己持有缓存（保存已获取数据的容器）。

```
// UserPage.ts (修改前)：页面代码持有缓存
const cache = new Map<string, User>();

async function showUser(id: string) {
  const user = cache.get(id) ?? (await fetchUser(id));
  cache.set(id, user);
  render(user);
}
```

变更后，获取数据的模块持有缓存，界面代码只负责调用。

```
// userApi.ts (修改后)：获取数据的模块持有缓存
const cache = new Map<string, User>();

export async function fetchUser(id: string): Promise<User> {
```

```

const cached = cache.get(id);
if (cached) return cached;
const user = await request(`/users/${id}`);
cache.set(id, user);
return user;
}

// UserPage.ts (修改后)：无须考虑缓存，只管调用
async function showUser(id: string) {
  render(await fetchUser(id));
}

```

缓存职责从界面代码移到了 `userApi.ts`。这就是转移职责的改动。

因此，若改动不局限于单个函数内部，而涉及其他代码依赖的约定（参数或返回值结构）或职责分工，就适合从 `/architect` 的设计流程开始。

04-design.md 明确说，`/poteto-mode` 接到请求后会自行应用这一标准，决定是否使用 `/architect`。

所以，**使用者无须每次亲自判断是否要调用 `/architect`。**

使用者直接调用 `/architect` 等 Skill，通常是希望比 `/poteto-mode` 的默认做法更仔细，或更简短地检查设计。

要点 10：实现揭示设计错误时，重新审视设计

如果实现中发现骨架与实际需求不符，或不断出现权宜性的绕路办法（不修问题根因，只暂时绕开的写法），Agent 会将其视为设计有误的证据，**重新审视设计；必要时舍弃骨架，从调查问题阶段重新设计。**

例如，实现后可能发现函数需要骨架里没有的参数，这就是骨架与实现的偏差。

又如，Agent 为读取数据类型中没有的字段，到处重复写下 `as any`（强行指定类型、关闭类型检查），也是重复出现的绕路办法。

文章的观点：无关位置出现同样的绕路办法，或类型需要逃生口，都是设计错误的证据

「Architecting bigger changes」一节在解释 `/architect` 的五阶段后，用下面这句话归纳何时应舍弃设计。

If the same workaround appears across unrelated call sites, or if the types require escape hatches like any or forced casts, that is empirical proof that the architecture is wrong.

若无关的调用位置出现同样的绕路办法，或类型需要 any、强制转换等逃生口，这就是架构错误的实证。

这段引文列出两个迹象。

- 无关的调用位置出现相同的绕路办法
- 类型需要 any 或强制转换（例如 as unknown as User）之类的逃生口

以下用本书的示例展示两种迹象。

骨架里的 Order 类型没有币种字段。

但 Agent 实际编写后发现，发票、邮件和管理界面这三个互不相关的地方都需要币种。

```
// 骨架中定义的类型：没有币种字段
type Order = { id: string; total: number };

// 要显示或发送的订单
declare const order: Order;

// invoice.ts: 通过类型逃生口 (as any) 读取类型中不存在的字段
const invoiceCurrency = (order as any).currency ?? "JPY";

// mail.ts: 在另一处不相关的代码中使用相同规避办法
const mailCurrency = (order as any).currency ?? "JPY";

// admin.ts: 在第三处使用相同规避办法
const adminCurrency = (order as any).currency ?? "JPY";
```

Agent 在三个位置重复同一种绕路办法，**并非各位置都写得不好，而是骨架的 Order 类型缺少币种字段。**

因此，当无关位置出现同样的绕路办法或类型逃生口时，**应检查设计本身，而非逐个修补调用位置。**

pstack：根据迹象是否反复出现，决定要不要舍弃设计

/architect 的 SKILL.md 要求：判断是否舍弃设计，不能只看迹象是否出现过一次，而要看同样的迹象是否反复出现。

该 SKILL.md 也要求看到迹象时，不立即断定设计有误。需要处理几个边缘情况（罕见输入或情境），不一定表示设计错误；问题本身也可能确实复杂。

文章把类型逃生口列为迹象，但按该 SKILL.md 的规则，只在某一处出现一次 any，不足以单独成为舍弃设计的理由。

相反，像发票、邮件和管理界面的例子一样，在三个无关调用位置都开始写相同绕路办法，就应将其视为可能需要舍弃骨架的迹象。

此外，同一 SKILL.md 的「Phase E: Scrap when the architecture is wrong」一节还列出以下四种迹象，供判断是否舍弃设计；这四种迹象是在上述相同绕路办法和类型逃生口之外的补充。

1. 多个互不相关的边缘情况，各自都需要专用分支
2. 骨架假设状态不共享，实现时 Agent 却发现需要加「锁」（防止并发改写的机制）
3. 调用方必须了解模块内部的约定才会正确使用（例如必须先调用初始化函数）
4. 实现中发现的骨架偏差，有相同形式的偏差在独立位置出现两次以上

下面用代码示范这四个迹象。

示例 1：无关边缘情况各自增加专用分支

一个显示订单总额的函数，为三个互不相关的边缘情况分别增加专用分支。

```
function formatTotal(order: Order): string {
  // 仅对旧格式订单另行计算
  if (order.id.startsWith("legacy-")) return formatLegacyTotal(order);
  // 仅对退款订单另行显示
  if (order.total < 0) return `返金 ${-order.total}円`;
  // 仅对礼品订单隐藏金额
  if (order.id.endsWith("-gift")) return "ギフト";
  return `${order.total}円`;
}
```

Oh My Stack 项目注（代码示例）：示例中的日文输出分别表示「退款若干日元」「礼品」和「若干日元」；它们是 formatTotal 的实际返回值，故保留。

各分支都从 id 字符串或金额正负号推测订单类型，因为 Order 类型缺少表示订单种类的字段。这是应重新设计类型，而非继续加分支的迹象。

示例 2：原本认为不共享的状态，结果需要加锁

骨架假设各发送方的计数只由一个处理过程读写。

```
// 骨架的前提：每个发送方的计数只由一个处理过程读写
const counts = new Map<string, number>();
```

但实现时发现，多个服务器会同时改写共享存储（store）里的同一计数。

于是试图加锁，避免从读取到写回期间被其他服务器插入。

```
// 实现过程中：试图加入骨架中没有的锁
async function tryAcquire(senderId: string): Promise<boolean> {
  await lock.acquire(senderId);
  try {
    const count = (await store.get(senderId)) ?? 0;
    await store.set(senderId, count + 1);
    return count < 60;
  } finally {
    await lock.release(senderId);
  }
}
```

之所以需要锁，是因为骨架「状态不共享」的前提有误。

示例 3：调用方必须了解内部约定才能使用

必须先调用 init()，否则 tryAcquire 就会报错的设计。

```
const limiter = createRateLimiter({ perMinute: 60 });

limiter.init(); // 内部约定要求先调用此方法
limiter.tryAcquire("sender-a"); // 若忘记调用 init()，这里就会报错
```

调用方若不知道「先调用 init()」这一内部约定，就无法正确使用。该约定既不体现在类型中，也不体现在函数形式中。

示例 4：相同形式的偏差在两个独立位置出现

骨架中，格式化金额的函数 `formatPrice` 只接受订单参数。

```
// 骨架：只接收订单
function formatPrice(order: Order): string {
  throw new Error("not implemented");
}
```

但实现发票和邮件这两个独立部分时，两边都需要显示语言（`locale`）参数。

```
// 实现 invoice.ts 时：发现需要骨架中没有的 locale 参数
formatPrice(order, locale);

// 实现 mail.ts 时：另一处也需要同样的 locale 参数
formatPrice(order, locale);
```

相同形式的偏差出现在两个不相关的位置，说明不只是某处的特殊需要，而可能是骨架遗漏了 `locale`。

要点 11：拆成可分别验证的小任务

设计确定后，Agent 使用规划用的「**Multi-phase or multi-PR plan**」Playbook，将任务拆成小规模、可分别验证的 PR。

同时，计划文档要写明每份 PR 的完成证据（例如日志行、截图或测试运行结果）。

文章的观点：规划 Playbook 通常在设计之后使用

poteto 在「Okay but I really want a planning doc」一节说，pstack 没有专门的规划 Skill，但有按多个阶段制定计划的 Playbook。

他通常等 Agent 拿出令自己满意的设计后，再用这份规划 Playbook（「**Multi-phase or multi-PR plan**」）把设计转化为具体到执行步骤的计划。

（该 Playbook 产出的计划文档示例见下文「计划文档的完整示例」。）

文章给出的请求只有一行。

```
/poteto-mode turn this design into a plan
// 把这个设计写成计划。
```


使用这一 Playbook 后，Agent 会围绕完成证据及其验证方法，安排计划中的每项任务。

人批准计划后，Agent 逐项执行。

计划中的每份 PR 都**规模小、自成一体，便于审查**。

pstack：计划文档逐份 PR 说明完成证据与验证方法

「**Multi-phase or multi-PR plan**」Playbook 的文件（playbooks/multi-phase-plan.md）有以下五个要点。

- 计划本身是成果物，暂不实现
- 编写前先用原型解决未定问题
- 每份 PR 包含一项变更及其证据
- 仅通过测试不足以算完成验证
- 使用者指示开始后才执行

最后会给出遵照这五点编写的完整计划示例。

计划本身是成果物，暂不实现

Agent 在这份 Playbook 中只编写计划文档，不编写代码。

执行计划由文档中点名的执行用 Playbook 负责。

执行用 Playbook 可选「**Autopilot-full**」或「**Autopilot-stack**」。对于持续多日的整个工作项目（program），则使用「**Orchestrate**」（均见第 16 章）。

编写前先用原型解决未定问题

对于「哪种方法更快」等未解决的问题，Agent 会分别运行「**Prototype**」Playbook（第 12 章）。

然后在计划文档附录中保留原型所得证据（分支、提交、截图）。

这与第 38 章要点 8（让实验回答疑问）相同。该章写道：

对于可以通过制作和运行来回答的疑问，人既不自行猜测，也不让 Agent 等待人的答复，而是让 **Agent 通过实验核验**。

每份 PR 一项变更及其证据

计划文档用标题（##）逐份划分 PR。例如，包含两份 PR 的计划文档形式如下。

```
# webhook 限流计划文档

## 创建按发送方计数的 RateLimiter (PR 1)
(请为 PR 1 填写以下各栏)

## 添加按发送方限流的功能 (PR 2)
(请为 PR 2 填写相同各栏)
```

每个标题下都按顺序填写以下字段。本节末尾会展示填完内容的示例。

- 依赖的 PR
- 要修改的文件
- 要进行的变更
- 可观察结果（日志行或界面状态）
- 单元测试
- 现场验证（实际操作运行中的应用来检查）
- 性能检查
- 使用者审查（第 16 章称为「操作者」审查；只适用于改变用户操作流程的 PR，其余写「无」）
- 合并

例如，在上面骨架的 PR 2 标题下填入这些字段，结果如下。

```
## 添加按发送方限流的功能 (PR 2)

**依赖的 PR**：PR 1 (创建按发送方计数的 RateLimiter)

**要修改的文件**
- [ ] 编辑 `src/webhooks/handleWebhook.ts`
```

****要做的改动****

- [] 在 `handleWebhook` 中调用 `limiter.tryAcquire(senderId)`，超过上限时返回 429

****可观察的结果****

- [] 同一发送方在 1 分钟内发送 61 次，第 61 次的日志出现 `rate\_limited sender=sender-a`

****单元测试****（单元测试不能单独算作验证完成）

- [] 在 `handleWebhook.test.ts` 中添加「第 61 次返回 429」测试，并运行 `npm test`

****实际运行验证****

- [] 向运行中的服务器发送 61 次 webhook 请求，确认第 61 次返回 429

****性能验证****

- [] 比较修改前后每次 webhook 请求的处理时间

****用户审阅****：无（用户操作流程未改变）****合并****

- [] 单元测试、实际运行验证和性能验证全部完成后再合并

仅通过测试不足以算完成验证

Playbook 用以下两句话规定验证标准。

Tests alone are not sufficient verification. A PR is verified only when its unit, live, and perf boxes are all checked.

单有测试还不足以构成验证。只有单元测试、现场验证、性能检查三个字段全都勾选，PR 才算验证完成。

使用者指示开始后才执行

Agent 将计划文档的位置和格式检查（check-plan.mjs）结果交给使用者，然后停下。

在使用者明确指示开始之前，不会执行计划。

计划文档的完整示例

示例：webhook 限流计划文档（两份 PR）

将上述两份 PR 骨架的各字段填入内容后，结果如下。

```

# webhook 限流计划文档

## 创建按发送方计数的 RateLimiter (PR 1)

**依赖的 PR**：无

**要修改的文件**
- [ ] 新建 `src/webhooks/rateLimiter.ts`
- [ ] 新建 `src/webhooks/rateLimiter.test.ts`

**要做的改动**
- [ ] 实现 `createRateLimiter`，按发送方统计每分钟的请求数

**可观察的结果**
- [ ] 用验证脚本以同一发送方身份调用 61 次，第 61 次显示 `false`

**单元测试**（单元测试不能单独算作验证完成）
- [ ] 添加「前 60 次返回 true、第 61 次返回 false」和「1 分钟后重新计数」两个测试，并运行 `npm test`

**实际运行验证**
- [ ] 实际运行验证脚本，确认观察结果符合预期

**性能验证**
- [ ] 测量调用 10,000 次的耗时，记录单次调用时间

**用户审阅**：无（用户操作流程未改变）

**合并**
- [ ] 单元测试、实际运行验证和性能验证全部完成后再合并

## 添加按发送方限流的功能 (PR 2)

**依赖的 PR**：PR 1（创建按发送方计数的 RateLimiter）

**要修改的文件**
- [ ] 编辑 `src/webhooks/handleWebhook.ts`

**要做的改动**
- [ ] 在 `handleWebhook` 中调用 `limiter.tryAcquire(senderId)`；超过上限时返回 429

**可观察的结果**
- [ ] 同一发送方在 1 分钟内发送 61 次，第 61 次在日志中出现 `rate_limited sender=sender-a`

**单元测试**（单元测试不能单独算作验证完成）
- [ ] 在 `handleWebhook.test.ts` 中添加「第 61 次返回 429」测试，并运行 `npm test`

**实际运行验证**
- [ ] 向运行中的服务器发送 61 次 webhook 请求，确认第 61 次返回 429

```

****性能验证****

- [] 比较修改前后每次 webhook 请求的处理时间

****用户审阅****：无（用户操作流程未改变）

****合并****

- [] 单元测试、实际运行验证和性能验证全部完成后再合并

真正的 Playbook 模板要求更细致。例如，现场验证分为十个方面，每方面都要写出截图及通过条件。PR 部分之前还有说明整体计划如何推进的章节，之后则附上原型证据等附录。

因此，**计划文档逐份 PR 规定完成证据及验证方法，并仅在使用者指示开始后执行。**

要点 12：计划文档是临时性的

计划文档用于「执行任务时」，或「向其他 Agent 说明大型任务进展时」，是一份**临时文档**。

文章的观点：任务完成后通常删除计划文档

poteto 写道，对于持续一周的大型项目，为让其他 Agent 了解进行中的工作，有时会临时把计划文档提交到仓库。

但**任务完成后，通常会删除计划文档**，避免给代码库留下混乱。

他还说，长期保留计划文档没有多少价值。

pstack：计划文档放在 Agent 存储区的 docs/ 下

「**Multi-phase or multi-PR plan**」Playbook 规定，除非使用者指定位置，**计划文档写在 Agent 存储区的 docs/ 中。**

Agent 存储区（原文称 agent store）……供 Agent 使用的存储区域。位置写在 Agent 的系统提示词（最初传给 Agent 的指示）里（见「Orchestrate」Playbook）。

要点 13：向 /poteto-mode 说明什么

人无需向 /poteto-mode 罗列步骤，而应说明**目标、约束、希望看到的结果，以及审查时机**。

文章的观点：通常只需要 /poteto-mode

文章说，由于 /poteto-mode 会自动使用介绍过的许多 Skill，**人通常只需向 /poteto-mode 提出请求**。

「The workflow in practice」一节给出四种场景的请求示例。

示例 1：调查原因不明的故障

第一种场景是调查生产环境中原因未明的故障。

```
/poteto-mode investigate why background workers periodically fail with timeout errors. give me a
breakdown of what we know, what data you used, and your best hypotheses.
// 调查后台 Worker 周期性因超时错误而失败的原因。分别列出已知事实、使用的数据和最有力的假设。
```

请求除了目标，还指定了结果格式：已知事实、所用数据和最有力的假设。

收到请求后，Agent 调查代码，并行核查指标（如处理耗时或错误次数等系统状态的量化记录）与历史提交，再对问题可能所在提出有依据的推测。

示例 2：请求设计

第二种场景是引入其他模块将依赖的新机制。

poteto 要求为外部发来的 webhook 设计限流机制（限制一定时间内接收的请求数量）。

```
/poteto-mode we need to add rate limiting for external webhooks. /architect this first, and answer
any open questions with prototypes. let me review before proceeding.
// 外部 webhook 需要限流。先用 /architect 做设计，用原型解答未决问题，再让我审阅后继续。
```

webhook……向指定 URL 发送 HTTP 请求，以通知另一个系统发生事件的机制（第 25 章）。

该请求除了规定如何推进设计，还指定「继续前让我审查」，明确停下的时机。

Agent 会调查现有 webhook 机制，让多个模型竞争设计方案，用一次性原型测量性能，最终形成结构清晰、经过验证的接口。

示例 3：规划迁移

第三种场景是规划跨越许多文件的迁移。poteto 要求把整个 UI 库迁至 StyleX（在 JavaScript 代码中编写样式的库），并制定分成多份 PR 的计划。

```
/poteto-mode create a plan to migrate our entire UI library to StyleX. break the migration into
small, verifiable PRs. each PR must have its visual regression tests and live verification steps. i
want the final result to be 100% identical compared to the original - bugs included
// 制定将整个 UI 库迁移到 StyleX 的计划，拆成可小步验证的 PR。每个 PR 都要有视觉回归测试和在运行中应
用验证的步骤。最终效果要与原来 100% 一致，包括已有缺陷。
```

除目标（整个 UI 库迁至 StyleX）外，请求还规定了以下三点，但没有指定具体迁移步骤。

- **如何拆分迁移**……规模小、可分别验证的 PR
- **每份 PR 应有的验证**……视觉回归测试（检查变更前后界面外观有无变化）及在运行中的应用里验证的步骤
- **最终结果的约束**……连既有缺陷也保持与原来相同

Agent 会把工作拆成彼此独立的阶段，写出可检查的清单，并为每阶段安全地完成构建、验证和合并作准备。

示例 4：修复 Slack 报告的故障

第四种场景是修复 Slack 上报告的故障。

这里有两个请求。

第一个适用于 Slack 讨论串里已经有足够背景的情况。

```
# thread already has sufficient context
/poteto-mode do it
// （讨论串中已有足够上下文）请照此执行。
```

第二个指定重现方法与证据。

```
/poteto-mode repro this with /control-app. if it repros on main, fix it and show me a video as proof
// 用 /control-app 复现这个问题。在 main 上复现后修复，并用视频展示证据。
```

验证 Skill……项目专属的 Skill，供 Agent 启动、操作应用并获取证据。
使用者运行 `/create-verification-skill` 后，由 Agent 创建。

`/control-app` 是文章对《The Complete Guide to pstack》[Part 1](#) 所创建验证 Skill 的称呼（[第 36 章](#)）。

四个示例的共同之处

四个场景中的请求都把**具体步骤交给** `/poteto-mode`，**只说明目标，并按需要补充以下内容**。

- 约束
- 希望看到的结果（结果形式）
- 审查时机（何时停下）

不过，也可以在请求中指定手段，例如[示例 2](#)中的 `/architect`，或[示例 4](#)中的 `/control-app`。

在[示例 4](#)的 `do it` 请求里，目标和上下文不在请求本身，而在 Slack 讨论串中。

pstack：只有想覆盖默认选择时才点名 Skill

随附指南 [docs/guide/02-poteto-mode.md](#) 说明，`/poteto-mode` 会根据请求类型选择 Playbook，并在 Playbook 的步骤中调用必要的 Skill。

因此，**使用者无需在请求里写明要按什么顺序使用哪些 Skill**。

该页面建议，只有下述情况才点名 Skill（[第 7 章](#)）。

Name a skill only when you want to override a specific choice.

只有想覆盖某项具体选择时，才点名 Skill。

但像[示例 2](#)那样引入其他模块依赖的新机制时，`/poteto-mode` 也可能依据[要点 9](#)（用 `/architect` 比较、整合多个设计）的标准，自行选用 `/architect`：

该指南将跨越函数边界的改动，或转移职责归属（哪个模块负责什么）的改动列为 /architect 的适用范围。

本书认为，[示例 2](#)明确写 /architect，是为了不让 /poteto-mode 自行决定是否使用它，而是要求先设计、后实现。

将 13 个要点连成一条流程

把 13 个要点排在一起，就形成第 6 部「本部章节」表中的三个阶段。

1. **理解问题**（要点 1~4，[第 37 章](#)）……人让 Agent 像自己一样理解问题，并继承过去工作的上下文。
2. **试验解决方案**（要点 5~8，[第 38 章](#)）……Agent 通过说明用法的文档与原型试验解决方案，人再选择设计方案。
3. **确定解决方案并转化为可执行计划**（要点 9~13，本章）……Agent 比较多个方案、确定设计，依据实现所得证据重新审视设计，并转化为可验证的计划。计划文档是临时性的；poteto 认为任务完成后通常删除。人向入口 /poteto-mode 说明的不是步骤，而是目标、约束、希望看到的结果与审查时机。

本书认为，三个阶段的共同点是：**判断的依据不是只有文字的计划，而是调查所得证据和实际运行结果。**

具体而言，有以下两类证据。

- 调查代码和历史所得证据（要点 3：用 /how、/why、/teach 了解机制和理由／要点 4：用 /recall 将过去的工作带进新对话）
- 实际制作并运行后得到的证据，例如操作原型的结果、测量值，或实现时在无关位置反复出现的同一种绕路办法（要点 7：比较多个方案的原型／要点 8：让实验回答疑问／要点 10：实现揭示设计错误时重新审视设计）

文章也在最后的「The art of planning」一节指出，**抽象计划（尚未落实为代码、只有文字的计划）只会给人工作正在推进的错觉。**

它还说，冗长的计划文档会让人和 Agent 看起来勤奋，但内容可能并不扎实。

随后文章指出，pstack 是用于**结合以下三项的工具**。

- 彻底调查
- 实际试验所得证据
- 严格验证

该节最后这样写道：

When you plan this way, engineering with agents stops feeling like a gamble. It becomes predictable and repeatable.

以这种方式规划时，与 Agent 一起做工程便不再像赌博，而会变得可预测、可重复。

总结

- **要点 9：用 /architect 比较并整合多个设计**……对于大型变更，人把时间用于架构、数据结构及系统间协作；Agent 负责填入实现细节。Agent 先用 /architect 比较多个设计并整合成一个，再开始实现。
- **要点 10：实现揭示设计错误时，重新审视设计**……如果发现骨架与实际需求不符，或相同的绕路办法、类型逃生口在无关位置反复出现，Agent 会将其视为设计错误的证据并重新审视；必要时舍弃骨架，从调查问题开始重做设计。
- **要点 11：拆成可分别验证的小任务**……Agent 使用规划 Playbook 将任务拆成可分别验证的小型 PR，逐份规定完成证据；证据不只有测试，还包括实际运行的结果。使用者指示开始后才执行计划。
- **要点 12：计划文档是临时性的**……计划文档用于执行任务及向其他 Agent 分享大型任务进度；poteto 通常在任务结束后删除。
- **要点 13：向 /poteto-mode 说明什么**……人向 /poteto-mode 说明目标、约束、希望看到的结果与审查时机，而非具体步骤。
- **13 个要点的流程**……本书认为，各阶段都以调查所得证据和实际运行结果作为判断依据，而非只有文字的计划。

第 6 部至此结束。

接下来的[附录](#)介绍 RGDD（README 驱动 with Grill Driven Development），作为实践本书内容的一种开发方法。

RGDD 是我构思的开发方法：先于代码准备 README 和术语表等供 Agent 持续阅读的上下文，再开展开发。

最后的「[结语](#)」将给出本书的结论，以及读完后的第一步。

附录：README with Grill 驱动开发 (RGDD)

kaito · 来源：日本語 · English · 2026-10-03

RGDD (README 驱动与 Grill Driven Development) 是我在个人开发中运用本书内容和 pstack 时，构思出的一套开发方法。

在 RGDD 中，开发者先于代码，将供 Agent 阅读的上下文放进代码库，再让 Agent 读取这些上下文进行开发。

上下文……主要是仓库中的文件，Agent 在每次工作时重新阅读。
对话结束后仍会保留。

第 4 章将代码库视为 Agent 的记忆，并写道：

让 Agent 继承过去判断的记忆，应放在**代码库**，而非模型内部。

在 RGDD 中，除代码外，以下文档也用作上下文。

- **README**……说明谁为了什么目的、如何使用软件的文档（例如用法或教程）。
- **GLOSSARY.md**……汇总领域术语（描述软件涉及的业务或对象的词语）及其定义的文档（例如「订单」具体指什么）。
- **ADR (Architecture Decision Record)** ……设计决策及其理由的记录（例如为什么选用某个数据库）。
- **编码规范**……代码写法的规则（例如写在 CODING_STANDARDS.md 中的规则）。
- **Feature Map 与验证 Skill**……验证 Skill 是项目专属的 Skill，供 Agent 启动、操作应用并获取证据。Feature Map 则是其中按应用功能汇总进入方式和验证方法的文档（第 36 章）。

开发者先于代码准备 README、GLOSSARY.md 和 ADR；第一个功能运行起来后，再让 Agent 创建 Feature Map 和验证 Skill。

本附录解释 RGDD 的理念与做法。

本附录结构

本附录安排如下。

- RGDD 结合 README 驱动开发与 /grill-with-docs 制作上下文，并用其进行开发和验证
- RGDD 分五个阶段增加上下文，并核查、修订和更新
- 每个阶段阅读前一阶段的上下文并添加内容，使上下文持续成长
- 先准备上下文，是为了让 Agent 从一开始就能读到正确的范例
- 总结：先于代码，准备上下文

上下文成长……随着工作推进，在上下文文件中增加术语和决策，同时持续核查、修订与更新已有内容。

RGDD 结合 README 驱动开发与 /grill-with-docs 制作上下文，再用于开发和验证

RGDD 将 README 驱动开发与 Matt Pocock 的 Skill /grill-with-docs 结合，用它们制作上下文，再将上下文用于开发与验证。

README 驱动开发……先从说明目标用户如何使用产品的 README 写起，再反推实现和架构的开发方法。

README 驱动开发将在「1. 在 README 中写下用户视角的用法」介绍，/grill-with-docs 则见「2. 用 /grill-with-docs 创建领域术语和 ADR」。

RGDD 的基础，是开头引述的第 4 章理念：将代码库视为 Agent 的记忆。

poteto 在《The Complete Guide to pstack》Part 1 的「Keep agents smart with Feature Maps」一节也写道：

Personally, I think your codebase is the ultimate form of memory. Code is a projection of the decision making you and your team have made and represents the source of truth for what's happened and how things actually work.

我个人认为，代码库就是记忆的终极形式。代码映照着你和团队作过的决策，也是了解发生过什么、系统实际上如何运作的可靠信息来源。

同一节中，poteto 如此解释 Feature Map：

A Feature Map is just a more compact form of that, designed to save tokens.

Feature Map 只是这种记忆（代码库）的精简形式，目的是节省 token。

Feature Map 包含：

- 有哪些功能，各自做什么
- 从用户视角如何进入每项功能
- 如何用验证 Skill 操作
- 出现什么结果才能说明功能正常

RGDD 进一步将上下文扩展到「谁为了什么目的、如何使用」（README）、「使用哪些术语」（GLOSSARY.md）和「为何这样决定」（ADR）。

以下汇总五类上下文的内容与创建者、工具。

上下文	包含什么	创建者与工具
README（或承担 README 作用的文档）	谁为了什么目的、如何使用	由开发者编写
GLOSSARY.md	领域术语及其定义、应避免的说法	/grill-with-docs
ADR（docs/adr/）	重要的设计决定及其背景和理由	/grill-with-docs
编码规范（例如：CODING_STANDARDS.md）	代码写法	由开发者按仓库确定并编写

上下文	包含什么	创建者与工具
Feature Map 与验证 Skill	功能清单、用户视角的进入方式与验证方法	<code>/create-verification-skill</code>

- **领域术语**……描述软件所处理的业务或对象的词语（例如点餐应用里的「订单」「取消」）。
- **应避免的说法**……指向同一事物、但决定不用的其他词语（例如选定「订单」为术语后，「购买」「交易」便是应避免的说法）。在 GLOSSARY.md 中写在术语定义之下。

编码规范并非五个阶段中创建的成果，而是 Agent 通过 `/poteto-mode`（将请求引向合适流程的 pstack 入口 Skill；第 7 章）编写代码时应遵守的规则，因此不列入下节的五个阶段。

另外，**README 不一定非要叫** README.md。

只要文档描述用户视角的用法，并作为其他上下文的起点，即使是规格说明，也可以承担 README 的角色。

RGDD 分五个阶段增加上下文，并核查、修订和更新

开发者按以下五个阶段推进 RGDD。

1. 在 README 中写下用户视角的用法
2. 用 `/grill-with-docs` 创建领域术语和 ADR
3. 用 `/poteto-mode` 开发
4. 用 `/thermos` 审查
5. 用 `/create-verification-skill` 验证

每个阶段负责**增加上下文，或核查、修订与更新**留在上下文中的内容。

下面依次从「制作什么」「使用什么工具」「留下什么上下文」三方面介绍。

1. 在 README 中写下用户视角的用法

- **制作什么**……从应用用户视角写成的用法或教程 README，暂不讨论实现。

- **使用什么工具**……在 RGDD 中，开发者自己编写 README，不使用 Skill 或 CLI 等工具。不过，第 38 章的要点 5（先写用法）介绍了让 Agent 编写的方式：

人在实现之前，让 Agent **先在 README 或教程中写出用法**，再由此反推 API 与内部设计。

- **留下什么上下文**……「谁为了什么目的、如何使用」等应用用法的上下文，是其他上下文的起点。

示例：点餐应用的 README

只写用户视角的用法，暂不讨论实现。

```
# 便捷点餐

这是一款用手机向附近店铺点餐的应用。

## 可以做什么

- 从菜单选择餐品并下单
- 查看订单状态（已接单、制作中、可取餐）
- 取餐前可以取消订单

## 使用方法

1. 打开应用，选择店铺
2. 从菜单选择餐品，点击「下单」
3. 状态变为「可取餐」后，到店取餐
```

RGDD 把这一阶段放在最前面，是为了先给下一阶段的 `/grill-with-docs` 准备提问素材。

写出用户视角的用法后，`/grill-with-docs` 才能针对其中的模糊之处提出具体问题，开发者也能具体回答。

例如，上述 README 示例说「取餐前可以取消订单」。`/grill-with-docs` 可以据此问「正在制作中的订单也能取消吗？」，明确 README 尚未决定的事项。

2. 用 `/grill-with-docs` 创建领域术语和 ADR

- **制作什么**……汇总领域术语的 `GLOSSARY.md`，以及记录设计决策的 ADR。

- **使用什么工具**.....Matt Pocock 的 `/grill-with-docs`。Agent 不会自行选择这项 Skill，需由开发者点名调用。
- **留下什么上下文**.....领域术语及定义等背景知识（`GLOSSARY.md`），以及设计决策和理由等历史判断（ADR）。

`/grill-with-docs` 会为厘清计划或设计，向开发者反复深入提问，并在过程中创建 ADR 与术语表（`GLOSSARY.md`）。

`/grill-with-docs` 在步骤中调用另外两项 Skill。

- `/grilling`.....围绕计划和设计反复询问开发者，厘清待定事项。
- `/domain-modeling`.....指出术语不一致或含糊的说法，立即把确定的术语写入 `GLOSSARY.md`。只有以下三项条件同时成立，才建议将判断记录成 ADR：
 - 难以推翻
 - 缺少背景时看起来出人意料
 - 确实比较过选项后才作出选择

下文解释 `/domain-modeling` 建议创建 ADR 的三项条件，并以点餐应用举例。

示例：创建 ADR 的三个条件

表中「含义」译自 `/domain-modeling` 的 `ADR-FORMAT.md` 中的「When to offer an ADR」一节。「示例」是本书将阶段 1 点餐应用 README 中「每家店的订单数据存入不同数据库」的判断套入条件后写成的。

条件	含义（ADR-FORMAT.md）	示例
难以推翻	日后改变决定需要付出很大代价	日后若合并到一个数据库，必须迁移所有店铺的数据
缺少背景时看起来出人意料	日后读代码的人会疑问「为何这样设计」	同为订单数据，却按店铺分开存储
确实比较过选项后作出选择	存在其他可行方案，并有明确理由选定其中之一	比较过合并为一个数据库的方案；为将故障影响限制在单家店铺，选择分开存储

反过来说，少了其中任何一项条件，就不写 ADR。

例如「订单列表按最新排在前面」的决定容易撤销，也不会有人质疑原因，因此不写 ADR。ADR-FORMAT.md 如此说明：

If a decision is easy to reverse, skip it: you'll just reverse it. If it's not surprising, nobody will wonder why. If there was no real alternative, there's nothing to record beyond "we did the obvious thing."

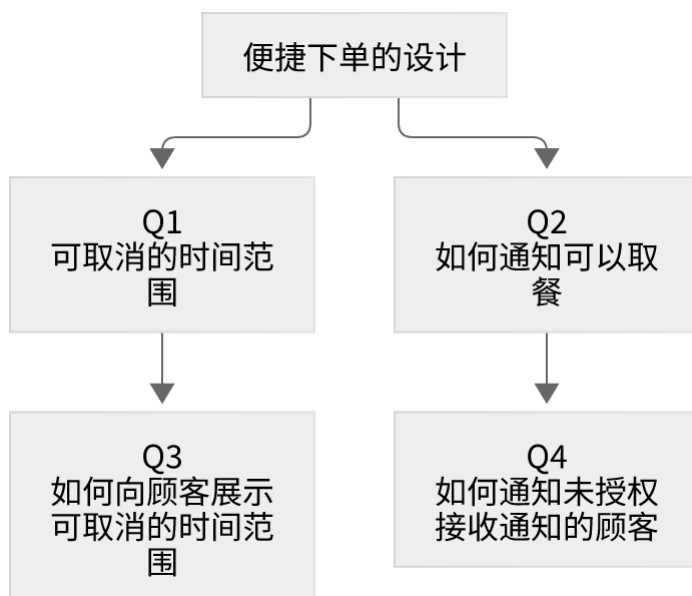
若决定容易撤销，就不必写 ADR，直接撤销即可。若决定并不令人意外，就不会有人追问原因。若没有真正可选的方案，除了「做了显而易见的事」以外便无须记录。

/grilling 会把待定事项按前置决策排序，并将当前就能回答的问题一起提出。具体方法与示例如下。

示例：/grilling 如何提问

/grilling 将待定事项整理成树状结构（原文称 design tree）。树枝表示「确定父决策后才可作出的子决策」。

以阶段 1 的 README 示例（简易点餐）为例，待定事项可以整理成以下决策树（本书自拟）。



[查看图示 1](#)

例如 Q3「如何向顾客展示可取消的期限」，只有确定 Q1「可取消的期限」后才能回答。若期限是「仅在已接单时」或「取餐前」，展示方法也会不同。同样，Q4「如何告知未允许通知的顾客」，只有 Q2 确定「通过推送通知告知」后才会出现。

/grilling 将树上的问题分成几轮（原文称 round）提出。每轮只问**所有前置决策均已确定、现在就能回答的问题**。原文把这组问题称为 frontier，并这样解释：

The **frontier** is every decision whose prerequisites are already settled: the questions you can ask now without guessing at answers you haven't heard yet.

frontier 指所有前提已经确定的决策：无须猜测尚未听到的答案，现在就能提出的问题。

上面的树中，第一轮是无须等待父决策的 Q1 和 Q2。Agent 会给它们编号，并附上推荐答案，一次性提问。

开发者回答 Q1 和 Q2 后，Q3 和 Q4 就可回答。Agent 在第二轮提出 Q3 和 Q4。

第一轮示例

这是按 /grilling 的 `SKILL.md` 的格式（为问题编号并附推荐答案），写出的上面决策树的第一轮（Q1、Q2）。

? ****Q1**** - ****可取消的阶段****: README 写着「取餐前可以取消订单」。制作中的订单也可以取消吗?

➡ 只有「已接单」的订单可以取消（避免浪费已经开始制作的餐品）

? ****Q2**** - ****如何通知可以取餐****: 如何告知顾客订单状态已变为「可取餐」?

➡ 通过应用推送通知

当开发者对 Q1 回答「仅在已接单时」，「取消」这一术语的定义便确定。于是 /domain-modeling 会将定义写入 `GLOSSARY.md`。

在这一阶段，**Agent 能把与开发者确定的领域术语写进文件（`GLOSSARY.md`），而非只留在对话中。**

对话中确定的术语，在对话结束后不会自动传给下一名 Agent。

而写在 GLOSSARY.md 中的术语，下一名 Agent 也能读到。

另外，在 /grill-with-docs 细化设计的期间，Agent 会将开发者的用词与 GLOSSARY.md 中的定义比对。

如果开发者的说法与定义不符，Agent 会当场指出（/domain-modeling 的 SKILL.md 中「Challenge against the glossary」一节）。

When the user uses a term that conflicts with the existing language in GLOSSARY.md, call it out immediately.

如果使用者（Skill 的使用者，在 RGDD 中即开发者）使用与 GLOSSARY.md 现有术语冲突的词，应立即指出。

判断不一致的线索是 GLOSSARY.md 中的「应避免的说法」（原文为 _Avoid_）。/domain-modeling 的 GLOSSARY-FORMAT.md 在「Rules」一节规定，若多个词指同一概念，选出最合适的一个作为术语，其余列为应避免的说法。

When multiple words exist for the same concept, pick the best one and list the others under _Avoid_.

如果多个词指同一概念，选择最合适的一个，其余列在 _Avoid_ 下。

例如，GLOSSARY.md 定义「订单」，并把「购买」列为应避免的说法（GLOSSARY-FORMAT.md 的示例）。

开发者说「购买」时，Agent 会指出它与既定术语不一致。

本阶段创建的 GLOSSARY.md 和 ADR，可以是以下这样的简短文档。

示例：GLOSSARY.md

写法参照 /domain-modeling 的 GLOSSARY-FORMAT.md，每个术语附上简短定义与应避免的说法。

订单

涵盖顾客向店铺点餐到取餐的过程。

术语

****订单**:**

顾客向店铺提交的餐品订单。

应避免的说法: 购买、交易

****取消订单**:**

顾客取消「已接单」的订单。制作中的订单不能取消。

应避免的说法: 取消

示例：ADR (docs/adr/0001-separate-database-per-store.md)

写法参照 /domain-modeling 的 ADR-FORMAT.md。ADR 是记录决定及其理由的短文，一段文字也可以。

按店铺分别在独立数据库中保存订单数据

为了避免一家店发生故障时影响其他店铺的订单，按店铺分别在独立数据库中保存订单数据。也比较过集中到一个数据库的方案，但因故障会波及所有店铺而未采用。

这份 ADR 记录的是折叠内容「示例：创建 ADR 的三个条件」中的表格所分析的决定。

3. 用 /poteto-mode 开发

- **制作什么**……实际代码。
- **使用什么工具**……通过 /poteto-mode 开发（第 7 章）。开发者想了解代码机制或过去的经过时，可使用以下 Skill（第 22 章）。
 - /how……调查运行机制
 - /why……调查设计理由
 - /recall……从聊天记录等收集近期工作的经过
- **留下什么上下文**……代码本身与 PR。Agent 会在功能开发、故障修复等 Playbook（按任务类型编写的流程；第 8 章）的最后一步创建 PR。

开发阶段，Agent 一边阅读此前创建的上下文及编码规范（若开发者已准备），一边编写代码。

对于回复反馈、QA、修复故障等较轻或重复性工作，我会从 Grok Bot（AI Bot 应用；第 5 章）启动 Cloud Agents（Cursor 在云端运行 Agent 的功能），让它们完成工作。

Grok Bot 只负责启动 Cloud Agents，不让它直接做具体工作；这一点与 poteto 的用法相同。

第 36 章这样描述 poteto 的用法：

从 Grok Bot 发出请求时，poteto 不让 Bot 自己执行任务，而是请它启动 Cloud Agents（Cursor 在云端运行 Agent 的功能）中的 Agent。

README、GLOSSARY.md、ADR 和编码规范都放在仓库中，因此包括 Cloud Agents 在内，所有执行工作的 Agent 都能阅读。

阶段 2 创建的 ADR，也有助于开发者在开发期间通过 `/why` 询问判断理由。

`/why` 针对不同信息来源，分别设有说明如何调查的文件。

其中，调查版本管理历史的文件包含寻找 docs/adr/ 等位置的 ADR 的步骤。

因此，只要 ADR 中记载了理由，`/why` 就可以把它作为回答的依据。

即使某项判断没有 ADR，`/why` 也会尝试重建理由：并行调查版本管理历史、工单和聊天记录，利用留下的线索。工单和聊天记录来自通过 MCP（连接 Agent 与外部工具或数据的机制）接入的服务。

若仍找不到线索，就会列明已查过的地方，并报告理由未知（Unknown）。

4. 用 `/thermos` 审查

- **制作什么……审查意见清单。**
- **使用什么工具……Cursor 插件 `/thermos`。**
- **留下什么上下文……Agent 根据意见修正后的代码。（※ `/thermos` 的步骤只负责把意见汇总成清单，不包括修正代码。）**

`/thermos` 会让两名子 Agent 并行审查阶段 3 中 Agent 编写的代码差异。

子 Agent……受运行 `/thermos` 的 Agent（主 Agent）委托，执行部分工作的另一名 Agent。

两项审查分别是：

- **安全性与正确性审查**（例如故障、破坏既有功能的变更、安全漏洞）
- **严格的代码质量审查**（例如维护困难、文件膨胀、不必要的抽象）

两项审查结束后，/thermos 会去除重复意见，汇总成一份清单。

两名子 Agent 都指出的同一个问题，权重会更高。

RGDD 加入这一阶段，是为了在权宜性的绕路办法（不修根因、只暂时绕开故障的写法）和难以维护的结构合并进入代码库前将其去除。留在代码中的绕路办法会被下一名 Agent 误认为正确范例，并照着复制。

第 4 章这样写道：

并非一切都值得留下作为记忆。若保留权宜性的绕路办法，下一名 Agent 会把它误认为正确的实现范例并复制；复制又会促成下一次复制，不良模式便悄悄扩散。

5. 用 /create-verification-skill 验证

- **制作什么**……项目专属的验证 Skill，包含操作应用的小型 CLI 与 Feature Map（第 36 章）。
- **使用什么工具**……pstack 的 /create-verification-skill，以及持续更新验证 Skill 和 Feature Map 的 /maintain-verification-skill（第 26 章）。
- **留下什么上下文**……有哪些功能，用户如何进入、如何验证每项功能。

/create-verification-skill 将 Feature Map 编排为目录 features/README.md 与各功能对应的文件。

目录链接到各功能文件。功能文件从用户视角说明以下四点。

- 功能是什么
- 如何进入
- 如何用验证 Skill 的 CLI 操作
- 看到什么结果才算功能正常

示例：功能文件（pstack 范例摘录）

这是 /create-verification-skill 的 references/feature-map-example/ 中，笔记应用「创建笔记」功能文件的节译，略去了一部分内容。

```
# 创建笔记
```

可以从浏览器或 CLI 保存带标题的笔记。也可以放弃未完成的草稿，并从其他页面查看已保存的笔记。

```
## Sub-features

- `create-open` 从浏览器的各入口打开空白编辑页
- `create-save` 保存标题和正文

## How to get to it (user POV)

- 在浏览器工具栏中选择 `New note` 按钮
- 当焦点在输入框外时，在浏览器中按 `n` 键
- 在终端运行 `notes create --title <title> --body <body>`

## Driving it with control-notes

- **打开编辑页。** 选择 `New note`，运行 `control-notes browser click --role button --name "New note"`。`Note editor` 表单出现，`Title` 输入框获得焦点。
- **填写内容。** 输入标题和正文，分别运行 `control-notes browser fill --role textbox --name "Title" --value "Release checklist"` 和 `control-notes browser fill --role textbox --name "Body" --value "Tag and publish"`。`Save note` 按钮变为可点击。
- **保存笔记。** 选择 `Save note`，运行 `control-notes browser click --role button --name "Save note"`。`Note saved` 提示出现，标题变为 `Release checklist`。

## Gotchas

- 当焦点位于输入框内时，按 `n`，不会打开新的编辑页，而是输入字符。
```

标题下的说明段落对应「功能是什么」；How to get to it (user POV) 对应「如何进入」；Driving it with control-notes 对应「如何操作」及「看到什么结果才算功能正常」。

control-notes 是范例验证 Skill 自带的应用操作 CLI。Sub-features 是以短 ID 逐行列出的细分功能；Gotchas 是可能使验证工作白费或结果无效的陷阱。四个标题在所有功能文件中都使用相同的英文名称和顺序。

Feature Map 的目录 features/README.md 与阶段 1 的 README 名称相似，作用却不同。

阶段 1 的 README 由开发者编写，描述计划建造的产品如何使用。

Feature Map 的 features/README.md 则是应用完成后，Agent 按 /create-verification-skill 步骤制作的目录。

《The Complete Guide to pstack》Part 1 将验证 Skill 视为最重要、最先要创建的 Skill（第 36 章）。RGDD 也持相同看法。

不过，从零创建应用时，尚无可运行和验证的对象，所以这里将其安排在最后一个阶段。

对于已有应用，本书强烈建议使用 `/create-verification-skill` 创建验证 Skill 与 Feature Map。

`/create-verification-skill` 的 `SKILL.md` 也以可运行的应用为前提。如果应用无法按当前状态构建或启动，该文件要求 Agent 在创建验证 Skill 之前先修复，或准确报告现状（见「1. Interview the repo, not the user」一节）。

因此在 RGDD 中，**第一个功能运行起来后**，开发者就运行 `/create-verification-skill`，让 Agent 制作验证 Skill。

此后，Agent 在每次任务中用它验证变更，开发者则使用 `/maintain-verification-skill`，让验证 Skill 和 Feature Map 保持最新。

我遵照《The Complete Guide to pstack》Part 1 的建议，每天至少运行一次 `/maintain-verification-skill`。运行工作交给用于维护的 Grok Bot。

这个 Bot 也遵照 Part 1 的建议，请 Dr Eggbot（第 36 章）协助制作。

I recommend running `/maintain-verification-skill` at least once a day to ensure that your agents always have the latest details on controlling your app.

我建议至少每天运行一次 `/maintain-verification-skill`，确保 Agent 始终掌握操作应用的最新细节。

每个阶段阅读前一阶段的上下文并添加内容，使上下文持续成长

上下文之所以持续成长，是因为 RGDD 的每个阶段都会阅读前一阶段留下的上下文，再添加内容，或核查、修订与更新保留在上下文中的成果。

阶段	阅读内容	添加的上下文，或核查、修订和更新的成果
1. README	无	谁为了什么目的、如何使用
2. <code>/grill-with-docs</code>	README	术语、决策（ADR）

阶段	阅读内容	添加的上下文，或核查、修订和更新的成果
3. /poteto-mode	README、GLOSSARY.md、ADR、编码规范	代码、PR
4. /thermos	阶段 3 编写的代码差异	差异中的问题（由 Agent 修正后保留在代码中）
5. /create-verification-skill	可运行的应用	Feature Map、验证方法

开发继续进行，开发者和 Agent 仍按以下方式添加、核查、修订和更新上下文。

- **出现新术语或新决策时**……开发者再次使用 `/grill-with-docs` 深化设计。Agent 将术语写入 GLOSSARY.md，并在一项决策符合以下三项条件时建议写成 ADR：
 - 难以推翻
 - 缺少背景时看起来出人意料
 - 确实比较过选项后才作出选择
- **审查发现差异中的问题时**……Agent 先修复 `/thermos` 的意见，再将代码保留在仓库里。
- **增加功能时**……开发者运行 `/maintain-verification-skill`。若发现 Feature Map 缺少功能等问题，Agent 会把修正汇入一份 PR。

这种持续添加、核查、修订和更新上下文的方法，最主要的目标是**为 Agent 持续保留合适的上下文，包括过去的背景**。

这一目标来自第 4 章介绍的「园丁」思路。第 4 章如此描述维护代码库记忆的工作：

演讲把维护这种环境的人比作「**园丁**」。花园不能只靠增加植物来维持，还要修剪不需要的枝条、清除杂草，持续维护，让值得保留的东西生长。

代码库也一样，**不只要添加知识，还必须持续清除过时的绕路办法和不良实现范例**。

在 RGDD 中，开发者与 Agent 不只维护代码，也对 README、GLOSSARY.md、ADR 与 Feature Map 做园丁工作。

- **添加**……将新确定的术语和决策写入 GLOSSARY.md 与 ADR。相当于在花园中培育值得保留的东西。

- **核查、修订与更新**……核查并修正过时的术语定义，或与应用不符的 Feature Map。相当于修剪枝条、清除杂草。

保留过去的上下文，主要是 ADR 的职责。它记录决策及理由，即使日后改了决定，下一名 Agent 仍能了解过去为何如此决定。

/domain-modeling 的 ADR-FORMAT.md 允许在 ADR 开头写状态（原文为 Status），以便日后修订决策。四种状态是提议中（proposed）、已采纳（accepted）、已弃用（deprecated）和被另一份 ADR 取代（superseded by ADR-NNNN）。

由于存在「被另一份 ADR 取代」这一状态，本书将其理解为：改变决策时保留旧 ADR，再增加新 ADR。

示例：改变决策后的 ADR

假设日后改变阶段 2 中的 ADR 示例（0001）。状态写法采用 ADR-FORMAT.md 列出的四种状态。

旧 ADR 不删除，只将状态改为「被 0002 取代」。

docs/adr/0001-separate-database-per-store.md

```
---
status: superseded by ADR-0002
---
```

按店铺分别在独立数据库中保存订单数据

为了避免一家店发生故障时影响其他店铺的订单，按店铺分别在独立数据库中保存订单数据。也比较过集中到一个数据库的方案，但因故障会波及所有店铺而未采用。

新 ADR 写明重新决定了什么，以及原因。

docs/adr/0002-single-database-for-orders.md

```
---
status: accepted
---
```

将订单数据集中到一个数据库

店铺超过 300 家后，已无法继续维护每店一个数据库，因此将订单数据集中到一个数据库。对于 0001 曾试图避免的故障波及所有店铺的问题，通过在多处复制数据库来降低影响。

下一名 Agent 阅读 0002 就能知道当前决策，阅读 0001 则能了解为何过去按店铺分开存储。

先准备上下文，是为了让 Agent 从一开始就能读到正确范例

保留上下文，就能将已决定的事项传给下一名 Agent。

先于代码准备上下文，则是**为了让 Agent 一开始读到的范例就是正确的**。

本书认为，如果按与 RGDD 相反的顺序先写代码，后来的上下文只能通过解读代码重建。

这时，混入代码中的权宜性绕路办法，也可能被当作正确上下文保留下来。

留在代码中的绕路办法，之后就成为了下一名 Agent 的范例（见[阶段 4](#)）。

总结：先于代码，准备上下文

- **RGDD 是什么**……先于代码准备 README、GLOSSARY.md 和 ADR，让 Agent 阅读它们并据此开发。
- **按什么顺序推进**……依次经历 README → /grill-with-docs → /poteto-mode → /thermos → /create-verification-skill 五个阶段。
- **为何上下文会持续成长**……每个阶段都会添加上下文，或核查、修订和更新已有内容。
- **为何先准备上下文**……为了让 Agent 从一开始就能读到正确的范例。

最后的「[结语](#)」将给出本书的结论，以及读完后的第一步。

后记

kaito • 来源: 日本語 • English • 2026-10-03

本书的结论：能委托多少工作，取决于可验证环境有多完善

pstack 之旅终于来到终点。

一路走来，本书以 poteto 的演讲和文章为线索，逐一解读了 pstack 的 Playbook、Principle 和 Skill。我们由此得出的结论是：

「能委托给 Agent 的工作范围，取决于它能在多大范围内自行验证成果，而非它有多聪明。」

也就是说，Agent 越能自行启动、操作应用并留下结果证据，使用者就越能把工作交给它，而不必每次亲自运行检查。

这一结论源自 poteto 的演讲（第 1 章）和《The Complete Guide to pstack》Part 1。两者都一贯把验证作为基础。

演讲指出，值得信任的对象是成果物，而非 Agent；能验证成果物，才有信任的前提。Part 1 的「Verification is all you need」一节则将高质量的验证 Skill 视为最重要的 Skill。

第一步：让一个小任务完整走过验证循环

pstack 有 23 份 Playbook、23 项 Principle、24 项 Skill，但无须从明天起就全部用熟。

起步时，用一个小任务完整走过一次验证循环就足够。

- **验证 Skill**……项目专属的 Skill，供 Agent 启动和操作应用、获取证据。
- **验证循环**……使用者设定可判定通过或失败的完成条件并委托工作；Agent 在真实应用中检查、修正直至满足条件，再返回证据的完整循环。

接下来提出读完本书后一周内可完成的四个步骤，以及此后扩大委托范围的顺序。

「一周」与「四个步骤」的安排是本书的建议。

但步骤 1（选择真实任务）和步骤 2（写出可判定成败的完成条件）的内容，也见于 pstack 随附指南（01-setup.md、06-verify-and-ship.md）。

一周内可完成的四个步骤

1. 选择一项失败也无碍的真实任务

起步时应选择「规模小、即使失败也无碍，但确实需要完成的任务」。

例如修复轻微的显示故障、增加一项 CLI 选项、修正配置值读取方式。这些改动都不大，运行后也能迅速核对结果。

避免练习题，选择真实任务。因为这一周的目的，是确认验证循环能否在真实应用中走通。

2. 写出可以判定成败的完成条件

使用者在编写请求前，先决定「确认什么之后才算完成」。

完成条件不能是「做得好一点」，而要能够通过执行来判定成败。

随附指南 06-verify-and-ship.md 的「State the finish condition up front」一节，以为 CLI 增加 JSON 输出为例，示范如何在请求中写出完成条件。

```
/poteto-mode add json output to this command. text output stays byte-identical, the json parses,
both run against the sample project. show me the evidence.
// 给这个命令增加 JSON 输出。文本输出必须逐字节保持不变。JSON 必须能够解析。两种输出都要在示例项目中
运行。请展示证据。
```

请求中的三个条件都可通过执行来判定成败。

如果使用 pstack，就直接将这些条件放进对 pstack 入口 `/poteto-mode`（第 7 章）的请求。

即使不使用 pstack，也可以对任何 Agent 采用同样的写法。

3. 要求 Agent 自行在真实应用中验证

这是最重要的一步。

使用者应要求 Agent 实际启动应用、操作修改过的功能，并**展示结果证据**，而不只是报告测试是否通过。测试通过，并不保证真实界面或输出正确。

例如，《The Complete Guide to pstack》[Part 1](#) 在委托新功能时使用以下写法（[第 36 章](#)）。

```
/poteto-mode build <description of feature, any useful context>. use /control-app to verify your changes and show me a video and screenshots as proof
// 创建<功能说明和有用的上下文>，用 /control-app 验证改动，并展示视频和截图作为证据。
```

`/control-app` 是文章中为通过 `/create-verification-skill` 创建的验证 Skill 使用的名称。

这份请求在功能说明之外，还指定了用哪项验证 Skill 核验（`/control-app`），以及证据格式（视频和截图）。

证据可以是：

- 命令输出
- 界面截图
- 读取已保存数值的结果

如果 Agent 无法自行验证，使用者应**优先消除阻碍它验证的原因**。否则使用者仍得亲自检查结果，委托范围便无法扩大。

本书认为，**Agent 无法验证而停下**，通常有以下三个原因。

- 不知道如何启动应用
- 没有操作应用的手段
- 缺少测试用户或测试数据

使用者记下 Agent 因哪一种原因受阻，便能明确下一步需要补齐什么。

其中，启动应用，以及测试用户和数据，也出现在 Part 1 的「Make it Reproducible」一节。

Part 1 在这一节建议，从应用的开发体验（原文称 dev experience）出发，考虑以下三件事。

seeding a dev database

how to handle auth, test users, API calls against a test/staging environment

installing and bringing up your dev environment in a consistent way

为开发数据库填充种子数据（开发或测试时预先放入的数据）

如何处理认证、测试用户，以及对测试或预发布环境的 API 调用

如何始终按一致的步骤安装并启动开发环境

验证 Skill（第 26 章、第 36 章）旨在消除 Agent 无法自行验证的状况。第 26 章这样说明了先准备验证 Skill 的理由。

使用者先准备验证 Skill，是因为**如果 Agent 无法自行验证工作，不论交给它多少任务，最后仍要人逐项检查，人就会成为阻碍工作推进的瓶颈。**

若使用 pstack，可以尝试运行一次创建验证 Skill 的 `/create-verification-skill`。

4. 同样的提醒说了两次，就将其写成机制而非文字

如果使用者在一周内对同一种错误提醒两次，就应考虑能否自动检测它；若能，就把提醒转化为机制。

这一思路借自 pstack 的 Principle。

「**Encode Lessons in Structure**」（第 21 章）规定了 Agent 发现自己重复写下同一指示时应如何处理。

Agent 会考虑将该指示改为以下任一形式；若可行，就改成机制并删除原文字指示。

- lint 规则（自动检查代码写法的工具规则）
- 元数据 flag（写在文件配置字段中的值）
- 运行时检查（程序运行时核对值或状态是否满足条件）
- 脚本

该 Principle 之所以这样规定，是因为**文字指示只有在读者注意到、记住并遵守时才会生效。**

其原文这样写道。

Textual instructions are easy to miss. They require the reader to notice, remember, and comply.

文字指示容易被忽略，需要读者注意、记住并遵守。

对使用者而言，lint 规则、CI 检查和脚本等都可替代文字指示。

例如，若已经提醒两次不要使用 any，就加入禁止 any 的 lint 规则。

eslint.config.js

```
// 编写 any 时 lint 应报错（假设已引入 typescript-eslint）。
import tseslint from "typescript-eslint";

export default [
  {
    files: ["src/**/*.ts"],
    languageOptions: { parser: tseslint.parser },
    plugins: { "@typescript-eslint": tseslint.plugin },
    rules: {
      "@typescript-eslint/no-explicit-any": "error",
    },
  },
];
```

一旦把提醒变成 lint 或 CI 检查，从下一周开始，不用使用者再次提醒，也不用依赖 Agent 记住，同类错误就会被自动拦截。

逐步扩大委托范围

使用者走完一周内可完成的四个步骤后，就会找到至少一类任务：对于这类任务，无须每次亲自检查差异和运行行为，阅读 Agent 提交的证据即可。

随后按任务种类逐一扩大这一范围。

即使在同一类任务内，也要分阶段扩大委托范围。

本书将其分为以下四个阶段。

委托阶段	使用者负责	交给 Agent
1	每次亲自检查差异和运行行为	编写代码
2	阅读 Agent 提交的证据	编写代码，自行运行验证并提交证据
3	设定完成条件并委托任务，只查看完成的 PR 和证据	按任务类型既定的流程（在 pstack 中为 Playbook）调查、修正、验证，并创建 PR
4	确定方针，批准不可撤销的操作（如部署、删除数据）	并行推进多项工作，合并已验证的变更

完成四个步骤的任务类型，便到达表中的阶段 2：阅读 Agent 提交的证据。

不要跳级，应从阶段 1 依次上升。

如果阶段 2 的证据尚不可信就进入阶段 4，只会并行增加一批无从核实的变更。

演讲中「每月 2,500 份 PR」的数字，也是按顺序完善成果验证环境后才达到的。第 1 章这样写道。

也就是说，必须先让成果物能够被验证，再将由此得到的知识留在代码库中，并在此基础上推进自动化，遵循这一顺序。

每月 2,500 份 PR 是依次完善这四项之后的结果。

这段引文也概括了演讲的四个主题：信任、提高信任的方法、把代码库当作记忆，以及自动化。

最后

pstack 至今仍在持续更新。

本书以 2026 年 9 月的 pstack version 0.15.5 为对象。

Playbook、Principle 和 Skill 的名称、数量，以及默认模型（使用者未指定时使用的模型），今后都会继续变化。

即便如此，本书认为，「先建立让 Agent 自行验证工作成果的环境」这一方针，不会因模型更聪明而失去价值。因为模型越聪明，能够承担的任务越大，也就越需要验证机制。

此外，即使聪明的模型也可能未经验证就自信地断言。

《The Complete Guide to pstack》Part 2 在「Building up a mental model」一节，这样描述最新的模型。

Even with the latest frontier models, (this also depends on the quality of the harness), in general I find that they still often state things confidently without backing it up with data or actually reading the code needed to build up a mental model of how it works.

即使是最新的前沿模型（也取决于 harness 的质量），我仍常看到它们不以数据支持结论，也不实际阅读理解机制所需的代码，却自信地下断言。

不妨先选一项小任务，让验证循环完整走上一轮。

从这里开始，能够委托的范围会逐渐扩大。

感谢您阅读到最后。

希望本书能为读者提供一些参考，让开发生活有所改善。

祝您享受愉快的 pstack life!

参考文献

<https://x.com/poteto>

<https://x.com/poteto/status/2102050467505430555>

<https://x.com/poteto/status/2094457600259842065>

<https://x.com/poteto/status/2097732320606507506>

<https://github.com/cursor/plugins/tree/main/pstack>

<https://github.com/cursor/plugins/blob/main/pstack/README.md>

<https://github.com/cursor/plugins>

<https://github.com/cursor/plugins/blob/main/pstack/docs/guide/README.md>

<https://github.com/cursor/plugins/blob/main/pstack/docs/guide/06-verify-and-ship.md>

<https://github.com/cursor/plugins/blob/main/pstack/docs/guide/02-poteto-mode.md>

<https://github.com/cursor/plugins/blob/main/pstack/docs/guide/01-setup.md>

<https://github.com/cursor/plugins/blob/main/pstack/docs/guide/07-overnight.md>

<https://github.com/cursor/plugins/blob/main/pstack/docs/guide/10-recipes-and-pitfalls.md>

<https://github.com/cursor/plugins/blob/main/pstack/docs/guide/05-build-and-clean.md>

<https://github.com/cursor/plugins/blob/main/pstack/docs/guide/09-make-it-yours.md>

<https://github.com/cursor/plugins/blob/main/pstack/docs/guide/08-principles.md>

<https://github.com/cursor/plugins/blob/main/pstack/docs/guide/03-understand.md>

<https://github.com/cursor/plugins/blob/main/pstack/docs/guide/04-design.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/bug-fix.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/babysit.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/prototype.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/multi-phase-plan.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/autonomous-run.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/orchestrate.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/feature.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/refactoring.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/investigation.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/runtime-forensics.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/trace-forensics.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/perf-issue.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/hillclimb.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/visual-parity.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/authoring-a-skill.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/eval.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/opening-a-pr.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/shipping.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/session-pickup.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/pause-safely.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/autopilot-full.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/autopilot-stack.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/worktree-cleanup.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-prove-it-works/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-test-behavior-not-implementation/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-build-the-lever/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-type-system-discipline/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-encode-lessons-in-structure/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-fix-root-causes/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-subtract-before-you-add/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-migrate-callers-then-delete-legacy-apis/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-never-block-on-the-human/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-laziness-protocol/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-guard-the-context-window/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-make-operations-idempotent/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-sequence-verifiable-units/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-separate-before-serializing-shared-state/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-model-the-domain/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-minimize-reader-load/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-exhaust-the-design-space/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-foundational-thinking/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-redesign-from-first-principles/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-attack-the-premise/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-outcome-oriented-execution/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-experience-first/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-boundary-discipline/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/create-verification-skill/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/maintain-verification-skill/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/references/bugbot-triage.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/no-comments/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/reflect/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/agents/poteto-agent.md>

<https://github.com/cursor/plugins/blob/main/pstack/agents/comment-sicko.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/architect/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/figure-it-out/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/arena/SKILL.md>

<https://github.com/cursor/plugins/tree/main/pstack/skills/poteto-mode/scripts/watch-pr>

<https://github.com/cursor/plugins/blob/main/pstack/skills/recall/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/scripts/check-plan.mjs>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/scripts/orch/orch.ts>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/scripts/worktree-audit.sh>

<https://github.com/cursor/plugins/blob/main/pstack/skills/how/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/why/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/teach/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/blast-radius/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/interrogate/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/swarm/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/automate-me/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/make-bot-ui/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/bro/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/tdd/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/show-me-your-work/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/typescript-best-practices/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/unslop/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/technical-writing/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/setup-pstack/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/show-me-your-work/scripts/log.sh>

<https://github.com/cursor/plugins/blob/main/pstack/skills/typescript-best-practices/references/patterns.md>

<https://github.com/cursor/plugins/tree/main/pstack/skills/create-verification-skill/references/feature-map-example>

<https://github.com/cursor/plugins/pull/422>

<https://github.com/cursor/plugins/tree/main/cursor-team-kit>

<https://github.com/cursor/plugins/tree/main/thermos>

<https://cursor.com/ja/changelog/0-48-x>

<https://github.com/poteto/verification-skill-example>

<https://diataxis.fr/>

<https://github.com/mattpocock/skills>

<https://cursor.com/ja/docs/bugbot>

<https://tailscale.com/>

<https://stylexjs.com/>

<https://x.ai/bot/marketplace/bots/dr-eggbot-v2>

<https://github.com/mattpocock/skills/blob/main/skills/engineering/grill-with-docs/SKILL.md>

<https://github.com/mattpocock/skills/blob/main/skills/productivity/grilling/SKILL.md>

<https://github.com/mattpocock/skills/blob/main/skills/engineering/domain-modeling/SKILL.md>

<https://github.com/mattpocock/skills/blob/main/skills/engineering/domain-modeling/ADR-FORMAT.md>

<https://github.com/mattpocock/skills/blob/main/skills/engineering/domain-modeling/GLOSSARY-FORMAT.md>

<https://x.com/poteto/status/2087977008253071816>

<https://x.com/poteto/status/2104744961904394699>

<https://github.com/cursor/plugins/blob/main/pstack/skills/architect/references/design-red-flags.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/create-verification-skill/references/feature-map-example/create-note.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/create-verification-skill/references/feature-map-example/search.md>

<https://cursor.com/ja/blog/cloud-agent-environment>

<https://cursor.com/ja/docs/cloud-agent/security-network>

<https://github.com/anthropics/skills>

https://github.com/anthropics/skills/blob/main/skills/skill-creator/scripts/quick_validate.py

<https://github.com/agentskills/agentskills/tree/main/skills-ref>

<https://developers.google.com/style>

<https://www.asd-ste100.org/>

<https://web.stanford.edu/~ouster/cgi-bin/book.php>

<https://zenn.dev/sc30gsw/articles/953334f11df507>

附加资料：术语表

这份术语表用于统一中文译本。命令、Skill 名称、文件名及代码标识符保持原样。

原文术语	译文用法	说明
agent / エージェント	Agent；首次写作「AI Agent（智能体）」	本书语境中能使用工具推进开发任务的智能体
Playbook	Playbook（工作流程）	为某类任务规定步骤和完成要求；后文保留 Playbook
Principle	Principle（原则）	支持 Agent 判断的工程原则；后文保留 Principle
Skill	Skill（技能）	可被 Agent 加载和执行的工作能力；后文保留 Skill
verification / 検証	验证	通过运行应用或检查实际成果判断工作是否完成
verification skill	验证 Skill	针对项目建立的启动、操作与取证能力
artifact / 成果物	成果物	代码、输出文件、应用行为等需要检验的结果
evidence / 証拠	证据	支持验证结论的输出、截图、录像或实际读取结果
finish condition / 完了の条件	完成条件	可以执行并判断通过或失败的条件
reproduction / 再現	复现	在可控条件下再次触发问题
codebase / コードベース	代码库	包括代码及项目内保留的知识、约束与文档
delegation / 委任	委派	将明确范围的工作交给 Agent 或子 Agent
parallelism / 並列化	并行执行	同时推进多个任务；不隐含共享写入或无限权限
diff / 差分	差异	代码或文件修改前后的差异
pull request / PR	PR（拉取请求）	首次解释，后文保留 PR

原文术语	译文用法	说明
regression	回归缺陷	原有正常行为因变更而失效
Feature Map	Feature Map（功能地图）	记录用户路径、操作步骤与预期结果
router	路由器	按请求选择并衔接合适工作流程的入口

译文不把「已读取源文件」「测试通过」「实际应用已验证」合并成同一种完成状态。

附加资料：图片文字说明

These are byte-preserving copies of the author's Japanese and English sign-up comparison figures, used in entry 22 (numbered chapter 17). Source URLs and SHA-256 hashes are recorded in [sources.json](#). They remain covered by the book's [attribution and authorization record](#).

中文图示说明：方案 A 把姓名、邮箱地址、密码放在同一张注册表单中，用户可以一次看全所有字段并填写。方案 B 分三个页面依次询问姓名、邮箱、密码，前两步点击「下一步」，最后点击「注册」；每页只处理一个字段。这是原图文字的中文说明，原图像素没有修改。